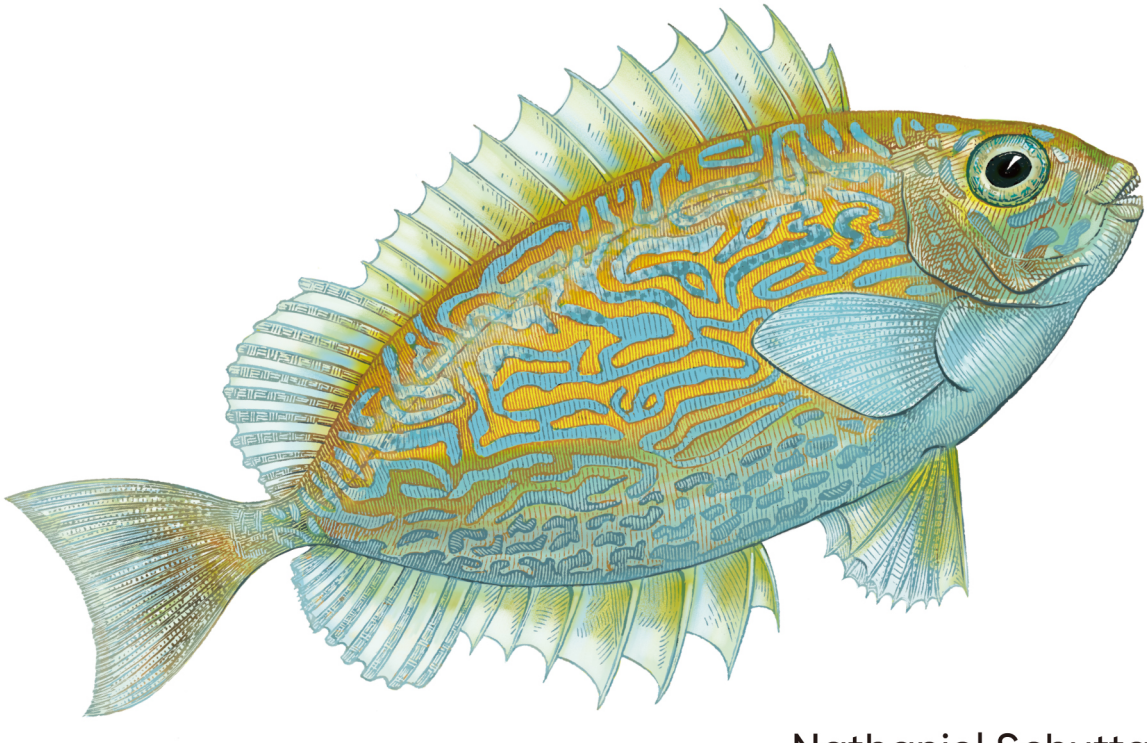
O'REILLY®
オライリー・ジャパン

ソフトウェアエンジニアリングの基礎

コーダーからエンジニアになるための実践ガイド



Nathaniel Schutta
Dan Vega
著
村上 列 訳

本書で使用するシステム名、製品名は、いずれも各社の商標、または登録商標です。
なお、本文中では、™、®、© マークは省略している場合もあります。

Fundamentals of Software Engineering

From Coder to Engineer

Nathaniel Schutta and Dan Vega

O'REILLY®

©2026 O'Reilly Japan, Inc. Authorized Japanese translation of the English edition of "Fundamentals of Software Engineering".
©2026 Code Monkey LLC and Nathaniel Schutta. All rights reserved. This translation is published and sold by permission of O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

本書は、株式会社オライリー・ジャパンが O'Reilly Media, Inc. の許諾に基づき翻訳したものです。日本語版についての権利は、株式会社オライリー・ジャパンが保有します。

日本語版の内容について、株式会社オライリー・ジャパンは最大限の努力をもって正確を期していますが、本書の内容に基づく運用結果について責任を負いかねますので、ご了承ください。

本書への賛辞

Nate と Dan は、長年の経験を活かし、ベテラン開発者から初心者まで、あらゆる読者に役立つ優れた知見を凝縮してくれました！

— Chris Kramer
Thoughtworks プリンシパル AI エンジニア

今ほど、このような本が求められている時代はありません。AI によって雇用市場が大きく変化し、リモートワークによって、現場での育成は従来のようには機能しなくなっています。本書は、こうした状況の中で、若手の開発者が成長し、適応し、成功するために必要な知恵と指針を提供します。

— Christopher M. Judd
Judd Solutions 社長

本書では、Dan と Nate が、従来の教育では見落とされがちな重要な知見を、わかりやすく解説しています。プロの開発者に向けた「すべきこと」と「すべきでないこと」を整理し、本書の随所に彼らの豊富な経験が活かされています。

— Venkat Subramaniam 博士
受賞歴のある著者であり、Agile Developer, Inc. の創設者

意外なことに、現代におけるソフトウェアエンジニアリングの本質を体系的に解説した書籍は、これまでほとんど存在しませんでした。しかし、その状況は本書によって変わりました。本書は、これまで大きな空白となっていた領域を埋めてくれました。プロフェッショナルの視点から、ソフトウェアエンジニアリングの多様な側面を整理しています。本書は、新人エンジニアにとっても経験豊富なエンジニアにとっても、これまで触れたことのない（あるいは知らなかった）領域を学ぶための、かけがえのないガイドとなります。強くおすすめします。

— Neal Ford
Thoughtworks ディスティンギッシュド・エンジニア

どの専門家も、「未知の未知」という不確実性に直面しています。これから何が待ち受けているのかもわからないのに、どう備えればよいのでしょうか？ 地図もないまま、未知の領域をどう進めばよいのでしょうか？
ソフトウェアエンジニアリングへの道のり、つまり「プログラマ」から「エンジニア」へと成長していく過程は、長い間、このような難題を突きつけてきました。次々と現れる流行語に翻弄され、刻々と変化する荒波を自力で切り抜けなければならず、さらに職場においては AI の影響力の高まりという新たな波にも直面しています。この道のりは、駆け出しの人にとってもベテランにとっても、非常に困難なものであり、途方に暮れてしまうこともあるでしょう。
しかし、心配はいりません。Nate と Dan が、読者が待ち望んでいた道しるべを示してくれました。本書は、エンジニアとしての思考法を身につけるための基礎を解説するものであり、単なるテクノロジー解説書ではありません。変化の激しい今日の企業の現場において、単に生き残るだけでなく、活躍するためのスキルを身につけるための本なのです。

— Raju Gandhi
ソフトウェア職人、テクノロジー愛好家、講師、
『Head First Software Architecture』『Head First Git』『JavaScript Next』の著者

Jen、Isabella、そして Juliana へ。私の心の支えであり、私の世界であり、私のすべてです。

— Dan

Christine と Everett へ。二人の愛情と支えは、私にとって何よりも大切です。

— Nate

はじめに

プログラマ、コーダー、開発者 ——ソフトウェアを作る人々を表す肩書きは数え切れないほどありますが、ソフトウェアエンジニアであるとはどういうことでしょうか？ 一般的なソフトウェア教育とは異なり、ソフトウェアエンジニアであるということは、単に文法的に正しいプログラムを作成すること以上の意味を持っています。

ブートキャンプや大学では、通常、コードを書くという技術的な側面を中心に教えており、プログラミングに精通した人材を育成しています。しかし、今日、優れたソフトウェアエンジニアになるために必要な知識は、プログラミング言語の習得にとどまりません。ソフトウェアプロダクトのライフサイクル全体について、深く理解していなければなりません。単にプログラミング言語の構文や文法だけでなく、テスト、アーキテクチャ、モデリングなどについても幅広い理解が求められます。レガシーコードを効果的に扱う方法や、本番環境へコードを確実に、しかも繰り返しデプロイする方法を知っていなければなりません。大きな成果を上げ、昇進し、最先端のプロジェクトに携わるためには、単にコードを書くだけにとどまらず、開発プロセス全体にわたりエンジニアリングの原則を適用する必要があります。人工知能（AI）やエージェント型コーディングツールの時代であっても、AI ツールを適切に活用するためには、基礎をしっかりと理解していなければなりません。

ソフトウェアエンジニアになるまでの道のりは多岐にわたり、コンピュータサイエンスの専門学校や大学の課程から、集中型のブートキャンプ、さらには独学に至るまでさまざまです（**図1**）。ソフトウェアエンジニアとしてのキャリアの初期段階では、まるで広大な世界への第一歩を踏み出したばかりのように感じられるかもしれません。圧倒されてしまうかもしれません。実は、筆者である私たちも、そう感じた経験があります！ どのようなバックグラウンドを持っている方でも、エンジニアとして働き始めると、標準的なカリキュラムでは教えられなかった重要なトピックや、まだ身につけていないスキルが山ほどあることにすぐに気づくでしょう。そして、そのようなスキルがないために、より上位の役割へ進む際に壁にぶつかってしまうことがあります。

ブートキャンプやコンピュータサイエンスの課程で学ぶ内容と、ソフトウェアエンジニアとして成功するために必要な知識との間には、隔たりがあります[†1]。（Thoughtworks のような技術系企

†1　ソフトウェア業界において、インポスター症候群（14 章で解説）が蔓延している一因となっています。

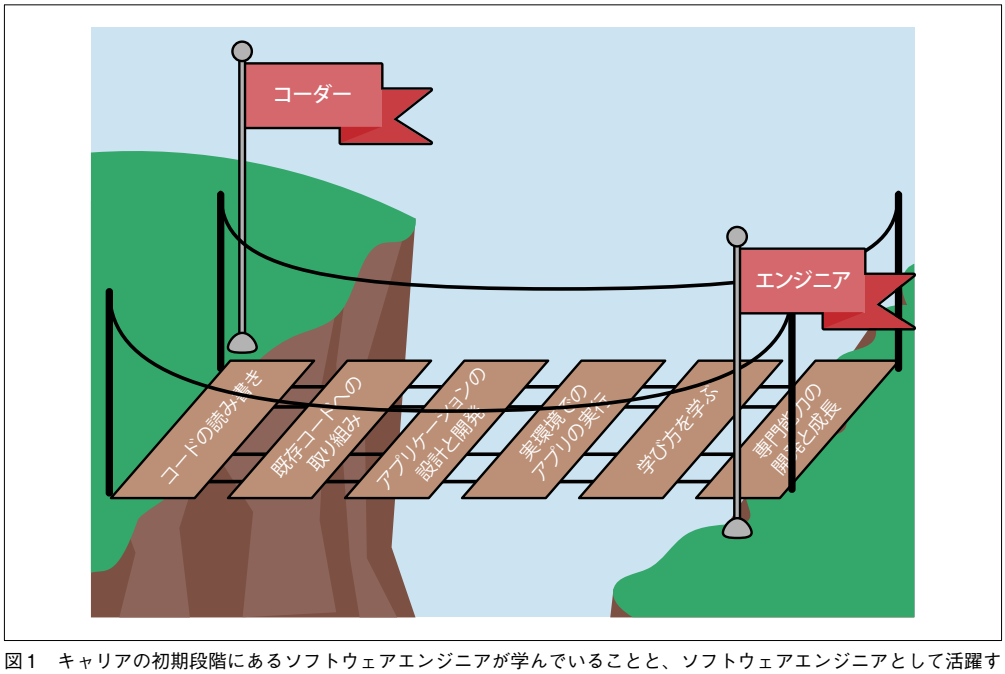


図1　キャリアの初期段階にあるソフトウェアエンジニアが学んでいることと、ソフトウェアエンジニアとして活躍するために必要なこととの間には隔たりがある

業は採用基準が厳しいにもかかわらず、新入社員を実践的な社内ブートキャンプに参加させています。こうした企業は決して珍しい存在ではありません。） 本書は、キャリアを築くために必要な背景知識と基礎を提供し、個人として、またプロフェッショナルとして成長する機会を見極める手助けをすることで、その隔たりを埋めることを目指しています。本書を、駆け出しのソフトウェアエンジニアのためのオンボーディングガイドだと考えてください。

本書は、その道のりを歩む読者の道しるべとなることを目指しており、読者自身もまだ気づいていない課題や学ぶべき領域を明らかにするものです。

本書の対象読者

本書は、特に新人ソフトウェアエンジニアの方を対象に執筆しました。私たち筆者の目的は「単にコードを書く」ことにとどまらず、一人前のソフトウェアエンジニアになるために必要なこと、そしてキャリアを築くために何が必要なのか、その全体像をお伝えすることです。

ただし、本書で取り上げる基礎知識は、初心者だけのものではありません。経験豊富なエンジニアにとっても、これらの必須スキルを習得することは大きなメリットとなります。特に、キャリアアップを目指し、より上級の役職に就きたいと考えている方にとってはなおさらです。確かなスキルセットを身につけることが、より大きな責任を担う機会や昇進への道を開くのです。

本書で学べること

ソフトウェアエンジニアとして活躍するために必要となるスキルとは何でしょうか？ 初心者と経験豊富なソフトウェアエンジニアの違いは何でしょうか？ コードの読み方から、読みやすいコードの書き方、テスト、ワークライフバランス、そして「学び方を学ぶ」ことまで――知っておくべきことを幅広く（そして、読者がまだ必要性に気づいていないことさえも）お伝えします。多くのソフトウェアエンジニアは、こうしたことを試行錯誤を通じて学びますが、その過程でプロジェクトに多大な損失をもたらすこともあります。しかし、このような経験をしなければならないわけではありません！ これらのスキルは、ソフトウェアエンジニアリングの基礎であり、読者のキャリアを支えるものです。そして、一過性のフレームワークや流行の言語とは異なり、これらのスキルは一生ものとなるでしょう。

本書は特定のトピックを深く掘り下げる本ではありません。その代わり、ソフトウェアエンジニアリングの幅広いトピックを紹介し、基本的な概念を理解するための土台を提供します。

本書では、筆者たちのキャリアにおけるエピソードや経験を共有し、そこから得られた教訓を紹介しています。各章の最後には、学んだことを実践するための関連リソースや実践的な課題を掲載しています。もし特定の章にピンとくるものがあれば、ぜひそのテーマについて深く掘り下げてみてください！

本書の読み方

ソフトウェアエンジニアになるための道が一つではないのと同じように、本書の読み方もさまざまです。多くの読者は本書を最初から最後まで通読することになるでしょうが、興味のあるトピックから章を選んで読む、「自分なりの読み方」を楽しむことも可能です。キャリアを積み、新しいプロジェクトに取り組み、新たに重要なトピックに出会うにつれて、指針を求めて特定の章に戻って読み直したくなることもあるでしょう。

本書は 4 つのパートで構成されています。1 つ目のパートでは、優れたソフトウェアエンジニアになるために必要なコアスキルについて解説します。2 つ目のパートでは、プロジェクトで必要となるさまざまな技術的実践を掘り下げ、3 つ目のパートではソフトウェアの設計と構築における細かなポイントについて詳しく解説します。4 つ目のパートでは、キャリアの成長と、自身のキャリアを主体的に切り開くための具体的なステップに焦点を当てています。

本書の内容は以下のとおりです。

1. コアスキル

- 「1 章　プログラマからエンジニアへ」では、ソフトウェアエンジニアになるためのさまざまな道筋について論じ、基礎的なスキルの重要性を重点的に解説しています。
- 「2 章　コードを読む」では、ソフトウェアエンジニアは、コードを書くことよりも、コードを読むことに多くの時間を費やしていることについて解説しています。また、初

めて触れるコードを読み解くための戦略も紹介しています。

- 「3 章　コードを書く」では、読みやすく、保守しやすいコードを書くことの重要性について詳しく解説しています。要するに、コンピュータのためではなく、開発者のためにコードを書くことがなぜ重要なのかを説明しています。

2. 技術的な実践

- 「4 章　モデル」では、多くのプロジェクトで目にした（あるいはこれから目にする）であろう箱と線で表される図、すなわちソフトウェアモデリングが、ソフトウェアエンジニア間のコミュニケーションにおいて果たす重要な役割について深く掘り下げます。
- 「5 章　自動テスト」では、コードの品質向上や開発者の自信につながる自動テストのメリットについて解説します。
- 「6 章　初めて触れるシステムの調査と変更」では、既存のコードベースを効果的に扱うための戦略に焦点を当てています。新しいチームに参加する場合でも、レガシーシステムを保守管理する場合でも、これらのスキルがあればどのようなコードベースに対しても自信を持って貢献できます。

3. アプリケーションの開発と設計

- 「7 章　ユーザーインターフェース設計」では、ユーザーインターフェース設計の概要を紹介します。ユーザーを理解し、設計原則を適用することが、ユーザーのニーズを満たすソフトウェアを作るうえでいかに重要であるかを説明しています。
- 「8 章　データの取り扱い」では、ほぼすべてのソフトウェアアプリケーションが何らかの形で行っている「データの取り扱い」について詳しく解説します。アプリケーション内のデータについて、十分な情報に基づいた判断を下すために必要な基本スキルを網羅しています。
- 「9 章　ソフトウェアアーキテクチャ」では、ソフトウェアアーキテクチャの複雑さを掘り下げ、あらゆるソフトウェアプロジェクトにつきまとうアーキテクチャ上のトレードオフについて解説します。
- 「10 章　本番環境へ」では、コードを本番環境へデプロイすることについて解説します。実際のユーザーや環境には予測できない要素が多いことを踏まえ、本番環境に対応したコードを構築するための戦略を概説します。

4. プロフェッショナルとしての能力開発と成長

- 「11 章　生産性を向上させる」では、日々の業務に立ち返り、自分なりのツールキットを構築することの重要性について解説します。
- 「12 章　学び方を学ぶ」では、継続的な学習の重要性について取り上げます。この章では、刻々と変化する分野についていくための戦略を提案しています。
- 「13 章　テクノロジー業界でソフトスキルを磨く」では、キャリアにおけるソフトスキルの重要性について深く掘り下げます。協調的なコミュニケーション、影響力の行使、

ステークホルダー管理について論じていきます。

- 「14 章　キャリアマネジメント」では、ソフトウェアエンジニアとしてのキャリアの計画と進め方に焦点を当てています。自分の情熱を見出すこと、キャリアパスの模索、そしてスキル習得への計画的な取り組みについて論じています。また、プロフェッショナルなネットワークの構築、面接を乗り切る方法、ワークライフバランスの確立についても取り上げています。
- 「15 章　AI を駆使するソフトウェアエンジニア」では、今日のソフトウェア業界において最も革新的なトピックの一つである、AI について触れています。

AI については専用の章で詳しく解説していますが、ソフトウェア開発ライフサイクルに与えるその広範な影響を考慮して、本書の随所で AI に関する議論を展開しています。

本書の表記法

本書では以下の表記規則を用いています。

太字（サンプル）
　強調や新出用語に使います。

等幅（`sample`）
　プログラムリストに使います。また本文の段落中でも、変数名や関数名、データベース名、データ型、環境変数、文、キーワードなど、プログラムの要素を示すために使います。



ヒントや提案を表します。



一般的なメモを表します。



警告や注意を表します。

コード例の活用

補足資料（コード例、演習問題など）は、https://github.com/Fundamentals-of-Software-

Engineering/book からダウンロードできます。

コード例に関する技術的なご質問や問題がある場合は、support@oreilly.com まで英語でメールをお送りください。

本書は、読者の業務を円滑に進めるお手伝いをするためにあります。一般的に、本書にサンプルコードが掲載されている場合、読者自身のプログラムやドキュメントでそれらをご利用いただけます。コードの大部分を複製するのでなければ、許可を得るために当社に連絡する必要はありません。例えば、本書からいくつかのコードの断片を使用したプログラムを作成する場合、許可は必要ありません。書籍からの例を販売または配布する場合は、許可が必要です。本書を引用し、例題コードを引用して質問に答える場合は、許可は必要ありません。本書の例題コードを相当量、自社製品のドキュメントに含める場合は、許可が必要です。

出典の明記は歓迎しますが、原則として必須ではありません。出典には「『ソフトウェアエンジニアリングの基礎』、Nathaniel Schutta、Dan Vega 共著（オライリー・ジャパン）、978-4-8144-0178-9」のようにタイトル、著者、出版社、および ISBN を含めてください。

コード例の使用がフェアユースの範囲外である、または上記の許可の範囲外であると思われる場合は、permissions@oreilly.com まで英語でお問い合わせください。

オライリー学習プラットフォーム

オライリーはフォーチュン 100 のうち 60 社以上から信頼されています。オライリー学習プラットフォームには、6 万冊以上の書籍と 3 万時間以上の動画が用意されています。さらに、業界エキスパートによるライブイベント、インタラクティブなシナリオとサンドボックスを使った実践的な学習、公式認定試験対策資料など、多様なコンテンツを提供しています。

https://www.oreilly.co.jp/online-learning/

また以下のページでは、オライリー学習プラットフォームに関するよくある質問とその回答を紹介しています。

https://www.oreilly.co.jp/online-learning/learning-platform-faq.html

お問い合わせ

本書に関する意見、質問等はオライリー・ジャパンまでお寄せください。連絡先は次の通りです。

株式会社オライリー・ジャパン
電子メール: japan@oreilly.co.jp

本書の Web ページには、正誤表やコード例などの追加情報が掲載されています。次の URL を

参照してください。

https://oreil.ly/fundamentals-of-software（原著）
https://www.oreilly.co.jp/books/9784814401789（和書）

本書に関する技術的な質問や意見は、次の宛先に電子メール（英文）を送ってください。

bookquestions@oreilly.com

オライリーに関するその他の情報については、次のオライリーの Web サイトを参照してください。

https://www.oreilly.co.jp
https://www.oreilly.com（英語）

謝辞

表紙だけを見れば、本は 1 人か 2 人の著者が単独で執筆した作品のように見えますが、実際には、チームによる深い協力の結晶です。編集者から校閲者、そして理解ある家族に至るまで、本は多くの人々の手によって生み出されたものです。素晴らしい編集者である Rita さん、Virginia さん、Louise さん、そしてオライリーチームの皆様からのご支援、忍耐、そしてご指導に、心から感謝申し上げます。皆様は決して私たちへの信頼を失うことなく、この「猫のように気ままな 2 人（私たち）」をまとめ上げるために全力を尽くしてくださいました。本当にありがとうございます！

素晴らしい技術レビューアーの方々への感謝の念は、いくら語っても語り尽くせません。彼らが費やしてくださった時間と心遣いは、私たちにとって計り知れない贈り物であり、その恩は容易には返せません。本書は、皆様からのご意見、ご質問、そして数々の具体例のおかげで、はるかに優れたものとなりました。Christopher M. Judd 氏には、いくつかの章の方向性を決定づけるような先見性のある見識をいただきました。共著者として名前を連ねたほうがよいくらいです。また、Venkat Subramaniam 博士が、本書の完成に向けて多大な時間を割いてくださったことに対し、心より感謝申し上げます。さらに、AI に関する助言をくださった Chris Kramer 氏、そして数十年にわたるご経験を本書に活かしてくださった Premanand Chandrasekaran 氏にも感謝申し上げます。James Erler 氏には、優秀な学習者であるだけでなく、熱心なレビューアーとしてもご尽力いただいたことに、特に感謝いたします。さらに、Raju Gandhi 氏、Sruthi Sathyamoorthy 氏、Murugan Lakshmanan 氏にも、お時間を割いてご意見をいただいたことに、心から感謝申し上げます。

また、私たちのキャリアを通じて共に仕事をしてきた数多くの開発者、アーキテクト、そしてその他のソフトウェアの専門家の皆様にも感謝申し上げます。お名前をすべて挙げることはできませんが、皆様のおかげで私たちは今の自分があるのです。（エスプレッソを片手に）本書の構想や初

期の章の執筆において、多大な貢献をしてくださった Jakub Pilimon 氏に感謝申し上げます。多くの方はご存じないかもしれませんが、本書の大部分は O'Reilly プラットフォームでのオンライン講座として始まりました。そのセッションにご参加いただいた皆様、この教材を練り上げるのにご協力いただき、誠にありがとうございます！

Nate です。ここで少し時間をいただき、私の親愛なる友人であり、「共犯者」であり、ゴルフ仲間であり、そして今や共著者でもある Dan Vega 氏に感謝の言葉を述べさせてください。君の献身的な努力、数々のエピソード、そしてアイデアに心から感謝しています。君は、私の漠然とした考えを、私のお気に入りの章の一つへと形にしてくれました！ あのツイートは取り消したいと思っているでしょうが、あの目標の一つを実現できたことを嬉しく思います。近いうちにゴルフのラウンドを予定できることを願っています！ このとんでもないプロジェクトのために、君のご家族が貴重な時間を割いてくださったことにも感謝しています。ありがとう、友よ！

また、私が新たな本を執筆するという、長く、時に困難な道のりを乗り越える中で、忍耐強く理解を示してくれた、愛する家族、Christine と Everett にも感謝したいと思います。こんな無謀な冒険につき合ってくれて、本当にありがとう！ 夕食が遅くなったり、家族との時間を犠牲にしたりして申し訳ありませんでした。この本が少しでもその埋め合わせになっていればよいのですが。これが作家としての最後の挑戦になるとは約束できませんが、当面の間は一休みするつもりです。

Dan です。初めての著書を執筆するこの道のりを、忍耐強く支えてくれた素晴らしい家族、Jen、Isabella、そして Juliana に感謝したいと思います。君たちがいなければ、この本は完成しなかったでしょう。共著者の Nate へ。人生には、運命的に出会うべき人がいるものです。友よ、君はまさにその一人です。同僚として、そして今は共著者として共に過ごせたことに感謝していますが、何よりも君を友人と呼べることを嬉しく思います。

また、John Kim 氏、Phil Rodopoulos 氏、Jason Delmore 氏、David Wintrich 氏、Tasha Isenberg 氏をはじめ、私のキャリアに多大な影響を与えてくださった多くの方々に感謝申し上げます。ここに名前を挙げきれないほど多くの方々がいらっしゃいますが、私を支えてくださった皆様が、私にとって特別な存在であることをお伝えしたいと思います。

目次

本書への賛辞 …… v
はじめに …… ix

1章　プログラマからエンジニアへ …… 1
1.1　ソフトウェアエンジニアとは …… 1
1.2　基本が重要 …… 2
1.3　ソフトウェアエンジニアになるためのさまざまな方法 …… 3
1.4　教えられたことと知るべきことの違い …… 4
1.5　怠惰なプログラマの精神を身につける …… 5
1.6　新たな視点の価値 …… 6
1.7　解決策に飛びつかない …… 7
1.8　ソフトウェアに黄金律を適用する …… 10
1.9　まとめ …… 11
1.10　実践に移す …… 12
1.11　関連リソース …… 12

2章　コードを読む …… 15
2.1　既存のコードを扱ううえでの課題 …… 15
2.2　認知バイアス …… 17
2.3　初めて触れるコードへの取り組み …… 18
2.4　ソフトウェア考古学 …… 20
2.5　効果的にコードを読む方法 …… 25
2.5.1　IDE の機能を活用する …… 25
2.5.2　テストを読むことで知見を得る …… 31
2.6　練習が上達へ導く …… 33
2.7　まとめ …… 35

2.8 実践に移す …… 35
2.9 関連リソース …… 35

3章 コードを書く …… 37
3.1 車輪の再発明は避ける …… 38
3.2 良いコードとは？ …… 39
3.3 少ないほど良い …… 42
3.3.1 コンピュータサイエンスの第ゼロ法則 …… 42
3.3.2 定型コードに注意する …… 42
3.3.3 継承よりも合成を優先する …… 43
3.3.4 メソッドは短くする …… 44
3.4 読まれることを意識してコードを書く …… 44
3.5 コードのコメントに関する問題 …… 45
3.6 テストをドキュメントとして活用する …… 47
3.7 「小賢しい」コードは避ける …… 48
3.8 コードレビュー …… 49
3.9 チェックボックス式のコードレビューは避ける …… 49
3.10 批判されるのは辛いこと …… 50
3.11 信頼の構築 …… 51
3.12 新しいプログラミング言語を学ぶ …… 51
3.13 まとめ …… 53
3.14 実践に移す …… 53
3.15 関連リソース …… 54

4章 モデル …… 57
4.1 ソフトウェアモデリングとは何か、なぜそれを行うのか …… 57
4.2 どの図が必要か？ …… 60
4.2.1 コンテキスト図 …… 61
4.2.2 コンポーネント図 …… 62
4.2.3 クラス図 …… 63
4.2.4 シーケンス図 …… 64
4.2.5 配置図 …… 65
4.2.6 データモデル …… 67
4.2.7 その他の図 …… 68
4.3 モデル作成のベストプラクティス …… 69
4.3.1 シンプルにする …… 69
4.3.2 図の利用者を把握する …… 70

4.3.3 色の選択には注意する …… 70
4.3.4 標準とテンプレートを策定する …… 71
4.4 ツール …… 71
4.5 まとめ …… 73
4.6 実践に移す …… 74
4.7 関連リソース …… 74

5章 自動テスト …… 75
5.1 自動テストのメリット …… 75
5.1.1 ドキュメントとしての役割 …… 76
5.1.2 保守性の向上 …… 76
5.1.3 自信を高める …… 77
5.1.4 一貫性と再現性につながる …… 78
5.2 自動テストの種類 …… 79
5.2.1 ユニットテスト …… 80
5.2.2 統合テスト …… 80
5.2.3 エンドツーエンドテスト …… 81
5.2.4 どのようにテストを組み合わせるべきか？ …… 81
5.2.5 テストすべきでないもの …… 81
5.3 コードカバレッジ …… 82
5.4 テストの作成 …… 83
5.4.1 はじめに …… 83
5.4.2 アサーション …… 84
5.4.3 ユニットテストの作成 …… 85
5.4.4 モック …… 86
5.4.5 統合テストの作成 …… 88
5.4.6 エンドツーエンドテストの作成 …… 89
5.5 まとめ …… 90
5.6 実践に移す …… 91
5.7 関連リソース …… 92

6章 初めて触れるシステムの調査と変更 …… 93
6.1 初めて触れるコードベースを理解する …… 93
6.1.1 全体像から始める …… 94
6.1.2 実行フローを理解する …… 97
6.1.3 メンタルモデルを段階的に構築する …… 109
6.2 手順の例 …… 114

6.3 安全に変更する …… 114
6.3.1 安全にリファクタリングする …… 115
6.3.2 スカウトのルール …… 117
6.3.3 小さく、元に戻せるように変更する …… 118
6.4 まとめ …… 123
6.5 実践に移す …… 123
6.6 関連リソース …… 124

7章 ユーザーインターフェース設計 …… 125
7.1 すべての人に向けたデザイン …… 125
7.1.1 ユーザビリティとは？ …… 127
7.1.2 アクセシビリティとは？ …… 128
7.1.3 ローカライゼーションと国際化とは？ …… 129
7.2 ユーザーを理解する …… 130
7.2.1 二次的なユーザー …… 132
7.2.2 読者はユーザーではない …… 132
7.2.3 文化の影響 …… 134
7.3 ユーザビリティを最大限に高める …… 135
7.4 デザインの原則 …… 136
7.4.1 コントラスト …… 136
7.4.2 反復 …… 137
7.4.3 整列 …… 137
7.4.4 近接 …… 138
7.5 デザインの原則を適用する …… 139
7.6 「正しいこと」を「わかりやすいこと」にする …… 143
7.7 適切なエラーメッセージの重要性 …… 149
7.8 破壊的な操作 …… 150
7.9 まとめ …… 153
7.10 実践に移す …… 153
7.11 関連リソース …… 154

8章 データの取り扱い …… 155
8.1 データ型と形式を理解する …… 156
8.1.1 構造化データと非構造化データ …… 156
8.1.2 一般的なデータ形式 …… 159
8.1.3 特化したデータに関する考慮事項 …… 162
8.2 データを効果的に保存する …… 165

8.2.1 データベースの種類とそのユースケース …… 165
8.2.2 データの永続化と管理 …… 171
8.2.3 データベース接続とトランザクション …… 174
8.2.4 一貫性モデルとキャッシュ戦略 …… 176
8.2.5 データ量の増加への備え …… 182
8.3 データクエリと管理の最適化 …… 184
8.3.1 効率的なクエリを作成する …… 185
8.3.2 ツールとベストプラクティス …… 189
8.4 データのマイグレーションと変換 …… 194
8.4.1 データ移動の基礎を理解する …… 195
8.4.2 スキーマの変更への対応 …… 197
8.5 まとめ …… 200
8.6 実践に移す …… 200
8.7 関連リソース …… 202

9章 ソフトウェアアーキテクチャ …… 203
9.1 アーキテクチャとは何か？ …… 203
9.2 トレードオフ …… 205
9.3 アーキテクチャと設計 …… 208
9.4 品質特性 …… 209
9.4.1 品質特性の特定 …… 210
9.4.2 ステークホルダーとの合意形成 …… 211
9.5 アーキテクチャスタイル …… 212
9.6 アジャイルアーキテクト …… 213
9.7 適応度関数 …… 214
9.8 アーキテクチャ図 …… 216
9.9 アーキテクチャ決定レコード …… 216
9.10 まとめ …… 219
9.11 実践に移す …… 219
9.12 関連リソース …… 220

10章 本番環境へ …… 223
10.1 本番環境の複雑さ …… 223
10.1.1 ユーザーは予測不能 …… 224
10.1.2「私のマシンでは動くのに」 …… 225
10.2 本番環境向けのコードの作成 …… 228
10.2.1 パフォーマンスの最適化 …… 228

10.2.2 環境固有の設定 …… 230
10.2.3 エラー処理とロギング …… 233
10.2.4 セキュリティの基本 …… 235
10.3 デプロイメントパイプライン …… 242
10.3.1 デプロイ環境 …… 243
10.3.2 バージョン管理の戦略 …… 243
10.3.3 デプロイの自動化 …… 248
10.3.4 デプロイ戦略 …… 251
10.3.5 継続的インテグレーションと継続的デプロイ …… 253
10.4 本番システムの監視と保守 …… 257
10.4.1 監視 …… 257
10.4.2 システム保守 …… 258
10.5 まとめ …… 260
10.6 実践に移す …… 261
10.7 関連リソース …… 262

11章 生産性を向上させる …… 263
11.1 開発環境の最適化 …… 263
11.1.1 自身の開発ツールを知る …… 264
11.1.2 自分だけのライトセーバーを作る …… 265
11.1.3 コマンドラインの力を活用する …… 266
11.1.4 キーボードショートカットの力を活用する …… 268
11.2 戦略的な自動化 …… 271
11.3 生産的な習慣を追い求める不断の努力 …… 271
11.3.1 共同学習 …… 271
11.3.2 個人のナレッジマネジメント …… 272
11.4 まとめ …… 274
11.5 実践に移す …… 275
11.6 関連リソース …… 275

12章 学び方を学ぶ …… 277
12.1 詰め込み学習は効果がない …… 278
12.2 スキルの習得 …… 279
12.3 学習の習慣 …… 280
12.4 AI を活かした学習 …… 283
12.5 FOMO（取り残されることへの恐れ） …… 285
12.6 どこに時間を割くべきか？ …… 287

12.7 イノベーションを実践する …… 289
12.8 アーキテクチャ共有会 …… 291
12.9 無理なく実践する …… 293
12.10 まとめ …… 293
12.11 実践に移す …… 293
12.12 関連リソース …… 294

13章 テクノロジー業界でソフトスキルを磨く …… 295
13.1 共同作業におけるコミュニケーション …… 296
13.1.1 コミュニケーション手段 …… 296
13.1.2 企業内伝言ゲーム …… 303
13.1.3 相手を知る …… 305
13.2 影響力を行使する …… 306
13.2.1 価値を理解し、明確化する …… 306
13.2.2 影響力を行使するための戦略的アプローチ …… 306
13.2.3 ステークホルダーの扱い方 …… 308
13.3 時間管理 …… 310
13.3.1 エンジニアのスケジュール …… 311
13.3.2 タスクに集中する …… 311
13.4 まとめ …… 312
13.5 実践に移す …… 312
13.6 関連リソース …… 313

14章 キャリアマネジメント …… 315
14.1 キャリアパスを計画する …… 315
14.1.1 自分の情熱を注げるものを見つける …… 316
14.1.2 キャリアの選択肢を探る …… 318
14.1.3 目標から逆算する …… 322
14.1.4 計画的なスキル習得 …… 323
14.1.5 人生の節目に合わせたキャリアの選択 …… 326
14.2 キャリアパスを歩む …… 329
14.2.1 成果を祝い、記録に残す …… 330
14.2.2 インポスター症候群を克服する …… 331
14.2.3 プロフェッショナルなコミュニティを築く …… 333
14.2.4 プロフェッショナルな人間関係を築く …… 335
14.2.5 次回の面接を成功させる …… 337
14.2.6 ワークライフバランスを実現する …… 346

14.3 まとめ …… 350
14.4 実践に移す …… 350
14.5 関連リソース …… 351

15章 AIを駆使するソフトウェアエンジニア …… 353
15.1 AIとは一体何なのか？ …… 354
15.1.1 AIの用語を紐解く …… 354
15.1.2 AIの能力と限界を理解する …… 360
15.2 ペアプログラマとしてのAI …… 363
15.2.1 スタンドアロン型チャットボットアシスタント …… 363
15.2.2 インラインIDEアシスタント …… 364
15.2.3 エージェント型AI搭載IDE環境 …… 365
15.3 プロンプトエンジニアリングの基礎 …… 367
15.3.1 プロンプトエンジニアリングとは何か？ …… 367
15.3.2 プロンプトエンジニアリングの必須テクニック …… 368
15.3.3 プロンプトエンジニアリングの高度なテクニック …… 370
15.4 AIがソフトウェアエンジニアリングに与える影響 …… 373
15.4.1 AIに仕事を奪われてしまうのか？ …… 373
15.4.2 バイブコーディング …… 375
15.4.3 AIを戦力増幅器として活用する：コード記述から問題解決へ …… 378
15.5 まとめ …… 379
15.6 実践に移す …… 380
15.7 関連リソース …… 381

訳者あとがき …… 383
索引 …… 385

コラム目次

大文字化という作業 …… 6
昔からそうしてきたから …… 7
5 つのなぜ …… 9
レガシーコードとは …… 16
メトリクスは誤解を招く …… 19
AI はどうでしょう？ …… 34
既存プロジェクトにメトリクスを追加する …… 40
メトリクスを活用したコードカバレッジの向上 …… 41
名前をつけるのは難しい …… 45
プログラミングの本質は、コミュニケーション …… 52
統一モデリング言語 …… 60
図にはコンテキストが必要 …… 70
これは本当に必要なのか？ …… 71
モデルからのコード生成 …… 72
ドキュメントの落とし穴 …… 95
後付け対応の難しさ …… 126
思い込みは禁物 …… 131
デフォルト設定の弊害 …… 133
すべての機能を発見しやすくする必要があるのか？ …… 145
分散システムに関する考慮事項 …… 182
リードレプリカとキャッシュの比較 …… 183
成り行きのアーキテクト …… 206
非機能要件 …… 209
間接的なアプローチの力 …… 211
名前が変われども… …… 214
パフォーマンスの問題 …… 215
「ユーザーは予測不能」という事例 …… 224
もっと良いキーボードを購入する …… 266
Emacs 対 vi …… 269
Notion …… 273
Keep It Simple（シンプルなままにする） …… 274
「新しいものへの憧れ」というパラドックス …… 277
テクノロジーのメリーゴーラウンド …… 282

情熱に導かれて …… 287
独自のテクノロジーレーダーを作成する …… 289
レガシープロジェクトの制約 …… 292
時間あたり数千ドルの損失 …… 299
険悪な雰囲気への対処 …… 304
誰が伝達するかが重要 …… 307
自分の問題を解決するサイドプロジェクト …… 318
私のキャリアにおけるプロフェッショナルネットワークの影響 …… 334
量より質を選ぶ …… 337
AI 模擬面接 …… 342

1章
プログラマからエンジニアへ

基礎スキルは習得が面倒だし、時代遅れに思えることもあります。
もしもエキスパートになる近道があるのであれば、基礎スキルは不要なのかもしれません。
しかし、エキスパートに近づくためには、しっかりとした基礎スキルが不可欠なのです。
— Ethan Mollick『Co-Intelligence』Portfolio、2024 年
（和書『これからの AI、正しい付き合い方と使い方』KADOKAWA、2024 年）

ソフトウェアエンジニアになるには、さまざまな分野にわたる幅広いスキルが必要です。顧客が実際に求めているものを理解すること。それらのニーズを保守しやすいコードへ落とし込むこと。ソフトウェアが想定通りに動作するのを検証するテストを書くこと。機能するユーザーインターフェースを作成すること。アーキテクチャを設計すること。データを取り扱うこと。コードを本番環境へデプロイすること。ソフトウェア技術者として成長していくのであれば、コードだけを書いていてはいけません。ソフトウェアエンジニアリング全体を理解しなければなりません。

プログラマからエンジニアになるには、ソフトウェア開発ライフサイクル（SDLC、Software Development Lifecycle）全体にわたる基礎スキルを習得しなければなりません。また、効率的に働き、わからないことを素直に認め、新しいことを学びつづける方法を身につけなければなりません。本章では、その第一歩を踏み出すためのヒントや考え方を提供します。エンジニアになるためのさまざまな方法と、その過程でよく見落とされがちな重要な知識について学びます。本章の目的は、筆者たちが苦労して獲得した教訓を読者に伝え、プログラマからエンジニアへの道のりを円滑にすることです。

まずは、ソフトウェアエンジニアになる、ということの意味について話しましょう。

1.1　ソフトウェアエンジニアとは

ソフトウェア業界は、複数の意味を持つような（しばしば別の用途から転用された）専門用語であふれており、曖昧な部分がたくさんあります。5 人のソフトウェア専門家に同じ言葉の定義を質問すれば少なくとも 5 通りの答えが返ってくるでしょう。もともと「コンピュータ」とは、計算を

行う人間のことを意味していました。今ではプログラマ、コーダー、開発者、ソフトウェアエンジニアといった言葉が飛び交っています。これらの言葉はすべて同じ意味なのでしょうか。

いいえ、違います。**プログラマ**や**コーダー**は、コードを書くというタスクだけを担う人を指すことが多いようです[†1]。一般的に、特定のプログラミング言語やフレームワークに精通しているかもしれませんが、ソフトウェア開発ライフサイクル（SDLC）全体は理解していないこともあり得ます。多くの人は、プログラマやコーダーとしてキャリアを始めますが、バグ修正や機能実装だけでは昇進することは難しいでしょう。

開発者と呼ばれる人は通常、ソフトウェアの提供という全体像を、より深く理解しています。複数の言語やフレームワークを扱い、さまざまなビジネス領域での経験を持ちます。プログラマやコーダーより経験が豊富で、SDLC のより多くの段階を経験しており、経験の浅いチームメンバーのメンターやペアプログラミングの相手を務めることもあります。

しかし、ソフトウェアエンジニアリングの領域へ進むには、SDLC 全体に**エンジニアリングの原則**を適用することが求められます。**ソフトウェアエンジニア**としては、コードを書く能力は当然の前提であり、スケーラビリティ、信頼性、効率性、セキュリティについて検討することも求められます。アルゴリズムや設計を表面的ではなく、深く理解しておかなければなりません。ルールを知っているだけでなく、そのルールを曲げるべき場合や、さらにはルールを破らなければならない場合も把握しておく必要があります。ソフトウェアエンジニアは、現在稼働しているシステムの中でも、非常に複雑で重要なシステムを担当します。

ブートキャンプや大学では通常、コードを書くという技術的な側面を重視して、プログラミングに精通した人材を育成します。つまり、開発者やプログラマを生み出します。これがソフトウェア業界でのキャリアの出発点です。大きな成果を上げ、昇進し、興味深いプロジェクトに携わるためには、単にコードを書くだけでは不十分なのです。

ソフトウェアエンジニアリングとは、文法的に正しいプログラムを作成するだけではありません。現代では、優れたソフトウェアエンジニアになるためには、単にプログラミング言語を学ぶだけでは不十分です。ソフトウェアプロダクトのライフサイクル全体に精通していなければならないのです。本書はその道のりのガイドとなることを目指し、読者自身もまだ気づいていない学ぶべき領域を示していきます。

1.2 基本が重要

基本が重要です。プロスポーツ選手は、競技を始めた頃に学んだことに大半の時間を費やします。ゴルファーはスタンス、グリップ、アライメントに集中します。バスケットボール選手はレイアップ、パス、フリースローに集中します。ハイライト映像で見かけるようなプレーに費やす時間は、ほんのわずかです。基本は面白くないかもしれませんが、しっかりとした基礎がなければ、彼らは決して高いレベルに到達することはできなかったでしょう。

†1 ちなみに、このような技術者は、エージェント型コーディングシステムに置き換えられてしまうおそれが高いでしょう。

あらゆる職業と同じように、ソフトウェアにも不可欠な基本原則があります。それらを身につけるための時間を確保してください。キャリアの大半は既存のコードと向き合うことになります。他人が書いたコードを読み取り、初めて触れるコードベースを素早く理解することはソフトウェアエンジニアとして成功するために不可欠です。見慣れないコードベースに恐れを抱くかもしれませんが、よくわからないものに向き合うことに慣れる必要があります。数か月経ってもコードベースの細部まで理解できないこともあります。変更を加える前にすべてを理解する必要はありません。

読者は、コードを書くための基本は知っていることでしょう。しかし、後任者の負担を軽減するようなコードの書き方については、それほど詳しくないかもしれません。テクノロジーはとどまることを知りません。したがって、新しいことを学んでいく能力が必須となります。多くのプログラマは新しいテクノロジーや言語の新機能だけを追い求め、昇進につながるような普遍的なソフトスキルを軽視しがちです。基本は流行ほど話題にはならないかもしれませんが、基本ができていれば、キャリアの幅を広げながら、より広い範囲と責任を伴う機会を獲得することができるのです。

1.3 ソフトウェアエンジニアになるためのさまざまな方法

ソフトウェアエンジニアになる方法はさまざまです。大学でコンピュータサイエンスの学位を取る方もいれば、ブートキャンプに参加する方、独学で学ぶ方もいます。電気工学などの関連分野の学位を取得後、プログラミングに転向する人も少なくありません。しかし実際のところ、ソフトウェアを構築する方法をどのようにして学んだかはあまり重要ではありません。

自分の選んだ方法が「より優れている」「より速い」「より安上がりだ」と主張する人もいるかもしれません。どのアプローチにもメリットとデメリットがあります。独学であれば授業料はかからず、適性によっては数年ではなく数週間で習得できるかもしれません。インターネットには動画やチュートリアルがあふれています。しかしながら、情報を探し出すのは大変で、厳しく自己管理しなければなりません。独学では品質にばらつきのある教材を使うため、かなりの試行錯誤を繰り返すことになるかもしれません。

ブートキャンプと学位プログラムでは、目的そのものが異なります。学部レベルのコンピュータサイエンス課程は、大学院への進学準備のために設計されています。アルゴリズム、言語設計、コンパイラ理論、オペレーティングシステムなどの関連分野に重点を置きます。しかし、これらは、業務で日々アプリケーション開発に取り組むための準備とは限りません。プロジェクトは小規模で、期間も数か月を超えることはほとんどなく、既存のアプリケーションに縛られることなく、ゼロからの開発がほとんどです。

ブートキャンプは、ある特定の知識を短期間、多くの場合は数週間で集中的に学ぶものです。流行の言語やフレームワークなどに重点を置きます。このアプローチはより実践的だと言えますが、言語やフレームワークの人気は浮き沈みするため、知識はあまり長い間役立つわけではありません。ブートキャンプは大学内に設置されることもあり、学部レベルのコンピュータサイエンス講座を担当する教員が教える場合もあります。つまり、教える側にはあまり違いがないのかもしれません！

必要な時間も、アプローチによって異なります。ブートキャンプは数週間で約 600 時間を要します。困難な状況に直面したとしても、指導者にほぼ常時対応してもらえる環境です。準学士号は通常 1〜2 年で 850 時間、学士号プログラムは 4〜5 年で約 1,400 時間を要します。

最初のうちは学部卒にメリットが多いと主張する人もいますが、時間の経過とともに差は埋まっていく傾向にあります。言うまでもなく、現在のコンピュータサイエンスの課程は、比較的歴史が浅いものです。ほとんどのプログラマが数学を専攻していた時代もそう遠くはないのです。



初期のコンピュータサイエンス学位プログラムは、数学を非常に重視し、実質的に数学の学位にコンピュータサイエンスを少し加えただけのものもありました。実際、教授陣の多くは数学の学位を持っていました。今日でも、数学的素養はソフトウェアエンジニアとして成功するための必須条件だと考える人がいます。実際にはそんなことはありません。コーディングはコミュニケーションであり、仕事の大半はコンピュータではなく他の開発者とのやり取りです。言語能力は数学的スキルよりもはるかに重要です（https://oreil.ly/qCWAx）。コードを書くことは数字ではなく言語を使うのです。これについては 2 章で説明します。

結局のところ、ソフトウェア業界で成功していくには、問題解決能力、探究心、創造性が必要です。そのマインドセットこそが重要なのです。スキルをどう身につけたかは重要ではありません。自分が選んだ方法について言い訳する必要もないのです。

1.4 教えられたことと知るべきことの違い

どのように学習してきたかにかかわらず、読者はコードの書き方を学んできたはずです。言語を 1 つや 2 つ学び、`foo` や `bar` といった汎用的な変数名について学びました。デバッグや効率的なアルゴリズム設計などの話題も少しは学んだことでしょう。しかし、重要なことの多くは、一般的なカリキュラムや独学では学ぶことができません。なぜなのでしょうか。それは、時間が不足していたり、その知識がすでに知られているものと想定されていたりするためです[†2]。

これらのコースや教材が割愛している内容は、長期的な視点で見れば、読者の成功に重大な影響を与え得るものです。このように割愛されるテーマには、他者との協働方法、コードの読み取り方、**可読性の高い**コードの書き方、レガシーコードの効果的な扱い方などが含まれます。本書では、このようなテーマも含めてさまざまなものを取り上げます。

多くの開発者は、既存のコードベースの技術的負債を一切持たない、白紙の状態の新規プロジェクト（グリーンフィールドプロジェクト）を学びます。学期やブートキャンプカリキュラムの時間的制約から、ほとんどの教育現場では特定のスキルを練習したり、概念をより深く理解したりするために、ゼロから始めることになります。プロジェクトは往々にして非常に小規模なもので、単独で動くものであるため、一人でコーディングすることも多いでしょう。これは教育環境では有効ですが、実際の業務内容を反映しているとは言えません。

†2 デバッガーやモダンエディタの使い方のようなものです。

ソフトウェアエンジニアとして働いている間、読者はほとんどの時間を既存のレガシーアプリケーションや過去のアプリケーション、そして読者が関わる前に下された決定のために負の遺産が残されているプロジェクトと向き合うことになります[†3]。たとえ新しいコードベースという恵まれた環境だったとしても、数か月後にはそこにも技術的負債が蓄積していきます。モダンソフトウェアプロジェクトでは、世界中のチームが数百万行ものコードを扱います。常に誰かと協働することになります。チームメンバーの場合もあれば顧客の場合もあるでしょう。読者のコードは評価用のリポジトリに眠っているのではありません。本番環境で稼働し、設計通りに機能すると人々が信頼することで、ビジネス価値を生み出していくのです。控えめに言っても、読者がソフトウェアの現実を初めて知ったときには、その衝撃は非常に大きいものになるでしょう。

学校でのプロジェクトは短期間です。数か月、あるいはそれ以下です。しかし、実際には、プロジェクトは決して終わりを迎えることはありません。芸術のようにいつか放棄されることはあっても、コードは生涯の大半を保守フェーズで過ごします。つまりソフトウェアは、明確な開始と終了を持つようなプロジェクトではなく、継続的な投資が必要なプロダクト群で構成されているものなのです。ソフトウェアプロダクトがビジネス価値を提供しつづける限り、投資は続いていくのです。

1.5 怠惰なプログラマの精神を身につける

怠惰なプログラマの精神とは、SNS でお気に入りの猫動画を見て一日を過ごすことでも、仕事をサボる手法でもありません。**怠惰なプログラマの精神**とは効率性に焦点を当てた哲学のことです。多くの新人ソフトウェアエンジニアは本能的に急いで解決策を見つけようとします。問題を十分に考えず、すぐにコードを書き始めます。あえて戦略的に怠惰であろうとすることで、思考の時間を持ち、最終的に生産性を高め、より良い解決策を見つけられます。

> 時間をかけることで、かえって時間を節約できるのです。
> — Pat Parelli（によるとされる）、著名な馬術トレーナー

力任せなアプローチには注意が必要です。そこから始めたとしても、何度も設計を見直すべきです。ビッグオー（Big O）記法のような概念を理解することは有益です[†4]。例えばバブルソートを実装したことがあったとしても、それが最も効率的なソート方法ではないと学んでいるはずです。エンジニアとして最良ケース・最悪ケース・平均ケースを考慮し、適切な選択をすべきです。

実際、ある特定の問題を解決するのは読者が初めてではない場合が多いのです。時間を割いて、既存の解決策やライブラリを探してください。多くの場合、10 分検索すれば数日分の作業を節約できます。もっと簡単な方法があるはずだと感じたら、おそらくそれは存在します。この点につい

†3 物事がなぜそうなっているのかを問うことは、どんな場合でも意味のあることですが、同僚の推測が自分と同じくらい当てずっぽうなものだったとしても驚かないでください。

†4 端的に言えば、ビッグオー記法は、あるアルゴリズムが他のアルゴリズムより効率的であることを説明するものです。

ては、「3.1 車輪の再発明は避ける」で詳しく説明します。

大文字化という作業

Nate です。この業界に入ったばかりの頃、とても単純なタスクを任されました。データベース内の請求先住所を大文字化して、請求書の印刷・郵送時に郵便局が混乱しないようにすることでした。テックリードが、何をどこで修正すべきか、修正すべきルーチンと、請求先住所を大文字化するのに使うべき関数（もういない誰かが書いたもの）を具体的に教えてくれました。その大文字化関数を見て、私はそれが複雑すぎると思いました。私の考えでは、引数は1つで十分なはずでした。その引数とは、大文字化したい文字列です。しかし、この関数の引数は1つではなかったのです。10個以上もあったのです。この自社開発の関数を理解するには何時間も必要でした。

これはプログラミング言語が解決すべき問題だと思いました。少し検索してみると、案の定、そのプログラミング言語には組み込みの大文字化関数が存在しました。私はそれを使うことにしました。翌週のチームミーティングで、各自の作業内容を順番に報告する中、私はこの大文字化作業について触れました。すると同僚の一人が私の方を見てこう言ったのです。「Joe が作った大文字化関数を使ったのですか？」 私は「いいえ、使っていません」と答えました。

すると、その場にいた全員がこちらを振り向きました。まるでレコードの針が突然止まったかのような瞬間でした。私がその場で最も経験の浅い開発者だったにもかかわらず、他の誰も言語に組み込みの大文字化関数があることを知らなかったのです。おそらく、以前の開発者が関数を書いた後に言語機能が追加されたのでしょう。詳しいことはわかりません。ただ、重要なのは、もっと簡単な方法があると思うなら、時間をかけて探すことです。選択肢や代替案を数時間調査するだけで、数日あるいは数週間の作業を節約できるかもしれないからです。

1.6 新たな視点の価値

最も危険な言葉は「我々は常にこの方法でやってきた」です。
— Grace Hopper、コンピュータの先駆者・海軍士官

経験豊富な同僚ほど多くの年数やコード行数を積み重ねていなくても、新人開発者には貴重な資質があります。それは新たな視点です。コードベース、チーム、ソフトウェア業界のいずれにおいても、新たなメンバーは偏りのない視点で課題を見極めます。過去の決定事項に縛られず、もはや有用ではなくなった慣習に囚われていないからです。この新たな視点は革新的な解決策を生み出し、非効率な点を指摘し、長年続いてきた前提に疑問を投げかけます。また、停滞気味のプロジェクトに活気をもたらし、チームに新たなエネルギーと意欲をもたらすこともあります。

昔からそうしてきたから

Nate です。数年前、あるクライアントのアプリケーションについてパフォーマンス問題を解決するお手伝いをしました。その問題とは、夜間バッチジョブでウィジェットを処理する方法に関するものでした。会社が立ち上がった当初は、クライアントの顧客も、ウィジェットの数も少なく、夜間の処理はすぐに終わっていました。しかしクライアントが成功するにつれ、クライアントの顧客も増え、ウィジェットの数も増えていきました。私が関わった頃には、夜間のバッチ処理はもはや一晩では終わらなくなっていたのです。

私は彼らと一緒に、アーキテクチャを検証しました。数時間後、私は単純な質問を投げかけたのです。「ウィジェット A のプロセスは、ウィジェット B のプロセスと何か関係があるのですか？」 答えはノーでした。最終段階で集計処理は行われるものの、個々のウィジェットは独立していたのです。そこで私は提案しました。ウィジェットを並列処理できないでしょうか？ 次のウィジェットを取得し、それを処理して、集計のために結果をキューに入れる一連のワーカーを立ち上げる、という案です。

一瞬、時が止まりました。そして、そうだ、そうだ、それでうまくいく、という声が上がりました。確かに、上司には、私に特別な洞察力があると思わせたかったのですが、実際には私にはそのような洞察力があったわけではありません。ただ組織において最も危険な言葉「昔からそうしてきたから」に縛られていなかっただけだったのです。昔からそうしてきたからといって、それが正しい方法だとは限りません。ましてや最善の方法だとは言えません。何年も前に誰かが採用した手法は、偶然の産物か、単に最も手っ取り早い選択肢だっただけかもしれないのです。おそらく、もっと良い方法があるはずです。質問したり、別のアプローチを探したりするようにしてください。

異なる視点の価値は、決して過小評価してはならないのです。物事がなぜそのようになっているのか、遠慮なく問いかけるようにしてください。奇妙に思えるもの、場違いなもの、あるいは単に理にかなわないものに出会ったら、チームメンバーに説明を求めてみてください。場合によっては、型破りと思えるようなアプローチにも正当な理由があるかもしれません。しかし多くの場合、人々は単に自分が着任したときにあったものをそのままコピーしているだけです。過去の（そして現在の）開発者を侮辱するつもりではなく、学びたいという姿勢で質問するのであれば、ほとんどのソフトウェアエンジニアは前向きに議論に参加してくれるはずです。

1.7 解決策に飛びつかない

問題に直面したとき、すぐに解決策に飛びつきたい、という衝動を抑えてください。代わりに、時間を割いて問題の根本原因を理解するようにしてください。考えるための余裕を持つのです。中断されずに考える機会はそれほど多くはありません。仕事の予定表は会議で埋まり、メッセージ

ツールの通知にも一日中何度も割り込まれることでしょう。デイリースタンドアップ、プロダクトオーナー、上司、チームリーダー、アーキテクトとのやり取りで、数時間おきに状況報告をしているはずです。やるべきことは減らず、テスターはまた厄介なバグの山を発見します。時間的プレッシャーは強いと思いますが、解決策を考える時間を確保し、安易な対処法に飛びつくのは避けるようにしてください。

とりあえずの解決策は当面の問題を解決しますが、長期的にはさらなる問題を引き起こすおそれがあります。この問題に対処するため、開発サイクルの早い段階で問題を発見するためにテストを行う動き、「シフトレフト」が提唱されています。本番環境でバグが発見されてしまうと、修正コストが圧倒的に高くなるからです。顧客に影響が及んでおり、会社に金銭的損失をもたらすおそれがあります。データが破損している危険性もあり、その復元は困難です。根本的な問題の修正には複数のルーチンにまたがる複雑なリファクタリングが必要となり、他チームとの調整も行わなければならない場合も多いのです。

シフトレフトは、問題修正についての話だけではありません。価値あるものをプロセス全体において前倒しするということが本質です。作業完了の数日（あるいは数週間）後に行っていたコードレビューは、ペアプログラミングを活用することでリアルタイムなものへと変わっていきます。テストを継続的な活動とすることで、問題が発生した直後に特定し、解決しやすくなります。継続的インテグレーションは、早く頻繁にマージすることでコード変更の競合という問題を軽減します。さらに、継続的デリバリーと組み合わせれば、顧客に影響を与える前に環境における問題が発見できます。また、開発プロセス全体でセキュリティを意識することで、壊滅的な脆弱性が発生しないようにします。

テスト段階でこれらの問題を発見するほうがはるかにコストが低くなります。コードを記述してすぐにバグが発見できれば、開発チームはその作業の内容をまだ覚えているため、コードの解読に費やす時間を節約できます。テスト用データストアは最小限の手間で復元できます。解決する問題が複雑な場合でも、顧客に影響は出ないので、迅速な修正を迫られるという時間的プレッシャーも大きくありません。品質保証（QA）テストと QA エンジニアは、このような理由から生まれました。開発段階で問題を発見すればさらにコスト削減が可能であり、これがテスト駆動開発（5 章参照）を推進する一因となっています。

しかし、そもそも最初からバグを作り込んでいなければどうでしょうか。問題が設計段階で解決されていたとしたらどうでしょう。

> ソフトウェアで直面する大きな問題のほとんどは、誤解に起因します。実行する前に何をすべきかをはっきりと理解していないのです。そして「急げ、急げ」とすべてを押し進めてしまいます。
>
> — Rich Hickey、Clojure Conj 2010 基調講演より

誤解の問題はソフトウェア開発の現場に蔓延しており、開発における共通の課題です。「クライアント」と「カスタマー」という言葉は同じ意味に思えます。プロダクトオーナーが「そのアプ

ローチはクライアントには有効だが、カスタマーには通用しない」と言うまで、実はこの２つの言葉が異なる意味を持つことに気づかないのではないでしょうか。この２つの言葉は類似していますが別物であり、その微妙な差異が重要だったのです。どんなにテストを重ねても、どんな型システムでも、誤解の問題は決して発見できません。技術的な解決策だけではこれらの問題は解決できないのです。理解が根本的に間違っていることに起因しているからです[†5]。

誤解から生じた問題に対処するより、そもそも誤解が生じないようにすべきです。では、どうすればよいのでしょうか？ 問題解決に集中することです。アプリケーションがどのような問題を解決するのか知っているでしょうか？ コンテキストと制約条件を理解していますか？ 関連する問題は何でしょう？ 問題領域をじっくり考える時間を持つようにしてください。

開発者は、解決策を過剰に設計しがちです。必ずしも悪いことではありませんが、問題を解決する方法は数多く存在します。解決しようとしている問題を見失わず、安易に解決策に飛びつかないようにしましょう。顧客は何を求めているのでしょうか？ 彼らの真のニーズは何ですか？ 経験から得た教訓として、自らを追い詰めるような状況には決して陥りたくないという理由で、必要になる**かもしれない**ものをすべて想定しがちです。しかし、後で捨てなければならない大量のコードを構築することも避けたいはずです。さらに悪いのは、そのコードが使われないまま放置されてしまうことです。

難しいことかもしれませんが、顧客の要望の背後にある「なぜ」、つまり根本原因を突き止めてください（コラム「5 つのなぜ」参照）。質問を投げかけましょう。顧客と話し、プロダクトオーナーと話し合ってください。多くの場合、顧客があることを要望していたとしても、実際には自分では気づいていない別のものが必要なのです。彼らは「昔からそうしてきたから」という固定観念に縛られていることも少なくありません。時には、顧客が具体的な解決策を提示してくれることもあります。例えば、「このテーブルをスプレッドシートにエクスポートできるようにしてほしい」といった具合です。すぐにその機能を開発したくなりますが、経験豊富なエンジニアであれば、まずその根底にあるニーズが何なのかを尋ねます。顧客が求めているものと、彼らのビジネス課題を解決するものの間には、しばしば乖離があるのです。

5 つのなぜ

トヨタ生産システムの一部として生まれた「5 つのなぜ」は、「なぜ？」という問いを 5 回繰り返し、その都度、前の「なぜ？」への回答を基に次の「なぜ？」を突き詰めることで、問題の根本原因を突き止めようとする問題解決手法です。表面的な症状の背後に潜む根本的な問題を掘り起こせます。例を示します。

- 問題：ロボットが動かなくなりました。
  - なぜ？ 回路がオーバーロードになり、ヒューズが飛びました。

†5 用語集は、プロジェクト成果物の中で最重要なものの一つです。プロジェクトに用語集がないなら、作成を検討すべきです。

- なぜ？ ベアリングの潤滑が不十分で固着したからです。
- なぜ？ オイルポンプが十分な油を循環させなかったからです。
- なぜ？ ポンプの吸入口が金属の削りくずで詰まっているからです。
- なぜ？ ポンプにフィルターが付いていないからです。

ソフトウェアの例で説明しましょう。Nate です。何年か前、ある Web ページに出くわしました。そこでは、すべての終了タグが次の行に書かれていました。インタプリタにとっては問題ありませんでしたが、行の先頭に毎回終了タグがあるのは見た目に違和感を覚えました。同僚に「**なぜ**誰かがこんな手法を使ったのか」と尋ねました。彼は、私たちが運用していた Web サーバーの古いバージョンにあるバグが原因だと答えました。「なるほど、では**なぜ**そのバージョンをまだサポートしているのですか？」と問うと、「ああ、もうサポートしていない」と返ってきました。「では、**なぜ**そのバグは修正されていないのですか？」「ああ、修正済みですよ」「では、**なぜ**まだこのパターンを見かけるのでしょうか？」「ええと、おそらく古い Web ページだったからでしょう」 私は、そのページが数日前に作成されたものだと彼に伝えました。そこで私たちは、開発者に対してこのパターンを使用しないよう通知すべきだと合意し、まだ古い手法が使用されているページを修正するためのクリーンアップタスクを追加しました。

多くの場合、要求事項を挙げてくるのは、主に旧システムのスーパーユーザーです。彼らの知識は極めて貴重なものですが、同時に、彼らが現在利用している環境に左右される面もあります[†6]。場合によっては、特定の機能が**なぜ**そうなっているのかという根本的な理由を理解しておらず、その機能が特定の問題を解決する唯一の方法であると想定してしまうことがあります。つまり、彼らは（意識的か無意識的かを問わず）新しいテクノロジーを用いて旧システムを単に複製するという方向へ導いてしまうおそれがあります。

視野を広く持つことが重要です。顧客が実際に抱える問題を解決するために、適切なツールを選んでください。多くのエンジニアは楽観的すぎます。現実をよく見て、徹底して実用的に考えてください。ただしスケーラビリティは重要です。すべてのアプリケーションがインターネットトラフィックの 3 分の 1 を処理できるわけではありません。

予測は極めて難しい。特に未来に関する予測はなおさら。
— デンマークのことわざ

1.8 ソフトウェアに黄金律を適用する

おそらく、学生時代の早い段階で、黄金律（https://ja.wikipedia.org/wiki/黄金律）について

†6 だから多くのアプリケーションがメールやスプレッドシートに退化してしまうのです。

学んだことでしょう。それは「自分が他人にしてもらいたいと思うように、他人にも接するべきだ」というものです[†7]。この原則は世界中で受け入れられており、文明社会で生きるための基本原則として、数千年の歴史を持つものです。しかし、実は、これは単なる倫理原則にとどまりません。コードに対して、同じ原則を適用できます（そして、そうすべきです！）。

自分の後を継ぐ開発者のことを考えてみましょう。そしてその開発者は、実は自分自身かもしれないのです[†8]！ 未来の**自分**がコードベースに何を求めるのでしょうか？ 次の開発者の負担を軽減するには、どのようにすればよいのでしょうか？ コードは、究極的にはコミュニケーションの手段です。確かに、コードはコンピュータにタスクを実行させるためのものではあります。しかし、他の開発者とコミュニケーションする機能に比べれば、その重要性ははるかに劣ります。読まれることを前提にコードを書いてください。最適化する際には、コンパイラではなく、人間のために最適化してください[†9]。

他の人とは一線を画したいのであれば、黄金律を守ってください。後任者が理解し、保守できるよう、読みやすく整理されたコードを書くようにしてください。ドキュメントを更新し、他者が古い内容に振り回されるのを防ぎましょう。混乱させるのではなく、状況を明確にするようなわかりやすい図を作成しましょう。質問に答え、他者を助けるための時間を確保しましょう。これらすべてを実践すれば、読者は重宝される存在となります。周囲の人々の力を引き出すことができるため、誰もが読者をチームに迎えたいと思うはずです。

1.9 まとめ

ソフトウェアエンジニアとしてのキャリアは、ただ画面を見つめてコードを書き連ねるだけではありません。単なるコーダーやプログラマを超えた存在になるためには、文法的に正しいコードを書くこと以上のものが求められます。そこには、多岐にわたるスキルセットが不可欠なのです。そのスキルをどこで、どのように学んだかは問題ではありません。コンピュータサイエンスの学部課程であれ、ブートキャンプであれ、オンラインで見つけた教材で学んだのであれ、重要なのは、それらのスキルをどのように組み合わせるかということです。

キャリアを成功に導くために必要な知識は、学校で教わったことだけではありません。本章では、解決策に飛びついてしまうこと、過剰に設計してしまうこと、そして力業的なアプローチの危険性など、よくある落とし穴を避けるためのアドバイスを共有しました。こうした教訓を学ぶのは、早ければ早いほどよいのです。

†7 より一般的な言い方では「他人にしてもらいたいことを、他人にもしなさい」です。

†8 繰り返しますが、すべての開発者は「一体どんな馬鹿が書いたんだ」と思いながらコードを見つめ、やがて自分がその馬鹿だと気づくものです。

†9 ただし、特定環境の非常に特殊なプログラミングの場合は例外です。どのようなものが例外となるかは、そのコードを見た瞬間にわかります。

1.10 実践に移す

キャリアの早い段階において、メンターを見つけると非常に効果的です。所属する組織を見渡し、経験豊富なソフトウェアエンジニアの方に、コーヒーを飲みながらお話をする時間を設けていただけないか、丁寧に尋ねてみてください。この際、読者がコーヒーをおごりましょう。その方々の経験や、学んだこと、もっと早く知っておけばよかったと思うこと、チーム内でより価値ある存在になれた要因、読者が学ぶべきだと思うことなどについて話し合ってみてください。また、イベントの講演者や著者に連絡を取るのもよいでしょう。彼らの多くは、自ら声をかけてくれる人を喜んで助けてくれます。

次にバグ修正や新機能の開発を任されたら、すぐにコードに取り掛かる前に、いったん立ち止まりましょう。解決しようとしている問題を正しく理解しているかどうかを確認しましょう。30 分ほどかけて調査を行ってください。既存の解決策を活用できないでしょうか？ 問題領域における概念や、考えられる解決策を図示したり、モデル化したりしてみてください。経験豊富な同僚と選択肢について話し合うのもよいでしょう。

本書の目次にざっと目を通してみてください。どの分野が自分の得意分野だと思いますか？ また、どの分野をさらに磨く必要があるでしょうか？ もっと学びたいと思う分野を一つ選び、そのための計画を立ててください。「実践に移す」の節を実際に試してみて、その章の関連リソースも読んでみてください。

お使いのフレームワークやライブラリ、あるいはクラウドプロバイダーは、日々の作業から特定の要素を抽象化してくれるかもしれません。例えば、Spring Data を使えば、数え切れないほどの SQL 文を書く手間を省けます。こうした抽象化による生産性の向上はぜひ活用すべきです。しかし、同時に、現在活用している抽象化レイヤーの**下にある**レイヤーについても理解しておく必要があります。時間をかけて、その内部を覗いてみましょう。そこで得られる知識は、特に何かが思ったように動作しない場合などに、非常に役立つことでしょう。

1.11 関連リソース

- 『The Pragmatic Programmer: Your Journey to Mastery, 20th Anniversary Edition』Andrew Hunt, David Thomas 共著（Addison-Wesley Professional、2019 年）
  和書『達人プログラマー（第 2 版）』（オーム社、2020 年）
- 『The Mythical Man-Month: Essays on Software Engineering, Anniversary Edition』Fred Brooks 著（Addison-Wesley、1995 年）
  和書『人月の神話 新装版』（丸善出版、2014 年）
- 『Design Patterns: Elements of Reusable Object-Oriented Software』Erich Gamma 他著（Addison-Wesley、1994 年）
  和書『オブジェクト指向における再利用のためのデザインパターン（改訂版）』（ソフトバンククリエイティブ、1999 年）

- 『Practices of an Agile Developer』Venkat Subramaniam、Andy Hunt 共著（The Pragmatic Bookshelf、2006 年）
  和書『アジャイルプラクティス 達人プログラマに学ぶ現場開発者の習慣』（オーム社、2007 年）
- 『The Productive Programmer』Neal Ford 著（O'Reilly Media、2008 年）
  和書『プロダクティブ・プログラマ』（オライリー、2009 年）
- 『Software Engineering at Google』Hyrum Wright 他著（O'Reilly Media、2020 年）
  和書『Google のソフトウェアエンジニアリング』（オライリー、2021 年）
- 『The Staff Engineer's Path』Tanya Reilly 著（O'Reilly Media、2022 年）
  和書『スタッフエンジニアの道』（オライリー、2024 年）
- "No Silver Bullet: Essence and Accidents of Software Engineering"、Frederick P. Brooks Jr. 著（The University of North Carolina at Chapel Hill Department of Computer Science、1986 年 9 月、https://www.cs.unc.edu/techreports/86-020.pdf）
- 『Code Complete, 2nd Edition』Steve McConnell 著（Cisco Press、2004 年）
  和書『CODE COMPLETE 第 2 版』（日経 BP、2005 年）

2章
コードを読む

コードは書くことよりも読むことのほうが多い。
— Guido van Rossum、Python の生みの親

一般的なコーディング教育とは異なり、ソフトウェアエンジニアはコードを書くよりも、はるかに多くの時間をコードを読むことに費やしています。ほとんどの初心者向けコーディング講座では、すぐにコードを書く作業に飛び込み、プログラミング言語のコアとなる概念や構文に焦点を当てます。もっとも、ポーランド語やポルトガル語を同じ方法で学ぶ人はいませんが。また、大学でのプロジェクトの多くは白紙の状態から始まるのに対し、実際の開発者はほぼ常に、何年もかかって現在の状態になっているコードを相手に作業しています。この点については、6 章で詳しく説明します。

エージェント型プログラミング／チャット指向プログラミング[†1]の登場により、ソフトウェアエンジニアにとってコードを読むことは、**これまで以上に**重要になるでしょう。自らが書いたコードではないものを扱うことになるかもしれませんが、心配はいりません。初めて触れるコードに直面した際、状況を把握するためのテクニックが**存在します**。本章では、コードを読むことがなぜ難しいのかについて説明し、そのプロセスを簡単にするためのヒントを提供します。

2.1　既存のコードを扱ううえでの課題

プログラミングをどのように学んだかにかかわらず、おそらく**グリーンフィールド開発**と呼ばれる幸せな環境で多くの時間を過ごしてきたことでしょう。そこでは過去の作業に縛られることなく、ゼロから始めるという仕事を体験します。しかし、プロとして働く場合は、真っ白なエディタからアプリケーションを構築する機会はほとんどありません。実際のソフトウェアエンジニアは、既存のコードベースの制約の中で、レガシーコードを扱う**ブラウンフィールド開発**に多くの時間を費やします。

†1　**バイブコーディング**とも呼ばれます。

レガシーコードとは

古いコードを読むということは、多くの開発者が避けたいと思う作業です。私たちはそれを**レガシーコード**と呼ぶことがよくありますが、この言葉が褒め言葉として使われることはほとんどありません。レガシーコードにはさまざまな定義があります。昨日書かれたコード、テストカバレッジが不十分あるいは過剰なコード、あるいは単に自分が書いていないコードなどです。

しかし、既存のアプリケーションがうまく動作しているという事実を軽く見てはなりません。あるプロダクトが何年もビジネス価値を提供し、継続的に投資されているのであれば、それは称賛に値します。「継承コード」や「既存コード」といった、より前向きな表現を使うほうがよいでしょう。この話題についてより深く知りたい場合は、6章を参照してください。

既存のコードに関わる問題の解決を求められた場合、実際には**4つ**の問題を解決する必要があります。

何よりもまず、解決しようとしているビジネス上の問題を理解する必要があります。開発者が取り組む分野は非常に複雑なのです！「ソフトウェアが世界を飲み込んでいる」(https://oreil.ly/lY6A9)と言われるように、ソフトウェアエンジニアは、ますます複雑化するビジネス上の課題に取り組まなければなりません。いわゆる「手の届く成果」の多くは、すでにもぎ取られてしまっているのです。

第二に、自分より前にそのコードを書いた開発者の視点に立って問題を見つめ直す必要があります。これこそがソフトウェア開発において最も難しい点です。開発者によって問題解決の方法は異なり、それぞれ習慣や傾向が違います。前任者は、読者にはあまり馴染みのない関数型プログラミングスタイルを好んでいたかもしれません。あるいは、読者にとって馴染みのないパターンを採用していたり、これまで使ったことのないライブラリを活用していたりするかもしれません。そのアプローチを好むかどうかにかかわらず、ある問題を解決する方法は数多く存在します。

第三に、既存コードの抽象化レベルが適切でないおそれがあります。モデル化が汎用的すぎたり、そのドメイン固有の微妙な問題を捉えきれていなかったりするかもしれません。あるいは、前任者が2つの概念を混同していたり、本来は別のパターンのほうが適切だったにもかかわらず、好んで使っていたパターンを無理に適用していたりするかもしれません。いずれにせよ、コードを理解しやすくするには、大規模なリファクタリングが必要になる可能性があります。

第四に、考古学者と同じように、技術的負債(https://oreil.ly/qFfTu)の山を次々と剥がしていくことになります。かつて使われていたテクノロジーや手法は、現在ではアンチパターンと見なされるかもしれません。時間の経過とともに、プログラミング言語やベストプラクティスは進化していきます[†2]。そのため、コードが書かれた当時の視点に立って、そのコードを見ていく必要が

†2 Neal Ford は「今日のベストプラクティスは明日のアンチパターンだ」と述べています。

あります。使われている、あるいは使われていないフレームワークや言語機能に注目するだけで、コードの「年代測定」さえできるかもしれません！ Spring のようなフレームワークは長年にわたって進化しており、特定の問題に対して複数のアプローチをサポートしていることがよくあります。何かを行うための「正しい」方法は、時とともに変化するものです。

さらに厄介なのは、パッチの上にパッチを重ねたコードを扱わなければならないことがほとんどであるという点です。前の開発者が問題を完全に理解していなかったのかもしれません。あるいは、新しい言語機能を使えば目の前の作業を大幅に簡略化できたのに、その機能を知らなかったのかもしれません。もしくは、前の開発者が AI コーディングアシスタントのプレミアムライセンスを所有しており、1 日に 2 万行ものコードを生成していたのかもしれません。さらに「すぐに直して」というよくある要求も加われば、たった一つのメソッドを解読することに、半日を潰してしまうことになりかねません。

既存のコードを扱うことは技術的な課題を伴うため、それが開発者にとって最も嫌われる作業の一つとなりがちなのも理解できます。しかし、その嫌悪感の背景には、技術的な問題だけでなく認知バイアスもあります。

2.2 認知バイアス

既存のコードを読む際、自分のコードと比べて、「あまり良くないコードだな」と思うかもしれません。もちろん、**読者自身**は悪いコードを書かないですよね？ 私たち（この本を書いている者）も、何度かコードに苦戦し、「一体誰がこんな馬鹿げたコードを書いたんだ？」と（もっと失礼な表現で）叫んだことがあります。ところが、それを書いたのは他ならぬ自分自身だったのです。率直に言って、数年前に自分が書いたコードを読むと、少しがっかりするのは当然のことです。それは成長の証であり、今の自分が当時よりも多くのことを知っているのですから。これは素晴らしいことです！

既存のコードを扱う場合、いくつかの認知バイアスが働きます。一つ目は「IKEA 効果」です。これは自分で作成したものに対して、実際よりも高い価値を感じてしまう現象のことです。ある研究によると、自分で組み立てた製品に対しては、他人が組み立てたまったく同じ製品よりも 63% 高い金額を支払うという結果が示されています[†3]。リンゴ狩りに行ったことがある方ならわかると思いますが、自分で作業する分、割高な料金を請求されることがよくあります。しかし、その体験や、リンゴの木から直接最高の果実を選べるという機会を得るために、喜んでその料金を支払うのです。ソフトウェアの世界でも、開発者は物事の「正しい」やり方についてこだわりを持つことが多く、自分のコードやアプローチを好む傾向があります。

さらに、「単なる接触効果」というものもあります。つまり、人はすでに慣れ親しんでいるものを好む傾向があるのです。こうしたことが、多くの開発者がプログラミング言語に対して抱きがちな

†3 “The ‘IKEA Effect’: When Labor Leads to Love”、Michael I. Norton, Daniel Mochon, Dan Ariely 共著（HBS Working Paper, No. 11-091（2011））、https://oreil.ly/zuETL

思い込みにつながっています。開発者は、自分が最初に学んだ言語こそがすべての基準であると考えがちです。Java が初めてラムダ式を導入した際、ある言語別のメーリングリストで、なぜ Java にこのような「新奇なラムダ」が必要なのか、と問う声が上がりました。しかし、ラムダはプログラミング言語において新しい概念ではなく、そもそも Java 自体の当初の計画にも含まれていたことに気づいていなかったのです！

開発者は、自分が好むツールに関して視野が狭くなりがちです。これは Paul Graham がエッセイ “Beating the Averages（普通のやつらの上をいけ）”（https://oreil.ly/1MAVi）で触れていることです。Graham は、プログラミング言語は「表現力の連続体」の上に存在すると述べています。しかし、ある言語が別の言語より**なぜ**優れているのか、その理由を認識できないことがよくあります。この主張を裏付けるため、彼は架空の言語「Blub」と、Blub を駆使する生産性の高いプログラマを例に挙げました。Blub のプログラマが機能の優劣を比較するために、Blub より表現力の低いプログラミング言語を見てみると、目に入るのは、自分が日常的に使っている機能が欠けている言語ばかりです。なぜそのような劣ったツールを選ぶ人がいるのか理解できません。一方、表現力の高いほうを見ると、Blub にない奇妙な機能ばかりが目に入ります。Blub にないそれらの機能がなぜ生産性を高めるために必要なのか、想像もつきません。

コードを扱う際や、他の開発者と協力する際は、こうしたバイアスを念頭に置いておきましょう。同僚が特定のツールや手法に強くこだわる理由がわからない場合は、それが「IKEA 効果」や「Blub パラドックス」の一例ではないか、と考えてみてください。もちろん、自分自身もこうした思い込みに囚われていないか、自身のバイアスを省みることも大切です。

2.3 初めて触れるコードへの取り組み

仕事の時間をすべて新しいコードの作成に費やせたらと願うかもしれません。しかし、実際にはキャリアを通じて既存のコードベースと向き合うことになるでしょう。どうすれば、ストレスを感じることなく新しいプロジェクトにスムーズに取り組めるでしょうか？ まずはチームメンバーに話を聞きましょう。基本的なプロジェクトの概要説明は、どんなオンボーディングでも欠かせない要素です。

ドキュメントを読むことに少し時間を割いてみてください。多くのプロジェクトには、全体像を把握するのに役立つ README ファイルが用意されています。また、概要を簡潔にまとめた Wiki や Web サイトが設けられているプロジェクトもあります[†4]。プロジェクトによっては、アーキテクチャ決定レコード（ADR、Architecture Decision Record）が用意されている場合もあります[†5]。ADR は、貴重な背景情報や、最新の不具合や障害への対応に追われる中で見失われがちな「なぜそうするのか」という極めて重要な理由を提供してくれます。デバッガーを使って何時間も費やすより、ドキュメントを数分間読むだけで、より多くのことを学べるでしょう。プロジェクト

†4 ドキュメントは古くなっている**可能性がある**ため、信頼はしても必ず確認するようにしてください。

†5 ADR について詳しく知りたい場合は、Raju Gandhi らによる『Head First Software Architecture』（O’Reilly、2024 年）の Chapter 3 を参照してください。

のコーディング標準を読んでおけば、コードベースの中で使われているプログラミングパターンに備えられます。

読んでいるコードのドキュメントが古くなっている場合は、学習しながらドキュメントを更新してください。もしドキュメントが存在しない場合は、作業を進めながら自分で作成することを検討してください。「黄金律」（1 章参照）を実践しましょう。ドキュメントを作成することは、プロジェクトを理解する助けとなるだけでなく、後任の開発者にとっても有益なものとなります。

ドキュメント化すべき要素は数多くあります。コードを読み進めながら、次のようなよくある疑問に答えることを目指し、軽量で形式ばらないアプローチを心がけてドキュメントを作成してください。

- このサービスはどのような機能を提供するものですか？
- どのように動作しますか？
- どのような要素に依存していますか？
- アプリケーションはどのように実行しますか？

ちょっと待ってください。今、読者が言いたいことが聞こえてきます。「ドキュメントはコードと同期していないことがある（そして往々にしてそうなっている）」と。しかし、そうでなければならないということは決してないのです。ドキュメントはコードとともに進化できます。それを実現する最善の方法は、テストをドキュメントとして活用することです。振る舞い駆動型のスタイルで書かれたテストは、適切に記述されていれば、実行可能なドキュメントとなります。この点については、5 章でさらに詳しく解説しています。

メトリクスは誤解を招く

コードカバレッジ（テスト実行時にコードベースのどの程度が実行されるか）は、プロジェクトにおいて有用なメトリクスとなり得ます。しかし、ソフトウェア開発に銀の弾丸（万能薬）（https://oreil.ly/QWpz_）など存在せず、コードカバレッジが 100% であっても問題が生じるおそれはあります。ある友人が、リリースごとにリグレッションが発生しているプロジェクトに参加しました。コードに慣れてきた頃、彼はテックリードにテストがあるかどうか尋ねました。テックリードは誇らしげに、「はい、コードカバレッジはおよそ 92 %です」と答えました。感心した友人は、これほどリグレッションが多いことに多少驚きましたが、分析を続けました。

テストコードを確認したところ、びっくりするようなことを発見しました。当初は、これらが単独の事例だと思っていましたが、やがてコードベース全体に蔓延していることに気づきました。彼はテックリードのところへ戻り、「どうしても気になったのですが、テストにアサートが一切含まれていないようですね」と伝えました。テックリードは、コードカバレッジの統計値を繰り返し挙げて返答しました。

ここから得られる教訓は、あらゆるメトリクスは誤解を招くおそれがあるので注意が必要だということです。その価値と目的を見失ってはいけません。もしそれが単なる形式的なものに過ぎないなら、期待したような成果は得られないでしょう。プロジェクトチームは、定期的に自分たちの取り組みやアプローチを見直すべきです。必要であれば、躊躇せずに方向転換してください。

2.4 ソフトウェア考古学

チームと話し合い、既存のドキュメントに目を通したら、次はお気に入りのエディタを開いて、ソフトウェア考古学を実践してみましょう。袖をまくりあげて、コードベースをくまなく探ってみましょう。Issac Newton 卿の言葉を借りれば、より滑らかな小石や、より美しい貝殻を探し求めるのです。コードの構造を見てみましょう。コードはどのように構成されているでしょうか？ 言語によっては、コードをパッケージ化するための構文が用意されているものもあります。一方で、慣習のようなものを使う必要があるものもあります。コードはどのように組み合わされているでしょうか？ これはモノリシックなシステムでしょうか、それとも数十から数百ものサービスからなる分散アーキテクチャでしょうか？ コードにはどのようなドメインの概念が表現されているでしょうか？ テストを読んでみましょう。テストは機能について何を教えてくれるでしょうか？ その情報をもとに、このクラスは**何を意図しているのか**理解できますか？

意図がわからないなら、さらに掘り下げて調べてみてください。モダンエディタを使えば、特定の関数を誰が呼び出しているかを簡単に確認できるため、呼び出し元を次々とたどっていけます。呼び出し元をたどることで、そのクラスが何を行うものか、またどのように使用されているかを把握する手がかりになります。次々とたどっていくと、HTTP 呼び出しのようなサービスエンドポイントまでたどり着くこともありますが、最終的には、特定のユーザー操作とコードとの関連性を見出せるはずです。

全体像が把握できたら、アプリケーションを実行してみてください。どのような動作をするでしょうか？ ユーザーインターフェース上の要素であれ、サービス呼び出しのパラメータであれ、特定の要素を見つけ、それをコードにマッピングしてみてください。課題リストを確認し、特定の機能やバグに焦点を当てて、コードの流れを追跡できるかどうかを確認してください。目印を探しましょう。ある操作が、データストアの更新につながることを知っていれば、コード内でその箇所を見つけ出してください。デバッガーを使ってコードを順に追って確認してください。予想通りに動作しましたか？ それとも、まったく異なるコードにたどり着いてしまいましたか？ 最終的には、コードのメンタルモデルを構築することになります。つまり、アプリケーションを脳内に読み込むのです。

実際にはどのようになるのでしょうか？ 例として、Spring PetClinic アプリケーション（https://oreil.ly/1q4UJ）を使ってみましょう。Java や Spring の専門家でなくとも、このアプリケーションを理解するのは比較的簡単です。しかも優れたドキュメントも用意されてい

ます。アプリケーションをクローンして実行すると、飼い主を検索する機能があることがわかります（**図2-1** 参照）。飼い主のテンプレートを調べてみると「findOwners.html」というわかりやすい名前がついたファイルが見つかります[†6]。このファイルは、`/owners` エンドポイントを参照しています。プロジェクト内で`/owners` を検索すると多くの結果が返されますが、少し勘を働かせれば、`OwnerController` ファイル内の `processFindForm` メソッドにある`@GetMapping("/owners")` アノテーションにたどり着くことでしょう。

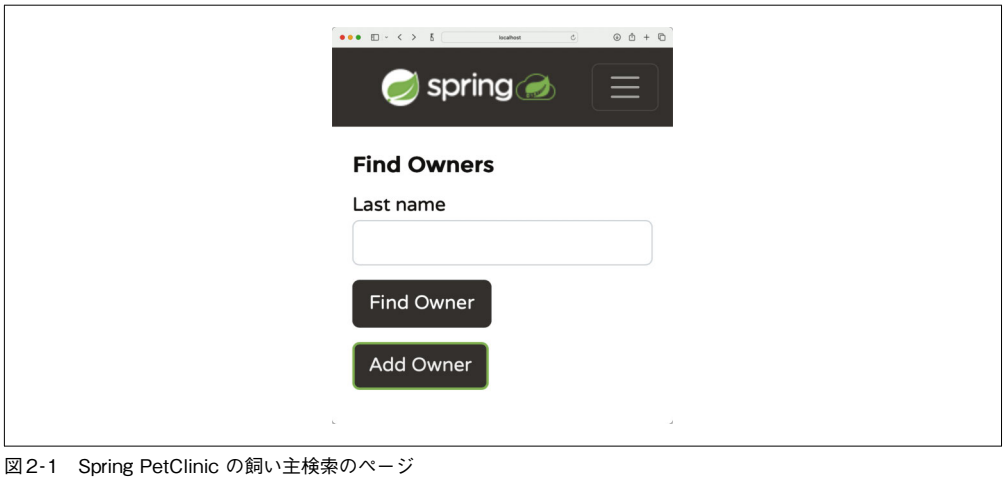


図2-1　Spring PetClinic の飼い主検索のページ

ブレークポイントを設定し、ブラウザから飼い主検索を実行して、何が起こるか見てみましょう。予想通り、デバッガーの画面は**図2-2** のような状態になっているはずです。もし直感が外れていた場合は？ 前の手順を繰り返してください。そうすれば、やがてその仕組みを把握できるようになり、コードをたどりながら、理解を深めていくことができるでしょう。

†6　もちろん、すべてのアプリケーションでこれほどわかりやすくファイルに名前がつけられているわけではありません。お気に入りの検索ツールを駆使する必要があるかもしれません。

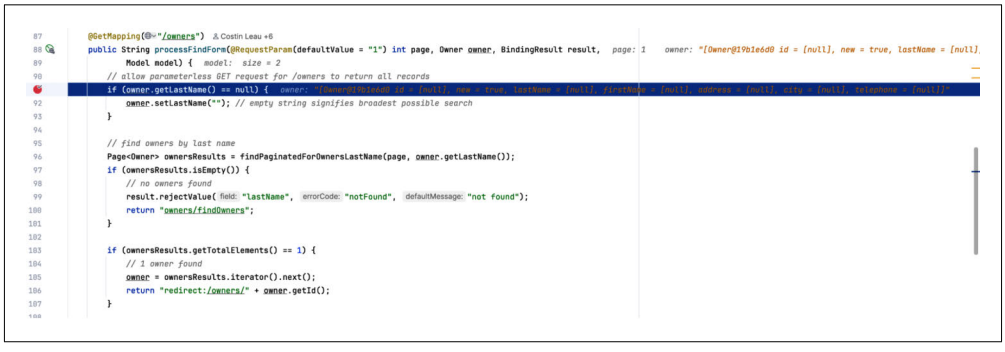


図2-2 OwnerController のブレークポイントで、アプリケーションの現在の状態を確認する

統合開発環境（IDE、Integrated Development Environment）を使用して、コード内を移動しましょう。多くの IDE では、コードを読み進める際に、他のクラスにあるメソッドへ簡単にジャンプできます（**図2-3** 参照）。メソッド本体をすべて折りたたんで、表示範囲を狭くするのもよいでしょう（**図2-4** 参照）。また、多くの IDE では特定のクラスのアウトラインを表示でき、コードをより高レベルな視点から把握できます。メソッド名を読んでみてください。そのモジュールの目的についてどのようなことがわかりますか？

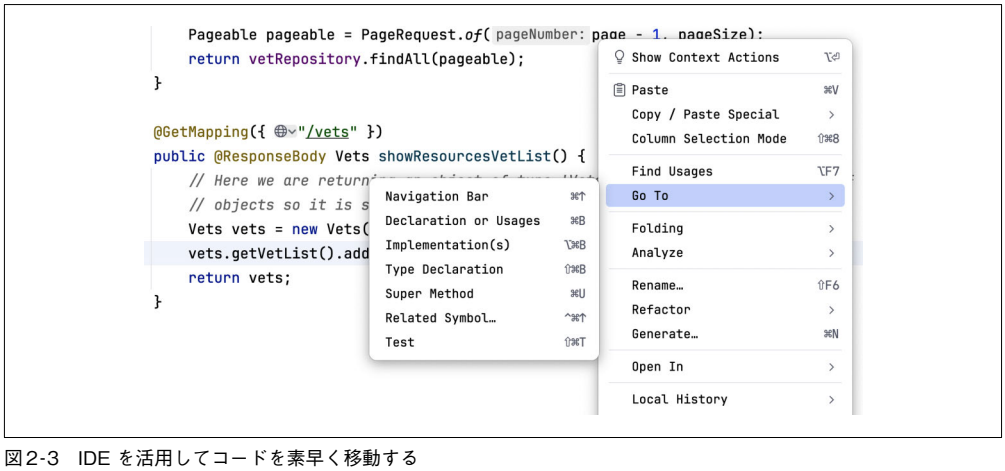


図2-3 IDE を活用してコードを素早く移動する

コードが名前通りの動作をすると決めつけないでください。コードが進化するにつれて、変数名やメソッド名が実態を反映しなくなることがあります。焦らずに、自分の直感が正しいかどうか確かめてください。誘惑に負けて手抜きをしてしまい、後で余計な手間を増やすようなことにならないようにしましょう。つまり、時間をかけて丁寧に作業することで、結果として時間を節約できます。

```
@GetMapping("/owners/new")  Mic +1
public String initCreationForm(Map<String, Object> model) {...}

@PostMapping("/owners/new")  Mic +4
public String processCreationForm(@Valid Owner owner, BindingResult result) {...}

@GetMapping("/owners/find")  Dave Syer +1
public String initFindForm() { return "owners/findOwners"; }

@GetMapping("/owners")  Costin Leau +6
public String processFindForm(@RequestParam(defaultValue = "1") int page, Owner owner, BindingResult result,
        Model model) {...}

private String addPaginationModel(int page, Model model, Page<Owner> paginated) {...}

private Page<Owner> findPaginatedForOwnersLastName(int page, String lastname) {...}

@GetMapping("/owners/{ownerId}/edit")  Mic +2
public String initUpdateOwnerForm(@PathVariable("ownerId") int ownerId, Model model) {...}

@PostMapping("/owners/{ownerId}/edit")  Dave Syer +5
public String processUpdateOwnerForm(@Valid Owner owner, BindingResult result,
        @PathVariable("ownerId") int ownerId) {...}

/** Custom handler for displaying an owner. ...*/
@GetMapping("/owners/{ownerId}")  Mic +1
public ModelAndView showOwner(@PathVariable("ownerId") int ownerId) {...}
```

図2-4　メソッドを折りたたんでコードベースを把握する

例外（Exception）は誤解を生むことがあります。これまでにも、起こり得るエラー状況について、例外が誤った想定をしているのを何度も見てきたことがあります。非常に大まかなレベルの例外をキャッチしている場合は、特に注意を払ってください。コードを書いた人にとっては手っ取り早い方法かもしれませんが、そうした例外は、起こり得る問題を曖昧にしてしまうおそれがあります。

IDE には、コード分析に役立つツールやプラグインが搭載されている場合もあります。例えば、IntelliJ IDEA では依存関係を素早く表示でき、コードがどのように連携して動作しているかを把握するのに役立ちます（**図2-5** 参照）。モダンな開発ツールは非常に強力です。コードを理解するために、ぜひそれらを活用してください。

ソースコード管理ツールも活用してください（**図2-6** 参照）。モダンなツールの多くは、プロジェクト内を素早く移動できるように設計されています。ファイルの変更履歴を確認してみましょう。どのような変更が頻繁に行われていますか？ コミットログからは、更新内容についてどのようなことがわかりますか？ 最も頻繁に更新されているクラスから確認してみましょう。Git では、次のようなコマンドでそれらを表示できます。

```
git log --pretty=format: --since="1 year ago" --name-only - "*.java"
| sort | uniq -c | sort -rg | head -10
```

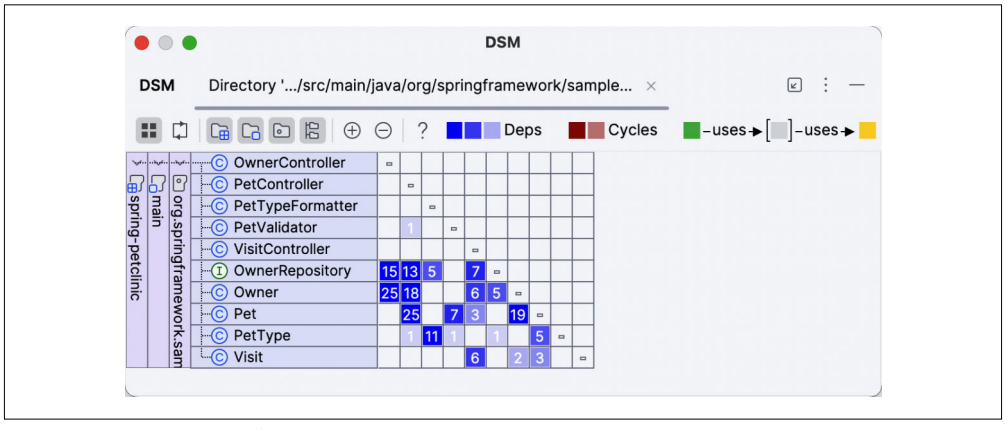


図2-5　Spring PetClinic アプリケーションの owner パッケージにおける依存関係

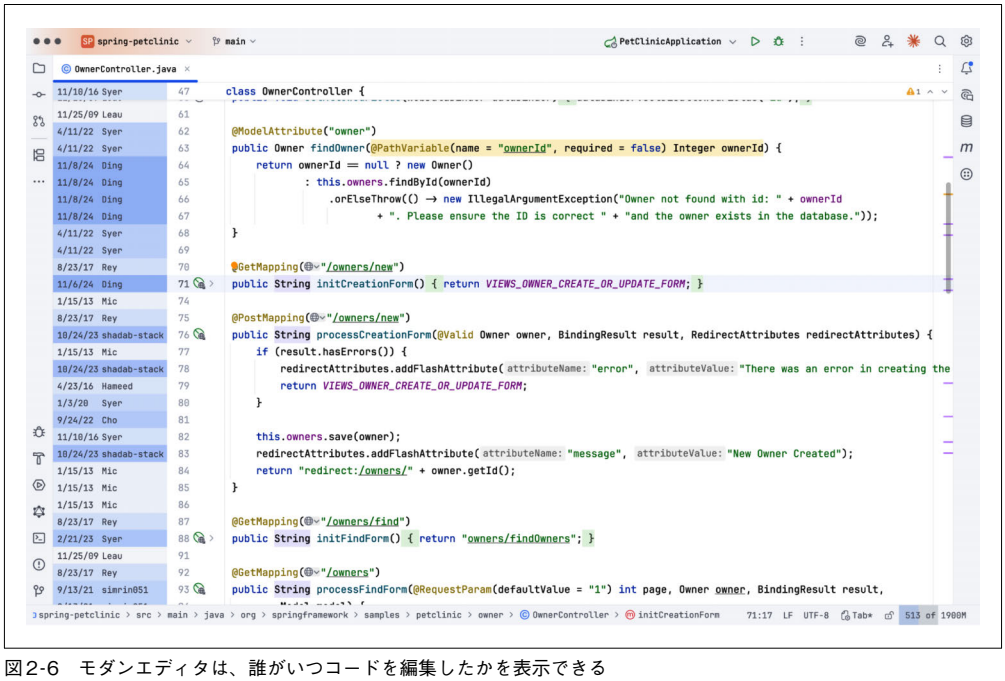


図2-6　モダンエディタは、誰がいつコードを編集したかを表示できる

また、`git blame` などのツールを使って、コードの変更箇所を可視化することもできます。新しいプロジェクトに参加したばかりの場合、このようなツールを使うことで、チーム内で誰が直近の変更を行ったのか、あるいは、誰が最も頻繁に変更を加えているのかを確認できます。コマンドラインを使うのが面倒な場合は、IDE でも変更履歴を確認できます。どのような方法でコードを調査する場合でも、恐れずにチームメンバーに質問するようにしてください。

アプリケーションの進化に伴い、そのプロジェクトのモデルはコードと整合性が失われること

がよくありますが、コードベースからいつでも図を**抽出**できます。一部の IDE では、簡単なキー操作でこれを行えます（例えば**図 2-7** 参照）[†7]。また、Umbrello、Doxygen、Structurizr などのツールを使用して、コードを視覚的に表現することも可能です。ビルドパイプラインに手順を追加して、コードがコミットされるたびに最新の図を自動的に生成することも検討してみてください。

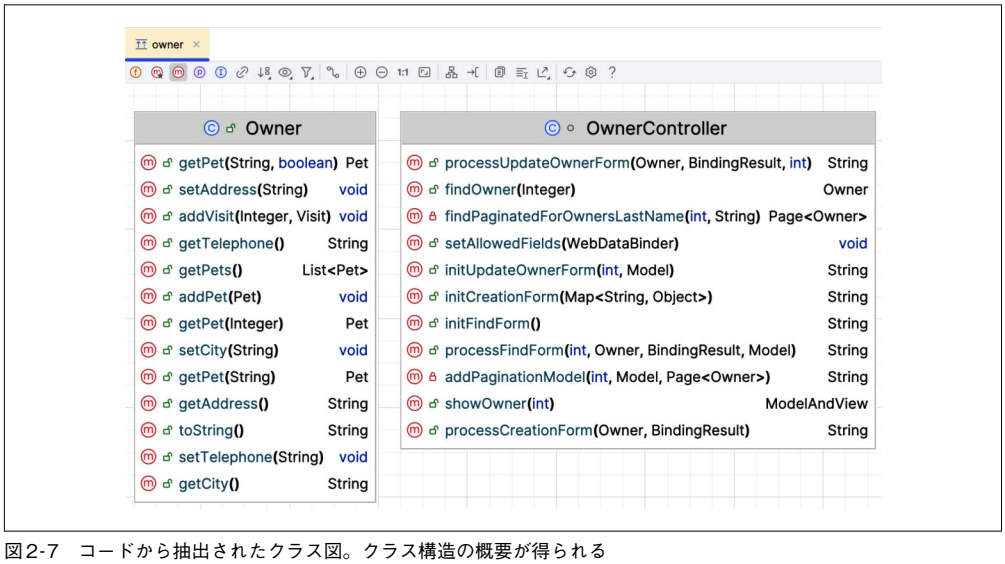


図 2-7　コードから抽出されたクラス図。クラス構造の概要が得られる

2.5　効果的にコードを読む方法

プロジェクトの目的と全体像を把握できたところで、次はコードを理解していく段階です。本節では、IDE の機能を活用したり、テストを分析して情報を得たりするなど、効果的にコードを読むための方法を学びます。

2.5.1　IDE の機能を活用する

IDE は、単にコードを書くためのテキストエディタというだけではありません。IDE には、コードの記述、デバッグ、コードベースの探索を容易にする数多くの機能が備わっています。既存のコードを扱う際には、IDE はコードを調査し、理解するためのツールとして、さらに便利に活用できます。

2.5.1.1　継続的な学習方法

最近の IDE は非常に高性能なため、そのすべての機能を把握するのはほぼ不可能です。その代

†7　例えば、IntelliJ IDEA では ⌃⇧⌘U（macOS）/ Ctrl-Alt-Shift-U（Windows/Linux）で UML 図を生成できます。

わり、毎週何か新しいことを学ぶようにしてください。例えば、JetBrains の IntelliJ IDEA は Java 開発者の間で人気があり、「Tip of the Day（今日のヒント）」という非常に便利な機能を備えています。IDE を起動すると、機能に慣れるためのヒントが表示されます。

これらのヒントは、ショートカット、ツール、リファクタリング、デバッグ、プラグインなど、あらゆる内容を網羅しています。お使いの IDE のショートカットを習得すれば、開発効率が大幅に向上します。まずはここから始めてみるとよいでしょう。現在使われている多くの IDE には同様の機能が備わっていますので、ドキュメントを確認して学習を始めてみてください。

もし IntelliJ の「Tip of the Day」を見逃してしまった場合でも、[Help] → [Tip of the Day] を選択すれば、いつでも確認できます。**図2-8** に示されているヒントでは、IntelliJ の「ライブテンプレート」という機能について解説しています。この機能は、頻繁に使用するコード構造を挿入するのに活用できます。

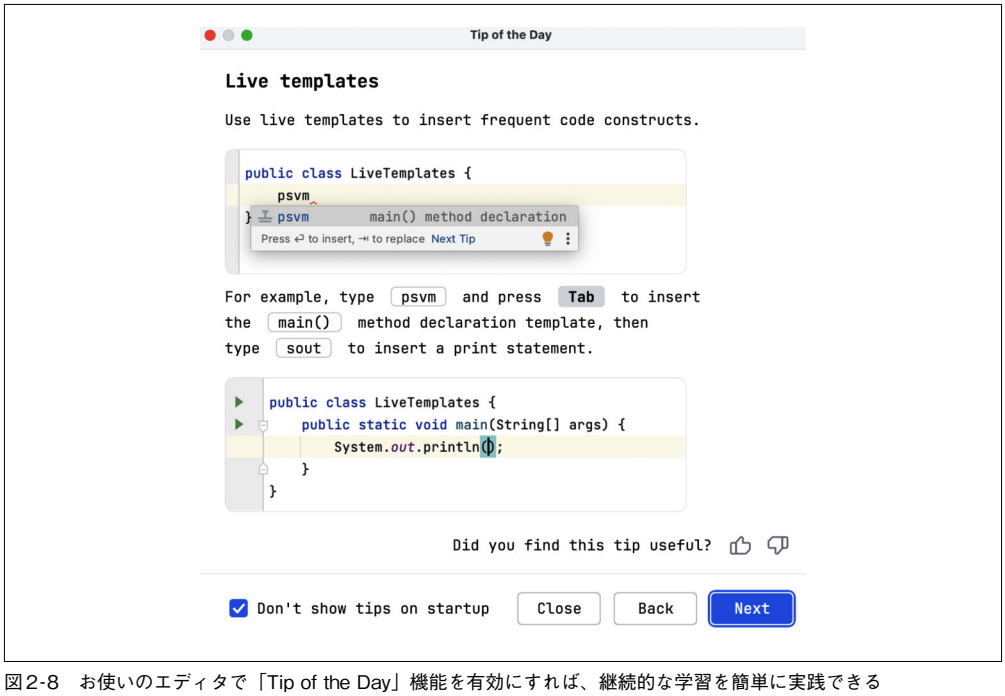


図2-8　お使いのエディタで「Tip of the Day」機能を有効にすれば、継続的な学習を簡単に実践できる



自分専用の「IDE 活用術」ドキュメントを作成し、発見した新しいショートカットや機能を記録しましょう。定期的に見直して、学んだことを定着させましょう。このドキュメントを参照し、記録したヒントを活用して、操作を無意識に行えるようになるよう練習しましょう。

2.5.1.2　コードナビゲーションツール

初めて触れるコードベースを調査する際に非常に役立つスキルは、コード内を効率的に移動する

能力です。多くの IDE では、ナビゲーションバーから手動でファイルを開くことなく、他のファイルへジャンプしたり、そこでコードを作成したりできます。以下の節では、コードの調査を劇的にスピードアップさせる実用的なナビゲーション機能について紹介します。

使用箇所や参照先を探す

クラスやコンポーネントを理解しようとする際は、コードベースのどこで使われているかを確認する必要があります。モダン IDE であれば、このような作業を簡単に行えるようになっているので、ぜひ使いこなせるようにしておくべきです。

例えば、IntelliJ で**図 2-9** のような Java インターフェースを開いた場合、ガターアイコンをクリックすると、そのインターフェースのすべての実装を検索できます[†8]。これは抽象化が実際にどう使われているかを理解しようとする際に、非常に便利です。同様に、メソッド名を右クリックして「Find Usages（使用箇所を検索）」を選択すると、そのメソッドが呼び出されているすべての場所を確認できます。

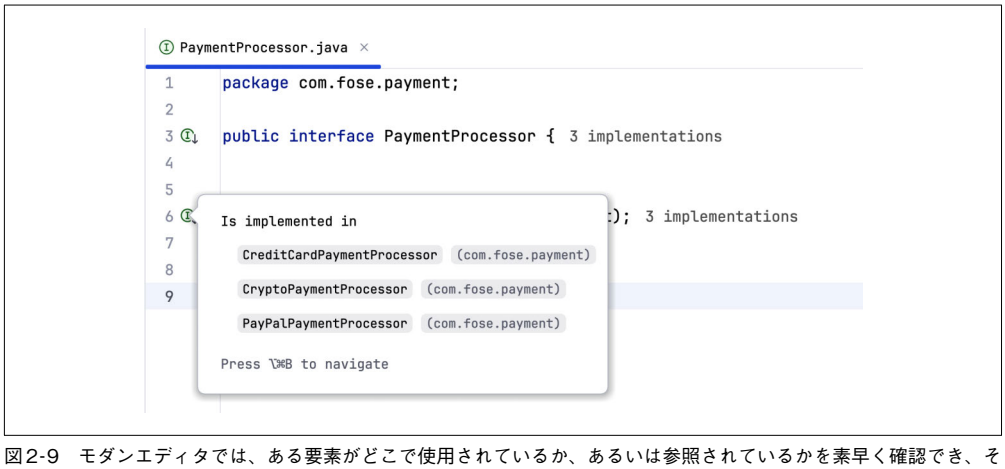


図 2-9　モダンエディタでは、ある要素がどこで使用されているか、あるいは参照されているかを素早く確認でき、そこへすぐ移動する機能も備わっている

定義へ移動する

IDE の便利なナビゲーション機能として、クラス、メソッド、変数の定義へジャンプする機能もあります。コードの論理的な流れに沿って、使用箇所から実装へと素早く移動できます。例えば、Java アプリケーションを作成しているときに `ArrayList` を使用しており、その動作を知りたい場合、`ArrayList` 型を Command キーを押しながらクリック（Windows ／ Linux では Ctrl キーを押しながらクリック）すると、ソースコードに移動し、その型のコードやドキュメントを確認で

†8　Java のインターフェースとは、実装クラスが提供しなければならないメソッドの仕様を定義する参照型です。Java プログラムにおいて抽象化とポリモーフィズムを実現するものです。

きます（**図2-10** 参照）。

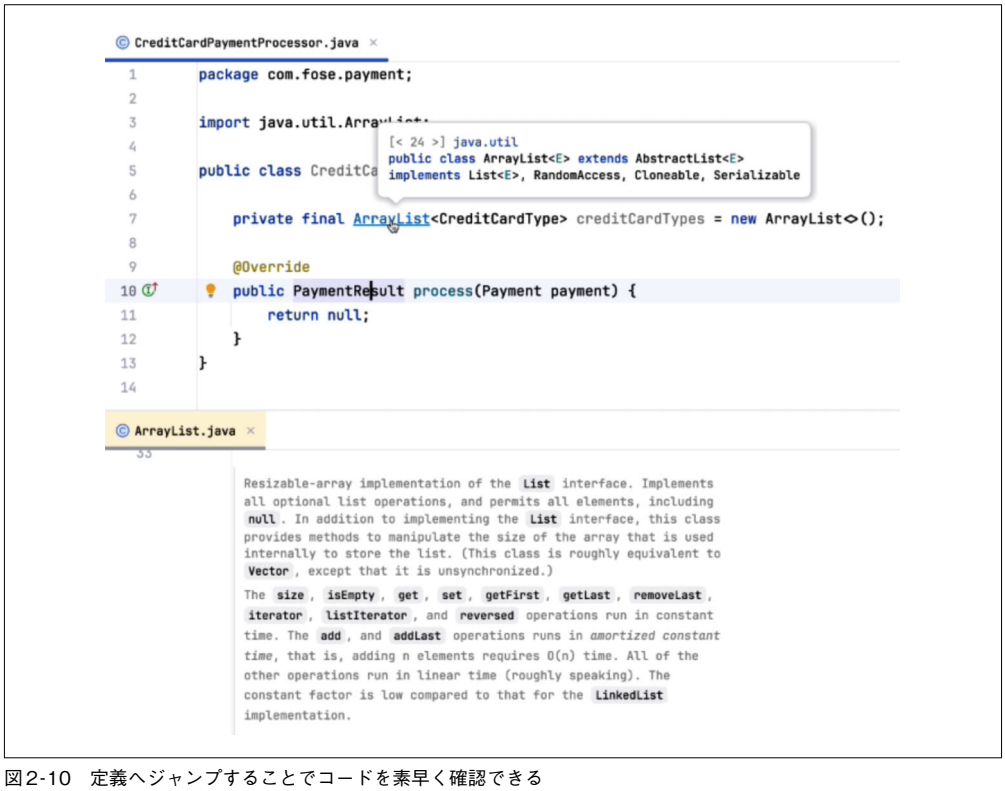


図2-10　定義へジャンプすることでコードを素早く確認できる

以下のようなものを扱う際、定義へジャンプする機能は大変役立ちます。

- サードパーティ製ライブラリ
- フレームワークのコード
- 基底クラスやインターフェース
- コードベース全体で使用されるユーティリティメソッド

呼び出し階層と依存関係ビュー

コンポーネント間のやり取りを理解することは、初めて触れるコードベースを把握するうえで極めて重要です。IDE には、こうした関係を可視化するための専用のビューが用意されています。

呼び出し階層

そのメソッドが何から呼び出され、どのメソッドを呼び出しているかを示す

型階層

継承関係を表示する

依存関係

モジュールやパッケージが互いにどのように依存しているかを可視化する

例えば、IntelliJ では、メソッドを右クリックして「Show Call Hierarchy（呼び出し階層を表示)」を選択すると、インバウンド呼び出し（このメソッドがどこから呼び出されているか）とアウトバウンド呼び出し（このメソッドが呼び出しているメソッド）の両方を確認できます。これにより、システム全体におけるそのメソッドの役割を素早く把握できます。

2.5.1.3 コード分析機能

モダン IDE には、コードの探索を支援する強力なツールが備わっていますが、IDE の能力はそれだけにとどまりません。IDE は、読者が記述しているコードを積極的に分析し、コードの品質向上だけでなく、これまで意識していなかった新機能の習得にも役立つ情報を提供してくれます。

自動検査と提案

コードを記述したり修正したりする際、IDE はコードのリファクタリング方法について提案してくれます。これらはあくまで提案に過ぎないということを覚えておいてください。必ずしもその提案を受け入れる必要はありませんが、確認しておくことは有益です。こうした提案は、コードの品質を向上させるだけでなく、その過程で新しいことを学ぶきっかけにもなり、よく使われるパターンやベストプラクティスを知るよい機会になることも多いのです。

以下は、Java で `Books` コレクションを反復処理する例です。コレクション内の各 `Book` に対して、単に書籍名を出力するだけのコードです。

```
List<Book> books = library.getBooks();
for (Book book : books) {
        System.out.println(book.getTitle());
}
```

このコードは動作します。また、コードにも問題はありません。ただし、IntelliJ は、次のようなより簡潔な方法を提案してくるでしょう。

```
library.getBooks().forEach(book -> System.out.println(book.getTitle()));
```

こうした提案は、より良いコードを書くのに役立つだけでなく、使用している言語やフレームワークにおける慣用的なパターンも学べます。

コード構造の可視化

コードを効果的に読むには、プロジェクト全体、あるいは単一のクラス内におけるコードの構造を理解することが重要です。一般的に、IDE には、コードの構造を可視化する機能が備わっています。例えば、**図2-11** のコードがどのように構成されているかを理解したい場合は、プロジェクト構造を確認することで、それがモノリシックなアプリケーションであり、コードが機能ごとにパッケージ化されていることがすぐにわかります。

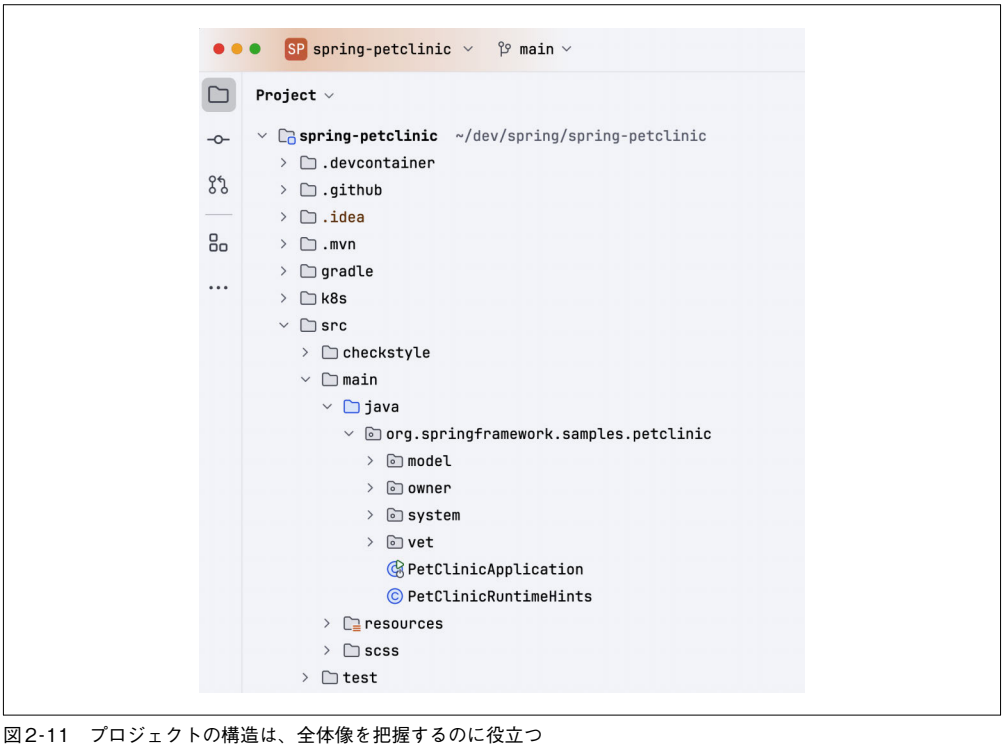


図2-11　プロジェクトの構造は、全体像を把握するのに役立つ

あるクラスでどのようなメソッドが利用可能かを理解しようとしているとしましょう。もちろん、API ドキュメントを確認することもできますが、それが最新バージョンのドキュメントかどうかすらわからないかもしれません。手作業でクラスファイルを開いてソースコードをスクロールして確認することもできますが、これには時間がかかることでしょう。ほとんどの IDE には、クラスやコンポーネントの構造を可視化する機能が備わっています。**図2-12** に示した例では、PetClinic というプロジェクト内の `OwnerController` の構造を確認しています。このクラスで利用可能なメソッドが一目でわかるため、このクラスの目的や機能について素早く把握できます。

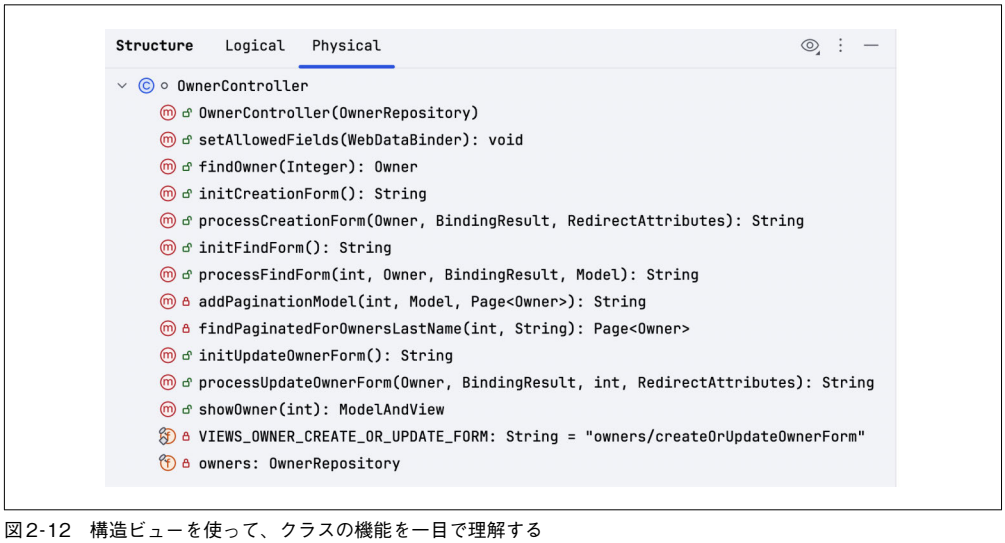


図2-12　構造ビューを使って、クラスの機能を一目で理解する

2.5.2　テストを読むことで知見を得る

新しいコードベースを調査する際、テストは開発者にとってドキュメントとしての役割を果たします。適切に書かれたテストは、コードが何を実行するかだけでなく、なぜそのように実装されているかについても明らかにしてくれます。コードは「実際にどう動作するか」を教えてくれますが、テストは「本来どう動作すべきか」を教えてくれるのです。

2.5.2.1　テストは生きているドキュメント

従来のドキュメントはすぐに古くなってしまうことがあります。しかし、テストの場合は、継続的インテグレーションのチェックに合格するために、常に機能しなければなりません。そのため、テストは、システムの挙動に関して常に最新の「信頼できる情報源」となります。

テストは、メソッドの期待される入力と出力、コンポーネント間のやり取り、システム全体の挙動をドキュメント化します。コンポーネントが実際にどのように動作するかを探る前に、まずテストを読むことで、そのコンポーネントが何を行うべきかを理解できます。このような重要な情報により、実装内容が理解しやすくなります。

次のテストでは、常連客が一定額以上の大量注文を行った際に 10% の割引が適用されることを検証し、価格設定サービスが顧客の種類ではなく注文規模に基づいて割引を正しく計算していることを検証しています。

```
@Test
public void shouldApplyBulkDiscountForLargeOrders() {
        // 前提
        Customer regularCustomer = new Customer(CustomerType.REGULAR);
```

```
        Order bulkOrder = new Order(regularCustomer, 500.00);

        // 操作
        double finalPrice = pricingService.calculateFinalPrice(bulkOrder);

        // 結果
        assertEquals(450.00, finalPrice, 0.01);
}
```

こうしたコンテキストを踏まえて、価格設定サービスクラスを詳しく調べ、calculateFinalPrice メソッドを確認することで、その計算結果を生成するコードを理解できます。

2.5.2.2 ワークフローとプロセスを理解する

テストは多くの場合、ユーザーストーリーと対応付けられるため、ビジネスプロセスやワークフローを理解するうえで非常に有効です。多くの場合、統合テストを見れば、コンポーネントがどのように連携してビジネス要件を満たしているかという全体像を把握できます。

プロセス全体を最初から最後まで検証するテストを探してください。こうしたテストには、実際の運用環境を想定したセットアップコードが含まれていることが多く、本番環境でのシステムの挙動を把握する手がかりとなります。

以下のテストでは、ユーザーが商品レビューを投稿した際に、レビューサービスが商品の平均評価を正しく更新し、管理者に通知を送信することを検証します。このテストにより、レビュープロセスのデータ永続化と通知の両方が期待通りに動作することを確認できます。

```
@Test
public void submitProductReviewShouldUpdateRatingAndNotifyAdmin() {
        // 前提
        Product product = productService.findProductById("XYZ789");
        User user = userService.findByUsername("janedoe");
        Review review = new Review(user, product, 4, "Great quality product!");

        // 操作
        ReviewResult result = reviewService.submitReview(review);

        // 結果
        assertTrue(result.isSuccessful());
        assertEquals(4.2, productService.findProductById("XYZ789")
        .getAverageRating(), 0.01);
        verify(notificationService).sendReviewNotification(anyObject(), eq("XYZ789"));
}
```

このテストのコンテキストを踏まえれば、ユーザーが新しいレビューを投稿した際の全体の流れが理解できるでしょう。

2.5.2.3 エッジケースや境界条件を発見する

ソフトウェア開発において、**エッジケース**とは、プログラムの想定される入力、動作条件、利用パターンの極端な境界で発生する状況を指します。多くの場合、こうした状況ではこれまでの前提が成り立たなくなります。これらの入力や条件は有効ではあるものの、通常とは異なるものであり、「典型的な」シナリオや「ハッピーパス」のシナリオでは発生しないようなバグが露見します。既存のコードベースでは、こうしたケースに関するテストがより多く見られることがあります。このような問題は、時間の経過とともに発見され、再発を防ぐためにテストが作成されてきたためです。

特に注意すべきなのは、`shouldHandleEmptyList` や `shouldRejectInvalidInput`、`shouldTimeoutAfterTenSeconds` のような名前がついたテストです。これらのテストは、システムの機能の限界を示し、発生しうる障害を明確にしてくれます。

以下のテストにより、システムが有効期限切れのクレジットカードを適切に処理し、正常に処理を中断することがわかります。実装コードを単に読み通すだけでは、この点はわからないかもしれません。

```
@Test
public void shouldRejectPaymentWhenCreditCardIsExpired() {
        // 前提
        CreditCard expiredCard = new CreditCard("4111111111111111", "05/20");
        Order order = new Order(new Customer(), 50.00);

        // 操作
        PaymentResult result = paymentService.processPayment(order, expiredCard);

        // 結果
        assertFalse(result.isSuccessful());
        assertEquals(PaymentFailureReason.EXPIRED_CARD, result.getFailureReason());
}
```

このコンテキストを踏まえて、支払い処理の実装内容を確認し、このエッジケースを捕捉する機能を探すとよいでしょう。

プロジェクトの内容を理解し、コードを効果的に読み解けるようになったら、次はコードを修正する段階です。しかし、どうすれば安全に修正できるのでしょうか。その点については、次の章で説明します。

2.6 練習が上達へ導く

いずれにせよ、自分自身に少し寛容になってください。モダンなコードベースは、非常に複雑で大規模なことがよくあります。一人の人間がすべてを理解することは不可能です。そして、そもそもそれが目的ではありません。知識は時間とともに増えていくものです。プロジェクトにおいて初

めて触れる部分に直面するたびに、同じプロセスを繰り返してください。最初は気が遠くなるかもしれませんが、すべての開発者が同じ道を歩んできたのです。きっと大丈夫です！

コードを読む力をどう伸ばせばよいのでしょうか。気が進まないかもしれませんが、コードを読む（https://oreil.ly/dFBe2）練習をしましょう。さまざまな言語で書かれた、良質なオープンソースのコードが数多く公開されており自由に利用できます。近道はありません。練習なくして上達することはできません。時間が経つにつれて徐々に楽になり、読むスピードも上がってくるでしょう。

AIはどうでしょう？

他の分野と同様、ソフトウェア開発ツールも進化しつづけています。シェルベースのテキストエディタから、統合リファクタリングツールやIntelliSenseによるコード補完機能を統合したIDEに至るまで、コードを書くことは以前より容易になりました。もちろん、開発対象となるアプリケーションの規模や複雑さも増しているため、結局は相殺されてしまうのかもしれません！ GitHub CopilotやChatGPTのような生成AIの登場により、開発者は間もなく置き換えられてしまうだろうとさえ指摘する声もあります。開発者の終焉を予言する噂は、多くの場合、大げさに誇張されたものです。COBOLから第4世代プログラミング言語、さらにはさまざまなドラッグ&ドロップ式の「プログラミング」ソリューションに至るまで、これらがソフトウェアエンジニアリングへの需要を大幅に減少させることはありませんでした。

これらのツールは、読者のツールボックスに欠かせない存在です。例えば、あるコードのまとまりが何をするものかChatGPTに尋ねることはできます[†9]。それでも読者自身がコンテキストを理解する必要があります。AIは、なぜ特定のクラスが存在するのかを知りません。やはり読者がコードを読み、ドキュメントを精査し、仲間の開発者と話し合う必要があります。AIは（現時点では）企業全体のコードベースを完全に理解しておらず、ハルシネーションのリスクは現実のものとして存在します。AIは助けにはなります。しかし、最終的には、複数のリポジトリにまたがるコードの細かな点や相互関係を理解するのは、**読者**の仕事です。

Danがよく言うように、AIを利用する際、読者は「パイロット」であって「乗客」ではありません。つまり、信頼はしても、検証も行ってください。AIは、個々のコード片であれば理解できることもありますが、ハルシネーションのリスクもあるため、企業レベルのコードベースを理解できるとは限りません。モダンアプリケーションは、多くのリポジトリにまたがり、複数のライブラリを活用していることがよくあります。開発者である読者が、その関係性や複雑さを理解してこそ、AIツールが開発を加速できるようになるのです。

†9 読者の所属する組織の法務部門は、こうしたツールの利用についてポリシーを設定しているはずです。AIモデルは貼り付けたコードを訓練データとして使用するおそれがありますので、価格設定など自社独自のアルゴリズムをAIに対して貼り付ける際には、必ず社内規定を確認してください！

2.7 まとめ

プログラミングの教育は、ある意味逆転していると言えるでしょう。読むことを学ぶ前に書くことを学ぶのに、実際のキャリアでは、他人が書いたコードを読むことにかなりの時間を費やすことになるからです。既存のコードを扱うことは、ゼロから開発するほど楽しいものではないかもしれませんが、それが報酬を得るための仕事なのです。その状況から逃げ出すのではなく、それを受け入れることを学びましょう。そうすることで、プロフェッショナルとして得られるものは多いからです。また、認知バイアスにも注意を払うようにしましょう。臆することなく、袖をまくりあげて、初めて触れるコードベースを掘り下げてみてください。必ず何かを学べるはずです。理解が深まったら、コードを元の状態よりも良い状態にしてください。そうすることで、次にそのコードを扱う開発者の負担を軽減することになります。その次の開発者は、読者自身かもしれないのですから！

2.8 実践に移す

コードを読む力を高めたいのであれば、近道はありません。とにかく多くのコードを読む必要があります。幸いなことに、豊富なオープンソースプロジェクトを利用できます。時間を確保して、仕事で使っている（あるいは使いたいと思っている）フレームワークや、興味のある他のプロジェクトのコードを読むようにしましょう。どこから手をつければよいか迷う場合は、GitHub のトレンドリポジトリ（https://oreil.ly/q27Ze）をチェックしてみてください。この章で学んだテクニックを活用してみましょう。数か月後、最初に調べたプロジェクトの別の部分をさらに探るか、あるいはまったく別のプロジェクトに挑戦してみてください。前回より読みやすかったでしょうか？ 継続することが大切です。時間をかけて取り組めば、コードを読むスキルは確実に向上していくでしょう。

既存のコードを扱うことも、磨くべきスキルの一つです。ここでもまた、オープンソースソフトウェアはたくさんのソースコードが公開されており探求しがいがあります。オープンソースへの貢献は、絶好の学習の場（https://oreil.ly/aDGcn）です。貢献を始めるのは想像しているほど難しくありません（https://oreil.ly/35Aqv）[†10]。プロジェクトを選んで取り組んでみてください。

2.9 関連リソース

- "Code as Design: Three Essays"、Jack W. Reeves 著（https://oreil.ly/KVTgx）
- "Reading Code Is Harder Than Writing It"、Trisha Gee 著（https://oreil.ly/mr3mU）
- "Reading Other People's Code"、Patricia Aas 著（https://oreil.ly/h_b1O）
- "How to Quickly and Effectively Read Other People's Code"、Alex Coleman 著（https://oreil.ly/2QpkE）

†10 多くのプロジェクトには「初めての貢献者向け」とマークされたバグリストがありますが、現在のプロジェクト貢献者に気軽に連絡を取るようにしてください。ほとんどの人が、喜んで手助けをしてくれるはずです。

- "How to Read Code Without Ripping Your Hair Out"、Sunny Beatteay 著（https://oreil.ly/8JKqG）

3章
コードを書く

どんな愚か者でも、コンピュータが理解できるコードなら書けます。
優れたプログラマは、人間が理解できるコードを書きます。
— Martin Fowler、英国のソフトウェア開発者、作家、
ソフトウェア開発に関する国際的な講演者

コードを書くことは、ソフトウェアエンジニアリングにおいて間違いなく非常に重要なことです。そして、コーディングそのものは広く教えられていますが、**良い**コードを書くための勘どころまでは、それほど広く共有されているわけではありません。問題を解決するためのコードを書くことが**できる**からといって、その問題を解決するためにコードを書く**べき**とはならないのです[†1]。人工知能やエージェント型コーディングツールの登場により、自らコードを書く作業が減っています。その代わりに、ソフトウェアエンジニアの仕事は、生成 AI ツールが作成したコードのバグ修正や、調整、レビューを行う業務へと変化しつつあります。しかし、コードレビューをうまく行うためには、コードを書く力がなければなりません。直感に反するように思えるかもしれませんが、AI の世界において、良質でクリーンなコードを書くことは、かけがえのないスキルなのです。

良くも悪くも、開発者は「良いコード」や「悪いコード」とは何かについてこだわりを持っていることがよくあります。メトリクスやツールを活用することで、重要な視点や指針が得られます。テストは、お金で買える最高のドキュメントの一つです。コードレビューを適切に行うことで、チームがミスしやすい形式や、複雑すぎるコードに頼らないようにできます。結局のところ、コードは人間が読むことを前提に書くべきものです。

よく教えられていることとは異なり、プログラミングとはそもそもコミュニケーション活動であり、それは単にコーダーとコンパイラの間だけのものではありません。忘れてはならないのは、コンピュータは**どのような**コードでも（少なくとも構文的に正しい限りは）理解します。ただ、コンピュータが理解するからといって、コードが何を意図しているのかを人間が理解できるとは限らな

†1　AI は現在のところ、問題を解決するためのコードを書くことができるかどうかと、書くべきかどうかを区別することができません。おそらく今後も決して区別できるようにはならないでしょう。

いということです。優れたソフトウェアエンジニアは、人が読んで理解できるようなコードを書くことに注力しています。つまり、コードは簡潔で、整理されている必要があります。コードはその意図を明確に伝えるべきであり、そうすることで人間がコードの動作を把握できるようになります。読みやすいコードであれば、保守も容易になります。これは、ソフトウェアプロジェクトが長期的に成功するために極めて重要なことです。

ソフトウェア開発の道を歩み始めたとき、最初に学ぶのはコードの書き方です。おそらく「Hello World!」からでしょう。しかし、大学の講義であれ、ブートキャンプであれ、独学のチュートリアルであれ、必ずしも教わらないことがあります。それは、良いコードとは何か、どうすれば良いコードを書けるのか、そして問題を解決するためにいつコードを書くべきか、ということです。本章で、この点を補っていくことにします。

3.1 車輪の再発明は避ける

（間違いなく素晴らしい）コードを次から次へと書き始める前に、少し時間を取って、その問題がすでに解決されていないか確認するようにしてください。開発者は、必要以上にコードを書いて、他者がすでに解決した問題を解決することがよくあります。新しいコードを書き始める前に、周囲を見渡してみてください。活用できるライブラリはありませんか？ ただし、活用できるライブラリがあるからといって、適当に機能を追加してよいという理由にはなりません。それらのライブラリの依存関係を分析することを忘れないでください。サードパーティ製ライブラリに関する組織の方針や手順を必ず遵守してください。

普段使っているプログラミング言語をもう一度確認してみてください。活用できる言語機能はありませんか？ 言語は進化しつづけています（1章のコラム「大文字化という作業」参照）。5分間検索するだけで、何時間もの労力を節約できるかもしれません。アプリケーションフレームワークについてはどうでしょうか？ もっと良い方法があるはずだと思うのであれば、実際に存在するかもしれません。自問してみてください。このようなコードを書いたのは、宇宙で読者が初めてなのでしょうか？ そうではない可能性が高いでしょう。（「1.5 怠惰なプログラマの精神を身につける」参照）。

新しいコードを書くのに時間を割く前に、自分自身にいくつかの質問を投げかけてみるとよいでしょう。 Christopher M. Judd（https://oreil.ly/euyYh）は、自身のブートキャンプで以下に挙げるような意思決定ツリーを教えています（自分の使用している言語やフレームワークに合わせて調整してください）。

- 現在のコードベースのどこかで、この処理を行っている箇所はありますか？
- JDK ではすでに実装されていますか？
- この問題を解決している Spring Framework や Spring Boot のプロジェクトはありますか？
- Google Guava には解決策がありますか？（https://oreil.ly/getTn）

- Apache Commons には解決策がありますか？（https://oreil.ly/QMWgj）
- プロジェクト内に、この問題を解決している他のライブラリがありますか？
- この問題を解決しているオープンソースライブラリはありますか？

すべての質問に「いいえ」と答えた方は、安心してコードを書き始めてください。その際は、忘れずにテストを書くようにしてください。いずれかの質問に「はい」と答えた方は、おめでとうございます。これで多くの時間を節約できました。既存のものを活用しましょう！

とはいえ、昨日のベストプラクティスは、往々にして明日のアンチパターンとなります。5 年前や 10 年前に正しいとされていたことが、**今日でも**正しいとは限りません。なぜそうするのかを問い、現状に疑問を投げかけるようにしてください。気づかないうちに、見つけたパターンやアプローチを無批判に真似してしまったり、AI が生成したコードを、その理由を十分に**理解しないまま**安易に受け入れてしまったりするおそれがあります。ソフトウェアの世界は目まぐるしく変化しています。時折立ち止まって周囲を見渡し、意識的に広い視野を持つようにしなければ、本当に重要なことを見逃してしまうかもしれません（「1.6 新たな視点の価値」参照）。言語、フレームワーク、テクノロジー、手法は絶えず進化しています。それらについていくことは、ソフトウェアエンジニアとしての当然の責務なのです。核心を突く質問を投げかけ、常に先を見据えておくようにしましょう。

3.2 良いコードとは？

とはいえ、問題を解決してくれるようなライブラリや言語機能が必ず存在するというわけではありません。その場合、読者はコードを自分で書くことになるでしょう！ そこで、「良いコードとは一体何なのか？」と自問するかもしれません（https://oreil.ly/ors7V）。Potter Stewart の言葉を借りれば、良いコードとはどういうものかを定義するのは難しいのですが、実際に目にすればそれが良いコードだとわかるものです。確かに、良いコードと悪いコードの区別は非常に主観的な概念であると言えます。ただ、しばらくすると、その感覚が身につき、コードには「スメル（臭い）」がある（https://oreil.ly/PDUiT）とさえ言えるようになるかもしれません。例えば、長すぎるメソッドは一般的に良くないものですが、それでもコードを詳しく調べる必要があります。もっとシンプルなアプローチが存在しない場合もあるからです。自分が悪いコードを書いたと認める開発者はほとんどいません。しかし、エディタを開いたことのある人なら誰でも、あまり美しくないソフトウェアを作った経験があるはずです。

メトリクスを使えば、コードベースに関する詳細な情報が得られます。例えば、サイクロマティック複雑度（https://oreil.ly/hw_0S）は、ソースコード内のパス数を示してくれます。パスが多いほどコードは複雑であり、リファクタリングを行うことで改善できるでしょう。多くのプログラミング言語には、PMD（https://oreil.ly/fg6Du）、SonarQube（https://oreil.ly/TyPeY）、JSHint（https://oreil.ly/2n5XP）などのソースコード解析ツールがあります。これらのツールを使えば、ある種のバグや不適切なコーディング慣行がソースコードに蔓延するのを防げます。

SonarQube や CodeScene のようなツールは、コードベースに関して非常に貴重な情報を提供してくれます。おそらく、読者の組織でも何かしらのツールが導入されていることでしょう。よくわからない場合は、周囲に聞いてみてください。もし導入されていないのであれば、プロジェクトに導入するための取り組みを率先して行ってみてはいかがでしょうか。

既存プロジェクトにメトリクスを追加する

理想的な状況であれば、プロジェクトの初期段階から、前述のような分析ツールを活用することになるでしょう。そうすることで、コードを「クリーンな状態で開始し、クリーンな状態を維持する」ことが（同僚が検出された問題を修正すると仮定すれば）可能になります。しかし、常にそのような理想的な環境ばかりではなく、数か月あるいは数年もの間開発が続いているプロジェクトにリンターやソースコードアナライザーを追加しなければならない場合もあります。

PMD のようなツールを導入して、すべてのルールを有効にしたくなるかもしれません！ しかし、それは避けてください。第一に、ルールの中には矛盾しているものがあり、あるルールを通過しても、別のルールで失敗してしまうことがあります。第二に、ルールを有効化しすぎると警告数が管理できないレベルになり、かえって逆効果になります。チームの士気を低下させるおそれもあります。数百、数千もの警告がある中で、誰かがコードをチェックインしてさらに数件の警告が追加されても気づきにくいでしょう。また、数十件の警告を修正するために数時間を費やしても、グラフ上の変化はごくわずかな数ピクセルに過ぎません。

その代わりに、チームと話し合い、全員が納得できるルールをいくつか選んでください。そのルールセットを見つけるのは、なかなか難しいかもしれません！ ルールを選んだら、そのルールセットだけを有効にしてください。そうすれば、警告の数は管理できるレベルになるはずです。チーム一丸となって、それらの修正に取り組みましょう。修正が終わったら、さらにいくつかのルールを有効にします。この手順を繰り返してください。数か月後には、プロジェクトに対して適用されるルールセットが充実し、コードもよりクリーンになっていることでしょう。

メトリクスを無批判に受け入れるべきではありません。避けられない複雑さもあるのです！ どのような経験則にも常識を働かせる必要があります。なぜなら、場合によっては、一般的なルールを適用することで、かえって事態を悪化させてしまうことがあるからです。ただし、ホーソン効果（https://oreil.ly/CGxL8）（人は観察されていると行動を変えるという現象）を活用することは有効です[†2]。プロジェクトにおいて、これを活用するのはまったく問題ありません。例えば、テストカバレッジを向上させたいなら、コードカバレッジの統計を目立つ場所に表示すればよいのです。

†2 高速道路で、中央分離帯に警察のパトカーがいることに気づくと、みんながスピードを落とすようなものです。

メトリクスを活用したコードカバレッジの向上

再び Nate です。何年も前、私は数か月間コードが次々と書きつづけられていたプロジェクトに参加したことがあります。そんな状況にもかかわらずテストが存在していたことには嬉しい驚きを覚えたものの、「失敗しつづける」という理由で、ごく一部を除いてすべてがビルドでスキップされていたことにがっかりしました。それでもめげずに、私はすべてのテストを実行した後に、何行のコードが実行されたかを示すコードカバレッジレポートを作成しました。予想通り、その数は一桁でした。しかし、私はそれを掲示し、そのことについて話しました。何度も何度も。ほぼすべてのスタンドアップやレトロスペクティブで。その数字が少しでも増えるたびに、私はチームメンバーたちを素晴らしいと大いに称賛しました。誰かがコードの特に難しい部分にテストを追加したときは、その方法を皆で話し合うように促しました。

ゆっくりとではありますが着実に、コードカバレッジの数値は上昇し、コードも堅牢になっていきました。数か月を要しましたが、私が次のプロジェクトに移る頃には、コードカバレッジは 70% に迫る水準に達していました。決して満足できる状態ではありませんでしたが、以前よりはるかに良い状況になっていました。そして、そのすべては単純にコードカバレッジレポートを実行し、それをチーム全体に浸透させたことから始まったのです。

メトリクスは悪用されたり、誤用されたりするおそれがあります（2 章のコラム「メトリクスは誤解を招く」参照）。例えば、多くの組織が、技術スタッフの評価に、記述したコード行数、削除したコード行数、修正したコード行数を用いようと試みてきました。こうしたメトリクスは賢く使うべきです（https://oreil.ly/fjXT0）。別の例として、外部結合度や、あるクラスに依存しているクラスの数について考えてみましょう。一般的に、結合度が高い場合は、リファクタリングしたほうがよいと捉えるでしょう。しかし、そのクラスが実際には一連のクラスを隠蔽するファサードである場合はどうでしょうか？ どのようなメトリクスであっても、コンテキストを考慮して解釈する必要があります。

何かを測定できるからといって、それが必ずしも意味のある情報をもたらすとは限りません。メトリクスをプロジェクトの目標と結びつけ、その推移に注目するようにしてください。時間の経過とともに改善しているでしょうか、それとも悪化しているでしょうか？ 短期的な視点に立ち、調整と適応を行う準備と意欲を持って臨みましょう。

パイプラインに新しいツールを追加するだけが選択肢ではありません。多くの IDE にはコード品質のメトリクスが備わっており、より良いコードを書くのに役立つプラグインも数多く存在します。フィードバックループを短縮するために、多くのコード分析ツールはエディタに連携できます。ビルドが失敗するのを待たなくとも、入力中に違反を通知してくれます。チームごとに推奨される設定があるかもしれません。よくわからない場合は、同僚に尋ねてみるとよいでしょう。

3.3 少ないほど良い

> ソフトウェア設計の目的は、人間の頭脳に収まるような「チャンク」や「スライス」を作り出すことにあります。ソフトウェアはどんどん大きくなりますが、人間の頭脳には限界があるため、変更を加えつづけたいのであれば、常に新たな方法でチャンク化やスライス化を行わなければなりません。
>
> — Kent Beck

コードベースの規模を考えると、開発者の報酬が文字数に応じて決まっているのではないかと考えてしまうこともあるでしょう。確かに、解決に数百万行のコードを必要とする問題もありますが、ほとんどの場合、コードベースは小さいほど良いと言えます。コードが少なければ少ないほど、人の頭に入れるべき情報も少なくて済むからです。多くのプロジェクトは、一人の開発者ではコード**全体**を理解できなくなるほどの規模に達しており、これがマイクロサービスや FaaS（Function as a Service）が台頭した要因の一つとなっています。

典型的な「ビッグ・ボール・オブ・マッド（無秩序に肥大化したシステム）」（https://oreil.ly/6DR8g）では、辞書並みの分厚い入門ガイドがあり、ビルドプロセスには数週間、あるいは月の満ち欠けほどの時間がかかります。プロジェクトに新しく参加した開発者が、コードを扱えるようになるまでには、数週間から数か月を要することもあります。コードベースが小さければ小さいほど、新しい開発者がコードを理解するまでの時間は短くなり、より早く貢献できるようになります。詳細については 6 章を参照してください。コードの長さを適切に管理する際、これから説明する点を考慮する必要があります。

3.3.1 コンピュータサイエンスの第ゼロ法則

ソフトウェアエンジニアがコードを整理するために提唱する手法の多くは、コンピュータサイエンスの「第ゼロ法則」に帰着します。それは、「高い凝集度、低い結合度」です。**凝集度**とは、要素がどの程度密接に関連しているかを示す尺度です。高い凝集度とは、簡単に言えば類似した要素がまとまっているということです。印刷ロジックも含まれている通知関数は、凝集度が低い例と言えます。**結合度**とは、モジュールやルーチン間の相互依存の度合いを指します。結合度の高いコードは、コードの一部を変更した際に、一見無関係に見える他のシステムにも予期せぬ影響が及ぶため、修正が困難になることがあります。通知サービスを変更しただけなのに、印刷モジュールが動作しなくなるようなことはあってはなりません。

凝集度が高く、結合度が低い状態であれば、コードの可読性を高め、保守や更新が容易になります。多くの設計手法は、さまざまな抽象化レベルで高い凝集度と低い結合度を実現する手法です。マイクロサービスは、サービスに対して高い凝集度と低い結合度を実現した例と言えます。

3.3.2 定型コードに注意する

定型コード（ボイラープレートコード）は、エディタやフレームワークによって生成されたもの

であっても避けるべきです。自分で**書く**必要はないかもしれませんが、そのコードはプロジェクトの存続期間中ずっとついて回ることになります。クラスは短く、数ページ以内に収めるべきです[†3]。どのプログラミング言語を選ぶかで、**短い**という定義は異なります。言語によっては、他の言語よりも記述が冗長になるものもあるからです。一般的に、スクロールが必要になるなら、そのクラスは長すぎる可能性があります。

3.3.3 継承よりも合成を優先する

多くのプログラミング言語では、クラスが他のクラスから継承できるようになっています。この機能は非常に強力であり、ソフトウェアにも確かに **is-a** 関係（猫 **is-a** 哺乳類）は存在しますが、継承は使われすぎている傾向があります。一部の開発者は、継承を再利用の手段として使っています。しかし、再利用は副産物であって、継承を正当化する理由にはならないのです！

具体的な例を見てみましょう。例えば、自動車とトラックを扱う場合を考えてみましょう。自動車（Car）とトラック（Truck）の両方が継承する車両（Vehicle）というスーパークラスを作成し、そこに内燃機関（Engine）を含められます。なぜなら、すべての自動車とトラックには内燃機関が搭載されているため、スーパークラスで内燃機関を定義することで、すべての自動車とトラックにも内燃機関が搭載されていることが保証されるからです。例えば、以下の Ruby の擬似コードを見てください。

```
class Vehicle {
        Engine engine
        Integer num_wheels
        Integer num_doors
        Function brake {}
        Function accelerate {}
}

class Truck < Vehicle {
        Number tow_capacity
}

class ElectricVehicle < Vehicle {
        Number range
        # あれ… EV には内燃機関がない…
}
```

しかし、内燃機関を一切搭載していない電気自動車（ElectricVehicle）が登場しました。EV は確かに車両ですが（is-a の関係）、**他の**車両とは異なります。合成を使うほうが柔軟性が高いので、継承よりも優先して使うべきです。継承を**絶対に**使わないようにすべきという意味ではなく、合成を優先すべきだということです。

†3　ただし、最近のモニターのサイズであれば、1 ページ以内に収めるほうがよいでしょう。

3.3.4 メソッドは短くする

メソッドは短くしてください。コード行数が一桁程度になるのが理想です。Linux や Unix のコマンドラインツールのように、メソッドは一つのことだけを行い、それをしっかりと行うようにしてください。そして、他のメソッドと連携して大きな目標を達成するのです。メソッド名に接続詞（and、or、but など）が含まれるようなメソッドは、そのメソッドが仕事をしすぎていることを示しています。メソッド名は明確かつ簡潔であるべきであり、小難しい表現は避けるべきです。メソッド名のつけ方に苦労している場合、そのメソッドは機能が多すぎる可能性があります。機能を分割してみて、どうなるかを確認してみてください。メソッド名は、メソッドの機能を説明する名前にしましょう。ロジックが重複している場合は、たとえ少しの重複でも排除してください。単純化し、さらに単純化してください。

全体的に、開発者が理解しやすく、生産性を高められるように、より小さく管理しやすいコードベースを目指しましょう。

3.4 読まれることを意識してコードを書く

読者の後を継ぐ開発者、つまり読者のコードを読み、保守を行う人のことを考えてみてください。コードを書く際には「黄金律」を適用することを忘れないでください（「1.8 ソフトウェアに黄金律を適用する」を参照）。次の開発者の仕事を楽にするために、どのようなコードを書けばよいでしょうか？ 次の担当者に何を伝える必要があるでしょうか？

関数の「適切な」長さについては、再利用性を重視する考え方から、行数だけを基準にする考え方まで、数多くの指針が存在します。Martin Fowler は、意図（何をするのか）と実装（どうやるのか）を分離するよう、示唆に富んだ意見（https://oreil.ly/1WZKx）を述べています。つまり、その関数が何をするのかを理解するのに、どれくらいの時間がかかるでしょうか。メソッド本体を確認しなくても、名前を読んだだけでコードの動作がすぐに理解できるようにすべきです。この原則に従うと、コードは数行、あるいはたった 1 行だけの関数になることもよくあります。**意図**を明確にしてください。クラス名はコンテキストを表し、メソッド名はクラスが何をするのかを理解する手助けとなるはずです。

コードは、「逆ピラミッド型」の新聞記事のように書けます。新聞記事はリード文から始まり、重要な事実へと移り、さらに深い背景へと展開していきます。新聞記事を読む人は、好きなところで読むのを止められます。最初の数段落だけかもしれませんし、最後まで読み通すかもしれません。出版業界でよく言われるように、肝心な部分を後回しにしてはなりません。

コードを記述するときも、新聞記事と同じようなモデルで記述するべきです。クラス名は、いわば記事の見出しのようなものです。クラス名を見た後、メソッドのシグネチャにざっと目を通すだけで、おおまかな内容を把握できるはずです。特定の関数や他のクラスの呼び出しについて詳しく調べたいなら、そうしても構いませんが、調べる前でもコードの要点は理解できているべきです。

名前をつけるのは難しい

開発者なら誰でも、変数やメソッド、クラスの名前をつけるのに苦労し、エディタ画面をじっと見つめた経験があるはずです。このように苦労するのは、問題に対する理解が不十分だったり、コードが複雑すぎたりするのかもしれません。しかし、`foo`、`bar`、`foobar`といった名前がこれほど頻繁に使われるのには理由があります。意味のある名前を思いつくのは、時間がかかるのです。焦らないでください！ 繰り返しになりますが、時間をかければ、結果的に時間を節約できます。努力して良い名前を考えることは、それだけの価値があります。チームメンバーに意見を求めたり、相談したりするようにしてください。自分が取り組んでいる内容を説明するだけで、完璧な名前が思い浮かぶこともあります。

IDE が役立ちます！ 多くの開発者は宣言をまず記述し、名前についてはエディタに提案をさせます。つまり、`new Arrays.asList({"Red", "Green", "Blue"});`をまず記述するのです。IDE は接尾辞を補完して変数名を提案してくれます。エージェント型コーディングツールもヒントを与えてくれます。

不適切な名前がつけられたコードは、遠慮なくリファクタリングしてください。モダンエディタには、名前の変更を簡単に行える強力なツールが備わっています。ただし、ドメインのコアとなる概念の名前を頻繁に変更しないように注意してください。頻繁に改名したいのであれば、それは一般的に、概念を誤解していると考えられるからです。

ジブリッシュゲーム（https://oreil.ly/YDkTZ）（意味のない言葉でコミュニケーションする遊び）をするのもよいでしょう。概念を定義する際に最初に思い浮かんだ言葉が最良のものとは限らないのに、その言葉が問題領域に対する考え方を形作ってしまうおそれがあります。その領域について初めて検討する際は、まず架空の言葉を考えてみてください！ さらに分析を進めた後、その意味のない言葉を実際の言葉に置き換えてみましょう。そうすれば、最初の直感とはまったく異なり、より明確な表現が思い浮かぶはずです。次に名前について本当に迷ったときは、意味不明な言葉をとりあえず使っておいて、改めて検討してみてください。

3.5 コードのコメントに関する問題

どのプログラミング言語にも、開発者がコンパイラやランタイムではなく、仲間の開発者に**語りかける**ためのコメント機能が備わっています。コードには積極的にコメントをつけるべきだと思われるかもしれませんが、実際には、積極的にコメントをつけるのはコードスメル（コードの悪臭）（https://oreil.ly/6LoPG）と言えるでしょう[†4]。コードのコメントは DRY 原則（Don't Repeat Yourself《繰り返すな》）（https://oreil.ly/m-B67）に違反します。DRY 原則違反は、コードのコピー＆ペーストやロジックの重複によって引き起こされることがほとんどですが、コメントもま

†4 開発者の経験レベルは、多くの場合、コードのコメントをどのように使っているか（使っていないか）によって判断できます。

た問題となる可能性があります。多くの場合、コードを書いた後に、そのコードについてコメントを書くことになるからです。

コードのコメントも、保守において悩みの種になり得ます。コードを修正した際、わざわざコメントも更新する時間を割くでしょうか？ 多くの場合、コメントはコードの内容と乖離し始め、数か月後や数年後にそのコードを目にする開発者にとって、さらなる混乱の要因となってしまいます。コードを読みやすいように書くべきです。自分が何をしたかをドキュメント化するよりも、コードを読みやすくすることに時間を費やすほうが賢明です。表現力豊かなプログラミング言語は、間違いなく読みやすいコードの作成に役立ちます。ただし、プログラミング言語の「魔法のような」機能は使わないほうがよいのかもしれません。適切にメソッド名や変数名をつけるほうが、コメントを記述するよりも優れています。コメントを書く必要があると感じた場合は、そのメソッドや変数の名前を変更してみてください。そうすることで、ほとんどの場合、コメントの必要性はなくなるでしょう。

おそらく、最も役に立たないコードのコメントと言えば、ファイルの最初の数百行（あるいは数千行）にしばしば見られる変更履歴の記述です。これらは、コードの「発掘」作業における興味深い手がかりとなり得るものの、追跡番号や日付、名前、変更内容の説明文が繰り返されることで、かえって作業の妨げになってしまいます。このような役割は、ソースコード管理ツールに任せましょう。

とはいえ、コードにコメントを**一切入れてはいけない**というわけではありません[†5]。もし、一見してわかりにくい処理を行っており、かつコードを簡潔にする方法が見つからない場合は、状況を説明するコメントが役立つことがあります。こうしたコメントは、「**何を**したか」ではなく「**なぜ**そうしたか」を中心に記述するようにしてください。いくつかのアルゴリズムの中から、あるアルゴリズムを選んだ理由を説明したり、特に複雑なロジックの部分を解説したりすることも、同様に役立ちます。とはいえ、コードに説明が必要になるようなら、書き直すべきです。

コメントは、将来の自分へのメモとして、あるいは「このハックは機能するが、その仕組みを（まだ）理解していない」といった警告を残すのにも役立ちます。一部の開発者は、次の例のように、将来の開発者への警告としてコメントを残すこともあります。

```
// 保守担当者さまへ
//
// このルーチンの
// 「最適化」を試みた後に、
// それがどれほど大きな
// 間違いであったかと気づかれたなら
// 次の担当者への警告として
// 以下のカウンターを
// ぜひ増やすようにしてください。
//
// total_hours_wasted_here = 42
```

†5　ソフトウェアであれ人生であれ、教条主義が正しいことはめったにありません。実用主義を重んじましょう。

こうしたコメントは良いものなのでしょうか？ それは状況によります。よく耳にする例の多くは、おそらく作り話でしょう。とはいえ、未来の自分へのアドバイスを残すようにしてください。

3.6 テストをドキュメントとして活用する

もしコメントがコードをドキュメント化するのに適切な方法ではないとしたら、どうすればよいのでしょうか？ テストを書きましょう。テスト、特に洗練されたスタイル（https://oreil.ly/h6kSc）で書かれたものは、本番コードとともに進化する実行可能な仕様書となります[†6]。コードのコメントであれ、README であれ、仕様書であれ、ドキュメントは一度書かれるとすぐにコードから乖離していきます。コードを書く際にテストを記述しておけば、自由にリファクタリングを行うことができ、アプリケーションの品質に対する自信を高められます。また、後任の開発者にとっての道しるべともなります。テストについては、5 章でさらに詳しく解説します。

既存のコードにテストを追加すれば、読者がコードの動作について学んだことを記録し、その知識を共有できます。そうすることで、他の開発者も読者の成果を活用できます。読者がメソッドや変数の名前を変更したり、不要なコードを削除したりするたびに、読者がコードに初めて触れたときよりも良い状態へと変えているのです。

一部の開発者は、自らが提供するサービスを利用するユーザーのために、膨大なコメントを書く必要があると主張します。こうしたコメントは、前述のものほど「臭いもの」ではありませんが、唯一の選択肢というわけではありません。繰り返しになりますが、テストこそが、より堅牢なドキュメント化の仕組みとなります。コンシューマ駆動契約を活用することで（https://oreil.ly/nHHcE）、サービスが何を実行するかを伝えると同時に、必要に応じて自信を持って柔軟に改善を重ねられます。契約に違反していない限り、下流のシステムを意図せず壊してしまうという心配を抱くことなく、コードを進化できます。利用者は、コードの期待される動作をシミュレートする一連のテストを実行できるため、サービスに対して信頼を寄せることができます。また、新たに不具合を引き起こすことを恐れることなく、自身のコードを変更することができるようになります。

コンシューマ駆動契約は、信頼性が高く、レジリエンスのあるソフトウェアにとって不可欠な要素です。多くの言語やフレームワークには、アプリケーションで活用できる（そして活用すべき！）コンポーネントが存在します。Spring Cloud Contract（https://oreil.ly/1SaQp）から、ほぼすべてのプラットフォームに対応した Pact（https://oreil.ly/fKeT7）に至るまで、選択肢は豊富に存在します。

†6 Spring REST Docs（https://oreil.ly/VDbKF）のようなライブラリも、ドキュメントとコードの整合性を維持するのに役立ちます。

3.7 「小賢しい」コードは避ける

ソフトウェアは難しく、読者が携わる分野も複雑です。しかし、すべての複雑さが同じというわけではありません。広く引用されているエッセイ「No Silver Bullet（銀の弾丸はない）」[†7]の中で、Fred Brooks は「偶有的な複雑さ」と「本質的な複雑さ」を区別しています。**本質的な複雑さ**はソフトウェアに内在するものであり、ビジネスルールの細かな違いから、チームとのコミュニケーション、さらには絶えず変化しつづけるコードベースの性質に至るまで多岐にわたります。この複雑さをソフトウェアから取り除くことはできません。それは仕事に伴う必然的なものです。一方、**偶有的な複雑さ**とは、余計な複雑さを持ち込むテクノロジーや重厚すぎるツールなど、開発者が必要以上に物事を難しくしてしまう要因のことです。迷ったときは、シンプルに保つようにしましょう。

ソフトウェアの世界も、いわゆる「怪しげな商品」（https://oreil.ly/WCFDX）とは無縁ではありません。多くの企業が、ソフトウェアの提供方法に革命をもたらすような製品やプロセスを売り込もうとしています。懐疑的な姿勢を持つことが大切です。もちろん改善の余地はありますが、科学的方法論が教えてくれるように、「常識を超える主張には常識を超えるような証拠」が必要です。可能な限り、偶有的な複雑さを排除するよう常に注意を払うべきです。詳細は 6 章を参照してください。

プログラミング言語やフレームワークには、ミスを招きやすい記述形式がよくあります。例えば、次の Java コードをご覧ください。問題点が見つかりますか[†8]？ Java では、単一ステートメントからなる `if` ブロックにおいて、中括弧は**言語仕様上**省略できます。さて、一部の開発者は、中括弧を省略するほうが冗長性が減り、中括弧を記述する形式は（実質的に）空行が縦方向のスペースを無駄にするだけで、中括弧を書かない形式のほうがよいと主張するかもしれません。しかし、別の開発者が 2 つ目のステートメントを追加したらどうなるでしょうか？ 彼らは中括弧を追加することを覚えているでしょうか、それともコードのインデントに惑わされてしまうのでしょうか？

```
if (condition)
  doFoo();
  doBar();
```

使用するツールチェーンにおいて、エラーが発生しやすい構文や設定は避けるようにしてください。自分ははっきりと理解しているからといって、その知識がチーム全体に広く浸透しているとは限りません。こうしたケースを防ぐため、コーディング標準を更新（または策定）するようにしてください。デプロイパイプラインに組み込めるような静的解析ツールが数多く存在します。このようなツールを組み込むことで、自分自身やチームがうっかりこうした問題を引き起こすのを防げます。

†7 https://learning.oreilly.com/library/view/mythical-man-month-the/0201835959/ch16.xhtml
†8 執筆者の一人（ここでは名前は伏せます）は、このコードブロックのデバッグに 2 週間近くを費やしました。

3.8 コードレビュー

コードレビューの形態は、同僚にメソッドについてフィードバックを求める程度のものから、複数の開発者が参加して数時間にわたるウォークスルーを行うものまで多岐にわたります。具体的な実施方法にかかわらず、コードレビューは学びや経験を共有し、知識を広めるための優れた手段です。コードを多くの人の目に触れるようにすることは有益であり、一部の組織がペアプログラミング（https://oreil.ly/i1r_o）を採用している理由の一つでもあります。

正式なものであれそうでないものであれ、以下に示すような実践方法を行えばコードレビューの質を高められます。何よりもまず、嫌味な態度を取らないようにしましょう。皮肉は避けてください。そもそもフィードバックを求めることは相手にとって大きなストレスです。そして、多くの人は批判を個人に対するものと受け止めてしまいます。コメントの伝え方が極めて重要です。チームメンバーに共感を持って接してください。コードレビューを、チームに対して小難しい豆知識を披露する機会として使いたくなるかもしれません。しかし、コードレビューの目的はコードを改善することであり、自分の技術的な専門知識を誇示することではありません。

> 何かを素晴らしいものにする唯一の方法は、それがまだ素晴らしいものではないかもしれないと認めることです。目標は最善の解決策を見つけることであって、それによって自分の価値を測ることではありません。
>
> — Jonas Downey、ソフトウェアデザイナー、開発者、作家

最も重要なことに集中してください。スタイルの評価も重要ですが、エネルギーはコードレビューのピラミッド（https://oreil.ly/Uj3Cj）の下部階層にある部分に費やすのが賢明です。書式やスタイルに関する問題は自動化できます（そしてそうすべきです）[†9]。それらについてはコンピュータに任せましょう。時間とエネルギーは、コンピュータでは検知できない部分に費やすべきです。メソッド名や変数名は明確かつ簡潔ですか？ コードは読みやすいですか？ 重複はありませんか？ コードには適切なロギング、トレース、メトリクスが実装されていますか？ インターフェースはコードの他の部分と整合性がありますか？ 開発者はエラーが発生しやすい形式を使用していませんか？ モデルは正しいですか？ 間違った抽象化を選択していませんか？

3.9 チェックボックス式のコードレビューは避ける

場合によっては、コードレビューがソースコード管理システム上のチェックボックス項目を確認するだけのこともあります。組織によっては、すべてのコードを main ブランチにマージする前にレビューすることを義務付けています。その目標自体は称賛に値するかもしれません。しかし、多くの場合、ある開発者が別の開発者に「レビュー」を依頼し、依頼された開発者が同僚のために

†9 いわゆる「LGTM（Looks Good To Me、ざっと見て OK）」とするだけのレビューは、人の時間の使い方としてはあまり良いものではありません。

チェックボックスにチェックを入れるだけという結果になりがちです。善意から行われているとはいえ、そのようなレビューは本来の目的を損なうことになります。小さな単位で少しずつレビューしていくことで、レビューはより単純かつ効果的になります。

一部の組織では、コード品質を確保する手段としてプルリクエスト（PR、Pull Request）を活用しています。コードについて議論するために午後の時間をビデオ通話に費やすよりも、PR のほうが気楽な場合もありますが、PR が必ずしもチームの結束や信頼の構築に役立つとは限りません。一部の開発者は、PR の些細な点を指摘したり、縄張り争いを始めたり、以前に解決済みの問題を蒸し返したりする機会として利用することがあります。また、PR は「LFTM（looks fine to me、問題なさそう）」という反応に流れやすいという側面もあります。これは場合によっては許容できるかもしれませんが、前述の「チェックボックス式レビュー」に陥ってしまうおそれがあります。

テキストによるコメントには、対面での会話のような口調やボディランゲージが欠けています。皮肉や辛辣な**意図がなくても**、相手にはそう受け取られてしまう可能性があります。チームメンバー、特に経験の浅いメンバーが PR を恐れているとしても無理はありません。繰り返しになりますが、相手の立場に立って考えることを心がけてください。自分がどう扱われたいかを自問し、それに応じて行動してください。シニアスタッフは率先して模範を示すべきです。

3.10 批判されるのは辛いこと

開発者は自身の仕事に多大な情熱を注ぐことが多いため、フィードバックを個人に対するものとして受け止めてしまうことがよくあります。コードレビューは、新しい言語機能を知らなかったり、より簡単な方法を思いつかなかったりしたからといって、相手に恥をかかせる場ではありません。完璧な人はいませんし、誰にでもミスはあります。コードレビューはより良いアプリケーションを構築するためのものであり、コードそのものが対象であって、開発者個人を批判する場ではありません。コードレビューでは、個人攻撃は控えてください。謙虚な姿勢を保ち、建設的な質問をしましょう。批判は、褒め言葉で挟まれると受け入れやすくなります。ぜひ、良い点も必ず指摘するようにしてください。

自分自身の経験を共有してください。実体験に基づく話は説得力があり、変化に対する心理的な抵抗感を和らげることができます。過去のプロジェクトで直面した問題についてどのような解決策を使ったのかを提示しましょう。断定的な発言には注意してください。何かが「うまくいかない」と断言する前に、詳細をすべて把握するようにしてください。何か見落としている背景情報はありませんか？ あるいは、読者が気づいていない制約条件があるのかもしれません。あくまでコードに基づいて判断しましょう。

> 私たち一人ひとりは、それぞれの意識の状態のもとで、できる限りの最善を尽くしています。
> — Deepak Chopra、インド系アメリカ人作家、代替医療提唱者

誰かのコードに気になる点があれば、遠慮せずにその開発者に直接話しかけてみてください。人

は、特に集団の場では、つい自己防衛的になりがちです。短時間の 1on1 が、解決の糸口になるかもしれません。チームメンバーを突然責め立てるようなことは避けてください。そのようなやり取りでは、誰も得することはありません。レビューは学ぶ機会であり、教える機会でもあるということを忘れないでください。

3.11 信頼の構築

チェックボックス式や LFTM レビューを行うべきではないとしたら、何を行うべきでしょうか？ どのようなコードレビューをするかにかかわらず、その目的を見失わないようにしましょう。チームとして経験を共有し、学び、成長しながら、混乱を招くようなスタイルや非推奨の手法といった問題のある記述方法を避けるべきです。チームメンバーが質問したり、建設的なフィードバックを提供したりすることを奨励しましょう。コードレビューは、コードに対してチーム全体でのオーナーシップを育み、チーム内の信頼関係を深めるものでなければなりません。「今週のバグ」を共有したり、自分が遭遇した問題について時間をかけて共有したりすることは、非常に大きな効果をもたらすことがあります。

メンバーが解決した面白い不具合について話し合う場を定期的に設けるのは、学習方法として非常に価値のあるものです。特定の問題に直面した際、誰もが対処方法を知っていると考えるかもしれませんが、チームメンバーとそうした状況を話し合うことで、知識が広まります。金曜日のスタンドアップミーティングの時間を少し割いて、その週にメンバーが経験した興味深い事例について話し合う、という単純なことでも、大きな成果をもたらすことがあります。

言うまでもありませんが、チームメンバーには敬意を持って接してください。親切に振る舞い、正しいことを行い、効果的な方法を実践しましょう。自分のやり方を振り返り、もっと良い方法はないかと考える時間を取るようにしてください。アジャイル開発手法を採用しているかどうかにかかわらず、定期的に状況に合わせて柔軟に対応して調整するべきです。もしうまくいっていないことがあれば、それを変えましょう！

3.12 新しいプログラミング言語を学ぶ

コードを書くスキルを向上させたいのであれば、やはり、コードを書く必要があります。新しいプログラミング言語を学ぶことで、そのスキルを飛躍的に伸ばせます。外国語を学ぶことを考えてみてください。何か月も、いや何年もかけて、その言語を母語に翻訳して理解することになるでしょう。やがて、その新しい言語で考え、夢にまで見るようになるはずです。しかし、それには時間がかかります。プログラミング言語も同じことです。もっとも、プログラミング言語には、非常に厳格な文法規則があり、キーワードははるかに少ないのですが。

重要なのは、その環境に没頭することです。もし本当にフランス語を習得したいのであれば、パリに移住することで上達のスピードが加速するでしょう。新しいプログラミング言語を習得する場合も、これと同じことが言えます。仕事や家庭で悩まされている問題を解決するアプリを、その新

しい言語で開発してみましょう。ソーシャルメディアでコミュニティをフォローし、関連するポッドキャストを聴いたり、その言語に関する最新のカンファレンスの動画を見たりしましょう。さらに良いのは、地元のユーザーグループに参加したり、そのテーマに関する次のイベントに出席したりすることです。

開発者は、初めて触れた言語に強く愛着を抱きがちです。この点には注意が必要です。プログラミング言語はあくまでツールに過ぎません。釘を打つにはドライバーよりハンマーのほうが適しているのと同じように、特定の問題に対しては、他の言語よりも適した言語があります。例えば、多くの組込みシステムはC言語で記述されています。これは、C言語が移植性が高く信頼性があり、特定のプラットフォーム向けに高度に最適化できるからです。他の言語を調べて、探求し、学ぶことを習慣にしましょう。いわゆるインフルエンサーと呼ばれる人（https://oreil.ly/2wY8L）の中には、1年か2年ごとに新しい言語を学ぶことを勧める人もいます。

プログラミングの本質は、コミュニケーション

世界中の大学にコンピュータサイエンス学科が次々と設立されるようになる前は、プログラミングは数学科に属することが多く、ソフトウェア分野で成功するには数学的素養が不可欠だと多くの人が考えていました。今でも、学生を選抜するのに数学の試験を行う大学があるほどです！ こうした通説とは裏腹に、研究によれば（https://oreil.ly/eKIhN）、新しいプログラミング言語を習得するには、数学のスキルよりも**言語**能力のほうが重要であることが示されています。特定の分野では数学が不可欠な場合もありますが、プログラミングという技術そのものはそうではありません。

プログラミングとは、何よりもまずコミュニケーションです。しかも、それはコーダーとコンパイラの間で行われるものではありません。忘れてはならないのは、コンピュータは、（少なくとも構文的に正しければ）**どんな**コードでも理解できます。しかし、人の場合は、読者の意図を理解してくれるとは限らないのです。優れたソフトウェアエンジニアであれば、コードを読む人のことを第一に考えます。優れた物書きは、（どんな分野であれ）常に読む人のことを念頭に置いているものです。

新しい言語を習得するには時間がかかります。仕事で毎日使うとは限らないものを学ぶために、その時間を費やすことをどう正当化すればよいのでしょうか？ たとえ毎日使う機会がなくても、新しい言語を学ぶことは、コーディングの仕方を変えることになるでしょう。新しい言語に挑戦する際は、仕事で使っているものとは異なるものを選んでみてください。経験豊富なJava開発者なら、C#のようなC系の言語（C#を学ぶことに何の問題もないことは言うまでもありませんが）ではなく、異なるパラダイムの言語に目を向けてください。代わりに、Rubyのような動的言語や、Haskellのような関数型言語を検討してみてはいかがでしょうか。普段使わない言語に軽く触れるだけでも、プログラミングへのアプローチが根本的に変わるはずです。普段使っている言語の良さ

を再認識できるかもしれません。あるいはまったく異なる方法でコードを書くようになるかもしれません。新しいことを学ぶために、ぜひ時間を割いてみてください。

新しい言語を学ぶことは、時間が経つにつれて簡単になっていきます。知っている言語が多ければ多いほど、比較できる対象も増えるからです。最初に学んだ言語を思い出してみてください。そのときは、ゼロからのスタートでした。3 つ目、4 つ目、5 つ目の言語を学ぶ頃には、ある記述方法が Java のそれとよく似ていること、あるいはある構文が Ruby から借用されたものであることに気づき始めるでしょう。

キャリアの早い段階では、日々使用している言語やフレームワークを深く掘り下げることにできる限り注力すべきです。しかし、他の選択肢を探求することをおろそかにしてはいけません。他の選択肢を探求することで、やる気が湧いてくるだけでなく、より優れた開発者になれるでしょう。

3.13 まとめ

開発者はコードを書きます。それは仕事の一部であり、その本質です。ミスを招きやすい形式や、小賢しいコードを避けるかどうかが、開発者が喜んで取り組めるコードベースと、まるで疫病のように避けられるコードベースとの分かれ目になります。コードレビューから分析ツールまで、より良いコードを書くための方法は数多くあります。長々としたコメントよりも、テストを書くことを優先しましょう。コードを批評する際は、相手の立場に立って考えてください。決して忘れてはならないのは、コードは人が読むために書かれるべきだということです。この原則を守ることが、他者から「良い」という評価を得られるコードを書くための重要な鍵となります。

良いコードを書くには、学習を続けていかなければなりません。新しいスキルを身につけることや、すでに身につけたスキルをさらに磨くことに、常に前向きな姿勢を持ちましょう。ソフトウェアエンジニアリングにおけるベストプラクティスを常に把握しておくことは、この分野で長期的に成功するための鍵となります。

3.14 実践に移す

近道はありません。開発者として上達したいのであれば、コードを書くしかないのです！ 昔からのジョークにこんなものがあります。57 丁目を歩いていた通行人が、タクシーから降りてくる音楽家を見て、「カーネギー・ホールへ行くにはどうすればいいのですか？」と尋ねました。音楽家は一瞬もためらうことなく、疲れた様子で「練習だよ」と答えました。より優れた開発者になりたいのであれば、練習を重ねる必要があります。

日々のタスクに、やや負荷のかかる学習を取り入れてみてはいかがでしょうか。2 か月に一回程度、定期的に数時間を確保して CodeKata（http://codekata.com/）に取り組んでみてください。武道における「形（カタ）」とは、生徒が学んで数え切れないほど繰り返し練習する、受け身や、蹴り、パンチの一連の動きのことです。CodeKata は、この考え方をソフトウェアに応用したものです。シンプルな課題を通じて、自身のスキルを磨く機会を提供し、日々のコーディング業務では遭

遇しないような技術に取り組めます。

CodeKata は、さまざまな方法で活用できます。例えば、1 つを選んで、2〜3 つのプログラミング言語で解いてみるのもよいでしょう。友人や同僚とペアを組んで、一緒にカタに取り組むのも一案です。1 つのカタには複数の解き方があります。同じカタを 2〜3 通りの方法で解くことに挑戦してみてください。数か月（あるいは数年！）前に解いたカタを今改めてコードレビューしてみましょう。今なら、どのような点を変えてみるでしょうか？

また、オープンソースソフトウェアの世界を活用することもできます。数時間を確保して、自分が使っている（あるいは使いたいと思っている）オープンソースライブラリへの貢献に取り組んでみてください。どこから手をつければよいかわからない場合は、GitHub のトレンドリポジトリ（https://oreil.ly/L_Eef）をチェックしてみてください。オープンソースへの貢献は、素晴らしい学習の場（https://oreil.ly/MhjGr）となりますし、始めるのは思っているほど難しくありません（https://oreil.ly/GeVpy）[†10]。

コードへの貢献をしなくても、優れたコードを書くための知識は十分に得ることができます。プロジェクトを一つ選び、数時間かけてコードを読んでください。気に入った点はどこでしょうか？ 気に入らない点はどこですか？ 自分ならどう書き直すでしょうか？ ソースコードアナライザーでコードを分析してみてください。そこからプロジェクトについてどのようなことがわかりますか？ もし、明らかな問題点が見つかったら、遠慮なく指摘したり、プルリクエストを送ったりしましょう！

変化についていくことは、ソフトウェア業界で成功するには重要な要素です。主要なプログラミング言語やフレームワークに関する知識を定期的に更新する習慣をつけましょう。日々の業務の中で自然と新機能に出会うことは少ないかもしれませんので、年に 1、2 回、数時間を割いて、ツールキットに何が追加されたかを確認しましょう。ほとんどのテクノロジーには提唱者や推進者がいますので、彼らをフォローしたり購読したりして、変化に遅れを取らないようにしましょう。新しい機能がすべて、少なくとも現時点では、読者のアプリケーションに役立つとは限りませんが、数年ごとに数十や数百もの新しいことを学ぼうとするよりも、定期的に少しずつ更新情報を消化するほうが簡単です[†11]。

最後に、地元の慈善団体や非営利団体に、自身のプログラミングスキルをボランティアとして提供することをぜひご検討ください。多くの団体が、技術コミュニティからの支援を常に求めています。ボランティア活動は、学びたいと思っている言語やフレームワークを使いながらスキルを磨く機会となるだけでなく、自分が関心を寄せる社会貢献にもつながります。

3.15 関連リソース

- "Portrait of a Noob"、Steve Yegge 著（https://oreil.ly/Ew3LD）

†10 多くのプロジェクトには、「初めて貢献する方向け」とマークされたバグのリストがありますが、遠慮せずにコントリビュータに連絡してみてください。ほとんどの人が喜んで手助けしてくれるはずです。

†11 そのテクノロジーの**アップグレード**についても同様ですが…

- 継承ではなく合成を使うべきという議論スレッド（https://oreil.ly/nfvEM）
- 『The Mythical Man-Month: Essays on Software Engineering, Anniversary Edition』Frederick P. Brooks 著（O'Reilly、1995 年）
  和書『人月の神話 新装版』（丸善出版、2010 年）
- "An Appropriate Use of Metrics"、Patrick Kua 著（https://oreil.ly/wsPB3）
- "Simple Made Easy"、Rich Hickey 著（https://oreil.ly/AaZYK）

4章
モデル

すべてのモデルは間違っています。だが、中には役に立つものもあります。
— George E.P. Box（の言葉とされる）、英国の統計学者

ここまでで、コミュニケーション、とりわけ他の開発者とのコミュニケーションが、ソフトウェアエンジニアとしての仕事において中心的な役割を果たすことは、おわかりいただけたかと思います。コンピュータは文法的に正しいコードさえあればよいのですが、人間同士のコミュニケーションではそれでは不十分な場合もあります。コードは、他の人にも理解してもらえるよう、十分にドキュメント化され、整理されているべきです（コードの書き方については 3 章を参照してください）。しかし、それでも不十分な場合があります。プロジェクト全体を通じて、技術的な意図を表現するために、ソフトウェアモデルや、箱と線による図を使用することになるでしょう。

良いコードと同じように、良いソフトウェアモデルは明確です。ステークホルダーにとって理解しやすいものです。モデルが明確でなければ、それを利用する人々は技術的な意図を理解できません。モデルは、多くの人に利用されます。ユーザー、テスター、他の開発者、セキュリティ担当者、予算を承認する人々、プロジェクトマネージャー、アーキテクトなどです。そして自分自身もその一人です。時には、図を利用するのが自分だけの場合もあります。とはいえ、単に図を描いただけでは、誰もがそれを理解してくれません。開発者にとって完璧な図でも、エンジニアリング担当副社長にとってはあまり役に立たないかもしれません。その逆もまたしかりです。問題は、どのような図を作成すべきか、そしていつ作成すべきかを把握することです。

4.1　ソフトウェアモデリングとは何か、なぜそれを行うのか

ソフトウェア産業は比較的新しい分野であるため、より成熟した分野から概念やアプローチを取り入れてきました[†1]。また、統一モデリング言語（https://www.uml.org/）（UML、Unified

†1　**アーキテクト**や**品質保証**といった用語に加え、データ関連ではライブラリアン、サイエンティスト、オントロジストといった役割にも出会うことになるでしょう。

Modeling Language）から C4 モデル（https://c4model.com/）に至るまで、モデリング手法にもさまざまな「潮流」がありました。建設業界では、新しいプロジェクトに着工する前に設計図一式を作成しますが、ソフトウェアプロジェクトも同様にすべきではないでしょうか？ そうかもしれません。

ソフトウェアモデリングとは、ソフトウェアシステムの構造、動作、および機能を、より深く理解し、分析し、伝達するために、その抽象的な表現を作成するプロセスです。これらのモデルは、開発者、設計者、ステークホルダーがシステムの設計・開発プロセスを進める際の指針となります。良いソフトウェアモデルは、現実世界の問題領域を反映しており、開発プロセス全体を通じて重要な情報を提供してくれます。

しかし、ソフトウェアの開発と家の建築との間には根本的な違いがあることを理解しておいてください。現実世界を「リファクタリング」するのは困難で費用もかかります。つまり、建設業者が、いわば「本番環境」で壁の耐荷重性能をテストするような余裕はないのです。設計図があれば、設計者はその家が地域の建築基準を満たし、水道、電気、冷暖房の配管・配線が適切に配置されていることを確認できます。着工前に設計図によって建設内容が伝えられるため、施主はすべてが期待通りであることを確認し合意できます。建設が完了した後は、設計図を用いてすべてが仕様通りに建設されたことを確認できます。もしさまざまな請負業者がすべて即興で作業することになったら、建設現場はどれほど混乱するでしょうか。

これに対し、ソフトウェアはリファクタリングが可能です[†2]。コードの変更は、数時間と数百ドルで済みます。一方、現実世界を「リファクタリング」するコストは、はるかに高額です。もし、ちょっとした変更が、数時間の追加作業と数百ドルの費用がかかるだけで済むなら、住宅の建て方は今とは大きく異なっていたでしょう。設計図は、建築プロセスにおいてこれほど重要なものにはならなかったはずです。ソフトウェアは柔軟性が高いため、図に表すことはプロジェクトの成功にとってそれほど重要ではないのかもしれません。

図はコンパイルされません。図は、プロジェクトの完成に必要な実行可能コードにはなりません。実際、開発者にとってコードこそが究極の設計成果物であるという主張は、かなり説得力があります。著名な論文「Code as Design」（https://oreil.ly/hMINX）の著者である Jack Reeves は、プログラミングは根本的に設計活動であり、その設計の最も純粋な表現はコードそのものであると論じています。

図がコンパイルできないのであれば、なぜそれが役に立つのでしょうか？ 図はコンテキストを提供してくれます。システムの複雑さを解きほぐし、問題を分解して理解・管理するために活用できます。図を使えば、9 章で学ぶ非機能要件、つまり品質特性を予測することも可能です。図は、特定の品質特性を考慮して設計を行ううえで役立ちます。

組織によっては、SDLC の一環としてさまざまな図を作成するよう求められることがあります。こうした要件は、メリットであると同時にデメリットにもなり得ます。

図を描くことは、システム設計の計画に役立ちます。図を描くことで設計内容を整理しやすくな

†2　ただし、想定以上に困難で費用がかかる場合もあります。

るのであれば、ぜひ図を描いてみてください。1〜2 時間かけてソリューションをスケッチするだけで、開発時間を数日短縮できるかもしれません。

図は、新しいシステムについて学んだり、初めてそのシステムに触れたりする際などソフトウェア考古学の調査においても役立ちます。もちろん、それは図が正確であることを前提としています。図は、新しいエンジニアのオンボーディングの際にも有用です。これもまた、図が正確であることを前提としています。さらに、チームの新メンバーに知識を伝達する際にも役立ちますが、これもやはり図が正確であることを前提としています。システムのデバッグにおいて、図は極めて重要になる場合があります。システムの境界やコアとなる役割を把握することで、テストケースの設計やデバッグの進め方を決めるのに役立つからです。

すでにお気づきかもしれませんが、図が正確で最新のものかどうかによって、その価値は大きく変わります。図が明確でなかったり、コードの現状を正しく反映していなかったりすると、その価値は大幅に低下してしまいます[†3]。そこで興味深い疑問が浮かびます。図はどれほど永続的なものなのでしょうか？ 図には有効期限を設けるべきであり、役に立たなくなった時点で廃棄してもまったく問題ないと主張する人もいるでしょう。図は、ホワイトボードに描かれたスケッチのように一時的なものでもかまいません。また、モデリングツールを使って作成された、形式的なものであってもかまいません。



同じ図を何度も繰り返し描いていることに気づいたら、それは何らかの形でその図を正式にまとめるべきだという兆候です。時間をかけてツールを使って作成し、プロジェクト用に 1 か所にまとめて保管しましょう。自分にとって役立つものであれば、チームメンバーにとっても役立つはずです。大きな図の場合はプロッタプリンターを活用しましょう。壁に貼った物理的なコピーは、チーム全体にとって役立つものです。場合によっては、厚紙に図を印刷することで、より実物としての存在感が増し、価値を感じてもらえることもあります。
もちろん、図を写真に撮って、ただ単にプロジェクトのドキュメントに追加するだけでもかまいません。ツールのために作業が遅れることのないようにしてください。

数多くのモデリングツールを利用できます。シンプルな図作成ツールから、ハイエンドで多機能なエンタープライズモデリングツールに至るまで多岐にわたります。世界中に分散した大規模なチームによる大規模なプロジェクトでは、特定のエンタープライズツールを標準化することで、大きなメリットが得られます。手作業でモデルを作成するためのツールもあれば、コードからモデルを生成できるツールもあります[†4]。後ほど、この章でツールについて詳しく見ていきますが、高機能なツールだからといって、必ずしもニーズに合った優れた図が作成できるとは限りません。

ツールだけではなく、図の種類も数多くあります。

†3　図が古すぎると、むしろ有害であるとさえ言えるでしょう。
†4　また、モデルからコードを生成することも可能です。主に安全性が極めて重要なシステムにおいて見られるものです。

4.2 どの図が必要か？

ソフトウェアモデリングの図に関しては、選択肢が山ほどあります。課題となるのは、ソフトウェア開発プロセス全体を通じて、どの図を作成すべきか、またいつ作成すべきかを把握することです。言い換えれば、どの図が必要なのでしょうか？ プロジェクトの現段階において、どの図が最も重要なのでしょうか？ もちろん、答えは「状況による」としか言いようがありません。どのようなことを実現しようとしていますか？ 問題はどれほど複雑ですか？ アプリケーションのリスクはどれほど高いですか？ これまでに同様のアプリを構築した経験はありますか、それともまったく新しい試みですか？ プロジェクトの予算はいくらですか？ このプロジェクトはビジネスにとって極めて重要ですか？

図には、形式的なものもあれば非形式的なものもあります。形式的なモデルでは、UML（**図4-1**）のような記法が用いられます。モデルを誰に提示するかを理解しておく必要があります。モデルが形式的であればあるほど、それを理解できるのは技術的な知識を持つ人に限られます。つまり、作成者の意図を理解できる人が限られることを意味します。例えば、Z 表記法をご存じでしょうか[†5]？ たとえ読者が Z 表記法を完全に理解していたとしても、組織内でどれほど広く知られているでしょうか？

手法が非形式的であればあるほど、伝えたい内容を理解してもらうのに、それほどの専門知識がなくてもよくなります。これにより、図を理解できる人の範囲が広がります。

統一モデリング言語

UML（Unified Modeling Language、統一モデリング言語）は、1980 年代から 1990 年代初頭にかけて開発されたさまざまな表記法を統一する手段として、1990 年代半ばに考案されました。1997 年にオブジェクト・マネジメント・グループ（OMG、Object Management Group）が標準として採用し、2005 年には ISO/IEC 19501 規格として発行されました。とはいえ、多くのソフトウェアエンジニアは、その代わりに（UML 要素を多用する）非形式的な図を好んで使用しています。なぜでしょうか？

UML 図における塗りつぶされた菱形と塗りつぶされていない菱形の違いがわかりますか[†6]？ 読者自身は、この図の微妙な違いを完全に理解しているかもしれません。しかし、相手はそうではないかもしれません。多くのソフトウェアエンジニアは、より幅広い層に理解してもらうために、UML よりも非形式的な表記法を選ぶ傾向にあります。効果的な手法は避けるべきではありません。シンプルなほうが、ほとんどの場合、より良いのです。

†5 Web で検索する手間を省くためにまとめておくと、**Z 表記法**は 1970 年代後半に開発された、数学に基づいた形式仕様記述言語です。

†6 これはカスケード削除に関係しているのですが、その細かい点については読者自身で調べてください。

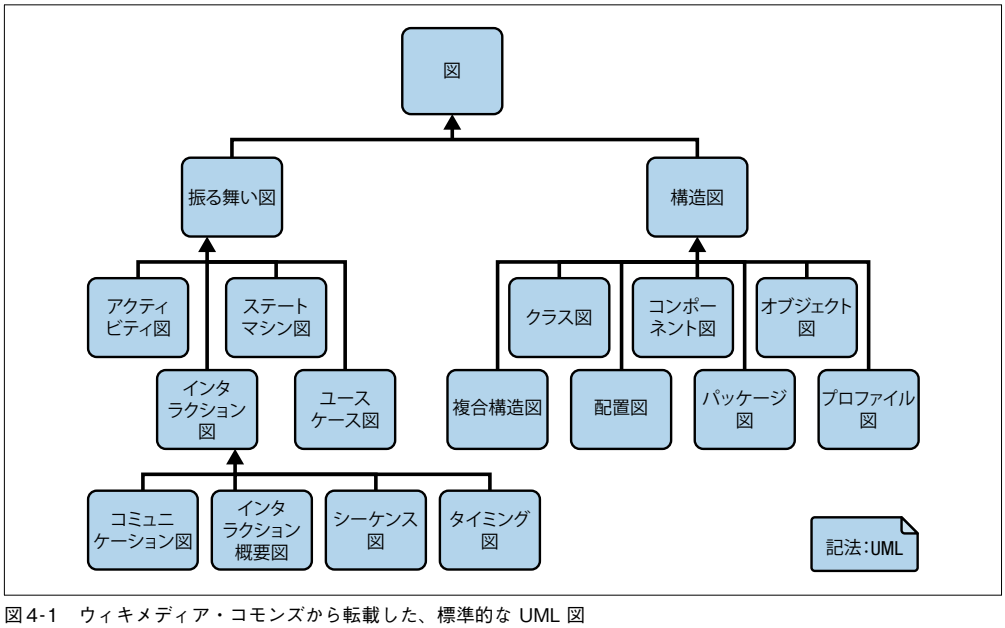


図4-1 ウィキメディア・コモンズから転載した、標準的な UML 図

4.2.1 コンテキスト図

コンテキスト図は、システム間、あるいはシステム内のコンポーネント間の境界を定義します。コンテキスト図では、環境だけでなく、エンティティ間のやり取りも示します。これらは論理データエンティティであり、多くの場合、データ量や頻度といった情報も示されます。コンテキスト図は非常に高レベルなものです。

コンテキスト図は、アーキテクト、意思決定者、プロジェクトマネージャー、プロダクトオーナーによって頻繁に活用されます。多くの場合、プロジェクトの初期段階で、境界を定義し、特定のアプリケーションに含まれるものと含まれないものについて関係者の認識を合わせるために用いられます。特定のシステムが何を指すのかについて概要を把握するのに役立ち、そのシステムの境界を明確に示してくれます。

コンテキスト図は、システムを最も俯瞰的に捉えた図と考えられます。例えば、**図4-2** は、自動運転車からのすべてのデータを整理・管理するシステムのコンテキスト図です。この車はデータサイエンティストが分析できるようにデータを送信するほか、車両の所有者に通知を送信します。

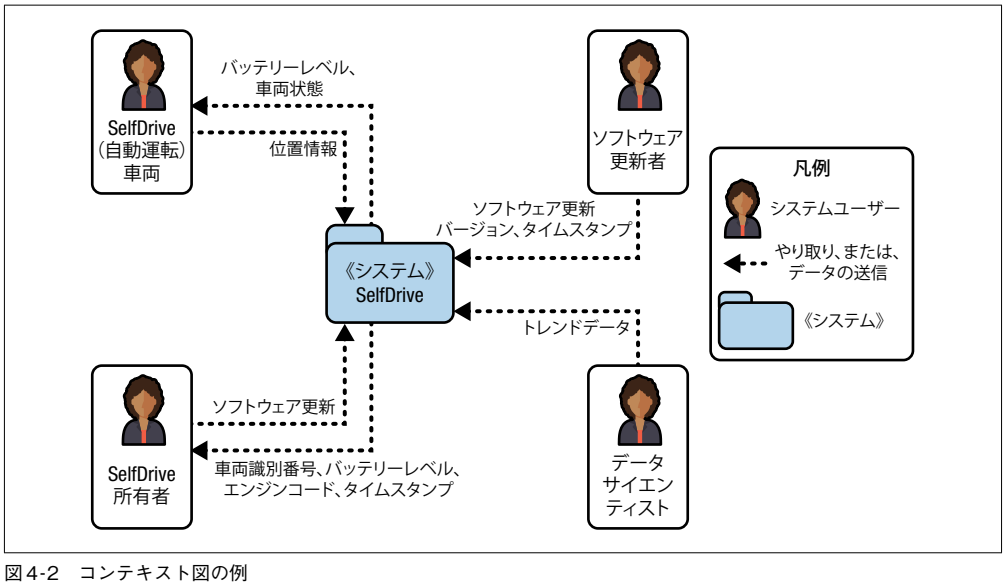


図4-2 コンテキスト図の例

4.2.2 コンポーネント図

コンポーネント図は、実行時のシステムの主要なコンポーネントを示します。コンポーネント図は、システムがどのように連携して動作するかを示し、その構造と挙動を明示します。また、情報の流れとインターフェースも表します。コンポーネント図は、開発者、アーキテクト、運用サポート担当者、DevOps エンジニアによって頻繁に利用されます。

コンポーネント図はプロジェクト全体を通じて活用されます。本来は想定されるやり取りを定義するために設計されたものです。このほか、より技術的な知識を持つ関係者に対して、プロジェクトの経緯を説明する際にも役立ちます。知識の引き継ぎやオンボーディングにおいても、非常に有用です。

例えば、**図4-3** に示すシステムは、利用可能な API のディレクトリとして機能します。2 つの異なるビューがあり、1 つはシステムの管理者向け、もう 1 つは API の利用者および作成者向けとなっています。認証認可のためのユーザーディレクトリがあり、その情報はデータベースに格納されています。

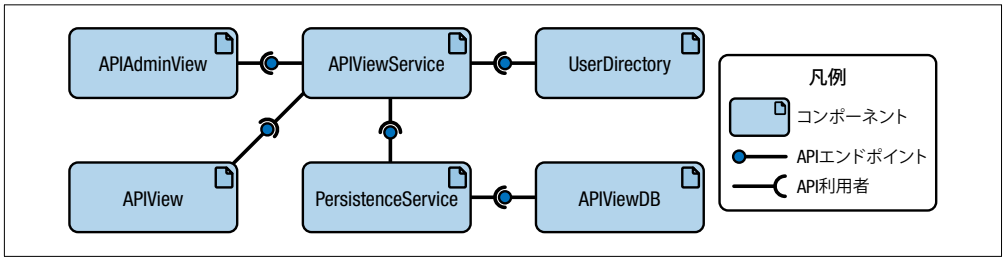


図4-3 コンポーネント図の例

4.2.3 クラス図

その名の通り、**クラス図**は、クラス同士がどのように関連しているかを示します。継承関係や合成関係を表すほか、多重度を含めることもできます[†7]。論理的なエンティティを表し、必要に応じてメソッドを含めることも可能です。

クラス図は、一度作成するとすぐにコードの更新に追いつかなくなるため、多くの場合、必要に応じて現在のコードから抽出されます。大規模システムでは膨大な量になり得るため、より利用しやすいように、論理的な境界やドメインの境界ごとに分割することがあります。

クラス図を利用するのは、その性質上、技術的な知識を持つ人々、通常は他のエンジニア、アーキテクト、開発者、運用担当者などです。クラス図は、プロジェクトの初期段階で全体像を理解してもらうために作成されることが多く、通常はプロジェクトの進行に合わせて改版されていきます。クラス図は、新規の開発者にとっても既存の開発者にとっても、システムの全容を把握するうえで非常に役立ちます。例えば、**図4-4**のクラス図では、`Person`と`NamedEntity`が`BaseEntity`のサブクラスであることが示されています。

†7 **多重度**とは、関連に結びつけられるインスタンスの数を意味します。

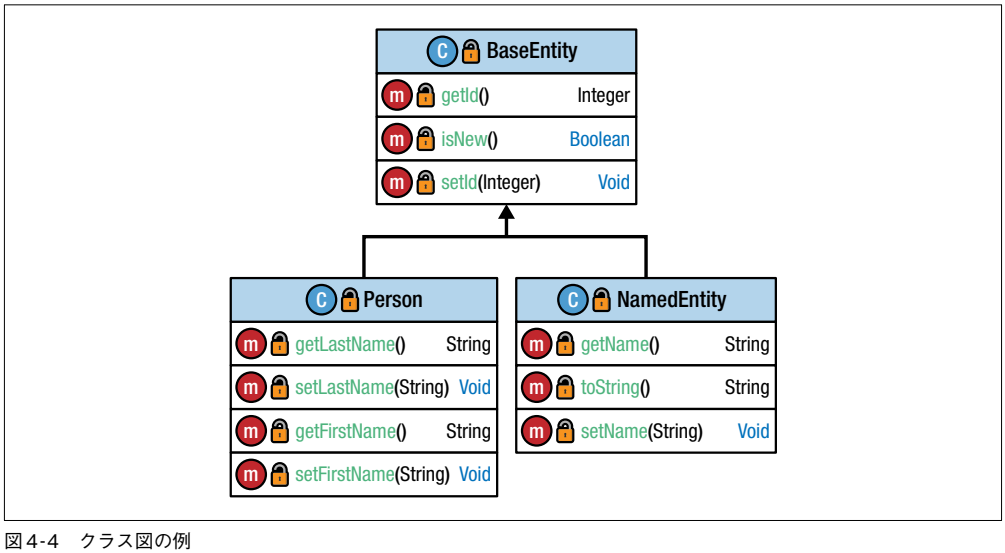


図4-4　クラス図の例

4.2.4　シーケンス図

シーケンス図は相互のやり取りの順序を示します。ただし、すべてのエンティティやすべてのやり取りを網羅することはほとんどありません。通常、特定のフローに関与するエンティティのみが含まれます。ほとんどのシステムでは、相互のやり取りはほぼ無限にあるため、すべてのエンティティやすべてのフローをドキュメント化することは現実的ではありません。シーケンス図は、特に注意すべきやり取りや、アーキテクチャ上重要なやり取りをドキュメント化するため、あるいは特定のパターンやライブラリの標準的な使い方を示す手段として頻繁に利用されます。

シーケンス図には、パラメータや戻り値の型を含めた操作が記載されることがあります。これは開発者にとって役立つ場合がありますが、コードとの整合性がすぐに失われてしまうおそれがあることも理解しておくべきです。既存のコードからシーケンス図を抽出でき、新しいコードベースに取り組む際に役立ちます。こうした図は通常、開発者やソリューションアーキテクトによって作成されます。この図を利用するのは主に技術者であり、開発者、アーキテクト、DevOps 担当者が含まれます。QA 担当者も、何をテストすべきかを理解するためにシーケンス図を活用することがあります。

シーケンス図はプロジェクト全体を通じて活用されます。プロジェクトの初期段階ではシステムのパターン設計に用いられます。知識の引き継ぎやオンボーディングにおいても非常に有用です。例えば、**図4-5** のシーケンス図では、API サービスに接続した後、検索処理においてユーザーがその機能を実行する権限を持っているかどうかが判定されます。権限がある場合には、そのリクエストがデータストアへと流れ、その後戻ってくる様子が示されています。

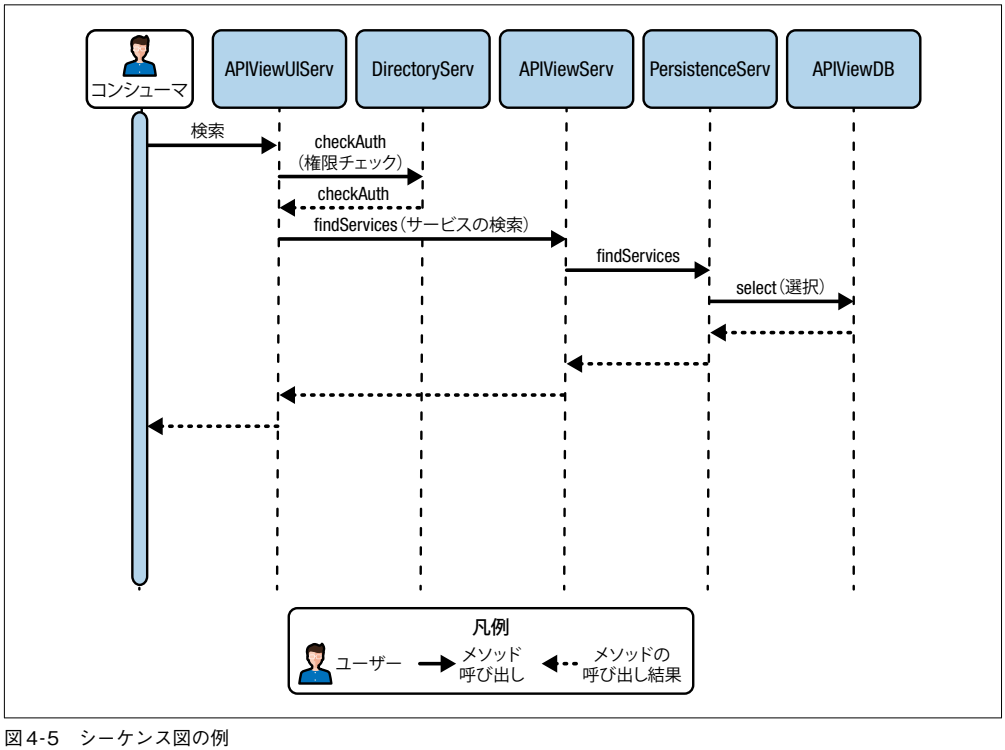


図4-5 シーケンス図の例

4.2.5 配置図

配置図は、システムの実行時の状態を示します。物理的なハードウェアノードだけでなく、それらの上で実行されているソフトウェアも表します。多くの場合、ハードウェアの接続関係や、プロトコル、多重度も含まれます。配置図は、アーキテクト、開発者、DevOps、運用サポートチームをはじめ、インフラストラクチャ、セキュリティ、ミドルウェアチームの担当者など、より技術的な知識を持つ人々が利用します。

配置図には論理図と物理図があり、それぞれ適切に記述する必要があります。例えば、論理配置図ではクラスタを単一のエンティティとして表現します。一方、物理配置図では、そのクラスタ内で特定の時点で稼働しているロードバランサやサーバーの数を示します。

配置図は、プロジェクトの初期段階で品質特性を検証するために使用されます。24 時間 365 日のサポートが必要なシステムは、稼働時間の要件がそれほど厳しくないシステムとは、まったく異なる配置になります。配置図は、アプリケーションがスケーリングの要件を満たすかどうかを判断したり、事業継続性を検証したりするのに役立ちます。また、より費用対効果の高い配置方法を見つけるのにも有用です。

多くの組織では、配置用の標準テンプレートを用意しています。ほとんどの企業では、標準的なクラウド環境やオンプレミス環境の設計が確立しています。多くの点で、配置図はレゴブロックの

ように考えられます。読者の会社でも、使用可能なツールやテクノロジーの定義済みセットがあり、それらを組み合わせていくことになるでしょう。多くの組織では、典型的なアプリケーションを記述するために標準化されたリファレンスアーキテクチャを用意しています。そして、配置の選択肢には一定の制約を設けています。無制限に何を使ってもよいという企業はほとんどありません。もし読者の会社が Amazon Web Services（AWS）を標準化している場合、Azure 固有の要素で埋め尽くされた図を作成することはないでしょう。また、PostgreSQL が社内で承認されたリレーショナルデータベースであるならば、MariaDB を使用した配置をモデル化することは適切ではありません。配置図の例については、**図4-6** を参照してください。

この例では、自動運転車を題材に、アプリケーションのどの部分がセキュアゾーン内に配置され、どの部分がファイアウォール外にあるかを示しています。また、これらの図では、コンポーネントの具体的なバージョンを示すこともできます。

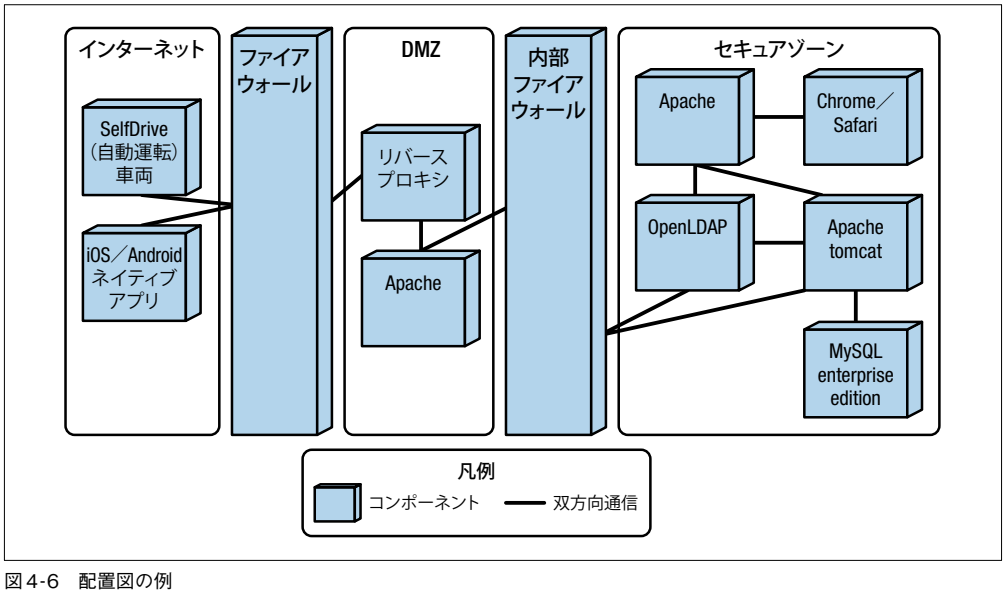


図4-6 配置図の例

セキュリティ図と呼ばれる、配置図の特殊な形態を目にすることがあるかもしれません。このより詳細なモデルは、アプリケーションのセキュリティ機構を記述するものです。セキュリティ図には多くの場合、プロトコルが含まれており、システムの配置やテクノロジーを可視化するのに活用できます。セキュリティ図は、開発者、アーキテクト、セキュリティ専門家など、より技術的な知識を持つ人々が利用します。

セキュリティ図は、プロジェクト全体を通じて活用されます。セキュリティ図は、パターンを定義するだけでなく、そのソリューションがプロジェクトのセキュリティ要件を満たしていることを確認するうえでも役立ちます。システムを運用する際、個人識別可能情報（PII、Personally

Identifiable Information）に加え、システムに適用される可能性のある規制や法律についても考慮することが重要です。例えば、**図4-7** をご覧ください。

この例でも、自動運転車を題材とし、アプリケーションのどの部分がセキュアゾーン内に配置され、どの部分がファイアウォールの外側にあり、その結果としてどう扱われるべきかについて示しています。HTTPS や LDAP といったプロトコルが追加されている点に注目してください。

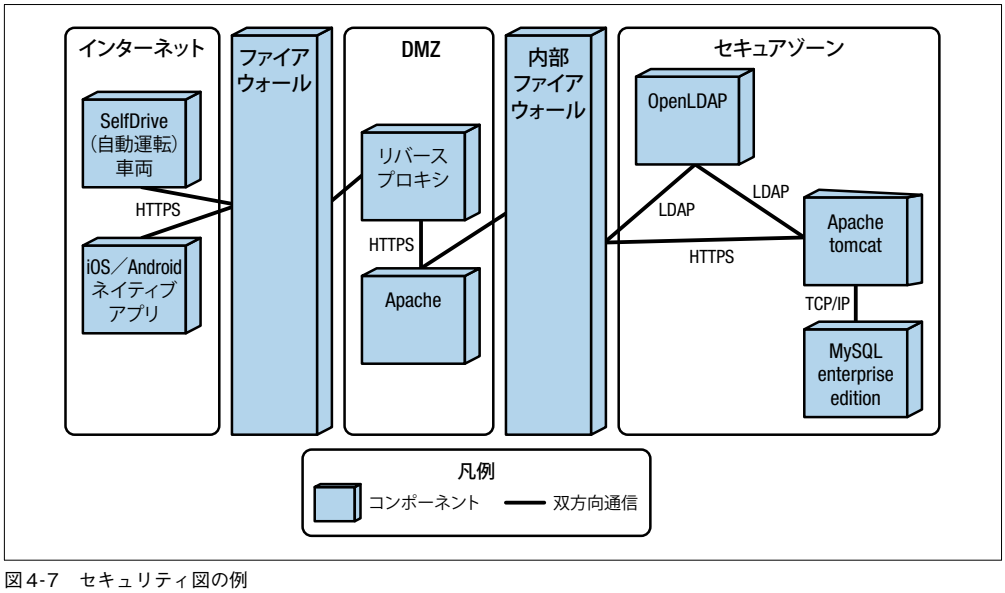


図4-7 セキュリティ図の例

4.2.6 データモデル

データモデルは、データエンティティだけでなく、それらの関係性も示します。データモデルは、より概念的なレベルから論理的なレベル、さらにはデータストアの物理的なレイアウトに至るまで、さまざまな粒度で定義されます。基本的には、高レベルな概念から具体的な実装へと段階的に進んでいきます。概念モデルは非常に高レベルであり、正規化はされていません[†8]。論理モデルはビジネス用語を表し、多くの場合、正規化されます。物理データモデルは、データ型を含む実装の詳細を表します。物理データモデルも、詳細度の低いものから高いものへと段階的に進んでいきます。

データモデルを利用するのは、顧客から情報アーキテクト、データベース管理者、さらにはソフトウェアアーキテクト、開発者、サポート担当者まで多岐にわたります。多くの場合、データモデルは、対象領域を可視化するために、プロジェクトの初期段階で作成されます。ただし、プロジェクトの全期間を通じて改版されることも少なくありません。例えば、**図4-8** には、API システム

†8 **正規化**とは、冗長性を減らし、データの整合性を高めるプロセスのことです。

におけるデータエンティティの一部が示されています。この図からわかることは、特定のテクノロジーには１つから複数のバージョンがあり、ベンダーには１つから複数のテクノロジーやプラットフォームがあります。

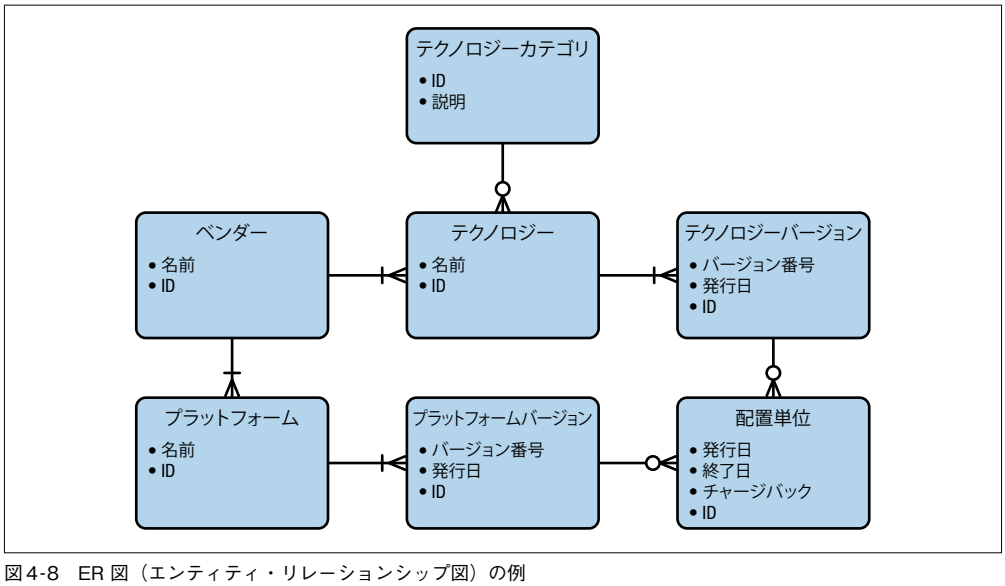


図4-8　ER図（エンティティ・リレーションシップ図）の例

4.2.7　その他の図

プロジェクトのライフサイクル中に、他にもさまざまな図表のような成果物を作成することになるでしょう。繰り返しになりますが、ほとんどの場合、その作成プロセスは、対象領域や特定の問題、あるいは複雑なやり取りを理解するうえで非常に有益なものとなります。問題に対する認識が人によって異なっていることに気づくこともよくあるでしょう。順不同ですが、以下に、プロジェクトで活用できるその他の図をいくつか紹介します。

イベントストーミング（https://oreil.ly/6kUM4）

ビジネス領域を探求するためのワークショップ形式の手法です。対面でもオンラインでも実施可能です。ビジネスおよび技術のステークホルダーが、対象領域のイベントや、そのイベントを発生させるコマンド、そしてそのコマンドを実行するアクターについて概要を整理します。色の異なる付箋紙を用いて、さまざまなカテゴリを表します。イベントストーミングは、非常に柔軟で負荷の少ないプロセスであり、さまざまな場面で活用できます。

バリューストリームマッピング（https://oreil.ly/sZIgA）

システムの現状を分析し、将来の最終状態をモデル化する手法です。対面でもオンラインでも実施可能です。バリューストリームマッピングを行うことで、プロセス内のどこに無駄や

ボトルネックがあるかを把握できます。

ユーザーストーリーマッピング（https://oreil.ly/hb59p）
付箋とラフスケッチを用いて、ソフトウェアシステムのユーザーエクスペリエンスを整理します。対面またはオンラインでも実施でき、ビジネスと技術のステークホルダーが協力してシステムを検討します。大まかなタスクはアクティビティとして表現され、その下に必要な手順や詳細が示されます。

ディザスタリカバリモデル（災害復旧モデル）（https://oreil.ly/e8fYy）
システムを詳細に把握し、プロジェクトの事業継続要件を定義するのに役立ちます。システム停止によって影響を受ける人数だけでなく、ダウンタイムによるコストも考慮することが重要です。これらのモデルには、実行中のエンティティと、それらが動作しているハードウェアが示されます。このモデルは、プロジェクトライフサイクル全体を通じて活用されます。本番環境への完全なリリースが近づくにつれて、その重要性は高まります。

4.3 モデル作成のベストプラクティス

前述したとおり、モデルは技術的な意図を伝えるうえで非常に有用です。簡易的な図は比較的短時間で作成できますが、これは利点であると同時に難点にもなり得ます！ 図が管理不能な状態になってしまうことがあるからです。以下に、図を適切に管理するのに役立つ方法をいくつか紹介します。

4.3.1 シンプルにする

図は、非常に雑然としたものになりがちです。見る者を圧倒し困惑させることを目的として、ただ「衝撃と畏怖」を与えるだけの図表を作成してしまうことは容易です。中には、文字がぎっしり詰めこまれており、自分の目が正常かどうか疑ってしまうような図もあります。また、注意をそらすような視覚要素があり、線が至るところに走り、あらゆる形や大きさの要素が散りばめられている図もあります。こうした図は、あまりにも多くの情報を詰め込みすぎており、結局のところ、混乱以外の何ものも伝えられないのです。

このような図はよく見かけますが、役に立つことはめったにありません。繰り返しになりますが、**図の目的は、他者に何かを伝える手助けをすることです。もし相手が読者の作成した図を理解できなければ、それは無意味です。**アプリケーションが成長するにつれ、エンティティの数も増え、図の数も増えていきます。すべてを１つの図にまとめようとするべきでしょうか？ このような図は、アプリケーションの全体像を把握するには役立つかもしれませんが、内容を理解するのが非常に難しいものになります。もし誰かがそれを印刷したとして、読みやすいものでしょうか[†9]？

すべての要素をたった１つの図に盛り込むべきという決まりはありません。多くの場合、図を分

†9 嘘だと思うかもしれませんが、印刷版を欲しがる人は必ず現れます。

割して、個々の図をわかりやすくしたほうがよいでしょう。ある図から別の図を参照するのは簡単です。コンポーネント図を例に挙げてみましょう。アプリケーションのすべてのコンポーネントを1つの図に示そうとすると、その図はすぐに使い物にならなくなってしまいます。一般的には、論理的な区切りで図を分割し、ある図から別の図を参照するようにします。すべてのコンポーネントを一覧できる概要図を用意したい場合もあるでしょうが、それがシステムの唯一のモデルとなることはめったにないはずです。

4.3.2 図の利用者を把握する

コードを書く場合と同じように（3章参照）、優れたソフトウェアエンジニアは、自分の図を見る人のことを第一に考えます。開発者にとっては完璧な、高度に技術的な図であっても、技術に詳しくない人に見せる場合には、あまり効果的ではないかもしれません。

図にはコンテキストが必要

Nateです。何年も前、私はある新しいWebアプリケーションのリード開発者を務めていました。このプロジェクトでの仕事の一環として作成したクラス図を、私はとても誇りに思っていました。複雑ではありましたが、交差する線が一本もなかったからです！ 私はアーキテクトとレビューの時間を設け、自分の成果を確認してもらうことにしました。誇らしげに図をテーブル越しに彼に手渡して確認してもらおうとしたところ、彼は肩をすくめて、そのまま返してきました。正直、少しがっかりしました。何しろ、交差する線などなかったのですから。努力に対してご褒美を期待していたわけではありませんが、せめて少しくらいは褒められるかもと思っていました[†10]。

当時、私が理解していなかったのは、担当のアーキテクトが、私のクラス図を理解するのに十分なコンテキストを持っていなかったということです。彼が私の職人技を評価してくれることを期待していましたが、彼には、一見してわからないミスを特定するすべがなかったのでした。彼は、そのプロジェクトにおいて私ほどビジネス領域を深く理解していなかったため、それについて効果的なコメントをすることができませんでした。彼は、私の作業が正しいと信じるしかなかったのです。これは、こうした図が役に立たないと言っているわけではありません。ただ、その限界を理解しておく必要があるということです。

4.3.3 色の選択には注意する

図の中で特定の要素を区別するために、色を使うことがよくあります。例えば、新規、変更済み、または変更されていないコンポーネントには、それぞれ異なる色が割り当てられる場合があります。色を使用する場合は、必ず色の割り当てを説明する凡例を記載してください。チーム内で「新

†10 ええと、もしかしたらクッキーを期待していたのかもしれません。

規」を表す色として緑色を使っているからといって、他のチームも同じようにしているとは限らないからです。

たとえ読者自身はここ数年何も印刷していなくても、組織内の誰かが図を印刷するのを想定すべきです。図はモニター上では色鮮やかで素晴らしく見えるかもしれませんが、グレースケールで印刷されると、その意味の多くが失われてしまうおそれがあります。また、閲覧者の中には色覚異常の方がいることも想定しておくべきです。つまり、色に頼らない代替案を用意しておくべきです。

4.3.4 標準とテンプレートを策定する

組織内で一連の標準を定めると、非常に役立つことがあります。日常的に使用する図のセットと、使用する色を決めておけば、プロジェクト間で一貫性を保てます。ただし、あまり細かく決めすぎないようにしましょう。プロジェクトの要件に応じて、図を追加したり削除したりできる余地を残しておくことが大切です。

> **これは本当に必要なのか？**
>
> Nate です。数年前、私はプロジェクトのドキュメント作成に関して非常に厳格な方針をとっている組織で働いていました。アーキテクトとして、私は数多くの図を作成することが求められていました。しかし、必要のないものはすぐに不要と判断し作成しませんでした。例えば、当時のテンプレートではユースケース図を含めることが求められていました。しかし、このプロジェクトは、ユースケースではなくストーリーを使用するアジャイルプロジェクトでしたので、図を作成するという目的のためだけに図を作成しているのは奇妙に感じたのです。そこで、私は作成しないことにしました。テンプレートにこだわらず、柔軟に考えてください。

4.4 ツール

おそらく、読者の組織では、すでに 1 つや 2 つのモデリングツールが利用できることでしょう。しかし、面倒な調達作業をしなくても、十分に実用的な図を作成することは可能です。

もし何も導入されていないのであれば、以下の選択肢を検討してください（以下は一例です）。

- OmniGraffle は、Omni Group が提供する macOS および iOS 向けの図作成ツールです。標準的なソフトウェア図を素早く作成できる、数多くのステンシルが用意されています。WYSIWYG 方式のドラッグ＆ドロップ環境であるため、比較的簡単に使いこなせるようになります。
- Microsoft Visio は、Windows 用の図作成ツールです。Visio を使えば、フローチャートやマインドマップから UML 図に至るまで、さまざまな図を作成できます。OmniGraffle と同じように、WYSIWYG のドラッグ＆ドロップ環境を採用しているため、非常に簡単に使

いこなせます。職場のノートパソコンに Microsoft Office スイートがインストールされている場合、おそらく Visio もすでにインストールされているでしょう。ソフトウェアエンジニアリングや、Azure、AWS、Google Cloud などのパブリッククラウドプロバイダーに特化したステンシルをダウンロードできます。

- Mural、Miro、Lucidchart は、分散型コラボレーションホワイトボードとして機能する、ブラウザベースのツールです。これらを使えば、図をいくつでも作成できます。複数のユーザーが同時に同じキャンバス上で作業できるため、共同作業に最適です。
- Mermaid は、JavaScript ベースの図作成・チャート作成ツールです。Markdown に似たテキスト定義を用いて、図を動的に作成・編集できます。特に、ドキュメントや GitHub Wiki など、Markdown に対応したプラットフォームに対して、図を埋め込む際に広く利用されています。Mermaid は、シンプルなテキスト構文を使用して、フローチャート、シーケンス図、クラス図、状態遷移図などを作成できます。
- Structurizr は「図をコードとして扱う」モデルを採用しており、Structurizr DSL（https://oreil.ly/nZ_oS）を使用して任意の数の図を作成できます。C4 モデルをサポートするように設計されていますが、C4 モデルを可視化する以外にも、一般的なクラウドベースのアーキテクチャ向けのテーマも用意されています。
- PlantUML は、一般的な UML 図を多数サポートしており、コマンドラインまたは GUI インターフェースを介してテキストファイルからモデルを生成できます。
- お使いの IDE。おそらく、お使いの IDE は、コードベースからモデルを抽出するだけでなく、モデルを直接作成することもネイティブにサポートしているはずです。プラグインを 1 つや 2 つ追加する必要があるかもしれませんが、今日のエディタであれば、多くの場合、実用的なモデリングツールが搭載されています。
- PowerPoint、Keynote、Google Slides などのプレゼンテーションソフトも、多くの企業で広く導入されていることから、非常に一般的に使用されています。これらはモデリングツールとして設計されたものではありませんが、ソフトウェアエンジニアリングに特化したステンシルを使用することで、標準で用意されている基本的な図形を補完できます。また、これらのツールは共同作業をサポートしており、複数の人が一緒に図を作成できます。
- AI ツールはますます普及しており、その多くはコードベースから図を抽出できます。ただし、誤った情報を生成するおそれは常にありますので、出力内容は慎重に確認する必要があります。

迷ったときは、誰もが利用でき、かつ／または使用ライセンスを持っているツールを選んでください。

モデルからのコード生成

何年も前、一部のモデリングツールでは、図からコードを生成できました。プロジェクトの

立ち上げ時には役立つこともあったかもしれませんが、こうした手法は、今やソフトウェアの歴史の片隅に追いやられてしまいました[†11]。なぜなら、これらのツールには重大な問題があったからです。開発者がツールのルールを厳密に守らない限り、生成されたファイルに追加したコードは、次にモデルからコードを生成する際に削除されてしまうことが多かったのです。

一部のツールは、少なくとも理論上は双方向で機能していました。つまり、コードを更新するとモデルが変更され、その逆も同様でした。しかし残念ながら、このようなツールは期待したような大幅な生産性向上にはつながりませんでした。

どのようなツールを使用する場合でも、図のバージョン管理を忘れないようにしてください。「コードとしての図」のほうがバージョン管理には適していますが、最新のバージョン管理ツールであれば、図形もかなりうまく扱えます。

複雑なツールには強力な機能が備わっていることが多いのですが、その反面、価格が高額であるだけでなく、習得に時間がかかるという欠点もあります。使いつづけることで後者の問題は解消されますが、こうしたツールは、必要以上に物事を複雑化してしまうおそれがあります。より簡単かつ迅速にメッセージを伝えられるのであれば、悩まずにシンプルなツールを使うようにしてください。

図を通じて伝えようとしているメッセージこそが、最も重要な部分であることを理解してください。ツールは、あくまでその意図を表現するための補助手段に過ぎないのです。気に入ったツールがあり、それが生産性の向上に役立つのであれば、ぜひ活用すべきです。しかし、ツールに縛られてはいけません。紙と鉛筆も、ホワイトボードと同様に優れたモデリングツールとなります。迷ったときは、シンプルに保つようにしましょう。

図は、実際に動作するコードの代わりにはならないということを忘れないでください。図は非常に役立つものですが、コードとの間に乖離が生じることもあります。一部の図は、必要に応じて、あるいはビルドパイプラインの一環として、コード自体から抽出できます。これにより整合性を失うという問題を軽減できます。しかし、常にコードこそが最終的に信頼できる情報源であると考えるべきです。

4.5 まとめ

モデルや図は、ソフトウェアエンジニアにとって欠かせないツールの一つです。コードこそが最終的に信頼できる情報源ですが、図を用いることで、技術系／非技術系のステークホルダーに対して重要な概念を効果的に伝えられます。課題となるのは、特定の方法をいつ使用し、いつ使用しないのかを判断することです。モデリングツールは、それが適切である場面で活用してください。

†11 今日の開発者は、バイブコーディングするだけですよね？

4.6 実践に移す

よく知っているアプリケーションの中から、興味深い処理の流れを選んでみてください。あるいは、少し挑戦したいなら、使い慣れたオープンソースのライブラリでもかまいません。シーケンス図を1つか2つ描いてみましょう。そして、同僚にフィードバックを求めてください。

アプリケーションからクラス図を抽出してみましょう。クラスの関係性について、どのような点が意外でしたか？ その結果得られたモデルから、アプリケーションについてどのような気づきを得られましたか？

担当のアーキテクトに、システム用に作成した図を共有してもらうよう依頼してください。時間をかけて、それらの図に慣れていきましょう。もし疑問点があれば、遠慮なく質問してください！ もし自分が同じ図を作成するよう求められたら、どのような点を変えて作成するでしょうか？ 何か不明な点があれば、アーキテクトと話し合ってください。プロジェクトによっては、CI/CDパイプラインの一環として図を生成しているものもあります。もし読者のプロジェクトでそうしていないのであれば、導入を検討する価値があるでしょう。

4.7 関連リソース

- The C4 model for visualizing software architecture（https://c4model.com/）
- 『User Story Mapping』Jeff Patton 著（O'Reilly、2014年）
  和書『ユーザーストーリーマッピング』（オライリー・ジャパン、2015年）
- 『Communication Patterns』Jacqui Read 著（O'Reilly、2023年）
  和書『開発者とアーキテクトのためのコミュニケーションガイド』（オライリー・ジャパン、2025年）
- 『Creating Software with Modern Diagramming Techniques』Ashley Peacock 著（Pragmatic Programmers、2023年）
- 『UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd Edition』Martin Fowler 著（Addison-Wesley Professional、2003年）
  和書『UMLモデリングのエッセンス 第3版』（翔泳社、2005年）

5章
自動テスト

品質とは、一度きりの行為ではありません。習慣なのです。
— 哲学者 Will Durant、Aristotle の思想を踏まえて

新年を迎えると、多くの人がより健康的な生活を送り、それを維持するという目標を立てます。そうした目標は、健康的な食事を一度とったり、ジムに一度行ったりしただけでは達成できません。重要なのは、**一貫して**健康的な選択をしつづけることです。同様に、高品質で保守性の高いソフトウェアを生み出すことは、コードの記述、レビュー、テストにおいて、良い習慣を**一貫して**実践した結果として得られるものです。健康的なライフスタイルを維持することと同様に、これは容易なことではなく、言い訳をしてもただ状況を悪化させるだけです。

しかし、一貫してこのようなことを行うことは決して簡単ではありません。多くの開発者は、自分の判断が正しいのか、あるいは問題解決のアプローチが疑問視されるのではないかと、自信を失い、悩んでいます。高品質なソフトウェアを書くことは、継続的な実践を必要とする技術です。時間が経つにつれて、テストへの取り組みは、チームメンバーや将来の自分にとって有益で効果的な安全装置となるでしょう。本章では、自動テストのメリット、さまざまな種類のテスト、そしてそれらの書き方について学びます。

5.1　自動テストのメリット

既存のコードを検証するために追加のコードを書くことに対して、懐疑的になるのは自然なことです。また、その労力に見合わないと感じることもあるでしょう。単なる流行や、履歴書に書くための活動ではないかと疑問に思うかもしれません。しかし、自動テストはそれ以上の価値があります。自動テストは身につけておくべき貴重なスキルであり、コードベースへの重要な投資なのです。

以降の節では、自動テストを作成することの具体的なメリットについて解説します。自動テストを書くことが、どのようにコードの品質を向上させ、仕事に対する自信を高め、最終的にはより効率的なエンジニアへと成長させるかについて、詳しく見ていきます。これらのメリットを理解する

ことで、テストへの取り組みの方向性が明確になり、自動テストに時間を割くことが、自身のキャリア形成においていかに価値のあることかが実感できるでしょう。

5.1.1 ドキュメントとしての役割

既存プロジェクトに参加するのは、気が重くなることもあるでしょう（既存プロジェクトに参加する際の対応方法については6章を参照してください）。プロジェクトにドキュメントがあるというのは、大変恵まれている環境です[†1]。運が良ければ、作業を始める前にリードエンジニアやドメインエキスパートと話し合いを行い、全体像を把握できるかもしれません。しかし、常にそのような環境とは限りません。ドキュメントが不十分だったり、あるいはまったく存在しなかったりすることもあります。そんなとき、頼りになるのは、しっかりと記述されたテストです。

ある特定の種類のクレジットカードに関して、決済処理の問題を解決するよう依頼された状況を想像してみてください。適切なドキュメントやテストがなければ、初めて触れる（おそらく複雑な）コードを丹念に調べたり、同僚にコードについて教えてもらったりしなければならないでしょう。

ここで、決済に関する包括的なテストスイートを見つけたとします。そこには、次のようなわかりやすいテスト名を持つ `PaymentProcesserTests` クラスがあるはずです。

- `shouldValidateValidCreditCardNumber`
- `shouldProcessCreditCardTransaction`
- `shouldProcessOrangePay`

これらのテストは、モジュールの機能に関する情報を提供し、その機能に関連しているコード部分を示してくれます。最新のIDEを使えば、テスト対象となる特定サービスへ容易に移動できます（効果的にコードを読む方法については2章を参照してください）。テストが失敗した場合は、コードベース内の原因となりそうな箇所を特定できます。

例えば、`shouldFailWhenCreditCardTypeIsOrangePay` という名前のテストが見つかるかもしれません。これにより、Orange Payがサポートされていないことが判明します。この情報は、問題を迅速に特定・修正し、フロントエンドを更新し、ドキュメントを改版するのに役立ちます。

この例からわかるように、テストはドキュメントとしての役割を果たし、コードベースをより早く理解するのに役立ちます。テストスイートの改善に注力することで、コード品質を向上させ、本番環境での問題を減らし、プロジェクトへの貢献を早められます。

5.1.2 保守性の向上

良質で保守性の高いコードを書くことは、通常、習得するのに何年もかかるスキルです（詳細については3章を参照してください）。テストを先に書くことは、話す前に考えるようなものです。考えを整理し、構成を組み立てるのに役立ちます。初心者の主な目標は、単にコードを動作させる

†1 そのドキュメントが最新かつ正確であることはさらにまれなことです。

ことであることが多いでしょう。もちろんそれも重要ですが、テストを書くことで、改善すべき点をより早く見つけられるようになります。

次の BlogPostController の例を見てみましょう（説明のため、コードは割愛しています）。

```
public class BlogPostController {
   public void publish(Post post) {
       // ブログ投稿をデータベースに保存する（isPublished を true に切り替える）
       // ブログ投稿が公開されたことをログに残す
       // 新しい投稿について購読者にメールをする
   }
}
```

一見すると、これはまったく問題がないように思えるかもしれません。しかし、このメソッドのテストを書こうとすると、すぐにその処理範囲が広すぎることに気づくでしょう。

- データベースとの通信
- 情報のロギング
- メールの送信

このコードは、「クラスが変更される理由は、1つだけであるべきだ」と定めた単一責任の原則に反しています。

このコードをどのようにテストするかを考えることで、自然と問題点が明らかになります。その後、ブログ公開プロセスの各部分を個別のクラスにリファクタリングすることで、テストや保守が容易になります。

テストを書くことは、機能性を検証するだけでなく、より良いコード設計へと導いてくれます。潜在的な問題が顕在化する前に発見できるようになり、結果として保守性が高く、再利用しやすいソフトウェアが実現します。

5.1.3 自信を高める

自動テストには多くのメリットがありますが、その中でも特に際立つのが「自由にコードを書けるという自信」です。人生のさまざまな場面でも、自信を持って臨むことで、プレッシャーに対処し、私生活や仕事上の問題に取り組めるようになります。ソフトウェア開発は、絶えずコードを書き、実験し、リファクタリングを繰り返すという、本質的に同じことを繰り返す作業です。テストがなければ、システムを壊してしまうような変更を避けるために、慎重になりすぎてしまうかもしれません。新しい手法や独創的な解決策を試すことを避けてしまうことでしょう。しかし、堅牢なテストスイートがあれば、セーフティネットが確保されます。このように自由に取り組める環境は、コーディングへの取り組み方を根本から変えてくれます。テストがあらゆる問題をしっかり捕まえてくれると確信していれば、自信を持って変更を加え、新しい解決策を試み、リファクタリングを行えるのです。

コードの変更やリファクタリングは、ストレスのたまる作業になりがちです。アプリケーションを完全に理解できていない場合もあり、思わぬ副作用が生じることも予測しがたいものです。テストなしで作業を行うことは、ロープなしで山に登るようなものです。すべてが順調に進めば、アドレナリンが湧き出るような高揚感を味わえるかもしれませんが、たった一つのミスが致命的な結果を招くおそれがあります。

テストは、コードベースを扱う際のセーフティネットとしての役割を担い、自信を持って行動できるようにしてくれます。本来の目的は機能を動作させることかもしれません。しかし、変更がシステム全体にどのような影響を与えるかについても考慮する必要があります。包括的なテストがなければ、一見正常に動作しているように見えても、本番環境で問題が発生するおそれがあります。

堅牢なテストスイートがあれば、チームは以下のことができるようになります。

- 自信を持ってコードをリファクタリングする
- 変更によって既存の機能が損なわれないようにする
- 本番環境に反映される前に潜在的な問題を特定する

包括的な一連のテストを作成・保守することで、以下のことが可能になります。

- システムの振る舞いをより深く理解する
- 修正が簡単でコストも低い開発プロセスの早い段階でバグを発見する
- リグレッションが発生するおそれを低減する
- 自動テストを通じて、機能が仕様を満たしていることを確認し、コードの品質を維持する

このアプローチは、コードの品質を向上させるだけでなく、開発者としての自信も高めてくれます。テストが潜在的な問題を確実に検出してくれるという確信を持って、変更を加えたり、機能を追加したり、リファクタリングを行ったりできます。

コーディングに対する自信は、知識、経験、そしてツールの組み合わせから生まれます。自動テストは、エンジニアとしてのキャリアを通じて、自信を築き、維持するための強力なパートナーとなります。

5.1.4 一貫性と再現性につながる

かつて、開発者は多くの場合、膨大な時間をかけた手動テストに頼っていました。新機能に負荷をかけるための手順リストを作成し、その手順リストに従ってテストを行っていたのです。たいていの場合、手順通りに進めたとしても、手作業によるアプローチはミスが起きやすく、再現性がなく、非常に労力と時間を要するものでした。人間には、同じ作業をまったく同じ方法で繰り返し行うことが本質的に不可能なのです。

自動テストは、一貫性があり再現性のある結果をもたらします。人間とは異なり、テストスクリプトは毎回まったく同じ手順を、ミスや抜け漏れなく実行します。これらのテストは、ボタンをクリックするだけで、あるいはビルド中に、最小限の手間で迅速に実行されます。特に、リグ

レッションテストは、コードが変更されても既存の機能が引きつづき正常に動作することを確認します。

包括的なテストスイートを活用すれば、環境にかかわらず、テストを一貫して繰り返し実行できます。自動テストは頻繁に実行できるため[†2]、リグレッションテストを効果的に実施できます。これらのテストにより、以前に修正されたバグがなぜか再発することを防ぐだけでなく、コードベースの変更によって生じた新たなバグも特定できます。新機能のテストを行う際は、コストのかかる手動テストを省き、自動テストに頼るようにしましょう。

5.2 自動テストの種類

テストを行うべき理由がわかったところで、次に何をテストすべきか（そして、何をテストすべきでないか！）について見ていきましょう。**自動テスト**には、ユニットテストだけでなく、UI テスト、エンドツーエンドテスト、統合テストなども含む、幅広い種類のテストがあります。これらのテストの種類は、**図5-1** に示すようなピラミッド構造として捉えるのが一般的です。

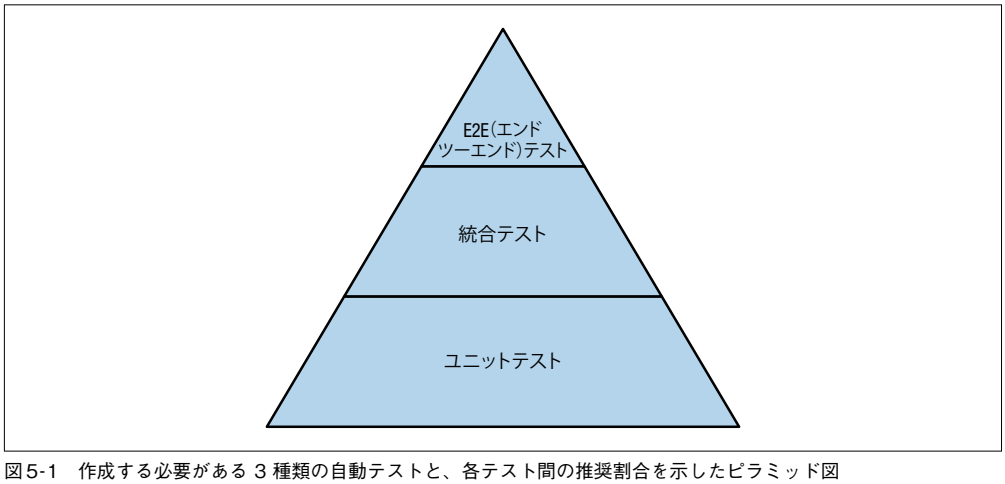


図5-1 作成する必要がある 3 種類の自動テストと、各テスト間の推奨割合を示したピラミッド図

テストのピラミッドは、ソフトウェアテストにおける重要な概念ですが、よく誤解されたり、完全に無視されたりしています。Mike Cohn によって考案されたこのモデルは、アプリケーションにどのような種類のテストが必要かを視覚的に示す指標となります。

自動テストには、以下の 3 つの種類があります。

ユニットテスト

システムの他の部分から切り離して個々のコンポーネントの機能をテストするように設計さ

†2 継続的インテグレーションプロセスの一環として、おそらくすべてのチェックイン時に実行されます。

れており、各部分が単独で正しく動作することを確認する

統合テスト
システムのさまざまなコンポーネントやモジュールが、一体となってどのように連携して機能するかを確認する

エンドツーエンドテスト
ユーザーインターフェースから始まり、バックエンドシステムに至るまで、アプリケーション全体をカバーする

このピラミッド構造は、ソフトウェアテストにおける基本的なトレードオフに基づいています。ピラミッドの上層へ進むにつれて、テストの実行時間は長くなり、保守コストは高くなり、失敗するおそれも高くなります。ユニットテストは数マイクロ秒で完了する一方、エンドツーエンドテストには数分かかることもあります。テストが数百から数千にも及ぶ場合、こうした差は劇的に大きくなります。それでは、それぞれについて詳しく見ていきましょう。

5.2.1 ユニットテスト

ユニットテストはテストピラミッドの基盤を成し、テストスイートの大部分を占めています。ユニットテストは、コードが満たさなければならない「契約」のようなものだと考えてください。コードの機能に対して変更が加えられた場合、対応するユニットテストは失敗し、潜在的な問題があることを知らせてくれます。これらのテストは、個々のコンポーネントや関数を独立して確認し、それぞれの部分が単独で正しく動作することを検証します。

ユニットテストは、静的解析やコードレビューと並んで、バグに対する最初の防衛ラインとしての役割を果たします。ユニットテストは、記述や実行が迅速に行えるものであり、開発中に即座にフィードバックを提供します。ユニットテストが失敗した場合、通常は特定の関数やコード行が指摘されるため、簡単にデバッグできます。この迅速さと正確さにより、ユニットテストは機能開発や継続的インテグレーションにおいて極めて有用であり、開発者は自信を持ってコードを迅速に反復・改善できます。

5.2.2 統合テスト

統合テストとは、システムのさまざまなコンポーネントやモジュールが、一体となってどのように連携して動作するかを詳細に検証するプロセスです。ユニットテストがコードの個々の部分を単独で検証するのに対し、統合テストはこれらのやり取りを検証することで、より広い範囲をカバーします。

しかし、システム全体の性能や機能を評価するシステム全体テストに比べると、そのカバー範囲は依然として限定的です。統合テストは、個別にテストされたコンポーネントを組み合わせた際にのみ発生する問題を発見するのに役立ちます。統合されたシステムが正しくかつ効率的に機能することを確認するうえで、極めて重要です。このステップは、システム全体の整合性と信頼性を維持

するために、SDLC において不可欠なものです。

統合テストは、その範囲と複雑さのために、通常、ユニットテストよりも実行に時間がかかり、実行頻度も低くなります。統合テストが失敗した場合、どのモジュール間のやり取りが問題を引き起こしたのかを特定するために、より詳細な調査が必要となります。

5.2.3 エンドツーエンドテスト

エンドツーエンド（E2E）テストとは、ユーザーインターフェースからバックエンドシステムに至るまで、アプリケーション全体を網羅する包括的なテストのことです。これらのテストは、実際のユーザーシナリオをシミュレートするように設計されており、実環境においてアプリケーションが期待どおりに機能することを確認します。

エンドツーエンドテストは複雑で広範囲に及ぶため、実行に時間がかかり、多大なリソースと期間を要します。その結果、他の種類のテストと比べ、エンドツーエンドテストの数は一般的に少なくなります。また、実行頻度は低く、通常は定期的に計画された工程の一部として実行されます。エンドツーエンドテストは不安定になりやすく、偽陰性が発生しやすくなります（機能上の問題ではなく、ユーザーインターフェース要素の些細な変更によってテストが失敗する場合もあります）。エンドツーエンドテストが失敗した場合、問題を特定して修正するために多大なデバッグ作業が必要になることがあります。

それでも、エンドツーエンドテストは、システムの全体的な機能に対して信頼性を高めてくれ、アプリケーションのすべてのコンポーネントがシームレスに連携することを確認するのに役立つため、非常に価値のあるものです。

5.2.4 どのようにテストを組み合わせるべきか？

実際のテスト数は、アプリケーションの複雑さ、使用されるテクノロジースタック、およびアプリケーションの重要度によって、プロジェクトごとに異なります。さらに、テスト数は組織の基準や慣行に基づいて決定される場合があり、これにはテストカバレッジや品質に関する具体的なガイドラインやベンチマークが含まれることがあります。プロジェクト固有のニーズに合わせてテスト戦略を調整すると同時に、アプリケーションに必要なあらゆる要素をすべてカバーできるようなバランスの取れたアプローチを維持することが重要です。

5.2.5 テストすべきでないもの

ソフトウェアの品質を確保するには、包括的なテストが不可欠です。ただし、何をテストするかについて戦略的に考えることも同じように重要です。ここでは、テスト戦略において避けるべき主なポイントを紹介します。

- テストの目的は自身のコードに置くべきであり、言語の機能やフレームワークのコードをテ

ストすべきではありません[†3]。

- ゲッター、セッター、ビルダーメソッド、データ転送オブジェクト（DTO、Data Transfer Object）などの自動生成されたコードのテストは避けてください。
- プライベートメソッドを直接テストするのは避けてください。代わりに、これらのプライベートメソッドを呼び出すパブリックインターフェースに対してテストを行ってください。
- 外部サービスに依存するテストの作成は避けてください。代わりに、ユニットテストではモックを、統合テストにはテストダブルを使用してください。

適切に設計されたテストスイートは、網羅的でありながら保守性が高く、管理が困難になるようなことがなく、十分なカバレッジを提供するものでなければなりません。自身のコードのテストに焦点を当て、不要なテストを避けることで、より効果的かつ効率的なテストスイートを作成できます。自動テストの種類や、どこに注力すべきか、またどこに注力すべきでないかが理解できたところで、次はコードカバレッジについて説明します。

5.3 コードカバレッジ

コードカバレッジとは、テストの実行時にコードのどの程度の割合が実行されたかを測定するためのメトリクスです。コードカバレッジは、ビデオゲームのマップのようなものだと考えてください。ゲームを始めた当初はマップ全体が暗く表示されていますが、移動してエリアを探索していくにつれて、その部分が明るくなっていきます。コードの一部を実行するテストは、その部分を「照らし出し」、そこは探索済みであることを示します。カバレッジが高いということは、マップの大部分を探索したことを意味し、カバレッジが低いということは、バグが潜んでいる可能性のある死角が残っていることを意味します。

テストの世界ではコードカバレッジが重要であるため、お使いの言語に対応したツールだけでなく、コードカバレッジをサポートするさまざまなIDEも存在します。開発者は、IDE上で直接カバレッジ分析を実行し、カバレッジに関するフィードバックを即座に得ることができます。また、CI/CDパイプラインに統合され、コードカバレッジの最低しきい値を満たすことを強制するツールもあります。カバレッジがそのしきい値を下回った場合、ビルドが失敗し、その結果、プルリクエストのマージがブロックされることになります。

コードカバレッジツールは、テストの実行結果を分析し、どの行、どの分岐、どの関数が実行されたかを正確に示すレポートを生成します。「行カバレッジ85%」や「分岐カバレッジ72%」といった数値が表示されることがあります。これらはテスト中にコードがどの程度実行されたかを表しています。

コードカバレッジは貴重な知見をもたらしますが、その限界を理解することも重要です。よくある誤解として、コードカバレッジが高いほどテストの質も自動的に高くなるという考えがありま

†3　その言語やフレームワークの保守を担当しているのは開発者であり、読者ではありません。

す。中には、デプロイを行う前に 80〜90% という最低限のカバレッジ基準を満たすことを義務付けている組織さえあります。こうした要件には確かに正しい意図がありますが、かえって逆効果となる行動を招くおそれもあります。

例えば、開発者が実際に意味のある動作を検証することもなく、ただコードを実行するだけのテストを作成してしまうことが考えられます。これは、本質的にカバレッジメトリクスを「ごまかす」ことになります。メソッドを呼び出すだけで、意味のあるアサーションを行わないテストであっても、カバレッジにはカウントされますが、バグに対する有効な防御策にはなりません。

カバレッジ 100% を目指すことは、単なる「見せかけの指標」に過ぎません。そうではなく、作成中のテストに対するフィードバックの手段として活用しましょう。ミッションクリティカルなビジネスロジックに対してカバレッジが低い場合は、コードのその部分にさらなる注意を払う必要があることを示しています。一方、単にデータを保持するだけのクラスや設定をテストしている領域でカバレッジが高い場合、テストが不要なコードをテストしている可能性があることを示唆しています。

動作を検証する意味のあるテストを書くことに注力し、コードカバレッジを参考に、より注意が必要なコードを特定しましょう。目標は、単に要件を満たすために恣意的な数値を達成することではありません。テストの真の目的は、自分が記述し、そして最終的に本番環境にデプロイするコードに対する信頼性を高めることにあります。コードカバレッジについて理解できたところで、これらのツールを活用できるよう、テストの作成を始めましょう。

5.4 テストの作成

さあ、本題に入り、テスト作成の仕組みを学んでいきましょう。

5.4.1 はじめに

どのようなプログラミング言語、フレームワーク、あるいはメタフレームワークを使用していても、多くの場合、複数のテストツールが利用可能です。この節では、Java プログラミング言語を使用します。お使いの言語によって設定や構文は異なるかもしれませんが、基本的な考え方は同じです。

プロジェクトを作成したら、次は使用するテストフレームワークを決めることになります。選択肢は数多くありますが、Java で最も人気のあるテストフレームワークは JUnit です。JUnit を導入するには、Maven の POM ファイルに適切な依存関係を宣言し、インストールしてください。

ここでは JUnit の詳細については触れません。もし JUnit についてさらに詳しく知りたい場合は、ユーザーガイド（https://oreil.ly/JIAwL）にある充実したドキュメントを参照してください。JUnit のインストールが完了したら、テストの作成を始めることができます。

テストの作成には、「テストファースト」と「テストラスト」という 2 つの主なアプローチがあります。

- テスト駆動開発（TDD、Test-Driven Development）などの**テストファースト手法**では、「レッド・グリーン・リファクタリング」というサイクルに従います。まず失敗するテスト（レッド）を作成し、次にテストに合格するための最小限のコードを実装して（グリーン）、最後に動作を変えずにコードを改善します（リファクタリング）。
- **テストラスト手法**では、機能実装後にテストを記述します。

どちらの手法にもメリットがあります。どちらを選ぶかは、多くの場合、プロジェクトの要件やチームの意向によって決まります。どのアプローチを採用する場合でも、すべてのテスト手法では、期待される結果を検証するためにアサーションが使用されます。



AIについて

テストの作成は、AIの支援を活用するのに適したユースケースです。既存のテストスイートから例をAIツールに提供することで、組織が好むテストのスタイル、命名規則、コーディング標準を理解させることができます。AIは、テストケースの生成、見落としがちなエッジケースの提案、定型的なテストコードの記述、さらには既存のテストの可読性を高めるためのリファクタリングまで支援してくれます。これにより、チームで確立したパターンとの一貫性を保ちつつ、テストプロセスを大幅にスピードアップできます。

5.4.2 アサーション

アサーションとは、プログラムの実行中に特定の条件が満たされているかどうかを確認するために使用する、関数やメソッドのことです。Javaには数多くのアサーションライブラリがあり、それぞれが異なる機能を提供しています。

JUnit 5には、さまざまな検証を行うために数多くのアサーションが用意されています。これらのアサーションはすべて、`org.junit.jupiter.Assertions`クラスの静的メソッドです。例えば、数学的な演算の和が正しいかどうかを検証したい場合を考えましょう。2つの整数を受け取る`assertEquals`メソッドがあり、その使い方を以下のコードブロックに例を示します。最初の引数は期待される値、2番目の引数は実際の値です。

```
@Test
void shouldAdd2Numbers() {
   int expected = 3;
   int actual = 2 + 1;
   assertEquals(expected, actual);
}
```

2つの数値が等しい場合、アサーションは成功し、したがってこのテストも成功します。等しくない場合、アサーションは失敗し、テストも失敗します。利用可能なアサーションについて詳しく知りたい場合は、使用しているアサーションライブラリのドキュメントをご確認ください。

5.4.3 ユニットテストの作成

先ほども説明したように、ユニットテストは、独立して実行され、非常に高速なテストです。数学的な演算を行うクラスがあり、それぞれの演算に対してテストが必要だと想像してみてください。以下に、Operations という名前のクラスの例を示します。

```
package com.fose;

public class Operations {
   public double add(double a, double b) {
       return a + b;
   }
   public double subtract(double a, double b) {
       return a - b;
   }
}
```

最初のテストを作成する前に、ソースコードがどこにあるのかを確認する必要があります。このクラスのコードは、com.fose パッケージ内にあります。テストクラスを作成する際は、**/src/test/java** ディレクトリに、またソースコードと同じパッケージに配置するようにしてください。以下の例では、加算および減算のテストを用意して、Operations クラスのすべての機能をカバーしています。

ここでは、機能を実装した後にテストを書く「テストラスト」のアプローチを採用しています。まずは、作成する各テストに対応するメソッドのスタブを追加することから始めましょう。

```
package com.fose;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class OperationsTest {
   @Test
   void add() {
   }

   @Test
   void subtract() {
   }
}
```

まず、Operations クラスのインスタンスが必要です。これは、現在テストしているシステムの特定のコンポーネント、モジュール、または部分を指すため、よく**テスト対象システム（SUT、System Under Test）**と呼ばれます。ここで記述しているのは、Operations クラスのテストにのみ焦点を当てたユニットテストです。

```
class OperationsTest {

   private final Operations sut = new Operations();
}
```

テスト対象システムのインスタンスが用意できたので、テストメソッドを記述できます。ここでは特別なことは行いません。実際のテストでは、ユーザーに問題を発見されることのないよう、あらゆるエッジケースを網羅するようにしてください。

```
package com.fose;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class OperationsTest {
   private final Operations sut = new Operations();
   @Test
   void add() {
       assertEquals(5, sut.add(2, 3));
       assertEquals(0, sut.add(-1, 1));
       assertEquals(-5, sut.add(-2, -3));
   }
   // 簡潔にするため、他のテストについては割愛
}
```

5.4.4 モック

ソフトウェアテストにおいて、**モック**とは、依存関係をシミュレートした代替物を使って、コードを独立した環境でテストできるようにする手法です。このような代替物は、一般に**テストダブル**と呼ばれます。

依存関係とは何でしょうか？ ユニットテストを作成するという場合には、**依存関係**とは、テスト対象システムがタスクを完了するために依存する、他のクラスのことを指します。モックを使用することでユニットテストにおける依存関係を分離できますが、統合テストにおいては大きな課題となります。統合テストでは、これらの依存関係がどのように連携するかを検証することを目的としていますが、この検証を行うのが難しい場合があります。複数の依存関係を管理し、それらが正しい状態にあることを確認し、それらのやり取りを処理することは、統合テストをすぐに複雑なものにしてしまいます。それは、一度に複数のボールをジャグリングしようとするようなものです。それぞれの依存関係がどのように振る舞い、他の依存関係にどのように影響を与えるかを常に把握しておく必要があります。

次の例では、`UserService` というクラスがあり、`UserRepository` と `EmailService` という2つの依存関係を持っています。`UserRepository` クラスは、ユーザーの読み取りとデータベースへの保存を担当し、`EmailService` はメール送信を担当します。

```
package com.fose;

public class UserService {

   private UserRepository userRepository;
   private EmailService emailService;

   public UserService(UserRepository userRepository, EmailService emailService) {
       this.userRepository = userRepository;
       this.emailService = emailService;
   }

   public void registerUser(String username, String email) {
       // ユーザー登録
   }
}
```

UserService クラスの registerUser メソッドのテストに焦点を当てたユニットテストを作成したい場合、どうすればよいのでしょうか？ 現在、このクラスには 2 つの依存関係がありますが、その両方を含めるとユニットテストではなく、統合テストとなってしまいます。

ここで、モックフレームワークの出番となります。本番環境の UserRepository を使う代わりに、そのクラスのモックを使います。このモックは元のクラスと**見た目は**似ていますが、実際にはそれとは異なります。「アヒル（duck）のように振る舞い、アヒルのように鳴くが、アヒルではない」という言葉を聞いたことがあるでしょう。このモックは UserRepository のように見えますが、実際は UserRepository ではなく、新しいユーザーをデータベースに保存するという**実際の**ビジネスロジックは実行しません。その代わりに、モックにプログラムしたとおりの動作を行います。

UserService クラスのテストにのみ焦点を当ててください。依存関係については、それぞれのユニットテストでテストされます。この例では、Java の世界で広く使われている Mockito というモックフレームワークを使用しています。ただし、ここで紹介する考え方はどの言語を使用している場合でも応用できます。

Mockito を使うと、次のような UserServiceTest クラスを記述できます。

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

   @Mock
   private UserRepository userRepository;
   @Mock
   private EmailService emailService;
   @InjectMocks
   private UserService userService;

   @Test
```

```
    public void testUserRegistration() {
        String username = "newuser";
        String email = "newuser@example.com";
        when(userRepository.existsByUsername(username)).thenReturn(false);
        userService.registerUser(username, email);
        verify(userRepository).save(any(User.class));
        verify(emailService).sendWelcomeEmail(email);
    }

}
```

先ほどのテストコードでは、@Mock アノテーションを使用してモックオブジェクトを作成し、依存関係をシミュレートしています。これにより、実際の実装を使用することなく、テストを行うことが可能になります。そして、これらのシミュレートされた依存関係は、テスト対象クラスに自動的に注入されます。

5.4.5 統合テストの作成

前の節では、UserRepository と EmailService の両方に依存する UserService の例を紹介しました。ユニットテストを作成する際、そのクラスを単体でテストしたかったため、これら 2 つの依存関係をモックで置き換えました。

統合テストでは、UserService が依存関係とどのようにやり取りするかをテストします。まず UserServiceIntTest という名前のテストを新たに作成します。名前に Int という文字列が含まれているので、このテストが統合テストであることがはっきりとわかります。

UserService はリポジトリとメールサービスの両方に依存しているため、UserService のインスタンスを作成するには、それぞれのクラスのインスタンスが必要となります。実際のアプリケーションで、依存性注入機能を備えたフレームワークを使用している場合、単にインスタンスを要求するだけで済むかもしれませんが、この例では手作業で作成する必要があります。

```
class UserServiceIntTest {
    private UserRepository userRepository;
    private EmailService emailService;
    private UserService userService;

    @BeforeEach
    void setUp() {
        userRepository = new UserRepository();
        emailService = new EmailService();
        userService = new UserService(userRepository, emailService);
    }

}
```

これで、依存関係の（モックではない）実体が含まれた UserService のインスタンスが用意で

きたので、統合テストを作成できます。

```
@Test
void shouldNotRegisterUserWithUsernameOfUser() {
   // 無効なユーザー登録のテスト
}

@Test
void shouldRegisterValidUser() {
   // 有効なユーザー登録のテスト
}
```

5.4.6 エンドツーエンドテストの作成

E2E テストは、バックエンド API に焦点を当てます。E2E テストを実行するには実環境が必要です。この例の場合は、Tomcat サーバーを組み込んだ Spring Boot 環境となります。このテストの目的は、実サーバーにリクエストを送信し、レスポンスを検証することです。以下に、E2E テストの概要を紹介します（完全版は GitHub で確認できます）。

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
class UserRegistrationE2ETest {

   @Autowired private MockMvc mockMvc;
   @Autowired private UserRepository userRepository;
   @Autowired private EmailService emailService;
   @Autowired private ObjectMapper objectMapper;

   @BeforeEach
   void setUp() {
       userRepository.deleteAll();
   }

   @Test
   void testUserRegistration() throws Exception {
       String username =
               "testuser";
       String email = "testuser@example.com";
       User request = new User(username, email);

       // リクエストの実行
       mockMvc.perform(post("/api/users/register")
              .contentType(MediaType.APPLICATION_JSON)
              .content(objectMapper.writeValueAsString(request)))
              .andExpect(status().isOk());

       // ユーザーがデータベースに保存されたことを検証
       Optional<User> savedUser = userRepository.findByUsername(username);
       assertTrue(savedUser.isPresent());
```

```
        assertEquals(email, savedUser.get().email());

        // // この他の検証（歓迎メールの送信）は簡潔にするため割愛
    }
}
```

このテストでは、以下の処理を行います。

1. サーバー用にランダムなポートを設定する
2. サーバーサイドの Spring MVC テストをサポートするために MockMvc を使用する
3. 各テストを実行する前にデータベースをクリーンアップする
4. ユーザー登録リクエストをシミュレートする
5. レスポンスとデータベースの状態を検証する

このアプローチにより、API リクエストからデータベースとのやり取りに至るまでの全フローをテストできます。そして、システムがエンドツーエンドで正しく機能することを確認できます。

5.5 まとめ

本章では、自動テストの世界と、ソフトウェア開発におけるその重要な役割について学んできました。テストが貴重なドキュメントとしての役割を果たし、コードの品質を向上させ、リファクタリングの際のセーフティネットとなる様子を見てきました。ユニットテストからエンドツーエンドテストに至るまで、それぞれのテストは、ソフトウェアが意図したとおりに動作することを確認するために、不可欠な役割を果たしています。

これから述べることは、ぜひ覚えておいてください。健康的なライフスタイルを維持するのと同じように、優れたソフトウェアを書くことも、良い習慣を身につけ、一貫して健全な選択を続けることなのです。堅牢なテストスイートを用いてコーディングすることは、トレーナーとともに明確なトレーニング計画を持つようなものです。テストがあれば、失敗するかもしれないという不安に常に苛まれることなく、自信を持ってコーディングを進められます。新しいことに挑戦し、変更を加えられます。この自信こそが、ソフトウェア開発の複雑さを乗り切る力を与えてくれ、既存の機能を壊してしまうことを恐れることなく、継続的な改善と新たな要件への適応を可能にするのです。

キャリアを積んでいく中で、テストを習慣づけるよう心がけてください。最初は余計な手間のように思えるかもしれませんが、コードの品質、プロジェクトの保守性、そして自身の成長にとって、長期的なメリットは計り知れません。品質向上は一度きりの作業ではなく、継続的な実践であることを忘れないでください。自動テストを取り入れるのは、単にバグを見つけるためだけではありません。より効果的で自信に満ち、熟練した開発者へと成長していくためなのです。

5.6 実践に移す

テストは一夜にして習得できるスキルではなく、継続的な実践を通じて磨かれるものです。以下で紹介する実践方法は、新機能の開発、バグの修正、コードベースの改善など、どのような状況であっても、自動テストの世界への入り口にできます。これらすべてを一度に実践しようと焦る必要はありません。代わりに、現在の状況に合った手法を１つか２つ選び、そこから始めてみてください。これらの習慣に慣れてきたら、徐々に他の手法も取り入れていきましょう。今日、たった１つのテストを書くことだけでも、何も書かないよりはるかにましです。以下にテストスキルを磨くのに役立ついくつかの手法を紹介します。

オープンソースのライブラリから学ぶ

お気に入りのオープンソースライブラリを探索しましょう。

- そのライブラリのテストをじっくり読んでみてください。
- ライブラリの機能について、これまで知らなかったことは何でしょうか？
- 彼らのテスト手法は、読者のやり方とどのように異なりますか？
- 彼らのテスト戦略の一部を、自分自身のプロジェクトに取り入れてみてはいかがでしょうか。

テストをドキュメントとして活用する

コードベースのよく知らない部分を担当することになった場合、以下の手順に従ってください。

- まずは既存のテストを確認してください。
- テストが不足していたり、あるいは不十分だったりする場合は、コードを理解しながら新しいテストを作成してください。

既存のコードベースを改善する

プロジェクトのテスト数が不十分だと感じた場合は、以下の手順に従ってください。

- すべてを一度に直そうとしないでください。その代わりに、少しずつ改善していきましょう。
- 各開発者（自分自身も含めて）に、１日に１つの新しいテストを書くよう促しましょう。

新機能はテストから始める

次に新しい機能の開発を任された際は、まずテストから書き始めてみてください。この習慣は、要件を明確にし、最初からより良いコードを設計するのに役立ちます。

テストを用いてバグを修正する

バグに対処する際は、以下の手順に従ってください。

- まず、バグが存在することを確認するテストを作成してください。このテストは、最

初は失敗するはずです。

- テストを作成することで、コードのリファクタリングが行えるようになり、品質、可読性、保守性を向上させるよい機会となるでしょう。

テストを推進する

適切なテストを行っていれば防げたと思われる問題が見つかった場合は、以下の手順に従ってください。

- その問題を検出できたはずのテストを追加してください。
- これをチームの学びの機会として活用してください。
- テストカバレッジを向上させることで、将来的に同様の問題をどのように防げるか、マネージャーと話し合ってください。

実践する

テスト作成のスキルを向上させる方法は、実践することしかありません。CodeKata を使って練習し、体系的かつ反復的な方法でテスト作成のスキルを磨いてみてはいかがでしょうか。

堅牢なテストスイートを構築することは、継続的なプロセスであることを忘れないでください。より良いテスト手法に向けて踏み出す小さな一歩ひとつひとつが、コードベース全体の品質と保守性に寄与するのです。

5.7 関連リソース

- 単一責任の原則（https://oreil.ly/Ui_00）
- Mockito（https://oreil.ly/DkLzi）
- JUnit 5（https://oreil.ly/eWw_h）
- AssertJ（https://oreil.ly/b5rLP）
- Hamcrest（https://oreil.ly/fOAgm）
- 『Clean Code: A Handbook of Agile Software Craftsmanship』Robert C. Martin 著（Pearson、2008 年）
  和書『Clean Code アジャイルソフトウェア達人の技』（KADOKAWA、2017 年）

6章
初めて触れるシステムの調査と変更

私が得た最も貴重な経験のいくつかは、レガシーアプリのサポートを通じて得たものです。
強くおすすめしますが、誰にも経験してほしくないですね。
— Dalia Abo Sheasha、ソフトウェア開発者

多くの開発者は、グリーンフィールドプロジェクトという白紙状態から始めることを好みます。しかし、ソフトウェア開発の厳しい現実として、私たちの仕事の大部分は既存のシステム上で行われます。この現実によって、ソフトウェア開発という職業に対する前向きな姿勢を失わないでください。既存のコードベースを扱うことには、メリットもあります。重要な決定の多くはすでに下されているため、インフラストラクチャについて心配することなく、目の前の問題に直接集中することができるのです。

新しいプロジェクトに参加する際や、新しいコードベースを調査する際には、初めて触れるシステムを調査し、修正するためのスキルを身につける必要があります。本章では、初めて触れるコードをどのように読み解き、理解するか、そして安全に変更を加える方法について学びます。新しいチームに加わる場合でも、レガシーシステムの保守を行う場合でも、これらのスキルにより、どのようなコードベースに対しても自信を持って貢献できるようになるでしょう。このスキルは、2章で取り上げたコードの読み方の基礎をさらに発展させた内容です。

6.1　初めて触れるコードベースを理解する

新しい会社に入社する場合でも、現在の職場で新しいプロジェクトに移る場合でも、初めて触れるコードベースを扱うことは、ストレスや不安の原因となることがあります。その原因は、知らない領域であることや、あるいは馴染みのない言語、フレームワーク、ツールを使用することにあるかもしれません。しかし、明確な計画と、この未知の領域を乗り切るためのテクニックを駆使して、この課題に取り組めば、成功率と自信の両方を高めることができるでしょう。

この節では、プロジェクトの全体像や目標を理解するためのテクニックを学びます。コードの流れや実行パスを追うことで、新しいプロジェクトに慣れる方法を説明します。最後に、複雑なシス

テムを扱いやすい単位に分解することで、少しずつメンタルモデルを構築する方法を解説します。本章を読みすすめたり、新しいプロジェクトを調査したりする際は、新しいコードベースのすべてをすぐに理解できなくても大丈夫だということを覚えておいてください。そのプロジェクトに何年も携わってきたチームメンバーでさえ、そのすべてを知っているわけではありません。

6.1.1 全体像から始める

「木を見て森を見ず」という言葉があります。これは、些細な詳細（使っているフレームワークのバージョンなど）にばかり気を取られていると、全体像や状況（このプロジェクトの目標）を見失ってしまうという意味です。おそらく、すぐにプロジェクトに飛び込んで貢献を始め、チームに自分の価値を証明したいと思っていることでしょう。どのような場合でも計画を立てることはよいことです。ここでは、全体像を把握するためのヒントをいくつか紹介します。

6.1.1.1 プロジェクトを理解する

コードを理解し始める前に、まずプロジェクトの目的と意図を理解する必要があります。大局的な視点から始めましょう。このプロジェクトは、何のために存在するのでしょうか？ このプロジェクトのステークホルダーは誰でしょうか？ このプロジェクトは雇用主にとってどのような意味を持つのでしょうか？ それはビジネスにおいて重要な部分を占めているのでしょうか？

プロジェクトを理解する方法はたくさんありますが、まずはプロダクトマネージャーなど、プロダクトについて深い知識を持つ方から話を聞くのがよいでしょう。その方と面談の機会を設けられるか確認し、学ぶ姿勢で臨んでください。後で参照できるよう、できるだけメモを取ってください。オンラインでのミーティングの場合は、後で復習できるよう、録画してもよいか尋ねてみてください。何よりも重要なのは、質問をすることです。愚かな質問などありません。聞かないことこそが愚かなのです。

プロダクトオーナーと話し合い、プロジェクトの概要を把握したら、次は技術的な面についてさらに深く掘り下げていきましょう。この段階で、現在そのプロジェクトに携わるソフトウェアエンジニアに連絡を取ってください。もし、当初からシステム開発に携わってきた人物が見つかったら、それはまるでゴールドラッシュで金脈を掘り当てたようなものです。理想を言えば、テックリードやアーキテクトといった役割を担っている人物に連絡を取り、過去の意思決定について貴重な知見を得られるようにするのがよいでしょう。

このプロダクトの顧客はどのような方々でしょうか？ 外に出て、このプロジェクトに関する公開情報を探してみてください。もし見つかったら、顧客の視点からその情報を分析してみてください。企業によっては、プロダクト向けに「ユーザーペルソナ」を作成しています。これは実際のユーザーをモデルにした架空のプロフィールであり、さまざまなタイプのユーザーや、その目標、課題を理解するのに役立ちます。このユーザーペルソナにより、技術的な決定がなされた背景について、貴重な手がかりが得られるでしょう。

6.1.1.2 既存のドキュメントを確認する

ステークホルダーと話し合った後、コードベースを理解するための次のステップは、ADR を含め、入手可能なあらゆるドキュメントを徹底的に精査することです。プロジェクトそのものについて大まかな理解が得られていれば、これらのドキュメントも徐々に理解できるようになるはずです。この段階では、ドキュメントを通じてコードベースについてできる限り多くのことを学んでいくのです。

その大きな部分を占めるのが、プロジェクトの「オンボーディングドキュメント」です。これには、ローカルマシンでプロジェクトを立ち上げて稼働させるために必要な情報がすべて記載されています。Docker などのツールの登場により、このドキュメントは年々改善されていますが、それでも確認作業は依然として手間のかかるプロセスです。

現時点では、まだ業務に慣れていないため、ドキュメントの作成は主な仕事ではありませんが、メモを取り、ドキュメントの不備を特定しておくべきです。このようにしておけば、後で振り返って修正することができるので、チームに貢献する素晴らしい機会となります。

> **ドキュメントの落とし穴**
>
> Dan です。以前、複雑な認証システムを理解しようと、最新だと思っていたドキュメントを丹念に読み込み、3 日間も費やしたことがあります。ようやくチームメンバーに助けを求めたところ、相手は笑いながらこう言いました。「ああ、そのドキュメントは 2 バージョン前のものだよ。去年、認証システムを完全に書き直したんだ。」ここから得られる教訓は？ ドキュメントは嘘をつくことがありますが、コードは決して嘘をつきません。読んだ内容と実際に動いているコードを、常に照合して確認してください。そして助けを求めることをためらわないでください。たった 3 分会話していれば、私は 3 日間も無駄にすることはなかったでしょう。

6.1.1.3 アーキテクチャとプロジェクトの構造を理解する

プロジェクトの全体像を把握し、入手可能なすべてのドキュメントを読み終えたら、次はコードベースのアーキテクチャと構成を確認する段階です。このステップは重要ですの、実際にコードや設定を読んだり、コードを書いたりする前に必ず行うようにしてください。プロジェクトの構造からは多くの情報が得られ、プロジェクトの全体像を俯瞰できます。そして、このような情報はコードベースを把握する際に非常に役立ちます。

ほとんどのプロジェクトは、意図的な設計によるものであれ、自然な進化によるものであれ、何らかのアーキテクチャパターンに従っています。まずは、コードベースがどのパターン（あるいはパターンの組み合わせ）に従っているかを特定することから始めましょう。

レイヤー別パッケージング（package by layer）は、技術的な役割に基づいてコードを水平方向に整理する手法です。コントローラ（controllers）、サービス（services）、リポジトリ（repositories）、

モデル（models）といった名前のトップレベルのディレクトリがよく使われます。このアプローチでは、技術的に類似したコンポーネントをまとめて配置するため、特定の種類のコンポーネントをすべて簡単に見つけられます。以下は、レイヤー別パッケージングの例です。

```
src/
  ├── controllers/
  │   ├── UserController.java
  │   └── ProductController.java
  ├── services/
  │   ├── UserService.java
  │   └── ProductService.java
  ├── repositories/
  │   ├── UserRepository.java
  │   └── ProductRepository.java
  └── models/
      ├── User.java
      └── Product.java
```

機能別パッケージング（package by feature）では、ビジネスケイパビリティや機能を中心にコードを垂直方向に整理します。ユーザー（users）、プロダクト（products）、注文（orders）などのビジネスドメインを表すトップレベルのディレクトリが配置されます。このアプローチでは、機能のあらゆるものがまとめてカプセル化されるため、ビジネスワークフロー全体を把握しやすくなります。以下は、機能別パッケージングの例です。

```
src/
  ├── users/
  │   ├── UserController.java
  │   ├── UserService.java
  │   ├── UserRepository.java
  │   └── User.java
  └── products/
      ├── ProductController.java
      ├── ProductService.java
      ├── ProductRepository.java
      └── Product.java
```

ヘキサゴナルアーキテクチャ（ポートとアダプタとも呼ばれます）は、ビジネスロジックと外部システムを分離することを目的にコードを整理します。外部の世界と接続するアダプタに囲まれている、コアドメインモデルを探してください。このパターンは、ビジネスルールと技術的な実装を分離することに重点を置いたものです。以下は、ヘキサゴナルアーキテクチャを採用したコードの例です。

```
src/
  ├── domain/                  // ビジネスロジック
```

```
│   ├── model/
│   └── service/
├── application/              // ユースケース、オーケストレーション
│   └── service/
├── ports/                    // アダプタ用インターフェース
│   ├── input/
│   └── output/
└── adapters/                 // 技術的な実装
    ├── web/
    ├── persistence/
    └── messaging/
```

マイクロサービスアーキテクチャでは、機能を複数の独立したサービスに分割します。マイクロサービス環境で作業する場合は、担当するサービスのアーキテクチャだけでなく、それがより大きなエコシステムの中でどのように位置づけられているかを理解する必要があります。以下の例では、`user-service` のようなトップレベルサービスが、それぞれ独立したマイクロサービスとなっています。

```
microservices-system/
├── user-service/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── models/
│   └── Dockerfile
├── product-service/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── models/
│   └── Dockerfile
├── order-service/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── models/
│   └── Dockerfile
├── api-gateway/
│   ├── config/
│   ├── filters/
│   └── routes/
└── shared/
    ├── common-models/
    └── utils/
```

6.1.2 実行フローを理解する

実行フローとは、プログラムが実行時にたどる命令の経路のことです。どのコードをどの順序で

実行するかを決定する、すべての分岐、ループ、関数呼び出しが含まれます。

プロジェクトの全体像を把握し、その目的を理解したところで、いよいよコードの検討に入ることができます。現時点では、実際にコードを記述したり、詳細な分析を行ったりするわけではありません。あくまで、コードの流れを理解することを目的としています。この節では、アプリケーションのエントリーポイントを見つけ、リクエストとレスポンスを追跡し、外部依存関係を特定する方法を通じて、実行フローを理解するためのテクニックを学びます。

6.1.2.1 アプリケーションのエントリーポイントを特定する

アプリケーションには、アプリケーションへのエントリーポイント（入り口）となる一連の「扉」があります。これらの扉には、アプリケーションのメインメソッド、公開 API、Web UI などが含まれます。アプリケーション内のさまざまな実行フローについて学びたい場合、これらのエントリーポイントを出発点とするのが最適です。

例えば、Spring Boot の Web アプリケーション[†1]では、`@SpringBootApplication` アノテーションを探し、main メソッドを特定することでメインクラスを見つけられます。

```
@SpringBootApplication
public class PetClinicApplication {
    public static void main(String[] args) {
        SpringApplication.run(PetClinicApplication.class, args);
    }
}
```

アプリケーションのエントリーポイントの 1 つを特定したら、そこからコードの実行フローを追跡できます。ここで、IDE のデバッガーを活用すると非常に役立ちます。`run` メソッドにブレークポイントをセットし、ステップ実行機能を使用してください。フレームワークが内部でどのような処理を行っているのかが把握でき、コードベースを理解するための絶好の出発点となります。

アプリケーションの他のエントリーポイントについて調べる場合、公開 API を探すとよいでしょう。この PetClinic サンプルアプリケーションでは、REST エンドポイントがそれにあたります。以下の `@GetMapping` により、/pets/new にリクエストを送信したときに、そのリクエストを実行するのがこのメソッドだとわかります。この後 IDE やデバッグツールを使って実行パスを追跡できます。

```
@GetMapping("/pets/new")
public String initCreationForm(Owner owner, ModelMap model) {
   Pet pet = new Pet();
   owner.addPet(pet);
   return VIEWS_PETS_CREATE_OR_UPDATE_FORM;
}
```

†1 本章では、例として Spring PetClinic アプリを使用します。Java や Spring の専門家でなくとも、比較的理解しやすい内容になっているはずです。

アプリケーションの実行フローを把握するのに役立つエントリーポイントは数多くあります。以下に一般的なものをいくつか挙げておきますが、他にもたくさんあります。

メインメソッド／ブートストラップメソッド
: 従来の実行開始点（Java の `main()` のようなもの）

公開 API ／コントローラ
: 外部システムに対して機能を公開するエンドポイント

イベントハンドラ／リスナー
: 特定のイベントやトリガーに応じて実行されるコード

スケジュールされたタスク／ジョブ
: あらかじめ設定された間隔で実行される機能

ライフサイクルフック
: コンポーネントの作成、起動、終了時に呼び出されるメソッド

プラグイン／拡張ポイント
: アプリケーションの機能を拡張するために設計されたインターフェース

メッセージコンシューマ
: キューやメッセージブローカーからのメッセージに応答するコード

コマンドライン引数処理ルーチン
: 起動パラメータを処理するロジック

データベースのトリガー／ストアドプロシージャ
: データの変更に応じて実行されるサーバーサイドのコード

重要なのは、どの扉を開けばよいかを見極めることです。その扉を見極めれば、それを足がかりとして、アプリケーションの仕組みを理解できます。

6.1.2.2 データを追跡する：リクエストの流れをたどる

既存のコードベース、特に Web アプリケーションや API（REST、GraphQL、gRPC）を扱う際、リクエストがシステム内をたどる経路を追跡することは、コードの動作や関連するシステムを理解するうえで非常に効果的です。ドキュメントやテストでは想定される動作が示されているかもしれませんが、リクエストを追跡することで、実際に何が起きているのかがはっきりとわかります。

以下のような理由により、リクエストをトレースすることには意味があります。

- 複数のコンポーネントやシステムにわたる実際の実行パスが明らかになる

- 関連するすべてのレイヤー（コントローラ、サービス、リポジトリなど）を特定するのに役立つ
- 処理の過程で発生するデータ変換が把握できる
- 隠れたビジネスロジックや検証ルールが明らかになる
- エラーが実際にどのように処理されているかがわかる

新しいコードベースを探索する開発者にとって、リクエストを追跡する際には、非常に優れたツールがいくつか利用可能です。この節では、それらのツールについて紹介します。すべてを使う必要はないかもしれませんが、どのようなツールが利用できるのかを知っておくことは役に立つでしょう。いくつか試してみて、自分の仕事のスタイルに合うツールを見つけてください。

ブラウザの開発者ツール

Web アプリケーションを扱う際、リクエストやレスポンスを確認するには、まずブラウザの開発者ツールを利用するのがよいでしょう。次の例では、PetClinic アプリケーションを実行し、飼い主に新しいペットを追加しています（**図6-1** 参照）。

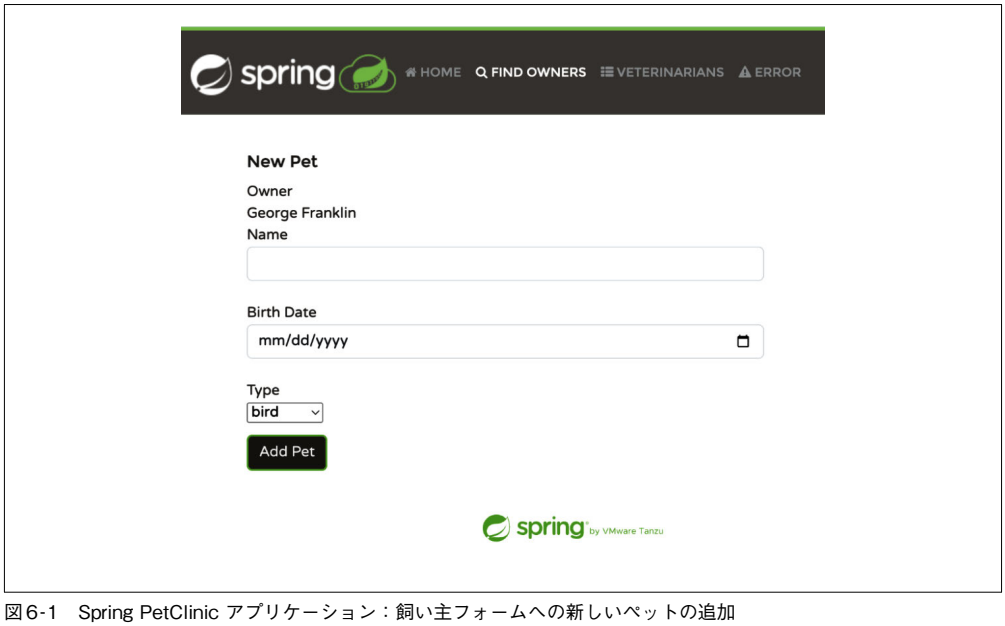


図6-1　Spring PetClinic アプリケーション：飼い主フォームへの新しいペットの追加

フォームに入力し「Add Pet（ペットを追加）」ボタンをクリックすると、以下のエンドポイントにリクエストが送信されます。

```
@PostMapping("/pets/new")
public String processCreationForm(Owner owner, @Valid Pet pet,
BindingResult result,
      RedirectAttributes redirectAttributes) {

   if (StringUtils.hasText(pet.getName()) && pet.isNew() &&
   owner.getPet(pet.getName(), true) != null)
      result.rejectValue("name", "duplicate", "already exists");

   LocalDate currentDate = LocalDate.now();
   if (pet.getBirthDate() != null && pet.getBirthDate().isAfter(currentDate)) {
      result.rejectValue("birthDate", "typeMismatch.birthDate");
   }

   if (result.hasErrors()) {
      return VIEWS_PETS_CREATE_OR_UPDATE_FORM;
   }

   owner.addPet(pet);
   this.owners.save(owner);
   redirectAttributes.addFlashAttribute("message", "New Pet has been Added");
   return "redirect:/owners/{ownerId}";
}
```

開発者ツールを開き、「Network」タブを確認すると、**図6-2** に示すように、**/pets/new** エンドポイントへの POST リクエストが表示され、ヘッダー、ペイロード、レスポンスなどを確認できます。

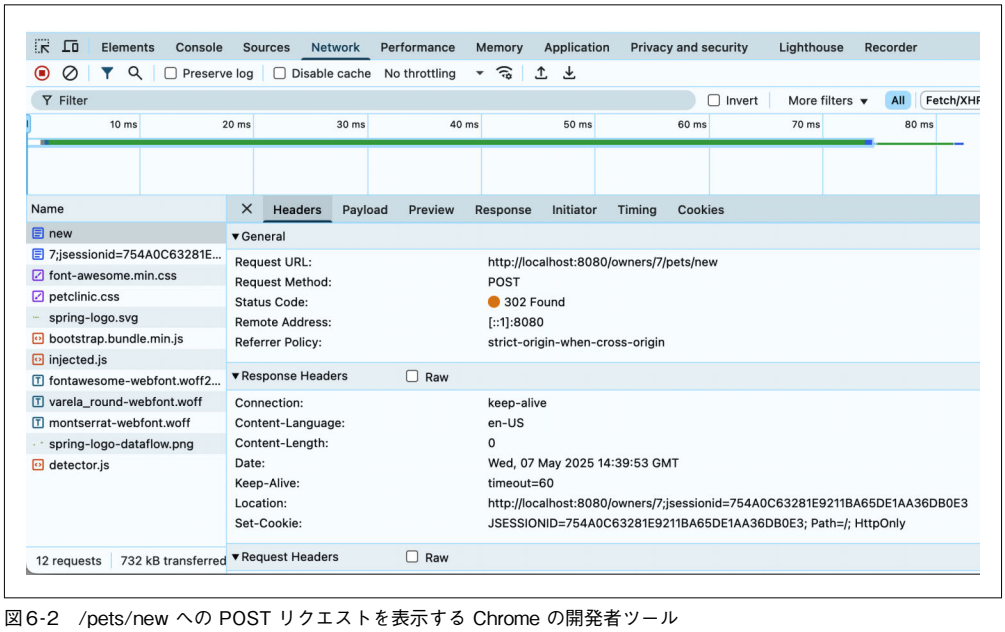


図6-2　/pets/new への POST リクエストを表示する Chrome の開発者ツール

API テストツール

API エンドポイントを直接テストしたい場合に、それが単純な GET リクエストであれば、ブラウザに URL を入力して結果を確認できます。しかし、POST、PUT、PATCH、DELETE などの他のリクエストメソッドを使用する場合は、API テストツールを活用する必要があります。世の中には Postman、Insomnia、Bruno など、非常に優れたツールがあります。また、主要な IDE の多くに、同様の機能を備えたプラグインが用意されています。

前節では、ブラウザからフォームを送信し、開発者ツールを使用して POST リクエストの内部構造を確認しました。もし UI を経由せずに、API エンドポイントを直接テストしたい場合はどうすればよいのでしょうか。ここで API テストツールの真価が発揮されます。ヘッダーや認証情報、リクエスト本文などを含めた POST リクエストを送信できます。リクエストを送信した後、レスポンスを確認できるため、UI をまったく使う必要がありません（**図6-3** 参照）。

Postman などのツールを使用して、このエンドポイントをテストし、以下の点を確認できます。

必須フィールドとデータ型

リクエスト本文の中で、どのフィールドが必須であるか、またそれらのフィールドがどのような形式（文字列、数値、配列など）を想定しているかを確認できます。

入力の検証動作

API が、無効なデータ、欠落したフィールド、不正な JSON、空の文字列や null 値などのエッジケースにどのように対応するかをテストできます。

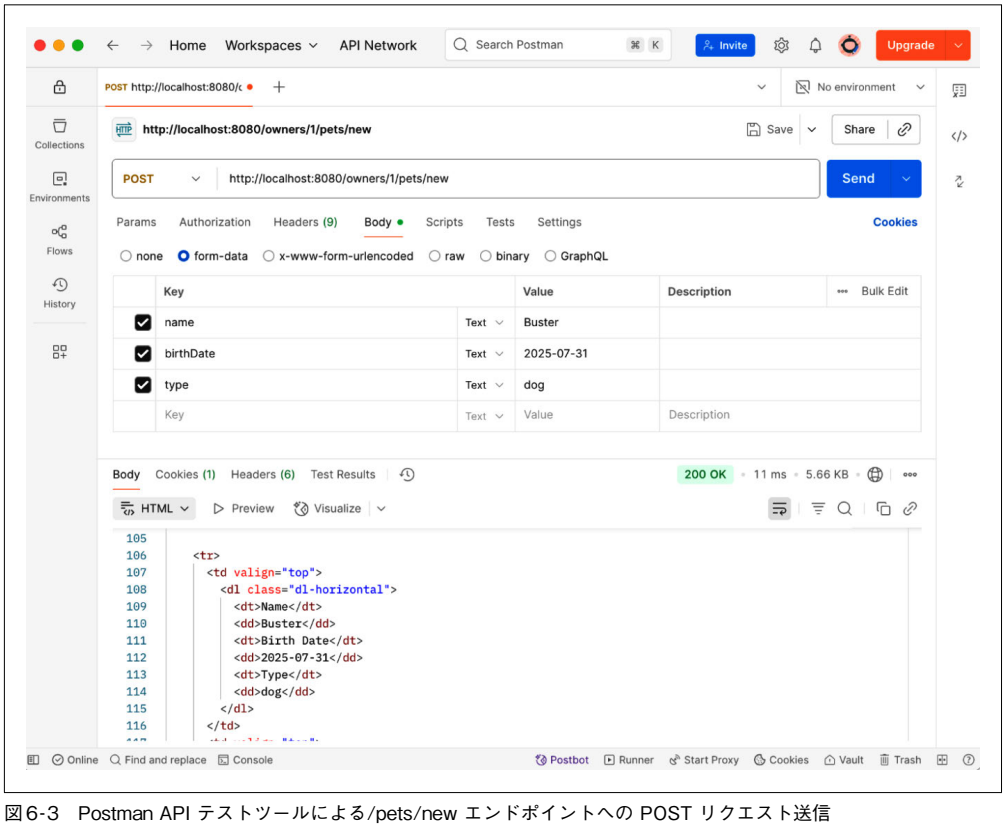


図6-3　Postman API テストツールによる/pets/new エンドポイントへの POST リクエスト送信

認証と認可

保護されたエンドポイントが、認証されていないリクエストを適切に拒否し、有効な認証情報を正しく受け入れることを確認できます。

レスポンスの構造とステータスコード

リクエストが成功した場合と、さまざまな種類のエラー（400、401、404、500 など）が発生した場合で、どのようなデータが返されるかを比較検証できます。

エラーメッセージの品質

エラー応答が、デバッグに役立つような、明確で具体的なフィードバックを提供しているかどうかを評価できます。

性能特性

応答時間を測定することで、異常に遅いエンドポイントを特定できます。

ロギング

ログは、システムの内部動作を理解する手がかりとなります。多くのアプリケーションにおいて、本番環境で発生した問題をデバッグする際や、リクエストの流れを追跡する際に、ログしか利用できない場合もあります。コードを確認する際は、次のようなロギング文を探してください。

```
log.info("Finding owner with id: {}", ownerId);
```

次のエンドポイントは、特定の飼い主が所有するすべてのペットを一覧表示します。有効な飼い主 ID を指定して、このエンドポイントにリクエストを送信すると、ログに 2 行が追加されます（**図6-4** 参照）。また、中括弧の部分が実際の値に置き換わっていることにも注意してください。

```
@GetMapping("/owners/{id}/pets")
public List<Pet> findAllPets(@PathVariable("id") int ownerId) {
   log.info("Finding owner with id: {}", ownerId);
   Optional<Owner> owner = ownerRepository.findById(ownerId);
   if (owner.isPresent()) {
      var pets = owner.get().getPets();
      log.info("Found {} pets for owner", pets.size());
      return pets;
   }
   return null;
}
```

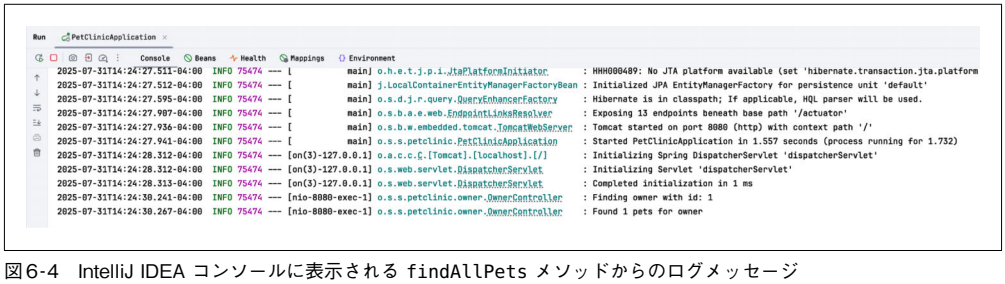


図6-4　IntelliJ IDEA コンソールに表示される `findAllPets` メソッドからのログメッセージ

デバッグ

IDE のデバッグ機能は、コードをステップ実行する際に非常に役立ちます。特に、複雑なビジネスロジックを理解する必要がある場合には、その価値は計り知れません。先ほど取り上げたペット作成時の検証処理をもう一度見てみましょう。

```
@PostMapping("/pets/new")
public String processCreationForm(Owner owner, @Valid Pet pet,
BindingResult result,
      RedirectAttributes redirectAttributes) {
```

```
    // ブレークポイントをここに設定する
    if (StringUtils.hasText(pet.getName()) && pet.isNew()
    && owner.getPet(pet.getName(), true) != null)
        result.rejectValue("name", "duplicate", "already exists");

    // その他の検証処理
}
```

アプリケーションが新しいペットの登録を受け付けないことがわかったとして、それはなぜでしょうか。この条件式は複数の部分から構成されており、重複した名前によるエラーが発生するには、そのすべてが真である必要があります。この行にブレークポイントを設定してステップ実行を行えば、以下のようなことを確認できます（**図6-5** 参照）。

各条件の評価結果

`StringUtils.hasText(pet.getName())` は真を返していますか？ `pet.isNew()` は真ですか？

オブジェクトの状態

この飼い主はすでにどんなペットを飼っていますか？ 具体的にどのような名前を確認しているのでしょうか？

メソッド呼び出しの結果

`owner.getPet(pet.getName(), true)` は、実際には何を返しますか？

IDE のデバッグツールを使用して複雑なビジネスロジックをステップ実行すると、デバッガーはコードが**どの**経路をたどるかを示すだけでなく、**なぜ**その経路をたどるのかという理由も明らかにしてくれます。これは、動作が期待どおりでない場合に特に役立ちます。例えば、ログにペットの名前がすでに存在すると表示されているにもかかわらず、それが間違いだと確信している場合、デバッガーを使えばこれを直接確認できます。

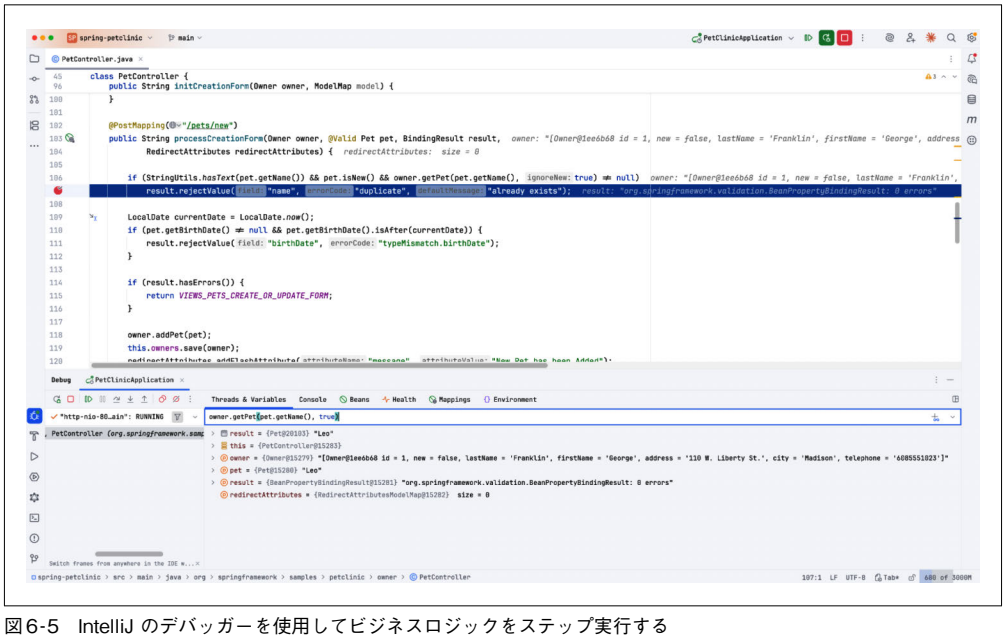


図6-5　IntelliJ のデバッガーを使用してビジネスロジックをステップ実行する

6.1.2.3　外部依存関係を特定する

完全に単独で動作するアプリケーションは、ほとんどありません。モダンアプリケーションは、公開 API を介して外部サービスを利用したり、データをやり取りするために内部サービスに依存したりしています。実行フローを理解するうえで重要なのは、アプリケーションが通信を行うあらゆる外部依存関係を特定することです。これには、他の API やデータベース、キャッシュなどが含まれます。

アプリケーション全体をまだ完全に理解していないとすれば、これらの依存関係をどのように特定すればよいのでしょうか？ まず手始めに設定ファイルを確認するのが効果的です。Spring Boot で構築された PetClinic アプリケーションでは、`application.properties` または `application.yaml` という設定ファイルを確認することで、どのような依存関係が宣言されているかを確認できます。次の例の場合、このアプリケーションが PostgreSQL データベース、外部サービス、メールホストに依存していることがわかります。

```
# データベース
spring.datasource.url=jdbc:postgresql://db-server:5432/petclinic \
  spring.datasource.driver-class-name=org.postgresql.Driver
# 外部サービス
payment.service.endpoint=https://payments.example.com/api/v2
# メールホスト
email.service.host=smtp.company.com
```

コンテナ化されたアプリケーションの場合、Docker Compose ファイルや Kubernetes マニフェストを確認して、関連するサービスを特定してください。

```
# docker-compose.yml ファイル内
services:
  app:
    image: petclinic:latest
    depends_on:
      - mysql
      - redis
      - kafka
```

6.1.2.4 内製のフレームワークとライブラリを特定する

既存のコードベースを調査するうえで最大の課題の一つは、社内で開発されたフレームワークやライブラリにたどり着くことです。これらは、特定のビジネス上の課題や技術的な課題を解決するために、社内で構築された独自のソリューションです。作成当時は適切な判断だったかもしれませんが、コードベースを調査・修正しようとする新しいチームメンバーにとっては、大きな障壁となることがよくあります。

チームはなぜ、外部のソリューションを採用するのではなく、社内で何かを構築することを決めるのでしょうか？ 内製フレームワークが生まれるのは、一般的に以下のような理由からです。

- 当時のチームは、既存ライブラリでは提供されていない機能が必要でした。
- 特定のセキュリティやパフォーマンスに関する要件がありました。
- 外部ソリューションを導入するには総コストが高すぎました。
- 時間の経過とともに変化した問題を迅速に解決するために、独自のシステムを構築しました。
- 解決策が存在することを知らなかったか、あるいはその内容を十分に理解していませんでした。

> メタ作業は、実際の作業よりも面白いものです。
>
> — Neal Ford、Thoughtworks 社ディレクター兼ソフトウェアアーキテクト、著述家、講演者

残念ながら、こうしたソリューションは、ドキュメントが不十分であったり、元々の開発者に連絡が取れなかったり、保守が一貫して行われていなかったりするといった問題を抱えていることがよくあります。

内製フレームワークを特定する

内製フレームワークは、コードベースに直接組み込まれているため、外部依存関係よりも見つけ

にくい場合があります。次のようなものをヒントに、内製のソリューションを見つけるようにしてください。

パッケージ名のつけ方

`com.yourcompany.framework`、`utils.common.core`、`internal.shared` といった、企業固有のプレフィックスや一般的な名前がついたパッケージを探します。

カスタム基底クラス

`BaseService`、`AbstractController`、`CommonEntity` などの名前のカスタム基底クラスを継承するクラスが多数ある場合は、フレームワークのコードであることを表しています。

カスタムアノテーション

一般的なフレームワークに含まれていない非標準のアノテーションは、多くの場合、独自のソリューションを表しています。

```
@Entity
@CompanyTable(name = "users", audit = true)
public class User extends BaseCompanyEntity {

    @CompanyField(encrypted = true)
    private String email;

}
```

適用範囲が広いユーティリティクラス

アプリケーション全体でさまざまな役割を担う `AppUtils`、`BusinessHelper`、`SystemManager` などの名前のクラスが存在することがよくあります。

複雑な設定

標準ライブラリのパターンに合致しない、大規模なカスタム YAML ／ XML ファイルや設定クラスが存在します。

内製フレームワークへの取り組み

以下のヒントを参考に、内製フレームワークを調査し、コードを元の状態よりも良い状態に整えてください。

まずは使用例を探してみる

コードベースを検索して、他の開発者がこのフレームワークをどのように活用しているかを確認してください。実際の事例は、ドキュメントよりも参考になることがよくあります。

テストを探す

フレームワークのコードは、複数のシナリオで動作する必要があるため、テストカバレッジが比較的高くなる傾向があります。テストを見ることで、意図している動作やエッジケースが明らかになります。

学びながらドキュメント化する

フレームワークの機能や使用方法についてメモを作成してください。基本的なドキュメントであっても、将来の開発者の助けになります。

バージョン管理の履歴を確認する

`git blame` やコミットメッセージを活用して、そのフレームワークがなぜ構築されたのか、そしてどのように進化してきたのかを理解しましょう。

トレードオフを検討する

そのフレームワークが依然として価値を提供しているのか、それとも保守の負担になりつつあるのかを検討してください。場合によっては、独自のソリューションを標準ライブラリに置き換えることが、長期的な観点からは正しい選択となることもあります。

少しずつ改善していく

フレームワークを使いながら、命名規則の改善、ドキュメントの追加、テストカバレッジの向上などを行い、スカウトのルール（詳細は後ほど説明します）を実践してください。

6.1.3　メンタルモデルを段階的に構築する

メンタルモデルとは、システムがどのように機能するかについて、自分自身の中に構築したイメージのことです。モデルをしっかりと構築するには、時間と意識的な練習が必要です。システムのメンタルモデルを構築する場合、システムを段階的に分解していくと、メンタルモデルの構築ははるかに容易になります。例えば、EC アプリケーション全体のメンタルモデルを構築しようとせず、決済処理などの単一のフローを取り上げ、そこから構築していくのがよいでしょう。また、ツールや完璧主義に邪魔されないようにすることも重要です。多くの場合、メモ用紙に手書きで描いたシンプルなモデルで十分です。

この節では、複雑なシステムを小さなシステムに分解したり、それらのモデルを可視化したり、少しずつ理解を深めていくなど、メンタルモデルを構築するための実践的な方法を学びます。



AI について

昨今、テキストベースの図を作成する場合、生成 AI を活用して最初の草案を作成し、それを手作業でさらに磨き上げるという手法を使えるようになりました。例えば、AI ツールにシステムやプロセスを説明し、Mermaid 構文を生成するよう依頼した後、その出力結果を自分のニーズに合わせて調整するのです。

6.1.3.1　複雑なシステムを分解する

複雑なシステムを分解することは、メンタルモデルを構築するうえで重要です。大規模なコードベースのプロジェクトに新たに参加する際、**すべてを一度に理解しようとすると、負担が大きすぎて非効率的です**。その代わりに、複雑なシステムを扱いやすい単位に分解することに注力してください。

前節でアプリケーションのエントリーポイントについて学びましたが、このエントリーポイントから始めるとよいでしょう。例えば、EC アプリケーションでは、以下のような実行フローを定義していることでしょう。

新規顧客フロー

新規ユーザーが登録し、顧客となり、プロモーションクーポンを受け取ります。

決済フロー

ユーザーがカート内の商品の購入を完了するとどのようなことが起こるのでしょうか。

商品レビューフロー

顧客が購入した商品についてフィードバックを提供します。

このように、複雑なシステムを理解しやすい小さな単位に分割できます。新しい商品レビューがどのように記録されるかを確認し、その手順に関連するコードを見れば、この処理についてメンタルモデルを構築できます。学習方法は人それぞれです。もしかしたら、処理ではなくコードを中心にメンタルモデルを構築したいという人もいるでしょう。アプリケーションが機能別パッケージングを採用しているのであれば、機能ごとにモデルを構築するとよいでしょう。次のようなパッケージング構成であれば、**カート**機能に関連するすべてのコードをまとめて確認できます。

```
src/
├── cart/
│   ├── CartController.java
│   ├── CartService.java
│   ├── CartRepository.java
│   ├── Cart.java
│   ├── CartItem.java
│   └── dto/
│       ├── CartDTO.java
│       └── CartItemDTO.java
```

6.1.3.2　メンタルモデルを可視化する

コード間の関係を可視化することで、こうしたメンタルモデルを構築しやすくなります。4 章では、ソフトウェアモデリングと、利用可能な図の種類について学びました。可視化は本格的なものである必要はありません。簡単なスケッチから完全な UML 図に至るまで、どのような形でもかま

いません。システムを理解するのに役立つレベルで作成してください。メンタルモデルを可視化する第一歩は、可視化の形式を選ぶことです。ソフトウェアエンジニアリングでは、さまざまな可視化手法があり、それぞれさまざまな目的で使われます。

フローチャート
: 一連のプロセスや分岐点について理解しやすくする

エンティティ・リレーションシップ図（ER 図）
: データ構造とその関係を明確にする

シーケンス図
: コンポーネントがどのようにやり取りするのかを時間軸で表す

コンポーネント図
: 全体的なアーキテクチャと依存関係を示す

マインドマップ
: 関連する概念を階層的に整理する手助けをする

まずシンプルな図から始め、システムへの理解が深まるにつれて詳細度を上げていってください。例えば、EC システムの決済フローを可視化する際、最初は主要なコンポーネントだけから始めるのがよいでしょう。以下は、Mermaid 構文で記述されたシンプルなフローチャートです。Mermaid は、人気のある Markdown ベースの図作成ツールです。

```
graph TD
    A[ショッピングカート] --> B[決済フォーム]
    B --> C[支払処理]
    C --> D[注文の確認]
```

このようにコードを記述すれば、**図 6-6** のようなフローチャートが生成されます。

理解が深まるにつれ、このフローチャートを拡張し、具体的なサービス、データベースとのやり取り、外部システムとの通信などを追加できます。以下に例を示します。

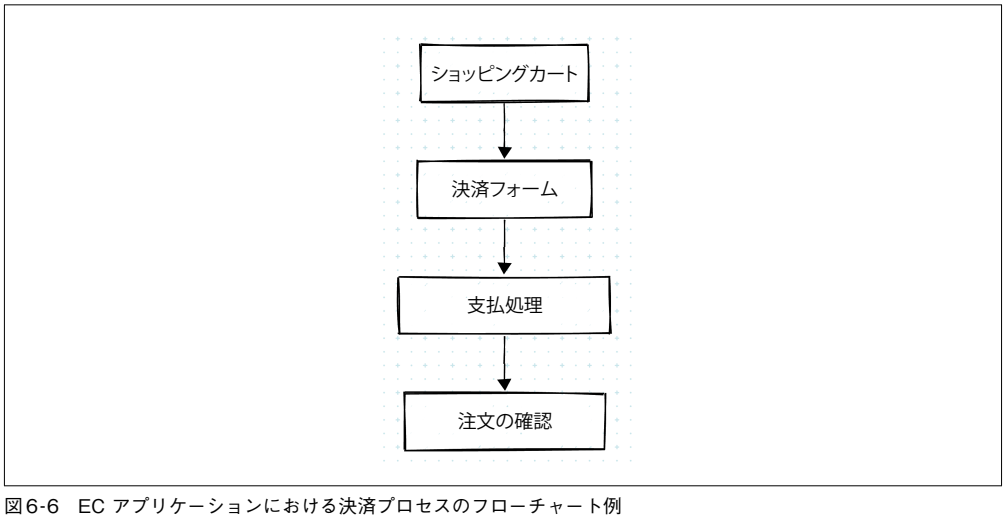


図6-6　EC アプリケーションにおける決済プロセスのフローチャート例

```
sequenceDiagram
    participant U as ユーザー
    participant FE as フロントエンド
    participant API as API レイヤー
    participant CS as CartService
    participant PS as PaymentService
    participant OS as OrderService
    participant DB as データベース
    participant PP as 決済サービスプロバイダー

    U->>FE: 「決済」をクリック
    FE->>API: POST /checkout
    API->>CS: validateCart()
    CS->>DB: getCartItems()
    DB-->>CS: カート中の商品
    CS-->>API: 確認結果
    API->>PS: processPayment()
    PS->>PP: authorizeCharge()
    PP-->>PS: 認可結果
    PS-->>API: 決済結果
    API->>OS: createOrder()
    OS->>DB: saveOrder()
    DB-->>OS: 注文の確認
    OS-->>API: 注文の詳細
    API-->>FE: 成功レスポンス
    FE-->>U: 確認画面の表示
```

このコードを使うと、**図6-7** に示すようなシーケンス図が生成されます。

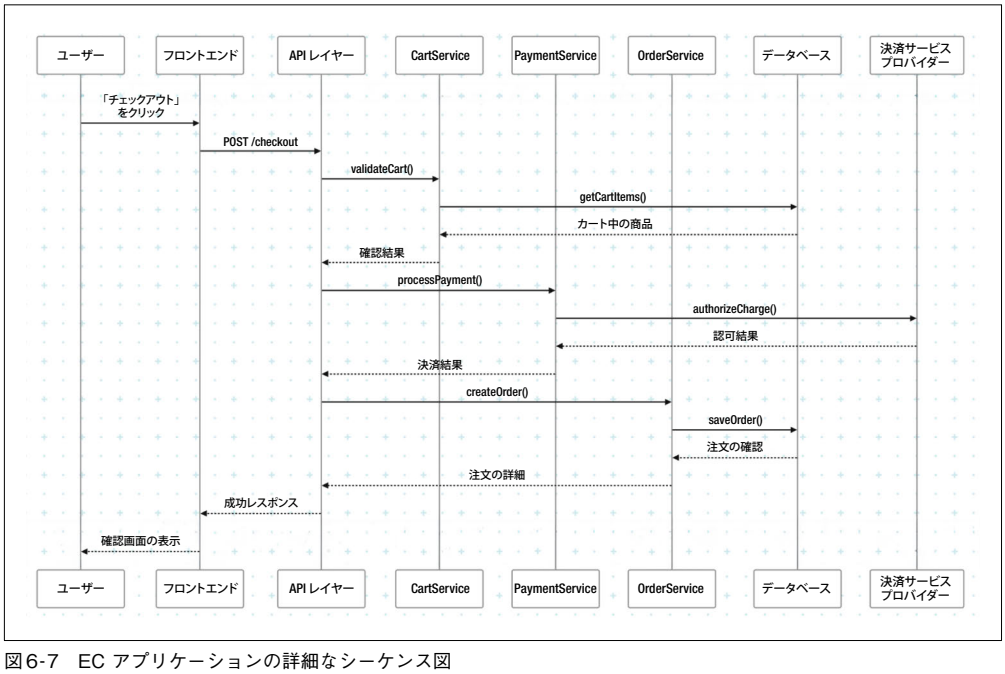


図6-7　EC アプリケーションの詳細なシーケンス図

6.1.3.3　スコープを少しずつ広げる

個々のコンポーネントを理解したら、次に示すようにスコープを少しずつ広げて、システム全体の挙動を見ていきましょう。

1. まずは個別のメソッドから始めます。
2. 次に、クラスとその直接的な依存関係に移ります。
3. その後、複数のコンポーネントにまたがる機能レベルのフローへと進みます。
4. 最後に、セキュリティやトランザクション処理など、横断的な関心事を理解します。

このように少しずつ広げていく方法を採用すれば、学習の負担を軽減しながらシステムの全体像を把握できます。

正確なメンタルモデルを構築するために時間を割くことで、コードベースを変更する際に、より適切な判断ができるようになります。これは継続的なプロセスであり、時間がかかるものであるということを忘れないでください。経験豊富なチームメンバーであっても、システム全体が進化するにつれて、システムへの理解を深めています。初めて触れるコードベースを理解する方法を学んだところで、既存のコードを扱う際の具体的な手順について例を見ていきましょう。

6.2　手順の例

経験を積むにつれて、既存のコードを扱う感覚が身についていくことでしょう。とはいえ、手順を定めておくと効率的になります。Christopher Judd（https://oreil.ly/G4f5f）は、自身のブートキャンプで以下のような手順を教えています。

1. ソースコード管理システムからプロジェクトをクローンします。
2. README、コーディング規約、アーキテクチャ、その他の関連ドキュメントを確認します。
3. 作業を進めながら、メモ（アーキテクチャ、ビルドコマンド、よく使われるSQL、業界標準、用語など）を取ります。これらのメモは、必ずチームメンバーと共有してください！
4. ビルドスクリプトを確認します。
5. プロジェクトの依存関係を確認します。
6. プロジェクト構造（パッケージ、名前空間、モジュール、成果物など）を確認します。
7. CI/CDパイプラインを確認します。
8. プロジェクトの依存関係（ビルドツール、ランタイム、言語）をインストールします。
9. IDEを使って以下の作業を行います。
    a. アプリケーションを実行し、デバッグします。
    b. 調査したいメソッドにブレークポイントを設定し、アプリケーションの実行パスと関連付けます。
    c. ユニットテストを実行しデバッグします。
    d. 調査したいメソッドにブレークポイントを追加し、アプリケーションの実行パスと関連付けます。
10. コマンドラインから以下の作業を行います。
    a. プロジェクトの成果物をビルドします。
    b. ユニットテストを実行します。
    c. 必要なコンテナ（データストアなど）を起動します。
    d. アプリケーションをローカル（ポート、URLなど）で実行します。

既存のコードベースを理解することは、反復的な作業になります。一度読んだだけではすべてを理解することはできませんが、それはごく自然なことです。課題や新しい機能に取り組むたびに、より深い知識が身につくでしょう。

6.3　安全に変更する

既存のコードベースを理解するためのコツと手順を学んだところで、次は安全にコードを修正する方法を学びましょう。既存のシステムは、おそらく数多くの改良や機能追加を経てきたことでしょう。まるで、荒くれ者のボクサーのようにリングに飛び込んで、最初のラウンドですべてのエ

ネルギーを使い果たしてしまうようなことは避けなければなりません。コードベースを体系的に調査し、システムを壊さないように注意を払う必要があります。「素早く動いて、破壊せよ（move fast and break things）」という言葉を耳にしたことがあるかもしれません。この考え方が有効な状況もありますが、既存のシステムに変更を加える場合には当てはまりません。

6.3.1 安全にリファクタリングする

レガシーシステムや既存のコードベースを扱う際、コードの可読性向上やパフォーマンス改善などの目的でコードのリファクタリングを行うことがよくあります。このような場合には、しっかりと計画を立て、一度にやりすぎないよう注意する必要があります。

6.3.1.1 既存のテストを活用する

前節で学んだように、テストはコードベースを理解するうえで大きな役割を果たします。そして、コードのリファクタリング作業をより安全かつ容易にするものでもあります。プロジェクトに包括的なテストスイートが用意されている場合は、それを大いに活用すべきです。テストはセーフティネットとしての役割を果たし、小さな変更を加えた後にテストを実行することで、プロジェクトの他の部分に悪影響を与えていないことを確認できます。

次の例では、`calculateTotal` メソッドをリファクタリングし、小計の計算と割引の適用を分離しています。リファクタリングを行う際は、動作を維持しつつコード構造を改善することが目的であるため、テストを変更しないようにしてください。リファクタリングを行った後には、このメソッドに関連するテストを実行し、動作に変更がないことを確認します。

```
// リファクタリング前
public double calculateTotal(Order order) {
    double total = 0;
    for (OrderItem item : order.getItems()) {
        total += item.getPrice() * item.getQuantity();
    }
    if (order.hasDiscount()) {
        total = total * (1 - order.getDiscountRate());
    }
    return total;
}
```

```
// リファクタリング後
public double calculateTotal(Order order) {
    double subtotal = calculateSubtotal(order);
    return applyDiscount(subtotal, order);
}

private double calculateSubtotal(Order order) {
    return order.getItems().stream()
        .mapToDouble(item -> item.getPrice() * item.getQuantity())
```

```
        .sum();
}

private double applyDiscount(double amount, Order order) {
    return order.hasDiscount()
        ? amount * (1 - order.getDiscountRate())
        : amount;
}
```

6.3.1.2　リファクタリング前にテストを追加する

前の例では、プロジェクト全体に影響が及ばないよう、テストを活用しました。これから行おうとする変更に対してアプリケーションのテストカバレッジが不十分な場合、既存のコードをリファクタリングする前にテストを追加してください。この手法は、**テスト駆動リファクタリング**とも呼ばれ、以下のような手順で行います。

1. 現在の動作をドキュメント化するテストを作成します。
2. 現在の実装でテストが成功することを確認します。
3. コードをリファクタリングします。
4. 新しい実装でもテストが成功することを確認します。

以下のユニットテストは、割引の対象となる注文の合計金額を計算する際、注文サービスが正しく 10% の割引を適用することを確認するものです。

```
@Test
public void calculateTotalAppliesDiscountWhenOrderHasDiscount() {
    // 前提
    OrderItem item1 = new OrderItem("Product1", 10.0, 2);
    OrderItem item2 = new OrderItem("Product2", 15.0, 1);
    Order order = new Order(Arrays.asList(item1, item2), true, 0.1);

    // 操作
    double total = orderService.calculateTotal(order);

    // 結果
    assertEquals(31.5, total, 0.001); // (10*2 + 15*1) * 0.9 = 31.5
}
```

変更を加える前にこのようなテストを追加しておくと、既存の動作を理解するのに役立ち、リファクタリングを行ってもその動作が維持されるという確信が持てます。



既存のコードベースにテストを追加する際、第一の目的は既存の動作をドキュメント化することであり、それを変更することではないということを肝に銘じておいてください。たとえコードをリファクタリングするより良い方法を知っていたり、間違いなくバグだと確信している箇

所を修正したりしたい場合でも、まずは現在の動作を確認するためのテストを作成すべきです。テストが整った後で、改めてバグの修正やさらなるリファクタリングに取り組むようにします。

6.3.2 スカウトのルール

スカウトのルールとは、ソフトウェア開発における原則の一つで、「コードは、見つけたときよりも良い状態で残すこと」というものです。これは、ボーイスカウトアメリカ連盟におけるキャンプの理念に由来するものです。スカウトたちはキャンプ場を、自分たちが来たときよりもきれいに片付けて去るよう教えられています。

スカウトのルールでは、ファイルに手を加えるたびに、コードの品質を少しずつ改善することを推奨しています。こうした改善には、次のようなものがあります。

- 不足しているドキュメントを追加する
- わかりやすさを考慮して変数名やメソッド名を変更する
- 複雑すぎるメソッドを分割する
- 不要なコードを削除する
- 発見した軽微なバグを修正する

次の例では、メソッド名を改善し、型パラメータを追加し、拡張 `for` ループを使用し、ドキュメントを追加しています。こうすることで、コードのコアとなる機能を変えることなく、保守性を大幅に向上させています。

```
// スカウトのルール適用前
public void prcs(List l) {
    for (int i = 0; i < l.size(); i++) {
        Object o = l.get(i);
        // オブジェクトを処理する
        // ...
    }
}
```

```
// スカウトのルール適用後
/**
 * 顧客レコードのリストを処理し、データベース内のステータスを更新する。
 *
 * @param customers 処理対象の顧客のリスト
 */
public void processCustomers(List<Customer> customers) {
    for (Customer customer : customers) {
        // customer（顧客）を処理する
        // ...
    }
}
```

スカウトのルールでは、作業を進めながらコードを整理することを推奨しています。ただし、コードの整理と目の前のタスクとのバランスを見極める必要があります。大規模なコードベースでは、コードの改善に時間を費やしすぎてしまい、結局、当初修正しようとしていた問題を解決できなかったということになりかねません。

改善の余地があるコードすべてを、すぐに改善すべきとは限りません。リファクタリングを行うかどうかを決める際には、以下の点を考慮してください。

以下のような状況では、コードのリファクタリングを行ってもよいでしょう。

- 現在取り組んでいるコードが理解しにくい。
- その部分に機能を追加しなければなりません。
- コードのバグを修正しています。
- コードがパフォーマンスの問題を引き起こしています。
- 複数の開発者が頻繁にその部分で作業しています。

以下のような場合には、リファクタリングを先送りすることを検討すべきです。

- コードは問題なく動作しており、変更が必要になることはほとんどありません。
- 納期が迫っており、時間的余裕がありません。
- コードを壊してしまうリスクが、そのメリットを上回ります。
- リファクタリングを行うには、現在のタスクの範囲を超えるような大規模な変更が必要となります。
- 安全性を確保するには、テストカバレッジが十分ではありません。

6.3.3　小さく、元に戻せるように変更する

システム全体に影響を与えることなく既存のコードを変更するには、段階的に、容易に元に戻せるように変更するのが最も安全な方法です。このようなスキルを習得するには、継続的な実践が不可欠です。複雑な問題を一度に解決したくなるかもしれません。プロダクトオーナーからはより大規模で影響力の大きい変更を求める圧力がかかるかもしれません。しかし、読者は、システム全体に混乱をもたらさないように、小規模な変更を行うようにすべきです。

6.3.3.1　変更管理戦略

ソフトウェアチームにおける**変更管理戦略**とは、アプリケーションの変更に対する体系的なアプローチです。コードの品質を維持し、チーム内の足並みを揃えるために、変更の提案、検討、実装、および追跡を行うための仕組みです。小規模で元に戻せる変更を基本とするのであれば、変更管理戦略では安全性と可視性を最優先すべきです。

以下に、変更管理における実践的な戦略をいくつか紹介します。

変更内容を可視化する

変更内容は、十分にドキュメント化し、他者が理解しやすいようにしてください。詳細に記録する必要はありませんが、変更の理由と、その実装方法については説明するようにしてください。

検証機能を組み込む

変更内容が期待どおりに機能することを確認する方法を組み込んでください。チームによる検証方法としては、自動テスト、コードレビュー、静的解析などが挙げられます。

ロールバックを計画する

マーフィーの法則によれば、「うまくいかないことは必ずうまくいかない」と言われています。問題が発生した場合に備え、変更をロールバックする計画を常に用意しておいてください。今回提案しているような小さな変更であれば、ロールバックする作業も比較的容易になるはずです。

効果を監視する

変更によって予期しない影響が生じていないか注意してください。そのためには、アプリケーションのオブザーバビリティを高めておくことが重要です。

6.3.3.2 テスト可能な小さな単位で変更する

テスト可能な小さな単位で作業するという考え方を身につければ、既存のコードベースを修正する際のストレスが軽減され管理しやすくなります。このアプローチを採用すると、次のようなメリットが得られます。

- 変更の範囲を限定することでリスクを低減する
- テストをより的を絞った効果的なものにする
- コードレビューが進めやすくなる
- 問題が発生した場合のトラブルシューティングを容易にする

言い換えれば、機能追加の規模が大きすぎる場合は、その機能をそのまま実装するのではなく、より小さな機能に分割したほうがよいでしょう。そのほうが変更を管理しやすくなります。バグを引き起こすような大規模な変更を１つ行うよりも、小規模で確実に機能する変更をいくつか提供したほうがよいということを忘れないでください。

例えば、システムの注文処理方法を変更してほしいという依頼があったとしましょう。以下の例では、`processOrder` メソッドの処理内容が多すぎるので、リファクタリングの対象となり得ることがわかります。メソッド全体のロジックを変更しようとするのではなく、一部の機能をより小さなメソッドに分割します。そして、それらのメソッドに対してテストを作成し、それぞれが正しく動作することを確認してから、次のメソッドに進むようにしてください。

```
// このメソッド全体を一度に変更するのではなく…
public void processOrder(Order order) {
    // 注文内容の確認
    // 合計金額の計算
    // 割引の適用
    // 配送料の計算
    // 税金の適用
    // 支払処理
    // 在庫の更新
    // 確認メールの送信
}
```

```
// より小さな変更に分割する
// 1. まず、注文内容の確認ロジックを抽出する
private boolean validateOrder(Order order) {
    // 注文内容の確認ロジックをここに記述する
    return isValid;
}

// 2. 次に、合計金額の計算を抽出する
private double calculateOrderTotal(Order order) {
    // 合計金額の計算ロジックをここに記述する
    return total;
}

// 3. 残りのロジックの抽出を続け、変更のたびにテストを行う
```

6.3.3.3　バージョン管理のベストプラクティス

新たに読者が参加するプロジェクトのほとんどには、おそらくすでにバージョン管理システムが導入されていることでしょう。しかし、状況によっては、適切なバージョン管理が行われていない場合もあります。また、一からベストプラクティスを確立する必要があるかもしれません。

レガシーなコードベース

バージョン管理は行われているものの、ベストプラクティスが守られていない古いプロジェクトに巡り合うかもしれません。こうしたアプリケーションでは、ブランチ戦略が確立されていないまま、すべてを main ブランチにコミットしている場合があります。コミットメッセージに一貫性がなかったり、マージされていない変更が大量にあったり、そもそもバージョン管理すべきではない生成ファイルや依存関係がリポジトリに残っていたりするかもしれません。

個人プロジェクトや小規模なプロジェクト

個人プロジェクトを始める際や、ごく少人数のチームで作業する際は、バージョン管理を省略したくなるかもしれません。「自分一人だけだし」とか「直接コミュニケーションが取れるから」と思うかもしれません。しかし、早い段階で良い習慣を身につけておくことで、技

術的負債を防ぎ、プロジェクトが大きくなったり、他のメンバーを迎え入れたりする際にスムーズに進められます。

オープンソースへの貢献

リポジトリをフォークしたり、オープンソースプロジェクトに貢献したりする際は、プロジェクトのバージョン管理に関する規約を理解してください。そして、その規約に従いながら、自身のフォークを適切に管理しなければなりません。

請負業務またはコンサルタント業務

既存のプロジェクトに参加する際は、これまではあまり重視されていなかったバージョン管理の取り組みを確立したり、改善したりする必要が生じる場合があります。

どのような状況にあっても、Git のようなバージョン管理システムは、既存のコードベースに安全に変更を加えるために不可欠です。バージョン管理システムを効果的に活用すれば、作業が失敗したり、機能を壊してしまったりすることを恐れることなく、実験を行ったり、履歴を追跡したり、共同作業を行ったりできます。

戦略的にコミットする

既存のコードを扱う際は、コミットによって、そのコードがどのように進化したのかが伝わるようにすべきです。

アトミックコミットを行う

各コミットは、そのコミットだけで成立するような単一の論理的な変更を表すべきです。こうすることで、変更内容の理解やレビューが容易になり、必要に応じて変更を元に戻すことも簡単になります。

意味のあるコミットメッセージを記述する

適切なコミットメッセージには、何が変更されたか、そして**その理由**の両方が明記されています。将来の開発者（将来の自分自身も含めて）は、このような内容がコミットメッセージに含まれていれば変更を理解しやすくなるでしょう。以下に、良いコミットメッセージと悪いコミットメッセージの例を示します。

```
# 不適切なコミットメッセージ
バグ修正

# より良いコミットメッセージ
割引コードが 2 回適用された場合の注文計算を修正。

顧客がモバイルとデスクトップの両方のセッションで割引コードを
入力した場合、システムが割引コードを二重に適用していました。
この修正により、1 注文につき割引が 1 回のみ適用されるようになりました。
```

こまめにコミットする

何十もの変更を加えるまで待たずにコミットしてください。小さな単位で論理的なコミットをこまめに行えば、問題がいつ、どこで発生したのかを特定しやすくなります。

ブランチを賢く使う

ブランチ戦略は組織、チーム、プロジェクトによって異なります。一般的に、既存のコードを扱う際には、以下のような原則が適用されます。

フィーチャーブランチを作成する

変更内容は、マージの準備が整うまで専用のブランチに分離してください。こうすることで、main ブランチの安定性を保ちつつ、自由に試行錯誤を行えます。

ブランチの存続期間を短くする

長期にわたって保守されるブランチは、時間の経過とともに main ブランチのコードベースから乖離していき、統合が難しくなります。変更内容は数週間ではなく、数日以内にマージするように心がけてください。

マージする前にリベースする

必要に応じて、変更内容を最新のメインブランチを基にリベースし、コードベースの最新バージョンで作業していることを確認してください。**注意：**リベースを行うとコミット履歴が書き換えられるため、共有ブランチでは慎重に行ってください。

プルリクエストとコードレビュー

プルリクエストとは、あるブランチの変更内容を別のブランチにマージするよう提案するものです。統合前にレビュー、議論、および自動チェックが行えるようにするためのものです。プルリクエストは焦点を絞って、管理しやすいものにしてください。20 ファイルにわたる 1,000 行ものコードを変更するプルリクエストは、レビューが困難です。その代わりに、単一の課題に対処する、小規模で焦点を絞ったプルリクエストを心がけましょう。

コードレビューでは、どのような問題を解決しようとしているのか、そして変更内容がどのようにその問題に対処しているのかを必ず説明してください。関連する課題や要件への対応があれば、それらについても記述するようにしてください。コードレビューは、開発者を批判するためではなく、解決策を改善するためのものです。提案には柔軟に対応し、自身のやり方を改善していく姿勢を持つようにしてください。

ここまで学んだように、既存のコードベースを軽率に修正すべきではありません。しかし、手を加えることを恐れる必要もありません。テストを活用して安全にリファクタリングを行い、「スカウトのルール」に従い、小規模で元に戻せる変更を行うようにしてください。

6.4 まとめ

既存のコードを扱うことは、単なるコーダーからソフトウェアエンジニアへと成長するための基本的なスキルの一つです。本章では、コードベースを把握し、理解し、安全に修正するための戦略について説明しました。全体像を把握することから始め、ドキュメントを確認し、プロジェクトのアーキテクチャを理解することで、初めて触れるコードベースを理解する方法を学びました。また、エントリーポイントを見つけ、リクエストの流れを追跡し、外部依存関係を特定することで、実行フローをたどる方法についても取り上げました。複雑なシステムを分解し、関係性を可視化することで、段階的にメンタルモデルを構築する方法も習得したはずです。さらに、慎重なリファクタリング、スカウトのルールに従うこと、そしてバージョン管理のベストプラクティスを活用することで、安全に変更を加える方法についても検討しました。

これらのスキルは、単に「あれば便利なもの」というだけでなく、ソフトウェアエンジニアとしての成長に不可欠なものです。ゼロから新しいアプリケーションを作成するのは楽しく刺激的です。しかし、現実には、キャリアの大半は既存のコードを扱う作業に費やされることになります。これらの技術を習得すれば、コードベースの規模や複雑さにかかわらず、迅速かつ自信を持って貢献できるようになります。

6.5 実践に移す

学んだことを実践するには、知識だけでなく行動が必要です。以下に紹介するような手法は、既存のコードを扱う際に、コードの閲覧や修正のスキルを向上させるのに役立ちます。

1. 興味のあるオープンソースプロジェクトを見つけ、コードを書かずに 30 分間、そのコードベースを調べてみましょう。その構造や構成を理解することだけに集中します。
2. Web アプリケーション（「PetClinic」の例など）を選択し、ブラウザの開発者ツール、ロギング、デバッグを順に活用して、UI 操作から API 呼び出し、データベースの変更に至るまでのユーザー体験の全行程を追跡する練習を行ってください。
3. オープンソースプロジェクトから理解しにくいメソッドを 1 つ選び、その動作を変えずに可読性を高めるようリファクタリングしてください。
4. テストカバレッジが非常に小さいレガシープロジェクトを見つけ、既存の動作をドキュメント化できるよう、テストを追加する練習をしましょう。
5. 実際のコードベースからビジネスプロセスを 1 つ選び、コードが実際にどのように流れるかを示す視覚的な図（手書きの図、フローチャート、シーケンス図、コンポーネント図など）を作成し、チームメンバーと確認してください。
6. 既存のコードベースに複雑な変更を加える際は、小規模で元に戻せるコミットを心がけてください。また、自身のコミット戦略について、同僚からフィードバックをもらってください。
7. 現在取り組んでいるプロジェクトに「スカウトのルール」を適用し、コードを元の状態よりも

良い状態にするために、3つの小さな改善点を見つけましょう。

8. プロジェクト内の外部依存関係を特定し、影響分析やロールバック計画を含め、それを安全にアップグレードまたは置き換える方法を調査することで、安全な依存関係の変更を実践してください。

6.6　関連リソース

- 『Refactoring:　Improving the Design of Existing Code』Martin Fowler 著（Addison-Wesley Professional、2018年）
  和書『リファクタリング（第2版）』（オーム社、2019年）
- 『Working Effectively with Legacy Code』Michael Feathers 著（Prentice Hall、2004年）
  和書『レガシーコード改善ガイド』（翔泳社、2009年）
- 『Getting to Know IntelliJ IDEA』Trisha Gee, Helen Scott 共著（JetBrains、2021年）

7章
ユーザーインターフェース設計

優れたデザインとは、デザイナーとユーザーとの間で交わされる
コミュニケーションでもあります。
ただし、そのコミュニケーションはすべて、デバイスのデザインだけを通じて
行われなければならないという点が異なります。
デバイスは、自らを説明できなければならないのです。
— Donald A. Norman、『The Design of Everyday Things』著者

あまりにも多くの場合、ユーザーインターフェース（UI）は、他のあらゆる要素を無視して、入力フィールドを単に「空いている場所」に配置するという設計になってしまっていました。開発者にとっては最も手っ取り早い方法だったかもしれませんが、その結果、使い勝手の良いアプリケーションになることはほとんどありませんでした。忘れてはならないのは、UI こそがユーザーエクスペリエンス（UX）の**「形」**であり、ほぼすべてのユーザーにとって、UI **こそが**アプリケーションそのものであるということです。優れた UI デザインとは、UX によって定められた目標を達成することです。

UI デザインとは、単に洗練された見た目や操作感を提供するだけのものではありません。ユーザーにとってアプリケーションを使いやすいものにすることです。使いやすさには、UI デザインのアクセシビリティや包摂性（インクルーシブデザイン）など、さまざまな要素が含まれます。もちろん、ソフトウェアエンジニアの多くは UI デザインの基礎を学んでいません。本章を読んだだけで専門家になれるわけではありません。それでも、本章を読み終える頃には、他の人たちよりも一歩先を行けるようになるでしょう！

7.1　すべての人に向けたデザイン

アクセシビリティ、ユーザビリティ、包摂性は、デザインにおいて異なる概念ではあるものの相互に関連し合うものであり、さまざまな背景や能力を持つ人々がプロダクトを利用できるようにすることを目的としています。これらは、アクセシビリティを「a11y」、ローカライゼーションは

「L10N」、国際化は「I18N」といった略語で呼ばれることがよくあります。



ヌメロニムとは、数字で部分的または完全に構成された単語のことです。さまざまな形態をとりますが、多くの場合、最初の文字と最後の文字の間のすべての文字を、その文字数に相当する数字に置き換えた短縮形として表現されます。正規表現が好きな人向けに言えば、この種のヌメロニムは `[a-zA-Z]\d{1,2}[a-zA-Z]` というパターンに一致します。

これらの要素のバランスを取り、設計プロセスを通じて統合し、さまざまなユーザーのニーズに効果的に対応することには、課題が生じることがあります。これらはアプリケーションの成否を左右する要因となり得ます。また、場合によっては、アクセシビリティを考慮した設計を義務付ける「障害を持つアメリカ人法（ADA、Americans with Disabilities Act）」、米連邦政府機関に対し情報技術のアクセシビリティ確保を規定する「リハビリテーション法」第508条、および障害を持つ人々の利便性を最大限に高めるよう製品の設計を義務付ける「欧州アクセシビリティ法（EAA、European Accessibility Act）」などの法令や規制の対象となることもあります。

後付け対応の難しさ

Nateです。数年前、私はあるソフトウェア会社のUIチームで働いていました。その会社には、プロのエンジニアに広く利用されている、定評のあるWebアプリケーションがありました。プロダクトマネージャーたちは、次期リリースではアクセシビリティを重点課題とすることを決定しました。そして、スクリーンリーダーやキーボード操作に対応できるよう、アプリケーションの改修を私たちのチームに指示しました。このアプリが、こうした概念を考慮せずに開発されたと言うのは、婉曲な表現に過ぎません。スクリーンリーダーを使って初めてページを確認したとき、その作業の膨大さが明らかになりました。

各フィールドには明確なラベルがなく、キーボードショートカットもほとんどありませんでした。マウスを使わずにアプリケーション内を移動するのは、非常に手間がかかりました。私が関わるよりずっと前に下された決定についてはコメントできませんが、最初からアクセシビリティ、ローカライゼーション、国際化を考慮して開発するほうがはるかに簡単であったことは言うまでもありません。とはいえ、このアプリは多言語対応という点では非常に優れていたと申し上げておきます！ そして、多大な労力を費やした結果、キーボード操作で（ほぼ）アプリを使えるようになり、少なくともスクリーンリーダーに対してはそれほど不親切なものではなくなりました。

以降の節では、ユーザビリティ、アクセシビリティ、ローカライゼーションについて説明していきます。なお、他のソフトウェアエンジニア向けにソフトウェアを開発しているのでない限り、自分自身はユーザー層の代表にはならないということを念頭に置いてください。さまざまな能力を持つユーザー、ハードウェアやソフトウェア、インターネット接続へのアクセス状況やその品質、そ

して地理的な場所などを考慮してください。読者の視力は鷹並みかもしれませんが、それは誰にでも備わっている能力ではありません。

7.1.1　ユーザビリティとは？

ユーザビリティは、ソフトウェアにおける「デンタルフロス」のようなものです。誰もがその重要性を理解しているにもかかわらず、プロジェクトでは真っ先に削られてしまう項目の一つです[†1]。運良く UI の専門家を頼れる環境にいる方もいるかもしれませんが、おそらくその作業の一部（あるいはすべて）を自ら担うことになるでしょう。一部の人々の考えとは裏腹に、ユーザビリティは重要です。少なくとも、ユーザーがアプリケーションから逃げ出したくなるような事態を避けたいのであれば、なおさらです。多くのソフトウェアエンジニアは、アルゴリズムやコンパイラ理論の実践については十分に学んでいますが、UI デザインの芸術と理論については基礎的な知識さえ欠いています。さらに「ユーザビリティは博士号を持つ専門家やグラフィックデザイナーに任せるべきだ」という長年の通説も加われば、ユーザビリティが長きにわたり軽視されてきたのもよくわかります。

多くのソフトウェアエンジニアが思い込まされていることとは裏腹に、使いやすいアプリケーションを構築することは決して不可能なことではありません。次の数ページで、アプリケーションを使いやすくするために必要な基本について探求し、この誤解されがちな「ユーザビリティ」を取り巻く通説を払拭していきます。では、ユーザビリティとは一体何でしょうか？

ユーザビリティとは、結局のところ、ある単純な問いに帰着します。それは、「ユーザーにとって、そのアプリケーションはどれほど使いやすいか」ということです[†2]。もちろん、これはかなり広範な概念であり、さまざまな解釈が可能です。ユーザビリティは、多くの場合、以下のさまざまな要素によって定義されます。

習得しやすさ

アプリケーションのインターフェースは、どれほど習得しやすいでしょうか？ 新規ユーザーは数分で使いこなせるでしょうか、それとも習得するのに何年もかかるでしょうか？ 初心者と熟練者の両方が、このアプリケーションを利用できるでしょうか？ 必要に応じて「補助機能」を追加したり、外したりすることはできるでしょうか？ 言い換えれば、アプリケーションはユーザーとともに成長できるでしょうか？ アプリケーションには、見つけやすくアクセスしやすい、適切なコンテキストヘルプやドキュメントが用意されていますか？

効率

アプリケーションのユーザーは、どれほど効率的に操作できますか？ クリック数は最小限に抑えられていますか、それとも最も単純な作業でさえ、複数の画面を移動する必要があり

†1　歯科医から、すでにフロスの習慣を褒められていますか？ その場合は、この例え話で、フロスの代わりに「定期的な運動」、「十分な睡眠」、「バランスの取れた食事」を当てはめてみてください。

†2　アプリケーションの対象とするユーザーは、自分ではないことを決して忘れないでください（開発者向けツールを構築している場合を除く）。自分にとってはわかりやすいことが、顧客にとってもわかりやすいとは限りません。

ますか？

記憶しやすさ

アプリケーションのインターフェースは、どれほど記憶しやすいでしょうか？ 自転車に乗るように、しばらく使わなくてもすぐに使いこなせるものですか？ それとも、ユーザーは起動するたびに、まるで歩き方を一から学び直しているような気分になるのでしょうか？

発見しやすさ

システムの機能を理解するのはどれほど簡単でしょうか？ ユーザーにとって、正しい操作が常に一目でわかりますか、それとも基本機能さえもわかりにくい場所に隠れてしまっているのでしょうか？

エラー処理

アプリケーションはエラーをどのように処理し、復旧しますか？ ユーザーの入力漏れに対して適切に対応していますか、それともミスが発生するとブルースクリーンになってしまいますか？ エラーが発生した場合、アプリケーションはユーザーのデータ損失を防ぐようになっていますか？ ヒント、検証、入力マスク、視覚的なガイドラインを通じてエラーを**未然に防ぐ**ようになっていますか？

ユーザー満足度

ユーザーは、アプリケーションを使って作業した後、どれほど満足しているでしょうか？ ユーザーは天にも昇るような気持ちになっていますか、それとも、毎日毎日、このひどいシステムを使わざるを得ない状況を作り出した責任者を罵っているでしょうか？

アクセシビリティ

そのインターフェースは、アクセシビリティに対応していますか？ そのインターフェースでは、支援技術が利用できますか？

このように、アプリケーション全体のユーザビリティには、さまざまな要因が影響します。一見すると大変に思えるかもしれません。しかし、適切なアプローチと入念な設計を行えば、ユーザーのニーズに反することなく、ユーザーを喜ばせるアプリケーションを作成できます。

7.1.2 アクセシビリティとは？

ユーザビリティとは、すべてのユーザーにとっての使いやすさと満足度のことですが、アクセシビリティとは、さまざまな能力を持つ人々がソフトウェアを利用できるようにすることです。あらゆる能力を持つユーザーが、インターフェースを操作して、やり取りできるようにする必要があります。法令や規制が適用される場合は、個別に考慮しなければならない項目もあるかもしれませんが、一般的には、以下の課題に留意する必要があります。

視覚的な課題

弱視から全盲、色覚異常まで[†3]

聴覚に関する課題

聴覚障害または難聴者

手先の器用さに関する課題

反復性ストレス障害や運動機能障害により、マウスによる精密な操作が難しい場合

認知面での課題

自閉症や読字障害（ディスクレシア）、あるいは読解力やコミュニケーション能力が低い状態

ソフトウェアのアクセシビリティを向上させることは、障害のある人々だけでなく、すべてのユーザーにとって有益です。車椅子の方にとって不可欠な縁石の切り欠けやスロープは、空港でキャリーバッグを引きずって移動する旅行者にとっても役立ちます。タップやポインタのターゲットを大きくしたインターフェースは、代替入力デバイスを使用する人々のためだけのものではありません。また、キーボードショートカットは、マウスやトラックパッドが使えない方々と同じように、熟練ユーザーにとってもメリットがあります。高速インターネット接続を必要としないソフトウェアは、ブロードバンドを利用できない人々だけでなく、通信エリアの境界を移動するモバイル端末ユーザーにとっても役立ちます。

UI を構築する際は、包摂性を念頭に置き、できるだけ多くの人々がソフトウェアを利用できるようにしてください。多くのモダンプラットフォームやオペレーティングシステムでは、アプリケーションのアクセシビリティを向上させるために、さまざまな API や機能がサポートされています。Apple のオペレーティングシステムには、カラーフィルターやライブキャプション、視線追跡機能など、さまざまな支援技術が搭載されています。Microsoft も自社製品に同じような機能を多数搭載しているほか、カスタマイズ可能なアダプティブボタンやマウスのラインナップも提供しています。

7.1.3 ローカライゼーションと国際化とは？

ローカライゼーション（L10N、localization）と国際化（I18N、internationalization）は、よく同じ意味で使われますが、両者は異なる概念です。ただし、互いに関連しています。**I18N** とは、アプリケーションをさまざまな言語や文化的背景に適応できるよう、**設計・構築する**プロセスのことです。例えば、ラベルをハードコーディングするのではなく、プロパティファイルからラベルテキストを読み込むように UI を構築します。また、ラベルの大きさを変更できるような設計も重要です。同じ単語やフレーズであっても、翻訳によってはテキスト量が大幅に増える場合があるからです[†4]。すべての文化が左から右へ読む形式（LTR、left-to-right）とは限りません。アラビア語

†3　男性の約 8%、女性の約 0.4% は何らかの色覚異常があります。

†4　例えば、英語からドイツ語への翻訳では、平均してテキストが 35% 増えます。

やヘブライ語など、いくつかの言語は右から左へ読みます（RTL、right-to-left）。日本語は上から下へ読むこともあります！

一方、**L10N** とは、対象となる市場の文化や言語に合わせて、インターフェースを**調整するプロセス**のことです。つまり、L10N とは、ラベルや入力項目を現地の言語に合わせて翻訳し、適切な通貨記号や、その地域で一般的な日付形式、単位を使用することを指します。国際化は、さまざまな言語や文化に対応するための基盤となるものであり、ローカライゼーションはカスタマイズを実施することです。

アプリケーションがある特定の地域以外では使用されないという保証がない限り、プロジェクトの初期段階から I18N と L10N を考慮して設計すべきです。アプリケーションがある一つの市場を超えて展開される可能性が少しでもあるなら、世界中のユーザーを念頭に置いて構築してください。将来、きっとその決断に感謝することになるでしょう！

7.2　ユーザーを理解する

ユーザーや、ユーザーがインターフェースを利用する環境を理解していなければ、使いやすいアプリケーションを構築することはできません。ユーザーについて一般化しすぎるのは危険を伴うおそれがあります（次のコラム「思い込みは禁物」を参照）。必ず、自分の思い込みに疑問を投げかけてみてください。もちろん、現場で何か月もかけてユーザーを分析してもよいのですが、ユーザーがアプリケーションをどのように使うかについて基本的な質問を投げかけるだけでも、ユーザビリティを向上できます。以下に例を示します。

- ユーザーは、コンピュータに詳しいですか？ どのようなオペレーティングシステムを使っていますか？
- ユーザーの中に、身体的な制約のある方はいますか？ アプリケーションはスクリーンリーダーにどのように対応していますか？ ユーザーが文字サイズを大きくしたり、画面の解像度を変更したりした場合はどうなりますか？
- ユーザーは、スマートフォンでアプリケーションにアクセスしますか？ タブレットですか？ ノートパソコンですか？ ヘッドセットですか？ テレビですか？ それとも、これらすべてですか？
- 対象となるユーザーの教育水準は、おおよそどの程度を想定されていますか？
- ユーザー層は比較的安定していますか、それとも入れ替わりが激しいですか？
- そのアプリケーションはどのくらいの頻度で使用されていますか？
- このアプリケーションはどこで使用されるのでしょうか？ オフィスでしょうか？ 誰かの食卓でしょうか？ ソファーでしょうか？ それとも現場でしょうか？
- 利用環境はどのような感じですか？ 騒がしいですか？ 静かですか？ ストレスがたまるような環境ですか？
- ユーザーは、このアプリケーションに関するトレーニングを受けることになりますか？

思い込みは禁物

ユーザー調査の手間を省くために、ユーザーについて勝手に決めつけたくなることもあるかもしれませんが、それは不必要な問題を招くおそれがあります。数年前、ある高齢者向け施設が地元の企業に連絡を取り、古いパソコンを寄付してもらえないか尋ねました。事務所の責任者は、高齢者はそれらを活用できるほどITに詳しくないだろうと思い、ただ埃をかぶるだけだろうと決めつけて、笑いながら承諾しました。数か月後、その高齢者施設から再び連絡があった際、責任者は機器を返却したいのだろうと考えました。しかし実際には、入居者たちがキーボードを酷使して消耗したため、追加のキーボードが必要だったのです。実は、入居者たちはオンラインで家族や友人と連絡を取り合うことをとても楽しんでいたのです。時間をかけて、対象となるユーザーについて理解するようにしてください。

想定ユーザーについて基本的な質問を投げかけることは、非常に有益です。その回答から、アプリケーションのユーザビリティを向上させ、価値のあるものにする方法がわかります。騒がしい環境でアプリが使用される場合、ユーザーは通知音を聞き逃すことが多くなるでしょう。通知音以外の通知手段を検討しましょう。長期間にわたるトレーニングを行うことを前提として設計することも可能ではありますが、実際にはユーザーはマニュアルすら読まないでしょう。このような場合は、最も簡単に行える操作が正しい操作となるようにしてください。例えば、数字のみの入力が必要な場面で、ユーザーが数字以外の文字を入力できないようにするなどです。また、現場で地下水を測定する水文地質学者が使用する堅牢なタブレット向けに設計されたアプリケーションは、大型モニターやトラックパッドを備えたオフィスワーカーを対象としたものとは、制約がまったく異なります。

ユーザビリティが高く価値のあるアプリは、ユーザーの満足度向上につながります。多くの開発者は「ユーザー満足度」という概念を軽視しており、中には「ユーザーには選択肢がないのだから」とまで言う人もいます。例えば、社内ユーザー向けに開発している場合などです。彼らは読者のアプリケーションを使用せざるを得ない状況にあります。だからといって、ユーザーに対してアプリケーションを無理やり押し付けるようなことは許されません。ユーザーには、もっと良いものを提供しなければならないのです！ この点は、競合他社とオープンな市場で競争している場合、より明白になります。多くの場合、ユーザーは、よりユーザビリティが高いソリューションへと流れていくからです。どのような状況にあっても、ユーザー満足度を高めることの重要性を過小評価してはなりません。

可能であれば、代表的な顧客がアプリケーションを使用している様子を、実際に傍らで観察してみてください。そこから得られる気づきには、きっと驚かれることでしょう。繰り返しになりますが、このような調査に何か月も現場で費やす必要はありません。アプリケーションが実際に使われる環境で、数日間かけて実際のユーザーを観察するだけで、ユーザーに喜ばれるアプリと、嫌われるアプリの差が生まれるのです。

7.2.1　二次的なユーザー

ソフトウェアの主なユーザーに注力すべきではありますが、**二次的な**ユーザーも忘れてはいけません！ 多くの場合、アプリケーションを利用している人々は、実際には別の誰か、つまり二次的なユーザーに代わって作業を行っています。例えば、カスタマーサービスの担当者とやり取りをしたときや、車の整備の予約をしたときなどは、読者が二次的なユーザーだったことになります。

直近で自分が二次的なユーザーだったときのことを思い出してみてください。カスタマーサービスの担当者は、「少々お待ちください、ご注文内容を確認しています。システムの反応がかなり遅いです」といったようなことを言いませんでしたか？ その体験は快適でしたか？ Nate は以前、予約担当者が予約を入れるのに数分もかかっているのを目撃しました。担当者は「新しいソフトウェアだと、11 回クリックする必要があります。数えてみました」と謝罪しました。彼の情報がすでにすべて登録されていたにもかかわらずです！

ユーザビリティが悪いと、実際にコストが伴います。それはエンドユーザーやその業務に直接的な影響を与えるだけではなく、市場における企業の評判にも悪影響を及ぼします。

7.2.2　読者はユーザーではない

言うまでもありませんが、（他の開発者向けのアプリケーションを設計している場合を除き）読者自身は実際のユーザーを代表しているわけではありません。読者にとって当たり前のことが、背景の異なる人にとっても当然だとは限りません。読者はターミナルに難解なコマンドを入力することに慣れているからといって、それがすべてのユーザーに当てはまるとは限りません。また、巨大な曲面モニターでは見栄えが良くても、それが小さな画面でもうまく機能するとは限りません。

また、ユーザーは、読者ならしないような行動をすることもあります。例えば、数年前、ある非常に信頼性の高い世論調査機関に、ある政治団体から問い合わせがありました。それは、無作為に選ばれた数人のボランティアに提示された選択肢の中に、その候補者の名前が含まれていなかったためです。世論調査会社は、候補者が調査に含まれていることを選挙陣営に保証し、問題の真相を究明することを約束しました。実際に調査会社がコールセンターを訪問したところ、すぐに問題の原因が判明しました。コールセンターの一部のスタッフが、質問文を読みやすくするため、フォントサイズを大きくしていたのです[†5]。

エンドユーザーによる行動としては特に驚くべきことではありませんが、その行動がどのような結果を招くか、誰もテストしていなかったことが判明しました。偏りを排除するため、候補者の名前は調査ごとにランダムな順序で表示されるようになっていました。そして、文字サイズを大きくした結果、コールセンターの担当者には一部の名前が見えなくなってしまいました！ 結局、その調査結果は破棄せざるを得なくなり、調査機関の評判が傷つく結果となりました。

†5　読みやすくするためにフォントサイズを大きくしたことがある人は、手を挙げてください！

デフォルト設定の弊害

デフォルト設定の力を決して侮ってはいけません。さあ、今すぐスマートフォンで確認してみてください。メールにはどのアプリケーションを使っていますか？ Web の閲覧には？ ナビゲーションには？ おそらく、モバイルオペレーティングシステムのデフォルト設定になっているアプリでしょう。デフォルトに設定されることは、莫大な利益をもたらす可能性があります。Google は、Apple の全デバイスでデフォルトの検索エンジンとなるために、数十億ドルを支払っていると推定されています。

おそらく、読者のデフォルト設定が数十億ドル規模の契約で選ばれることはないでしょう。ただ、だからといって、デフォルト設定について真剣に検討しなくてもよいということにはなりません。Nate の教え子の一人が顧客に機能のデモを行っていたところ、顧客が真っ先にやったことは、アプリのデフォルト設定をすべて変更することでした。教え子は驚いてその理由を尋ねると、顧客は「私たちはいつもそうしているんです」と答えました。教え子は自分のデスクに戻り、デフォルトの設定を更新して顧客のもとへ戻ると、顧客はその変更に大喜びしました。ここから得られる教訓とは？ 適切なデフォルト設定が何かを勝手に決めつけないでください。顧客に遠慮なく尋ねてみましょう。さらに良いのは、実際にテストしてみることです！

既存のアプリケーションについては十分な利用データがあるかもしれませんが、新しいものを構築する際には、さらに深く検討しなければなりません。つまり、何が正しい対応なのかは事前に把握できるとは限らないのです。このような場合、ユーザーインターフェースにテレメトリー機能を組み込むことで、アプリケーションがどのように利用されているかをより深く理解でき、直感ではなくデータに基づいて対応できるようになります[†6]。そして、インターフェースを適切に進化できます。

また、ユーザーが設定を変更した場合、アプリケーションはその設定を今後も記憶しておく必要があります。自分の好みに合わせて設定を変更したのに、次に起動したときにはアプリケーションが「自分のほうがよくわかっている」と判断して、設定が元に戻されていた、という経験は何度もありますよね？

最後に、デフォルト設定は「安全」であるべきです。特に、ユーザーがストレスを感じている場合や注意が散漫になっている場合はなおさらです。ある操作が取り返しのつかない破壊的なものである場合、その操作方法に「障壁」を設けることは、実は良いことなのです。デフォルト設定を「アカウントを削除」にしてはいけません。特に、ユーザーが無意識にクリックするよう習慣づけられてしまっているようなダイアログではなおさらです。

†6　あるいは、ユーザーがフィードバックを送信できる機能を提供するだけでも、たとえ「いいね」や「イマイチ」といったシンプルなものであっても、貴重な情報が得られる可能性があります。

7.2.3　文化の影響

文化が及ぼす影響を理解することも重要です。ユーザーのメンタルモデルは人それぞれで異なります。これは、ロンドンを訪れた観光客なら誰でも、道路を横断する際に身をもって実感できることでしょう（**図7-1** 参照）。明らかな違いもあります。例えば、右から左に読む言語では、左から右に読む言語とは異なる UI レイアウトが必要になります。**読者**にとって当たり前のことが、別の大陸に住む人にとっては不快に感じられることもあります。色にも異なる意味合いがある場合があります。例えば、多くの東洋文化では赤は幸運や繁栄を象徴しますが、南アフリカでは喪の色とされています。



図7-1　ロンドンの横断歩道では、右側から車が近づいてくることを歩行者に注意喚起する

文化の違いは、アイコンの視覚的なデザインにまで影響する場合もあります。より具体的なアイコンを好む文化もあれば、より抽象化された表現を好む文化もあります。さらに、「保存」を表すフロッピーディスクのような歴史的なシンボルについても考慮する必要があります。ディスクドライブを搭載したコンピュータを使ったことがない人々の割合が増えていることを考えれば、なおさらです[†7]。ローカライゼーションはそれだけで章を設けるようなテーマです。インターフェースの要素をある言語から別の言語へ翻訳すること自体は比較的単純な作業であっても、その後で要素のサイズ変更や配置の変更が必要になるケースも珍しくありません。

ユーザーがどのような人々で、ソフトウェアをどう使うのかを時間をかけて調査することで、ユーザーにとって不親切に映りかねない選択を避けられます。貴重な時間を無駄にしているように思えるかもしれませんが、基本的なユーザー調査は、その投資に見合うだけの価値を何度ももたら

†7　もちろん、多くのアプリケーションには自動保存機能があります。

してくれます。第一印象は取り返しがつきません。コードの向こう側にいる人々の声に耳を傾けるだけで信頼を得られるのです。この点を、決して過小評価してはいけません。

7.3 ユーザビリティを最大限に高める

この時点で、ユーザビリティのどの点（学習のしやすさ、効率性、記憶しやすさ、発見しやすさ、エラー処理、ユーザー満足度、アクセシビリティ）に重点を置くべきか、と尋ねたくなるかもしれません。これらの要素をすべて最大限に高めようと急ぐ前に、最も重要な問いについて考える必要があります。それは、「アプリケーションはどのように使われるのか」ということです。アプリケーションの中には、毎日一日中使われるもの（効率性、記憶しやすさ、ユーザー満足度が重要）や、本質的に複雑なもの（CAD ソフトなど）があります。複雑なものの場合は、ユーザーは通常、アプリケーションの使用に際して何らかのトレーニングを受けたり、ひたすら繰り返すことでインターフェースを習得したりすることになります[†8]。もし読者のアプリケーションが、次なるソーシャルネットワーキングの「ユニコーン」となることを目指しているなら、ユーザーに多くのことを学ぶのを我慢してもらうことは期待できません（つまり、習得しやすさと発見しやすさが鍵となります）。アプリは直感的であるべきです。そうでなければ、ユーザーは読者のサイトから、より直感的なサイトへと離れていってしまうでしょう。言い換えれば、ユーザーがどのような人たちかを踏まえたうえで、アプリケーションの目的とそのコンテキストを考慮しなければならないのです。

効率性についても同じような原則が当てはまります。ユーザーが 1 日の大半をアプリケーションで過ごすのであれば、クリック数をわずかでも減らすことが、すぐに大きな効果をもたらします。1 回の操作にかかる時間をほんの数秒短縮するだけでも、それが 1 日に何千回も行われれば、その効果は大きなものになります。決して忘れてはいけません。小さな数字に大きな数字を掛け合わせれば、大きな数字になるのです！

場合によっては、ユーザーは 1 時間や 1 日にどれだけの「タスク」を完了したかで評価されることがあります。そのため、ワークフローを可能な限り効率化することは、収益に大きな影響を与える可能性があります。逆に、使用頻度の低いアプリケーションであれば、記憶しやすさや習得のしやすさといった点を向上させるために、効率性については多少の妥協をしなければならないこともあります。

「記憶しやすさ」、「習得しやすさ」、「発見しやすさ」は、密接に関連しています。理想的には、アプリケーションは記憶しやすく、かつ習得しやすいものであるべきですが、ここでもコンテキストが重要になります。例えば、毎日使用されるアプリケーションであれば、（できれば、良い意味で）おそらく記憶に残るものとなるでしょう。このような場合、習得や発見がそれほど容易ではないような操作が含まれていてもかまいません。一方、アプリケーションが時折しか使われない場

†8　とはいえ、トレーニングを口実に、使い物にならないようなひどいものを作ってよいというわけではありません。トレーニングもソフトウェアのコンポーネントの一つであり、すぐに切り捨てられてしまうものです。

合、ユーザーに何時間もかけてその使い方を学んでもらうことは期待できません。このような場合では、習得しやすさと発見しやすさは不可欠ですが、記憶しやすさは重要ではありません。パズルゲームは、発見しやすさを意図的に低く設計されています。物事を自分で解き明かすことこそが、その醍醐味だからです。繰り返しになりますが、コンテキストが判断の指針となります。

7.4 デザインの原則

デザインのスキルを身につけるには時間がかかりますが、よく知られる原則に従えば、初心者でも魅力的なデザインを作成することが可能です。Robin Williams は、高く評価されている著書『The Non-Designer's Design Book』（Peachpit Press、和書『ノンデザイナーズ・デザインブック』マイナビ出版）の中で、デザインを学び始めたばかりの人々に、コントラスト、反復、整列、近接の重要性を解説しています。

彼の著書は主として、魅力的で効果的なドキュメント、ニュースレター、名刺の作成を目的としていますが、これらの概念はアプリケーションのインターフェースデザインにも同じように適用できます。これらの概念はシンプルで実践しやすいものです。逆に、これらの概念を無視すると、アプリケーションのユーザビリティに深刻な悪影響を及ぼすことになります。

キャリアを積むにつれ、こうしたルールを柔軟に解釈したり、あえて破ったりするほうがよい場合があることを実感するようになるでしょう。ただし、デザインしているものは何かをよく考慮する必要があります。ギザギザの配置は印象的な広告キャンペーンには効果的かもしれませんが、企業の勤怠管理アプリケーションには適しません。

7.4.1 コントラスト

コントラストは、非常に効果的な手法です。通常、コントラストといえば 2 つの色の違いを思い浮かべますが、異なる 2 つの要素があれば、それだけでコントラストが生まれます。大きなフォントと小さなフォントを対比させたり、太い線と細い線を組み合わせたり、小さな画像と大きな画像を対比させたりすることで、視覚的なコントラストが生まれます。色や形の違い、さらには配置の違いさえも、ページを際立たせるのに役立ちます。

コントラストを効果的に活用することで、デザインに視覚的な魅力を加えられますが、効果を発揮させるためには、**本当に**異なる要素を用いることが大切です。大胆にいきましょう。あるいは、Williams が言うように「臆病になってはいけません」。2 つのグレーの色合いを対比させるよりも、赤と黒のようなまったく異なる 2 色を使うほうが、はるかに効果的です。思い切って挑戦するようにしてください。要素が似通いすぎてしまうよりは、少しやりすぎたほうが良いのです。

コントラストは、構成を整理する手段としても使えます。ページを一目見ただけで、ユーザーがデザインの流れを素早く把握できるようになります。適切にコントラストを使わなかった場合、ユーザーを混乱させ、意図していないグループを誤って認識させてしまうおそれがあります。見出し、小見出し、本文は、ユーザーが明確に判別できるよう、それぞれ異なる特徴を持つようにすべきです。

7.4.2 反復

デザイン全体で視覚的要素を繰り返し使用することで、アプリケーションに親しみやすさと統一感が生まれます。主に、常に表示されるナビゲーション要素やヘッダ画像で見られるものですが、より控えめなものでもその効果は現れます。同じフォント、配色、画面レイアウト、箇条書きのスタイル、イタリック体など、デザインを統一するのに役立つ要素であれば反復の効果を活用できます。

これは単なる統一感だと考えるかもしれませんが、視覚的に優れたデザインを生み出すうえで重要な要素となります。最も基本的なレベルでは、反復によってユーザーは自分がまだ同じサイトにいることを認識できます。視覚的なデザインが絶えず変化するアプリケーションを使ったら、どれほど違和感を覚えるか想像してみてください。

見出しのフォントや太さをすべて統一したら、意図的に繰り返すことができる要素が他にないか考えてみましょう。繰り返し使われる線や箇条書きは、デザインを一段階引き上げる、ちょっとしたアクセントとして使えます。Nate が担当したあるアプリケーションでは、ページの下部に、本文コンテンツと定型的なプライバシー・著作権表示を区切る細い線が 1 本引かれていました。このフッターは繰り返し使用されていましたが、その反復性をさらに強調するため、細い線を 2 本重ねて追加しました。些細な違いに見えるかもしれませんが、この工夫によって、フッターはアプリケーションの他の部分とは文字通り一線を画すものとなりました。

コントラストと同様に、反復も視覚的なまとまりを作り出し、ユーザーがアプリケーションを操作するのを助けます。ユーザーはレイアウトにすぐに慣れ、日々の作業を進める中で、安心して「無視」できる要素を把握できるようになります。繰り返される形状によって、ユーザーがページを素早く理解できます。そして、異なる要素がわかりやすくなります。

繰り返し使用する要素の中でも、色に関してはやりすぎてしまうことがよくあります。赤が自社ブランドを強く象徴する色だとしても、アプリケーション全体を赤で埋め尽くしてよいというわけではありません。統一感をもたらす要素と、派手すぎる過剰な装飾は紙一重の差ですので、慎重に進めてください。

7.4.3 整列

整列は、実践するのが非常に簡単な原則です。すべての要素を丁寧に配置してください。ページ上にたまたま空きスペースがあるからといって、適当な場所に要素を配置してはいけません。要素を整列させることで、すっきりとした統一感のある見た目が生まれ、視覚的に要素が結びつきます。整列を活用することで、互いに離れて配置された項目同士でも、視覚的なつながりを保てます。

左揃えや右揃えにすると、垂直方向の端がシャープになります。シャープな端は、洗練されたプロフェッショナルな印象を与えます。中央揃えは避けてください。端がギザギザになり、見た目が悪くなるだけでなく、テキストが読みづらくなってしまいます。ただし、整列するのはテキストだけではありません。画像やアイコンも、ページ上の他の要素に合わせて整列する必要があります。要素を配置する際は、常に何かと揃えるようにしてください。

整列方法を決めて、デザイン全体を通してそれを一貫して使用してください。繰り返し適用してください。デザイン全体で同じ整列方法を使用することで、ユーザーに安心感を与え、構成をよりわかりやすくします。

7.4.4 近接

「近接」（距離が近いかどうか）は、人が目にするものをどのように解釈するかに重要な影響を与えます。一緒にまとめられたものは、たとえ関連性がなくても、関連しているように認識されます。以下のリストを見てください。

- Shorts（ショートパンツ）
- Sunscreen（日焼け止め）
- Socks（靴下）
- Sunglasses（サングラス）
- Sandals（サンダル）
- Shirts（シャツ）

これらの項目はまとめて配置されているため、人は本能的にそれらを一つのグループとして捉えます。これらの項目がどのように関連しているかは重要ではありません。誰かが旅行の準備をしているとは思いもよらず、しかも「S」で始まるものにこだわりがあるということに気づかなかったとしても、項目間に何らかの共通点があると感じ取れるでしょう。では、リストを少しだけ変えてみましょう。

- Shorts（ショートパンツ）
- Sunscreen（日焼け止め）
- Socks（靴下）
- Sunglasses（サングラス）
  - Sandals（サンダル）
  - Shirts（シャツ）

一見すると、サンダルとシャツはリスト全体の中で特別な関連性があるように見えます。その関係性が具体的にどのようなものかは明らかではありませんが、一部のユーザーは（意識的か無意識的かはともかく）そのつながりが何なのかを特定しようとするでしょう。

もう一つの例として、Apple の Fitness アプリでは、獲得したさまざまなアワードを管理しています。以前は、すべてのバッジが一覧表示されていたため、操作が少々煩雑でした。アップデートにより、インターフェースが変更され、類似したアワードがカテゴリカードごとにまとめられるようになりました（**図7-2** 参照）。メインのアワード画面では最新のチャレンジ状況が表示され、カテゴリをタップすると、そのカテゴリで獲得した他のバッジが表示されます。

逆に、関連性のない項目はまとめてはいけません。ユーザーを混乱させないためにも、性質の異なるものは離して配置してください。

これらの視覚的なグループを作成する際は、各グループの間に適度な余白を設けるようにしてください。グループを余白で区切ることで、グループ内の項目間のつながりがさらに強まり、ユーザーがページの内容を理解しやすくなります。項目同士が近接していると、視線が自然と誘導される視覚的なグループが形成されます。混乱を避けるため、ページ上の視覚的なグループの数は最小限に抑えてください。グループが多すぎる場合（5 つ程度以上）、配置を変更してグループを再編

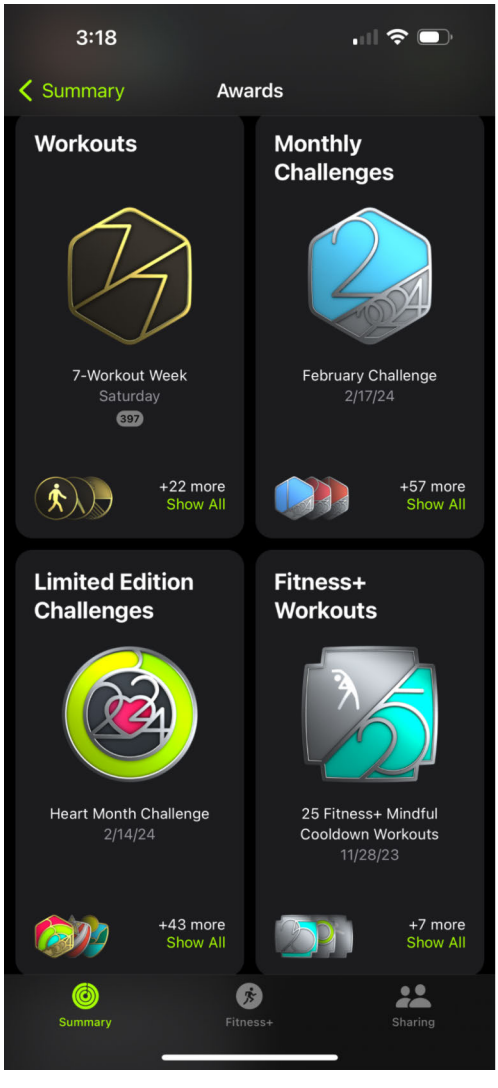


図7-2　Apple の Fitness アプリケーションの「アワード」セクションにおける「近接」の使用例

成してみてください。

7.5　デザインの原則を適用する

オライリー学習プラットフォーム（O'Reilly Learning Platform）を題材に、これらの原則をさらに詳しく分析してみましょう。そのデザイン全体を通じて、これらのデザインの原則が活かされ

ており、その結果、使いやすく、見た目にも美しいサイトとなっています。

図7-3 を見てください。ここでは、コントラストの効果が見て取れます。大きな表紙画像が際立っており、コントラストのある色使いが視線を惹きつけると同時に、ブランドアイデンティティをさりげなく表現しています。また、各リソースのフォーマット種別を示す色使いにも注目してください。現在のトピック領域は、ページ上の他のどの要素よりも大きなフォントを使用することで、一目でわかるように強調されています。

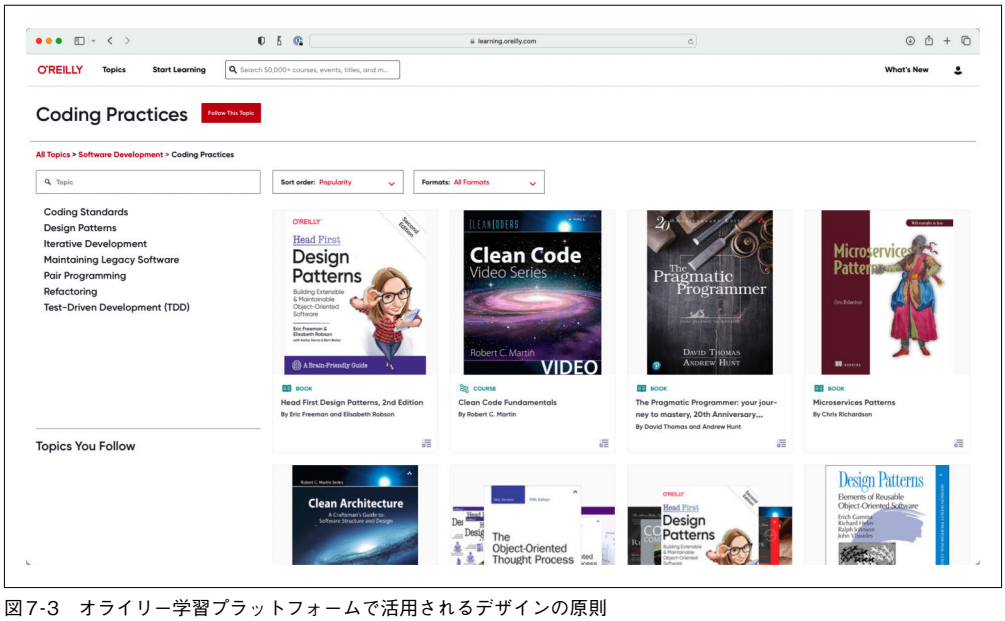


図7-3　オライリー学習プラットフォームで活用されるデザインの原則

図7-4 では、検索結果ページを構成するさまざまなブロックを確認できます。「検索」ボックスがトピック領域と明確につながっていることが一目でわかり、2つのドロップダウンメニューで検索結果をさらに絞り込めます。特に「Topics You Follow（フォロー中のトピック）」ボタンのコントラストのある色に注意してください。ページ上で一際目立ち、自然と視線を惹きつけます。

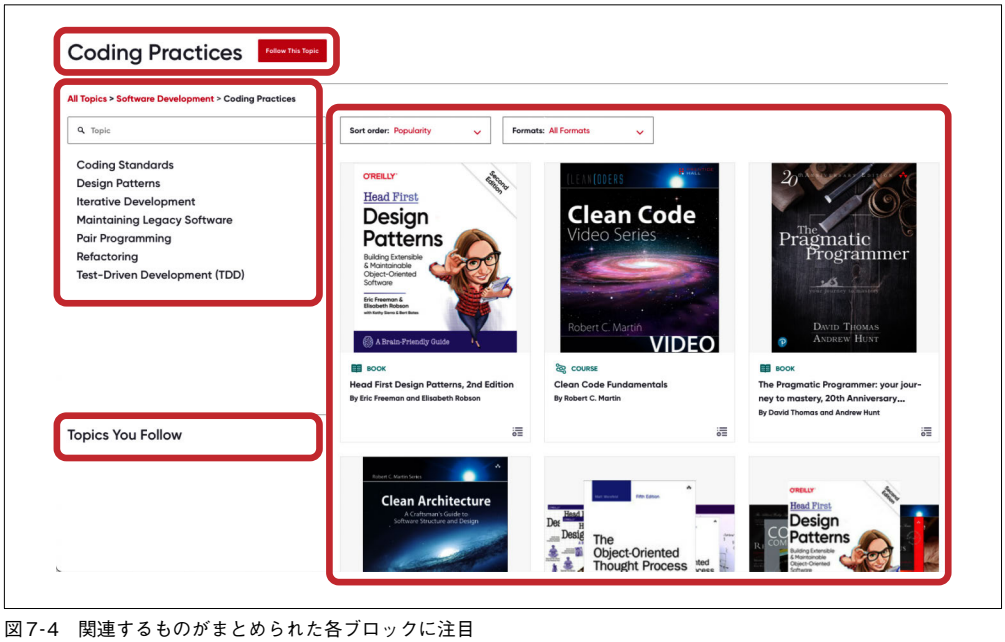


図7-4　関連するものがまとめられた各ブロックに注目

これらの構成要素は、コンテンツの詳細ページにも引き継がれています（**図7-5** 参照）。ここでもおなじみのコントラストのある色使いが目に入ります。視線を引くには十分でありながら、過剰ではない絶妙なバランスです。同じフォントファミリーが使用されているため、同じサイト内にいるという安心感を与え、ページ上部のメインメニューが統一感を生み出しています。ここでも、コントラストのある色で強調されたボタンが１つ配置されており、このコンテンツを読み始めるよう促しています。

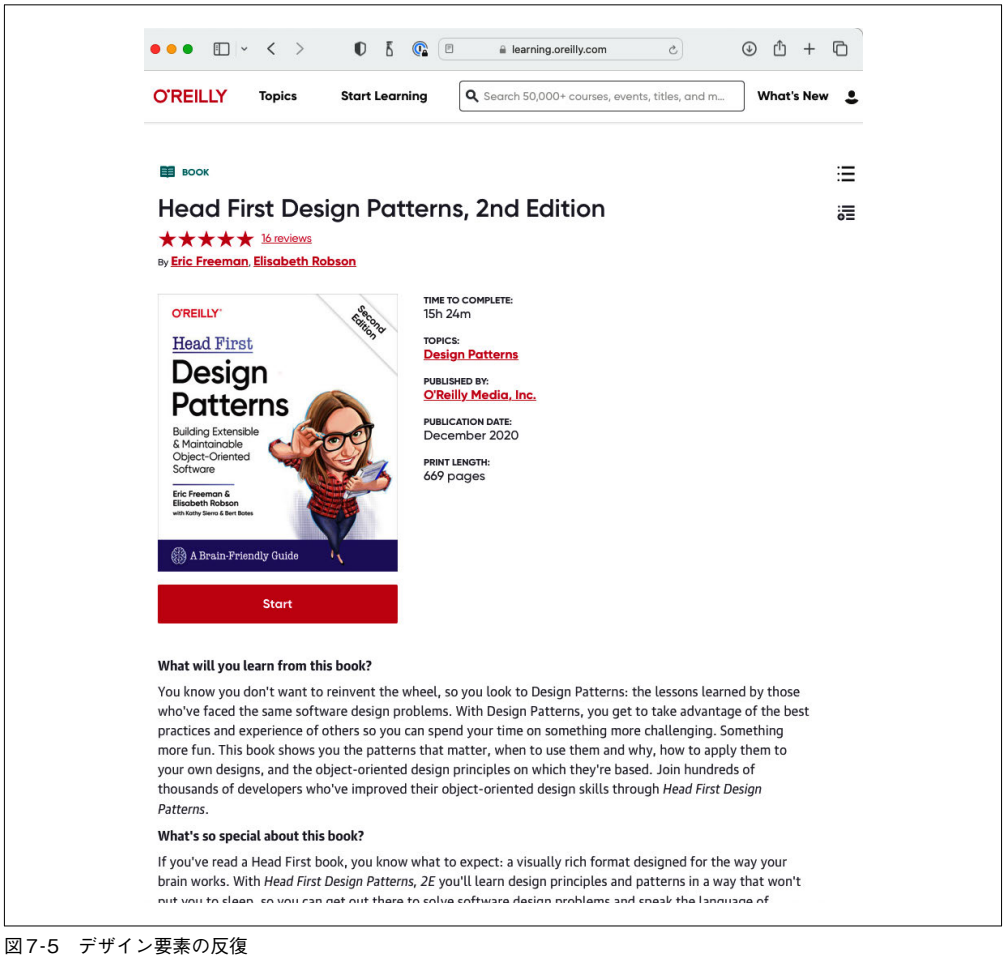


図7-5　デザイン要素の反復

ただし、これらの原則は Web アプリケーションに限った話ではありません。**図7-6** の Apple Music、Apple Books、App Store の各アプリケーションのサイドバーを例に挙げてみましょう。これら 3 つはそれぞれ対象となる分野に合わせてカスタマイズされていますが、いずれも検索バー、統一されたフォント、関連するアイコンを備えています。また、微妙に色を変えることで、それぞれのアプリケーションの独自性を際立たせています。もしアプリ間でまったく同じ色を使っていたら、ユーザーは自分がどのアプリケーションを利用しているのかわからなくなってしまったかもしれません！



図7-6　Apple のサイドバーの類似性

7.6 「正しいこと」を「わかりやすいこと」にする

アプリケーションを習得するうえで、発見しやすさは非常に重要です。もしすべての機能がユーザーの予想しない場所に隠されていると、システムを習得するのに非常に時間がかかってしまいます[†9]。発見しにくい場合でも、一度覚えれば忘れなかったり、毎日システムを使用したりする場合は問題ない**かもしれません**。しかし、他の方法よりも操作性が格段に優れていない限りは、発見しやすい方法を採用するのが最善です。

†9　もしユーザーがその機能を知っていたとしても、機能をしっかりと隠しておけば、ユーザーはアプリケーションにその機能がないと思い込んでしまうことでしょう。

かつて筆者の一人が、ずいぶん昔にほぼ毎日のように苦労させられた、あるソフトウェアを例に挙げてみましょう。**図7-7** のダイアログを踏まえて、音声はどのように録音すればよいのでしょうか。考えてみてください。



図7-7　発見しにくい機能の例：音声はどのように録音するのか？

このソフトウェアを初めて利用するユーザーは、普段よくこのソフトウェアを使っている人の助けや詳細なマニュアルがなければ、音声の録音方法がわからずに困ってしまいます。このダイアログは、発見しやすさを軽視している例です。答えは**図7-8** にあります（またしてもわかりにくい形で）。もう諦めましたか？「Conference Number（会議番号）」を設定した後、「TestAudio（音声テスト）」をクリックし、「Remain connected to meeting audio after testing（テスト後も会議音声に接続したままにする）」にチェックを入れるのを忘れないようにしてください。ほら、簡単でしょう。



図7-8　ここで音声を録音する。操作方法を見つけられる？

まあ、これは「ひどすぎる」という点で記憶に残る、つまり、機能させるために何か「奇妙な」操作が必要だったと、多くの人が漠然と覚えていることでしょう。しかし、もし数か月間録音しなかったらどうなるでしょうか？ その操作方法をまだ覚えているでしょうか？ おそらく覚えていないでしょう。研究によると、発見しやすさは記憶しやすさに大きく寄与することが示されています。迷ったときは、よりわかりやすいアプローチを選ぶのが無難です。

先ほどの例の場合は、どのように修正すればよいでしょうか？ まず、なぜわざわざ音声を「テスト」する必要があるのでしょうか？ 音声レベルメーターは、最初のダイアログに組み込むだけで十分なはずです。そうすればこの煩わしい2つ目のダイアログは完全に不要になります。もしそのアプローチが現実的ではない場合、「Include audio with meeting recording（会議の録音に音声を含める）」を選択するだけで、音声が含まれるようにすればよいでしょう。ユーザーは「Remain connected to meeting audio after testing（テスト後も会議音声に接続したままにする）」を選択する必要はまったくありません。このオプションは削除すべきです。

また、可能な限り、ユーザーが誤った操作を行わないように配慮すべきです。フォーム上で適切なフィールドタイプを選択するという単純な方法だけでも、多くのエラーを未然に防げます。ヒント、フィールドマスク、視覚的な手がかり、入力例などを活用し、ユーザーにとって正しい選択が明確になるようにしてください。

すべての機能を発見しやすくする必要があるのか？

場合によっては、理想的な操作性を追求できないこともあります。機能が多すぎるアプリケーションでは、すべての機能を前面に出すことはできず、一部の機能は目立たない場所に配置せざるを得ません。モバイルファーストやモバイル専用のソフトウェアが増えるにつれ、デスクトップ時代のソフトウェアに比べて、操作スペースがさらに狭くなる可能性もあります。

例えば、iPhone のピンチズーム機能を考えてみましょう。初めて使うユーザーは、Web ページや画像をズームできることさえ気づかないかもしれません。インターフェースを見ただけでは、ピンチ操作で何かが起こることは決してわかりません。しかし、ピンチ操作がズームに相当することを教えるのにかかる時間は、わずか2秒です。この手法は、非常に習得しやすく、記憶に残りやすい機能であり、一度その仕組みを知れば、それはごく自然な操作のように感じられます。あまりにも自然であるため、もしインターフェースがこの機能をサポートしていなければ、ユーザーのメンタルモデルを崩してしまうことになりかねません！

ただし、ピンチズームは例外です。この操作は実にうまく設計されており、直感的です。この「隠れた」機能のメリットは、考えられるデメリットをはるかに上回っています。Apple は、当初の広告キャンペーンにおいてズーム機能を中心に据えていたため、ほとんどのユーザーがこの機能を知っていると考えられます。また、Apple は iPhone の公式サイトでさまざまな操作説明動画も提供していました。さらに、ピンチ操作によるズームがなくても、インターフェースは十分にユーザビリティが高いものでした。状況によっては発見しやすさを多少

犠牲にすることもあります。しかし、重要な機能を目立たないようにするのであれば、その操作のメリットが利用者にすぐに伝わるようにしなければなりません。

インターフェースは、時に非常に発見しやすくても、それほど記憶に残らないこともあります。例えば、キーボードショートカットは、ユーザーの作業効率を高めるために不可欠なものです。熟練ユーザーにとって欠かせないものとなりますが、必ずしも記憶しやすいとは限りません。頻繁に使いつづけるうちに、ショートカットは自然と身についていきます。ほとんどのユーザーは「保存」、「コピー」、「貼り付け」といった操作を、意識せずに素早く実行できるでしょう。しかし、あまり使わない組み合わせについては、発見しやすいようにしておくことが重要です。多くの IDE は、まさにこの点において素晴らしい機能を提供しています。例えば、IntelliJ には「Find Action」という汎用的なコマンド（**図7-9** 参照）が搭載されており、アクション名を入力することで、このコマンドがなければ思い出せなかった機能を見つけられます[†10]。

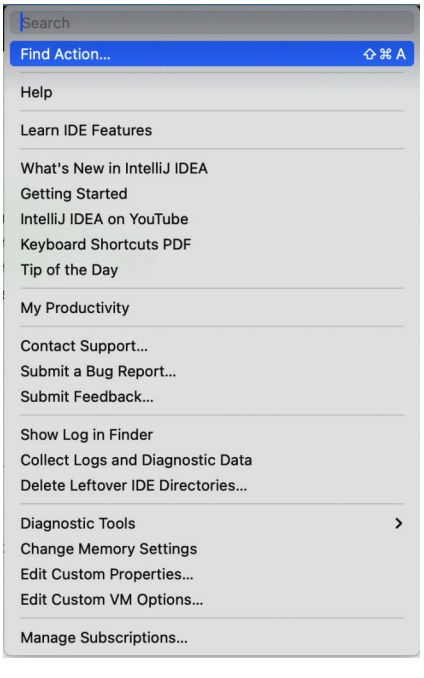


図7-9　IntelliJ IDEA の「Find Action」コマンド

†10「Key Promoter」というプラグインもあり、ショートカットキーを使うのを忘れたときに（親切に）教えてくれます。

また、ショートカットをメニュー階層に表示するのも効果的です。このようにすれば、ユーザーがメニューを使用するうちに、自然とショートカットを覚えるようになります。例えば、モダンな開発ツールにはファイル間を素早く移動する機能が備わっていますが、エディタによってその操作方法は異なる場合があります。例えば、**図7-10** の VSCode の「Go」メニューや、**図7-11** の IntelliJ IDEA の「Navigate」メニューをご覧ください。

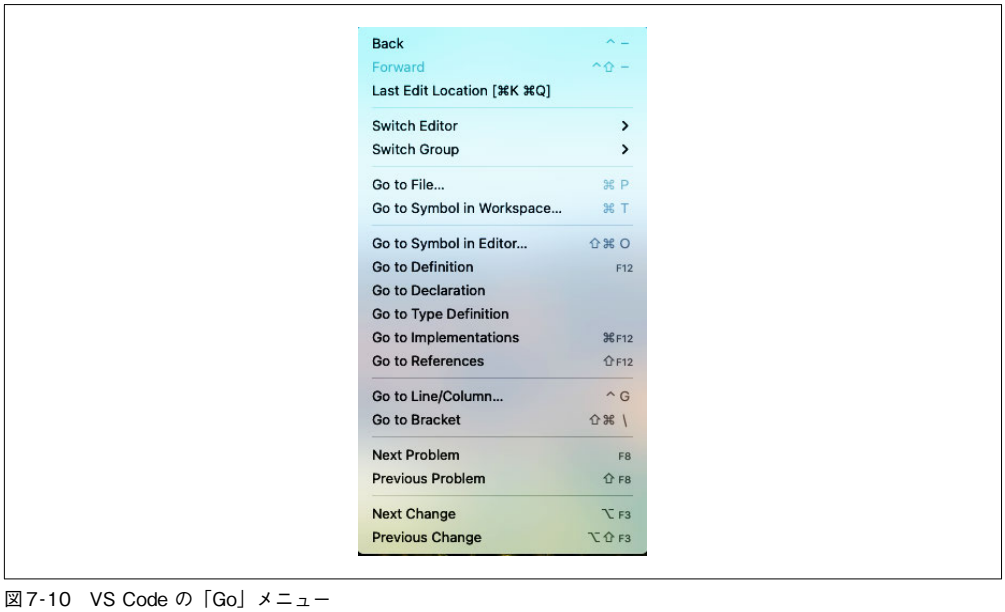


図7-10 VS Code の「Go」メニュー

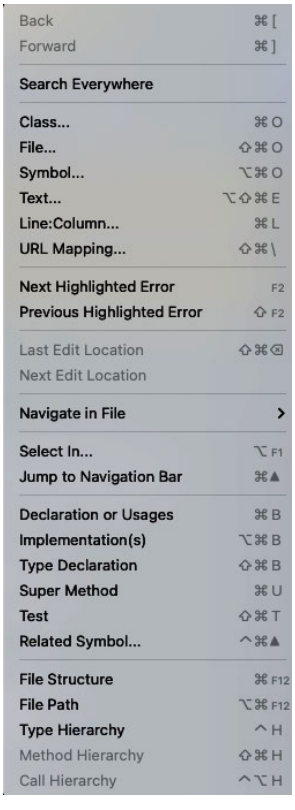


図7-11　IntelliJ IDEA の「Navigate」メニュー

一部のオペレーティングシステムやツールには、メニュー項目を素早く発見して確認できる強力な検索機能が備わっています。例えば、**図7-12** は、IntelliJ IDEA で「file」を検索している様子を示しています。

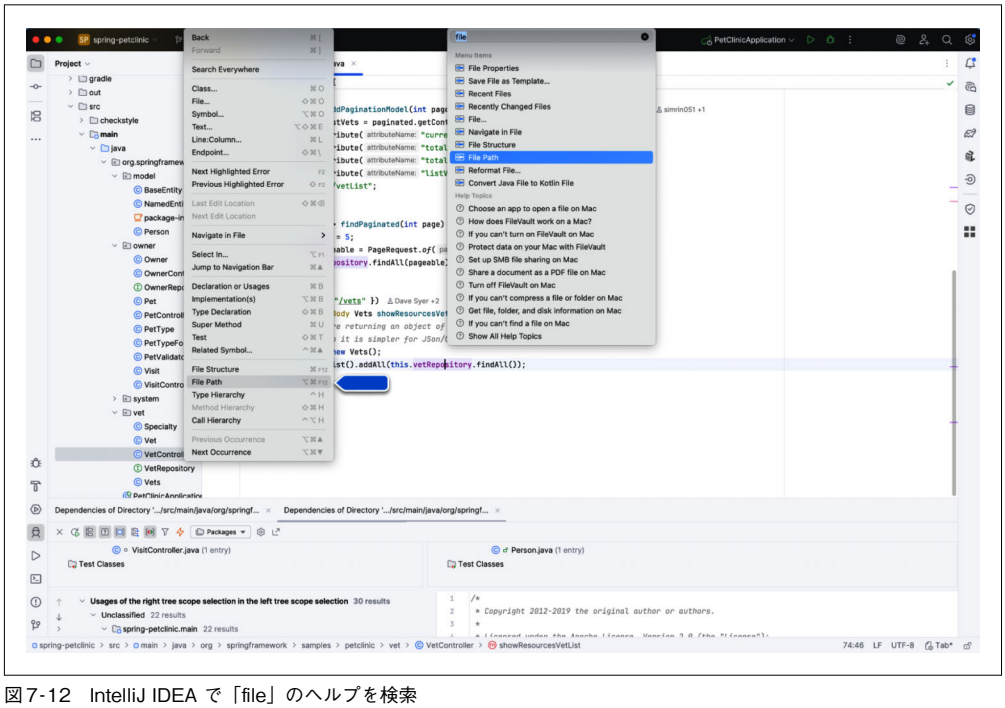


図7-12　IntelliJ IDEA で「file」のヘルプを検索

7.7　適切なエラーメッセージの重要性

いくらそうではないと思いたくても、ユーザーは完璧ではなく、間違いを犯すものです。エラーに対して適切に対応することは、アプリケーション全体のユーザビリティを左右する重要な要素となります。できる限り、そもそもエラーが発生しないようにしたいところですが、ユーザーがどうしても「ハッピーパス」から外れてしまった際には、意味のあるメッセージが表示されるべきです。**図7-13** に示されているような「Interface not registered（インターフェースが登録されていません）」といったものよりも、もっと意味のあるメッセージにしてください。適切なエラーメッセージは、ユーザーが遭遇した問題をデバッグし修正する際にも役立ちます。

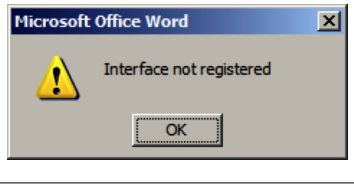


図7-13　あまり役に立たないメッセージ

アプリケーションはユーザーに状況を伝えるだけでなく、ユーザーが操作を元に戻せるようにする必要があります。郵便番号を間違えたからといって、ハードドライブをフォーマットしてしまってはいけません。元に戻す機能や取り消し機能を提供することは、ユーザーが自由に操作を試せるようにするための重要な要素です。操作を元に戻せないようでは、ユーザーは新しい操作を試そうとはしないでしょう。

適切なエラーメッセージは、単にどの行のコードで問題が発生したかを開発者に伝えるだけでなく、ユーザーに対して何が問題なのかをわかりやすく説明し、対処方法を選択できるようにするものでなければなりません。**図7-14** では、Google の Gmail で問題が発生したときのエラーメッセージを紹介しています。意味不明なメッセージを表示するのではなく、インターネット接続に問題があることが一目でわかり、アプリが自動的に再接続を試みる一方で「Try now（今すぐ試す）」という選択肢も提示されます。このメッセージは明確かつ簡潔です。アプリケーションはデフォルトでユーザーに代わって再試行を行いますが、ユーザーが希望すれば、自らアクションを起こすことも可能です。

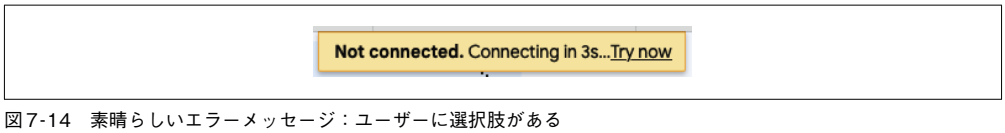


図7-14　素晴らしいエラーメッセージ：ユーザーに選択肢がある

7.8　破壊的な操作

「ユーザーにとって操作を容易にする」という原則には、大きな例外が一つあります。それは、データなどを破壊する操作に関するものです。顧客がデータを編集しにくくすることは、通常は望ましくありません。しかし、アカウントの削除のような取り返しのつかない決定に関しては、プロセスに 1〜3 つの手順を追加することが、多くの場合、適切な対応となります。

例として、GitHub からリポジトリを削除する場合を考えてみましょう。まず、デザイナーは削除などの設定をページの下部に配置しています（これも「近接」の例です）。ユーザーにスクロールさせてその部分まで移動させることで、誤ってこれらの操作をクリックしてしまわないようにしています。「Danger Zone（危険ゾーン）」という明確なタイトルは、これらの設定には危険が伴うことをはっきりと示しています（**図7-15** 参照）。また、ボタンのテキストには目を引くような色使いが施されている点にも注目してください（これは「コントラスト」の活用例です）。

「Delete this repository（このリポジトリを削除）」ボタンをクリックしても、実際にはリポジトリはすぐには削除されません！ 削除を実行するには、その操作を明示的に示す別のボタンをクリックする必要があります（**図7-16**）。

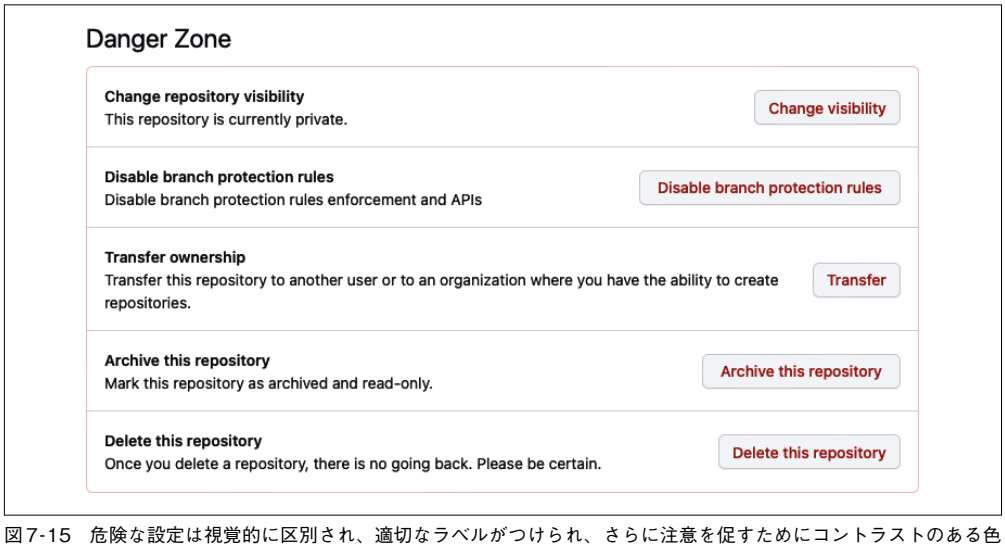


図7-15　危険な設定は視覚的に区別され、適切なラベルがつけられ、さらに注意を促すためにコントラストのある色（この例では赤）が使用されている



図7-16　リポジトリを削除する手順のステップ1：ボタンのラベルには単に「Delete（削除）」と表示するのではなく、このボタンをクリックすることでユーザーが実行する操作を具体的に明記している

その後、これから行われる操作についてさらに詳しく説明し、その操作が元に戻せないものであることを示す警告が表示され、その警告内容を読み理解したことを確認する**別の**ボタンが表示されます（**図7-17** 参照）。



図7-17　リポジトリを削除する手順のステップ２：その操作の結果に関する警告を表示し、ここでもラベルには、ユーザーがクリックすることで実行される操作が詳しく記述されている

しかし、まだ完了ではありません！ リポジトリを誤って削除したくはないため、GitHub はリポジトリ名を完全に記述するよう求めてきます。そして、リポジトリ名を正しく入力した場合にのみ、明確に「Delete this repository（このリポジトリを削除）」と表示された最終的なボタンが有効になります（**図7-18**）。やりすぎのように思えるかもしれませんが、そうしない場合の結果はさらに深刻です。また、このような破壊的な操作は、アプリケーションにおいて比較的まれなユースケースであるはずです。

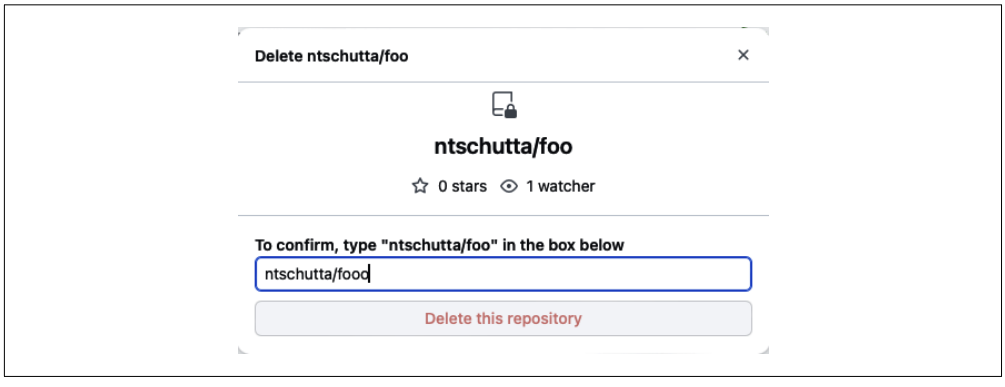


図7-18　リポジトリを削除する手順の最後のステップ：ユーザーがリポジトリ名を正しく入力するまで、「ラベル付き」ボタンは有効にならない

7.9　まとめ

プロジェクトにおいて、ユーザビリティは真っ先に頭に浮かぶことではないかもしれません。しかし、エンドユーザーを常に念頭に置くことが重要です。ユーザビリティの低さや行き当たりばったりのデザインがもたらすコストを過小評価してはいけません。アプリケーションチームにデザインの専門知識があれば確かに役立ちますが、少しの知識でも大きな効果を発揮します！ コントラスト、反復、整列、近接といった原則は、単純で基本的なものに思えるかもしれませんが、考え抜いて組み合わせることで、平凡なアプリケーションと最高水準のユーザー体験との差を生み出します。本章では、これらの原則が実際のアプリケーションにどのように適用されているかを見てきました。読者自身のシステムに対して、これらを活用するためのヒントが学べたのではないかと思います。

優れたユーザーインターフェースを設計するには、アクセシビリティ、ローカライゼーション、国際化についても考慮する必要があります。ユーザビリティの高いアプリケーションを構築するということは、英語に堪能な人や視力が完璧な人だけでなく、あらゆる人が利用できるようにすることです。

7.10　実践に移す

普段、読者は毎日いくつものアプリケーションを使われていることでしょう。その中には、ユーザーインターフェースがうまく設計されているものもあれば、そうでないものもあります。本章を読み終えた今、使い心地の良いアプリケーションと、そうでないアプリケーションの違いについて、より深く理解できたはずです。そこで、今すぐそうしたアプリケーションの一つを見て、コントラスト、反復、整列、近接の例を挙げてみてください。

これまで携わってきたアプリケーションのユーザーインターフェースを確認してみてください。

ユーザビリティ上の明らかな問題はありますか？ 本章を読んで得た知識を活かし、そのアプリケーションにどのような変更を加えますか？ 1時間ほどかけて、インターフェースを再設計してみてください。普段使っているアプリケーションのUIデザインを1〜2時間かけて分析してみてください。本章で学んだ原則がどこに現れていますか？ どのような変更を加えますか？ 1時間かけて、自分自身のノートPCやスマートフォンにあるアプリケーションの中から、これらの原則に反している箇所をどれだけ見つけられるか試してみてください。繰り返しになりますが、どのように変更を加えたいですか？

最後に、実際のユーザーが自身の環境でアプリケーションを使用する様子を観察してください。何がうまくいき、何がうまくいかず、何が改善できるかについてメモを取りましょう。インターフェースについて、気に入っている点や気に入らない点を顧客に尋ねてみてください。顧客はどのような変更を希望していますか？ デフォルト設定は適切ですか？ 操作性は最大限効率的ですか？ 顧客からさまざまな意見が出ても、驚かないでください！

7.11　関連リソース

- 『The Design of Everyday Things, Revised and Expanded Edition』Donald A. Norman著（Basic Books、2013年）
  和書『誰のためのデザイン？ 増補・改訂版』（新曜社、2015年）
- 『The Non-Designer's Design Book, 4th Edition』Robin Williams著（Peachpit Press、2014年）
  和書『ノンデザイナーズ・デザインブック』（マイナビ出版、2016年）
- 『About Face: The Essentials of Interaction Design, 4th Edition』Alan Cooper他著（John Wiley & Sons、2014年）
  和書『ABOUT FACE　インタラクションデザインの本質』（マイナビ出版、2024年）
- 『Designing Interfaces, 3rd Edition』Jennifer Tidwell他著（O'Reilly、2019年）
- The Work of Edward Tufte（https://oreil.ly/QTviB）

8章
データの取り扱い

データは貴重なものであり、システムそのものよりも長く残るものです。
— Tim Berners-Lee、コンピュータ科学者

ソフトウェアエンジニアとしてのキャリアをスタートさせる際、プログラミング言語やフレームワーク、ツールの習得にばかり気を取られがちです。しかし実際には、キャリアを通じて、さまざまな形式のデータを扱うことに多くの時間を費やすことになります。データこそが、読者の仕事の中心なのです。データを中心にアプリケーションを設計し、データを収集、保存、変換し、最終的にはユーザーにとって意味のある形で提供する方法を見出すことになるでしょう。

データと聞くと、まず思い浮かぶのは、何らかのデータベースに保存されているデータかもしれません。しかし、それだけにとどまらず、さまざまなデータ型の理解、適切なストレージソリューションの選定、効率的なデータモデルの設計、クエリの最適化、データ整合性の確保、そして時間の経過に伴うデータの変化への対応などが含まれます。これらのスキルは、開発者としての成功において、基本的なコーディング能力と同様に不可欠なものです。

データを適切に扱う方法を習得するのは難しいことです。開発環境ではうまくいくことが、本番環境では失敗に終わることもあります。顧客データベースの処理速度の遅さから、複雑な移行作業や非効率的なクエリに至るまで、こうした現実的な課題はソフトウェア開発においてよく見られるものであり、それらに備えておくことが重要です。

本章は、データを効果的に扱うための基礎を築き、データに精通した開発者になるためのガイドとなります。さまざまなデータ型の理解、適切なツールの選択、効率的なストレージソリューションの検討、クエリの最適化、そして円滑なマイグレーションの管理について学びます。データベース管理やデータエンジニアリングの深い部分には踏み込みませんが、アプリケーション内のデータについて適切な判断を下し、多くの開発者が陥りがちな落とし穴を回避できるように、すべての開発者に必要となる重要なスキルを身につけることができます。

8.1　データ型と形式を理解する

ソフトウェアを構築する際、データを処理・変換・表示するシステムを構築することがよくあります。小規模アプリケーションであれ、エンタープライズ向けアプリケーションであれ、プロジェクトの成否は、そのデータと、データをどのように構造化し、検証し、処理するかという点にかかっていると言えます。

データ型や形式を扱う際は、以下の点を考慮してください。

利用者

このデータを誰が読み取ったり処理したりするのでしょうか？ 人間でしょうか、それとも機械でしょうか？

性能要件

容量や処理速度が重要でしょうか？

互換性

このデータはどのようなシステムで利用されるのでしょうか？

複雑さ

データ構造はどの程度入れ子になっていますか、あるいはデータ構造は変わりやすいのでしょうか？

検証の必要性

スキーマの検証が必要でしょうか？

この節では、アプリケーション内でさまざまな種類のデータを効果的に扱うための概念について学びます。構造化データと非構造化データの根本的な違いを探り、それぞれの特徴、利点、および限界について理解を深めます。

8.1.1　構造化データと非構造化データ

データには、主に「構造化データ」と「非構造化データ」の２種類があります。**構造化データ**はスプレッドシートのように整理されており、明確なカテゴリや関連性があるため、データベースへの保存や検索が容易です。一方、**非構造化データ**には通常のテキスト、画像、ドキュメントなどが含まれ、これらを扱うには特別なツールが必要となります。アプリケーションにおいて効果的な設計を行うためには、構造化データと非構造化データの違いを理解することが重要です。これらの基本的な概念について見ていきましょう。

8.1.1.1　構造化データ

構造化データとは、あらかじめ定義されたモデルやスキーマに従って整理され、フィールドの型

や関係性が一貫しているデータのことです。スプレッドシートやリレーショナルデータベースの行と列に簡単に収まる情報だと考えてください。プログラミングでは、データの構造はデータモデルを定義するクラスを通じて表現されます。以下の例では、`Customer` という名前の Java クラスが、顧客（Customer）のプロパティを定義しています。

```
public class Customer {
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private LocalDate birthDate;
    private Address address;

    // 簡潔にするためゲッターとセッターは割愛
}
```

構造化データの主な特徴は以下のとおりです。

一貫した形式
: すべてのレコードは同じスキーマに従います。アプリケーションで新しい顧客を作成するたびに、その顧客にはクラスで定義されたプロパティが設定されます。

明確に定義された関係
: データエンティティ間の関係性が明確です。

クエリが容易
: 詳細な検索や絞り込みが可能です。顧客の場合、名前、姓、あるいはその両方を組み合わせて検索できます。

効率的な保管
: 固定長フィールドを持つデータベース向けに最適化されています。`customer` オブジェクトは、テーブルの列にそのまま対応します。

構造化データは、厳密なデータ整合性、複雑なクエリ、そして確立されたビジネスルールが求められる場面で真価を発揮します。その例としては、金融取引、在庫管理、ユーザープロファイルなどが挙げられます。



PostgreSQL や MySQL などのリレーショナルデータベースを使用している場合、おそらく構造化データを扱っていることでしょう。スキーマ内のテーブルや列の定義によって、既存のフィールドや、各フィールドが保持できるデータ型が決まります。

構造化データは、とても魅力的に見えますよね？ 確かにそうですが、限界もあります。フィー

ルドの追加や変更が必要になった場合、データベースのスキーマとアプリケーションコードの両方を修正しなければならないことが多く、このため比較的柔軟性に欠けます。また、構造化データでは、複雑で入れ子になった情報を効率的に表現することが難しいという課題もあります。



AIについて

構造化されたスキーマに合致する、現実味のあるテストデータが必要な場合、AIツールが役立ちます。何十件もの架空の顧客レコードを手作業で作成したり、適切なデータや生成ライブラリを探したりする代わりに、単に「名前、メールアドレス、住所、生年月日が記載された現実味のある顧客レコードを20件、JSON形式で生成してください」と依頼するだけで済みます。AIに対して、「古書店向けのEC商品データを生成してください」や「活動レベルの異なるフィットネスアプリのユーザープロファイルを作成してください」といったように、具体的な要望を伝えればよいのです。これは読者が学習する際やプロトタイプを作成する際、アプリケーションにリアルな感覚を与えるような有意義なデータを素早く入手できるため、特に役立ちます。

8.1.1.2　非構造化データ

非構造化データには、あらかじめ定義されたデータモデルがありません。行や列にきれいに収まるものではありません。非構造化データには、電子メール、ソーシャルメディアの投稿、チャットの記録などの自由形式のテキストが含まれます。その他の例としては、画像、音声、動画、形式が統一されていないログ、明確な構造を持たない文書（PDFやWord）などが挙げられます。

非構造化データを扱う場合、その生データ（処理されていないデータ）という性質上、通常は特別な方法で処理しなければなりません。例えば、テキスト分析のための自然言語処理（NLP）や、画像や音声のためのコンピュータビジョンなどです。

次の例で示す関数は、生の顧客フィードバックを受け取ります。NLPを用いてそのフィードバックが肯定的か否定的かを判断し、キーワードやトピックを特定したうえで、その情報を元のフィードバックと共に保存します。

```
// 非構造化データの処理には、特別な方法が必要になる場合が多い
public void processCustomerFeedback(String feedbackText) {
    // 自然言語処理を用いてセンチメントを抽出する
    Sentiment sentiment = nlpService.analyzeSentiment(feedbackText);

    // 重要なトピックやキーワードを特定する
    List<String> keywords = nlpService.extractKeywords(feedbackText);

    // 元の非構造化テキストと抽出された情報の両方を保存する
    feedbackRepository.save(new Feedback(feedbackText, sentiment, keywords));
}
```

非構造化データの特徴には、次のようなものがあります。

可変形式

一貫したスキーマや構造がありません。

マルチメディアコンテンツ

感情や意見、視覚的な情報など、微妙なニュアンスが含まれています。

従来の方法ではクエリが困難

全文検索などのデータに特化した技術が必要です。

ストレージに関する課題

多くの場合、大規模なオブジェクト向けに最適化された専用システムが必要となります。

非構造化データは、人間のコミュニケーション、創造的なコンテンツ、複雑な実世界情報を扱うアプリケーションにとって、極めて価値があるものです。非構造化データを扱うには、通常、構造化データとは異なるツールが必要となります。SQL クエリの代わりに、Elasticsearch のような全文検索エンジンや NLP ライブラリ、あるいは柔軟に保存するために設計された専用のデータレイクなどを利用することになります。

8.1.2 一般的なデータ形式

ソフトウェアエンジニアとして、キャリアを通じてさまざまなデータ形式を扱うことになるでしょう。読者は、それぞれの形式や長所・短所、そしていつどの形式を使うべきかを理解しておかなければなりません。適切な形式を自分で選べる場合もあれば、すでに決定済みの場合もあります。いずれにせよ、データ形式を理解することで、より良い設計判断を下したり、アプリケーション開発をより効率的に進めたりできるようになります。それでは、キャリアの中で扱うことになる一般的なデータ形式について見ていきましょう。

8.1.2.1 JSON

JavaScript Object Notation（JSON）は、その読みやすさと簡潔さから、モダンアプリケーションにおけるデータ交換の事実上の標準形式となっています。その名称からは JavaScript 専用であるかのように思われますが、実際には言語に依存せず、オブジェクト、配列、文字列、数値、ブール値、null 値などの型をサポートしています。以下は、ユーザーに関する情報を含む JSON オブジェクトの例です。

```
{
  "id": 1001,
  "firstName": "Jane",
  "lastName": "Smith",
  "email": "jane.smith@example.com",
  "birthDate": "1985-03-15",
  "address": {
```

```
    "street": "123 Main Street",
    "city": "Boston",
    "state": "MA",
    "zipCode": "02108",
    "country": "USA"
  }
}
```

JSONは、Web API、設定ファイル、ドキュメントデータベースに適した選択肢です。その制限事項としては、コメントや日付形式、エンコーディングされていないバイナリデータに対応していない点が挙げられます。モダンアプリケーションの多く、特にJavaScriptをフロントエンドに採用しているアプリケーションにおいては、JSONが自然な選択肢となります。

8.1.2.2　XML

XML（eXtensible Markup Language）は、JSONよりも記述が冗長ですが、XMLスキーマ定義（XSD、XML Schema Definition）と文書型定義（DTD、Document Type Definition）を通じて堅牢な検証機能を提供します。XSDはスキーマ定義機能を提供し、XMLドキュメントのデータ型、制約、構造ルールを指定できます。また、XPath式を使用してデータを正確に抽出したり、XSLT（eXtensible Stylesheet Language Transformations）を用いてXMLドキュメントを他の形式に変換したりすることも可能です。

XMLは名前空間をサポートしています。異なる語彙やXMLスキーマに属する要素に一意の識別子を付与することで、複雑なドキュメントにおける名前衝突を回避できます。非常に大規模なXMLドキュメントを処理する場合には、ストリーム処理、イベント駆動型解析、メモリ効率の高い専用パーサーなどを利用することで、パフォーマンスを大幅に向上させることができます。以下は、ユーザーに関する情報を含むXMLドキュメントの例です。

```
<?xml version="1.0" encoding="UTF-8"?>
<customer>
    <id>1001</id>
    <firstName>Jane</firstName>
    <lastName>Smith</lastName>
    <email>jane.smith@example.com</email>
    <birthDate>1985-03-15</birthDate>
    <address>
        <street>123 Main Street</street>
        <city>Boston</city>
        <state>MA</state>
        <zipCode>02108</zipCode>
        <country>USA</country>
    </address>
</customer>
```

XMLは、エンタープライズシステムやドキュメント形式（DOCXやSVGなど）、複雑なアプ

リケーションの設定ファイルにおいて、依然として広く利用されています。スキーマの検証、名前空間、ドキュメント指向システムとの相互運用が重要な場合は、XML を選択してください。

8.1.2.3 CSV

CSV（Comma-Separated Values）は、表形式のデータをプレーンテキストとして保存する形式です。値を区切るにはコンマを、レコードを区切るには改行を使用します。通常、最初の行には列見出しが含まれます。CSV は扱いやすい形式ですが、組み込みのデータ型定義や、特殊文字を扱うための一貫したルールがありません。以下の例には、ユーザー情報の列見出しと、サンプルデータが 1 行含まれています。

```
id,firstName,lastName,email,birthDate,address.street,address.city,address.state,
address.zipCode,address.country
1001,Jane,Smith,jane.smith@example.com,1985-03-15,"123 Main Street",
Boston,MA,02108,USA
```

CSV は、データのエクスポートや簡易なデータ交換、表計算アプリケーションとの相互運用に適しています。構造が単純な表形式データには適していますが、国際文字やフィールド内のコンマ、複雑なデータ型については注意が必要です。



CSV を扱う際は、常に特殊ケースへの対処方法を考慮してください。具体的には、空のフィールド、コンマや引用符を含むフィールド、フィールド内の改行などです。ほとんどの CSV ライブラリには、こうしたケースに対応するオプションが用意されていますが、利用する側でそれらを明示的に設定する必要があります。

8.1.2.4 YAML

YAML（YAML Ain't Markup Language）は、JSON や XML よりも人にとって読みやすい構文を備えており、コメント、参照、複数行のテキストをサポートしています。インデントを使うことで構造が定義され、可読性が向上しています。しかし、スペースを 1 つでも間違えると解析エラーが発生するおそれがあるため、書式設定には細心の注意が必要です。ベストプラクティスとしては、一貫したインデント（通常はスペース 2 文字）を使用すること、デプロイ前に YAML ファイルを検証すること、書式設定の問題を早期に発見するために YAML 対応エディタを使用することが挙げられます。以下は、YAML 形式で定義されたユーザーの例です。

```
customer:
  id: 1001
  firstName: Jane
  lastName: Smith
  email: jane.smith@example.com
  birthDate: 1985-03-15
```

```
  address:
    street: 123 Main Street
    city: Boston
    state: MA
    zipCode: 02108
    country: USA
```

YAML は設定ファイルに最適であり、特に Docker Compose、Kubernetes、CI/CD パイプラインなどの DevOps ツールにおいて活用されています。可読性が高いため、人間が編集するファイルには最適ですが、インデントや特殊文字には注意が必要です。

多くのアプリケーションでは、API には JSON、設定には YAML、顧客データのダウンロードには CSV など、複数の形式が使用されています。どのデータ形式を選択するのが適切なのかは、具体的な要件や制約によって異なります。

8.1.3　特化したデータに関する考慮事項

前節で学んだ一般的な形式以外にも、あまり一般的ではありませんが、扱いに工夫が必要な特化したデータ型を扱うことになるでしょう。このようなデータ型についても理解しておくことが重要です。

8.1.3.1　バイナリデータ

バイナリデータには、画像やドキュメントから音声ファイル、暗号化されたコンテンツに至るまで、あらゆるものが含まれます。テキストベースの形式とは異なり、バイナリデータは適切なエンコードとデコードを行わなければ、直接読み取ったり操作したりすることはできません。以下の例では、画像を読み込み、それをバイナリデータ（この場合はバイト配列）に変換する方法を示しています。

```
// 画像ファイルをバイナリデータとして読み込む
try (FileInputStream fis = new FileInputStream("dogs-playing-poker.png");
     ByteArrayOutputStream bos = new ByteArrayOutputStream()) {

    byte[] buffer = new byte[1024];
    int bytesRead;
    while ((bytesRead = fis.read(buffer)) != -1) {
        bos.write(buffer, 0, bytesRead);
    }

    byte[] imageBytes = bos.toByteArray();
    // これでバイナリデータを処理できるようになる
}
```

画像ファイルをバイト配列としてメモリに読み込んだら、それを使ってさまざまな処理を行うことができます。

- GUI アプリケーションで画像を表示する
- 画像を処理または編集する（サイズ変更、トリミング、フィルターの適用など）
- 異なる画像形式に変換する（PNG から JPEG など）
- サーバーやクラウドストレージにアップロードする
- HTTP リクエストに含めて送信する
- PDF ドキュメントに埋め込む
- 画像からメタデータを抽出する

バイナリデータを扱う際には、いくつかの重要なポイントがあります。特定のファイル形式用に設計された専用ライブラリを利用してください。バイナリデータをテキスト形式に含める必要がある場合は、Base64 エンコーディングが便利です。16 進数表記を使用すれば、デバッグや分析の際に個々のバイトを確認するのがわかりやすくなります。大容量のバイナリファイルを扱う場合は、メモリ管理に細心の注意を払ってください。また、破損したデータファイルや不完全なデータファイルを検出できるよう、常にエラー処理を実装してください。

8.1.3.2 日付と時刻のデータ

ソフトウェアエンジニアリングのコミュニティでは、コンピュータサイエンスにおいて最も難しい問題は「名前のつけ方」と「キャッシュの無効化だ」と冗談交じりに言われることがよくあります（3 章のコラム「名前をつけるのは難しい」参照）。日付と時刻の処理は、一見単純そうに見えて実は複雑であるため、そのリストの上位に名を連ねるくらいのものです。日時データの取り扱いには、タイムゾーン、夏時間の調整、さまざまな暦システム、フォーマットの変換などに起因する数多くの課題が存在します。

以下の Java アプリケーションでは、ローカルの日付と時刻を取得し、特定のタイムゾーンに関連付け、移動時間を加算したうえで、別のタイムゾーンに変換します。最後に結果として得られた `ZonedDateTime` を表示用にフォーマットすることで、目的の地域での到着時刻を正確に表します。

```
// Java の java.time パッケージによるモダンな日付・時刻の処理
// ある特定の日時を作成する
LocalDateTime departure = LocalDateTime.of(2025, 3, 29, 3, 20);

// タイムゾーンの処理をする
ZonedDateTime departureNY =
      ZonedDateTime.of(departure, ZoneId.of("America/New_York"));
ZonedDateTime arrivalTokyo =
      departureNY.plusHours(14).withZoneSameInstant(ZoneId.of("Asia/Tokyo"));

// 表示用にフォーマットする
DateTimeFormatter formatter =
      DateTimeFormatter.ofPattern("MMM dd, yyyy HH:mm z");
String formattedArrival = arrivalTokyo.format(formatter);
```

日付・時刻データを扱う際には、以下のようなことを念頭に置いてください。

- 日付は常に内部では UTC で保存し、表示時のみローカルタイムゾーンに変換する
- システム間の日付・時刻のやり取りには ISO 8601 形式を使用する
- 独自の解決策を構築するのではなく、モダンな日付・時刻ライブラリを活用する
- ユーザーインターフェースでは、タイムゾーンを明確に表示する

8.1.3.3　大規模なデータセット

アプリケーションの規模が拡大するにつれて、扱うデータ量も増加します。かつては基本的なデータ型を使用する単純なシステムであったものが、いつしかメモリ上でまとめて処理しきれないほど巨大なデータセットが含まれるようになり、特別なアプローチが必要となります。以下の例では、大規模な CSV ファイルを処理する方法を示します。このコードでは、レコードを 1 つずつ読み込んで処理するストリーム処理を用いています。

```
// 大規模 CSV ファイル用のストリーム処理手法
try (Reader reader = new FileReader("massive-data.csv");
CSVParser parser = CSVFormat.DEFAULT.withHeader().parse(reader)) {

parser.forEach(record -> {
   // 1 レコードずつ処理する
   // すべてのデータをメモリに読み込まずに処理する
   processRecord(record);

   });
}
```

大規模なデータセットを扱う際は、以下の点を考慮する必要があります。

- データを段階的に処理するストリーム処理を採用する
- API のレスポンスやユーザーインターフェースにページネーションを実装する
- インデックス作成やクエリのチューニングなど、データベースの最適化を検討する
- 非常に大規模なデータセットに対しては、データウェアハウスや分散処理フレームワークなどの専用テクノロジーを検討する
- 複数のスレッドが共有データに同時にアクセスする際のデータ競合を防ぐため、ロック、アトミック操作、不変データ構造を用いて適切な並行制御を実装する
- 開発の初期段階で、実際のデータ量を用いてテストを行う

アプリケーションの複雑さや規模が大きくなるにつれ、こうした専門的な考慮事項がますます重要になってきます。すぐにこれらすべての分野の専門家になる必要はありませんが、こうした考慮事項を認識しておくことで、標準的なアプローチでは不十分な場合や、より専門的なソリューションや専門知識を求めるべきタイミングを見極めるのに役立ちます。

この節では、構造化データや非構造化データなど、さまざまなデータ型について学びました。また、今後のキャリアを通じて扱うことになる一般的なデータ型や特化したデータ型についても触れました。データ型の概要がわかったところで、次はデータを効果的に保存する方法について見ていきましょう。

8.2 データを効果的に保存する

扱うデータ構造を決定したら、データに適した保存方法を選択する必要があります。早い段階で適切な保存方法を選択しておけば、後々のトラブルを未然に防ぐことができます。この節では、アプリケーションの要件に合わせてデータベースを選ぶ方法を学び、さまざまなデータベースの種類ごとのメリットと限界について学びます。これらの基礎知識は、アプリケーションのパフォーマンス、スケーラビリティ、保守性に関して、根拠に基づいた判断を下すのに役立ちます。

8.2.1 データベースの種類とそのユースケース

次にアプリケーションを開発するときに、どのデータベースを使用するのか選ぶ際は、ソーシャルメディアのトレンドに流されたり、同僚の選択を真似たりしないでください。アプリケーションの具体的な要件を踏まえて、十分な情報に基づいて判断を下すようにしましょう。ストレージソリューションを選ぶ際には、以下の重要な要素を考慮してください。

データ構造の複雑さ
: データの関係性はどれほど複雑ですか？

読み取りと書き込みのパターン
: アプリケーションでは、読み取り操作と書き込み操作のどちらがより多く行われるでしょうか？

クエリの複雑さ
: データに対してどのような質問をしますか？

スケーラビリティの要件
: 今後、データ量はどのくらい増加するでしょうか？

一貫性の要件
: アプリケーションにとって、データの一貫性はどれほど重要ですか？

ここでは、データベースの要件が異なる 2 つのアプリケーションについて考えてみましょう。まず、知識や感想を共有するためのブログアプリケーションを考えてみましょう。エンティティ（ユーザー、投稿、タグ、コメントなど）とその関係を定義することから始めます。ユーザーは多くの投稿を書き、投稿には多くのタグやコメントがつきます。このように明確に定義された構造か

ら、リレーショナルデータベースが適切な選択であることが示唆されます。

もう一つは、オーダーメイド商品の製品カタログを作成する場合を想像してみてください。この場合、製品は統一された構造を持っておらず、頻繁に変更される可能性のある属性を持っています。このような状況では、ドキュメントデータベースのほうが適しているでしょう。



データベースの選択はアプリケーションの進化に合わせて変更可能です。また、一度決めた選択が絶対的なものでもありません。多くの成功プロジェクトはシンプルに始めて、ニーズの変化に応じてより複雑なソリューションへ移行していきます。最初から「完璧な」ソリューションを選ばなければならなかったり、複雑すぎる設計をしたりする必要はありません。

それでは、データベースの種類とそれぞれをいつ選択すべきかについて見ていきましょう。

8.2.1.1　リレーショナルデータベース

リレーショナルデータベースは、データを行と列からなるテーブルに整理し、キーを通じてテーブル間の関係を確立します。これらは堅牢な数学的基礎（関係代数）に基づいており、ACID 特性を通じてデータの整合性を保証します。

不可分性（Atomicity）
: トランザクションは「すべてか、あるいは何もなし」という性質を持っています。トランザクション内の変更は、すべて正常に完了するか、あるいは一つも完了しないかのいずれかです。一部でも失敗した場合、データの一貫性を保つためにトランザクション全体がロールバックされます。

一貫性（Consistency）
: 各トランザクションの前後を通じて、データベースは常に有効な状態を維持します。すべてのデータ整合性ルール、制約、関係性が保持されるため、データベースが不正な状態になることは決してありません。

独立性（Isolation）
: 並行するトランザクションは互いに干渉しません。複数のトランザクションが同時に実行されている場合でも、各トランザクションは独立して実行されているかのように振る舞うため、ダーティリードやロストアップデートといった問題を防ぐことができます。

永続性（Durability）
: トランザクションがコミットされると、その変更内容は永続的に保存され、システム障害が発生しても失われることはありません。コミットの直後にデータベースがクラッシュしたり、電源が切れたりした場合でも、データは保持されます。

以下のような場合には、リレーショナルデータベースを使用してください。

- データ内のエンティティ間に明確な関係性があります。
- 複雑なクエリや結合が必要です。
- トランザクションとデータの一貫性が極めて重要です。
- 構造があまり変化しないデータを使用します。

代表的なものとしては、PostgreSQL、MySQL、Oracle、SQL Server などが挙げられます。Java でリレーショナルデータベースにテーブルを作成する簡単な例を以下に示します。

```
// JDBC を使用してテーブルを作成
String createUserTable = """
           CREATE TABLE users (
               id SERIAL PRIMARY KEY,
               username VARCHAR(50) UNIQUE NOT NULL,
               email VARCHAR(100) UNIQUE NOT NULL,
               created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
           )
       """;

String createPostTable = """
           CREATE TABLE posts (
               id SERIAL PRIMARY KEY,
               user_id INTEGER REFERENCES users(id),
               title VARCHAR(200) NOT NULL,
               content TEXT NOT NULL,
               published_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
           )
       """;

try (Connection conn = dataSource.getConnection();
    Statement stmt = conn.createStatement()) {
   stmt.executeUpdate(createUserTable);
   stmt.executeUpdate(createPostTable);

   // ユーザーを挿入
   String insertUser = "INSERT INTO users (username, email) VALUES (?, ?)";
   try (PreparedStatement pstmt = conn.prepareStatement(insertUser)) {
       pstmt.setString(1, "john_doe");
       pstmt.setString(2, "john@example.com");
       pstmt.executeUpdate();
   }
}
```

8.2.1.2 ドキュメントデータベース

ドキュメントデータベースは、固定的なテーブルではなく、柔軟性のある JSON 形式のドキュメントとしてデータを保存します。各ドキュメントは異なる構造を持つことができるため、リレーショナルデータベースよりも柔軟性が高くなります。

次のような場合には、ドキュメントデータベースを使用してください。

- レコードごとにデータ構造が異なります。
- 大規模なデータセットに対応するために水平スケーラビリティが必要です。
- アプリケーションの要件が頻繁に変更されます。
- コンテンツ管理システムやカタログを構築しています。

代表的なものとしては、MongoDB、Couchbase、Firebase Firestore などが挙げられます。
ドキュメントデータベースにブログ記事を保存する例を次に示します。

```
// MongoDB の Java ドライバーを使用
Document post = new Document()
    .append("title", "Getting Started with MongoDB")
    .append("author", new Document("name", "Jane Developer")
    .append("email", "jane@example.com"))
    .append("tags", Arrays.asList("database", "nosql", "mongodb"))
    .append("comments", Arrays.asList(
        new Document("user", "reader1").append("text", "Great article!"),
        new Document("user", "reader2").append("text", "Thanks for sharing!")
    ))
    .append("publishedAt", new Date())
    .append("createdAt", new Date());

try {
    collection.insertOne(post);
} catch (MongoException e) {
    log.error("Error: " + e.getMessage());
}
```

8.2.1.3　キーバリューストア

キーバリューストアは、NoSQL データベースの中で最もシンプルな形態であり、データをキーと値のペアのコレクションとして格納します。単純な操作においては非常に高速ですが、クエリ機能には制限があります。

キーバリューストアは、次のような場合に使用してください。

- 極めて高速な読み書き性能が必要です。
- データアクセスのパターンが単純です（主にキーによるアクセス）。
- キャッシュレイヤー、セッションストア、設定管理機能を構築しています。
- 複雑なクエリよりもスピードを重視するユースケースです。

代表的なものとしては、Redis、Amazon DynamoDB、Riak などがあります。
セッション管理にキーバリューストアを使用する例を以下に示します。

```
// Jedis を利用（Redis の Java クライアント）
try (Jedis jedis = jedisPool.getResource()) {
```

```
    // 有効期限 30 分でセッションを保存する
    String sessionId = generateSessionId();
    jedis.setex("session:" + sessionId, 1800, userJson);

    // セッションを取得する
    String storedSession = jedis.get("session:" + sessionId);
}
```

8.2.1.4 グラフデータベース

グラフデータベースは、情報をノード（エンティティ）とエッジ（エンティティ間の関係）として表現することで、相互に複雑に関連したデータを効率的に保存・検索できるように設計されています。テーブルや外部キーを使用するリレーショナルデータベースとは異なり、グラフデータベースでは、関係そのものがデータモデルの中心的な概念となっています。グラフデータベースでは、一般的なクエリ言語が利用でき、データ内のパターンを見つけ出したり、つながりを追跡したりすることが容易です。

以下のような場合には、グラフデータベースを使用してください。

- データが相互に密接に関連しています。
- 関係性はデータそのものと同じくらい重要です。
- 複雑な探索や経路探索を行う必要があります。
- ソーシャルネットワーク、レコメンデーションエンジン、ナレッジグラフを構築しています。

代表的なものとして、Neo4j、JanusGraph、Amazon Neptune などがあります。

以下に、Java を使用した Neo4j の具体的な使用例を示します。プロパティを持つ `Person`（ユーザー）ノードを作成し、2 人のユーザー間に `FOLLOWS`（フォロー）関係を確立する方法を示しています。

```
// Neo4j の Java ドライバーを使用
try (Session session = driver.session()) {
    session.writeTransaction(tx -> {
       tx.run("CREATE (user:Person {name: $name, email: $email})",
              parameters("name", "Alice", "email", "alice@example.com"));
       tx.run("MATCH (a:Person {name: $person1}), (b:Person {name: $person2}) " +
              "CREATE (a)-[:FOLLOWS]->(b)",
              parameters("person1", "Alice", "person2", "Bob"));
       return null;
    });
}
```

グラフデータベースは、従来のリレーショナルデータベースでは見つけにくいような、隠れたパターンや関連性を明らかにするのに非常に優れています。シンプルなモデルを採用しており、データ同士の関連性を見つけるのに適しています。アプリケーションが情報間の関係性を理解し、活用

する必要がある場合、グラフデータベースが適しています。

8.2.1.5　ベクトルデータベース

ベクトルデータベースは、高次元の数値ベクトルを扱い、類似度検索を効率的に行えるように設計されたデータベースシステムです。テキストや数値などの離散的な値を格納する従来のデータベースとは異なり、ベクトルデータベースは、データの意味を表す数学的表現（埋め込み表現）を格納します。これにより、AIを活用したアプリケーションを実現できます。

ベクトルデータベースについて言えば、理解しておくべき重要な概念が2つあります。それは、ベクトル埋め込みと類似度検索です。

ベクトル埋め込みとは、データを数値ベクトルとして表現したもののことです。高次元空間におけるデータポイントを表す数値の配列（通常は数百から数千の次元）が用いられます。類似した項目は、この空間内でベクトルが互いに近接しており、一方、類似していない項目は互いに離れた位置にあります。

類似度検索とは、類似したデータを見つける検索方法です。これは、探しているものに最も近い項目を見つけるという、基本となる操作です。「これと完全に一致するものを見せて」と言うのではなく、「これと似たようなものを教えて」と尋ねるようなものだと考えてください。

ベクトルデータベースは、多くのAIアプリケーションで利用されており、以下のような場合に利用されます。

意味的な検索
　キーワードではなく意味に基づいてドキュメントを検索する

レコメンデーションシステム
　類似の製品、コンテンツ、ユーザーを提案する

RAG（Retrieval-Augmented Generation、検索拡張生成）
　大規模言語モデル（LLM）に適切なコンテキストを提供する

画像・動画検索
　視覚的に似たメディアを探す

異常検知
　高次元データにおける外れ値の特定

重複排除
　類似コンテンツの検出

代表的なものとしては、Pinecone、Weaviate、Chroma、Qdrantなどがあります。

適切なデータベースを選択するには、データの構造とアプリケーションの要件を理解することが重要です。リレーショナルデータベースは、明確に定義された関係性に対して構造と一貫性を提供

します。ドキュメントデータベースは、スキーマの変更に対応する柔軟性を提供します。キーバリューストアは、単純なデータアクセスパターンにおいて高速性を発揮します。グラフデータベースは、複雑な関係性の管理に優れています。ベクトルデータベースは、多くの AI アプリケーションを支えています。

データベース選びがキャリアの成否を左右するものではないということを忘れないでください。ユーザー数が数百万人に達した時点でデータベースの切り替えが必要になるかもしれませんが、それは実は成功の証なのです！ 重要なのは現在のニーズに基づいて選択し、前へ進むことです。

8.2.2 データの永続化と管理

データベースの種類について理解できたところで、次はデータベースを効果的に活用する方法について見ていきましょう。どのような言語やフレームワークを使用している場合でも、おそらくいくつかの抽象化レベルが存在するはずです。適切なデータベースを選ぶのと同じように、開発中のアプリケーションにとってどの抽象化レベルが適切かを見極める必要があります。

このような見極めでは、チームの状況も意思決定において大きく影響します。SQL の経験はあるものの、Hibernate のようなオブジェクトリレーショナルマッパーの経験がまったくない開発者が多数いるチームの場合、プロジェクトに Java Persistence API を導入するのはあまり得策ではないかもしれません。

この節では、さまざまな永続化パターン、接続管理、一貫性モデル、データ増加への備えについて学びます。ここでの例のほとんどは Java で書かれていますが、もし Java を使用していなくても心配ありません。そのような場合には、パターンや抽象化のレベルに注目し、そしてそれらを読者が使っている言語やフレームワークにどのように適用できるかを考えるようにしてください。

データ永続化パターンについて、いくつかの選択肢を見ていきましょう。データ永続化においてどの抽象化レベルを選択するかは、開発速度、コードの保守性、アプリケーションのパフォーマンスに大きな影響を及ぼします。ここには万能な解決策はありません。選択にあたっては、アプリケーションの複雑さ、チームの専門知識、プロジェクトのスケジュールを考慮に入れる必要があります。

8.2.2.1 直接データベースアクセス

直接データベースアクセスは、最も抽象度が低い方法です。生の SQL やデータベース固有のクエリ言語を記述します。このアプローチではデータアクセスを完全に制御できますが、データベースに関する深い知識が必要となります。

次の例では、クエリを使って `users` テーブルから必要な列を選択し、そのクエリを実行した後、結果セットを順に処理します。

```
// 直接 JDBC アクセスの例
String query = "SELECT id, username, email FROM users WHERE active = true " +
            "ORDER BY created_at DESC";
```

```
try (Connection conn = dataSource.getConnection();
     PreparedStatement stmt = conn.prepareStatement(query);
     ResultSet rs = stmt.executeQuery()) {

    while (rs.next()) {
        User user = new User(rs.getLong("id"),
                             rs.getString("username"),
                             rs.getString("email"));
        users.add(user);
    }
}
```

8.2.2.2　リポジトリパターン

リポジトリパターンは、ビジネスロジックとデータをアクセスするコードの間に抽象化レイヤーを提供します。このレイヤーは、ドメインオブジェクトにアクセスする、コレクションのようなインターフェースとして機能し、データベース操作の複雑さを隠蔽します。このパターンには、次のような重要な利点があります。

関心の分離
: データアクセスロジックをビジネスロジックから分離する

テスト
: モックリポジトリを使用できるようにすることで、ユニットテストを容易にする

コードの構成
: データアクセスロジックを 1 か所に集約する

以下のコード例を見ると、`UserRepository` がこれらの原則をどのように実現しているかがわかります。

1. リポジトリは、すべての SQL クエリとデータベース操作をカプセル化しています。
2. データベースの抽象化により、アプリケーションはデータベーススキーマの変更や実装の詳細から隔離されます。
3. 明確でドメインに特化したインターフェースを提供します（`findActiveUsers`）。
4. エラー処理は、`RepositoryException` を通じて標準化されています。
5. 接続管理は、`try-with-resources` によって適切に処理されます。

```
// リポジトリパターンの例
public class UserRepository {
    private final DataSource dataSource;
```

```
    public UserRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public List<User> findActiveUsers() {
        List<User> users = new ArrayList<>();
        String query = "SELECT id, username, email FROM users WHERE " +
                "active = true ORDER BY created_at DESC";

        try (Connection conn = dataSource.getConnection();
             PreparedStatement stmt = conn.prepareStatement(query);
             ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                User user = new User(rs.getLong("id"),
                                     rs.getString("username"),
                                     rs.getString("email"));
                users.add(user);
            }
        } catch (SQLException e) {
            throw new RepositoryException("Failed to find active users", e);
        }

        return users;
    }
}
```

8.2.2.3 オブジェクトリレーショナルマッピング

Hibernate、JPA、Entity Framework などの**オブジェクトリレーショナルマッピング（ORM、Object-relational mapping**）フレームワークは、データベースとのやり取りにおいて最も抽象化レベルの高い方法です。これらのツールは、ドメインオブジェクトをデータベーステーブルに直接マッピングすることで、オブジェクト指向プログラミングとリレーショナルデータベースの間のギャップを埋めます。これにより、ほとんどの場合、手作業での SQL 記述や結果のマッピング作業が不要になります。

以下に示すコードは、JPA アノテーションを使用して、データベースマッピングのメタデータを持つ、`User` エンティティを定義しています。リポジトリでは、生の SQL の代わりに簡略化されたクエリ言語を使用して、作成日順に並べ替えたアクティブなユーザーを取得します。

```
// JPA/Hibernate の例
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;
```

```
    private String email;
    private boolean active;
    private LocalDateTime createdAt;

    // ゲッターとセッターは割愛
}

// JPA でのエンティティの使用
@Repository
public class UserJpaRepository {
    @PersistenceContext
    private EntityManager entityManager;

    public List<User>  findActiveUsers() {
        return entityManager.createQuery(
                "SELECT u FROM User u WHERE u.active = true " +
                        "ORDER BY u.createdAt DESC",User.class)
                .getResultList();
    }
}
```



初心者の開発者がよく犯す間違いの一つは、抽象化レベルが高いもの（ORM など）が常に優れていると思い込んでしまうことです。場合によっては、直接 SQL を使用するほうがパフォーマンスが向上し、より細かく制御できることもあります。ORM は導入が容易な反面、デバッグが難しくなることがよくあります。一つの手法を無条件に使用するのではなく、自身の具体的なニーズに合わせて、そのメリットとデメリットを適切に評価することを身につけましょう。

8.2.3　データベース接続とトランザクション

使用するデータベースと、そのデータベースとの連携方法を決定したら、アプリケーションのパフォーマンスと信頼性を向上させるために考慮する点が他にもいくつかあります。データベース接続は、アプリケーションがデータベースにどのように接続するかを定義します。トランザクションは、単一の作業単位内での複数の操作がどのように動作するかを定義します。

8.2.3.1　データベース接続

データベースとその接続方法について初めて学んだ際、おそらく単一の接続を想定したコードを書かれたことでしょう。実際のアプリケーション開発に取り組む際、適切な接続管理とはどういうものか理解しておくと、パフォーマンスと信頼性において極めて役立ちます。ほとんどのアプリケーションでは、操作ごとに新しい接続を作成するのではなく、接続プールを通じてデータベース接続を再利用しています[†1]。

†1　**接続プール（コネクションプール）**とは、メモリ内に保持されるデータベース接続のキャッシュのことです。将来データベースへのリクエストが必要になった際に、データベース接続を再利用できるようにすることで、新しい接続を何度も確立するオーバーヘッドを排除します。

以下の例では、Java で Hikari Connection Pool（HikariCP）を使用してローカルの PostgreSQL データベースに接続する方法を示しています。

```
// HikariCP 接続プール設定
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:postgresql://localhost:5432/myapp");
config.setUsername("user");
config.setPassword("password");
config.setMaximumPoolSize(10);

HikariDataSource dataSource = new HikariDataSource(config);
```

Java で Spring のようなフレームワークを使っている場合、内部で自動的に設定・処理されます。

8.2.3.2 トランザクション

データベーストランザクションとは、1 つ以上のデータベース操作（挿入、更新、削除など）を含む論理的な処理単位のことです。単一の不可分な操作として実行され、完全に成功するか、完全に失敗するかのいずれかとなります。トランザクションは、部分的な変更が適用されないようにすることでデータの整合性を維持します。以下に示す例は、口座間の送金を行っています。トランザクションを使用して両方の操作が成功するか、あるいは両方とも失敗するようにすることで、口座残高の整合性を維持しています。

```
// 簡単なトランザクションの例
try (Connection conn = dataSource.getConnection()) {
    conn.setAutoCommit(false);

try (PreparedStatement withdrawStmt =
        conn.prepareStatement(
            "UPDATE accounts SET balance = balance - ? WHERE id = ?");
     PreparedStatement depositStmt =
        conn.prepareStatement(
            "UPDATE accounts SET balance = balance + ? WHERE id = ?")) {

    // 振出口座から引き出す
    withdrawStmt.setBigDecimal(1, amount);
    withdrawStmt.setLong(2, fromAccountId);
    withdrawStmt.executeUpdate();

    // 指定口座への入金
    depositStmt.setBigDecimal(1, amount);
    depositStmt.setLong(2, toAccountId);
    depositStmt.executeUpdate();

    conn.commit();
} catch (SQLException e) {
    conn.rollback();
```

```
    throw e;
}
```



トランザクション管理の責任は、使用するデータベースシステムやクライアントの設定によって異なります。データベースは通常、適切に定義されたトランザクションに対して ACID 特性を保証します（次節で説明）。一方で、トランザクションの境界をどこに設定するのかは、一般的に、アプリケーション開発者の責任となります。関連する操作を同じトランザクションのスコープにまとめるようにしなければなりません。

8.2.4　一貫性モデルとキャッシュ戦略

本章の冒頭で、データがほとんどのアプリケーションの根幹をなすものであることを学びました。つまり、あらゆるシステムの基盤は、高いパフォーマンスを発揮しつつ、データを確実に管理する能力にあるということです。本節では、こうしたニーズに応えるための 2 つの重要な概念、一貫性モデルとキャッシュ戦略について解説します。これらはしばしば別々に議論されます。しかし、モダンなデータ駆動型アプリケーションにおいては、これらの要素が連携して、データの信頼性と処理速度のバランスを保っています。

8.2.4.1　一貫性モデル

これまでに学んだトランザクションの概念を踏まえると、**一貫性モデル**は、システム内のさまざまな部分におけるデータの正確性をアプリケーションがどのように扱うかを定義するものです。トランザクションは個々の操作におけるデータ整合性を維持します。これに対し、一貫性モデルは、複数のサーバーにまたがる場合や、複数のユーザーが同時に同じ情報にアクセスする場合に、その整合性をどのように維持するかを規定します。

データベースにおける一貫性とは、トランザクションによりデータベースをある有効な状態から別の有効な状態へと移行させた際に、すべてのデータが定義されたルール、制約、関係に矛盾なく準拠することを保証することを指します。この原則は、データベースシステムにおける信頼性の高いトランザクション処理を保証する、4 つの ACID 特性（不可分性、一貫性、独立性、永続性）の一つです。

一貫性モデルの種類

アプリケーションによって、データへの変更がシステム全体に反映されるまでの時間に対する要件は異なります。データベースでは、こうした要件を満たすために、さまざまな一貫性モデルが用意されています。

強一貫性

システム内のどの部分も常に同じデータを参照できるようにし、変更が発生した順序通りに

反映されることを保証します。PostgreSQL では、ユーザーの口座残高を更新すると、その後のすべての読み取り操作において、その変更が即座に反映されます。このような状態を保証するため、システムは一時的に操作をブロックする場合があり、可用性よりも正確性を優先します。

結果整合性

時間の経過とともに解消される一時的な不整合を許容することで、アプリケーションの稼働を優先します。Amazon DynamoDB では、商品の価格を更新しても、その変更がグローバルに反映されるまでに数秒かかる場合があります。その結果、地域によってユーザーに表示される価格が一時的に異なることもありますが、システムはそのような場合でも利用可能なままです。このモデルは、即時の整合性が必須ではない場合に適しています。

因果整合性

因果関係のある操作間でのみ順序を保持します。例えば、Google ドキュメントのような共同編集ツールでは、ユーザー A がユーザー B の編集内容にコメントした場合、すべてのユーザーにはコメントより前の編集内容が表示されます。一方で、ユーザー C による関連性のない編集は、ユーザーによって異なる順序で表示される場合があります。これは強一貫性と結果整合性の間の中間的なアプローチとなります。

セッション一貫性

ユーザーごとのセッション内での一貫性を確保しつつ、ユーザー間で差異が生じるのを許容します。EC アプリケーションでは、商品をカートに追加すれば、そのセッション中はその商品がカートに表示されたままになります。ただし、他のユーザーは、在庫について多少古い情報を見ることになるかもしれません。こうすることで、システム全体を同期させることなく、各ユーザーに対して一貫した利用体験を提供できます。

CAP 定理とその意味

CAP 定理は、分散システムにおける根本的なトレードオフを説明するものです。データが複数のサーバーに分散されている場合、以下の 3 つの特性のうち、同時に保証できるのは 2 つだけとなります。

一貫性（Consistency）

すべてのサーバーで、同時に同じデータが表示されます。

可用性（Availability）

一部のサーバーがダウンしてもシステムは正常に動作をしつづけます。

分断耐性（Partition tolerance）

サーバー間のネットワーク接続が切断されても、システムは正常に動作しつづけます。

このようなトレードオフにより、以下のようなシステム設計が生まれます。

CP システム

CP システムは、可用性を犠牲にして、一貫性と分断耐性を優先します。ネットワークに問題が発生した場合、システムは誤ったデータを表示するリスクを避けるため、リクエストの受け入れを停止することがあります。例としては、データの正確性が極めて重要な、銀行アプリケーション、在庫管理システム、金融サービスなどが挙げられます。Apache HBase や Google Spanner といったテクノロジーは、このカテゴリに分類されます。

AP システム

AP システムは、可用性と分断耐性に重点を置き、結果整合性を許容します。AP システムは、即時の一貫性よりもシステムの稼働時間が重要視される、コンテンツ配信ネットワーク、ソーシャルメディアのフィード、レコメンデーションエンジンなどに適しています。Amazon DynamoDB や Cassandra といったテクノロジーは、このアプローチの好例です。フィード全体が利用できなくなるよりは、多少古くてもソーシャルメディアの投稿を表示するほうが望ましいのです。

CA システム

CA システムは一貫性と可用性を提供しますが、ネットワークの分断には耐えられません。CA システムは、単一ノードのデータベースや信頼性の高いネットワーク上で密結合されたクラスタにおいて、うまく機能します。MySQL や PostgreSQL のような、分散機能を設定していない従来のリレーショナルデータベース管理システム（RDBMS）は、このモードで動作します。

適切な一貫性モデルを選択する

モダンアプリケーションでは、同じシステム内でデータ型に応じて、一貫性モデルを使い分けることがよくあります。ここで重要なのは、一貫性の要件とビジネスへの影響を適切に照合することです。

正確性が極めて重要であり、一時的な利用不能が許容される操作には、**強一貫性**を使用してください。一般的に、金融取引、在庫の更新、ユーザー認証では、このアプローチを採用します。

わずかな遅延であれば許容され、可用性が極めて重要なデータには、**結果整合性**を採用してください。ユーザーアクティビティログ、レコメンデーションシステム、コンテンツフィードなどは、パフォーマンスと稼働時間が向上するのであれば、一時的な不整合は許容できます。

システム全体の同期よりも個々の整合性が重要となるユーザー向けの機能には、**セッション一貫性**を使用してください。ショッピングカート、ユーザー設定、下書きコンテンツなどは、このモデルに適しています。

適切な一貫性モデルを選択する際は、それが恒久的なものではないことを念頭に置いてください。重要なのは、現在の要件を正確に把握し、トレードオフを慎重に検討したうえで、適切なモデ

ルを選択することです。

8.2.4.2 キャッシュによるパフォーマンスの最適化

キャッシュとは、頻繁に使用されるデータを高速な保存場所に格納し、応答時間とデータベースへの負荷を軽減する技術です。これは、よく使うアプリをスマートフォンのホーム画面に置いて、毎回すべてのアプリを探し回る手間を省くようなものです。アプリケーションがデータを要求するときに、キャッシュはアプリケーションとデータベースの間に位置する、高速アクセス可能な保存レイヤーとして機能します。

データベースへのアクセスを行う場合、たとえ単純なものであっても、ネットワーク通信やディスク読み取りが伴い、数十ミリ秒から数百ミリ秒かかることがあります。キャッシュはデータをメモリに保存するため、状況やアーキテクチャによっては、応答時間を 10 倍から 100 倍以上短縮できます。

しかし、キャッシュにはトレードオフが伴います。データの鮮度に関する複雑な問題が生じ、新たな障害発生要因となるおそれがあります。そのため、キャッシュ戦略を明確に定義することが極めて重要です。適切な戦略を策定することで、アプリケーションにおいて、キャッシュされたデータとデータベース内の「信頼できる情報源」との関係をどのように管理するかが決まります。

一般的なキャッシュ戦略

モダンアプリケーションでは、主に 3 つのキャッシュ戦略が採用されており、それぞれ、パフォーマンス、一貫性、複雑性の間でさまざまなトレードオフを抱えています。

キャッシュアサイド（遅延読み込み）

キャッシュアサイド方式では、アプリケーションがキャッシュとデータベースの両方の管理を担当します。データに対するリクエストがあった場合、アプリケーションはまずキャッシュにそのデータがあるかどうかを確認します。データが存在しない場合（**キャッシュミス**）、アプリケーションはデータベースからデータを取得し、今後のリクエストに備えてキャッシュに保存します。データが存在する場合（**キャッシュヒット**）、アプリケーションはキャッシュからデータを取得します。

次の例では、ID に基づいてキャッシュからブログ記事が検索されます。該当する記事が見つかった場合、アプリケーションはその記事を返します。見つからない場合、アプリケーションがデータベースから該当するブログ記事を探し出し、次のリクエストに備えてキャッシュに保存します。

```
public BlogPost getPostById(Long id) {
    BlogPost cached = cache.get(id);
    if (cached != null) return cached;
```

```
    BlogPost post = database.findById(id);
    cache.put(id, post);
    return post;
}
```

この戦略は、キャッシュの対象を細かく制御したい、読み取りが中心のアプリケーションにおいて効果的です。その代償として、アプリケーションのコードは複雑になります。また、どのデータに対しても、最初のリクエストはキャッシュの検索やヒットミスが発生するため、常に処理に時間がかかります。

ライトスルー

ライトスルーキャッシュでは、書き込み操作が行われるたびに、キャッシュとデータベースの両方が同時に更新されます。これにより、データベースとキャッシュの両方に同じデータが保持されることが保証されます。ただし、両方の場所に書き込みが行われるため、パフォーマンスが低下するおそれがあります。以下の例では、データベースとキャッシュが同じメソッド内で更新されています。

```
public void updatePost(BlogPost post) {
    database.save(post);
    cache.put(post.getId(), post);
}
```

この戦略は、キャッシュとデータベースの間で強一貫性が必要であり、読み取りパフォーマンスが書き込みパフォーマンスよりも重要である場合に有効です。その代償として、書き込みパフォーマンスが低下するほか、あまり読み取られないデータがキャッシュされるおそれがあります。

ライトビハインド（ライトバック）

ライトビハインドキャッシュでは、データは直ちにキャッシュに書き込まれますが、データベースへの更新は非同期で行われます。これにより、書き込み操作のパフォーマンスは向上しますが、データベースが更新される前にキャッシュに障害が発生した場合、データが失われるリスクが生じます。以下の例では、まずキャッシュが更新され、その後データベースを更新するためのバックグラウンドプロセスが起動されます。

```
public void updatePost(BlogPost post) {
    cache.put(post.getId(), post);
    asyncQueue.schedule(() -> database.save(post));
}
```

この戦略は、パフォーマンスの向上と引き換えに、ある程度のデータ損失のリスクを許容で

きる、書き込み量の多いアプリケーションに適しています。ライトビハインド戦略を採用する例としては、一部のデータポイントが失われても許容できるユーザアクティビティ、ロギング、メトリクスなどが挙げられます。

キャッシュをいつ使用すべきか

キャッシュは強力なツールですが、それにはトレードオフが伴います。キャッシュはシステム内の可動部品のようなものであり、障害が発生したり、情報が古くなったり、リソースを消費したりするおそれがあります。重要なのは、キャッシュが良いか悪いかということではなく、アプリケーションにキャッシュを導入することによるメリットが、それに伴う複雑さの増加に見合うかどうかということです。

オンラインストアの商品カタログを例に考えてみましょう。ユーザーがストアにアクセスして商品を検索すると、アプリケーションは商品詳細、在庫、レビュー、価格といった複数のテーブルから情報を取得する必要があります。各商品ページの処理に平均 200 ミリ秒かかり、1,000 人の買い物客が同時にストアを利用している場合、アプリケーションは短時間で 4,000 件ものデータベースへのリクエストを送信することになります。データベースは、許容範囲内の応答時間を維持しつつ、この大量の同時クエリを処理しなければなりません。

こうしたリクエストのほとんどは、めったに変化しない情報に関するものです。このような場合には、キャッシュを活用するのが理にかなっています。データベースが注文処理のような本質的に動的な操作を処理していたとしても、メモリから平均 5 ミリ秒でリクエストに応答できるからです。

これに対し、価格が 1 秒間に何度も変動するようなリアルタイム取引プラットフォームのような場合を考えてみましょう。たとえキャッシュの削除ポリシーを 30 秒間隔に設定したとしても、古い価格が表示されてしまうおそれがあり、その結果ユーザーに金銭的な損失をもたらしてしまうかもしれません。この場合、キャッシュの無効化の複雑さと、古いデータが表示されるリスクが、パフォーマンス上のメリットをはるかに上回ってしまいます。

特定の状況においてキャッシュを使うのが適切かどうかを評価する際は、以下の点を検討してみてください。

- このデータはどのくらいの頻度で更新されますか？
- ユーザーに古いデータを提供した場合、どのようなコストが発生しますか？
- 現在のクエリの実行コストはどの程度ですか？
- 現在、パフォーマンスのボトルネックはありますか？

これらの質問への回答が、読者の具体的な利用シーンに適したソリューションを見つける手助けとなります。

キャッシュと一貫性

キャッシュは、一貫性に関して特有の課題をもたらします。一貫性については、本節ですでに学びました。データをキャッシュする際、一時的なコピーが作成されますが、元のデータが変更されると、そのコピーは古くなってしまうおそれがあります。キャッシュ戦略によって、この問題への対処方法は異なります。

- **TTL（有効期間）付きのキャッシュアサイド**は、設定された時間が経過するとキャッシュされたデータを自動的に削除することで、結果整合性を確保します。
- **ライトスルーキャッシュ**は、キャッシュとデータベースの間に強一貫性を維持しますが、その代償として書き込みパフォーマンスが低下します。
- **ライトビハインドキャッシュ**は、パフォーマンスを優先するため、一時的な不整合を許容します。

重要なのは、ビジネス要件とトレードオフを慎重に検討することです。金融業界では、強一貫性を確保するためにライトスルーキャッシュが必要となる場合があります。一方で、個人のブログでは、結果整合性でも問題がないので、TTL 付きキャッシュアサイドを採用することも可能です。

分散システムに関する考慮事項

アプリケーションが複数のサーバーにまたがってスケールアウトする場合、次のような課題が生じます。

- キャッシュの一貫性がより複雑になります。
- 分散トランザクションが必要になる場合があります。
- 結果整合性のパターンを実装する必要があるかもしれません。
- データのレプリケーションやシャーディング戦略が重要になります。

これらのトピックについては、それぞれ個別の章を設けて説明する価値があるようなものです。アプリケーションが拡大するにつれて、データ永続化の戦略も進化させる必要があることに留意してください。

8.2.5 データ量の増加への備え

本章を通じて学んできたように、データに関して「完璧な」計画というものはありませんが、まったく計画なしに始めることは避けるべきです。事前にデータ戦略についてしっかりと検討しておくことで、後々の労力や時間、ストレスを大幅に軽減できます。

データ量の増加を見据えた計画を立案するには、パフォーマンスメトリクスやリソース要件を定

期的に見直すことが不可欠です。これは、開発チーム、データベース管理者、運用担当者が協力して行うのが最も効果的です。クエリの応答時間が許容範囲を超えている、データベースの CPU 使用率が常に一定割合を上回っている、メモリ使用量が着実に増加している、読み込み速度の遅さに関するユーザーからの苦情が増えているといった、重要な指標に注意を払ってください。データ量が増加しているアプリケーションについては、こうした検討を毎月行うことで、問題が深刻化する前に対処できます。

8.2.5.1 スケーリング戦略の活用

データ量の増加のきざしが見られた場合、状況に応じて以下のような選択肢があります。

垂直スケーリング

データベースサーバーにリソース（CPU、RAM）を追加します。これは簡単でアーキテクチャの変更も不要なため、多くの場合、最初に講じられる対策となります。ただし、いずれハードウェアの限界に直面することになります。

水平スケーリング

データを複数のサーバーに分散させます。これは、垂直スケーリングの限界に達した場合や、グローバルなアプリケーション向けに地理的な分散が必要な場合に有効です。

リードレプリカ

読み取り処理用にデータベースのコピーを作成することで、プライマリデータベースへの負荷を軽減できます。読み取りが中心となるアプリケーションでは、まずこのアプローチを検討してください。これは完全な水平スケーリングよりも実装が容易であり、パフォーマンスを大幅に向上させることができます。

シャーディング

シャードキーに基づいて、データを複数のデータベースに分割します。この方法は、最も複雑な選択肢ですが、単一のサーバーでは収まりきらないような膨大なデータ量を持つアプリケーションには不可欠です。

リードレプリカとキャッシュの比較

リードレプリカとキャッシュは、それぞれ読み取りパフォーマンスを向上させるものですが、その仕組みは異なります。

- **リードレプリカ**は、データベースの完全なコピーであり、レプリケーションを通じてプライマリデータベースと同期が保たれます。
- **キャッシュ**は、頻繁にアクセスされるデータを一時的なコピーとしてメモリに保存する

もので、そのコピーは古くなるおそれがあります。

リードレプリカはデータベースレベルで結果整合性を提供し、アプリケーションのコードを変更する必要はありません。一方で、キャッシュを使用する場合は、アプリケーション側でキャッシュの無効化と一貫性の管理を行う必要があります。リードレプリカは、地理的な分散や読み取り中心のワークロードにおいて特に有用です。一方、キャッシュは、頻繁にアクセスされるデータの応答時間を短縮するのに効果的です。

8.2.5.2　データ量増加に対するパフォーマンスの維持

アプリケーションの規模が拡大するにつれ、定期的なデータメンテナンスの重要性はますます高まります。パフォーマンスを維持するには、以下の方法があります。

- 古いデータに対してデータアーカイブを行い、作業対象データの量を管理可能な範囲に保ちます。
- データベースのインデックスを適切に設定します。クエリの傾向の変化に合わせて定期的に見直してください。
- クエリパフォーマンスの分析や、処理が遅いクエリの最適化など、定期的にデータベースの保守を実施してください。

さまざまなデータベースの種類や永続化の抽象度のレベルを理解することで、アプリケーションのアーキテクチャや方向性について、十分な情報に基づいて判断を下すことができます。どのような選択が適切なのかは、アプリケーションの要件やチームのスキルによって異なり、万能な最善の解決策というものは存在しません。

8.3　データクエリと管理の最適化

アプリケーションの動作が遅いと、見た目がどれほど優れていても、ユーザーを苛立たせてしまいます。バックエンドチームはユーザーへのデータ提供に注力するあまり、パフォーマンスを軽視しがちです。アプリケーションが動作さえすれば、チームは満足してしまうのです。小規模なWebアプリケーションであれ、複雑なエンタープライズシステムであれ、データのクエリや管理の方法は、アプリケーションのパフォーマンスを左右する要因となります。

この節では、データアクセスを効率的にする基本原則について解説します。これらの基本原則は、将来的に発生するかもしれないパフォーマンスの問題を回避するのに役立ちます。非常に一般的な原則であるため、多くのアプリケーションで目にすることでしょう。クエリを最適化するための実践的な手法を学び、データベースのインデックスについて理解を深め、パフォーマンスツールをいつ使用すべきかを学びます。これらのスキルは、アプリケーションのパフォーマンスを向上さ

せるだけでなく、データベースの基礎を深く理解していることの証にもなります。

8.3.1 効率的なクエリを作成する

開発の初期段階では、まず目的の結果が得られるクエリを作成したいと思うものです。ただし、その次のステップとして、「このクエリのパフォーマンスをどのように改善できるか」と自問すべきです。効率的なクエリの書き方を理解することは、アプリケーションのパフォーマンスに多大な影響を与える基本的なスキルです。最適化が不十分なクエリは、サーバーリソースを過剰に消費し、ボトルネックを引き起こし、ユーザー体験を著しく悪化させる原因となります。

最適化がうまくいっているクエリとそうでないクエリとの間には、大きな差が生じます。実行時間が数秒から数ミリ秒にまで短縮されることもあります。データアクセスを最適化し、高スループット下でも応答性を維持できる、そのようなアプリケーションを構築するための、いくつかの重要なテクニックについて見ていきましょう。

8.3.1.1 基本的なクエリの最適化

効率的なクエリを作成するための第一歩は、そもそも何がクエリを非効率にしているのかを理解することです。以下のようなよくあるケースを考えてみましょう。

```
public List<User> getAllActiveUsers() {
    return jdbcTemplate.query(
        "SELECT * FROM users WHERE active = true",
        (rs, rowNum) -> {
            User user = new User();
            user.setId(rs.getLong("id"));
            user.setUsername(rs.getString("username"));
            user.setEmail(rs.getString("email"));
            user.setActive(rs.getBoolean("active"));
            user.setCreatedAt(rs.getTimestamp("created_at"));
            // ... さらに 15 のフィールドのマッピングを行う
            return user;
        });
}
```

このクエリは一見問題なさそうですが、2 つの重大な問題があります。まず、SELECT *を使用すると、必要な列が 1 つか 2 つであっても、データベースからすべての列が取得されてしまいます。次に、一度に数千件ものレコードがメモリに読み込まれてしまうおそれがあります。改善してみましょう。

```
public List<UserSummary> getActiveUserSummaries() {
    return jdbcTemplate.query(
        "SELECT id, username, email FROM users WHERE active = true LIMIT 100",
        (rs, rowNum) -> {
            UserSummary summary = new UserSummary();
```

```
            summary.setId(rs.getLong("id"));
            summary.setUsername(rs.getString("username"));
            summary.setEmail(rs.getString("email"));
            return summary;
        });
}
```

必要な列のみを選択し、結果セットを絞り込むことで、データベースからアプリケーションへ転送されるデータ量を大幅に削減できました。この単純な変更により、特に大規模なテーブルを扱う場合、パフォーマンスが桁違いに向上します。



選択する列については常に具体的に指定し、大量のデータを取得する際は、ページネーション（1 ページあたり 20 件など、結果を小さなブロックに分割すること）の使用を検討してください。パフォーマンスが向上するだけでなく、アプリケーションが必要とするデータを明確に記述できるため、コードの保守性も高まります。

8.3.1.2　プリペアドステートメント

基本的なクエリの最適化を終えたら、次はデータベースがそのクエリを可能な限り効率的に実行できるようにする必要があります。**プリペアドステートメント**とは、事前にコンパイルされた SQL クエリのことです。実行時に安全にデータ値を挿入できるだけでなく、パフォーマンスを向上させ、SQL インジェクションを防ぐことができます。次の例では、値を SQL に直接連結することでステートメントが構築されています。

```
public User findUserByEmailAndStatus(String email, boolean active) {
    String sql = "SELECT id, username, email FROM users WHERE email = '" +
                 email + "' AND active = " + active;
    return jdbcTemplate.queryForObject(sql, User.class);
}
```

このメソッドが実行されるたびに、データベースはまったく新しいクエリを解析、コンパイル、最適化しなければなりません。それでは、プリペアドステートメントを使ってみた場合と比較してみましょう。

```
public User findUserByEmailAndStatus(String email, boolean active) {
    return jdbcTemplate.queryForObject(
        "SELECT id, username, email FROM users WHERE email = ? AND active = ?",
        new Object[]{email, active},
        User.class);
}
```

プリペアドステートメントを使用すると、データベースはクエリを解析して最適化します。その

後、異なるパラメータで呼び出される際にもその実行プランを再利用します。このクエリプランのキャッシュにより、頻繁に実行されるクエリの実行時間を 20%〜50% 短縮できます。

パフォーマンス面だけでなく、プリペアドステートメントはパラメータのエスケープ処理を自動的に行うため、手作業での入力サニタイズを行う必要がなく、アプリケーションを SQL インジェクション攻撃から保護します。

8.3.1.3 インデックス管理

索引がない状態で、本書全体から「キャッシュ」に関連する記述を探すことを想像してみてください。すべてのページを読み、その都度メモを取らなければならないでしょう。データベースのインデックスは、検索テーブルを作成して、値を保存する場所に直接リンクすることで、この問題を解決し、処理に時間がかかる全テーブルスキャンを高速な検索に変えます。以下の例では、クエリにインデックスが設定されていないため、実行に約 2,000 ミリ秒必要です。

```
// インデックスを使用しない場合のクエリ実行時間： 約 2000 ミリ秒
UserProfile profile = jdbcTemplate.queryForObject(
    "SELECT * FROM user_profiles WHERE email = ?",
    new Object[]{"john.doe@example.com"},
    UserProfile.class);
```

email 列にインデックスがない場合、データベースはテーブル全体のスキャンを実行します。それでは、インデックスを追加してみましょう。

```
CREATE INDEX idx_user_profiles_email ON user_profiles(email);
```

このインデックスを使用することで、同じクエリの実行時間が 2,000 ミリ秒から 5 ミリ秒に短縮され、400 倍の高速化が実現します！ ただし、インデックスにはコストがかかります。ストレージ容量を消費するほか、データが変更されるたびに、データベースが各インデックスを更新する必要があるため、書き込み処理の速度が低下してしまいます。

インデックス化する対象を決める際は、以下のガイドラインを参考にしてください。

- WHERE 句で頻繁に使用される列をインデックス化してください。
- JOIN 条件で使用される列をインデックス化してください。
- ORDER BY 句や GROUP BY 句で使用される列をインデックス化してください。
- 複数の列でフィルタリングするクエリでは、複合インデックスの使用を検討してください。
- 一意の値が少ない（カーディナリティが低い）列にはインデックスを設定しないでください。

8.3.1.4 大規模な結果セットの処理

開発業務において、見落としがちな問題の一つは、本番環境に存在する膨大なデータ量です。大

規模なデータセットを扱っていない場合、本番環境で大量のオブジェクトが返されて、パフォーマンスの問題が発生し得ることを忘れがちです。前述のとおり、この問題に対処する方法の一つは、クエリで返すレコード数を制限することです。ここで役立つのがページネーションです。これにより処理しやすい単位で結果を操作できるようになります。

次の例では、Java メソッドがカテゴリでフィルタリングされた商品データをページ単位で取得します。このメソッドは 2 つのデータベースクエリを実行します。1 つは特定のページの商品データを取得するためのものであり、もう 1 つは総件数を取得するためのものです。その後、このメソッドはこれらの結果を組み合わせて、クライアントアプリケーションに必要となる、商品データとページネーションのメタデータの両方を含むページオブジェクトを作成します。

```
public Page<Product> getProductsByCategory(String category, int page, int size) {
    int offset = page * size;

    List<Product> products = jdbcTemplate.query(
        "SELECT id, name, price FROM products WHERE category = ? " +
        "ORDER BY name LIMIT ? OFFSET ?",
        new Object[]{category, size, offset},
        (rs, rowNum) -> {
            Product product = new Product();
            product.setId(rs.getLong("id"));
            product.setName(rs.getString("name"));
            product.setPrice(rs.getBigDecimal("price"));
            return product;
        });

    int totalCount = jdbcTemplate.queryForObject(
        "SELECT COUNT(*) FROM products WHERE category = ?",
        new Object[]{category},
        Integer.class);

    return new Page<>(products, page, size, totalCount);
}
```

このアプローチには、パフォーマンス上の問題が生じるおそれがあります。テーブルが大きくなるにつれて、一致するすべての行をカウントする、2 つ目のクエリの処理負荷が高くなる可能性があるからです。このコードではオフセットベースのページネーションを使用しています。この方式では、データベースがオフセットポイントより前のすべての行をスキャンして破棄する必要があるため、オフセットが大きくなるほど非効率になっていきます。

以下に、検討すべきいくつかの選択肢を紹介します。

- オフセットによるページネーションを、キーセットによるページネーション（`WHERE id > last_seen_id` を使用する）に置き換えてください。
- 頻繁に変化しないのであれば、総件数の結果をキャッシュしてください。
- 非常に大規模なデータセットの場合は、総件数を推定するようにしてください。

- 総件数を表示する必要のない、無限スクロールのような UI パターンを検討してください。

8.3.2 ツールとベストプラクティス

「森の中で木が倒れても、それを聞く人が誰もいなければ、音はするのでしょうか？」という古くからある思考実験をご存じでしょうか。さて、読者が自宅で何事にも気づかないうちに、ユーザーがその森の中にいて、データベース上で木が倒れる音を聞いていると想像してみてください。問題は必ず発生するものですが、重要なのは、そうした問題をどのように把握するかということです。

8.3.2.1 クエリの実行プランを理解する

データベースクエリプランナー（**オプティマイザ**とも呼ばれます）は、あらゆるデータベース管理システムにおいて不可欠なコンポーネントであり、SQL 文を**実行プラン（実行計画）**と呼ばれる実行可能プログラムに変換する役割を担っています。SQL コードを受け取り、要求されたデータを取得するために最も効率的な方法を決定するコンパイラのようなものだと考えてください。

データベースクエリプランナーとは？

クエリプランナーは、SQL クエリを分析し、利用可能なインデックス、テーブルの統計情報、さまざまな実行戦略を考慮して、最適な実行プランを生成します。クエリプランナーには主に 2 つの種類があります。ルールベースのオプティマイザは厳格なルール（利用可能なインデックスを常に使用するなど）に従います。一方、コストベースのオプティマイザは複数の実行プランを生成し、推定コストが最も低いものを選択します。モダンデータベースシステムのほとんどは、コストベースのオプティマイザを採用しています。

データベース固有のクエリプランナー

各データベースシステムには、それぞれ独自の特徴を持つクエリプランナーが備わっています。

PostgreSQL

PostgreSQL のプランナー／オプティマイザは、クエリ内の各リレーション（テーブル）をスキャンするためのプランを生成し、使用するインデックスを決定し、さまざまな結合順序を検討して、最もコストの低いものを見つけることで、実行プランを作成します。結合が多数含まれる複雑なクエリの場合、PostgreSQL は遺伝的クエリオプティマイザを使用して、妥当な時間内に妥当な（必ずしも最適とは限りません）プランを見つけ出します。

MySQL

MySQL のクエリオプティマイザもコストベースのアプローチを採用しており、特に得意とする読み取り操作の最適化に重点を置いています。さまざまなアクセス方法、結合の種類、結合順序を評価し、最も効率的な実行プランを見出します。

Oracle

Oracleのオプティマイザは非常に高度な機能を備えており、エンタープライズレベルでの最適化機能を提供しています。実行時の統計情報に基づいて、クエリの実行中に変更可能な適応型実行プランなどが含まれます。

クエリの実行プランの利用

ほとんどのデータベースでは、クエリの実行プランを確認するために、EXPLAINコマンドが用意されています。その使い方は以下の通りです。

```
 -- PostgreSQLにおける基本的なEXPLAIN
EXPLAIN SELECT u.username, p.bio
FROM users u JOIN profiles p ON u.id = p.user_id
WHERE u.active = true;

-- EXPLAIN ANALYZEを使用したより詳細な分析（PostgreSQL）
EXPLAIN ANALYZE SELECT u.username, p.bio
FROM users u JOIN profiles p ON u.id = p.user_id
WHERE u.active = true;

-- MySQLのEXPLAIN形式
EXPLAIN SELECT u.username, p.bio
FROM users u JOIN profiles p ON u.id = p.user_id
WHERE u.active = true;
```

EXPLAINは、PostgreSQL、MySQL、Oracleなど主要なデータベースシステムでサポートされていますが、構文や出力形式はシステムによって異なる場合があります。また、PostgreSQLとMySQLでは、EXPLAIN ANALYZEも利用できます。これを使用するとクエリを実行し、オプティマイザの予測と実際の実行結果がどのように一致したかなどの追加情報を得ることができます。

実行プランの出力には、通常、以下のような内容が含まれます。

- どのテーブルがどのような順序でアクセスされるか
- どのような結合方式が使用されるか（ネストループ結合、マージ結合、ハッシュ結合）
- 各テーブルに対してどのインデックス（もしあれば）が使用されるか
- どのような種類のスキャンが実行されるか（シーケンシャルスキャン、インデックススキャン）
- 時間およびリソースの観点からの推定コストおよび／または実際のコスト
- 行数（ANALYZE使用時の、推定値と実際の値）

実行プランを読み解くには練習が必要ですが、このスキルはパフォーマンスの低いクエリをデバッグするうえで重要です。実行プランを分析する際は、次のような警告サインに注意してください。

- インデックスを使用すべき場面でのテーブル全体のスキャン
- データ量に対して非効率的な結合方法
- インデックスの欠落または未使用
- 推定コストまたは行数が大きい
- 推定値と実際の値の間に大きな乖離がある（ANALYZE を使用する場合）

EXPLAIN を定期的に使用することで、データベースがどのように動作し、判断を下しているかを理解できるようになります。その結果、より効率的なクエリを作成したり、より適切なスキーマやインデックスを設計したりすることが可能になります。



個々のクエリの最適化に加え、環境間（開発環境、ステージング環境、本番環境）で実行プランを比較することで、デプロイに関する重大な問題を発見できる場合があります。インデックスの欠落、統計情報の古さ、データ分布の違いといった環境間に不整合があると、テスト環境では良好に動作していたクエリが、本番環境では失敗する原因となることがあります。デプロイ後にパフォーマンスが急激に低下した場合、環境間で実行プランを比較してください。他のデバッグ手法よりも迅速に根本原因を特定できることがよくあります。

8.3.2.2 データベースの監視と分析

EXPLAIN を使用して、理想的な条件下でのクエリのパフォーマンスを把握することは重要です。しかし、実際のパフォーマンスは、負荷がかかった状況下でアプリケーションがデータベースとどのようにやり取りするかによって左右されます。スロークエリログや統計テーブル（pg_stat_statements）といったデータベース固有のツールだけに頼っても、全体像の一部しか把握できません。

全体像を把握するには、アプリケーションの視点から監視しなければなりません。そこで役立つのがアプリケーションレベルのオブザーバビリティです。データベースとのやり取りがアプリケーション全体の健全性やパフォーマンスにどのような影響を与えるかについての情報を提供します。ソフトウェアエンジニアリングにおいて、オブザーバビリティとは、システムが出力する結果に基づいて、その内部状態をどの程度理解できるかを指します。

モダンアプリケーションフレームワークには、多くの場合、メトリクス、ロギング、トレースを統合した、強力なオブザーバビリティツールが備わっています。Spring Boot では、Actuator モジュールと Micrometer メトリクスライブラリによって、このようなオブザーバビリティを最小限の設定で利用でき、データベースとのやり取りを詳細に把握できます。

オブザーバビリティについて議論する際には、以下のような概念を理解しておくことが重要です。

ロギング

イベントやエラーを記録し、メトリクスやトレースと関連付けることで、問題を診断します。

メトリクス

システムパフォーマンスに関する定量的なデータを収集します。データベースの場合、接続プールの使用状況（アクティブ、アイドル、待機中の接続）、クエリの実行時間、トランザクション時間、エラー率などが含まれます。

トレース

リクエストがアプリケーションのさまざまな部分、さらには分散システム全体をどのように伝播していくかを追跡します。データベースへの呼び出しにかかる時間を含め、遅延が発生している箇所を特定するのに役立ちます。

以下は、オブザーバビリティが実用的な知見をもたらす事例です。

接続プールはボトルネックになっていないか？

`jdbc.connections.pending` メトリクスの値が高い場合は、スレッドが接続を待ちすぎていることを示しています。`jdbc.connections.active` が常に `jdbc.connections.max` の上限に達している場合は、接続プールが小さすぎるか、接続が適切に解放されていない（メモリリークの可能性や長時間実行されるトランザクションなど）ことが考えられます。

データベースリソースを無駄遣いしていないか？

`jdbc.connections.active` の値が常に低く、`jdbc.connections.idle` の値が高い場合は、接続プールのサイズが大きすぎる可能性があり、不要なメモリやデータベースリソースを浪費しているおそれがあります。

なぜ特定のユーザーのリクエストが遅いのか？

分散トレーシングを利用すれば、リクエストのライフサイクル全体を把握できます。もしトレースの結果、リクエスト時間の 90% が単一のデータベーススパン（クエリやトランザクションを表します）で消費されていることが判明した場合、最適化の取り組みをどこに集中させるべきかがはっきりします（おそらく、その特定のクエリに対して `EXPLAIN` を実行する必要があるでしょう）。

「N+1 クエリ問題」は発生していないか？

トレースを見ると、1 つの論理操作の中で、数十件もの高速で小規模なデータベースクエリが連続して実行されている場合があります。この典型的な「N+1 問題」は、ORM のマッピング設定の不備に起因するものですが、トレースを見ればすぐにわかります。

特定のクエリの実行速度は、時間の経過とともに低下していないか？

クエリの実行時間に関するメトリクスを監視し（有効になっている場合）、アプリケーションのデプロイやデータ量の増加と照らし合わせることで、重大なインシデントを引き起こす前に、最適化が必要なクエリを事前に特定できます。

データベースのエラーはユーザーにどう影響するのか？

データベースのエラーログやメトリクスの急上昇を、アプリケーションレベルのエラー率やリクエストの失敗トレースと照らし合わせることで、データベースの問題（制約違反、デッドロック、接続障害など）がユーザーに与える影響を把握できます。

Spring Boot Actuator は、監視用に本番環境対応エンドポイントを提供し、Micrometer はベンダーに依存しないアプリケーションメトリクスのファサードを提供します。これらを併用することで、`DataSource` ビーン（Spring Boot のデフォルトである HikariCP など）を自動的に計測し、前述した重要な統計情報の多くを収集できます。これにより、これらの知見を得るために必要な生データを得ることができます。

8.3.2.3 複雑さとパフォーマンスのバランス

ソフトウェアエンジニアとして、コードにおけるトレードオフについて、十分な情報に基づいた判断を下すことは非常に重要なスキルです。データベースを扱う際にはこのスキルがよく必要とされます。コードの可読性や保守性と、クエリのパフォーマンスとのバランスをとることが頻繁に求められます。ここでは、注文処理という実際のシナリオを例に、このバランス調整について考えてみましょう。

次の例では、注文データを取得するための 2 つのアプローチについて見ていきます。1 つ目のアプローチでは、Spring Data JPA のメソッドチェーンとストリーム処理を使用します。これは簡潔で理解しやすいのですが、大規模なシステムではパフォーマンスが低下するおそれがあります。2 つ目のアプローチでは、生の SQL を使用します。これはより効率的ですが、保守にはより細心の注意が必要です。

```
// シンプルではあるが、非効率となるおそれのあるアプローチ
List<Order> recentOrders =
orderRepository.findByUserIdAndStatusOrderByCreatedAtDesc(
    userId, OrderStatus.COMPLETED);

List<OrderDto> result = recentOrders.stream()
    .filter(order -> order.getTotal().compareTo(new BigDecimal("100.00")) > 0)
    .map(orderMapper::toDto)
    .collect(Collectors.toList());
```

```
// より効率的だが、複雑なアプローチ
List<OrderDto> efficientResult = jdbcTemplate.query(
    """
    SELECT o.id, o.created_at, o.total, u.name AS user_name
    FROM orders o JOIN users u ON o.user_id = u.id
    WHERE o.user_id = ? AND o.status = ? AND o.total > 100.00
    ORDER BY o.created_at DESC
    """,
    new Object[]{userId, OrderStatus.COMPLETED.name()},
```

```
    (rs, rowNum) -> {
        OrderDto dto = new OrderDto();
        dto.setId(rs.getLong("id"));
        dto.setCreatedAt(rs.getTimestamp("created_at"));
        dto.setTotal(rs.getBigDecimal("total"));
        dto.setUserName(rs.getString("user_name"));
        return dto;
    });
```

2つ目のアプローチは、フィルタリングをデータベース側で行い、必要なデータのみを取得するため、より効率的です。しかし、その分複雑であり、データベースの構造との依存関係も強くなります。

アプローチを決める際には、以下の点を考慮してください。

- データの規模
- パフォーマンス要件
- 保守の負担
- チームのSQLの習熟度
- 将来的な柔軟性の要件

多くの場合、ハイブリッドなアプローチが最も効果的です。つまり、ほとんどの操作にはORMを使用し、パフォーマンスが重要な部分では生のSQLに切り替えるのです。

効率的なデータアクセスとは、単にクエリを高速化することだけではありません。アプリケーションにおけるデータライフサイクル全体を理解することが重要です。必要なデータのみを選択し、適切なインデックスを活用し、結果セットが大きい場合にはページネーションを使用し、パフォーマンスを監視することで、データが増えても応答性を維持できるアプリケーションを構築できます。

8.4　データのマイグレーションと変換

データのマイグレーションは、すべてのソフトウェアエンジニアがいずれ直面する一般的な課題です。開発チームは、データベースの種類を変更する（リレーショナルデータベースからNoSQLデータベースへのマイグレーション、あるいはその逆など）、外部サービスと連携する、複数のソースからのデータを統合するなど、定期的にデータベースシステムのアップグレードを行う必要があります。こうした作業は、キャリアを通じて頻繁に発生します。データのマイグレーションと変換を適切に処理する方法を理解することは、基本となるスキルです。

この節では、システム間のデータ移行やスキーマ変更への対応、プロセス全体を通じたデータ整合性の確保など、データマイグレーションと変換の基礎について学びます。これらのスキルを身につけることで、自信を持って課題に取り組むことができ、データの損失や破損につながるようなよ

くあるミスを回避できるようになります。

8.4.1 データ移動の基礎を理解する

ソフトウェアエンジニアにとって、問題解決能力は非常に重要なスキルです。一見すると、あるシステムから別のシステムへデータを移行するという問題は、ごく単純なものに思えるかもしれません。あるシステムからデータをコピーし、別のシステムに貼り付けるだけで済む、というものでしょう？ しかし残念ながら、実際にはそう単純なものではありません。データ移行を成功させるには、綿密な計画、実行、そして検証が必要です。

幸いなことに、あるシステムから別のシステムへデータ移行する際には、テンプレートとして活用できる移行戦略がいくつか存在します。他のあらゆる意思決定と同様に、どの戦略を選択するかは、データ量、システムの複雑さ、確保可能なダウンタイムの期間、許容できるリスクの度合い、ビジネス要件といった要因によって決まります。これらの要素を理解しておけば、読者自身の具体的なマイグレーションシナリオに適したアプローチを選択できます。

8.4.1.1 ビッグバンマイグレーションとフェーズドマイグレーション

ビッグバンマイグレーションは、通常はダウンタイムの期間を利用して、すべてのデータを一度に移行します。この方法は、比較的簡単なアプローチですが、多くのリスクも伴います。あるデータベースから別のデータベースへ従業員データを移行する場合を考えてみましょう。ビッグバン方式では、データベース全体の従業員データをすべて新しいデータベースに移行することになります。

対照的に、**フェーズドマイグレーション**では、データを段階的に移行します。これもおそらくダウンタイムの期間に実施することになります。ただし、データの一部のみを移行し、検証を行ったうえで、次のデータの移行へと進めていきます。先ほどの従業員の例で言えば、このフェーズでは人事部門の従業員のみを移行するといった形になります。ここで重要なのが綿密に計画を立てることです。計画によって、このフェーズドマイグレーションをどのように分割するかが決まるからです。万が一問題が発生した場合でも、影響が最も少ないと思われるグループから移行を開始できます。

データマイグレーションプロジェクトの経験がない場合は、フェーズドマイグレーションから始めるのが安全かもしれません。時間はかかるかもしれませんが、リスクを大幅に軽減できるうえ、より重要なデータを移行する前に、ノウハウを身につける機会を得ることができます。



これらのデータセットをコードで処理する場合は、システムのメモリを圧迫しないよう、必ずバッチ処理を行ってください。1,000 件から 10,000 件程度のバッチが一般的な目安となりますが、システムの処理能力を確認するため、まずは概念実証（PoC、Proof-of-Concept）を実施することを推奨します。

8.4.1.2　ETL プロセス

ETL（抽出《Extract》、変換《Transform》、ロード《Load》）とは、システム間でデータを移動させるプロセスです。

抽出

ソースシステムからデータを取得する

変換

データを、対象システムの形式や要件に合わせて変換する

ロード

変換されたデータを対象システムに挿入する

この 3 段階のアプローチにより、各プロセスの処理範囲が明確に定義されます。より大きなプロセスの一部として処理が行われますが、これらの各段階にはそれぞれ固有の課題が存在します。抽出の段階では、本番システムにおいて処理速度の制限やパフォーマンスへの悪影響が発生するおそれがあります。変換の段階では、小規模な PoC では明らかにならなかったデータ品質の問題がしばしば発生します。ロードの段階では、対象システムにおいてリソースの制約や検証エラーが発生するおそれがあります。

ETL プロセスを実装する際には、問題を捕捉・特定するために、詳細なエラー処理とロギングに時間を割く必要があります。プロセスを迅速に改善していくためには、レコードの移行がいつ、どのような理由で失敗したのかを知っておくことが重要です。

8.4.1.3　データの同期

場合によっては、移行期間中に複数のシステムを同期しなければなりません。フェーズドマイグレーションアプローチを採用した従業員データベースの例に戻ってみましょう。この手法ではリスクを軽減できますが、別の課題も生じます。それは、新しい従業員が入社した際、フェーズドマイグレーションが完了するまでは、その従業員が両方のデータベースに存在しつづけなければならないということです。

同期が必要な場合、以下の方法を検討してください。

- 更新を非同期で処理するためのメッセージキュー
- データベースの変更を追跡するための変更データキャプチャ（CDC、Change Data Capture）
- 不整合を特定・修正するためのリコンシリエーション（照合）プロセス

本節では、システム間でデータを移行するためのデータマイグレーションおよび変換の戦略について学びました。次節では、基盤となるデータベーススキーマに変更を加える方法について学び

ます。

8.4.2 スキーマの変更への対応

プロジェクトが時間とともに進化するにつれ、基盤となるデータベーススキーマの変更は避けられません。テーブル構造が変更されたり、フィールドが追加・削除・名前変更されたり、型が変更されたりすることがあります。データの整合性を維持しながら、これらの変更を管理することは、信頼性の高いアプリケーションを維持するために重要です。本節では、データベーススキーマのバージョン管理、Flyway のようなマイグレーションツール、スキーマ変更時のデータ変換手法、そしてシステムへの影響を最小限に抑えながらシームレスに更新を行うためのベストプラクティスについて学びます。

8.4.2.1 データ構造におけるバージョン管理の利用

目を閉じて、バージョン管理なしでコードベース全体が保存されている様子を想像してみてください。少し考えただけで、背筋が凍るような思いがするはずです。変更履歴を追跡したり、効率的な共同作業を行ったりせずにソフトウェアを開発することはあり得ないのと同じように、データベーススキーマも同様の保護が必要です。

バージョン管理を行わないと、重要な履歴が失われたり、同僚の作業を上書きしてしまったり、問題のある変更を元に戻せなくなったりするリスクがあります。データインフラストラクチャの基盤となるスキーマについても、アプリケーションのコードを保護するのと同じくらい慎重なバージョン管理が必要です。

Flyway、Liquibase、Rails Migrations などのツールは、データベーススキーマのバージョン管理のためのフレームワークを提供します。これらのツールは、各環境にどのスキーマ変更が適用されたかを追跡し、開発環境、テスト環境、本番環境全体での一貫性を確保します。

Flyway を使った具体的な例を見てみましょう。Flyway は、データベースのマイグレーションに対してシンプルなアプローチを採用しています。Flyway の基本的な仕組みは以下の通りです。

- データベース内に特別なテーブル（通常は `flyway_schema_history` と名前がつけられます）を保持し、どのようなマイグレーションが適用されたかを追跡する
- 指定されたフォルダ内をスキャンし、特定の命名規則に従ったマイグレーションスクリプトを探す
- 利用可能なスクリプトと、履歴テーブルに記録されている内容を照合する
- まだ適用されていない新しいスクリプトを実行する

例えば、マイグレーションをバージョン番号付きの SQL ファイルとして管理することもできます。

```
V1__Create_users_table.sql
V2__Add_email_to_users.sql
V3__Create_orders_table.sql
```



Flywayを含めほとんどのマイグレーションツールは、マイグレーションを順次実行するように設計されています。つまりバージョン100から250へ移行するには、各マイグレーションスクリプトを順番に実行する必要があります（100 → 101 → 102... → 250）。これは非効率に見えるかもしれませんが、実際にはデータの整合性を確保し、各マイグレーションが前の状態を正しく引き継げるようにするための安全機能です。高度なシナリオでは、マイグレーションの「統合」や直接的なマイグレーションパスの作成が可能です。しかし、一貫性を保証し、問題が発生した際のトラブルシューティングを容易にするため、順次実行するアプローチが標準的な手法となっています。

スキーマのバージョン管理において、最も重要なのは以下の点です。

- 各変更スクリプトに冪等性をもたせる（何度実行しても問題がないようにする）
- 可能な限り、スクリプトの後方互換性を確保する
- 変更内容と必要なデータ変換の両方を、同じスクリプトに含める

8.4.2.2　データの依存関係と変換の管理

スキーマの変更に伴いデータ変換が必要になる場合、スキーマの変更と併せてデータ変換もバージョン管理を行ってください。これにより、すべての環境においてデータベース構造とデータが常に同期された状態を維持できます。

Flywayの例の場合、マイグレーションはSQLスクリプトに限定されません。FlywayはSQLベースとJavaベースの両方のマイグレーションをサポートしており、プログラミング言語の機能を最大限に活用して複雑な変換を柔軟に処理できます。

```
// V4__Add_Address_Components.java - FlywayによるJavaベースのマイグレーション
public class V4__Add_Address_Components implements JdbcMigration {

    public void migrate(Connection connection) throws Exception {
        // まず、スキーマを変更して新しい列を追加する
        try (Statement stmt = connection.createStatement()) {
            stmt.execute("ALTER TABLE users ADD COLUMN street VARCHAR(255)");
            stmt.execute("ALTER TABLE users ADD COLUMN city VARCHAR(255)");
            stmt.execute("ALTER TABLE users ADD COLUMN state VARCHAR(255)");
            stmt.execute("ALTER TABLE users ADD COLUMN zip_code VARCHAR(10)");
            stmt.execute("ALTER TABLE users ADD COLUMN country VARCHAR(255)");
        }

        // 次に、既存のデータをマイグレーションする
        try (PreparedStatement select =
```

```
        connection.prepareStatement("SELECT id, address FROM users");
             PreparedStatement update = connection.prepareStatement(
                "UPDATE users SET street = ?, city = ?, state = ?, zip_code = ?,
                country = ? WHERE id = ?")) {

            ResultSet rs = select.executeQuery();
            while (rs.next()) {
                long id = rs.getLong("id");
                String fullAddress = rs.getString("address");

                // 新しいスキーマのコンポーネントを解析する
                AddressComponents components = addressParser.parse(fullAddress);

                // 新しい形式でユーザーを更新する
                update.setString(1, components.getStreet());
                update.setString(2, components.getCity());
                update.setString(3, components.getState());
                update.setString(4, components.getZipCode());
                update.setString(5, components.getCountry());
                update.setLong(6, id);
                update.executeUpdate();
            }
        }
    }
}
```

コードによるアプローチを使えば、アプリケーションの既存のビジネスロジックを活用し、複雑な解析処理を行い、SQL だけでは困難なマイグレーション時の検証機能を実装することが可能になります。

このアプローチにより、以下の点が保証されます。

- スキーマ変更とデータの変換が一緒にバージョン管理され、一貫性が保たれます。
- マイグレーションは不可分となり、スキーマとデータの変更は両方とも成功するか、あるいはどちらも失敗します。
- Flyway のスキーマ履歴を使用することで、どの環境に変換が適用されたかを追跡できます。

複雑な変換を行う場合は、以下のような方法も検討してください。

- システムへの影響を最小限に抑えるため、変換処理をバックグラウンドジョブとして実行する
- 移行期間中は二重書き込み方式を導入する
- 変換処理で問題が発生した場合に備え、ロールバック計画を策定する

データ品質の問題が、データ変換の過程で表面化することが多いのを忘れないでください。欠損データや無効な形式、予期していなかった特殊なケースに対処できるよう、準備をしておきま

しょう。

データ変換を行う際は、まず少量のサンプルから始め、結果を入念に検証してから、データセット全体を処理するようにしてください。このアプローチにより、後々の大きなトラブルを未然に防ぐことができます。

8.5　まとめ

読者のキャリアを通じて開発していくアプリケーションにおいて、データはその基盤となるものです。プログラミング言語やフレームワーク、ツールだけに注目したくなるかもしれませんが、データを効果的に扱う能力は貴重なスキルであり、アプリケーションの成否を左右することも少なくありません。

本章では、データを取り扱う際のさまざまな側面について見てきました。

- さまざまなデータ型と形式（構造化データと非構造化データ）の理解
- 具体的なユースケースに適したデータベースの種類の選定
- 効果的なデータ永続化パターンと接続管理の実装
- パフォーマンス向上のためのクエリの最適化
- データの増加と移行を見据えた計画

本章から得られる重要な教訓は、データ管理において完璧な解決策などほぼ存在せず、その特定の用途に適した解決策があるだけだということです。どのような決定も絶対的なものではありませんので、どのデータ構造を使用するか、あるいはどのようなデータベースを基盤とするかについて、十分な情報に基づいて判断を下し、次に進むべきです。将来の「もしも」についてあれこれ悩むよりも、現在の要件に基づいて決断を下すほうが賢明です。そもそも、想定している期間やユーザー規模まで、そのアプリケーションが使われるとは限らないのですから。

これは**オーバーエンジニアリング**と呼ばれ、しばしば不必要に複雑なシステムを生み出し、保守が困難になったり、実際のニーズへの対応が難しくなったりする原因となります。仮定でしかない将来のシナリオのために精巧なアーキテクチャを構築するのではなく、設計に適切な柔軟性を保ちつつ、現在の問題を効果的に解決することに注力すべきです。

8.6　実践に移す

データを効果的に扱うスキルは、経験を積むことで身につくものです。これまで取り上げた概念は、データ活用の道のりの基礎となります。しかし、どんな技術でもそうであるように、熟達するには実践を重ねることが不可欠です。

これらの考え方を今日から実践するための具体的な方法を紹介します。

既存のアプリケーションのデータモデルを分析する

GitHub 上のオープンソースプロジェクト（シンプルな EC プラットフォームやブログエンジンなど）を選び、そのデータ構造を整理してください。エンティティ間の関係を特定し、選択したデータベースの種類がアプリケーションの要件に適しているかどうかを評価してください。

動作の遅いクエリを最適化する

現在進行中のプロジェクトの中で、パフォーマンスが低下しているクエリを探してください。`EXPLAIN` コマンドを使用してその実行プランを分析し、適切なインデックスを追加したり、クエリを書き直して効率化したりするなど、改善策を講じてください。

ミニマイグレーションを作成する

基本的なデータベーススキーマを持つ小さなアプリケーションを作成し、その拡張方法を練習しましょう。既存のデータを保持したまま、新しいフィールドを追加したり、データ型を変更したり、テーブルを分割したりします。これらの変更を管理する際には、Flyway や Liquibase のようなマイグレーションツールを使用してください。

基本的なキャッシュ戦略を実装する

既存のアプリケーションの中から、読み込みが頻繁に行われる機能を選び、簡単なキャッシュレイヤーを実装してください。実装前後のパフォーマンスを測定し、改善効果を定量化してください。

さまざまなデータ形式を使って練習する

ある形式（CSV など）のデータセットを取り込み、それを別の形式（JSON など）に変換するコードを作成してください。特殊文字、欠損値、適切な型変換といったエッジケースへの対応に重点を置いてください。

非構造化データをどのように保存するかを検討する

シナリオ（ユーザー生成コンテンツの保存など）を選び、保存と効率的な検索の両方に対応するソリューションを設計してください。構造化されたメタデータと非構造化コンテンツを組み合わせたハイブリッドなアプローチも検討してください。

データ集約型アプリケーションのプロトタイプを作成する

大規模なデータセットを処理する小規模なアプリケーションを作成してください。例えば、ログファイルの分析や公開データセットの可視化などが考えられます。効率的なデータ読み込み、変換、クエリの最適化に注力してください。

新しいデータの概念を取り入れる際は、まずは小さなことから始めましょう。今日から、読者が興味を持つような簡単なことから始めてみてください！ 実践的な演習を重ねるごとに自信がつき、特別な直感が養われ、データに基づいて賢明に判断を下せるようになります。この実践的な知識

は、キャリアを通じて極めて貴重な資産となり、新たな可能性を切り拓きます。一見すると手に負えないように思える問題の解決にも役立つでしょう。誰もがどこからか始めるものです。初めてのデータプロジェクトが完璧である必要はありません。とにかく始めてみることが大切です！

8.7　関連リソース

- 『Designing Data-Intensive Applications, 2nd Edition』Martin Kleppmann, Chris Riccomini 共著（O'Reilly、2026 年）
- 『Seven Databases in Seven Weeks, 2nd Edition』Luc Perkins 他著（Pragmatic Programmers、2018 年）
- "Mastering SQL for Web Developers: Full Course"（freeCodeCamp）（https://oreil.ly/vGKvH）
- 『Refactoring Databases: Evolutionary Database Design』Scott W. Ambler, Pramod J. Sadalage 共著（Addison-Wesley Professional、2006 年）
- 『Fundamentals of Data Engineering』Joe Reis, Matt Housley 共著（O'Reilly、2022 年）和書『データエンジニアリングの基礎』（オライリー・ジャパン、2024 年）
- 『NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence』Pramod J. Sadalage, Martin Fowler 共著（Addison-Wesley Professional、2012 年）

9章
ソフトウェアアーキテクチャ

アーキテクチャとは重要なものについてのことです……それが何であれ。
— Ralph Johnson、コンピュータサイエンティスト、
『Design Patterns: Elements of Reusable Object-Oriented Software』共著者
（和書『オブジェクト指向における再利用のためのデザインパターン』）

ソフトウェアアーキテクチャは、非常に広範なテーマです。筆者の一人はミネソタ大学でこのテーマに関する講義を行っていますが、大学院の 1 学期という期間では、その表面に触れる程度にとどまります。このテーマについては、人によって定義はさまざまですが、プロジェクトの長期的な成功にとって重要であるという点では、誰もが一致しています。本章を読んだだけでソフトウェアアーキテクトになれるわけではありません。しかし、トレードオフの重要性、答えがほぼ常に「状況による」となる理由、そして進化しつづけるアプリケーションを構築するうえで品質特性が鍵となる理由を確実に理解できるようになるでしょう。

9.1　アーキテクチャとは何か？

20 人のアジャイルエンジニアを一つの部屋に集め、それぞれに「アーキテクチャ」の定義を書き出してもらったことはありますか？ 私は一度、そうしたことがあります。すると、20 通りの異なる答えが返ってきました。
— Matt Parker、著者、エンジニアリングリーダー

4 章で述べたように、ソフトウェア業界はまだ歴史が浅く、そのため、用語はより成熟した分野から借用することがよくあります。**アーキテクチャ**はその典型的な例であり、この用語は建築業界に由来しています。新しい家の基礎を掘る前に、アーキテクト（建築家）は構造を設計し、完成した家が地域の建築基準に適合すると同時に、将来の居住者のニーズも満たすようにします。建築家は、全体像、構造、プロジェクトのビジョン、つまり電気、配管、換気など、後から変更が難しい部分について責任を負います。一度コンクリートを打設してしまえば、リファクタリングするのは

本当に難しいのです！

ソフトウェアにおいても、アーキテクチャはほぼ同様です。ソフトウェアプロジェクトにおいて、アーキテクトはシステムの全体設計を担当し、スケール、パフォーマンス、セキュリティ、その他の品質特性（「〜性」と呼ばれるもの）に関する、あまり明文化されていない要件も含めて考慮に入れます。過去の経験はこうした課題を乗り越える助けにはなりますが、アプリケーションはそれぞれ異なります。データストアからエラーメッセージが表示された場合、インターネットで検索すれば答えが見つかることでしょう[†1]。しかし「Foo アプリにはどのデータストアを使うべきか？」という問いに対しては、いくら検索しても満足のいく答えは得られないでしょう。

> アーキテクチャとは、Google で検索しても見つからないものです。
>
> — Neal Ford と Mark Richards、『Fundamentals of Software Architecture, 2nd Edition』（和書『ソフトウェアアーキテクチャの基礎 第 2 版』）の著者

アーキテクチャとは、後から変更するのが難しい決定事項である、と定義されることがあります。一戸建て住宅と集合住宅では要求が異なるのと同じように、ソフトウェアアーキテクトはアプリケーションの全体像を理解しなければなりません。ソフトウェアは家屋よりもリファクタリングが容易ですが、アーキテクチャ上の決定の中には、後から修正するのが極めて困難なものもあります[†2]。例えば、アプリケーションを Spring を基盤として Java で記述し、多くの小さな独立したモジュール（マイクロサービスと呼ばれます）に分割して構築したとします。しかし、プロジェクト開始から半年後に考えを変えてしまうと、大変な問題になります。アーキテクチャには批判的思考が求められるのです！ アーキテクトは、そのプロジェクトにおける重要な品質特性を見極め、設計がそれらの要件を満たしていることを保証します。

結局のところ、アーキテクチャとは、後で変更するのが難しい事項について、トレードオフの分析に基づいて判断を下すことです。そのため、アーキテクトは、アプリケーションが機能要件を満たすだけでなく、それ以上の価値を提供できるよう、時間をかけて問題領域を徹底的に検討します。アーキテクトは、次のような質問を頻繁に投げかけます。

- このアプリケーションは、何人のユーザーに対応する必要がありますか？ そのうち、同時接続ユーザー数はどれくらいですか？
- ユーザーはどの地域にいますか？ 主に同じ地域に集中していますか、それとも世界中に分散していますか？
- このアプリケーションの可用性目標はどの程度ですか？
- アプリケーションの需要や利用が急増する要因は何ですか？
- このアプリケーションは通常、いつ利用されますか？ 営業時間内ですか？ それとも、一日

†1 答えが多すぎることもあります。

†2 将来を予測することはできませんが、アプリケーションがどのように進化していくかを慎重に検討したうえで設計しておけば、将来的な変更を容易に行えます。

中いつもですか？

- どのような制約がありますか？ このアプリケーションはどのようなシステムと連携しますか？
- 組織はどのクラウドプロバイダーと契約していますか？
- 競合他社はどのような状況ですか？ 競合他社に対して、どのような点で同等かそれ以上の水準を維持する必要がありますか？
- 理想的な導入環境はどのようなものですか？
- 現在の導入環境はどのようなものですか？
- アプリケーションはどのようなデータを保存すべきでしょうか？ そのデータはどのくらいの期間保持すべきでしょうか？
- アプリケーションのカーボンフットプリントはどの程度であり、どのように最適化できるでしょうか？
- このアプリケーションのコストはどの程度であり、どのように最適化できるでしょうか？
- どのような法令や規制を遵守する必要がありますか？
- システムの各要素において、許容される応答時間はどの程度でしょうか？
- このアプリケーションは、どのようなセキュリティポリシーを満たす、あるいは上回る必要がありますか？
- このアプリケーションにはどのようなプライバシー上の懸念がありますか？

アプリケーションは一つひとつ異なるため、これらの質問に一概に答えられるものではありません！ だからこそ、アーキテクチャの選択において「正解」というものはないのです。あらゆる決定について、そのメリットとデメリットを慎重に検討する必要があります。例えば、ブラジルで300人のユーザーを対象としたドキュメント管理システムを構築する場合、アプリケーションはブラジル国内またはその近隣のデータセンターに配置することになるでしょう。ブラジル市場における特定の法律や規制を理解するために法務部門への相談も必要です。また、アプリケーションが1か月あたり処理するドキュメントの数や、それらのドキュメントの一般的なサイズについても把握しておく必要があります。さらに、ブラジル国外への展開計画があるかどうか、ドキュメントの翻訳が必要になるかどうか、どのような種類のドキュメント分析が必要となるかについても確認しておくのが賢明でしょう。

9.2 トレードオフ

> プログラマは、あらゆるものの利点を知っていますが、トレードオフについては何も知りません。アーキテクトは、その両方を理解する必要があります。
>
> — Rich Hickey、Clojure プログラミング言語の作者

アーキテクチャとは、単純な答えが存在しない問いについて考える分野です。だからこそ、アー

キテクトはほぼすべての質問に「状況による」と答えるのです[†3]。学校では、A 評価がもらえる答え、B 評価がもらえる答え、C 評価がもらえる答えがあることを学びました。しかし、ソフトウェアの世界では、Neal Ford の言葉を借りれば「アーキテクチャに正解も不正解も存在せず、あるのはトレードオフだけ」なのです。さらに事態を複雑にしているのは、ソフトウェアの世界には「最良の」答えが存在しないということです。ほとんどの場合、最も悪くない答えを受け入れるしかありません。ソフトウェアプロジェクトにおけるあらゆる決定にはトレードオフが伴います[†4]。

アーキテクトの仕事は、プロジェクトにおいて検討されている（数多くの）選択肢について、トレードオフ分析を行うことです。あらゆるアーキテクチャスタイルには、さまざまな特性に対して、異なる「星評価」がつけられますが、その分析は単に星の数が最も多いものを選ぶほど単純なものではありません。ある特性が他の特性**よりも重要**となる場合があり、その結果、一見すると理想的ではないと思われるアーキテクチャスタイルを選ぶことになるかもしれません。多くのプロジェクトが、学術的には完璧なアーキテクチャを創り出そうとして失敗しますが、それはそもそも達成不可能なものです。優れたアーキテクトは、プロジェクトの納期を守りつつ、可能な限り最善の決断を下します。

成り行きのアーキテクト

多くの組織では、役職に関してやや「縄張り意識」のようなものがあります。しかし、エンジニアがアーキテクトという肩書きを持たなくてもアーキテクチャ関連の業務を行うことは非常に一般的です。これは筆者たち自身も身をもって実感していることです。また、アーキテクトが余るほどいる組織を見たことがありません。多くのアーキテクトは複数のプロジェクトに携わり手一杯の状態です。そのため、「アーキテクト」に相談しにくい（あるいはまったく相談できない）状況に置かれ、知らず知らずのうちにアーキテクトの役割を担うことになるかもしれません。もし自分がアーキテクトの役割を果たしていることに気づいたら、全体像を把握するために時間を割くようにしてください。そして、チームや組織内の他の人々からフィードバックを積極的に求めてください。

トレードオフを慎重に検討する必要があります。例えば、エンジニアは、モジュール間で共通して使用される機能を、再利用可能なライブラリとして抽出して、再利用を促進し保守性を高めることがよくあります。しかし、そのライブラリは、実行時に呼び出されるサービスとすべきでしょうか、それとも呼び出し側と一緒にビルドされるライブラリとすべきでしょうか？ 言い換えれば、実行時の依存関係とすべきか、ビルド時の依存関係とすべきか、ということです。これは、状況によります！

†3 筆者たちは、会議の時間を節約するために、「It Depends（状況による）」と書かれた T シャツを着ていることさえあります。

†4 それに、人生で下すあらゆる決断についても。

では、どのような状況によるのでしょうか？ いくつか考慮すべき点があります。まず、そのライブラリの**変動の度合い**、つまり変更の頻度はどの程度か、という点を考えるのがよいでしょう。これに関連して、呼び出し元にとって最新バージョンを利用することがどれほど重要か、という点も考慮すべきです。もしライブラリがほぼ絶えず変更されているのであれば、実行時依存関係を採用するのが合理的です。必要に応じてライブラリの新しいバージョンをデプロイでき、すべての呼び出し元が常に最新バージョンを利用するため安心できます。

しかし、アプリケーションから外部プロセスを呼び出すようにしたことで、新たなエラーが発生するおそれが生じます。外部ライブラリサービスに接続できない場合、アプリケーションはどのように対応するのでしょうか？ そのライブラリは地理的に分散配置する必要があるのでしょうか？ ライブラリの機能によっては、代替手段やキャッシュされた回答、あるいはデフォルトの結果を用意する必要があるかもしれません。ライブラリをビルド時の依存関係とすることを選択すれば、ネットワーク経由で通信を行わないので、エラーの要因が一つ減ります。テストも容易になります。ただし、そのライブラリのアップグレードを組織全体に展開するのは難しくなり、ライブラリの利用者によって異なるバージョンがデプロイされる可能性が高くなります。

ソフトウェアアーキテクチャに関する役立つヒント

- 良いものは悪いものより良いものですが、そうでない場合もあります。
- また、何事も良いとも悪いとも言えません。
- それは状況によります。
- どんな質問にも答えは「状況によります」ですが、そうでない場合もあります。状況によります。
- 好きなものを 3 つ挙げてください。それらを同時に手に入れることはできません。
- ダメです。
- ソフトウェアアーキテクチャには多くの定義がありますが、どれも正しいものではありません。
- ソフトウェアアーキテクチャなどというものは存在しません。

— Ken Scrambler

どのアプローチが「正解」なのでしょうか？ 状況によります！ 例えば、配送先に基づいて消費税を計算するユーティリティを考えてみましょう。明らかに、そのルールは定期的に変更される可能性があり、システムが最新のルールを使用することは重要です。しかし、ネットワークの問題でアプリケーションが税金を計算できなくなった場合に備えて、バックアップを用意しておく必要があります。もし、バックアップを用意していなければ、組織の上層部から緊急の連絡が入ることを覚悟しなければなりません。このような状況では、どのような対策を推奨するでしょうか？

9.3 アーキテクチャと設計

アーキテクチャと設計の境界は曖昧です。どのプロジェクトにおいても、設計上の決定はアーキテクチャに影響を与え、その逆もまた然りです。これは「ツインピークスモデル」と呼ばれることもあり、ソフトウェアプロジェクトでよく見られる「鶏が先か卵が先か」のような状況を表しています。現在の設計がアーキテクチャの柔軟性を損なう可能性があり、また、現在のアーキテクチャが設計の自由度にも影響を及ぼすのです！ 決定事項の中には、アーキテクチャか設計のどちらかに明確に分類できるものもありますが、その大部分は両者の間の中間領域に位置しています。例えば、アプリケーションに最適なアーキテクチャスタイルを選択することは明らかにアーキテクチャ上の決定です。一方、クラス内のデータ型を選択することは設計上の決定です。

アーキテクチャと設計の区別が、多くの場合、程度の問題に過ぎないとしたら、その区別は重要なのでしょうか？ それは状況によります！ 多くの場合、最終的な決定を下すのに最も適した立場にあるのは誰か、という点に尽きます。アーキテクトと開発者は、両者の中間領域にある決定事項について、協力して取り組む姿勢（そして意欲）を持つべきです。

もう一つの視点として、この決定は戦略的なものか、それとも戦術的なものかを自問してみてください。**戦略的**思考では、長期的な目標やプロジェクト全体の方向性に焦点を当て、先を見据えて考える必要があります。多くの場合、戦略的な対応には少し時間がかかったり、場合によっては少し後戻りする必要があるかもしれません[†5]。**戦術的な**決定は、多くの場合、差し迫った深刻な問題を解決するための短期的なものであり、後日、より長期的な解決策が必要になる可能性があります[†6]。これは、恒久的な解決策ではないと承知のうえで、ダクトテープで何かを固定するようなものだと考えてください。

決定が戦術的であればあるほど、それは設計に近いものとなります。決定が戦略的であればあるほど、それはアーキテクチャの領域に近づきます。その決定には何人の人が関わるのでしょうか？ 彼らの役割は何でしょうか？ 開発者 1〜2 名で決定できるようなことは、戦術的な決定の典型例です。一方、CTO を交えて数週間にわたる会議を経て初めて決定されるような場合は、戦略的な決定と言えます。

この違いを別の角度から考えるのであれば、次のように自問してみてください。「変更にかかるコストはどれくらいか？」 メソッドのリファクタリングは機械的な作業であり、IDE に任せられます。しかし、広く利用されているサービスのシグネチャを変更することはそうではありません。メソッドのリファクタリングは戦術的なものですが、モノリシックシステムをマイクロサービスにリファクタリングするのは戦略的なものです。メソッドの命名は戦術的なものですが[†7]、無数の顧客が利用する外部サービスの命名は戦略的なものです。プロジェクトを進める中で、数え切れないほどの戦術的・戦略的な選択に直面することになり、不完全な情報や限られた時間の中で決断を下さなければならないことも多々あるでしょう。

---

†5 時間をかけてこそ、結果的に時間を節約できるのです。
†6 その場しのぎの対策が、何年経ってもまだ続いているとしても、驚かないでください。
†7 とはいえ、その重要性や、活発な議論を軽視するものではありません。

9.4 品質特性

前述の通り、アーキテクトの関心事は、アプリケーションの機能や性能にとどまらず、より広範なものです。アーキテクトは、完全なソリューションを提供するために必要となる、品質特性についても考慮しなければなりません。これらは、非機能要件、アーキテクチャ特性、品質目標、制約、サービス品質目標、アーキテクチャ上重要な要件、あるいは「〜性」という表現で呼ばれることもあります[†8]。アーキテクトとして、アプリケーションにとって最も重要な品質特性を見極めることが極めて重要です。

非機能要件

「非機能要件」という概念に馴染みがある方も多いと思います。なぜ筆者たちがこの一般的な用語を使わないのか、不思議に思われるかもしれません。筆者たちがこのような選択をしたのは、長年にわたる経験から得た気づきによるものです。ステークホルダーと**非機能的な**事柄について話すと、彼らは（意識的か無意識かは別として）「非機能的な話はいいから、私のビジネス要件に集中してほしい」と考えてしまい、話を聞かなくなってしまうことがよくあるのです。

ステークホルダーは、アプリケーションに求めるものに強く意識を向けがちです。それ自体は重要なことであり、このようなニーズに応える必要があります。しかし同時に、より大きな視点、つまり多くのステークホルダーが考えていないような点にも目を向ける必要があります。**品質特性**という言葉を用いることで、議論の雰囲気は一変します。多くの顧客は、品質について話し合うことを喜んでくれます。

保守性、拡張性、信頼性、セキュリティ、導入の容易さ、簡潔さ、ユーザビリティ、互換性、耐障害性など、品質特性にはさまざまなものがあることはご存じでしょう。品質特性を挙げていくときりがありません[†9]。では、最も重要な品質特性は何でしょうか？ それは状況によります[†10]！ どの特性が最も重要かは、開発するアプリケーションの種類によって決まります。例えば、デバイスドライバーを作成する場合、保守性よりもパフォーマンスのほうがはるかに重要になるでしょう。一方で、EC プラットフォームであれば、可用性とスケーラビリティが優先されるでしょう。

さらに事態を複雑にしているのは、すべての設定を最大値まで上げられるわけではないという点です。実際、品質特性の中には、互いに逆相関の関係にあるものもあります。つまり、ある特性を最大化すれば、別の特性は最小化されてしまうのです。セキュリティとユーザビリティは、時に相

†8 よく言われる冗談ですが、どんな単語にも「〜性」という接尾辞をつけるだけで、アーキテクトが関心を寄せるものになるそうです。

†9 もっと多くのアーキテクトが、シンプルさを追求すべきです。

†10 アーキテクチャの世界では、どんな質問に対しても「それは状況による」という答えになるというのは、決して冗談では言っていません。

反する関係になることがあります。完全に安全なシステムを設計し、ネットワークに接続せず、施錠された部屋で稼働させ、さらに言えばログイン画面すら設けない方法もあります。確かに安全ではありますが、ユーザビリティという点では決して良いとは言えません。

読者の課題は、品質特性を特定し、それらを互いに適切なバランスで調整することです。さらに、目に見えにくい品質特性の価値について、ステークホルダーを納得させる必要があることもよくあります。

9.4.1 品質特性の特定

プロジェクトにおいて、どのような品質特性が重要なのか、どのように判断すればよいのでしょうか？ 品質特性を示す特定の言葉やフレーズに、目を留めるべきです。例えば、以下の Concert Comparison kata（コンサート比較カタ）（https://oreil.ly/phDwU）を考えてみましょう。

人気アーティストの公演を多数取り扱う大規模なコンサートチケット販売サイトには、チケット販売のための柔軟なソリューションが必要です。

- ユーザー数：数千人の同時接続ユーザー。チケット発売開始時には、1 秒あたり最大 10,000 人までのアクセス集中が見込まれます。
- 要件
  - 複数のチケットを同時に購入できるようにします。
  - 同じ座席を重複して販売しないでください！
  - 購入者は残席状況を一覧で確認できます。
- 補足情報
  - スペースベース型（共有メモリを使った分散アーキテクチャ）とマイクロサービス型の両方のアーキテクチャスタイルでの実装を検討してください。
  - 各ソリューションにおけるトレードオフを特定してください。

これらの要件を読み進めると、いくつか注目すべき点があるはずです。チケットの販売開始時に数千人の同時接続ユーザーが発生し、アクセスが急増することを想定すると、高いスケーラビリティとレジリエンスを兼ね備えたソリューションが必要となります[†11]。また、最初のアクセス集中が収まった後は、ほぼゼロに近いレベルまでスケールダウンできることが望ましいでしょう。

当然ながら、同じ座席を 2 度販売することはできません。つまり、座席を予約済みとして確保する方法を考えなければなりません。一方で、一人のユーザーが数十席や数百席もの座席を独占できないような仕組みにする必要があります。また、ここでは触れられていませんが、ボットを排除するための対策はどのようなものになるのでしょうか？

読者の状況において、どの品質特性が最も重要か見当がついたら、時間をかけてそれらを順位付けしましょう。順位付けは、マインドマップや表、あるいは単に番号付きリストを使って行えま

†11 Taylor Swift ですかね？

す。リストが完成したら、次はフィードバックを求めましょう。作成した資料をステークホルダーに共有し、そのフィードバックを踏まえて改善を重ねていきます。

9.4.2 ステークホルダーとの合意形成

ステークホルダーに対して、品質特性の重要性をどのようにして理解してもらうのでしょうか？ここで、影響力を行使するスキルが役立ちます（影響力の行使やステークホルダーの管理に関する詳細は 13 章を参照してください）。意思決定者に対してセキュリティの重要性を説明するのは比較的簡単かもしれませんが、保守性や簡潔さの価値を理解してもらうためには、影響力を行使する能力を発揮する必要があります。

まずは、相手にとって関連のある用語や具体例を用いて、品質特性のメリットを概説することから始めてみてください。物事を純粋に技術的な観点だけで捉えないよう注意してください。技術的な側面は読者にとって重要かもしれませんが、例えば顧客にとっては、技術的な選択について深く考える必要はないのです[†12]。まずは共通の土台を見つけ、顧客の懸念事項について話し合いましょう。彼らの懸念が重要ではないと思われるかもしれませんが、最後まで話を聞いてください。そこには必ず何かを得るものがあります。顧客の懸念を考慮することは重要です。結局のところ、対話を行うことが不可欠なのです。

また、組織内の影響力のある人物を見極めることも重要です。多くの場合、こうした人物は、組織図だけではわかりません。アーキテクトとして、意思決定者に直接アプローチできるとは限りません。目的を達成するためには、誰かを介して働きかける必要があるでしょう。対等な立場で接し、自分のアイデアの説得力と評判を頼りにしましょう。共感できる点を見つけ、相互利益の力を活用してください。

> 改革は、回り道をすることで最も急速に進展することがよくあります。
>
> — Frances Willard、米国の禁酒運動・女性参政権運動家

間接的なアプローチの力

Nate です。数年前、ある組織に Git を導入しようと試みました。私たちは、既存のバージョン管理ツールを置き換えたい理由をまとめて、非常に説得力のある提案書を（しかも Gist 形式で）作成しました。そのプレゼンテーションにより、アーキテクチャグループからは承認を得ることができました。しかし、既存のツールを管理しているチームを説得しようとしたところ、きっぱりと拒否されてしまいました。「すでにツールはあるし、新しいものには一切応じない」と告げられたのです。

私たちは一歩引いて、より影響力のある協力者、つまり CTO の右腕として活躍する人物

†12 率直に言って、自分のほうが読者よりもソフトウェアについて詳しいと思い込んでいる顧客が最も厄介です。

を味方につけました[†13]。私たちのアプローチを説明したところ、彼は「Git を既存のツールキットの**代替**としてではなく、**補完するもの**として導入すべきだ」という鋭い意見を示してくれました。彼は Git を、コードの共同作業における課題を解決する手段、つまり成果物をより容易に共有できるようにするツールとして位置づけたのです。この間接的なアプローチは、既存のツールを支持していたチームからの反発を和らげるのに十分でした。何しろ、Git は補完的なものだったからです[†14]。私たちはすぐにパイロットプロジェクトを立ち上げ、活用できる機会を何度も見出し、数か月以内に全社的なソリューションを構築できました。

この話から学べる教訓とは？ 直接的な道ほど、抵抗に遭いやすいものです。新しいアイデアや技術、ツールを導入する際は、別のアプローチや切り口を検討してみてください。

9.5 アーキテクチャスタイル

アプリケーションの主要な品質特性を特定すれば、アーキテクチャスタイルを分析して適切なものを選べます。アーキテクチャスタイルとは、コードの全体的な構造や、データストアとの連携方法を指します。数多くのスタイルから選択できます。ご想像の通り、万能な解決策というものは存在しません。

大まかに言えば、アーキテクチャスタイルは、モノリシック（単一のデプロイ可能なユニット）と分散型（複数のデプロイ可能なユニット）の 2 つの主要なタイプに分類できます。モノリシックなアーキテクチャは、保守性が低いと批判されることがよくありますが、その原因はデプロイトポロジーではなく、構造の悪いシステムにあります。保守性の高いモジュール型のモノリシックアーキテクチャは実現可能です。一方、適切な規律と技術的負債への配慮がなければ、高度に分散化されたシステムを進化させるのは極めて困難になります。

モノリシックなスタイルには、「ビッグボールオブマッド（大きな泥団子）」、レイヤード、マイクロカーネル、パイプラインなどがあります。分散型のスタイルには、スペースベース、サービス指向、マイクロサービス、イベント駆動型などがあります。さまざまなアーキテクチャスタイルを網羅的に解説することは、本書の範囲を超えてしまいます[†15]。しかし、各アーキテクチャスタイルには、さまざまなアーキテクチャ特性においてそれぞれ異なる長所と短所があることを理解しておくことが重要です。例えば、マイクロサービスアーキテクチャは優れた弾力性とスケーラビリティを提供します。しかし、ネットワークホップが増えるため、パフォーマンスが低下するおそれがあり、他の選択肢と比較して複雑さとコストが増大します。また、マイクロサービスには監視とオブザーバビリティが求められ、デバッグやエラーの追跡が困難になる場合があります。

†13 確かに、実際には組織図上で役職名がついていたわけではありません。しかし、この人物が経営陣から高く評価されていることは明らかでした。

†14 また、組織内で影響力の大きい人物が提案を行ったことも追い風となりました。

†15 詳細については、『Fundamentals of Software Architecture』Mark Richards, Neal Ford 共著（O'Reilly、2020 年）、和書『ソフトウェアアーキテクチャの基礎 第 2 版』（オライリー・ジャパン、2026 年）を参照のこと。

適切なスタイルを選ぶには、トレードオフの検討が不可欠です。アプリケーションにスケーラビリティと弾力性が**必要な場合**、マイクロサービスは優れた選択肢となり得ます。しかし、それらが重要な特性ではない場合は、別のスタイルのほうが適しているでしょう。あるアプリケーションにとって正しいアプローチが、別のアプリケーションにとっては間違っていることもあります。繰り返しになりますが、「どのアーキテクチャスタイルを使うべきか？」とインターネットで検索しても答えは見つかりません[†16]。トレードオフを慎重に検討する必要があります。特定の状況に適したアーキテクチャスタイルを見極めることは、アーキテクトが行う極めて要な仕事です。

9.6 アジャイルアーキテクト

アーキテクチャとは、後から変更するのが難しい決定事項であると定義されることがよくあります。しかし、技術の世界において唯一不変なのは「変化しつづける」ということです。では、アーキテクチャは本質的にアジャイルプロジェクトと相容れないものなのでしょうか？ 一部の組織は、自らがアジャイルであるとして（誇らしげに）アーキテクトを置いていないと公言しています。肩書きの如何にかかわらず、プロジェクトにはアーキテクチャ上の決定を下す人々が存在します。彼らが適切な判断を下していることを願うばかりです。その判断が正しかったかどうかは、1、2 年後には明らかになるでしょう。

アーキテクチャとアジャイルは間違いなく共存できます。ただし、アーキテクチャに関するアクティビティの一部、おそらくその大部分はイテレーションの期間内には完了しないでしょう[†17]。アジャイルなアーキテクチャを構築するには、これまでの前提を少し変える必要があるかもしれません。将来の要件を難なく処理できるアーキテクチャを作ろうとするのではなく、物事が変化することを前提にアーキテクチャを設計したらどうでしょうか？ この考え方は、進化的アーキテクチャのコアをなす理念の一つです。

> 進化的アーキテクチャは、複数の次元にわたる、計画的かつ段階的な変更をサポートします。
> — 『Building Evolutionary Architectures, 2nd Edition』（Neal Ford 他著）より。

進化的アーキテクチャを採用すれば、ビジネスニーズや優先順位の変化、技術の進歩に応じて、段階的に変更を加えることが可能になります。ここで重要なのは**「段階的」**というキーワードです。アジャイルソフトウェア開発は、究極的には入れ子状のフィードバックループが核心となります。数か月かけて多数の機能の開発に取り組んだ挙句、目標から外れていたことに気づくような事態を避けるため、定期的なデモンストレーションを行うようにしてください。主要なステークホルダーが実際に動作するソフトウェアに直接触れ、調整を行える機会を提供するのです。仮説駆動型開発は、データを収集し、それに応じて開発を進めるのに役立つ手法です。

†16 状況によります。

†17 調達部門や法務部門と仕事をしたことがある方なら、スケジュールを自分でコントロールできないことにすぐに気づくでしょう。

仮説駆動型開発では、次のような枠組みを活用します。

- 私たちは**<この変更>**により
- **<この結果>**がもたらされると信じている
- **<このメトリクスで X という変化が確認された場合>**成功したと判断する

例を以下に示します。

- 分散型キャッシュを追加することにより
- 起動時間が短縮されると信じている
- 起動時間が 15 秒未満になれば、成功したと判断する

データに基づいて判断するほうが、1 時間も議論を重ねた挙句、結局は最も声が大きい人の意見に流れてしまうよりも、はるかに賢明です。

名前が変われども…

多くの組織において、役職名は時に奇妙なものであり、企業によって役割や責任の範囲は大きく異なります。「アーキテクト」も例外ではなく、むしろ状況はさらに複雑かもしれません。企業によっては、アーキテクチャ担当副社長の直属となる限られたメンバーにのみ、アーキテクトの肩書きを与える場合もあります。肩書きの如何にかかわらず、シニア開発者が、本来はアーキテクトの管轄であるような意思決定に積極的に関与することも、ごく一般的なことです。

アーキテクトは多忙を極めることが多く、アーキテクチャ関連の業務の一部を開発者に任せることもあります。これは業務負担を分散させるだけでなく、将来アーキテクトを目指している人にとって、素晴らしい成長の機会となります。

もちろん、アーキテクトという役職自体が存在しない企業もあります。繰り返しになりますが、役職の如何にかかわらず、プロジェクトにおいてアーキテクチャに関する決定を行っている人々がいます。彼らが適切な判断を下していることを願うばかりです。それが正しかったかどうかは、おそらく 1〜2 年後になればわかるでしょう。

9.7　適応度関数

多くのアーキテクトや組織は、プロジェクトに適したアーキテクチャを構築するプロセスに多大な投資を行っています。しかし、そのアーキテクチャを長期的に**維持する方法**については、あまり深く考えられていないことが多いようです。残念ながら、熱力学の第 2 法則はソフトウェアにも当

てはまり、無秩序化に対抗する努力をしなければ、アーキテクチャは劣化していきます[†18]。

最初のコミットから、アーキテクチャは変化しつづけています。開発に深く関わってきたメンバーは、アーキテクチャについて主要な考慮事項を詳しく熟知しているかもしれませんが、新しいチームメンバーにはその背景知識がないおそれがあります。十分な注意を払わないと、顧客からパフォーマンスやスケーラビリティ、あるいはその他の重要な品質特性について不満の声が上がるでしょう。

パフォーマンスの問題

Nate です。アーキテクトとしてのキャリアを始めたばかりの頃、古い VB6 アプリケーションのパフォーマンス問題を解決する任務を任されました。プロジェクト（そして、その役割！）に不慣れだった私は、この問題がいつから始まったのかを尋ね、最近の更新と関係があるのではないかと期待しました。しかし、問題が過去 3 年のどこかで始まったと聞いたとき、私たちには膨大な作業が待ち受けていると悟りました。そしてアーキテクトになりたいという自分の思いに疑問を抱き始めました。

今思えば、この問題の特定は、それほど困難な作業になるはずはなかったのです。もしチームが定期的にパフォーマンスをテストする適応度関数を作成していれば、おそらく顧客が気づく前に、もっと早くこの問題を発見できていたでしょう。

アプリケーションが「ビッグボールオブマッド」のような状態に陥らないことを願うだけという方法もあります[†19]。それよりも、積極的に対処することもできます。適応度関数を活用してアーキテクチャを継続的にテストし、何かが許容範囲を外れた際に警告を発するようにするのです。適応度関数とは、アーキテクチャが目標達成にどれだけ近づいているかを測定するものです。適応度関数の概念は、遺伝的アルゴリズムに由来しています。アルゴリズムは突然変異させられ、その結果がテストされます。この変異は成功でしょうか？ このような考え方をアーキテクチャに適用します。適応度関数を利用することで、アプリケーションの重要なアーキテクチャ特性を把握し、それを維持することが可能となります。

適応度関数とは、リファクタリングや新機能の導入によって、アーキテクチャが劣化していないことを確認するテストのことです。適応度関数は、アーキテクトから開発者への「TODO リスト」であり、負担が少なく形式ばらないガバナンスであると考えることができます。簡単に言えば、適応度関数とは、アーキテクチャの妥当性を検証するために実行する、一連のテストのことです。例えば、以下のようなことを確認するためのテストを作成します。

†18 端的に言えば、熱力学の第 2 法則は、外部からの介入がなければ、孤立系におけるエントロピーは決して減少しないことを定めています。言い換えれば、10 代の若者の寝室のようなものです。

†19 願うことは戦略ではありません。ただ、願うことこそが変革の礎となるものです。

- サービスへの呼び出しには、平均 100 ミリ秒以内に応答すること
- サイクロマティック複雑度は 5 を超えてはならない
- ユーザー数やリクエスト数が増加しても、平均応答時間は妥当な水準を維持すること

理想を言えば、適応度関数はすべて自動化され、CI/CD パイプライン内で実行されるべきです。しかし、テスト対象によっては、手作業で行う必要があるテストもあるでしょう。アーキテクトは、プロジェクトのライフサイクルの早い段階で適応度関数を設定しますが、それらは時間とともに進化し、変化していきます。そのため、定期的に見直しを行い、依然として適切であるかを確認する必要があります。また、より良いテスト方法が見つかる可能性もあります。

9.8 アーキテクチャ図

結局のところ、ソフトウェアエンジニアリングのあらゆる要素と同じように、アーキテクチャとはコミュニケーションの手段であり、その表現方法の一つがモデルです（モデリングの詳細については 4 章を参照してください）。活用できるアーキテクチャ図は数多く存在しますが、どの図をいつ、どのような対象者に向けて使用すべきかを理解しておく必要があります。開発者にとっては完璧なアーキテクチャ図であっても、エンジニアリング担当副社長にとっては理解不能なものになってしまうでしょう。

図を作成する前に、その対象となる人々を考えてみてください。このモデルは開発者向けでしょうか？ それとも同じアーキテクトでしょうか？ あるいは、ビジネスパートナーでしょうか？ 時には、問題を分解したり、解決策を探ったりするために、自分だけのために図を作成することもあるでしょう。図は、コンテキストを提供し、複雑さを説明・管理し、品質特性を見極めるのに役立ちます。

コンポーネント図から配置図、シーケンス図に至るまで、選択肢は豊富にあります。適切なタイミングで適切な図を作成することは、アーキテクトが身につけるべき重要なスキルです。迷ったときは、その図がストーリーを伝えるのに役立つかどうか自問してみてください。役立つのであれば、ぜひ作成してください。そうでない場合は、遠慮なく省略してもかまいません。

9.9 アーキテクチャ決定レコード

どのようなソフトウェアプロジェクトにおいても、数多くの決定を下すことになります。変数の命名のような些細なものもあれば、クラウドプロバイダーの選定のような、より重大な決定もあります[†20]。必然的に、プロジェクトの進め方について難しい判断を迫られる状況に直面することになるでしょう。多くの場合、アーキテクトは、時間や情報が不十分な状況で決断を下さなければなりません。数時間あるいは数日間の検討を経て、チームは選択を行い、その先へと進んでいきます。

†20 変数名の重要性を軽視しているわけではありません。

しかし、なぜその選択をしたのか、**その理由**を書き留める時間をとりましたか？ ソフトウェアアーキテクチャの第2法則では、「なぜ」は、「どのように」よりも重要である（https://oreil.ly/yQfLw）と述べています。

もちろん、少なくともその瞬間においては、その決定に至った経緯はわかっていることでしょう。しかし、読者の後任となる人はどうでしょうか？ また、ほんの数週間後、危機的な状況に頭がいっぱいの自分はどうでしょうか？ **アーキテクチャ決定レコード（ADR、Architectural Decision Record）**は、組織の知見を時系列のログとして残すものです。将来のチームメンバーがADRを読むことで、プロジェクトがどのようにして現在の段階に至ったのかを理解できるようになります。

ADRは、Michael Nygardによって導入された（https://oreil.ly/z8H19）、意思決定の背景にある動機を記録する手段です。これらは分厚い大著になることを意図したものではありませんが、「なぜ」を説明するのに十分な情報を含んでいる必要があります。ほぼすべてのワードプロセッサで作成できますが、MarkdownやAsciiDocのようなプレーンテキスト形式を使用するのが一般的です[†21]。

Web上にはいくつかのテンプレートが公開されています（https://oreil.ly/PYV5G）が、ADRの基本的な構成は以下のとおりです。

タイトル

決定内容の説明には、3桁の連番IDを前置します。このIDを使用することで、ADRを簡単に時系列順に並べ替え、今日に至る経緯を素早く確認できます。わかりやすい名前をつけてください。

ステータス

組織によってステータスの呼び方は異なる場合もありますが、一般的には以下のようなものが使用されます。

- コメント募集：本ADRは、チームおよびステークホルダーによって、現在精査されています。
- 提案中：本ADRは現在作成中であり、承認待ちの状態です。
- 承認済み：本ADRは審査を経て、採用されました。
- 無効化：ADRは変更不可ですが、後のADRによって以前のADRが無効化される場合があります。無効化されたADRは、上書きしたADRを参照するように更新できます。

コンテキスト

どのような意思決定にも、制約や固有の状況が伴います。例えば、CIOが指定する優先ベンダーリストがある場合、それによって選択肢が不自然に狭まってしまうおそれがあります。コンテキストを軽視しないでください。

†21 プレーンテキストを使用することは、検索性を高めることにもつながり、これは非常に重要です。

選択肢

どのような選択肢を検討しましたか？ もし検討対象から除外したものがあれば、その理由は何ですか？

決定内容

どのような決断を下しましたか？ その決断に至った理由は何でしょうか？ どのような選択肢を却下しましたか？ そして、その理由は何でしょうか？ 繰り返しになりますが、情報は多すぎるくらいのほうがよいでしょう。

結果

あらゆる決定にはすべて（良い面も悪い面も）結果が伴います。この ADR にはどのような結果が伴うのでしょうか？ 繰り返しになりますが、情報が多すぎるくらいのほうが無難です。

ガバナンス

その決定が実際に守られるようにするには、どのようにしますか？ この ADR を適用するために、どのような適応度関数を作成しますか？

メモ

この項目には、必要に応じてその他の情報に加え、作成者、各種日付（受理日、無効化日など）、承認者といった ADR に関するメタデータを記述します。

実例は非常に参考になりますが、入手するのは難しいものです。何しろ、ほとんどの企業は自社のソフトウェアプロジェクトの内部事情をインターネット上で公開することはないでしょうから。幸いなことに、第 1 回「Architectural Katas」ライブトレーニングの成果（https://oreil.ly/7NL2O）が公開されています。この内容は将来アーキテクトを目指す方々にとって非常に貴重なリソースとなるでしょう。3〜5 人のグループが、健康食品スタートアップのアーキテクチャを構築し、ADR も作成しました。その成果は GitHub で公開されています。ADR の例として、ArchColider（https://oreil.ly/colarCH）、Miyagi's Little Forests（https://oreil.ly/-l5vz）、Jiakaturi（https://oreil.ly/zv-E5）の 3 つが参考にできます。

ADR には、図やその他の補足資料が含まれることがよくあります。「常識」と思われる部分を省略したくなるかもしれませんが、いずれ誰かがまったく新しい視点で読者の ADR を読むことになるでしょう。もし自分が初めてそのプロジェクトに携わる立場であった場合、その決定内容を理解するために何が必要でしょうか？

ADR はバージョン管理システムに保存されることが多く、ADR 専用のシンプルなツールを構築している組織もあります。Wiki を活用しているところもありますが、どのような方法を選ぶにせよ、検索機能があることを確認してください。例えば、ネットワーク設定に関するあの厄介な問題を解決した ADR がどれだったか、正確に覚えていない可能性が高いからです。迷ったときは、シンプルにしましょう。ソフトウェアプロジェクトはそれだけでも十分に難しいのですから、不必

要に複雑にしてはいけません。

9.10 まとめ

アーキテクチャは、数多くの書籍や動画、ポッドキャストが扱っているほど広範なテーマであり、数えきれないほどの時間をかけて実践する必要がある分野です。多くの開発者がアーキテクトを目指しており、多くの人にとってそれはキャリアの頂点でもあります。しかし、アーキテクトになるということは、フロントエンドのライブラリを選んだり、ホワイトボードに図を描いたりすることよりも、はるかに複雑なものです。

アーキテクチャ設計には、アプリケーションの全体像を把握し、重要な品質特性を特定し、意思決定のトレードオフを次々と判断することが求められます。その一方で、組織内の政治的な駆け引きにも対処し、組織図の上下左右に円滑にコミュニケーションを取らなければなりません。アーキテクトになる道のりは、困難を伴いますが、非常にやりがいのあるものです。もしこの道を進むことを選ぶのであれば、これらの指針が、その道のりを歩むうえでの助けとなることを願っています。

9.11 実践に移す

ニューヨークを訪れた観光客が、バイオリンケースを携えた人にカーネギーホールへの行き方を尋ねると、その音楽家は「練習するんだ」と答えるという古いジョークがあります。アーキテクチャも、これと同じです。上達したいのであれば、多くのシステムを設計する必要があるのです！しかし、キャリアを通じて何百ものシステムに携わる機会を得られるアーキテクトはほとんどいません。そこで Ted Neward はこう問いかけました。「キャリアを通じてアーキテクチャ設計の機会がわずか半ダースにも満たないとしたら、どうやって優れたアーキテクトを育てればよいのでしょうか？」 Ted の鋭い観察眼は、彼にインスピレーションを与え、一連のアーキテクチャ・カタ（https://oreil.ly/4PBOM）を作成するきっかけとなりました。このアーキテクチャ・カタは、さまざまな領域を網羅し、共通の制約条件を持ち、解釈の余地を十分残したものです。

アーキテクチャ・カタは、さまざまな方法で活用できます[†22]。一つを選んで取り組むのもよいでしょう（https://oreil.ly/cPY8w）。ランダムに選んで（https://oreil.ly/8d4kO）、1〜2 時間かけて個人または少人数のグループで取り組むのもよいでしょう。アーキテクチャを構築するために、どのような疑問を解決すべきでしょうか？ そのカタにおいて、最も重要な品質特性は何でしょうか？ アプリケーションで、最も複雑な部分またはリスクの高い部分はどこでしょうか？ アーキテクトとして、最も不安に感じることは何でしょうか？ どのようなアーキテクチャスタイルを提案しますか？

その解決策を数日間、あるいは数週間ほど置いておきましょう。改めて見直してみてください。どこを変更しますか？ 最初に検討した際に見落としていた点はありませんか？ 今なら別のアプ

†22 パーティーゲームとしても楽しめます。少なくとも、参加者次第では。

ローチを提案しますか？ 非公式なアーキテクチャレビューを行ってみましょう。同僚からはどのような質問が出ましたか？ 提案した解決策にはどのような課題がありますか？ 1〜2 か月ごとに、別のカタに取り組むか、以前の解決策をさらに磨き上げていきましょう。

自分自身のアーキテクトとしての腕前を、もっと厳しい環境で試してみたいと思いませんか？ チームを結成し、オライリー学習プラットフォームで開催される Architectural Katas ライブセッション[†23]に登録してください。楽しく安全な環境で、実際の問題に取り組みながら、自身のスキルを磨く機会を得られます。

カタは面接の手段としても有効です。特定の都市圏におけるピアノ調律師の人数について質問したり、コンピュータサイエンスの知識を暗記する能力をテストしたりするのではなく、一緒にカタを実践してみてください。彼らが投げかける質問からは、その人の思考や問題への取り組み方がうかがえます。また、必然的に生まれる協働を通じて、実際のプロジェクトでこの人と一緒に仕事をするのがどのようなものか、十分に把握できるでしょう。

9.12　関連リソース

- 『Thinking Architecturally』Nathaniel Schutta 著（O'Reilly、2018 年）
- 『Head First Software Architecture』Raju Gandhi 他著（O'Reilly、2024 年）
- 『Fundamentals of Software Architecture』Mark Richards, Neal Ford 共著（O'Reilly、2020 年）
  和書『ソフトウェアアーキテクチャの基礎 第 2 版』（オライリー・ジャパン、2026 年）
- 『How to Win Friends and Influence People』Dale Carnegie 著（Simon & Schuster、1936 年）
  和書『人を動かす 新装版』（創元社、1998 年）
- 『Building Evolutionary Architectures, 2nd Edition』Neal Ford 他著（O'Reilly、2022 年）
- "Code as Design"、Jack W. Reeves 著（https://oreil.ly/tNgFe）
- 『Influence, New and Expanded: The Psychology of Persuasion』Robert B. Cialdini 著（Harper Business、2021 年）
  和書『影響力の武器［新版］』（誠信書房、2023 年）
- The C4 model for visualizing software architecture（https://c4model.com/）
- 『Communication Patterns』Jacqui Read 著（O'Reilly、2023 年）
  和書『開発者とアーキテクトのためのコミュニケーションガイド』（オライリー・ジャパン、2025 年）
- 『Creating Software with Modern Diagramming Techniques』Ashley Peacock 著（Pragmatic Programmers、2023 年）

†23 https://learning.oreilly.com/live-events/architectural-katas-q4-2025-ai-enabled-architecture/0642572249625/

- 『UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd Edition』Martin Fowler 著（Addison-Wesley Professional、2003 年）
  和書『UML モデリングのエッセンス 第 3 版』（翔泳社、2005 年）

10章
本番環境へ

うまくいかない可能性があることは、必ずうまくいきません。
— マーフィーの法則

プログラマとしてのキャリアをスタートしたばかりの頃は、本番環境への移行は、少し不安に感じられる一方で、ワクワクするものでもあります。本番環境とは、開発段階のコードが、実際に人々が利用するコードへと移行する場所です。これは、これまでの計画立案や問題解決、そして徹夜作業の集大成でもあります。

本番環境へのデプロイは、SDLC の最終段階の一つです。ここで、アプリケーションは真の試練に直面することになります。つまり、実際のエンドユーザーへのサービス提供、実データの処理、そして負荷の高い環境下での運用が行われるのです。

自分とコードしか存在しない、ローカル開発環境という「安全地帯」を離れ、実世界の人々が自分の仕事を操作する世界に足を踏み入れることになります。問題が発生すれば、それは誰の目にも明らかになります。コードベースを単なるアプリケーションに過ぎないと考えるかもしれません。しかし、ユーザーの手に渡れば、それはビジネス価値を生み出し、世界に有意義な変化をもたらすツールとなるのです。

本章を読むことで、本番環境についてより深く理解し、その重要性を認識できるようになります。また、SDLC の早い段階から、本番環境向けのコードを構築することについて考え始めるようになります。重要なポイントは、開発環境と本番環境の根本的な違いを理解し、そのギャップをうまく埋める方法を学ぶことです。

10.1　本番環境の複雑さ

「ヒューストン、問題が発生しました。」大惨事寸前までいったアポロ 13 号のミッションで発せられた、有名な言葉です。この言葉からわかるように、厳密に管理された環境であっても事態が思わぬ方向に進むことがあります。ソフトウェア開発において、生死にかかわる事態になることはめったにありませんが、異なる環境、特に本番環境への移行時には、予期せぬ問題が発生するおそ

れがあります。

読者のチームが数か月にわたり新製品の開発に取り組んできたと想像してみてください。テストカバレッジは十分であり、QA チームがさまざまな環境で徹底的に検証を行いました。関係するすべてのチームから承認を得て、いよいよリリースできる段階になりました。

必要な手順をすべて完了し、プロジェクトは本番環境への移行を果たしました。最初はトラフィックも順調に流入し、すべて問題がないように見えました。しかし、突然、大混乱が巻き起こります。ユーザーからバグの報告が相次ぎ、予期せぬパフォーマンスの問題が発生し、監視システムとメールボックスはアラートで溢れかえってしまいます。

一体何が起きたのでしょうか？ そのコードは、読者のマシンでは正常に動作していました。テストしたすべての環境でも問題なく動いていたはずです。もしこのような状況を経験したことがあるなら、それは読者だけではありません。これは、本番環境が如何に重要であるかを物語っています。本番環境は、単にコードが最終的に行き着く先というだけではありません。そこは現実の世界であり、実際のユーザーがアプリケーションとやり取りする、究極のテストの場なのです。

ソフトウェアはテスト環境では正常に動作するのかもしれませんが、その真価は、本番環境で実際のユーザーが利用した際のパフォーマンスによって決まります。開発環境と本番環境の違いは、主に 2 つの形で現れます。一つは、ユーザーが予期せぬ方法でアプリケーションを操作してしまうことです。もう一つは、環境によってコードの挙動が異なるという、「私のマシンでは動くのに」という一般的な問題です。ここでは、これらの課題について考察し、この問題を解決するための実践的な戦略を探っていきましょう。

10.1.1　ユーザーは予測不能

開発者は、ユーザーがアプリケーションとどのようにやり取りするかを予測できると思い込んでしまうことがよくあります。ユーザー層について全体像を把握できていない場合もあります。実際に知っている以上に、ユーザーを理解していると思い込んでしまうこともあります。どちらの場合も、開発者とユーザーの間で、アプリケーションに対して期待するものがズレてしまうおそれがあります。

テストやシミュレーションをいくら作成しても、現実の世界でどのように利用されるかが予測不可能なことを再現するのは困難です。このようなテストが、プロジェクトの成功にとって極めて重要なのにもかかわらずです。だからこそ、ユーザーこそが究極のテストとなるのです。

「ユーザーは予測不能」という事例

Dan です。大学を卒業して最初に就いた仕事の一つが、レベル 2 サポートスペシャリストでした。あるユーザーから、「パソコンの『ドリンクホルダー』が故障している」という問い合わせがありました。私は「お客様、パソコンにドリンクホルダーなんてありませんよ」と答えました。ところが、その方が言っていたのは CD-ROM ドライブのことだったのです。こ

の出来事は、ユーザーが CD-ROM ドライブをドリンクホルダーと間違えるような事態に対しては、どんなにテストを重ねても備えることはできない、ということを物語っています。

以下に示すのは、ユーザーの行動が予測しにくい要因です。

実世界データ

どんなに綿密に設計されたテストデータであっても、ユーザーがフォームに対してどのようにやり取りするかについて、すべてのやり取りを予測できるわけではありません。ユーザーは、まるでハリウッドの脚本の中でしかありえないような方法でデータを入力してくるのです。

スケーラビリティと並行処理

実際のトラフィックでは、テストデータではわからなかったスケーラビリティの問題が露呈することがあります。

多様な環境

実際のユーザーは、さまざまな環境で利用しています。これには、OS の種類、システムリソース、セキュリティ設定、ブラウザのベンダーやバージョン、ネットワークの遅延などが含まれます。

環境の違い

環境によってソフトウェアやライブラリのバージョンが異なる場合、テスト時には発生しなかった不整合が生じるおそれがあります。

予想外の利用事例

ユーザーは、開発者が想定していなかった方法でソフトウェアを操作することがあります。その結果、エッジケースや予期せぬシナリオが明らかになることがあります。

実世界の不確実性に備えるため、カナリアリリース、A/B テストといった手法を使います。さらに、ロギング、メトリクス、分散トレーシングを組み合わせて、しっかりとしたオブザーバビリティを構築することも重要です。これらの手法は、リスクを最小限に抑えつつ、実環境でユーザーテストを行うことの利点を最大限に活用するものです。

10.1.2 「私のマシンでは動くのに」

本番環境で新たな不具合が報告されたという通知が届きます。内容を確認すると、それは自分が担当した機能であることがわかりました。ノート PC を開いて再現手順を確認し、不具合を再現できるかどうか試してみましたが、当然ながら再現できませんでした。自分の環境では正常に動作するので、ユーザーの操作ミスに違いありません。「再現不可」、開発者がこうした状況で課題をクローズする際に使う決まり文句です。

ソフトウェア開発において、「私のマシンでは動く」とは、開発者が自身のコンピュータ上ではコードが正常に動作するものの、他の環境にデプロイすると失敗してしまう場合に使う言葉です。「私のマシンでは動く」という言い訳は、バグ報告を素早く解決するのには役立つかもしれませんが、エンドユーザーの問題を解決するものではありません。開発環境と本番環境の間に生じている差異を特定する必要があります。

環境間によって異なる可能性があるものは以下の通りです。

環境の違い

例えば、16GB の RAM を搭載した MacBook Pro で開発を行い、本番環境はメモリの制約が異なる Linux サーバーの場合を想像してみてください。ローカルの MySQL データベースはバージョン 8.0 であるのに、本番環境では PostgreSQL 14 が使用されているかもしれません。OS、利用可能なリソース、ネットワーク設定、データベースベンダー、依存関係のバージョンといった違いにより、自分のマシンでは問題なく動作していたコードが、本番環境ではまったく機能しなくなるおそれがあります。

構成管理

開発段階では、アプリケーションが接続しているのは、自分のノート PC 上で動作しているローカルデータベースかもしれません。また、デバッグのために、あらゆるログを記録したり、架空のクレジットカード番号を使用する決済プロバイダーのテスト API にリクエストを送信したりしているでしょう。しかし、その同じコードを本番環境にデプロイする際には、自動的にセキュアなクラウドデータベースに切り替え、ログ記録をエラーの捕捉のみに絞り込み、実際の料金を請求する本物の決済処理システムに接続するようにしなければなりません。

並行処理と負荷

次のような状況を想像してみてください。ショッピングカート機能をローカル環境でテストする際、「カートに入れる」ボタンを数回クリックして、正常に動作することを確認していました。しかし、ブラックフライデーになると、何千人もの顧客が同時にその同じボタンをクリックします。すると、単独でのテスト時には余裕で処理できていたデータベースが、リクエストの洪水に押し流されてしまいます。使用率が 30% 程度で順調に推移していたサーバーのメモリ使用率は、一度に数百ものユーザーセッションを処理しようとして、95% まで急上昇します。これこそが、開発環境と本番環境の間の現実的なギャップなのです。

データの多様性

ローカルのテストデータベースには、おそらく自分で作成した、整然とした形式のレコードが数件あることでしょう。「John Smith」という人物の電話番号が「555-123-4567」のように完璧で、メールアドレスの形式も整っていることでしょう。しかし、本番環境では事情が異なります。実際の顧客データは、ばらばらで予測不可能です。レガシーシステムではデー

タの保存方法が現在とは異なっていたため、現在の検証ロジックでは正しく検証できない場合があります。さらに、膨大なデータ量という問題もあります。テストクエリは数百件のレコードに対しては瞬時に実行されますが、本番環境には長年にわたる事業活動で蓄積された数百万人の顧客データが存在します。だからこそ、現実的で多様なデータセットを用いたテストが重要になるのです。本番環境のデータは、常に予想外の事態を引き起こすものです。

外部依存関係

モダンアプリケーションはチームスポーツのようなものです。一人の選手だけで試合に勝つことはできません。アプリケーションがユーザーインターフェースを処理する一方で、決済処理は決済システムに委ねられます。また、メール配信のために外部システムにリクエストを送信することもあります。ユーザー認証は、Google に依存しているかもしれません。しかし本番環境では、「チームメンバー」の誰かが調子を崩すことがあります。例えば、決済処理システムが遅くなったり、メールサービスがレート制限に達したり、認証プロバイダーが停止したりするような状況です。そのような場合は、どれほどまとまりの良いチーム（アプリケーション）でも試合に負けてしまうおそれがあります。

セキュリティ上の制約

本番環境は、ローカルマシンとは異なり、厳重に保護されています。自分のノート PC ならどこから送られてきた HTTP リクエストでも問題なく受け入れるかもしれません。これに対して、本番サーバーは予期しないトラフィックをブロックするファイアウォールの内側に配置されています。ローカル環境での API 呼び出しはプレーン HTTP で問題なく動作しますが、本番環境では有効な TLS 証明書を備えた HTTPS が必須となります。ローカルでテストしているフロントエンドはバックエンドに自由にアクセスできますが、本番環境では CORS ポリシーが適用され、同じリクエストがブロックされるおそれがあります。これらのセキュリティレイヤーはユーザーデータや金銭を保護してくれるものです。しかし、開発マシンでは完璧に動作するコードであっても、セキュリティレイヤーのために本番環境では動作しないことがあります。

「私のマシンでは動く」と「本番環境でも動く」というギャップを埋めるために、以下の点を考慮してください。

コンテナ化

Docker などのツールを活用して、開発環境と本番環境の間で一貫性のある環境を構築し、「私のマシンでは動くのに」といった事態を解消します。

オブザーバビリティ

包括的なロギング、メトリクス、分散トレーシングを導入し、本番環境でのアプリケーションの動作を把握し、ユーザーに影響が出る前に問題を特定します。

環境の均一性

本番環境と非常に近いステージング環境を用意することで、環境固有の問題を早期に発見できるようにします。

継続的インテグレーションと継続的デプロイ（CI/CD）

CI/CDを実践し、本番環境に近い条件下でコードが定期的にテストされるようにします。CI/CDについては、本章において後ほど説明します。

覚えておいてください。ユーザーにとって重要なのは、読者のマシンで動作するかどうかではありません。ユーザーのマシンで動作するかどうかです。これが本番環境特有の複雑さであり、テスト環境と実際のユーザーの予測不可能な振る舞いとのギャップを埋めるためのヒントです。次の節では、開発ライフサイクル全体を通じて、これらの問題について考慮していくことで、それらをどのように回避できるかについて学びます。

10.2　本番環境向けのコードの作成

子供が初めての登校日を迎える親御さんなら、考えなければならないことが山ほどあることでしょう。ただ子供を起こして、リュックサックを渡して、ドアの外へ押し出すだけ、なんてことはしないでしょう？ もちろん、そんなことはありません。しっかり食事をさせ、適切な服装を着せ、必要な持ち物をすべて揃え、万が一のときにどう連絡すればよいかを教えるでしょう。これから子供が直面する世界に向けて、しっかりと準備させるはずです。

本番環境へのデプロイに向けてコードを準備する場合も、同じことが言えます。コードは開発者にとって大切な「我が子」のようなものです。実環境における予期せぬ課題に確実に対応できるよう、万全の準備を整える必要があります。しかし、親が子供に起こりうるあらゆる状況を予測できないのと同様に、開発者も本番環境で発生しうるすべての問題を予見することはできません。それでも、開発プロセス全体を通じて、パフォーマンスの最適化、環境固有の設定、エラー処理やロギング、セキュリティ上の必須事項を考慮して、適切な準備を行うことで、コードが成功を収める可能性を最大限に高められます。

10.2.1　パフォーマンスの最適化

パフォーマンスの最適化は、単に動作をスムーズにしたいときに「ダイヤルを回す」ようなものではありません。コードを書く前に身につけておくべきことであり、開発プロセスを通じて常に見直すべきアプリケーションの重要な要素です。

パフォーマンス向上とは、確かにコードの実行速度を上げることも含まれますが、最終的にはユーザーに最高の体験を提供することにあります。パフォーマンスを向上させるためには、いくつかの戦略を活用できます。これらは「構成要素」と考えてください。すべてを一度に実装する必要はなく、開発プロセスに段階的に取り入れていきましょう。

非同期プログラミング

非同期プログラミングでは、タスクが完了するのを待ってから次のタスクに進む（同期実行）のではなく、タスクをバックグラウンドで実行し、完了時にメインプログラムに通知するようにできます。これは一般的な解決策ですが、非同期プログラミング自体に複雑さやトレードオフが生じるため、慎重に使用する必要があります。

ネットワーク呼び出しの削減

アプリケーションが行うリクエストには、すべてコストが伴います。そのコストとは、時間とリソースのことです。小さなリクエストを何度も行うのではなく、それらをまとめて1つのリクエストにすることを検討してください。

キャッシュ戦略

頻繁にアクセスされ、内容が変化しないデータを特定できれば、それをメモリや何らかの高速ストレージシステムに格納できます。これにより、データベースへの負荷が軽減され、応答時間が向上します。キャッシュのトレードオフについての詳細は、8章を確認してください。

データベースクエリの最適化

効率的なクエリを使ったり、適切なインデックス設定を行ったり、N+1クエリの問題を回避したりしてください。

コードミニフィケーション

モダンなフロントエンド開発において、コードミニフィケーションとは、機能を損なうことなくコードから不要な文字を削除することです。これは、本からすべてのスペースと改行を削除しつつ、単語はすべて残すようなものだと考えてください。意味は変わらないまま、スペースを節約できるのです。ミニフィケーションでは、空白や改行、コメントなどの文字を削除することで、ファイルサイズを小さくし、ダウンロードを高速化します。

コードバンドリング

バンドリングとは、複数の個別のコードファイルを、大きなファイルにまとめてファイル数を少なくすることです。これにより、ブラウザは20個のJavaScriptファイルをダウンロードする代わりに、バンドルされた数個のファイルをダウンロードすることになります。WebpackやViteなどのモダンビルドツールでは、バンドリングに加えて、ツリーシェーキング（未使用コードの削除）やミニフィケーションなど他の最適化処理も行えます。

遅延読み込み

リソース（画像、コードスニペット、コンポーネントなど）を、最初から読み込むのではなく、実際に必要になったときに読み込みます。これにより、ページの初期読み込み時間が短縮されます。

コンテンツ配信ネットワーク（CDN、Content Delivery Network）

CDN は、ユーザーから物理的に近い場所からアセットを配信することで、ミニフィケーションやバンドリングなどの最適化を補完し、読み込み時間を大幅に短縮できます。

パフォーマンスの最適化とは、単にコードの実行速度を上げるだけでなく、ユーザーに最高の体験を提供することです。そのためには、考え方を変え、この節で学んだ最適化の手法について検討してみてください。

10.2.2　環境固有の設定

「設定はコードである」という言葉を耳にすることがあるかもしれません。しかし、コードとは異なり、設定は環境によって異なります。設定は、照明条件に合わせてスマートフォンのカメラ設定を調整することに似ています。明るい日差しに合わせて最適化された設定が、暗い場所ではうまく機能しないのと同じように、アプリケーションも、ある特定の環境で確実に動作するためには、その環境に合わせた設定が必要です。

よく見かけるアンチパターンとその解決策を見てみましょう。次の例では、開発者がローカルマシンのデータベースの認証情報をハードコーディングしています。このように認証情報をハードコーディングするのではなく、環境ごとに設定できるようにしましょう。

```
// こういうことは行わないように
public class DbConnection {
   private final String url = "jdbc:postgresql://localhost:5432/fose";
   private final String username = "admin";
   private final String password = "secret";
}

// 代わりにこのようにする
public class DbConnection {
   private final String url;
   private final String username;
   private final String password;

   public DbConnection(AppConfiguration config) {
       this.url = config.getDatabaseUrl();
       this.username = config.getDatabaseUsername();
       this.password = config.getDatabasePassword();
   }
}
```

先ほど、開発環境固有のデータベース接続文字列をハードコーディングした例を紹介しました。コードを別の環境に移行する際、この方法では対応が難しくなります。環境によって変更される可能性のある設定の例を、いくつか紹介します。

- ロギング設定

- 外部サービスのエンドポイント
- データベース設定
- キャッシュ設定
- メール設定
- セキュリティ設定
- フィーチャーフラグとアプリの動作

この点について理解できたところで、環境をまたいでこれらの項目を設定する方法について見ていきましょう。

10.2.2.1 設定ファイル

設定ファイルを使用すれば、データベースの接続文字列から API キー、アプリケーション固有の設定に至るまで、あらゆる設定を行えます。これらはコードではありますが、多くの場合、人間が読みやすい形式になっているため、非常に便利です。また、設定ファイルを使えば、デプロイ先の環境ごとに個別のファイルを用意することもできます。

次の例では、開発用と本番用の 2 つの設定ファイルを定義しています。これにより、環境ごとにプロパティを定義できます。

```
# application.yaml（開発用）
services:
 service1:
   url: http://localhost:8080
   api-key: 123456
database:
 url: jdbc:postgresql://localhost:5432/fose
 username: postgres
 password: postgres
logging:
 level: INFO

# application-prod.yaml
services:
 service1:
   url: http://real-service1.com
   api-key: 789012
database:
 url: jdbc:postgresql://fosedb.com:5432/fose
 username: postgres
 password: f@3ser#4il_m$%@$LW0
logging:
 level: ERROR
```

10.2.2.2 環境変数

前節の設定ファイルを注意深く見た方なら、そこに機密情報が含まれていることに気づかれたか

もしれません。これらの設定ファイルは特定の環境向けのものであっても、やはり問題を引き起こすおそれがあります。まず、バージョン管理システムにチェックインすると、機密情報を公開してしまうという問題があります。もう一つの問題は、API キーのような機密情報は通常、定期的に更新されるということです。つまり、API キーを変更するにはアプリケーション全体を再デプロイする必要があり、これは理想的ではありません。次の例では、機密情報を環境変数を使って定義しており、ハードコーディングしていません。

```
# application-prod.yaml
services:
 service1:
   url: ${SERVICE1_URL}
   api-key: ${SERVICE1_API_KEY}
```

10.2.2.3　フィーチャーフラグ

フィーチャーフラグ（**フィーチャートグル**とも呼ばれます）は、開発者が新しいコードをデプロイすることなく、実行時に機能を有効または無効にできるソフトウェア開発の手法です。これは、アプリケーションの機能に対するコントロールパネルのようなものと考えてください。

フィーチャーフラグには、開発チームとユーザー双方にとって、いくつかの利点があります。ユーザーに対して機能を隠したまま本番環境にコードをデプロイできるため、より安全にリリースでき、コードのリリースと機能のリリースを明確に分離できます。また、ユーザーグループごとに段階的な展開が可能になり、全ユーザーにデプロイする前に、一部のユーザーを対象に機能をテストできます。さらに、フィーチャーフラグを使用すれば、異なるユーザー体験を比較できるため A/B テストが容易になります。コードのデプロイを必要とせずに機能を迅速にロールバックすることも可能です。

よく使われるフィーチャーフラグには、主に 3 つの種類があります。**リリースフラグ**は、開発中の機能を本番環境から隠し、作業を継続しながら main ブランチをデプロイ可能な状態に保つのに役立ちます。**実験フラグ**は、ユーザー体験を比較したり、ユーザーの行動に関するデータを収集したりするために使われます。**許可フラグ**は、ユーザーのロールやサブスクリプションレベルに基づいて機能へのアクセスを制御し、サービス階層化されたプロダクトを提供する際に便利です。



フィーチャーフラグは定期的に見直し、整理する必要があります。一時的なフラグが恒久的なものになってしまうと、コードベースが不必要に複雑になります。チームに合った整理スケジュールを策定し、新しいフィーチャーフラグを作成する際には有効期限を設定してください。また、各フラグの目的と、想定される有効期間を明確に記したドキュメントを作成してください。

10.2.2.4　シークレット

シークレットとは、API キー、パスワード、トークンなど、特別な注意を要する機密性の高い設

定値のことです。シークレットを適切に管理することは、アプリケーションのセキュリティにとって極めて重要です。いくつかのベストプラクティスに従う必要があります。

決して、シークレットをコード内に直接記述したり、バージョン管理システムにコミットしたりしないでください。このようなセキュリティ上の規則違反は、金銭的な損失を招き、ユーザーの信頼を損なうおそれがあります。万が一、これらをバージョン管理システムにコミットしてしまった場合、Git の履歴を書き換える必要が生じます。そして、これは非常に複雑な作業となる可能性があります。さらに、コードベースにシークレットが埋め込まれていると、第三者とソースコードを共有する際に重大なセキュリティリスクとなります。

その代わりに、AWS Secrets Manager、HashiCorp Vault、Azure Key Vault などの専用のシークレット管理サービスを利用してください。これらのサービスは、機密データのために特別に設計された安全な保存機能、暗号化、アクセス制御を提供します。認証情報は定期的に更新してください。可能であれば、シークレット管理プラットフォームを通じて自動化します。このようにすれば、万一認証情報が漏洩した場合でも、脆弱性が生じる期間を最小限に抑えることができます。

本番環境のシークレットへのアクセスは厳重に管理し、どうしても必要なチームメンバーに限定してください。「最小権限の原則」を適用し、認証および認可を適切に制御してください。最後に、アプリケーションがデプロイされる環境ごとに、別々のシークレットを使うようにします。開発環境、ステージング環境、本番環境で絶対に同じ認証情報を共有してはいけません。環境ごとにシークレットを変更しておくことで、開発やテスト中に本番環境が誤って公開されるのを防ぐことができます。

このように、複数の環境向けにアプリケーションを設定する際には、留意すべき点が数多く存在します。本番環境向けのコードを構築するうえで次に考慮すべき点は、問題が発生した際にそれを捕捉できるよう、適切なエラー処理とロギングを導入することです。

10.2.3　エラー処理とロギング

これまで学んできたように、本番環境は予測不可能です。マーフィーの法則が示すように、何かがうまくいかない可能性があるなら、それは必ずうまくいかなくなるものです。このような厳しい環境において、適切なエラー処理とロギングは単なるベストプラクティスにとどまらず、本番アプリケーションにとって不可欠な「生き残るためのスキル」です。問題がシステム全体の障害へと波及する前に早期に発見できます。また、問題を解決する際に、極めて重要な情報を得ることができます。

新しい機能の開発を始める際、自然と「ハッピーパス」に注力してしまうものです。これは、すべてが完璧に機能する、ユーザーにとっての理想的なシナリオを指します。しかし、ソフトウェア開発の現実においては、これは考慮すべき事項のごく一部に過ぎません。パレートの法則に従えば、ハッピーパスはコード全体の 20% を占めるに過ぎません。エッジケースやエラーへの対応には、多くの場合、時間と労力の 80% が費やされます。開発時間の大部分を費やすのはこの部分であり、経験豊富な開発者が真価を発揮する場でもあります。

10.2.3.1　エラー処理

では、実際に効果的なエラー処理とはどのようなものなのでしょうか？ ここでは、日常的に活用できる具体的な対策を紹介します。

グレースフルデグラデーション

機能が利用できない場合は、完全に利用不能にするのではなく、簡易版を提供してください。例えば、天気 API がダウンしている場合は、エラーページを表示するのではなく、情報が古くなっている可能性があるという注意書きを添えて、キャッシュされた天気データを表示するようにしてください。

わかりやすいエラーメッセージ

専門用語を、ユーザーにとってわかりやすい言葉に置き換えてください。「データベースへの接続に失敗しました」という表示の代わりに、「現在、データの読み込みに問題が発生しています。数分後に再度お試しください」とユーザーに伝えてください。

デバッグ用のロギング

後で修正できるよう、内部で何が問題だったかを記録しておきましょう。開発者にとって役に立つ技術的な詳細（エラーコードやスタックトレースなど）をログに残しつつ、ユーザーには必要な情報のみを表示するようにしてください。

代替手段

常に代替案を用意しておきましょう。例えば、決済処理中にメインの決済サービスに不具合が生じた場合、自動的に予備の決済サービスに切り替えることで、顧客が購入を完了できるようにします。

10.2.3.2　ロギング

効果的なロギングは、本番環境の問題を把握する手助けとなります。課題は適切なバランスを見極めることにあります。ロギングが不十分だと状況が把握できず、逆に過剰にロギングするとノイズを生み出し、重要な情報を覆い隠してしまうおそれがあります。

プログラミング言語によって実装は異なる場合がありますが、ロギングレベルの概念は共通しています。以下に、Java のロギングレベルを紹介します。ログメッセージの深刻度や重要度に基づいて分類・フィルタリングを行うのに役立つでしょう。

ERROR

直ちに対応が必要な重大な問題の場合に使用する

WARN

問題が発生するおそれのある状況に対して

INFO

通常のビジネスプロセスにおけるイベント

DEBUG

デバッグに関する詳細情報

ログを記録する際は、あまり詳細になりすぎないようにしましょう。重要な項目に焦点を当ててください。以下の項目はログに記録しないでください。

- パスワード、API キー、認証トークン、シークレットなどの機密情報。
- 個人識別情報（PII、Personally Identifiable Information）。本章で後ほど詳しく説明します。
- 大きなオブジェクトやレスポンスペイロード全体。その代わりに、問題の特定に役立つ重要な情報をログに記録してください。
- アプリケーションの複数レイヤーにわたって、同じ情報を重複してログに記録しないでください。

先ほど学んだように、本番環境で問題が発生した際に状況を把握するためには、エラー処理とロギングが不可欠です。本番環境向けのコードを構築するうえで、もう一つ重要な点は、そのコードが安全であることを確認することです。

10.2.4　セキュリティの基本

セキュリティは、一度考えればそれで終わりというものではありません。絶えず警戒心を持ち、定期的に注意を払う必要があります。アプリケーションを本番環境にデプロイする際は、セキュリティは開発プロセスの最初から組み込むべきものであり、最後に後付けするものではないということを忘れないでください。セキュリティの専門家である必要はありません。しかし、セキュリティの基本を理解することで、より安全なアプリケーションを構築できるようになり、チームにとってより価値のあるメンバーになれるでしょう。Web アプリケーションのセキュリティについて学ぶうえで、非常に優れたリソースの一つが OWASP（Open Worldwide Application Security Project）です。OWASP は、無料で実践的なセキュリティガイダンスを提供しています。世界中の開発者から広く信頼されています。本節では、アプリケーションのセキュリティを強化するための方法をいくつか学んでいきます。

10.2.4.1　HTTPS による通信の保護

ローカル環境で開発を行う際は、率直に言って便利だからという理由で、HTTP（Hypertext Transfer Protocol）を使いたくなるかもしれません。しかし、本番環境にデプロイする前には、HTTPS（HTTP Secure）でのテストを行うことが重要です。ローカル開発用に信頼できる証明書を作成できるツールも存在します。

誰に見られても問題のないメッセージもあれば、受信者だけにしか見られたくないメッセージもあります。休暇中に誰かに絵葉書を送る場面を想像してみてください。その絵葉書の表にはビーチの写真、裏には誰でも読める手書きのメッセージが書かれています。これが、HTTP 経由でデータを送信する際の状況です。HTTP は、ブラウザとサーバーの間で平文の（暗号化されていない）データを転送します。パスワード、クレジットカード番号、秘密情報、個人データなどの機密情報を、通信を傍受した人なら誰でも閲覧できてしまいます。

休暇中のメッセージを他人に知られたくない場合は、封をした封筒に入れるでしょう。HTTPS はまさにそれと同じ役割を果たします。HTTPS は、TLS（Transport Layer Security）プロトコルを使用して暗号化レイヤーを追加します。

TLS 証明書は、Web サイトにとってのデジタル ID のようなものです。HTTPS サイトにアクセスすると、サーバーはその身元を証明するために証明書を提示します。これらの証明書は、Web サイトの所有者の身元を保証します。証明書は信頼できる認証局（CA、Certificate Authorities）によって発行されます。

ブラウザが証明書を確認すると、TLS はこの信頼された接続を利用して、すべてのデータ転送に対して強力な暗号化を設定します。この暗号化により、第三者に情報を読み取られることを防ぐだけでなく、ブラウザとサーバー間の転送中に情報が改ざんされることもありません。



SSL（Secure Sockets Layer）は、TLS に取って代わられた古いプロトコルですが、今でも多くの方が習慣から「SSL」という用語を使用しています。SSL には既知のセキュリティ上の脆弱性があるため、本番環境で使用すべきではありません。今日、誰かが SSL について言及する場合、通常は TLS を意味していますが、システムが実際に TLS プロトコルを使用しているかどうかを確認することが重要です。

HTTPS だけでセキュリティ上の問題がすべて解決するわけではありません。これは、アプリケーションを本番環境に移行する際、考慮すべきセキュリティ対策の一つです。以下に、実践すべきベストプラクティスをいくつか紹介します。

- ログインページだけでなく、すべてのページで HTTPS を使用してください。
- HTTP から HTTPS へ自動的にリダイレクトするように設定してください。
- 証明書を常に最新の状態に保ってください（証明書管理ツールによる自動化を検討してください）。
- 安全な TLS バージョン（1.2 以降）を使用してください。

10.2.4.2　認証に関するベストプラクティス

セキュリティ侵害は、機密データの漏洩、顧客信頼の失墜など、組織に対して多くの問題を引き起こすおそれがあります。さらには重大な金銭的損失や評判の低下につながる場合があります。適切な認証は、こうした脅威に対する有効な防御策となります。

認証とは、ユーザーが自称する人物であることを確認するプロセスです。一方、**認可**とは、認証済みのユーザーがどのような操作を行ってよいのかを決定するものです。最も重要なルールは、いかなる状況下でも、独自のセキュリティ対策を絶対に作成しないということです。独自の認証システムを構築することは簡単そうに見えるかもしれませんが、セキュリティの専門家たちは、何十年もの歳月を費やして、一見してわかりにくく複雑な脆弱性を特定し、その対策を講じてきているのです。

Java の世界における Spring Security は、20 年以上にわたるセキュリティ開発の成果を体現しています。パスワードの保存など、一般的なセキュリティ上の課題に対し、実績のあるソリューションを提供します。平文でパスワードを保存するという危険な処理を避けるために、パスワードエンコーダーが提供されます。次の例では、パスワードのエンコードに使用する`BCryptPasswordEncoder`の新しいインスタンスを作成しています。

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

そして、このパスワードエンコーダーを使用して、次に示すように、パスワードを暗号化後、データベースに保存してください。

```
// 新しいユーザーエンティティを作成する
User user = new User();
user.setUsername(registrationDto.getUsername());
user.setEmail(registrationDto.getEmail());

// 保存する前にパスワードを暗号化する
user.setPassword(passwordEncoder.encode(registrationDto.getPassword()));
// 新しいユーザーを保存し戻る
return userRepository.save(user);
```

認証はアプリケーションのセキュリティにおいて不可欠な要素であり、不正アクセスに対する最初の防御線としての役割を果たします。認証システムのコンポーネントについて見ていきましょう。

システムを不正アクセスから保護するには、**強固なパスワード管理**が不可欠です。例えば、大文字、小文字、数字、特殊文字を組み合わせた、12 文字以上のパスワードを義務付けます。一般的なパスワードや漏洩したパスワードのデータベースと照合し、ユーザーが以前のパスワードを再利用できないようにします。また、パスワードごとに固有のソルトを使用したモダンなハッシュアルゴリズムを採用します。**ソルト**とは、レインボーテーブル攻撃を防ぐために、ハッシュ化の前に追加されるランダムなデータのことです。自動入力機能を備えたパスワードマネージャーの利用を推奨しサポートします。

こうした技術的なセキュリティ対策をすべて講じていても、パスワードの安全性は結局のところ人間の行動にかかっているということを忘れないでください。たとえ極めて厳格なパスワードポリシーを導入しても、ユーザーがパスワードを付箋に書き留めたり、同僚と共有したりしてしまえば、何の役にも立ちません。だからこそ、多層的なセキュリティ対策（後述する多要素認証など）を導入することが非常に重要なのです。

ユーザーアカウントの保護には、不正アクセスに対する多層的な防御策を講じる必要があります。例えば、ログイン失敗時の「指数関数バックオフ」を実装します。最初は5秒程度の短い遅延から始め、徐々に30分以上に増やしていきます。同一IPアドレスからの複数アカウントにわたるログイン失敗を追跡し、単一のアカウントが複数の場所から標的にされている場合は監視してください。自動攻撃を防ぐため、数回失敗した後にCAPTCHAを追加します。新しいデバイスや不審な場所からのログイン試行があった場合はユーザーに通知します。最後に、適切に本人を確認する手順を備えた安全なアカウント復旧フローを実装し、攻撃者の侵入を防ぐとともに、正当なユーザーがアクセスを回復できるようにします。

セッション管理は、ユーザーがシステムを利用する際の安全を確保し、アクティブなセッションへの不正アクセスを防ぎます。セッションが容易に推測されたり乗っ取られたりしないよう、十分なエントロピーを持つ暗号的に安全なトークンを使用してください。アイドルタイムアウトと絶対的なセッションタイムアウトの両方を実装します。例えば、30分間操作がない場合や合計8時間経過した後に自動的にユーザーをログアウトさせます。銀行取引などの機密性の高い操作については、より短いタイムアウトを設定します。侵害されたセッションの再利用を防ぐため、失効したトークンのブロックリストを、その有効期限が切れるまで保持してください。

認証段階を追加することで、パスワードだけでは不十分な認証セキュリティを強化できます。認証アプリ、セキュリティキー、生体認証など、複数の認証方法をサポートし、ユーザーの状況に合わせて適切な選択肢を提供しましょう。SMSやメールは便利で広く利用されていますが、SIMスワッピングなどの攻撃に対しては脆弱である点に注意してください。

不審な動作があれば、追加の認証を実行するのもよいでしょう。例えば、ユーザーが新しい場所やデバイスからログインする際に、追加の認証を要求するようにします。攻撃者がセキュリティを迂回できないよう、アカウントの復旧プロセスにはMFA（多要素認証）を必須とします。主となるMFA方法が利用できない場合でも、ユーザーがアカウントにアクセスできなくなることがないように、バックアップコードを使えるようにします。ユーザーが普段使用しているデバイスで毎回認証を求められることがないように、適切なセキュリティ対策を実施したうえで、信頼できるデバイスに対して一定期間認証を免除する設定を検討してください。

10.2.4.3　ユーザーデータの保護

ユーザーは読者に自らのデータを預けてくれています。その信頼を守ることは、プロフェッショナルとして、そして倫理的にも責任を負う必要があります。ユーザーのプライバシーとデータのセキュリティを尊重することは、法的要件があるからではなく、当然の責務です。開発者としての読者の判断は、人々の生活や経済的な安定、そして身の安全に真に影響を及ぼすかもしれないのです。

ユーザーデータを扱うアプリケーションの開発を始めると、おそらく**個人識別情報（PII、personally identifiable information）**という用語を見かけることになるでしょう。この種のデータには、氏名や社会保障番号といった明らかに個人を識別できるような情報だけでなく、IP アドレスやデバイス識別子など、一見してそうとはわからない情報も含まれます。また、他のデータと組み合わせることで、PII と見なされる情報もあります。

開発の段階でこの問題について検討する際は、データを機微情報としてマークすることから始めてください。以下の Java で書かれた例では、カスタムアノテーションを使用して、データを PII としてマークしています。このカスタムアノテーション内では、さまざまなマスキング手法を使用できます。

```
public class UserProfile {
   private String userId;
   private String hashedPassword;
   @Sensitive  // PII をマークするためのカスタムアノテーション
   private String email;
   @Sensitive
   private String phoneNumber;
   @Sensitive
   private LocalDate dob;
}
```

10.2.4.4　暗号化：最後の防衛線

金融情報（クレジットカード番号、銀行口座番号、確定申告書など）のような機密データをデータベースに保存する必要がある場合、どうすればよいでしょうか？ こうした情報は、絶対に平文で保存してはいけません。開発段階から暗号化について検討し、それをセーフティネットと位置づけるべきです。これを安全な金庫だと考えてみてください。たとえ誰かが侵入したとしても、鍵がなければ中にあるものを悪用することはできないのです。

以下のクラスは、Java で機密データを暗号化するためのメソッドを提供します。このクラスは、高度に安全な暗号化アルゴリズムである AES（Advanced Encryption Standard）を使用しています。AES は、機密性とデータの完全性の両方を保護します。AES を利用する際には、鍵の長さを選択する必要があります。現在、ほとんどのアプリケーションでは 128 ビットで十分なセキュリティが確保されています。しかし、計算能力が向上する未来に備えて、多くの開発者が 256 ビット鍵への移行を進めています。

```
class DataEncryptionService {
  private final SecretKey key;
  DataEncryptionService(SecretKey key) { this.key = key; }

  String encrypt(String s) throws GeneralSecurityException {
    byte[] iv = new byte[12]; new SecureRandom().nextBytes(iv);
    Cipher c = Cipher.getInstance("AES/GCM/NoPadding");
```

```
    c.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
    byte[] ct = c.doFinal(s.getBytes(StandardCharsets.UTF_8));
    byte[] out = ByteBuffer.allocate(iv.length + ct.length)
                .put(iv).put(ct).array();
    return Base64.getEncoder().encodeToString(out); // 「IV+ 暗号文」をエンコードする
  }
}
```

10.2.4.5　コンプライアンス要件

ソフトウェアのコンプライアンスとは、特定のルール、基準、規制に従うことを意味します。これには、社内ポリシーと法規制の両方が含まれます。コードを書く際は、以下のコンプライアンスに関する慣行を念頭に置いてください。

- 「プライバシー・バイ・デザイン」を基本として、システムの初期段階からデータ保護の仕組みを組み込んでください。
- 日々の業務において、セキュアコーディングの標準を遵守してください。
- 転送中および保存中の機密データに対して、適切なデータ暗号化を実施してください。
  - 転送中：すべてのデータ転送に HTTPS を使用してください（これにより、ユーザーとサーバー間で転送中のデータを保護できます）。
  - 保存中：データベースに保存されている機密データを暗号化してください（これにより、たとえ誰かがデータベースへのアクセス権を得た場合でもデータを保護できます）。
  - サーバーのメモリ内でデータを復号する際は、特に注意を払ってください。熟練した攻撃者が、そこからデータにアクセスする方法を見つけ出すおそれがあるためです。
- データの削除およびデータエクスポートに関する要求に対応する際の明確な手順を策定してください。
- 機密情報に関するすべてのやり取りを記録してください。

すべての法規制を覚えておく必要はありません。しかし、読者のコードが法規制の対象となるデータを扱う可能性がある場合には、その点を認識し、チームメンバーに助けを求めてください。セキュリティはチーム一丸となって取り組むものです。開発者自身でセキュリティに関するベストプラクティスを実装する一方で、各プロジェクトの初期段階から、組織のセキュリティ専門家が設計の意思決定に関与すべきです。開発者として、セキュリティリスクを引き起こしかねない状況に直面していることに常に気づけるとは限りませんが、それは仕方のないことです。

以下に、プライバシー関連の法規制の主なものについて概要をまとめました。開発者なら、このような法規制について知っておくべきです。

EU 一般データ保護規則（GDPR、General Data Protection Regulation）およびカリフォルニア州消費者プライバシー法（CCPA、California Consumer Privacy Act）

ユーザーデータの取り扱い方法に影響を与える重要なプライバシー規制です。これらの規制

では、基本的に以下のことが求められています。

- データ収集に関するユーザーの明確な同意
- ユーザーデータの削除機能（「忘れられる権利」）
- データの最小化（必要なもののみを収集する）
- 適切なデータの取り扱いとセキュリティ対策

医療保険の携行性と責任に関する法律（HIPAA、Health Insurance Portability and Accountability Act）

アプリケーションで健康情報（医療記録、保険データ、フィットネスデータなど）を扱う場合、HIPAA が適用されます。開発者が留意すべき主な点は以下のとおりです。

- 保存時および転送時のすべての健康データを暗号化してください。
- 厳格なアクセス制御を実施してください。ユーザーには、自身の健康情報のみを表示するようにしてください。
- 誰が、いつ、どの健康データにアクセスしたかについて、詳細な監査ログを保持してください。
- 連携するサードパーティのサービスも、HIPAA に準拠していることを確認してください。
- アプリケーションの目的上必要な期間を超えて、健康情報を保存しないでください。

PCI（Payment Card Industry）

アプリケーションでクレジットカード情報を扱う場合、PCI DSS（Payment Card Industry Data Security Standard）の要件に従う必要があります。規格全体は複雑ですが、開発者として留意すべき重要な点は以下のとおりです。

- 機密性の高い認証情報（CVV コードなど）は絶対に保存しないでください。
- 転送時および保存時にはカード会員データを暗号化してください。
- 厳格なアクセス制御を実施してください。
- システムおよびアプリケーションのセキュリティを維持してください。

SBOM（Software Bill Of Materials）

SBOM は、アプリケーションの「成分表」のような役割を果たします。コードで使用されているすべてのライブラリ、フレームワーク、依存関係とそのバージョン番号を記録したものです。ソフトウェアを構成するすべての要素の目録と考えてください。コンプライアンスの観点から、SBOM は重要な役割を果たします。セキュリティ上の脆弱性は、広く利用されているライブラリに頻繁に発生するからです。SBOM があれば、影響を受けるコンポーネントを、アプリケーションが使用しているかどうかを迅速に特定し、速やかに更新できます。ほとんどの開発ツールは SBOM を自動的に生成するため、手作業でのメンテナンスは不要です。



セキュリティに関連する操作は常にログに記録してください。ただし、機密データは決してログに記録しないでください！ 判断に迷った場合は、セキュリティチームに相談して指示を求めてください。依存関係の更新に関するメンテナンススケジュールを作成してください。定期的な間隔（毎月または四半期ごと）で SBOM を確認し、既知の脆弱性をチェックしたうえで、ライブラリを最新の安全なバージョンに更新するようにしてください。

セキュリティは「多層防御」が重要であることを忘れないでください。単一のセキュリティ対策だけで完璧なものは存在しませんが、複数のセキュリティレイヤーを構築することで、攻撃者がシステムを侵害するのを非常に困難にします。これは開発プロセスの中で常に意識しておくべき点ですので、セキュリティを意識する習慣を身につけるためにも、セキュリティ対策を段階的に導入するよう心がけてください。

本番環境向けコードを作成する際に、開発段階から留意すべきセキュリティの基本事項について理解できたところで、次はデプロイのプロセスそのものについて説明しましょう。

10.3　デプロイメントパイプライン

デプロイメントパイプラインとは、コードをある環境から本番環境へと、安全かつ確実に移行するためのロードマップです。各コンポーネントが特定の役割を果たし、コードを実際のユーザーがアクセスできるアプリケーションへと変換します。デプロイメントパイプラインは、円滑に機能する組み立てラインのようなものだと考えてください。

デプロイメントパイプラインは、4 つの相互に接続されたコンポーネントで構成されます。これらのコンポーネントが、機械の歯車のように連携して動作します。

デプロイ環境

デプロイ環境とは、ローカルの開発環境から、顧客がアプリケーションを利用する本番サーバーに至るまで、コードが配置され実行される場所のことです。各環境は、コードがユーザーに届く前にその妥当性を確認するために、個別の役割を果たします。

バージョン管理の戦略

複数の開発者が同じコードベースに変更を加える際、互いの作業が干渉しないよう調整します。こうした戦略は、リリースの管理や緊急事態への対応におけるセーフティネットとなります。

デプロイの自動化

環境間でコードを移行する際に発生しやすい手作業によるミスを排除します。チェックリストに従って何か見落としがないか心配する代わりに、自動化により、毎回同じ信頼性の高いプロセスが確実に実行されます。

CI/CD（継続的インテグレーションと継続的デプロイ、continuous integration and continuous deployment）

変更をコミットした瞬間から本番環境で実行されるまで、コードはパイプラインを通じて自動的に処理されます。これにより、一連のプロセスを一元的に管理できます。

本章を読み終える頃には、これらの要素がどのように組み合わさり、信頼性が高く、ストレスのないデプロイプロセスが構築されるのかが理解できることでしょう。

10.3.1 デプロイ環境

自社のデプロイプロセスを理解する前に、コードをデプロイする環境を理解することが重要です。デプロイ環境は組織によって異なり、同じ組織内でもチーム間で異なる場合があります。以下に、一般的に使われる環境とその目的をまとめておきます。

ローカル開発

読者のコーディングの旅が始まる、ローカルの「遊び場」です。ここは実験のために使える、読者だけのワークスペースです。ただし、他の開発者も同じ設定を共有していることを忘れないでください。

テスト／ QA（品質保証）

読者はローカル環境にも包括的なテストスイートを用意していることでしょう。しかし、テスト環境が最初の公開チェックポイントとなります。ここで品質保証が行われます。テストスイートの挙動はローカル環境とは異なる場合があります。

ステージング

ステージング環境は、本番に向けたリハーサルのようなものだと考えてください。本番環境前の最終的なテストの場となる、本番環境に近い環境です。性能テストやユーザー受け入れテストを行うのに適しています。

本番環境

本番の場であり、読者の最終目標です。読者が多くの時間と労力を費やして開発したアプリケーションを、ユーザーが実際に利用する環境です。

本章で、構成管理について学びました。アプリケーションを環境間で移行するうえで重要なことは、それぞれの環境に適した設定を確実に適用することです。構成管理は、後から付け加えるものではなく、デプロイプロセスに不可欠な要素であるべきです。ある環境から別の環境へコードを移行する方法を理解するには、それらの環境で何が変わるのかを理解しておく必要があります。

10.3.2 バージョン管理の戦略

バージョン管理の戦略は、コードの安定性、信頼性、デプロイの効率性が重視される本番環境に

おいて、特に重要となります。本番環境では、これらの戦略により、チームは稼働中のサービスを中断することなく機能をリリースしたり、問題のあるデプロイに迅速に対処（以前のバージョンへのロールバックや、修正を施したロールフォワードのいずれか）したり、コンプライアンスの目的で詳細な監査証跡を保持したりすることが可能になります。

教育や仕事を通じて、バージョン管理について経験していることでしょう。個人的なプロジェクトでも利用したことがあるでしょう。そして、リポジトリのクローン作成、ファイルの追加や削除、変更のプッシュといったGitの基本概念についてはよく理解されていると思います。この基礎知識は最初の一歩を踏み出すのには十分です。しかし、組織という環境に移ると、チームで作業を行うため、バージョン管理に新たな複雑さが加わります。

このような問題に対処するための一般的なワークフローは以下のとおりです。

Git Flow
: 複数の長期ブランチを使用して、異なる種類の作業（機能開発、リリース、ホットフィックス）を分離します。これにより、開発コードと本番コードが厳格に分離されます。本節でGit Flowについて詳しく見ていきます。

GitHub Flow
: mainブランチと短期間のfeatureブランチのみを使用する、よりシンプルなアプローチです。変更内容は、レビュー後にfeatureブランチから直接本番環境に反映されます。

トランクベース開発
: すべての開発者が単一のmainブランチで作業し、featureブランチの存続期間を非常に短く（多くの場合1日未満）することで、頻繁な統合を推奨します。

リリーストレイン
: 機能を一括してデプロイするために、定期的なリリース（週次や月次など）を計画・調整します。

Git Flowについて見ていきましょう。Git Flowの仕組みは明確であるため、チームがバージョン管理システムを使ってどのように連携するかを理解しやすいからです。Git Flowのブランチ戦略では、それぞれ特定の目的を持つ異なる種類のブランチを用いて作業を整理します。プロジェクトのライフサイクルを通じて存続するブランチもあれば、特定のタスクを完了すると消滅するブランチもあります。

各ブランチの種類について詳しく見ていき、チームが互いの足を引っ張ることなく協力し合う方法を学びます。

10.3.2.1　コアブランチ

Git Flowは、2つのコアと位置づけられる長期ブランチを中心に構成されています。この2つのブランチがプロジェクトの基盤となります。

main ブランチは、本番環境向けのコードを格納するブランチです。これが「信頼できる情報源」となります。このブランチは常に安定してデプロイ可能な状態であるべきです。チームで作業する際は以下のようにしてください。

main ブランチへの直接コミットを防止する

Git Flow では、main ブランチには develop（開発）ブランチを通じてレビューとテストが完了したコードが取り込まれます。このため、ここで重視されるのは、コードレビューではなく、誤って直接コミットしてしまうことを防ぐことです。

main ブランチがすべての自動テストに合格していることを確認する

自動テストは、本番環境で問題が発生しないようにするためのセーフティネットとなります。



main ブランチは従来 **master** と呼ばれていましたが、現在では多くの組織が、よりインクルーシブな用語である **main** を使用しています。どちらの用語も、同じ概念を指しています。

Git Flow において、**develop** ブランチは日々の開発活動が行われる場所です。これはチームの共有ワークスペースのようなものです。本番環境へ移行する準備が整う前に、個々の機能が統合されていきます。

develop ブランチで作業する際は次のようにしてください。

develop ブランチをコードレビュー必須で保護する

Git Flow において、実際にコードレビューが行われるのはここです。feature ブランチが develop ブランチにマージされると、チームメンバーが変更内容をレビューし、問題を発見し、コーディング標準を遵守しているかを確認します。

develop ブランチは常にビルド可能な状態に保つ

機能が不完全な部分があるかもしれませんが、コードは常にコンパイルでき、基本的なテストに合格する状態にします。

定期的に統合テストを行う

develop ブランチでは、複数の機能が同時に開発されているため、統合に関する問題を早期に発見できるよう、包括的なテストスイートを頻繁に実行してください。

develop ブランチは、機能がリリースとしてパッケージ化され最終的に main にマージされる前に、それらをまとめてテストするためのステージング領域として機能します。

10.3.2.2 補助ブランチ

Git Flow では、コアとなるブランチの他に、特定のタスクのために一時的で短期間のブランチ

を使用します。これらの補助ブランチは、作業を整理したり、チームメンバー間の競合を防いだりするのに役立ちます。

feature（フィーチャー）ブランチは、開発時間の大部分を費やす場所となります。新しい機能やバグ修正の作業を開始する際は、develop ブランチから専用の feature ブランチを作成します。これにより作業が隔離されるため、他の開発者や安定版コードに影響を与えることなく、試行錯誤や変更を行えます。

これらのブランチにわかりやすい名前をつけることで、他の開発者（そして将来の自分）に、そのブランチでどのような作業を行っているのかを伝えられます。**表10-1** には、適切なブランチ名と不適切なブランチ名の例を示しています。

表10-1　ブランチ名の例

適切なブランチ名	不適切なブランチ名
・ feature/user-authentication	・ my-branch
・ bugfix/login-timeout	・ stuff
・ enhancement/performance-optimization	・ feature/123（123 が何を指すかについての情報なし）
・ feature/JIRA-123-user-authentication （チケット管理システムを使っている場合）	
・ bugfix/GH-456-login-timeout (GitHub の issue 番号)	

feature ブランチについて学んだところで、「リリース準備が整ったら、develop ブランチを main ブランチに直接マージすればよいのでは？」と疑問に思われるかもしれません。それは実に良い質問です。release ブランチは、チーム環境で作業する際に生じ得るいくつかの問題を解決してくれます。

安定化期間

release ブランチで最終テストやバグ修正が行われている間も、チームは次のリリースに向けた新機能の開発を続けられます。

リリース準備

バージョン番号の更新、最終的なドキュメントの作成、デプロイ構成の変更には、進行中の開発の邪魔をしない専用のスペースが必要です。

品質ゲート

多くの組織では、ステークホルダーが本番環境にデプロイされるコードを承認する、「リリース候補」フェーズを設けています。

リリーススケジュール

チームが 2 週間ごとにリリースを行う場合、次のサイクルの開発を続けながら、一連の機能を「フリーズ」させる方法が必要になります。

すべてのチームに release ブランチが必要というわけではありません。それは、組織によって異なります。チームに正式なリリースプロセスや、スケジュールされたデプロイ期間がある場合、release ブランチを使うことで適切にコードを分離できます。

hotfix ブランチを設けることもあります。ここでも「develop ブランチを使えば済むのに、なぜホットフィックス用に別のブランチが必要なのか？」と疑問に思われるかもしれません。本番環境で深刻な問題が発生した際に、develop ブランチに未完成の機能が含まれているため、本番環境へリリースができない場合、どうすればよいのでしょうか？

以下に示すように、hotfix ブランチを使うことでこの緊急事態に対処できます。

不安定なコードを回避する

develop ブランチから未完成の機能をデプロイすることなく、本番環境の問題を解決できます。

更新を適切に管理する

hotfix ブランチでの修正は、main ブランチ（即時デプロイ用）と develop ブランチ（将来のリリースで適用漏れを防止するため）の両方に適用されます。

スピードと集中

読者のチームは、他の進行中の開発作業を気にすることなく、この重要な修正作業に取り組むことができます。

最小限のリスク

hotfix ブランチは安定版の main ブランチから分岐するため、変更するのは、変更しなければならない部分のみとなります。

本節では、Git Flow におけるブランチの種類について学びました。ご覧のとおり、それぞれのブランチには、チームの作業を整理し、適切にバージョン管理するための特定の役割があります。コアブランチ（main と develop）は安定性と調整機能を提供し、補助ブランチ（feature、release、hotfix）は、メインのワークフローを妨げることなく特定のタスクを処理できるようにします。最初は少し難しく感じるかもしれませんが、このような戦略で作業を積み重ねていけば、チームが安全に協力し合えるような明確な境界線を築く仕組みであることが理解できるでしょう。

10.3.2.3 バージョン管理のライフサイクル

では、これらすべてのブランチはどのように連携して、バージョン管理戦略を形成しているのでしょうか？ Git Flow やその他の多くの戦略では、開発は feature ブランチで行われ、develop ブランチにマージされ、リリース後に main ブランチに反映されます。必要に応じて、hotfix ブランチの内容は main ブランチと develop ブランチの両方に直接適用されます。

Git Flow のワークフローを**図 10-1** に示します。

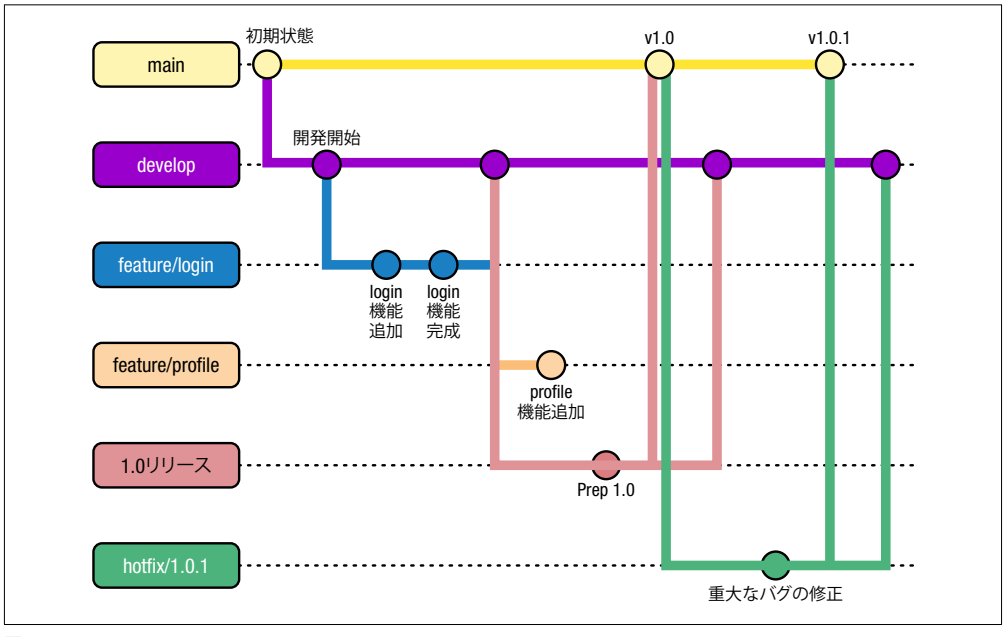


図10-1　Git Flow のワークフロー

Git Flow を理解することで、チーム開発のための強固な基盤が築けます。ただし、これは数あるアプローチの一つに過ぎないことを忘れないでください。さまざまなチームやプロジェクトに携わる中で、これらの戦略にはさまざまなバリエーションがあることに気づくでしょう。ただし、その目的は変わりません。コードを保護し、作業を分離し、変更を統合するための明確なプロセスを確立することです。次の節では、慎重に管理されたコードを確実かつ一貫して本番環境にデプロイするために、デプロイを自動化する方法を解説します。

10.3.3　デプロイの自動化

本番環境への手動デプロイは、多くの場合、時間がかかり、ミスが発生しやすく、ストレスのたまる作業です。コードの準備やパッケージ化、テストと検証、サーバーの準備、デプロイの実行、デプロイ後の検証など、デプロイ手順を一つひとつ手作業で行う必要があるためです。幸いなことに、デプロイを自動化することで、この問題に対処できます。本節では、デプロイプロセスを効率化するための鍵となることについて学んでいきます。

10.3.3.1　スクリプトによるデプロイ

手作業でのデプロイは、例えて言うなら、本棚が必要になるたびに、その都度手作業で特注の本棚を作るようなものです。たとえ以前と同じ本棚を作るとしても、手順を一つ飛ばしたり、些細な違いが生じたり、必要以上に時間がかかってしまったりするおそれがあります。デプロイの自動化は、正確な仕様に基づいてまったく同じ本棚を製造する工場を持つようなものです。正確で、再現

性が高く、人為的なミスもありません。デプロイスクリプトは、その工場の製造指示書のようなものです。アプリケーションを開発環境から本番環境へ一貫して移行するために、必要となるすべてのステップをドキュメント化したものです。

以下に示すのは、ディレクトリの作成、既存ディレクトリのバックアップ、アプリケーションのデプロイ、および起動といった手順を実行するデプロイスクリプトの簡単な例です。

```
#!/bin/bash

# 設定
APP_NAME="myapp"
JAR_FILE="target/${APP_NAME}.jar"
DEPLOY_DIR="/opt/applications/${APP_NAME}"
BACKUP_DIR="${DEPLOY_DIR}/backups"
LOG_FILE="${DEPLOY_DIR}/app.log"
PID_FILE="${DEPLOY_DIR}/${APP_NAME}.pid"
JVM_OPTS="-Xmx512m -Xms256m"

# ディレクトリの作成
mkdir -p "${DEPLOY_DIR}" "${BACKUP_DIR}"

# 既存のデプロイをバックアップ
if [ -f "${DEPLOY_DIR}/${APP_NAME}.jar" ]; then
    mv "${DEPLOY_DIR}/${APP_NAME}.jar" \
    "${BACKUP_DIR}/${APP_NAME}-$(date +'%Y%m%d_%H%M%S').jar"
fi

# 実行中のアプリケーションの停止
if [ -f "${PID_FILE}" ]; then
    kill $(cat "${PID_FILE}") 2>/dev/null || true
    rm -f "${PID_FILE}"
    sleep 2
fi

# 新しいバージョンのデプロイ
if [ -f "${JAR_FILE}" ]; then
    cp "${JAR_FILE}" "${DEPLOY_DIR}/${APP_NAME}.jar"
else
    echo "Error: JAR file not found: ${JAR_FILE}"
    exit 1
fi

# アプリケーションの起動
nohup java ${JVM_OPTS} -jar "${DEPLOY_DIR}/${APP_NAME}.jar" \
    > "${LOG_FILE}" 2>&1 &
echo $! > "${PID_FILE}"

# 簡単なヘルスチェック
sleep 5
if kill -0 $(cat "${PID_FILE}") 2>/dev/null; then
    echo "Deployment successful. PID: $(cat ${PID_FILE})"
else
```

```
    echo "Deployment failed. Check ${LOG_FILE} for details."
    exit 1
fi
```

この例は基本的なものですが、デプロイの自動化における主な要素が含まれています。

- 明確でドキュメント化された手順
- エラー処理
- バックアップ手順
- 体系的なアプローチ

たとえ簡単なスクリプトであっても、何もないよりはましです。まずは小規模から始め、時間をかけてデプロイの自動化を徐々に改善していきましょう。今このプロセスを自動化することに時間を割いておけば、将来の自分がきっと感謝してくれるはずです。

10.3.3.2　ロールバックの手順

デプロイを自動化すれば、時間を節約でき、ストレスも軽減できることはすでに説明しました。しかし、十分にテストされた自動化デプロイであっても、時には問題が発生することがあります。

完璧なものなどありません。予期せぬ問題に備えておく必要があります。変更したものを迅速に正常な状態に戻すための計画を立てておくことで、顧客はアプリケーションを使いつづけられます。

以下は、最新のバックアップを検索し、アプリケーションを停止して復元し、起動した後、アプリケーションが正常に動作していることを確認するロールバックスクリプトの例です。

```
#!/bin/bash

# 設定（deploy.sh と一致している必要がある）
APP_NAME="myapp"
DEPLOY_DIR="/opt/applications/${APP_NAME}"
BACKUP_DIR="${DEPLOY_DIR}/backups"
LOG_FILE="${DEPLOY_DIR}/app.log"
PID_FILE="${DEPLOY_DIR}/${APP_NAME}.pid"
JVM_OPTS="-Xmx512m -Xms256m"

# 最新のバックアップを検索
LATEST_BACKUP=$(ls -t ${BACKUP_DIR}/${APP_NAME}-*.jar 2>/dev/null | head -1)

if [ -z "${LATEST_BACKUP}" ]; then
    echo "Error: No backup found in ${BACKUP_DIR}"
    exit 1
fi

# 現在のアプリケーションを停止
```

```
if [ -f "${PID_FILE}" ]; then
    kill $(cat "${PID_FILE}") 2>/dev/null || true
    rm -f "${PID_FILE}"
    sleep 2
fi

# バックアップを復元
cp "${LATEST_BACKUP}" "${DEPLOY_DIR}/${APP_NAME}.jar"

# アプリケーションを起動
nohup java ${JVM_OPTS} \
    -jar "${DEPLOY_DIR}/${APP_NAME}.jar" \
    > "${LOG_FILE}" 2>&1 &
echo $! >  "${PID_FILE}"

# 簡単なヘルスチェック
sleep 5
if kill -0 $(cat "${PID_FILE}") 2>/dev/null; then
    echo "Rollback successful. Restored: $(basename ${LATEST_BACKUP})"
    echo "Application running with PID: $(cat ${PID_FILE})"
else
    echo "Rollback failed. Check ${LOG_FILE} for details."
    exit 1
fi
```

ロールバックスクリプトを実行する前に、以下の準備を整えておく必要があります。

- デプロイの前に、バックアップバージョンが存在することを確認します。
- ロールバック手順を定期的にテストします。
- データベースマイグレーションのロールバック手順をドキュメント化します。
- 複数のバックアップバージョンを保持してください。
- ロールバック中はシステムを監視してください。

デプロイの自動化の目的は、単に時間を節約することだけではありません。信頼性が高く、再現性のあるプロセスを構築し、デプロイに対して確信を持てるようにすることです。まずは小規模から始め、例えば数ステップを自動化するようなシンプルなスクリプトから着手し、徐々に包括的な自動化へと発展させていきましょう。改善を重ねるごとにリスクが軽減され、予測可能でストレスのないデプロイという理想に一歩ずつ近づいていけます。

10.3.4 デプロイ戦略

コードを本番環境にデプロイする際、すべてのケースにふさわしいような万能な戦略というものはありません。アプリケーションに対してどのような戦略を採用するかによって、スムーズな展開ができるか、それとも週末中ずっと緊急連絡の待機を強いられるか、その差が生まれます。ここでは、最も一般的なアプローチと、それぞれをいつ選択すべきかについて見ていきましょう。

10.3.4.1　一括デプロイ（ビッグバン方式）

この戦略では、アプリケーション全体を一括で置き換えます。旧バージョンは、新バージョンに一度に完全に置き換えられます。実装は簡単ですが、問題が発生すると直ちにすべてのユーザーに影響が及ぶため、リスクが高くなります。段階的な移行期間がないため、問題が発生した場合のロールバックは複雑になります。このアプローチは、徹底したテストが行われている小規模なアプリケーションや、ダウンタイムが許容される環境に適しています。

ビッグバン方式は、次のような場合に適しています。

- ユーザー数が限られた小規模なアプリケーション
- 開発およびテスト環境
- ある程度のダウンタイムを許容できるシステム

10.3.4.2　段階的デプロイ（フェーズドアプローチ）

段階的デプロイとは、お気に入りのレストランが、少数の客にだけ新メニューを提供するようなものです。シェフはフィードバックを集め、改良を重ね、徐々にそのメニューの提供範囲を広げていけます。段階的デプロイも同様のアプローチを採用しています。変更内容を一部のユーザーやサーバーに先行して導入し、その後、より広範囲に展開します。このようなアプローチにより、問題を早期に発見し、軌道修正を行うことが可能になります。

一般的な例としては、**カナリアデプロイ**（まず少数のユーザーに対してデプロイを行う手法）や**ローリングデプロイ**（サーバーを 1 台ずつ段階的に更新する手法）が挙げられます。どちらも、ブルーグリーンインフラストラクチャや、ロードバランサのトラフィックシフトといった手法を用いることで、多くの場合ダウンタイムゼロで実施できます。

以下の場合には、段階的デプロイが適しています。

- 大規模なアプリケーション
- ユーザーからのフィードバックが活かされる機能
- 綿密な監視が必要なシステム

10.3.4.3　ダウンタイムゼロに関する考慮事項

一括デプロイか段階的デプロイのいずれを選択する場合でも、ブルーグリーンデプロイ（2 つの同一環境を維持し、それらを切り替える手法）やローリングアップデートなどの手法を活用することで、ダウンタイムを最小限に抑えるか、あるいは完全に排除できます。設定は複雑になりますが、これらの手法は、ミッションクリティカルなアプリケーションやトラフィックの多いサービスにおいて、ユーザー体験を極めて快適なものにします。

10.3.4.4　戦略の選択

それぞれのアプローチは、どのような場合に選択すべきでしょうか？ ここまで読んできた方な

ら、答えはたいてい「状況による」であることはもうおわかりでしょう。アプリケーションに適した戦略を選択する際には、システムの複雑さ、ダウンタイムに対する許容度、監視機能などの要素を考慮してください。

覚えておいてください。まずはシンプルに始め、アプリケーションやチームが成熟するにつれてデプロイ戦略を進化させていくという方法に、何ら恥ずかしいところはありません。

10.3.5 継続的インテグレーションと継続的デプロイ

5章では、自動テストがコードの記述やコミット時に自信を与えてくれるだけでなく、開発ライフサイクルの早い段階で問題を発見するのに役立つことを学びました。しかし、リリースが本番環境などの新しい環境に展開されるたびに、これらのテストが確実に実行されるようにするにはどうすればよいでしょうか？ また、手作業によるミスを排除し、コードのビルド、テスト、デプロイを確実に実行するにはどうすればよいでしょうか。

ここで、**継続的インテグレーションと継続的デプロイ（デリバリー）**、つまりCI/CD（Continuous Integration and Continuous Deployment/Delivery）が役立ちます。これは、開発段階からユーザーが利用する本番環境に至るまで、コードを一貫して管理する一連の自動化されたプロセスです。**継続的インテグレーション**は、開発チーム全体からの変更を自動的に統合しテストを行います。**継続的デリバリー**は、テスト済みのコードをリリースに向けて準備します。**継続的デプロイ**は、それを本番環境に自動的にリリースします。

例えば、入社して2週目あたりで、チームから小規模なマイクロサービスのデプロイを手伝ってほしいと頼まれたとします。「アプリケーションをビルドする」「テストを実行する」「データベースを更新する」「サーバーにデプロイする」といった曖昧な手順ばかりが並んだ、分厚いデプロイドキュメントを開きます。たった一つの些細な手順を見逃しただけで本番環境がダウンしてしまうかもしれないとわかっているため、項目を一つずつ確認していくにつれ焦りが募っていきます。

かなりストレスがたまりそうですね。だからこそ、自動化が必要なのです。自動化を活用し、繰り返し可能で信頼性の高いプロセスを構築することで、多くの問題を解消できます。自動化のメリットには、次のようなものがあります。

一貫性

「私のマシンでは動くのに」という状況、つまりローカルではアプリケーションが正常に動作するものの、別の環境に移行すると問題が発生するケースを覚えていますか？ ビルドプロセスを自動化することで、そのような問題を最小限に抑えることができます。

自動テスト

開発者がコードをコミットする前に一連のテストを実行してくれることを期待するのではなく、CI/CDプロセスでテストを自動的に実行できます。

ストレスの軽減

デプロイプロセスの各ステップを手作業で注意深く確認する必要がなくなります。自動化に

より、毎回確実に同じ手順で実行されます。スクリプトは決して手順を飛ばしたり、コマンドを誤入力したりすることはありません。

統合に関する問題の早期発見

明確に定義されたCI/CDプロセスは、本番環境への不具合の混入を防ぎ、時間とコストの節約につながります。

開発から本番環境への移行までの時間を短縮

明確に定義された自動化プロセスを活用して、デプロイ時間を最小限に抑えることができるため、頻繁に本番環境へデプロイできます。コードはエンドユーザーの手元に届くまでは、真の価値を持たないということを忘れないでください。

10.3.5.1　基本的なCI/CDワークフローの構築

CI/CDワークフローはレシピのようなものです。開発環境から本番環境へコードを移行させるために必要な手順を定義するものです。基本的なワークフローを見てみましょう。

コード

開発者がリポジトリにコードをプッシュします。

ビルド

アプリケーションはコンパイルされビルドされます。

テスト

自動テストが実行されます。

パッケージ化

アプリケーションはデプロイ用にパッケージ化されます。

デプロイ

アプリケーションは対象環境にデプロイされます。

これを実践するために、基本的なGitHub Actionsワークフローを作成してみましょう。**GitHubワークフロー**とは、GitHubリポジトリ内で設定できる自動化プロセスです。このワークフローを活用して、コードのビルド、テスト、パッケージ化、リリース、デプロイを行えます。GitHub Actionsは、導入が簡単でGitHubリポジトリとシームレスに連携するため、CI/CDパイプラインの実装に広く利用されています。

以下は、Javaアプリケーション向けに、これらの手順を実装したワークフローファイルの例です。

```
name: Java CI/CD Pipeline

on:
 push:
   branches: [ main ]
 pull_request:
   branches: [ main ]

jobs:
 build-and-test:
   runs-on: ubuntu-latest

   steps:
   # リポジトリのコードをチェックアウトする
   - uses: actions/checkout@v4

   # Java 開発環境のセットアップ
   - name: Set up JDK 25
     uses: actions/setup-java@v4
     with:
       java-version: '25'
       distribution: 'temurin'

   # アプリケーションのビルド
   - name: Build with Maven
     run: ./mvnw -B package --file pom.xml

   # 自動テストの実行
   - name: Run tests
     run: ./mvnw test

   # アプリケーションのパッケージ化（JAR ファイルの作成）
   - name: Package application
     run: ./mvnw package -DskipTests

   # パッケージをアーティファクトとして保存
   - name: Upload artifact
     uses: actions/upload-artifact@v4
     with:
       name: app-package
       path: target/*.jar
```

このワークフローファイルでは、いくつかの重要な概念を紹介しています。

- on セクションでは、ワークフローがいつ実行されるか（main ブランチへのプッシュ時またはプルリクエスト時）を定義します。
- 各ジョブは、新しい仮想環境（ubuntu-latest）で実行されます。
- ステップは順次実行されます。各ステップは前のステップが正常に完了した場合に実行されます。
- このワークフローでは、アプリケーションのビルド、テスト、パッケージ化を行います。

このワークフローを導入することで、開発プロセスにおけるいくつかの重要なステップを自動化したことになります。コードをプッシュしたりプルリクエストを作成したりすると、GitHub は自動的に以下の処理を行います。

- コンパイルエラーを早期に検出するためにアプリケーションをビルドする
- 潜在的なバグを検出するために、テストスイートを実行する。
- デプロイ可能なパッケージを作成する
- 後で使用するためにパッケージを保存する

CI/CD ワークフローは、一連の事前定義されたステップを通じて、開発から本番環境へのコードの移行プロセスを自動化します。この自動化により、手作業による介入が不要になり、エラーを早期に検出し、品質基準を遵守できます。最終的には再現性のあるプロセスを通じて、より確実かつ効率的にソフトウェアをリリースすることが可能になります。

10.3.5.2　高度な CI/CD パターン

一度にすべてを覚えなければならないと焦る必要はありません。まずは簡単なことから始め、この章で学んだ基礎を土台にして進めていきましょう。まずはビルドとテストのプロセスを自動化することから始めてください。次第に仕組みが理解できるようになり、自信がついてくれば、そこから徐々に高度なデプロイ戦略を取り入れていくことができます。

キャリアを積むにつれて、次のような高度な CI/CD パターンに出会うことでしょう。

カナリアリリース

まず、一部のユーザーやサーバーに新しいコードをデプロイし、リスクを最小限に抑えながら本番環境でテストを行い、問題がなければ徐々に範囲を広げていきます。

ブルーグリーンデプロイ

2 つの同一の本番環境（ブルーとグリーン）を維持し、1 つだけが稼働するようにします。新しいバージョンは非稼働環境にデプロイされ、検証が完了した時点でトラフィックが完全に切り替わります。

高度なパターンを導入し、CI/CD プロセスを段階的に改善する方法を紹介します。

1. まずは、コードのビルドとテストを行う基本的なパイプラインから始めましょう。
2. 徐々に、自動化されたチェックや検証機能を追加していきましょう。
3. 迅速なフィードバックループを確保するため、パイプラインの処理速度を維持しましょう。
4. チームのニーズに合わせて、定期的にパイプラインを監視・最適化しましょう。
5. チームでの協業を円滑にするため、パイプラインをドキュメント化しましょう。

CI/CD は、単なるツールや自動化の話ではありません。継続的な改善と信頼性の高いソフト

ウェアを提供する、という文化を育むことこそが本質です。キャリアの早い段階からこうした実践を取り入れることで、プロフェッショナルとして成長するための強固な基盤を築くことができ、チームにとって欠かせない存在になれるでしょう。

10.4　本番システムの監視と保守

コードを書き上げ、徹底的にテストを行い、いよいよ本番環境への移行準備が整いました。アプリケーションは適切な CI/CD パイプラインを経て、現在本番環境で稼働しています。これで一息ついてリラックスできるはずですよね？ そうとも限りません。ローカル環境で動作するアプリケーションが、本番環境において信頼性の高いシステムとなるまでの道のりは、まだ続くのです。

10.4.1　監視

適切な監視とロギングを行えば、アプリケーションの状態について多くのことがわかります。この仕組みが整っていなければ、システムが現在、あるいは長期的にどのようなパフォーマンスを発揮しているのか把握できません。多くの開発者は、アプリを本番環境に投入して、うまくいくことを願うだけという傾向があります。「知らせがないのは良い知らせだ」と考えているのです。しかし実際には、ユーザーは問題が発生してもそれを伝えてくれることはめったにありません。ただ、離れていくだけです。

アプリケーションは、2 種類のシステムによって監視・運用されます。

リアルタイム監視

現在の状況をリアルタイムで把握します。アプリケーションの応答時間が遅すぎませんか？ サーバーの負荷や処理量が高すぎませんか？ 個々のメトリクスだけでは、全体像を把握することはできません。300 ミリ秒の応答時間は速く思えるかもしれませんが、通常 50 ミリ秒で応答しているアプリケーションであれば、これは問題の兆候であるおそれがあります。

ログ

ログは、事後検証のための記録として役立ちます。問題が発生した際、適切なログがあれば、何がいつ問題になったのかを再現するのに役立ちます。ログは、トラブルシューティングのために、重要なコンテキストを提供してくれます。

筆者の場合、アプリケーションでいくつかの重大なインシデントが発生したことを受け、以下のような監視ルールを策定しました。

1. 問題が発生した際に必要となる情報をログに残してください。
2. 必ずタイムスタンプとユーザー ID を含めてください。
3. エラーはエラーとして明示してください。重要なアラートを info（情報）ログの中に埋もれさせないでください。

4. ログには個人情報を記録しないでください（パスワードや個人情報は記録しないでください）。
5. 古いログは、ストレージ容量を圧迫する前に削除してください。



最初は監視システムをシンプルに保ちましょう。応答時間、エラー数、サーバーのリソースを監視してください。まずはそれで十分です。高度なツールは後回しにしましょう。

適切な監視体制があれば、安心感が得られます。ソフトウェアには、いずれ問題が発生するという性質があります。問題が発生した際、メトリクスやログがあれば、迅速に問題を診断・修正できるため、その重要性を実感できることでしょう。



AIについて

ログとメトリクスを収集できるようになったら、AIツールを活用することで監視機能を大幅に強化できます。AIは大量のログを分析し、深刻化する前に、問題の兆候を示す異常やパターンを特定できます。さらに、メトリクスデータに基づく監視ダッシュボードを構築するのも、従来は手作業で面倒な設定が必要でしたが、現在ではAIが支援してくれます。AIを活用することで、監視体制の基盤が堅固になり、ダッシュボードの構築作業が大幅に効率化されます。

10.4.2　システム保守

システムが本番環境にデプロイされても、それで終わりではありません。それはあくまで始まりに過ぎません。システムの保守は、ソフトウェアを効率的かつ安全、そして確実に稼働させつづけるうえで極めて重要な要素です。この節では、多大なコストを伴う問題を未然に防ぎ、最適なパフォーマンスを確保するために、本番システムをどのように保守すべきかについて解説します。

10.4.2.1　システムを最新の状態に保つ

本番環境にデプロイされたソフトウェアには、定期的な保守が必要です。本番環境ではすべてが順調に動作している場合、保守を先延ばしにしたくなるかもしれません。しかし、それは間違いです。最初は些細で費用もかからない問題であっても、あっという間に重大で多額の費用がかかる問題へと発展してしまうおそれがあります。

ユーザーが頻繁に利用するシステムを扱う場合、そのコストは高くなります。重要なインフラストラクチャは、安全性と信頼性を確保するために、継続的な保守が必要です。しかし、ソフトウェア開発の現場では、リスクがあるにもかかわらず、チームが必要な更新を先延ばしにしてしまうことがよくあります。

10.4.2.2　システムの更新

オペレーティングシステムの更新プログラムやセキュリティパッチは、重大かつ多額の損失を招く問題に立ち向かうための最前線の防衛線となります。これらの更新プログラムは、多くの場合、

以下の問題に対処するものです。

- 重大なセキュリティ上の脆弱性
- パフォーマンスの改善
- バグの修正
- 新機能の追加

セキュリティ更新プログラムは決して無視しないでください。機能の更新は後回しにできる場合もありますが、セキュリティパッチは、システムを脆弱性から守るために速やかに適用する必要があります。

10.4.2.3 依存関係の管理

多くの場合、依存関係の管理は、システム保守において見過ごされています。アプリケーションは数十、あるいは数百もの外部ライブラリに依存している場合があり、そのそれぞれにセキュリティ上の脆弱性が潜んでいるおそれがあります。この章で取り上げた SBOM を覚えていますか？正確な SBOM を維持することで依存関係を追跡・更新できるため、SBOM は非常に重要です。

以下のコードは、Java アプリケーションの依存関係の中に既知の脆弱性がある古いバージョンが含まれていることを示しています。

```
// pom.xml 内の依存関係の例
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <version>2.5.5</version>  // 既知の脆弱性がある古いバージョン
</dependency>
```

このアプリケーションの SBOM を生成すると、セキュリティツールが脆弱性データベースと照合してスキャンを行います。バージョン 2.5.5 に既知のセキュリティ上の問題があることを特定し、より新しい安全なバージョンへの更新を推奨してくれます。

Maven や Gradle などのモダンビルドツールには、既知の脆弱性を通知してくれる依存関係アナライザーが備わっています。こうした問題を早期に発見するために、自動スキャンを設定しましょう。次の例では、GitHub Action を使用してコードの依存関係をチェックします。

```
// GitHub Actions ワークフローのスニペット例
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run Dependency Check
        uses: dependency-check/Dependency-Check@main
```

10.4.2.4　行動を起こす

チームやプロジェクトの更新ポリシーを策定することで、システムの保守体制の改善に着手できます。

1. 定期的な更新間隔を設定します。
2. 緊急パッチの適用手順を定めます。
3. 自動スキャンを導入します。
4. 更新プロセスをドキュメント化します。
5. 更新ログを管理します。

予防は復旧よりもコストがかからないことを忘れないでください。開発ワークフローにおいて、システムの更新を最優先事項としてください。定期的な更新の確認を行うよう、スケジュール管理システムにリマインダーを設定しましょう。自動化が進んでいるとはいえ、システムの健全性を維持するためには、人間の監視が依然として不可欠なのです。

10.5　まとめ

安全で安心なローカル環境から、予測不可能な本番環境へとコードを移行するのは、最初は気が重く感じるかもしれません。しかし、読者だけがそう感じているわけではありません。ほとんどの開発者が、キャリアのどこかで同じようなストレスを感じたことがあるのです。適切なデプロイプロセスを確立するためには、事前の準備が鍵となります。本番環境へのコード移行は、経験を積むにつれて上達していくスキルです。

以下の重要なポイントを覚えておいてください。

- 開発の初日から本番環境での運用を想定しましょう。コーディングの際には、パフォーマンス、セキュリティ、エラー処理を考慮してください。
- 環境変数やフィーチャーフラグを活用し、さまざまな環境に合わせてアプリケーションを適切に設定してください。
- アプリケーションの状態を監視し、適切なロギングを設定してください。ただし機密データは決してログに記録しないでください。
- Git Flow などのバージョン管理戦略を活用し、チームと安全に共同作業を行いましょう。
- CI/CD を利用してデプロイを自動化し、信頼性と再現性を確保しましょう。
- ユーザーはアプリケーションが開発者のマシンで動作するかどうかは気にしません。重要なのは、ユーザー自身のマシンで動作するかどうかです。

自身のアプリケーションを本番環境にデプロイし始めると、課題に直面したり、時には失敗したりすることもあるでしょう。しかし、それは問題ありません。経験する失敗の一つひとつが、新し

いことを学ぶ機会となり、読者をより優れたソフトウェアエンジニアへと成長させてくれるはずです。

まずは良い習慣を身につけることに集中しましょう。そうすれば、本番環境へのデプロイも開発ワークフローの一部として自然に定着するでしょう。そう遠くないうちに、読者は経験豊富な開発者となり、他のメンバーがコードを本番環境という「約束の地」へと送り出すのを手助けできるようになるはずです。

10.6 実践に移す

アプリケーション全体が本番環境でどのように動作するかを予測するのは難しいため、この章で学んだベストプラクティスを少しずつ取り入れていきましょう。以下に挙げるのは、本番環境へデプロイする前に、さまざまな段階で実践できる良い習慣です。次に機能開発の担当になった際は、これらの中から 1 つか 2 つを選んで自分の作業に取り入れてみてください。一度にすべてを実践しようとせず、これらの習慣を徐々に身につけることに集中しましょう。

コーディングを始める前：

- その機能で発生し得る問題点を簡単にリストアップしてください。
- これらの問題をデバッグするために、どのような情報をログに記録する必要があるか計画してください。
- ワークフローにおいて、ローカル環境から他の環境へ移行する際に、どのような設定が変更される可能性があるかを特定してください。

コーディング中：

- 作業を開始する前に、明確でわかりやすい名前をつけた feature ブランチを作成してください。
- 失敗する可能性のある上位 3 つのケースに対して、基本的なエラー処理を実装してください。
- 重要なアクションにはログを出力するようにしてください（ただし、機密情報のログ出力は避けてください！）。
- 実際のユーザーの動作を模倣したテストを作成してください。
- 環境によって変更される可能性があるものはすべて、設定で値を変更できるようにしてください。

デプロイ前：

- デプロイ手順を実行しながら書き留めておきましょう。
- 簡単なロールバック計画を立てておきましょう。万が一問題が発生した場合、この機能をどのように無効化しますか？
- 本番環境での運用を念頭に置き、チームメンバーに変更内容を確認してもらいましょう。

デプロイ後：

- デプロイ後、1〜2時間ほどログの監視を行ってください。何か気になる点はありますか？
- 今回のデプロイに関するレトロスペクティブを開催してください。レトロスペクティブは、プロジェクトやデプロイの完了後に開催される会議や議論であり、チームにおいて何がうまくいったのか、どのような課題に直面したか、そしてどのような教訓が得られたのかを振り返る場となります。

10.7　関連リソース

- 『Continuous Delivery』Jez Humble, David Farley 共著（Addison-Wesley Professional、2010年）
  和書『継続的デリバリー』（KADOKAWA、2017年）
- 『The Phoenix Project』Gene Kim 他著（IT Revolution Press、2013年）
  和書『The DevOps 逆転だ！』（日経BP、2014年）
- 『Head First Git』Raju Gandhi 著（O'Reilly Media、2022年）
- 『Learning GitHub Actions』Brent Laster 著（O'Reilly Media、2023年）
- 『Feature Flags』Ben Nadel 著（自費出版、Lulu、2024年）

11章
生産性を向上させる

忙しくするのではなく、生産的であることに集中しましょう。
— Tim Ferriss、アメリカの起業家、投資家、作家

いわゆる「10 倍開発者」という言葉を耳にしたことがあるかもしれません。これは、平均的な開発者の 10 倍の生産性を持つとされるソフトウェアエンジニアのことです。彼らが実際に存在するかどうかは議論の余地がありますが、他の人よりも迅速かつ効率的にタスクをこなす人々が**確かに存在する**ことは否定できません。そんな人になるためには、常に改善の方法を模索してください。まずは、開発ツールキット、コードエディタ、コマンドライン、そしてシニアエンジニアなら誰もが知っているような小さなユーティリティから始めましょう。この章では、開発環境をカスタマイズすることなど、より生産性の高いソフトウェアエンジニアになるためのヒントを紹介します。エディタを自在に操る方法、コマンドラインをスムーズに操作する方法、そしてキーボードショートカットをマスターすることがなぜ「超能力」とも言えるほどの強みになるのかについて学んでいきます。

11.1　開発環境の最適化

大昔の開発者たちは、かなり原始的なツールで何とかやりくりしなければなりませんでした。幸いなことに、現在では IntelliJ IDEA や VS Code のような、驚くほど充実した機能を備えたエディタが数多く存在します。しかし、ツールをインストールしたからといって、それで終わりではありません。それはあくまで始まりに過ぎないのです。



さまざまな設定を自由に試してみてください。ただし、時間をかけ過ぎないように注意してください。気をつけないと、本来仕事をしなければならないときに、フォントやテーマをいじくり回している自分に気づくかもしれません。試行錯誤には時間制限を設けるのが賢明です。カスタマイズをやりすぎると、ペアプログラミングや他者との効果的な共同作業がほぼ不可能になってしまうこともあります。Nate は以前、Vim でしか作業できない開発者とペアプログラ

ミングを試みたことがあります。相手は、特殊なキーボードを使い、Dvorak 配列の設定をしていました。結果はうまくいきませんでした。

11.1.1　自身の開発ツールを知る

多くの職業において、道具の管理は各自の責任となります。プロの厨房には共用の包丁などありません。シェフたちは各自で自分の包丁を持参する責任があります。整備士は、多くの場合、自分好みの工具を自分の基準で整理した、自分専用の工具箱を持っています。多くの職業において、必要な工具を購入することは、その業界に入るための初期投資となります。会社がソフトウェアライセンスの費用を負担してくれる場合でも、どのようなツールキットを揃えるかについては依然として読者自身の責任であり、それを自分らしくカスタマイズするようにしてください。

ソフトウェアエンジニアとして、開発ツールに対して主体的に取り組むべきです。モダン IDE は、適切なプラグインを選択して組み合わせることで、最適な環境を構築することを推奨しています。自分の仕事の道具について学ぶ必要があります。それは、隅々まで知り尽くしたり、あまり知られていないコマンドやメニュー項目をすべて把握したりする必要があるという意味ではありません。ただし、日々使い慣れているものについては、自分の手のひらのように使いこなせるようにしておくべきです。

目標は、Mihaly Csikszentmihalyi が著書『Flow: The Psychology of Optimal Experience』(Harper & Row、和書『フロー体験：喜びの現象学』世界思想社）で述べた「フロー状態」を維持することです。**フロー状態**とは、目の前の作業に完全に没頭し、深い集中状態にある期間のことです。まるで魔法のように、画面上にコードが次々と現れてくるように感じられるでしょう。キーボードショートカットを使えば済むところを、数分おきに作業を中断してコマンドを調べたり、トラックパッドに手を伸ばしたりすると、フロー状態が妨げられ、集中力が途切れてしまいます(キーボードショートカットについてはこの章で後ほど詳しく説明します)。

> コードが頭の中から IDE へと自然に流れ出て、気がつけば（IDE のショートカットキーは完全に覚えているので)、ビルドやテスト、実行を行っているのです。
>
> — Mark Heckler、『Spring Boot: Up and Running』の著者

お使いのエディタやソースコード管理ツールについて、時間をかけて詳しく学んでおきましょう。不用意に main ブランチへのプッシュを強行し、チームの 3 週間分の作業を無駄にしてしまうような人にはなりたくないですよね。

モダン IDE は膨大な機能を備えており、そのすべてを把握することはほぼ不可能です。効果的な習熟方法としては、毎週 1 つの新機能を学ぶことを習慣にすることです。例えば 2 章でも触れたように、JetBrains IntelliJ IDEA では起動時に「Tip of the Day」が表示されます。これは、自分では気づかなかったショートカットやツール、リファクタリング、デバッグ、プラグインなどを発見する絶好の機会となります。他の IDE でも同様の機能を探して、新しいことを学んでみましょう！

11.1.2 自分だけのライトセーバーを作る

車のハンドルを握るとき、シートやハンドル、ミラーを自分に合うように調整しますよね。いわゆる「工場出荷時の設定」のままでは、快適（あるいは安全）とは言えません。これと同じように、開発環境も**読者**自身のニーズに合わせてカスタマイズすべきです。ツールにはさまざまな設定があるので、少し時間をかけて、自分の好みに合わせて最適化する価値は十分にあります。それはまるで、自分だけのライトセーバーを作るようなものです。ツールは自分の手や性格、ニーズにぴったり合うものであるべきです。自分の環境を自分らしくカスタマイズするようにしてください。

エディタでコードを何時間も見つめつづけることになるでしょう。デフォルトのフォントも悪くはないかもしれません。しかし、少し時間をかけて他のフォントも試してみると、目にも優しい環境になるはずです。別のフォントに変えることで、見た目が良くなるだけでなく、自分の個性を反映させた環境作りにつながるかもしれません。

優れた等幅フォントは数多く存在します。インターネットで検索すれば、コーディングやプログラミングに適した等幅フォントが数多く見つかるでしょう。ぜひ試してみてください。1 日か 2 日ほど使ってみて、どう感じるか確かめてください。さまざまなフォントを試してみてください。また、11 ポイントや 12 ポイントというフォントサイズにこだわる必要もありません。もし他のサイズのほうが目に優しいと感じたら、遠慮なく変更してください！

ほとんどのエディタには、ツールのほぼすべての外観を変更できる無数のテーマが用意されています。多くの開発者はダークモードを好んでいます。一日中非常に明るい画面を見つめつづけると、目に負担がかかるからです。とはいえ、ダークテーマが自分に合うかどうかは人それぞれですので、自分に合ったものが見つかるまでいくつか試してみてください[†1]。一部のオペレーティングシステムでは、周囲の環境や時間帯に応じて画面の明るさを自動調整する機能があり、これもまた目の疲れを軽減できます。好みがどうであれ、テーマを調整して、自分に最適な設定を見つけることをおすすめします。

設定を変更するのはエディタだけではありません。お使いのオペレーティングシステムも、自身のニーズに合わせて調整できる可能性が高いです。デフォルトの設定が必ずしも最適とは限らないため、好みに合わせて微調整してみてください。ドックの配置を変えてみたり、拡大率を変更したり、背景画像を自分にとって心地よいものに変更したりしてみてください。

作業環境をカスタマイズするための無料または低価格のユーティリティが数多く存在しています。クリップボード管理ツール（https://oreil.ly/nMxA7）から、よりきめ細やかなサウンドコントロール（https://oreil.ly/6dMwg）、さらにはメニューバーの配置を変更できるアプリ（https://oreil.ly/6bztj）まで、日々の作業の煩わしさを解消する手段は、ほんの少しのインストール作業で手に入ります。同僚のメニューバーに見慣れないツールがあれば、ぜひ尋ねてみてください。そんな質問ひとつで、月に数時間の時間を節約できるかもしれません。インターネットで検索すれば、読者のツールボックスに加えられるようなツールが数多く見つかるはずです！

†1　気に入ったテーマを見つけても、少し手を加えたいと思うかもしれません。既定の設定に妥協する必要はありません。

物理的な環境も調整できることを忘れないでください。椅子やモニターの高さを、体が自然な姿勢を保てる快適な位置に調整しましょう。スタンディングデスクを好む方もいます。また、1日に何度でも姿勢を変えられる優れた製品が数多くあります[†2]。会社に人間工学の担当部署がある場合は、ワークスペースの評価を依頼し、より良い椅子や高さ調節可能なデスクを遠慮なくリクエストしてください。在宅勤務の場合、会社から機器購入のための手当が支給されることもあります。もし支給されない場合でも、質の良いデスクと椅子に投資することをおすすめします。そうすれば、腰の負担が軽減され、きっとその恩恵を体感できるはずです[†3]。

目の疲れは確実に現れるものであり、一日中画面を見つめつづけることは問題を引き起こすおそれがあります。日中はこまめに休憩を取り、モニター以外の遠くの物を見るようにしましょう。眼鏡をかけている場合は、眼科医に画面を長時間見ていることを伝えてください。さまざまなコーティングや特殊なレンズなど、目の疲れを和らげる選択肢はたくさんあります。中には、コーディングをする際に使うために、度数を調整した別の眼鏡を用意している開発者もいます。

> **もっと良いキーボードを購入する**
>
> キーボードについては、より良いものを購入することをためらわないでください。おそらく、会社が一括購入したものは、読者には合わないでしょう。ErgoDox EZ のような人間工学に基づいた分割キーボードは、自由にカスタマイズできるため、日々の作業を大きく変えてくれます。好みのキースイッチを自由に選べるのですから、自分に最も合うものを使うべきです。多くの企業では、相談すれば費用の一部または全額を負担してくれます[†4]。

11.1.3　コマンドラインの力を活用する

ウィンドウシステムや WYSIWYG（What You See Is What You Get、見たままがそのまま出力される）型のコンピュータ環境が登場する以前、コンピュータは純粋なテキストベースのシステムでした。難解で不可解、そしてしばしば融通の利かないコマンドを使わなければなりませんでした。モダンなコンピュータシステムを最大限に活用すべきですが、ソフトウェアエンジニアとしてコマンドラインインターフェース（CLI、Command-Line Interface）にも慣れておく必要があります。

コマンドラインは最初は難しそうに思えるかもしれませんが、CLI はすぐに強力な武器となり、作業効率を飛躍的に高められます。一般的な作業の多くが、ほんの数回のキー操作で非常に効率的に行えるからです。週に数分でも、新しいコマンドを試してみてください。

---

†2　もし、原因のわからない痛みや不快感に悩まされているなら、座っている姿勢から立っている姿勢に切り替えるだけで、症状が和らぐかもしれません。

†3　高級な椅子やデスクの価格は一見すると高額に感じるかもしれませんが、上質な家具は長く使えます。

†4　反復性ストレス障害による痛みが表れるまで我慢しないようにしましょう。何か問題を感じているなら、より良いキーボードを使うことで驚くほどの効果が得られるかもしれません。



マニュアルはコマンドラインに組み込まれていることをお忘れなく！ 何かがどのような機能を持つのかわからない場合は、man コマンドを使って調べてみてください。

シェルコマンドは頼もしい味方です。cd、pwd、ls、mkdir といったコマンドには馴染みがあるかもしれませんが、他にも数多くのコマンドが利用可能です。コマンドラインから、open コマンドを使えば、アプリケーションを簡単に起動できます。cat、cut、grep、pbcopy といったコマンドを使えば、貴重な時間を節約できます。もちろん、これらはすべてパイプ（|）コマンドを使って組み合わせられます。パイプを使えば、単純で単一の目的を持つツールを連結し、より複雑な出力を得られます。例えば、Git リポジトリの URL を取得したいと思ったことはありませんか？ そんな時は、cat .git/config | grep url | cut -f2 -d= | pbcopy が役立ちます。

パソコンに話しかけてもらいたいと思ったことはありませんか？ それなら、これを試してみてください。

```
say I can't let you do that Nate.
```

他にもさまざまな CLI ツールがあります。構文ハイライト機能付きの cat をお探しではありませんか？ それなら bat を試してみてください（https://oreil.ly/g15BD）。見やすい diff 表示をお求めですか？ diff so fancy をチェックしてみてください（https://oreil.ly/BeBJp）。JSON の解析に多くの時間を費やしていませんか？ そんな時は、ツールセットに fx を加えてみてはいかがでしょうか（https://oreil.ly/PHrKh）。より見やすい色使いの ls をお探しですか？ それなら exa をインストールしてみてください。

多くの開発者は、コマンドラインで頻繁に行う操作のために、時間を割いてシェルのエイリアスを作成しています。シェルの設定ファイル（例えば、bash の場合は~/.bashrc、Z シェルの場合は~/.zshrc など）に 1〜2 行追加するだけで、数え切れないほどのキー入力を省くことができます。これらは、自分の設定が反映されているマシンで作業するときは非常に役立ちますが、それ以外の場合はその設定は使えません。うまく使うようにしてください。頻繁に行う作業や、入力に時間がかかる作業には、ぜひエイリアスを活用してください。以下に例を示します。

```
alias k=kubectl

alias d='docker'

alias dc='docker compose'
```

シェルの履歴機能もお忘れなく。シェルの履歴機能は、コマンドライン作業において、あまり評価されていない機能の一つです。しかし、履歴機能を使えば直近のコマンドをスクロールして確認したり、Tab キーを押して最近実行したコマンドを自動補完したりすることもできます。history

コマンドを実行すると、これまでに実行したコマンドの一覧が表示されます。また、Ctrl-R キーを押すと、コマンド履歴を逆方向に検索できます。

これらはすべて調整可能です。履歴の保存容量を増やしたり、特定のコマンドを無視したりすることもできます。活用できるシェルハック（https://oreil.ly/PueTb）は数え切れないほどあります。設定にファジーファインダー（https://oreil.ly/HHYdF）を追加すれば、同じコマンドを入力する必要がなくなる可能性も十分にあります！

最後に、OS のデフォルトのシェルだけにこだわらないようにしましょう。Oh My Zsh（https://ohmyz.sh/）など、優れたシェルが数多くあります。Oh My Zsh には、便利な組み込み機能が多数あるだけでなく、何千ものプラグイン、テーマ、補助ツールが用意されています。

11.1.4　キーボードショートカットの力を活用する

モダンなオペレーティングシステムは、マウス、トラックパッド、あるいは指といったポインティングデバイスの力を活用しています。これにより、確かにコンピュータは格段に手軽になりました。しかし、目標とすべきは、常に両手をキーボードのホームポジションに置いておくことです[†5]。キーボードショートカットを覚えましょう。ショートカットは、操作が速く効率的であるだけでなく、マウスやトラックパッドに手を動かす手間を省くことができます。また、ショートカットを使うことで、手首や手への負担も軽減されます。



タッチタイピングができないなら、ぜひ習得すべきです。本当に習得すべきです。スマートフォンは、ある世代に親指でのタイピングを教えました。友人との会話には便利ですが、開発者としてコードを書く際には、すべての指を使うべきです。タッチタイピングは速く、手や目、首への負担も少なくなります。学校でタイピングを習わなかった方も多いでしょうが、タッチタイピングの学習プロセスをゲーム化したオンライン教材は数多くあります。ぜひタッチタイピングを習得してください。きっと恩恵を享受できるはずです。お約束します。

キーボードショートカットを習得することは、初心者と熟練者のわかれ目であり、すぐに作業効率を飛躍的に高めてくれます。使用しているエディタのショートカットを覚えるだけでなく、オペレーティングシステム自体のショートカットも習得しておくべきです。

ショートカットの中には、無意識のうちに手が動いてしまうほど体に染みついているものもあります。ほとんどのコンピュータユーザーは「Cmd-C」や「Cmd-V」といったショートカットには慣れ親しんでいますが、実はそれ以外にも数多くのショートカットが利用可能です。今、ブラウザウィンドウをいくつ開いていますか？ 最前面のアプリのすべてのウィンドウを表示したい場合は、Ctrl キーを押しながら下矢印キーを押せば表示できます（macOS の場合）。最前面のアプリケーションを非表示にしたいですか？ Cmd-H が役立ちます。IntelliJ では、任意の 2 つのファイルを選択して Cmd-D を押すと、その 2 つのファイルの差分が表示されます。これは JAR ファイルや

†5　**ホームポジション**とは、タッチタイピングの際に指が自然に置かれるキーの列のことです。ほとんどのキーボードには、人差し指を置くべき位置を示す突起が付いています。

ZIP ファイルでも同様に機能します。

多くのアプリケーションやオペレーティングシステムでは Emacs のキーバインドが利用可能であり、その多くは、特にテキスト（例えばコードファイルなど）内を移動するときに非常に役立ちます。1 文字ずつ前後に移動するには、Ctrl+F と Ctrl+B を使用します。また、Meta+F と Meta+B を使用すれば、それぞれ単語単位で前後に移動することも可能です[†6]。行の先頭や末尾に移動するには、Ctrl-A と Ctrl-E を使用します。ほんの数分かけて Emacs のキーバインド（https://oreil.ly/nBVek）をいくつか覚えるだけで、大幅な時間の節約につながるでしょう。

Emacs 対 vi

プロジェクトルームで本当に大論争を巻き起こしたいなら、Emacs 対 vi の議論を始めてみてください。冗談抜きで、タブ対スペースの議論などかわいらしい問題に思えてきます。「Emacs は素晴らしいオペレーティングシステムだが、まともなエディタだけが欠けている」、「Vim は素晴らしいエディタだが、まともなウィンドウマネージャがまったく欠けている」とだけ言っておきましょう。覚悟して臨んでください。

とはいえ、vi の基本を学んでおけば非常に役立つことがあります。場合によっては、エディタが vi しか使えない状況もあるからです。また、繰り返しになりますが、Emacs のキーバインドを習得すれば、ファイル操作の達人になれるかもしれません。

最もよく使われるキーボードショートカットの組み合わせである「Ctrl-C」や「Ctrl-V」でさえ、さらに効率化できる可能性があります。**図 11-1** に示すように、Pastebot（https://oreil.ly/lr61H）のようなツールを利用すれば、貼り付け履歴にアクセスできます。あるアプリケーションから別のアプリケーションへ、5 つの項目をコピーする必要があったことは何度ありましたか？ ウィンドウを切り替えながら作業するのは、楽しいものでしたか？ おそらくそうではなかったでしょう。Pastebot のようなツールを使えば、コピー＆ペーストを一括で行うことができ、大幅な時間の節約になります。また、Pastebot では履歴をさかのぼって（パスワードやその他の機密情報には注意してください）、その日の早い時間にコピーした内容を再度貼り付けることも可能です。

テキストエクスパンダーも、生産性を高める優れたツールの一つです。TextExpander（https://oreil.ly/geQcT）のようなスタンドアロンツールは、生産性を飛躍的に向上させることができます。また、多くのオペレーティングシステムには、ユーザーが自由にカスタマイズできるテキスト置換機能が標準で搭載されています（**図 11-2** 参照）。繰り返しになりますが、こうした小さな時間の節約を積み重ねていくのです。

これまで説明してきたように、お使いの環境にはカスタマイズオプションが豊富に用意されています。デフォルトの設定に妥協する必要はありません。ツールに合わせるのではなく、ツールを自

†6 Meta キーは、お使いのオペレーティングシステムによって、Alt キー、Caps Lock キー、Escape キーのいずれかに割り当てられるのが一般的です。

分に合わせるべきなのです。環境の調整に費やす時間は、生産性への投資となります。



図 11-1　Pastebot を使えば、クリップボードの履歴にアクセスできる



図 11-2　テキスト置換オプションを活用すれば、作業効率が向上する

11.2　戦略的な自動化

一日中、他人のためにコードを書いているのですから、自分のためにもコードを書くようにしましょう。ほんの数行のコードが、最終的には何時間もの作業時間を節約してくれるかもしれません。また、他の言語や技術を試してみる絶好の口実にもなります。多くの開発者は正規表現に対して愛憎入り混じった感情を抱いていますが、ほんの少しでも正規表現を学んでおくと、非常に役立つことがあります。

決して忘れないでください。コンピュータサイエンスにおいて、重要な数字は 3 つしかありません。0 回行うこと、1 回だけ行うこと、そして n 回行うことです。もし何かを 2 回以上行うのであれば、それは数え切れないほど繰り返すことになるでしょう。2 回以上行う作業については、時間をかけて自動化することを検討すべきです。



自動化を追求することは、危険なほど時間を浪費する原因になりかねません。場合によっては、自動化するコードを書くのに、その作業自体を行うよりも時間がかかってしまうこともあります。もしその作業が本当に 1 度きりのもので、2 度繰り返す可能性が低いのであれば、その作業を自動化するために時間を費やすのは得策ではありません。その作業を数回ほど行ってから、完全に自動化しようとするのがよいでしょう。その場合でも、必ず作業に時間制限を設けてください。最終的には、読者自身の判断で適切に決めてください。

そのためには、スクリプトを作成しましょう。たった 30 行の Python コード（https://oreil.ly/iCp0Q）で、何時間も節約できるかもしれません。この話の教訓は、自動化は読者の味方だということです。もっと多くの作業をコンピュータに任せましょう。もし自分でスクリプトを書くのが面倒なら、AI に書いてもらうという手もあります！ コードを読み、テストすることは必要ですが、自動化へのハードルはかつてないほど低くなっています。

11.3　生産的な習慣を追い求める不断の努力

ショートカットやユーティリティ、コマンドの数は実に膨大で、そのすべてを覚えるのは気が遠くなるような作業に思えるかもしれません。1、2 か月ですべてをマスターしようとは考えないでください。これは継続的な取り組みなのです。目標は、毎日少しずつ上達することです。週に 1 つ、新しいことを学ぶように心がけてください。この方法なら、取り組みやすく、継続しやすいだけでなく、学んだことをしっかりと身につけることができるでしょう。

11.3.1　共同学習

ヒントやコツを得るのに最適な情報源の一つは、同僚です。チームメンバーに生産性向上のための工夫について尋ねる習慣をつけましょう。もし同僚が、自分がこれまで見たことのないツールやコマンド、ショートカットを使っているのを見かけたら、ぜひ尋ねてみてください。知らないから

といって気後れする必要はありません。同じように、チームメンバーが手間のかかる方法で作業をしているのを見かけたら、もっと簡単な方法を教えてあげてください。その知識を定着させるために、3〜4回ほど繰り返して説明するようにしましょう。もし同僚が、読者が愛用しているツールの使い方がわからないようであれば、使い始められるよう手助けをしてあげてください。そうすることで、自分の知識がさらに定着するだけでなく、自分にとっても使い慣れた環境を広められます[†7]。

毎週数分間、自分のツールについて新しいことを学ぶ時間を設け、学んだことを友人や同僚と共有するようにしてください。ペアプログラミングをしている際、相手が自分にとって初めて見るツールやユーティリティを使っている場合は、教えてもらうよう頼んでみましょう。読者の知っているショートカットを相手が使っていない場合は、その使い方を教えてあげてください。皆が自分と同じ知識を持っているとは限らないことを念頭に置き、自分が使っている便利なものを共有する場として、不定期でも「ランチタイム勉強会」を開催することを検討してみてください。同様に、自分がよく知っているツールに関するセッションやウェビナーも、自分には学ぶことが何もないと思い込んで欠席したりしないでください。

このアプローチは、プログラミング言語にも当てはまります。言語は進化しつづけるものですので[†8]、コーディングのスタイルもそれに合わせて進化させる必要があります。新しいバージョンがリリースされたら、リリースノートを確認し、時間をかけて新機能を学びましょう。学んだことを他の人に教え、その新しいアプローチを実践する機会を常に意識してください。勉強会を主催してみましょう。「見て、やって、教える」というアプローチは効果的です。

11.3.2 個人のナレッジマネジメント

テクノロジーの変化は信じられないほど速く、その結果、私たちは情報に溺れることになります。ポッドキャストからブログ記事、動画、チュートリアルに至るまで、自分なりのナレッジマネジメント戦略を立てておくことが重要です。複雑である必要も、専用のツールを導入する必要もありません。ただ、後で活用できるよう、興味を持った情報を保存しておく場所を確保しておくべきです。自分の周囲に流れる情報を確実に捉え、整理できるよう準備を整えておく必要があります。ある質問に対する正確な答えを**確かに知っていたはず**なのに、それがどこにあったか思い出せないことほど、もどかしいことはありません。

メモを取っておきましょう。その問題には、**間違いなく**また直面することになるでしょう。チームメンバーも同じです。書き留めておきましょう。次回には忘れてしまいますから。頭の中から外に追い出しましょう。外部記憶装置こそが勝利の鍵です。方法は問いません。形式ばったものでなくてもかまいません。クリップで留めた雑多な紙切れでも？ それでもかまいませんが、電子データのほうが検索は間違いなく簡単ですし、紙だと同僚と結果を共有するのが難しい場合もあります。

開発者の中には、ノートPCにメモやドキュメントを保存する人もいれば、Field Notes

†7 チームは、ペアプログラミングを円滑に行うために、ツール面での一定の制限を定めることがあります。

†8 多くのプログラミング言語やフレームワークでは、コミュニティへの貢献が奨励されています。ぜひ積極的に参加して、自身の開発環境の未来を形作ってみてください。

（https://oreil.ly/CtA8_）や Moleskine（https://oreil.ly/4-Nr-）のようなシンプルな手帳と、Mark Two（https://oreil.ly/HJ9U2）や Space Pen（https://oreil.ly/spenn）のようなポケットサイズのペンを好む人もいます。また、Org Mode（https://oreil.ly/B45xe）、Evernote（https://oreil.ly/nnd8U）、Notion（https://oreil.ly/U9i4U）といった、より技術的なソリューションを選ぶ人も少なくありません。さらに、1 日のうちに見つけたさまざまな Web 上のリソースをどのように扱うかも検討してみてください。（整理された）ブックマークをデバイス間で同期させるのも一つの方法ですが、Pinboard（https://oreil.ly/OLAtu）のような専用のブックマーク管理ツールを利用するのもよいでしょう。

Notion

Dan です。ソフトウェアエンジニアであり、整理整頓が大好きな私ですが、これまで数多くのナレッジマネジメントツールを試してきました。中でも Notion が私のお気に入りです。最小限でも最大限でも、必要な分だけを柔軟に使えるところが気に入っています。ここでは、ソフトウェア開発者として私が Notion をどのように活用しているか、その一部を紹介します。

学習

新しい言語やフレームワーク、ツールを学ぶ際に、私は調べた情報をすべて Notion にまとめています。メモやコードスニペット、画像、参考資料などを組み合わせることで、学習の効率を上げています。

ドキュメント作成

新しいプロセスや機能をドキュメント化する際は、まず Notion から始めます。その自由形式のエディタは Markdown に対応しており、リアルタイムプレビュー機能も備えているため、ドキュメント作成が効率化されます。

チェックリスト

Notion に、コードレビュー用など、さまざまなチェックリストを保存しています。レビューの際に必要なポイントをすべて網羅できているかを確認するのに役立っています。

メモ取り

Notion は私のメモの中心的な保管場所となっています。Notion では、自分に合った方法でメモを整理できます。モバイルアプリを使えば、外出先でもメモにアクセスでき、Notion Clipper というブラウザ拡張機能を使えば、後で参照するために Web ページを保存しておけます。

Notion の真の強みは、その汎用性とカスタマイズ性にあります。Notion を使えば、自分だ

けのニーズやワークフローに合わせたワークスペースを構築できます。Notionは私にとって「第二の脳」のような存在であり、エンジニアとしての生活において、Notionなしでは仕事になりません。

とはいえ、この分野にはツールが数多く存在します。Notionは私のワークフローにおいて欠かせない存在ですが、人によって向き不向きはあるでしょう。自分に合ったツールを使うようにしてください。

カンファレンスに参加することは、新しいスキルを身につけ、既存のスキルを磨くための非常に貴重な機会です。学んだことを今後の学習に活かすためにも、その内容を記録しておきましょう。多くのカンファレンスでは、イベント終了後にプレゼンテーションの動画を公開しています。少し時間をかけて、興味のある講演を選び、後で簡単に再生したり振り返ったりできるように、自分だけの動画プレイリスト（https://oreil.ly/Vzym9）を作成しましょう。開発者の中には、こうした講演をポッドキャストのように扱い、散歩中や運動中、通勤中に聴くためにリストに追加する人もいます。再生速度を1.0倍より速く設定すれば、より多くのコンテンツを短時間で消化できるでしょう。

どのようなアプローチをとるにしても、特定のデバイスだけに縛られないようにしてください。ポータブルで、モバイル環境でも使えるものであるべきです。いつインスピレーションが湧き、仕事上の問題に対する重要な気づきが得られるかはわかりません。その瞬間を逃さないよう、準備をしておきましょう。また、友人が素晴らしい本やポッドキャストを勧めてくれるのもいつのことかわかりません。その情報をすぐに記録できるように、準備をしておきましょう。

Keep It Simple（シンプルなままにする）

Nateです。シンプルであることに何の問題もありません。私がこの仕事を始めた頃に、バインダークリップで束ねたメモ用紙を何枚も持っていた同僚がいました。彼はそこに、長年にわたって集めたさまざまなコマンドやヒント、手順、その他のメモを走り書きしていました。彼に質問をすると、そのメモ用紙をめくって、まさに求めていた情報を教えてくれることが多くありました。彼のナレッジマネジメントシステムはシンプルなものでしたが、彼にとってはうまく機能していたのです。そして、それが重要なのです。

11.4　まとめ

優れたソフトウェアエンジニアは、自分のツールキットを自在に使いこなします。新車に乗ったときに、シートやミラー、ハンドル、温度設定を数分かけて調整するように、開発環境も自分自身のニーズに合うように調整するべきです。デフォルトの設定をそのまま受け入れるのではなく、エ

ディタやオペレーティングシステムを自分好みにカスタマイズしましょう。熟練ユーザーは強力なツールを使います。キーボードショートカットやコマンドラインは、生産性を飛躍的に高めてくれます。無料や低価格のユーティリティが、日々の業務をどれほど変えてくれるか、きっと驚くことでしょう。新しいものを試すようにしてください。

これらすべてのショートカットやユーティリティを把握するのは大変ですので、記憶に頼らないようにしましょう。ツールキットを充実させながら、ナレッジマネジメントのスキルを磨いていきましょう。同僚から学び、また同僚と教え合うことも大切です。

11.5 実践に移す

生産性を高めるために新しい方法を試してみてください。ただし、その試みが時間の無駄にならないよう注意してください。繰り返し行うような作業を選び、その手間を自動化できないか検討してみましょう。既存の自動化システムを改良し、より重要な業務に対応できるようにすることも考えてみてください。そして、そこで得た知見を同僚と共有しましょう。

今後数か月間、週に 1 つ新しいキーボードショートカットを覚えるように心がけましょう。

金曜日の午後、15 分ほど時間を取って、お気に入りのエディタのドキュメントをじっくり読んでみてください。今まで知らなかったことは何か学べましたか？ エディタやターミナルの「Tip of the Day」機能を有効にしてみてください。どんな機能に驚きましたか？

自分自身のナレッジマネジメントのやり方を見直してください。うまく機能していますか？ あまり機能していないようであれば、別の方法を試してください。複雑である必要も、高機能なツールを導入する必要もありません。ただ、自分自身に合っていればそれでよいのです。

新しいコマンドを覚えたら、書き留めておきましょう！ 外部記憶を活用しましょう。

11.6 関連リソース

- 『Flow: The Psychology of Optimal Experience』Mihaly Csikszentmihalyi 著（Harper & Row、1990 年）
  和書『フロー体験：喜びの現象学』（世界思想社、1996 年）
- 『The Productive Programmer』Neal Ford 著（O'Reilly、2008 年）
  和書『プロダクティブ・プログラマ』（オライリー・ジャパン、2009 年）
- 『Building a Second Brain: A Proven Method to Organize Your Digital Life and Unlock Your Creative Potential』Tiago Forte 著（Simon Element / Simon Acumen、2022 年）
  和書『SECOND BRAIN』（東洋経済新報社、2023 年）
- 『The Passionate Programmer, 2nd Edition』Chad Fowler 著（Pragmatic Press、2009 年）
  和書『情熱プログラマー』（オーム社、2010 年）

12章
学び方を学ぶ

明日死ぬかのように生きなさい。永遠に生きるかのように学びなさい。
— Mahatma Gandhi

テクノロジー業界に長く携わっている方なら、テクノロジーは絶えず変化しつづけるという現実をご存じでしょう。つまり、定期的に新しいことを学ぶという課題に直面することになります。多くのエンジニアは、自分はコードを書くために給料をもらっていると考えています。それは事実です。しかし、**考えること**に対しても報酬が支払われているのです。頭脳を使って生計を立てているのですから、知識を吸収して脳を活性化させなければなりません。つまり学びつづける必要があるのです。

新しいことを素早く習得する能力こそが、成功するエンジニアにとって最も重要なスキルであると断言できるでしょう。さらに、新しいことを理解するだけでなく、学んだことを実際の現場で活用することも求められます。「見て、やって、教える」というアプローチが、多くの人にとって効果的です。

「新しいものへの憧れ」というパラドックス

ソフトウェアエンジニアは、蛾が炎に飛び込むように「新しいもの」に惹かれます。何かが初めて登場した際には、すぐにそれを取り入れなければ取り残されてしまうかのような、一時の熱狂的な盛り上がりが見られることがよくあります。しかし、登場したばかりのテクノロジーは、まだ実用段階に達していないかもしれません。未熟な部分が残っているのです。時が経つにつれ、その熱狂は冷めていきます。しかし、その頃こそ、その新しいテクノロジーが導入に適した状態になっていることが多いのです。決して忘れないでください。新しいテクノロジーはバスのようなものです。15 分もすれば、また次のものがやってくるのです。

12.1　詰め込み学習は効果がない

おそらく、学生時代には、重要な試験のために一夜漬けで勉強した経験があるのではないでしょうか。そして、少なくとも短期的には、その方法は成功したかもしれません。しかし、その知識は定着しましたか？ 何かを真に学ぶためには、情報を「エンコード」する必要があります。つまり、その情報は詳細で、意味があり、コンテキストを伴っていなければなりません。物語は非常に効果的であり、何千年もの間、教えたり知識を共有したりするために用いられてきました。多くの場合、学習には繰り返すことがが必要です。適切な間隔を空けて繰り返すことで、知識や記憶の定着が促進されます。

エンジニアである読者にとって、こうした物語はどのような意味を持つのでしょうか？ 例えば`rm`のようなコマンド、特に`-f`オプションを使う際は十分に注意するようにと誰かに言われるのと、`rm -f`を含むスクリプトを作成中に、うっかり自分のノートPCのハードドライブを消去してしまったという体験談を誰かが共有してくれるのとでは、まったく次元が異なります[†1]。

どんなに努力しても、物事を忘れてしまうことはあります。しかし、これは実は「欠点」ではなく、「機能」なのです！ 脳は、もはや必要のない情報を排除するのが驚くほど得意なのです。一部の記憶は決して消えないかもしれませんが[†2]、あっというまに消えてしまう記憶もあります[†3]。脳は、いわば積極的に「ゴミ出し」を行っているのです。情報を忘れていくことは避けられない現象ですが、人によって、あるいは情報によってその度合いは異なります。コンピュータは一定の間隔で質問を繰り返すことで、記憶の定着を支援できます（新しい言語の習得に成功している人は、この手法を、高度なゲーミフィケーションと組み合わせています）。読者の中には、ローテクながらも非常に効果的なフラッシュカードを試された方もいるでしょう！

何かで悩んでいるときは、壁に頭を打ちつけるように考えつづけるよりも、ひとまずそのことを脇に置いておくほうがよい場合もあります。シャワーを浴びたり、散歩に出かけたり、夕食を作ったりするといった単純な行動が、気を紛らわせて答えを引き出すきっかけになることがあります。脳にはRモードとLモードという2つの異なる処理モードがあります。**Rモード**は、非言語的で、情報を検索・呼び出す脳の機能です。これはバックグラウンドで動作します。Rモードは、直接制御することはできないため、予期せぬ結果をもたらすことがあります。映画を観ているときに、ある俳優の名前がどうしても思い出せなかったのに、5分後にふと頭に浮かんだという経験があるでしょう。これこそが、Rモードが働いている証拠なのです。

一方の**Lモード**は、言語的、分析的、直線的であり、論理と計算に重点を置きます。Lモードの注意をそらすことで、Rモードが本来の能力を発揮できるようになります。問題について考えた後、シャワーを浴びたり、家の掃除をしたりといった日常的な作業を行ってみてください。

エンジニアとして、この2つの処理モードをうまく活用してください。問題に行き詰まってしまった場合、無理に突き進むことが必ずしも最善の解決策とは限りません。その問題を一旦脇に置

†1　筆者自身が、実際にそのような経験をしているかどうかは定かではありません。
†2　例えば、若い頃の電話番号や、小学校のときの大好きな先生の名前などです。
†3　お名前は何とおっしゃいましたっけ？

いて、もっと日常的な作業（散歩など）をしてLモードの意識をそらし、様子を見てみましょう。多くの場合、解決策がふと思い浮かぶものです。

12.2 スキルの習得

> 自分への投資こそが、最も価値のある投資です。学べば学ぶほど、収入も増えます。
> — Warren Buffett、米国の投資家・慈善家

スキル習得にはいくつかのモデルがありますが、それらはすべて似たような概念に基づいています。武道に由来する例として、**「守・破・離」**が挙げられます。Bruce Leeがかつて語ったように、「原理を学び、原理に従い、そして原理を解き放つ」というものです。別の考え方としてClark Terryによる「模倣、同化、革新」があります。「守」または「模倣」の段階では、生徒は指導者の動きを忠実に真似ます。ここでは、その理由や根底にある理論をあまり気にせず、指導者の動きを再現することに重点が置かれます。

「破」の段階では、活動の幅が広がり始めます。基礎を理解し、主要な動きを繰り返すのに十分な筋肉の記憶が身についています。ここで初めて理論を補完し始めます。他の指導者や情報源を探し求め、それらを自らの理解に組み込んでいくようになります。「離」の段階では、本質的には自分自身から学んでいることになります。他人が定めた道に従うのではなく、自らの実践そのものが学びを牽引していくのです。

ドレイファスモデルは、この概念をさらに発展させ、学習を5つの段階（**図12-1**参照）に分け、それらが段階的に進行していくものとしています。「初心者」の段階では、学習者は「レシピ」を必要とします。つまり「**まさにこの通りに**やってください」という指示です。新しいスポーツを習う場合、この段階では、コーチが読者の足の位置を何度も動かしたり、同じ動作を延々と繰り返させたりすることがあるでしょう。「初級者」段階では、こうした厳格なルールを離れ始めます。グリップを変えたりスタンスを調整したりしたときに何が起こるか、試行錯誤し始めるのです。

「中級者」段階では、トラブルシューティングを始められるようになります。ボールの軌道を見ることで、自分のミスが何だったのかわかるようになるでしょう。「上級者」段階に到達すれば、自分で修正できるようになります。ミスを視覚的または感覚的に把握し、コーチに指摘されなくても修正できるようになります。十分な時間をかけて練習すれば「熟練者」の段階に到達し、すべてが直感でできるようになります。ゴルファーとして、「このボールをカットショットしたい」とただ思うだけで、カットショットできるのです！ 実際、この段階では、何をどのようにしてやり遂げたのかを説明するのは難しいでしょう。ただ、それが正しいと感じられるのです。

ゴルフクラブを手に取ったばかりの人に、「カットショットを打つことを意識して打て」と言っても、あまり効果がないことは明らかでしょう。始めたばかりの頃は、段階的な指導が必要です。だからこそ、多くのプログラミング言語やフレームワークには、「はじめに」というチュートリア

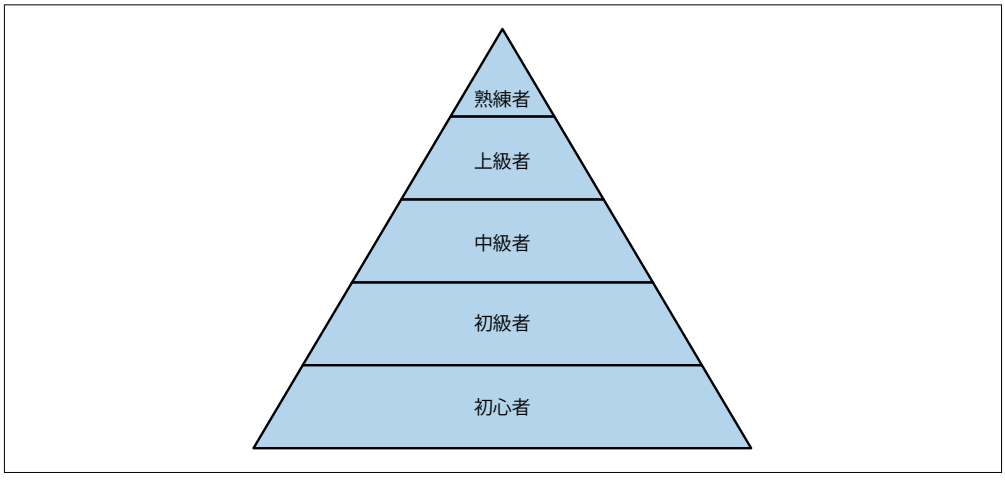


図12-1　ドレイファスモデルでのスキル習得過程

ルが用意されているのです[†4]。ルールや手順は初心者にとって不可欠です。一方で、熟練者の場合には、それらは熟練者が苦労して培った直感を阻害してしまうため、かえってその成長を妨げてしまうこともあります。そして、ショートゲームを上達させる場合と同じように、単に動画を見たりコツを読んだりするだけでは限界があります。練習しなければなりません！

新しいフレームワークを学ぶ際のことを考えてみてください。いきなり複雑なアプリケーションの開発に取り掛かりますか？ おそらくそうではないでしょう。誰もが知るところの「Hello World!」というサンプルから始めるはずです[†5]。最初はチュートリアルからコードをそのままコピーし、エラーのトラブルシューティングには検索やチャットボットに頼るでしょう。しかし、時間が経つにつれて直感が養われ、パターンを見分けられるようになります。エラーメッセージを見ただけで、どこに問題があるかがわかるようになるのです。さらに時間が経つと、そのフレームワークの限界や、可能なら変更したい部分が見えてくるでしょう。マスタリーに近づくにつれ、もはや自分の意図をコードに落とし込むのではなく、フレームワークを通じて自分の意図を表現するだけで済むようになります。課題を与えられたら、その解決策が「見える」ようになるのです。

12.3　学習の習慣

> プログラマとして最も重要なスキルは、新しいことを効果的かつ効率的に独学で身につける能力です。プログラマは常に成長しつづけ、新しいテクノロジーを習得していくことになります。その能力こそが、個々のツールやテクノロジーよりもはるかに重要なのです。
>
> — Ali Spittel、ソフトウェアエンジニア

†4　それに、「ToDo リスト」が必要ない人なんていないでしょう？
†5　これまでに、いくつ ToDo リストアプリを作りましたか？

はっきりと定まった決まりはありませんが、エキスパートになるにはかなりの時間がかかります。そのため、多くの人は中級レベルを超えることができません。テクノロジーの進化のスピードを考えると、これは深刻な問題です。テクノロジーの進化は**急速**です。次の大きなトレンドに移るまでに、一つのテクノロジーにどれだけの時間を割けるでしょうか？ とはいえ、キャリアを成功させるために、ソフトウェアのあらゆる側面について専門家になる必要があるというわけではありません。キャリアを通じて、ある分野では専門知識が深まる一方で、他のスキルは中級者レベルにとどまることもあるでしょう。しかし、それには何の問題もありません！

重要なのは、限られた学習時間を最大限に活用することです。探求したいことをリストアップし、今後数か月で学びたいことを優先順位づけしましょう。このリストや優先順位は流動的なものです。興味が変われば、学習目標も変わっていくでしょう。

すべての学習において、同じレベルの集中力や注意力が求められるわけではありません。追求する深さは、テーマによって異なります。Christopher Judd（https://oreil.ly/OrgJ2）は、このアプローチを「学習の深さ戦略（Learning Depth Strategy）」と呼んでいます。

Survey（調査）

ポッドキャストを聴いたり、ユーザーグループやカンファレンスのセッションに参加したり、技術カンファレンスの基調講演を見たり、リリースノートに目を通したりしましょう。

Shallow dive（浅い潜水）

短い動画を見たり、ブログ記事や解説記事を読んだり、チュートリアルを進めたりしましょう。

Deep dive（深潜）

対面またはオンラインのワークショップや研修に参加したり、そのテーマに関する本を読んだりしましょう。

Ultra deep dive（超深潜）

そのテーマについてプレゼンテーションを行ったり、ワークショップや研修を主催したりしましょう。

人間は物事を忘れてしまうため、学習は習慣にしなければなりません。1880 年代、Hermann Ebbinghaus は記憶について研究し、人は学習してからわずか 30 分後には、学んだことの半分を忘れてしまうことを発見しました！ しかし、その内容に繰り返し触れることで、記憶の減退は大幅に遅くなります。彼は「間隔反復」という概念を提唱しました。これは、学習後に徐々に間隔を空けて復習を行うことで、概念が完全に忘れ去られる前に定着させるというものです。このアプローチを 1 年を通してどのようにスケジュールに組み込めるか、考えてみてください。

週単位

ポッドキャストを聴いたり、動画を 1〜2 本見たり、記事を 1〜2 本読んだりしましょう。

月単位

技術書やビジネス書を読んだり、ユーザーグループやミートアップに参加したり、オンライン講座を受講したりしましょう。

年単位

技術カンファレンスに参加したり、新しいプログラミング言語を学んだり、サイドプロジェクトに取り組んだり、資格を取得したり、技術的なプレゼンテーションを行ったりしましょう。

また、「声に出して学ぶ」ことも効果的です。進捗状況を SNS に投稿してみてはいかがでしょうか。そうすれば、同じ志を持つ人々のコミュニティが見つかり、彼らから支援や励ましを得られることでしょう。新しい発見があった際は、学んだことを動画や記事にまとめてみてください。将来、その質問について検索した際に、自分が作成したコンテンツがヒットしても驚かないでください[†6]！ 時間を割いて、自分の取り組みを記録するようにすれば、きっと良かったと思えるはずです。

テクノロジーのメリーゴーラウンド

Nate です。キャリアの初期の頃、当時のマネージャーと話していたとき、なぜ技術職から管理職に転向したのか尋ねたことがあります。すると、常に新しいことを学ばなければならないこと、つまり、テクノロジーの「メリーゴーラウンド」に乗っているような感覚に疲れてしまったのだと言われました。もう十分に新しいものを見てきたのだそうです！ 大学を卒業したばかりの当時は、その発言の意味を完全には理解できませんでしたが、今ではその気持ちがよくわかります。読者は、これまでのキャリアの中で、どれだけのプログラミング言語やフレームワーク、フロントエンドライブラリ、データストアを学んできましたか？

とはいえ、管理職が楽な道だと言っているわけではありません。人を導くには、数多くの特有の難題があります。しかし、ソフトウェアエンジニアであるということは、知識の幅をほぼ絶え間なく広げていくことを意味します。それは非常にやりがいのあることですが、その大変さを甘く見てはいけません。

学ぶべきことは常に尽きないことを踏まえて、どうすればそのペースについていけるでしょうか？ それを習慣にする必要があります。自分に合ったルーティンを確立しましょう。読書や探求のための時間をスケジュールに確保してください。時折、その時間を他の予定で埋めてしまうこともあるかもしれません。金曜の午後は一般的に空いているものですが[†7]、人によっては火曜日の午前中や木曜日のランチタイムが適しているかもしれません。結局のところ、自分の時間の管理は

†6　私たち筆者二人とも、自分が作成したことをすっかり忘れていた資料を偶然見つけた経験があります。
†7　金曜の午後に会議を設定する人は、敵対的な行為を行っていることになります。

自分自身が行わなければなりません。Trello のボードに 1 年分の学習計画を立てたり、カレンダーにリマインダーを設定したりする人もいます。自分に最も合う方法を試してみてください。ただし、どのような方法をとるにせよ、学習のための時間を明確に確保することが大切です。

筆者が**モーニングコーヒー**と呼んでいる習慣を、ぜひ取り入れてみてください。一日の最初の 15 分から 30 分間、お気に入りの飲み物（エスプレッソ、紅茶、エナジードリンクなど、何でもかまいません）を片手に座り、ニュースに目を通す時間を取ってみてください。その「ニュース」が読者にとって何を意味するかは自由です。SNS を閲覧したり、ポッドキャストを聴いたり、チュートリアルをこなしたり、新しい技術書の序盤を読んだりすることでもかまいません。しかし、これを習慣にし、一日があっという間に過ぎてしまう前に実行してください。これまでにも、緊急対応の連続ですっかり時間が吸い込まれてしまったような日があったことでしょう！ このモーニングコーヒーの時間を優先することは、忙しすぎて何も成し遂げられなかったと感じるような日でも、救いになるはずです。少なくとも、何か新しいことを学んだことになりますから！

12.4 AI を活かした学習

AI は学習の助けとなります。実践的な経験に代わるものではありませんが、チャットボットはトピックの調査に役立ちます。チャットボットは、個々のニーズに合わせた学習パスを構築したり、テクノロジーについて会話形式でわかりやすく深く掘り下げたりする、素晴らしいリソースとなります。例えば、あまり使ったことがない Java のクラスを使用する必要がある場合や、リファレンスモデルについてさらに詳しい情報を得たい場合などを考えてみましょう。もちろんドキュメントを読むこともできますが、**図 12-2** と**図 12-3** に示すように、AI にその作業を代行させることもできます。



図12-2　クラスを説明するチャットボットの例

ただし、AIの「ハルシネーション」には注意が必要です。必ず出典を確認するようにしてください。それでも、新しい情報を学んだり、過去に学んだ概念を復習したりするのに、AIが役立つことは間違いありません。

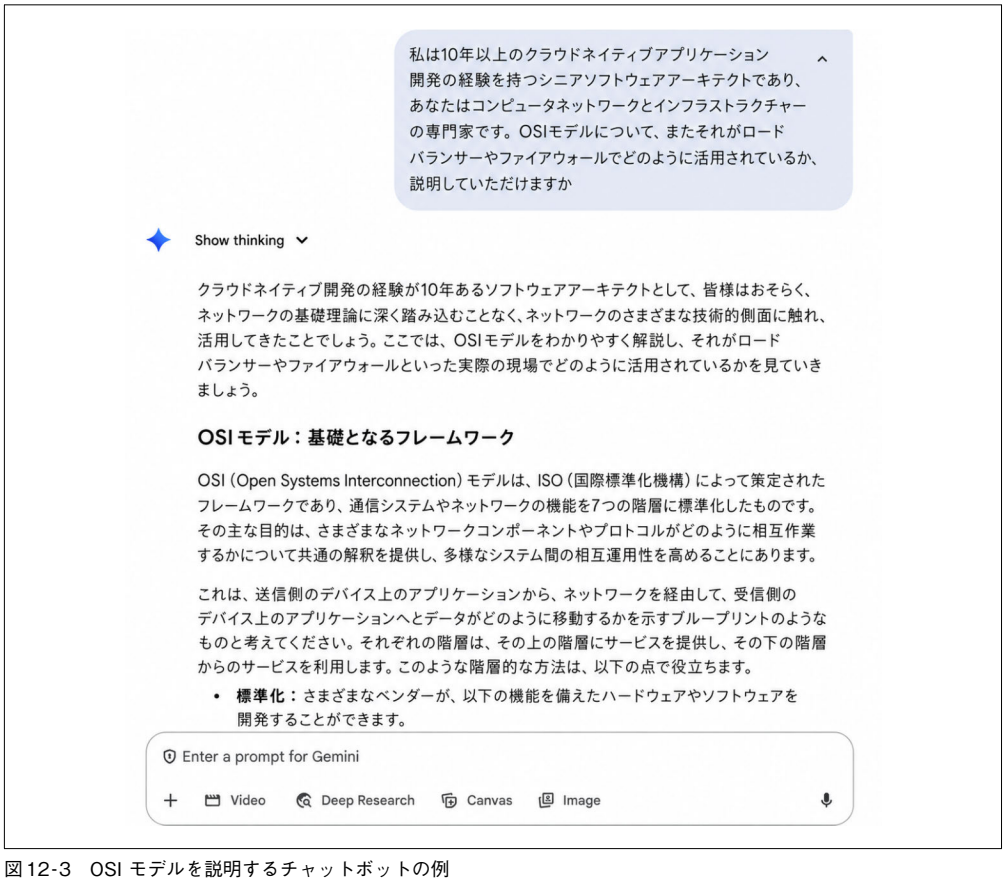


図12-3　OSI モデルを説明するチャットボットの例

12.5　FOMO（取り残されることへの恐れ）

世の中には、新しい言語、新しい技術、新しいアプローチなど、数多くの情報が溢れています。どうすればそれらについていけるでしょうか？ 読者の学習スタイルが、その取り組み方に大きく影響します。記事や本を読むのが好きな人もいれば、聞いたり見たりすることを好む人もいますし、実際にやってみることで最もよく学べるという人もいます。Web 検索、推薦図書の閲覧、ポッドキャストの聴取、動画の視聴、ユーザーグループやカンファレンスへの参加、さらには Technology Radar（https://oreil.ly/LCQ02）のような業界リソースの活用など、利用できる学習リソースは尽きることがありません。

「注意力」は貴重な資源であり、いくらでも増やせるものではありません。無駄にしないでください。すべての情報を読んだり、見たり、聞いたりすることはできません。選択せざるを得ません。多くの開発者は、もし「すべて」を積極的に取り入れていなければ、何か重要なことを見逃してしまうのではないかという、いわゆる「FOMO（Fear of Missing Out、取り残されることへの

恐れ)」を抱えています。

期待を裏切るようで申し訳ありませんが、文字通り、すべてのコンテンツを読み、視聴し、聞くことは不可能です。実際、この段落を読んでいる間に、YouTube にはおよそ 1 週間分の動画が新たにアップロードされていることでしょう！ しかも、他にも、ポッドキャストや入門ガイド、ソーシャルメディア、その他読者が消費したいと思うコンテンツが存在します。

テクノロジーに関する「FOMO」は確かに存在しますが、もしテクノロジー分野で**本当に**大きな出来事があれば、必ず耳に入るという現実を安心材料にしてください。最初の発表を見逃してしまうかもしれませんが、それでかまいません。実際、あまり興味のない分野のことでも、知らず知らずのうちに耳にしていることだってありますよね！ また、自分のネットワークを活用するようにしてください。友人や同僚は、読者とは異なる興味を持っている可能性が高いのです。その点を活かしましょう。自分の専門分野で見つけた興味深い情報を彼らと共有したり、逆に彼らから情報を得たりすることもできます。学びを複数の人と分かち合うことに、何の問題もありません！

> 「集中する」ということは、海にあるすべての波に乗れるわけではないということです。
>
> — Kelsey Hightower、米国のソフトウェアエンジニア、デベロッパーアドボケイト

読者の時間と注意力こそが、非常に貴重な資源なのです[†8]。そして、これらはまさに「資源」そのものです。つまり、いくらでも増やせるものではありません。注意力を無駄にする余裕はありません。その使い道には慎重でなければならないのです。履歴書のためだけの学習を避けましょう。新しいことを学ぶには、それを行う**理由**が必要です。SNS では常に新しい流行が次々と現れますが、実際に必要になるまで学ぶのを待ってもかまいません。新しいテクノロジーを評価するのは難しいものですが、経験を積むにつれてパターンが見えてくるようになります。その「新しい」ものが実は数年前のものと非常に似ていることに気づくでしょう。もし直感で「これは過大評価されている」と感じたら、そこに時間を投資する前に少し立ち止まっても何ら問題はありません。

ここでも、読者のプロフェッショナルなネットワークが大いに役に立つことがあります。なぜなら、学習を他の人に任せることができる場合もあるからです。もし、まだよくわからない分野を友人が探求しているのを知っているなら、定期的に連絡を取り、進捗状況を確認してみてください。その友人の経験が読者の道しるべとなり、場合によっては、その人が問題を解決してくれることで、自分が苦労しなくても済むこともあります。

> 「注意力」というのは、不動産に少し似ています。どちらも、これ以上増えることはないという点で。しかし、不動産とは違って、その価値は上がりつづけるのです。
>
> — Seth Godin、米国の作家、マーケティング専門家、起業家

†8　時間は再生不可能な資源です。賢く使いましょう。

情熱に導かれて

Nate です。以前、データベースについてもっと学びたいと思い、ある日のこと、ふと腰を据えてその分野の本を読み始めました。20 分ほど経ったところで、本を置いて、文字通り 2 階へ上がり、靴下入れを整理し始めました。婉曲表現ではありません。靴下入れの方が優先度が高いと判断したのです。これは、私が実際には、データベースというトピックに対して情熱を持っていなかったという、紛れもない証でした。学習時間を比較的に簡単に確保できたとしても、それでもなお、非常に重要な疑問が残ります。それは、「何を学ぶべきか？」ということです。最終的には自分次第ですが、情熱に導かれるようにすべきです。もしそのトピックにワクワクできないのであれば、日々の生活の中で学習する時間を確保するのは難しいでしょう。

自分に活力を与えてくれるものを選び、それを深く掘り下げてみてください。専門家の動向を追い、ポッドキャストを聴き、動画を見て、ドキュメントを読みましょう。「14.1.1 自分の情熱を注げるものを見つける」を読んでみてください。なお、いつでも別のものに変えることができる、ということを忘れないでください。本を置いたり、タブを閉じたり、動画を途中で止めたりしても、何の問題もありません。最後までやり遂げたからといって、特別な評価が得られるわけではありません。もしそのトピックにまだワクワクしているなら、素晴らしいことです。続けてください！ もしもうワクワクしなくなったら、それもまた素晴らしいことです。新しいものを選んでみましょう。これを繰り返してください。

ソフトウェアの世界では、あらゆる新技術に追いつくことができないのと同じように、すべての新技術を導入することもできません。特定のツールやテクノロジーに対する情熱があるからといっても、それが必ずしも自分のプロジェクトや組織に適しているとは限りません。多くの開発者は、一度は「履歴書目当ての開発」を行った経験があるでしょう。つまり、プロジェクトに有意義な利点をもたらすからではなく、単に履歴書に書き加えたいという理由で、新しいテクノロジーを取り入れてしまった経験です。

新しいものに対しては、常に批判的な目を持つことが大切です。ソフトウェア開発には常にトレードオフが伴います。よく言われるような「最善の」方法など、実際にはほとんど存在しません[†9]。テクノロジーの利点だけに注目するのではなく、あらゆる面を考慮してください。十分な知識を持った利用者となり、Rich Hickey のこの格言を心に留めておきましょう。「プログラマはあらゆるものの利点は知っていますが、トレードオフについては何も知りません」

12.6 どこに時間を割くべきか？

直感は経験と共に育まれ、年月を重ねるにつれて、何が長く続くものか、何が単なる一過性の

†9 たいていの場合、「比較的マシな方法」を選んでいるのです。

流行に過ぎないか、感覚的にわかるようになってくるでしょう。最終的には、トレンドが業界をどこへ導いていくのかが見えてくるはずです。ただし、直感には少し慎重になるべきです。Paul Graham はかつて「ハッカーのレーダー」について書き（https://oreil.ly/IyYUl）、新しいテクノロジーを見る際に彼が重視する基準について述べています。

そのハッカーのレーダーを Java に当てはめた結果、彼は「（Java）はあまり成功しない言語になる、という予感がする」と結論づけました。さて、彼がこの発言をしたのは 2001 年のことで、当時は Java がまだ比較的新しい技術でした。それから 20 年以上が経ち、今となっては、Java が史上最も成功したプログラミング言語の一つであることは明らかです。Paul Graham のように経験豊富で成功を収めている人物でさえ、これほど見当違いの予測をすることもあるのですから、読者の直感には少し慎重になったほうがよいでしょう。

とはいえ、表面的な判断も役に立つことがあります。ソフトウェアの世界では、パターンが繰り返し現れることが多く、物事は振り子のように極端な両極を行き来する傾向があります。古いものが掘り起こされ、改良され、名前を変えられ、まるで誰もやったことがないかのように再登場することは珍しくありません。次の説明を考えてみてください。

> 「異なるプログラミング言語で記述され、さまざまなコンピュータ上で動作するソフトウェアコンポーネントが、相互に通信し、連携して動作できるようにする手法」

この定義は、どのようなテクノロジーを表していると思いますか？ マイクロサービスを思い浮かべますか？ 確かに当てはまりますね！ しかし、これは 1990 年代半ばから後半にかけて普及した CORBA（Common Object Request Broker Architecture）の定義であり、マイクロサービスアーキテクチャが検討されるようになる何十年も前の話です。

経験を積むにつれ、新しいテクノロジーが、数年前に使っていたものとよく似ていることに気づくようになるでしょう[†10]。そのテクノロジーの問題点は解決されているかもしれませんし、そうでないかもしれません。深く掘り下げるべきです。新しいからといって、必ずしもプロジェクトに役立つとは限りません。

「群衆の知恵」は、有益な指針と**なり得ます**。コミュニティの関心を集めているものは何でしょうか？ そこから興味深い発見につながるかもしれません。しかし、時には大衆が過度な期待に踊らされてしまうこともあります。やはり、自分自身の経験も併せて活用してください。

Thoughtworks の Technology Radar（https://oreil.ly/sEOtv）など、業界のリソースも活用できます。Technology Radar は年 2 回更新され、ツール、技術、プラットフォーム、プログラミング言語、フレームワークの現状をまとめたものです。このレポートは、複数のプロジェクトでの経験を統合し、可能な限り客観的なテクノロジー分析を提供しています。Technology Radar を

†10　あるいは、当初は特定の目的のために開発されたテクノロジーが、後に別の問題を解決するために再び活用されることもあります。例えばインターネットは、当初軍によって開発されましたが、後に民間利用へと転用されました（訳注：インターネットの起源は、米国国防省傘下の ARPA が資金提供した研究プロジェクトである ARPANET にあります。ただし、これは軍が直接開発したものではなく、主に大学や研究機関を結ぶコンピュータネットワークの研究として始まったものです）。

フォローすることで、新しいテクノロジーに触れられるだけでなく、それらのテクノロジーが導入サイクルのどの段階にあるのかという感覚も得ることができます。

独自のテクノロジーレーダーを作成する

幸いなことに、読者の会社では充実した学習機会が用意されていることでしょう。しかし、ソフトウェアエンジニアとして、自身のキャリア形成に対して、最終的な責任を負っているのは読者自身です。新しい言語やテクノロジーが絶え間なく登場している中、貴重な時間をどのように配分すべきかを決めるのは、非常に難しい課題です。選択肢があまりに多すぎて決断できなくなる「分析麻痺」に陥りがちですが、独自の「テクノロジーレーダー」を構築する（https://oreil.ly/evKiw）ことで、この状況を打破できます。

複雑なものを用意する必要はありません。また、Thoughtworks のプロダクトのように見栄えを良くする必要もありません。しかし、**自分が**興味を持ち、ワクワクすることを定期的に確認する習慣を身につければ、学習において大きな違いを生むでしょう。その方法については 14 章で詳しく説明しています。

12.7 イノベーションを実践する

学びや探求を重ねる中で、きっと読者のプロジェクトやチーム、あるいは組織にとって大いに役立つ何かが見つかるはずです。そうしたアイデアを組織に取り入れることで、社内での評価が高まり、昇給や昇進につながる可能性があります。では、新しいものを導入するには、具体的にどうすればよいのでしょうか？

読書会は、新しいテクノロジーを導入するのに、非常に簡単で効果的な方法の一つです。志を同じくするエンジニアのグループを見つけ、技術書を選んで読み始めましょう。毎週、例えば朝一番や昼休みに集まるようにしてみてください。ほとんどの組織では書籍代を負担してくれます。昼食や朝食のケータリング代まで出してくれるところも少なくありません。読書会はモチベーションを高める役割も果たします。火曜日にチームで話し合うとわかっていれば、その章を読むのもずっと楽になります。

多くの組織では、**ハッカーデイ**や**ハッカソン**などと呼ばれる、何らかの形の**イノベーションデイ**を実施しています。呼び方はともかく、そのコンセプトは同じです。つまり、新しいテクノロジーを自由に試せる、安全で実践的な場を提供することです。ハッカーデイは、新しいテクノロジーを試してみたいという、誰もが抱く自然な欲求に対する「安全弁」のような役割を果たします。もしこのような場がなければ、多くの開発者は、許可を求めるよりも後で許しを請うほうが得策だと考え、こっそり新しいライブラリや言語をプロジェクトに導入しようとしてしまうでしょう。

もし読者の会社に「ハッカーデイ」のような定期的な取り組みがないなら、ぜひ導入してみてください！ 実験的な試みとして提案し、まずは小規模から始めてみましょう。金曜の午後だけでも

かまいません。組織内のできるだけ多くの部門を巻き込んでみてください。思いがけない場所から、状況を一変させるようなアイデアが現れることに驚くはずです。その結果生まれたコードが本番環境に導入されることはなくても、何かを学ぶことができるでしょう。ある新しいフレームワークが非常に有望に見えたとしても、自社の具体的な環境に当てはめると、ほぼゼロから書き直す必要があることにすぐに気づくかもしれません。あるいは、その新しい言語が、数週間も頭を悩ませてきた厄介な問題を、見事に解決してくれるかもしれません。試してみなければ、その答えはわかりません。

部単位であれ全社規模であれ、定期的に開催されるプロジェクト発表会は、組織内でどのようなテクノロジーが活用されているかを把握する絶好の機会です。その形式は、ライトニングトーク程度の短いデモから、各グループが成果を発表するブース形式まで多岐にわたります。デモデイは、社内でどのようなテクノロジーが活用されているかを学ぶ絶好の機会であるだけでなく、自身のプロジェクトで何か必要が生じた際に相談できる相手を見つける機会にもなります。

イノベーションを生み出す手段は、ハッカソンだけではありません。定期的な技術セミナーシリーズは、新しい概念を社内に導入するのに非常に有効です。講演者が社内の同僚であれ業界の専門家であれ、ランチタイム学習会（lunch-and-learn）を定期的に開催することで、変化しつづけるテクノロジーの動向についていくのが容易になります[†11]。繰り返しになりますが、もし自社に既存の技術セミナーシリーズがないのであれば、ぜひ立ち上げてみてください！ 自分が学びたいテクノロジーについて自ら進んで講演を申し出ることは、大きなモチベーションにつながります。

ランチタイム学習会は、全社規模でなくても十分に効果を発揮します。より小規模な取り組みも素晴らしいものです。興味深い記事を読んだり、ためになる動画を見たりしたことはありませんか？ ぜひチームで共有してみてください。新しい本を読むためのモチベーションをお探しですか？ 志を同じくする仲間数人と読書会を立ち上げ、議論をリードする役割を順番に交代してみてはいかがでしょうか。もし新しいテクノロジーに本当に興味があるなら、隔週の技術ディスカッションの会を始めてみてはいかがでしょうか！

ある程度の規模を持つ大都市圏であれば、地元のミートアップやユーザーグループの会合が定期的に開催されています。たいていの夜には、興味深い技術についての講演を見つけることができるでしょう（その際、夕食や少なくとも軽食が提供されることも多いはずです）[†12]。ラスベガスやサンフランシスコで開催される大規模な業界イベントのほうが、テック系メディアの注目を集めがちです。しかし、何千ドルもの旅費をかける必要のない、素晴らしい地域イベントも数多く存在します。

すでに多くの参加者を集めるランチタイム学習会を開催しており、地域のイベントにも参加した経験があるなら、さらに一歩踏み込んで社内カンファレンスを企画してみてはいかがでしょうか。その規模は、数名の社内スピーカーによる半日程度の気軽なイベントから、社外の専門家を招いた

†11　あるいは、ある組織が呼んでいたように、“chew and spew（噛んで吐き出す）"というものです。もっとも、筆者にとってはあまり好ましくない表現です。

†12　地域のイベントで講演をボランティアで行うことは、もしその道に興味があるなら、カンファレンスでの講演への足がかりとなります。

数日間にわたる複数会場での本格的なイベントまで多岐にわたります[†13]。たとえ小規模な社内イベントであっても、その企画・運営には多大な労力が必要です。しかしその見返りは非常に大きいものです。

社内イベントは人々を結びつけ、新しいアイデアの広がりを促します。普段接する機会の少ない人々と交流する機会を提供してくれます。また、プレゼンテーションの腕を試す絶好の機会でもあります。プレゼンテーションは多くの人にとって本能的な恐怖を呼び起こすものです。しかし、新しいことを学ぼうとするときには、モチベーションを強力に高めてくれます。言うまでもなく、そこで存在感を示すことはキャリアアップを加速させ、昇進やプロジェクトにおける責任の拡大につながるでしょう。

12.8 アーキテクチャ共有会

組織によっては、アーキテクチャ共有会（https://oreil.ly/nDbse）による体系的なアプローチを採用しているところもあります。基本的には、1 人が調査を行い、その結果をチームに報告する形となります。なお、アーキテクチャ共有会を行うのは、必ずしもアーキテクトである必要はありません。検討すべき項目は数多くありますが、以下の点については必ず触れるようにしてください。

- なぜこのテクノロジーを使うべきなのでしょうか？ なぜこのテクノロジーを使うべきでは**ない**のでしょうか？
- 前の質問に答えるためには、何を知っておく必要がありますか？
- このテクノロジーを使うためには、何を知っておく必要がありますか？

最終的なプレゼンテーションは、何時間もかける必要はありません。45 分から 60 分もあれば十分です。詳細なハウツー解説をする必要はありませんが、基本的な「はじめに」よりも詳しい内容であるべきです。セッションでは双方向のやり取りを重視し、参加者の積極的な関与を促すようにしてください。参加者はメモを取り、的確な質問を投げかけ、自身の経験を活かして議論に参加することが求められます。

共有会の段階を終えたら、いよいよ袖をまくり上げて、実際に手を動かして取り組むときです。ワークショップの時間です。残念ながら、ワークショップを行うのに十分な時間は決して確保できません。競合する 2 つのフロントエンドテクノロジーのどちらを選ぶか迷っていますか？ その選択を確実に行う方法があります。それぞれのテクノロジーを使ってアプリを 2 回構築し、気に入らない方を捨ててしまえばよいのです。この提案が承認されるかどうかは運次第でしょう[†14]。限られた時間の中で、何を検証すべきかという最も重要なことを判断しなければなりません。

†13 筆者らは、これまでクライアントのためにこうしたイベントに**数多く**参加してきました。何かお役に立てることがあれば、気軽にお問い合わせください。

†14 もし承認されたら、ぜひ筆者までご連絡ください。話を詳しく伺いたいです。

主な機能の探索に焦点を当て、参加者が簡単に始められるようにしてください。目的を明確に伝え、セッション後にさらに深く学びたい方のために、フォローアップ資料を用意しましょう。新しいことを試したり、隅々まで探ってみたりしてください。そこで発見したことに、嬉しい（あるいは不愉快な）驚きを覚えるかもしれません。何がうまくいくのか？ 何がうまくいかないのか？「ハッピーパス」から外れてみると、何が起こるのでしょうか？

ワークショップでは、対象プロジェクトでそのテクノロジーを使う際に直面する制約をすべて考慮してください。既存のレガシープロジェクトに、このテクノロジーを導入する場合は、そのレガシー環境特有の制約下でテクノロジーをテストしてください。グリーンフィールド環境で動作させることは参考にはなりますが、全体像を把握できないままになるおそれがあります。

レガシープロジェクトの制約

Nate です。数年前、私のワークショップに参加された方が、自身のプロジェクトで新しいユーザーインターフェーステクノロジーを使おうとした際の経験を話してくださいました。その方は、数年前に選んだライブラリで行き詰まり、変更が必要になっていました。チームは市場を調査し、候補を３つに絞りました。そして、各ライブラリを分担して、グリーンフィールド型のワークショップを実施しました。最終的に、ある１つの選択肢に落ち着き、それをアプリケーションに少しずつ取り入れていく計画を立てました。

半年後、このライブラリを追加するだけではうまく問題が解決せず、真に活用するためにはアプリケーションを全面的にリファクタリングする必要があることを痛感したそうです。彼らは、選んだライブラリについて、期待しすぎていたことを認めました。しかし、既存のプロジェクトをフォークして検証を行うべきだったのは明らかです。そうしていれば、より大規模なリファクタリングの必要性に、もっと早く気づいていたでしょう。もちろん、それでも彼らはそのライブラリを選んでいたかもしれません！ しかし少なくとも、状況を十分に理解したうえで取り組んでいたはずです。

この話の教訓は、ToDo リストに挙がっている項目がグリーンフィールド環境ではうまくいったからといって、そのテクノロジーが実際の運用環境でうまく機能するとは限らないということです。本番では、練習したとおりにしか動けないのです。現実とはかけ離れた環境で新しいことを試すのはやめましょう。

実際に触れてみてからであれば、そのテクノロジーについて十分な情報に基づいた判断を下せます。すべてが問題なければ、組織内で試験的なプロジェクトを探しましょう。その試験プロジェクトが適切であることを確認し、万が一失敗しても、戦略的に重要なプロジェクトに致命的な影響が及ばないようにしてください。

12.9 無理なく実践する

急速に変化するテクノロジーの動向についていくことは重要です。率直に言って、ソフトウェアエンジニアとしてのスキルセットにおいて不可欠な要素です。しかし、それは非常に負担に感じられることもあります。自分に優しく、余裕を持って取り組みましょう。ソフトウェアのように、多岐にわたる業界の最新動向を常に把握しようとすると、あっという間に燃え尽きてしまうおそれがあります。学ぶことは楽しいものであって、人生のストレス源であってはなりません。良質な睡眠や、精神面（あるいは身体面）の健康を犠牲にしてはいけません。

責任を持って先送りできるぎりぎりのタイミングまで、何かを学ぶのを待っても何の問題もありません。テクノロジー業界には「ハイプサイクル」が蔓延しています。何かを学ぶ前に、その動向が落ち着くのを待つようにしてください[†15]。忍耐は美徳です。ソフトウェア業界には、先月まで「最新トレンド」とされていたのに、今では放置されたままの残骸が散らばっています。何かを学ぶのに「期限」などありません。もしそのテクノロジーに持続力があると証明されれば、来週でも、来月でも、来年でも、いつでも学び始められます。

12.10 まとめ

テクノロジーは急速に変化します。キャリアを通じて、多種多様なツールや言語、テクノロジーに触れることになるでしょう。成功するためには、学び、適応し、柔軟に対応する能力が不可欠です。スキルを常に最新の状態に保つことを習慣にしましょう。そのため、スケジュール管理システムに時間を確保し、新しいことに挑戦する時間を設けてください。1 年分の学習を長期休暇に詰め込むより、定期的に取り組むほうが継続しやすいものです。その過程を楽しんでください！

12.11 実践に移す

結局のところ、新しいテクノロジーを習得するには、文字通り実践に移すことが必要です。自分の「やりたいこと」を実現するために、個人用のアプリケーションを作ってみてはいかがでしょうか。例えば、増えつづけるワインのコレクションを管理したい、子供のスポーツの成績を記録するアプリケーションを作りたいなどです。もし特に必要なものが思いつかないなら、地元の学校や非営利団体に連絡してみてください。こうした団体は技術的な支援を求めていることが多く、一般的に、自由に試せるような環境を与えてくれるでしょう。

少し時間を取って、テクノロジー分野を見渡してみてください。今日、最も注目を集めているものは何でしょうか？ それは読者の心に響きますか？ もし響かないなら、心に響くものを選んでください！ 興味をそそられる分野を深く掘り下げ、ソーシャルメディアでその分野のオピニオンリーダーをフォローし、ポッドキャスト、ライブ配信、ニュースレター、動画などを購読しましょ

†15 時間は再生不可能な資源です。大切に使いましょう。

う。地元のユーザーグループに参加してはどうでしょうか？ その分野に特化したカンファレンスはありますか？ オンラインイベントはどうでしょうか？ その世界に入り込んでみてください。そのテクノロジーを使って個人的なプロジェクトを作ってみましょう。同僚の前で、地元のユーザーグループで、あるいはカンファレンスで、その技術に関するプレゼンテーションを行ってみてください。

1〜2か月ごとに、見直してみましょう。そのテクノロジーに対して、まだ情熱は持てていますか？ もしそうでないなら、新しいものを選んでみてください！ これを繰り返しましょう。以前取り組んだことに戻るのもよいでしょう。すべてを一から学びなおす必要はありません。知識を深めてください。

結局のところ、数日あるいは数週間かけて実際に動作するコードを作成することほど効果的なものはありません。そのコードが洗練されている必要も、あらゆるエッジケースを網羅している必要もありません。そのテクノロジーが機能することを実証するのに十分なものであればよいのです。学習に近道はありません。単に「情報を頭の中に詰め込む」ような方法など存在しないのです。

学んだことを共有するのも、効果的な学習方法となります。友人や同僚数人と協力して、小さなプロジェクトに取り組んでみてください。たとえ一人でも、目を通してくれる人がいるだけで助けになります。ペアで取り組むのも効果的です。プロジェクトをオープンソース化するか、少なくとも公開リポジトリにホストして、他の人からコメントや貢献をもらうことを検討してみてください。また、学習の様子をライブ配信するのもよいでしょう。少し気後れするかもしれませんが、コミュニティの力は絶大です。他の人も似たような問題に直面している可能性が高く、たいていはアドバイスをくれるでしょう。学習の道のりを歩んでいるのは自分一人ではない、という単純な気づきだけでも、非常に心強いものになるはずです。

12.12　関連リソース

- "Thinking Architecturally"、Nathaniel Schutta 報告（O'Reilly、2018年）
- 『The Pragmatic Programmer』Andrew Hunt, David Thomas 共著（Addison-Wesley Professional、1999年）
  和書『達人プログラマー（第2版）』（オーム社、2020年）
- 『The Passionate Programmer』Chad Fowler 著（Pragmatic Bookshelf、2009年）
  和書『情熱プログラマー』（オーム社、2020年）
- Thoughtworks Technology Radar（https://oreil.ly/QXCHr）
- 『The First 20 Hours: How to Learn Anything…Fast!』Josh Kaufman 著（Portfolio、2014年）
  和書『たいていのことは20時間で習得できる』（日経BP、2014年）
- 『Pragmatic Thinking and Learning』Andy Hunt 著（Pragmatic Bookshelf、2008年）
  和書『リファクタリング・ウェットウェア』（オライリー・ジャパン、2009年）

13章
テクノロジー業界でソフトスキルを磨く

人と接するときは、相手が論理で動く存在ではなく、
感情で動く存在であることを忘れないでください。
— Dale Carnegie、『How to Win Friends & Influence People』（和書『人を動かす』）より

習得したスキルにはすべて「有効期限」があります。貴重な時間をどう配分するか考える際には、この点を考慮すべきでしょう。技術が絶えず変化していることは、もうご理解いただいているでしょう。API は進化し、新しいものに置き換えられます。ある言語の旧バージョンではベストプラクティスとされていた手法も、新機能によって不要になってしまうことがあります。理解し始めた途端に、その知識が時代遅れになってしまうように感じられるかもしれません。それは間違いではありません。そして、そう感じているのは読者だけではありません。

ソフトウェアエンジニアになるまでの道のりがどうであれ、おそらく技術的なスキルの向上に注力してきたことでしょう。何と言っても、技術的スキルはこの分野の基盤となるものです。プログラミング言語を理解していなければ、コードを書くことはほぼ不可能だからです。キャリアの初期段階で成長するためには、技術的なツールキットを拡充し、より多くのフレームワークを学び、クラウドプロバイダーの活用に精通し、そして自分が選んだプログラミング言語の最新動向を常に把握しなければなりません。

しかし、キャリアを通じてずっと役立つスキルが一つあります。それは、多くのエンジニアが軽視しがちな「ソフトスキル」です。他者と協力し、明確にコミュニケーションをとる方法を学ぶことは、新たな言語やフレームワークを習得することと同じくらい、成功するためには重要なことです。人間はテクノロジーほど急速には変化しません。私たちの「オペレーティングシステム」を更新するには、何千年もの時間がかかるのです。だからこそ、ソフトスキルは決して時代遅れになることがないのです。効果的なコミュニケーションの方法、他者との協働や影響力の与え方、そして時間管理の方法を学ぶことは、キャリア全体を通じて大きな成果をもたらします。

30 年、40 年、あるいは 50 年にもわたって役立つスキルを習得するために費やす時間は、投資対効果の面で非常に価値が高いと言えます。もちろん技術的なツールキットの変化には常に追いついていく必要があります。しかし、他者と差別化を図るためには、より普遍的なスキルも軽視して

はいけません。一見大変に思えるかもしれませんが、結局のところ、優れたソフトスキルを身につけることは、良い習慣を身につけることに尽きます。さっそく始めてみましょう！

13.1　共同作業におけるコミュニケーション

今日、テクノロジー分野に足を踏み入れる人の中には、人間関係の厄介な側面を避けられると期待している人もいます。開発者は内向的で孤独な存在だという固定観念は、プロジェクトの規模が小さく、多くの場合個人によって推進されていたコンピュータの黎明期までさかのぼります。さらにさかのぼれば、**コンピュータ**という言葉は、数学的な計算を行う人を意味していました。当時、企業には、数百万行にも及ぶコードベースに取り組む、ソフトウェアエンジニアの大軍など存在しませんでした。しかし、今日では、キャリアで成功を収めたいのであれば、コミュニケーションの技術を習得しなければなりません。

だからといって、ソフトウェア開発で成功するために必ずしも外向的である必要はありません。実際、カンファレンスの登壇者の多くは内向的な方々です！ 人によってはより多くの努力が必要になるかもしれませんが、ソフトウェア開発にはチームワークが不可欠です。今日のアプリケーションの規模を考えれば、「一匹狼」的な開発者の時代は終わりました。もはや一人の人間がコードベース全体を頭の中に収めることはできません。モダンなコードベースには、チームでの協力が不可欠なのです。

チームの働き方も変化してきました。窓のないキューブ型オフィスで黙々と作業するのではなく、多くのチームがプロジェクトルームで業務を行っています。プロジェクトルームは共同作業には非常に効果的ですが、内向的な人にとっては疲れる場所になり得ます。実際、開発者の中にはスケジュール管理システムに「一人になる時間」をわざわざ確保する人もいます。プロジェクトルームを離れて一人で過ごす休憩時間を取ることで、彼らは「社交的なエネルギー」を充電する機会を持つのです。上司と話し合う必要があるかもしれません。しかし、成功するために必要だと感じることは遠慮なく主張してください。

ソフトウェア開発は共同作業であり、チームと効果的に連携するためには、さまざまな手法を活用する必要があります。Kent Beck がかつて声高に述べたように、「ソフトウェア設計は人間によるプロセスであり、人間のために人間が行うもの」（https://oreil.ly/iLvNB）です。新しいプログラミング言語を習得するのと同じくらい自然にできることではないかもしれませんが、技術コミュニケーションは学ぶことができます。コミュニケーションとは、単に口から言葉を発することだけではありません。適切なコミュニケーション手段を選び、対話の準備を整え、組織の上下や左右のつながりにおいてどのようにコミュニケーションを図るかを学ぶことも含まれます。

13.1.1　コミュニケーション手段

もちろん、コミュニケーションはプログラミング言語だけにとどまるわけではなく、数多くのコミュニケーション手段が利用可能です。それらは、距離の近いものから距離感のあるもの、個人的なものから組織的なもの、密接な関わりから疎遠なものまで多岐にわたります。やり取りの記録が

残るものもあれば、後になって否定できる余地を残すものもあります。選択肢は実に豊富にあるため、適切なアプローチを選ぶことが極めて重要です。課題は、適切なタイミングで適切な方法を選択することです。しかし、これは口で言うほど簡単ではありません。**図13-1** に示す「コミュニケーションのグラデーション」のように、さまざまなコミュニケーション手段を可視化しておくと理解しやすいでしょう[†1]。

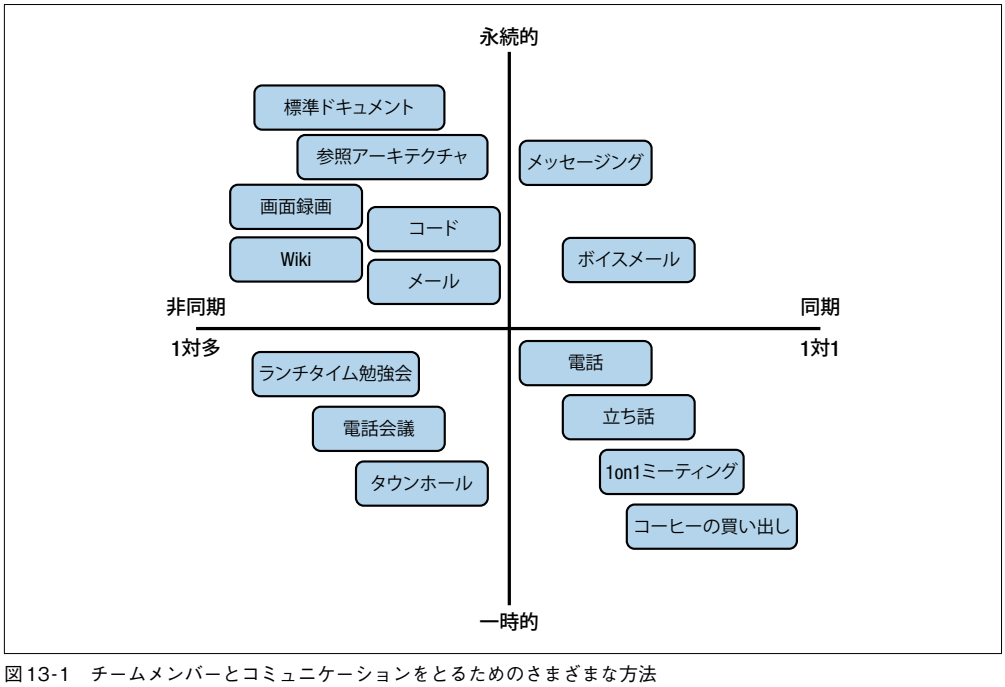


図13-1　チームメンバーとコミュニケーションをとるためのさまざまな方法

議論の内容を記録しておくことが極めて重要になる場合があります。例えば、重大な脆弱性が発見され、アプリケーションの主要コンポーネントをアップグレードしなければならないような場合です。問題とその解決策を全員が把握しておくことで、組織が致命的なハッキング被害にあうのを防ぐことができます。

コミュニケーション手段を選ぶ際には、相手についても考慮する必要があります。メールを頻繁に利用する人もいれば、数か月も受信箱を確認していない人もいます。組織によっては社内メッセージングツールが主流である場合もあれば、すべてを会議で行うべきだと考えるところもあります。自組織における情報の流れを理解することで、判断できるでしょう。コミュニケーションのあらゆる要素を網羅することは、本書の範囲を超えています。ここでは、一般的な選択肢と、キャリアアップにとって特に重要な選択肢について見ていきましょう。

†1　コミュニケーションスタイル（https://oreil.ly/dMOV8）にもさまざまな種類があります。

13.1.1.1　メッセージング

組織のリモートワークに対する方針がどうであれ、Microsoft Teams、Slack、Google Chat などの企業向けメッセージングツールをかなりの時間利用することになるでしょう[†2]。メッセージングツールを使えば、個人やグループに対して非同期のテキストメッセージを簡単に送信できます。また、多くのツールでは、ファイル、コード、画像、GIF、あるいはコピー&ペーストできるものなら何でも添付することもできます。メッセージングは通常、カジュアルな形式で行われ、簡単な質問や、問題に関して事前連絡するのに向いています。メッセージングの非同期性こそが、その最大の強みの一つです。チームメンバーに連絡すれば、相手にとって都合の良いタイミングで返信できます。

会社の組織構造がどれほどフラットか（あるいはそうでないか）にもよりますが、一般的に、メッセージのやり取りは直属のチーム内で行うのが適切です。より上層部のリーダーに対してやり取りができるかどうかは、場合によります。昨今、多くの企業ではメッセージングが日常的に活用されています。たとえチーム全員が同じ場所にいても、オフィスの話し声を抑えたり、相手の作業を邪魔しないようにしたりするために、隣に座っている人にメッセージを送ることは非常に一般的です。また、次回のコミットでリファクタリングが必要な関数などの情報を、短期記憶から解放するための手段としても役立ちます。

メッセージは非同期であるため、相手がすぐに確認しない場合があります。メッセージに関するチームのルールを定めることは有効ですが、相手の時間を尊重するよう心がけてください。「ちょっとした質問」であっても、相手の午前の予定を台無しにしてしまうおそれがあります。相手の通知設定や集中して作業できる時間帯、会議の予定などに配慮しましょう。

繰り返しになりますが、ソフトスキルとは、結局のところ良い習慣を集めたものです。例えば、仕事が山積みで忙しいときでも、誰かから質問メッセージが届いたら、たとえ「今日は手一杯ですが、今週の後半ならお話できる時間があります」と伝えるだけでもよいのです。できるだけ迅速に返信するように心がけてください。1章で述べた「黄金律」を忘れないようにしましょう。自分が扱われたいように他人を扱うことは、キャリアアップのためにできる最も簡単なことです[†3]。昇進は、単なる計算式や資格の取得だけで決まるものではありません。多くの場合、組織内で読者がどのように見られているかが反映されます。読者がその場にいないときに、他人が読者について何を語っているかが、しばしば決定的な要因となります。周囲の人々に、読者自身と読者の仕事に対して好意的な印象を持ってもらうことが大切です。

組織の記録保持ポリシーにかかわらず、メッセージングツールに入力した内容はすべて記録されており、後日、弁護士や経営陣に転送されたり、閲覧されたりする可能性があると考えておくべきです。メッセージングツールに関するポリシーや慣行には、その組織の企業文化が反映されることが多いのです。新しい笑いのネタを練るためにメッセージングツールを利用するのは、一般的に得策ではありません。企業内のメッセージングツールに入力した内容は後日閲覧される可能性がある

†2　どれを使うことになったとしても、おそらくそれが一番嫌いなものになるでしょう。残念ですが。
†3　言い換えれば、同僚に対して上司と同じような態度で接するようにしてください。

と考えておくべきです。

最後に、文章では口調が必ずしも伝わるわけではないため、相手が読者の意図したニュアンスを誤解してしまうおそれがあることを忘れないでください。皮肉や冗談のつもりで書いたとしても、相手はそれを文字通り、真剣に受け取ってしまうかもしれません。つまり、自分の頭の中で考えているのとは、受け取られ方が異なるおそれがあるのです。慎重を期すようにしましょう。

13.1.1.2 会議

調査によると、開発者は日常的に会議にかなりの時間を費やしていることが明らかになっています。一般的なソフトウェアエンジニアに「仕事で最も嫌いなこと」を尋ねると、「会議が多すぎる」という答えがトップ5に入ることでしょう。とはいえ、問題の解決や新しい取り組みの伝達において、会議が最も手っ取り早い手段となる場合もあります。

会議は同期型で行われ、その場限りのものですが、後で視聴できるよう録画されることが多く、スライド資料が共有されることもあります。会議の目的は多岐にわたり、朝のスタンドアップミーティングから、上司との 1on1、直近の財務報告について話し合う全社ミーティングに至るまでさまざまです。会議は解決策を見出すための最も手っ取り早い手段となり得ますが、それでも終了後に参加者の認識が一致しない場合があります。

時間あたり数千ドルの損失

Nate です。以前、私の仕事の一部として、毎週木曜日の朝に 2 時間の定例会議がありました。出席者リストをざっと見ただけでも 200 名以上の名前が並び、その全員が社内のシニア技術者でした。ざっと計算すると、この会議には毎週数千ドルの費用がかかっていたことになります。しかし、その費用は十分に価値があったと言えるでしょう。会議の議題は、アプリケーションの定期的なデプロイから新しいインフラストラクチャの稼働開始に至るまで、環境の変更に関するものばかりだったからです。

全員に情報を確実に共有することで、業務への支障を最小限に抑えることができました。また、変更内容に見逃されている影響の可能性があった場合には、参加者が意見を述べることができました。幸いなことに、この会議の運営も非常にスムーズでした。主催者は毎週詳細なアジェンダを送付してくれたため、参加者は必要に応じて会議に出入りできました。会議にコストがかかるからといって、時間と費用をかける価値がないというわけではありません。

場合によっては、予定が二重や三重に重なってしまうことがあるでしょう。会議への招待状が非常に広範囲の配布リストに送られたり、役職上自動的に CC に入れられたりすることもあります。主催者が読者を特に招待しているわけでも、意見を求めているわけでもなく、単にリストに名前が入っているだけということもあります。読者の時間は貴重です。会議への参加について、慎重に判断することはまったく当然のことです。主催者に連絡して会議での自分の役割を確認したり、上司

に相談したりしてください。上司なら適切なアドバイスをくれるはずです。

会議にはアジェンダを用意するのが最低限の礼儀ですが、自分が何のために出席するのかを尋ねてみると、状況が明確になることがあります。会議にはさまざまな目的があります。今回の会議の目的は何でしょうか？ 会議の最初から最後まで出席する必要はあるのでしょうか？ メールで済ませることはできないでしょうか？

いずれ、CTOやエンジニアリング担当副社長、大口クライアント、あるいはその他の「非常に重要な人物」との**会議**に出席することになるでしょう。多くの場合、これはキャリアアップの証なのですが、こうした会議は緊張してしまうものです。主催者に遠慮なく質問をしてみてください！ 定期的な会議であれば、毎回出席している人や招待リストに載っている他の人々に、会議の進め方や流れについて事前に聞いておくとよいでしょう。これは上司が意見を求める共同作業の場なのでしょうか、それとも単に指示を出すだけの場なのでしょうか？ 質問や意見を気軽に挟んでもよいのでしょうか、それとも「指名されるまで待つ」という暗黙のルールがあるのでしょうか？ 上級管理職の中には、一般には知られていない特定の**段取り**を念頭に置いている人もいます。内部事情を知らずに臨むと、読者の昇進を左右する人物に、知らず知らずのうちに悪い印象を与えてしまうかもしれません。

多くの場合、上司やシニア技術者を味方につけておくことは、非常に役に立ちます。特に、経営陣と初めて接する場合にはなおさらです。親しみやすい顔ぶれがいるだけで、緊張が和らぐこともあります。また、上司に自分の発言がうまく伝わっていない場合に、こうした味方からその点を指摘してもらったり、今後の会議に向けて個人的にアドバイスしてくれたりすることもあります。

会議の前に下準備をしておきましょう。確認すべき具体的な事案がある場合は、会議の前にそれらを読む時間を確保してください。対面であろうとビデオ通話であろうと、眉をひそめる、腕を組む、微笑むといった非言語的なサインに注意を払うようにしましょう。また、会議の権力構造を見極めるよう心がけてください。意思決定者は誰でしょうか？ その方は会議に参加していますか、それとも会議にいない方でしょうか？

オフィスへの出社義務があっても、多くの組織には複数の拠点があるため、業務時間の一部は音声通話やビデオ通話に費やすことになるでしょう。オンライン会議にもマナーがあります。組織の文化によっては、カメラをオンにすることや、背景を整理しておくことなど、暗黙の「ルール」が存在する場合があります。良質のヘッドセットに投資しましょう。新しいものを購入したら、CTOとの通話に参加する前に試してみてください。多くのヘッドセットでは、マイクの感度を調整できます。状況に合わせて調整してください。言うまでもありませんが、話していないときはマイクをミュートにしてください。特に食事中は注意が必要です[†4]。誰かが昼食を食べ終わる音を聞くほど、不快なことはありません[†5]。

社内のメッセージングツールを用いた裏でのやり取りは、多くの会議において役立つことがあります。確認のための質問をしたり、話を遮らずにコメントを挟んだりするために、会議の内容につ

†4　あるいは、その会議について皮肉なコメントを言うときもです。
†5　ダース・ベイダー並みの荒い息遣いのほうが、もっとひどいかもしれません。

いて裏で話し合うことはごく一般的です。場合によっては、会議で自分の意見を主張する手段として、メッセージ機能を活用することもあります。

また、会議に参加しているのが**誰なのか**を正確に把握しておくことも大切です。IT 部門の人たちだけでしょうか？ 顧客はいますか？ 経営陣は？ 勝手な思い込みは禁物です！ 最新のビデオ会議テクノロジーでは、顔の見えない参加者は大幅に減りましたが、それでも誰が参加しているのかを知っておくことは重要です。

読者自身も会議を主催することになるでしょう。効果的な会議運営を心がけてください。アジェンダを明確に設定し、出席者は必要最小限に留めてください。可能であれば、同僚に議事録の作成を依頼しましょう。一般的な企業のスケジュール管理システムでは会議時間は 60 分とデフォルト設定されていますが、だからといってすべての会議を 60 分に設定する必要はありません。読者も経験されたことと思いますが、多くの会議は与えられた時間枠に合わせて長引くものです。「1 時間もかかるはずがない」といった言葉を、どれほど耳にしたことでしょう[†6]。制約はかえって生産性を高めることもあります。会議を 30 分に設定すれば、議題に集中せざるを得なくなります。また、1 時間を予約したものの、最初の 30 分で議題をすべて終えた場合は、残りの時間を無駄にせず、参加者に返しましょう。



多くのスケジュール管理システムでは、デフォルトの会議時間を変更できます。長い会議には 50 分、短い会議には 25 分といった「思い切った」設定も、ぜひ試してみてください。また、移動時間や、どうしても発生してしまう会議の延長に備えて、開始時刻を通常の 0 分や 30 分から始めるのではなく、5 分または 10 分後に始めることも検討してみてください。これにより読者の集中力が維持されます。また、次の会議の前にトイレに行ったり、コーヒーをもう一杯飲んだりする時間が持てるため、参加者にも喜ばれることでしょう。組織全体でイベント設定を変更している組織もありますが、読者自身の会議について個別に設定を変更しても何ら問題ありません。

会議は時間通りに開始することで、参加者に敬意を表しましょう[†7]！ どうしても会議が長引いたり、休憩が必要になったりすることもありますが、できるだけ予定通りに進めるよう努めてください。準備期間にも配慮しましょう。複雑な設計案のレビューを依頼する場合は、相手が資料を確認する時間を確保してください。遠方の参加者のために会議室を予約する場合は、実際に利用する施設について事前に確認しておくとよいでしょう。会議室などの物理的なリソースの予約に関しては、オンラインの予約システムが常に最新の状態であるとは限りません。

主催者として、会議を予定通りに進めるよう努めてください。話が脱線した場合は軌道修正しなければならないでしょう。それが議論すべき内容であった場合、別途フォローアップの会議が必要になる場合もあります。また、全員が参加できるよう配慮し、各メンバーに発言の機会が与えられるようにしてください。会議は必ずしも楽しいものではありませんが、一人チームでない限り、避

†6　もちろん、その際に問うべきは「では、なぜ 1 時間も予約したのですか？」ということです。
†7　多くの場合、最もシンプルな解決策が最善の策です。

けられないものです。とはいえ、必ずしも苦痛に満ちたものにする必要はありません。

13.1.1.3　プレゼンテーション

人々に「何が怖いですか？」と尋ねると、必ず「人前で話すこと」という答えが返ってくるでしょう。プレゼンテーションを嫌がる人は多いのですが、キャリアアップを目指すなら、何らかの形で講演を行う必要があります。カンファレンスの常連になることはなくても、さまざまな聴衆を前にして、自信を持ってプレゼンテーションができるようにならなければなりません。

プレゼンテーションにはさまざまな形式があり、必ずしも1時間単位で行われるとは限りません。1分程度の短い「エレベーターピッチ」を常に用意しておくことをおすすめします。組織の上級幹部と、いつ突然、短い会話をする機会が訪れるかわかりません[†8]。短く話すのは往々にして難しいものです。要点を後回しにせず、伝えたい核心に集中するよう心がけてください。

プロジェクトチーム以外の聴衆に向けて、5分から10分程度の短いプレゼンテーションを行うよう求められる場合があります。例えば、エンジニアリング部門やアーキテクチャチーム、あるいはCTOの直属の部下たちに対して、プロジェクトの進捗を報告することなどが挙げられます。特定の聴衆の心に響くようメッセージを工夫することが重要です。聴衆の理解度や状況に合わせて、内容を調整してください。

プレゼンターとして上達するための近道はありません。上達したいのであれば、数多くのプレゼンテーションを行う必要があります。地元のユーザーグループやミートアップで、自ら進んで発表を申し出てみてはいかがでしょうか。読者が尊敬している講演者の多くは、地元での活動からキャリアをスタートさせています。また、プレゼンテーションを行うことは、新しいことを学ぶための大きな原動力にもなります。なぜなら、発表するという約束は動かしようのない壁のようなものなので、発表に備えて準備を整えざるを得なくなるからです。

練習は欠かせません。ペットを前にリハーサルをすれば、話の流れやタイミングをつかめます。また、練習の相手をしてくれる聴衆を募れば、貴重なフィードバックを得られます[†9]。プレゼンテーションはコードと同じように、時間とともに進化していきます。思いがけず、探しても見つからなかったような貴重な気づきを得られることもよくあります。講演することの真の価値は、思いがけないところにある、ということに気づくでしょう。例えば、NateはかつてReactとAngularを比較する講演を行いました。3回目か4回目の発表を終えた後、そこから**あらゆる**テクノロジーの比較・対照について語る、より汎用性の高い講演に発展させられることに気づいたのです。

スピーチスキルを向上させるための優れたリソースは数多く存在します。多くの都市や一部の大企業には、トーストマスターズクラブ（https://oreil.ly/3fxk4）があります。より技術的なプレゼンテーションに特化したクラブもあります。トーストマスターズのカリキュラムでは重要なテクニックを学べますが、プレゼンテーションを学び、上達するうえでは、安心して練習できる場が欠かせません。

---

†8　エレベーターの中で文字通り偶然会うことはないかもしれませんが、会議の冒頭で45秒間だけ自分のメッセージを伝える機会がいつ訪れるかは、誰にもわかりません。

†9　練習相手に、何かしらのインセンティブを用意する必要があるかもしれません。食べ物などが効果的です。

このテーマに関する優れた資料も数多くあります。筆者の一人である Nathaniel は『Presentation Patterns』を共著しており、オライリー学習プラットフォームで視聴できる動画[†10]もいくつか公開しています。Nancy Duarte 著『Resonate』（和書『ザ・プレゼンテーション』ダイヤモンド社）や Garr Reynolds 著『Presentation Zen』（和書『プレゼンテーション Zen』丸善出版）も素晴らしい書籍です。

13.1.1.4 コミュニケーション手段としてのコード

ソフトウェアエンジニアにとって、他の開発者に情報を伝える最も一般的な手段がコードそのものです。技術の世界において、優れたコミュニケーション能力とは、明確で効果的なコードを書く能力にもつながります。コードこそが究極の信頼できる情報源であり、自分の意図を他の開発者に正しく伝えられることが非常に重要です。

3 章で学んだように、コードを書く際には、それを読む人のことを念頭に置くことが大切です。適切な命名規則を守り、コードを簡潔に保ち、小賢しいコードを避けることは、読者の後を引き継ぐ開発者のために不可欠な配慮です[†11]。コードを読む人のために最適化しましょう。

> どんな愚か者でも、コンピュータが理解できるコードなら書けます。優れたプログラマは、人間が理解できるコードを書きます。
>
> — Martin Fowler、英国のソフトウェア開発者、作家、ソフトウェア開発に関する国際的な講演者

ソフトウェアプロジェクトにおけるドキュメントは、作成後すぐに古くなってしまうのが常です。しかし、コードは違います。コードそのものが「信頼できる情報源」であり[†12]、利用できる最新のドキュメントなのです。テストの実施からコーディング規範の遵守に至るまで、時間をかけて、コードが明確かつ効果的に情報を伝えられるようにしてください。

コミュニケーションは広範なテーマであり、読者はまだその表面をなぞったに過ぎません。キャリアを重ねるにつれ、コミュニケーションのグラデーションにおけるさまざまな側面を学び、習得していくことでしょう。効果的なコミュニケーション能力は、キャリアにおいて非常に重要です。すべてが容易に、あるいは自然に身につくわけではないかもしれませんが、これらのスキルを習得するために時間を割いてください。そうすれば、将来きっと良かったと実感するはずです！

13.1.2 企業内伝言ゲーム

学校やパーティーで「伝言ゲーム」という遊びをしたことがあるかもしれません。円になって座り、一人の人が右隣の人にひそひそと言葉を伝え、その人がまた右隣の人にその言葉を伝え、というように順番に伝えていき、最後の人までたどり着くと、その人がメッセージを声に出して言い

†10 https://learning.oreilly.com/course/presentation-patterns/9781491954980/
†11 そして、忘れてはならないのは、多くの場合、読者自身こそが、その引き継ぐ開発者なのです…
†12 もちろん、それが**正しい**とか、ビジネス上のニーズを満たしているということではありませんが、それはまた別の問題です。

ます。

当然のことながら、メッセージは人から人へ伝わるうちに変化していきます。発音が不明瞭だったり、聞き手が聞いたことをそのまま繰り返すのではなく、自分の解釈で受け取ったりすることもあるでしょう。あるいは、ある人がその表現に馴染みがなく、自分にとって馴染みのある言葉に置き換えてしまうこともあるかもしれません。

組織内でも、驚くほど頻繁に同じようなことが起こります。会議で誰かが不用意な発言をしてしまうこともあれば、単に人々が聞きたいことだけを聞いてしまうこともあります。いずれにせよ、いつの間にか、知らず知らずのうちに**企業内伝言ゲーム**に巻き込まれていることに気づくかもしれません。たとえどんなに配慮をしたり、優れたコミュニケーション能力を備えたりしていたとしても、それを避けることはできません。しかし、このような事態に備えることはできます。

企業内伝言ゲームでは、特に企業標準に関する話題がよく聞かれます。「Alice が『コンプライアンス違反だ』と言っていた」とか「○○が企業標準だと聞いた」といった話を耳にしても、驚かないでください。実際にそのような標準は存在するのでしょうか？ それは、読者のアプリケーションにも適用されるのでしょうか？ 遠慮なく、いわゆる「証拠」を提示してもらうようにしてください。

もし、企業内伝言ゲームに巻き込まれてしまったら、どうすればよいでしょうか？ まず、冷静さを保ち、その状況に気づくことが大切です。周囲は騒がしくなり、機嫌の悪い人も出てくるでしょう。次に、どこで情報が歪んでしまったのかを特定してみてください。誰かが話の一部しか聞いていなかったのでしょうか？ 多くの場合、コミュニケーションがどこでうまくいかなくなったのかを知ることで、どこを改善すべきなのかが明確になります。

そして、誰が関与しているのかを把握しましょう。これまで気づいていなかったのかもしれませんが、情報を共有しておくべきステークホルダーはいませんか？ 積極的に連絡を取り、その方々と問題について話し合いましょう。最後に、その状況について振り返りを行ってください。このようなことを防ぐために、何か別の対応はできなかったのでしょうか？

険悪な雰囲気への対処

Nate です。数年前、クライアント側でレンダリングされるユーザーインターフェースに関して、ベストプラクティスを作成しました。当時は業界全体で非常に一般的な手法でした。スマートフォンやタブレットなど、その他のデバイスの普及に伴い、アプリケーションはもはや特定のモニター解像度向けに構築できなくなっていました。そのため、アプリケーションは、各クライアントに適した UI を構築できるように、JSON 形式のデータを送り出すような一連のサービスへと進化しました。このベストプラクティスに異論はありませんでしたが、当社の全プロジェクトで確実に実践させるには、多大な教育や説明が必要でした。

数か月後、ある事業部門のチーフアーキテクトから、チームメンバーが懸念を抱いているとのことで、ベストプラクティスについて説明に来てくれないかと依頼がありました。彼は私の

スケジュールに時間を確保してくれ、私はそのテーマに関するプレゼンテーションを誠実に実施しました。質疑応答の時間になると、ある人物が明らかに激怒し、Windows MVC を使用するためにアプリケーションを完全に再設計するよう求めていること、そしてその要求がいかに理不尽であるかを訴えてきました。

具体的な実装技術については何も言及していなかったため、この反応には驚かされました。そこで、彼らの現在のアーキテクチャについて詳しく説明してもらうと、彼らはすでにベストプラクティスへの準拠が 90% ほど進んでいたことがわかりました。

さらに詳しく調べてみると、彼らのビジネスパートナーの一人が、私が行ったベストプラクティスに関するプレゼンテーションを聞いていたことがわかりました。そのプレゼンテーションでは JavaScript MVC ライブラリについて触れていました。彼らはそれが何なのか分からなかったため Windows MVC だと誤解してしまい、アプリケーションが標準に違反しているのではないかと心配していたのです。その誤解が解けると、会議の雰囲気はかなり前向きなものになりました。

13.1.3 相手を知る

ソフトウェアエンジニアとして、同僚のエンジニア、アーキテクト、テスター、プロダクトマネージャーなど、他の技術者とのコミュニケーションに多くの時間を費やすことになります。しかし、キャリアを重ねるにつれて、技術的知識がそれほど深くないステークホルダーに対しても、技術的な概念をわかりやすく説明しなければなりません。開発者の言葉をビジネス用語に置き換えることを学ぶことは、練習が必要な重要なスキルです。相手を理解し、それに応じて伝え方を調整する必要があります。

専門的な技術の細部までに精通していることをどれほど誇りに思っていても、プロジェクトルームの外にいる人々が、読者の専門用語の知識に感心することはめったにありません[†13]。セールスエンジニアから学びましょう。セールスエンジニアは、高度に技術的な製品について特定の人々の心に響くように説明しなければならないため、メッセージを適切に調整する能力が必要です。一部のベンダーは「経営層向けの話し方」（つまり、テクノロジーをビジネス課題の解決に結びつけること）に非常に長けています。読者もこのスキルを習得すべきです。意思決定者は「これはクールな新テクノロジーです」という言葉にはめったに心を動かされません。そのクールな新テクノロジーが**どのように**ビジネス価値をもたらすのかを明確に説明できるようになりましょう。

コミュニケーション方法は非常に重要です。上司に対して「マニュアルを読んでください」と言いたくなることもあるかもしれません。しかし、ほとんどの場合、皮肉や嘲笑を交えた態度をとる人は、対立を避けようとする人ほどキャリアアップできません。

また、ソフトウェアのビジネス上の価値を理解し、明確に説明することも重要です。自分が携

†13 多くの人は「Spring」を人気の Java フレームワークではなく、季節として認識しています。

わっている分野について理解していますか？ ビジネスパートナーが最も懸念していることは何か、把握していますか？

13.2　影響力を行使する

> リーダーシップとは影響力に他なりません。それ以上でもそれ以下でもありません。
> — John Maxwell、米国の作家・講演家

「影響力」とは、相手に自分の望む行動をとらせる技術であり、理想を言えば、相手に「自分のアイデアだ」と思わせながらそうさせることです。9章で学んだように、多くの人は単に命令を出せば済むと考えていますが、創業者やCEOでない限り、単に人に何かを命じるだけではうまくいきません。その代わりに、影響力という繊細な技術を身につける必要があります。次のプロジェクトでどのフレームワークを使うか決めることから、休憩室にどんなお菓子を置くかまで、影響力を発揮する機会は尽きることがありません。しかし、最終的な決定権を持っていない場合、どうすれば自分の望む結果を得られるのでしょうか？

会社には、組織図だけではっきりとわからないものの、影響力のある人々がいます。その人が誰なのかわからない場合は、特に興味深い業務を多く任されている人物に注目してみてください。優れたマネージャーは、自身の専門性の限界を認識しており、情報や助言を得るために、頼れる専門家たちを育成しています。CTOと直接連絡が取れない場合でも、その周辺にいる人物を通じてアプローチすることは可能です。「影響力のある人々に働きかける」ことは、非常に効果的です。

13.2.1　価値を理解し、明確化する

まず、説得したい相手に、読者が推奨する選択肢のメリットを明確に説明することから始めましょう。双方の立場の共通点を探しましょう。組織の実情に基づいてメッセージを構成してください。意思決定者は、市場投入までの時間を重視していますか？ 開発者の効率性でしょうか？ それともコスト削減でしょうか？ 流れに逆らうのではなく、読者の望む選択肢が、彼らのミッション達成にどのように貢献するかを示してください。例えば、意思決定者がリリースに伴うトラブルを減らしたいと考えていることがわかっているなら、自分のアプローチがその目標を達成する方法を主張に盛り込んでください。

同様に、意思決定者がどのように情報を入手しているのかを理解しましょう。彼らはどのような情報源を信頼しているのでしょうか？ 特定のアナリスト会社を頼りにしているリーダーもいれば、特定のテクノロジーベンダーの記事に特に耳を傾けるリーダーもいます。そうした情報源を引用しましょう。

13.2.2　影響力を行使するための戦略的アプローチ

相手が自分と意見が合わない場合、考えを変えてもらうのは難しいものです。基本的なアプローチは2つあります。1つは「ハンマー」です。相手に何かするよう命令する方法です。もう1つは

「忍者」です。自分が望んでいる行動を、あたかも相手自身のアイデアであるかのように思わせる方法です。誰かと人間関係を築いた経験があれば、前者の方法はめったにうまくいかないことはご存じでしょう。巧妙に、さりげなく働きかける必要があります。

強引な態度に出る人もいます。自分の意見を相手に押し通そうとして、大声で怒鳴ることを選ぶのです。しかし、そのやり方はほとんどの場合、逆効果になります。その代わりに、対話に努めましょう！ 目の前の課題を甘く見てはいけません。誰かを変えようと説得するのは非常に難しいことであり、忍耐と粘り強さが求められます。まず、小さなことから始めて成果を出し、そこから徐々に進めていきましょう。

読者の情熱が攻撃的だと受け取られるおそれがありますので、影響を与えようとしている相手の話にじっくりと耳を傾けてください。相手は何を言っているのでしょうか？ どのような懸念を抱いているのでしょうか？ 何が相手の心に響くのでしょうか？ 時には、相手の求める表現に合わせて、言い回しを調整するだけで済むこともあります。

読者の評判は、アイデアの質と同じくらい重要です。読者がその場にいないときでも、評判が読者のことを代弁してくれます。自分の評判がどんなものか、ご存じでしょうか？ もし確信が持てないなら、遠慮なく尋ねてみてください。その答えに納得できないかもしれませんが、状況を変えるにはそれしか方法はありません。

迷わず仲間を集めるようにしましょう。同じ考えを持つ人々に声をかけて、目標達成に向けて協力し合いましょう。数に力があることはよくあります。また、別の視点から物事を見ることで、自分では気づかなかった方向性が見えてくるかもしれません。

メッセージを伝える人物も、影響力において重要な要素となり得ます。時には、自分の主張を、自分以外の誰かから発信してもらう必要がある場合もあります。

誰が伝達するかが重要

数年前、筆者の一人が新しい会社に入社し、テスト駆動開発（TDD、Test-Driven Development）を導入することで「自分の存在をアピールしよう」と考えました。彼はTDD の概要、それが不具合の削減にどのように役立つか、そしてなぜ想像しているほど難しくないのかについて説明をまとめ、発表を行いました。しかし、彼のメッセージは、彼が期待していたような好意的な反応を得ることができませんでした。

しかし、彼の友人の一人（ここでは Steve と呼びましょう）はテスト導入に熱心だったため、彼は Steve のチームと密接に協力しました。数か月の間に、コードベースは、いわゆる泥沼のような状態から抜け出しました。TDD が自分のチームやコードにどれほどの影響を与えたかを確認した後、Steve は前述の TDD に関する講演を、ほぼ同じ内容で再び行いました。しかし今回は、反応が違いました。ほぼ全員から称賛の声が上がったのです。Steve のチームが行った取り組みを、コードベースの他の部分にも広げていこうという決意が示されました。

何が違っていたのでしょうか？ 誰もが Steve のことを知っていました。Steve は古参であ

り、長年その会社に勤めていました。一方、筆者である私は「新入り」でした。私に何がわかるというのでしょう？「誰が伝達するか」が重要です。意思決定者は誰に助言を求めるでしょうか？ その人は読者と同じ考えを持っているでしょうか？ 社内でより大きな権力や影響力を持つ人物を通じて物事を進めることを厭わないでください。スポットライトを浴びるほど満足感はないかもしれませんが、非常に効果的な方法となり得ます。

ステークホルダーとのコミュニケーションの取り方も重要です。また、組織内でどのようなメッセージが最も効果的かについても見極める必要があります。市場投入のスピードを最優先する企業もあれば、何よりもコスト削減を重視する企業もあります。状況に応じてメッセージを調整しましょう。例えば、スタートアップで働いている場合は、自分のアプローチによっていかに早く顧客に機能を提供できるかを強調します。一方、大企業で働いており、予算の制約があることがわかっている場合は、自分のアイデアがいかにコスト削減につながるかに焦点を当ててください。

影響力を行使する方法は、多くのソフトウェアエンジニアにとって教わる機会の少ないものです。しかし、その方法を学ぶことで、物事を成し遂げやすくなり、キャリアアップにもつながります。人々を動かすうえで重要なのは、ステークホルダーを理解し、適切に扱うことです。これについては、次で詳しく説明します。

13.2.3　ステークホルダーの扱い方

ステークホルダーが誰であるかを把握し、彼らを理解してこそ、効果的なコミュニケーションが可能となります。どのプロジェクトにも、チームメンバーから経営陣、さらには最終的に開発したソフトウェアを利用する人々まで、複数の利害関係者が存在します。ステークホルダーの中には、その存在が非常に明白な場合もあります。例えば、CTO が最優先で推進している取り組みに携わっている場合、CTO が関与するのは明らかです。しかし、中にはその存在がよくわからないステークホルダーもいるかもしれません。

時には、プロジェクトに直接関与していないにもかかわらず、プロジェクトを確実に助けることもあれば、妨げることもある人たちがいます。こうした「二次的な影響力を持つ人々」の存在は、「副社長がそんな案を承認するはずがない」といったコメントから感じ取れることがよくあります。場合によっては、こうした人々が存在することを示唆されるだけで、プロジェクトの進行が妨げられることもあります。その場にいない相手と交渉するのは困難であり、行き詰まりを打開するためには、彼らを文字通り「同じ部屋」に集める必要があるでしょう。

すべてのステークホルダーが同等な立場にあるわけではなく、組織内での関心度や影響力に応じて、彼らに割くべき時間や労力の量は異なります（**図13-2** を参照）。関心も影響力も低い人々には、あまり注力する必要はありません。しかし、関心も影響力も高い人々は、プロジェクトの成功にとって極めて重要です。

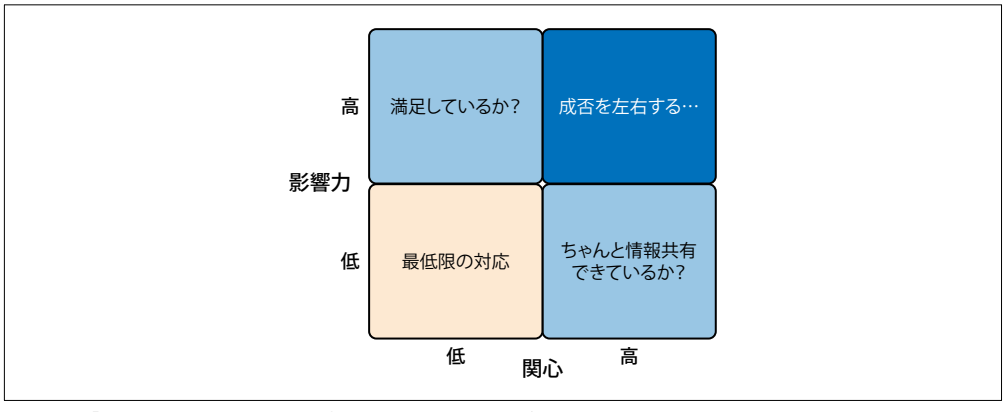


図13-2 「影響力／関心マトリックス」を理解することは、プロジェクトの成否を左右する

左上の象限に注目すると、この象限にいるステークホルダーについては、プロジェクトに対して安心できるよう、必要な情報を確実に提供する必要があります。彼らの時間を尊重し、希望や必要に応じて情報を得られるようにしましょう。時には、関心の度合いが低い状態から高い状態へと急速に変化することもあります。影響力は大きいが関心度が低いステークホルダーに対しては、透明性を保つことが重要です。

右上の象限に移ると、プロジェクトの成功に欠かせない人々がいます。彼らはプロジェクトの成否を左右する存在です。昇進や賞与の決定にあたっては、彼らの意見を軽視してはいけません。影響力が強く、関心も高い人々は、一緒に仕事をするのが難しい場合があり、多忙なスケジュールが障害となることもあります。しかし、彼らを無視することはできず、時折、土壇場で現れて、さまざまな事態を混乱させることもあるでしょう。

影響力があり関心の高い人々と積極的に関わりを持つ必要があります。また、相手のニーズに適応できるよう、自身の働き方を調整する必要があるかもしれません。プロジェクトを成功させるために、どのように協力するのが最善かについて話し合ってください。

また、沈黙が常に最善とは限らない、ということも重要です。相手にとって最適な方法を用いて、常に情報を共有するようにしましょう。具体的には、定期的なデモ、概要の説明、あるいは1対1のミーティングなどが考えられます[†14]。

右下の象限にいる人々を軽視するのは危険です。「関心は高いが影響力は低い」人々は、非常に大きな助けとなる可能性があります。彼らの情熱をうまく活用しましょう。その熱意は周囲にも伝染するものです。彼らは組織内で読者の強力な支持者となり、進んで協力してくれることも多いでしょう。また、彼らは「関心も影響力も高い」人々とのつながりを持っている場合もあります。新機能のテスト、数時間の業務見学、あるいはインタビューへの協力など、関心が高い人々はかけが

†14 そう、TPS レポートには表紙をつける必要があるかもしれません（訳注：映画"Office Space"《邦題『リストラ・マン』》に登場する話です。無駄な手続きや面倒な社内文化を皮肉るときに、よく使われます。「TPS レポート」という言葉には特に意味がありません。意味のないものの代名詞として使われています）。

えのない存在です。彼らを軽視すればプロジェクトに積極的に反対してしまうかもしれません。逆に、彼らを味方につけておけば、物事は順調に進むでしょう。

ソフトウェアそのものは0と1の羅列ですが、実のところソフトウェアとは「人」がすべてです。プロジェクトを取り巻くステークホルダーのネットワークを理解できるかどうかが、取り組みの失敗と大成功を分ける鍵となります。

13.3　時間管理

時間管理は極めて重要です。時間は再生不可能な資源であり、増やすことも、さらに手に入れることもできません。スケジュールに割り当てられる時間は常に不足しています。会議、コードレビュー、本番環境の問題への対応、TPSレポートの作成など、1日の時間は決して足りることはありません。読者は自分自身の時間の番人でなければなりません。繰り返しになりますが、手元にある重要な業務に取り組むために、中断されない時間を確保するようにしてください[†15]。少し息をつく余裕を持つようにしましょう。

また、自身の生活リズムにも気を配る必要があります。読者は朝型の人ですか、それとも午後の早い時間帯に最も頭が冴えているでしょうか？ できる限り、自分が最も調子の良い時間帯に合わせてスケジュールを組み、集中力が落ちていると自覚している時間帯には、ルーチンワークをこなすようにしましょう。柔軟に対応する必要はありますが、自分のリズムに**合わせて**働き、現実的な目標を設定することで、より幸せで生産的な日々を送ることができるでしょう。

考えるための余裕を持つことは、機能の完成や不具合の修正において、大きな違いを生むことがあります。Rich Hickeyは、ハンモック駆動開発（https://oreil.ly/uoX_I）について素晴らしい講演を行い、中断されない思考時間の重要性を訴えました。本番環境で発見されたバグを修正するのが最もコストがかかるのは明らかです。だからこそ、多くの開発者は自動テストを作成し、ソフトウェア開発プロセスには品質保証のステップが組み込まれているのです。しかし、最もコストのかからない解決策は、そもそもバグを発生させないことです。

> 「ソフトウェアに関する大きな問題のほとんどは、誤解に起因するものです。私たちは、行動を起こす前に、自分が何をしようとしているのかをよく理解できていません。そして、とにかく急いで、急いで、急いで、すべてをこなしてしまうのです。」
>
> — Rich Hickey、Clojureプログラミング言語の開発者

ソフトウェアの世界では、誤解による問題が常につきまといます。用語は複数の意味を持つことが多く、一見すると同じ意味の言葉のように見えるものが、実際には微妙なニュアンスの違いによって区別されることがよくあります。スケジュールに余裕を持たせることで、設計の及ぼす影響についてじっくりと考えるための貴重な時間を確保できます。時間をかければ、結果的に時間を節

†15　スケジュール管理システムによっては、コア業務時間とともに集中時間を簡単に設定できるものもあります。

約できます[†16]。また、役割が異なればスケジュールのタイプも異なるということを理解しておくことも重要です。日々ものづくりに専念する人々（ソフトウェアエンジニアなど）と、管理業務に注力する人々の間には、明確な違いがあります。

13.3.1 エンジニアのスケジュール

ソフトウェア開発は、科学というよりはむしろ職人芸に近いものです。そこには Paul Graham が「エンジニアのスケジュール（maker's schedule）」（https://oreil.ly/aPQld）と呼ぶようなものが必要です。一方、マネージャーのスケジュールとは、1 時間単位の会議の連続に過ぎません。文字通り、どの 1 時間も他の 1 時間と変わりません。そのため、マネージャーはスケジュールを組む際、「自分のカレンダーに空きがある場所を探す」というアプローチをとりがちです。そして、このアプローチは、意思決定の合意形成や予算案の検討には有効です。しかし、ソフトウェア開発においては、あまりうまく機能しません。

ソフトウェアエンジニアとして、普段どのような時間単位で物事を考えていますか？ 1 日単位でしょうか？ 半日単位でしょうか？ 1 日は、1 時間単位のブロックの寄せ集めではありません。午前 10 時の 30 分間の会議は、午前の時間を有意義な作業を行うには小さすぎる時間ブロックに分割します。結果として、午前中を台無しにしてしまいます。難しい作業を始めても意味はありません。なぜなら、タイミングの悪い会議のために作業を中断し、コンテキストを切り替えなければならないからです。読者は自らの時間の番人でなければなりません。それが「エンジニアのスケジュール」です。

ソフトウェア開発とは、結局のところ、問題を頭の中に読み込み、アプリケーションのメンタルモデルを構築することです。デバッグの場面を考えてみてください。ブレークポイントを設定し、プログラムを再実行して問題を再現し、システムの内部状態をたどっている最中に、突然、誰かに邪魔をされるのです。頭の中で積み上げてきた思考の積み木が、一瞬にして崩れ落ちてしまいます。その状態に戻るまでに、どれだけの時間がかかるでしょうか？

まだ手遅れではありません。生産性を守るための対策を講じることができます。テクノロジーをうまく活用し、通知をオフにしてください。ついつい確認したくなってしまう場合は、メールやメッセージングアプリを閉じてください。集中が必要なときは、通知の設定を見直し、「サイレントモード」を活用して、気が散る要因を最小限に抑えましょう。1 週間のスケジュールの見直しも検討してみてください。一部の組織では、会議のない時間帯や曜日を定めており、エンジニアが仕事に集中できるようにしています。会議を週に 1 日か 2 日に集中させることで、業務の妨げを一か所にまとめ、より集中できる時間を確保できます。もし読者の会社でそのような取り組みが行われていない場合は、ぜひ試してみることを提案してみてください！

13.3.2 タスクに集中する

時間管理において重要なのは集中力を維持することです。しかし、これは非常に難しいことでも

†16 あるいは、ゆっくりこそがスムーズであり、スムーズこそが速いのです。

あります。絶え間なく届くメールやインスタントメッセージ、そしてインターネットによる無限の気晴らしに囲まれていると、タスクに集中しつづけるのは一苦労です。時には、そのような精神的な葛藤は、その作業が後回しにできるものであることを示している場合もありますが、ToDo リストを片付けるために何らかの仕組みを取り入れるようにしてください！ 計画やアプローチを持つことで、精神的な負担が軽くなり、最終的には集中力を習慣化できます。あちこちと気を散らすのではなく、仕事を成し遂げるための枠組みを自分自身に与えるのです。

ポモドーロ・テクニック（https://oreil.ly/6Wl00）を取り入れてみてはいかがでしょうか。端的に言えば、これは極めて小規模なアジャイルソフトウェア開発のようなものです。取り組むタスクを選び、タイマーを 25 分にセットして、そのタスクに取り組みます[†17]。もし他にやらなければならないことが思い浮かんだら、それをメモに残して、手元のタスクに戻ってください。タイマーが鳴ったら、タスクを切り替えて休憩を取ります。これを繰り返します。4 つのタスクを終えたら、少し長めの休憩で自分を労ってください。

Focus Course（https://oreil.ly/p1Jrp）のような手法も、目標を設定し達成するための仕組みを提供してくれます。James Clear（https://oreil.ly/-lvOQ）などが示しているように、習慣には非常に大きな力があります。インターネット上には、コンピュータサイエンス関連だけでなく、それ以外にも質の高い情報があります。このような情報を積極的に取り入れるようにしましょう！

現役のソフトウェアエンジニアとして、1 日の時間がいくらあっても、予定していることをすべてこなすには足りないと感じることがあるでしょう。時間を主体的に管理する方法を身につければ、重要なことに集中でき、生産性を高められます。

13.4　まとめ

ソフトウェアエンジニアとしてのキャリアにおいて、ソフトスキルは最初に学ぶことではないかもしれません。しかし、効果的なコミュニケーション能力を持つ人は昇進しやすい傾向にあります。適切なタイミングで適切な手法を用いることが、目標達成において決定的な差を生むのです。時間管理を徹底すれば、周囲が慌てふためいている中でも冷静さを保てます。最新のテクノロジーに情熱を注ぐことは大切ですが、キャリア全体を通じて役立つソフトスキルを軽視しないようにしてください。

13.5　実践に移す

ソフトスキルを磨く機会は、さまざまな場面で得られます。チームメンバーの前や地域のユーザーグループで、進んでプレゼンテーションを行ってみてください。作業や思考に充てる時間を、スケジュール管理システムに積極的に確保しましょう。会議をスケジュールする際は配慮するようにしてください。アジェンダを明確にし、必要な人だけを招待し、会議時間を 60 分以外の時間枠

†17「ポモドーロ（トマト）」型のタイマーなら、なお良いでしょう。

で設定するよう心がけてください。

数週間、ポモドーロ・テクニックを試してみてください。生産性は向上しましたか？ またメールを送る前に、ひと息ついて、別のコミュニケーション手段のほうが効果的ではないか考えてみてください。チームに新しいアイデアやツール、テクニックを取り入れて、影響力を高めるスキルを磨いてみましょう。

13.6 関連リソース

- 『Thinking Architecturally (report)』Nathaniel Schutta 著（O'Reilly、2018 年）
- 『How to Win Friends and Influence People』Dale Carnegie 著（Simon & Schuster、1936 年）
  和書『人を動かす』（創元社、2023 年）
- 『Influence, New and Expanded: The Psychology of Persuasion』Robert B. Cialdini 著（Harper Business、2021 年）
  和書『影響力の武器［新版］』（誠信書房、2023 年）
- "Maker's Schedule, Manager's Schedule"、Paul Graham 著（https://oreil.ly/S7vkU）
- "Hammock-Driven Development"、Rich Hickey による講演（https://oreil.ly/otsDg）

14章
キャリアマネジメント

素晴らしい仕事をする唯一の方法は、自分の仕事を愛することです。
まだそれを見つけていないのなら、探しつづけてください。妥協してはいけません。
— Steve Jobs

ソフトウェアエンジニアとしてのキャリアをスタートした当初は、その道のりは一本道のように見えるかもしれません。つまり、コーディングを学び、スキルを磨き、そしてコーディングを続けるという道です。しかし、キャリアを重ねるにつれて、技術の世界には、技術力、創造性、そして成長の機会を組み合わせた、数多くの道が開かれていることに気づくでしょう。

本章の最初の節では、キャリアパスの計画の立て方について学びます。ここでは、自分が何に情熱を持っているかを見極めることが中心となります。その情熱が、長く充実したキャリアにつながるからです。次に、自分の興味が何であるかを見つけたら、自分にどのような選択肢があるかを知ることが重要です。

次の節では、キャリアを築いていくうえで役立つ実践的なヒントやコツを紹介します。自身の成果を記録する方法、インポスター症候群を克服する方法、プロフェッショナルなネットワーク（人脈）を築く方法、そして面接のコツなど、キャリアアップに役立つ戦略を学びます。

最初は選択肢が多すぎて圧倒されるかもしれません。本章では、読者のキャリアの道筋を見定め、その可能性を最大限に活かすための手助けをします。自分の進路を自ら切り開き、どこへ向かいたいのかを見据えたうえで、決断を下す必要があります。

14.1　キャリアパスを計画する

キャリアの初期段階では、プログラミングに深い情熱を抱いていることでしょう。それは素晴らしいことです。しかし、時が経つにつれて興味が移り変わることもあります。興味や目標は時間とともに変化していくものだと理解しておくことが大切です。

本章では、まずキャリアプランを立てるお手伝いをします。まずは、重要な第一歩である「自分の情熱を見つけ、コアとなる興味を特定すること」から始めましょう。これを土台とすることで、

充実感があり、かつ持続可能なキャリアパスへとつながっていくことでしょう。

次に、さまざまなキャリアの選択肢について探っていきます。このような選択肢については、これまで考えたこともなかったのかもしれません。そして、自分の情熱を見極め、キャリアの方向性を決めた後は、選んだ道を歩んでいくために、どのように戦略的な決断を下すべきかについて議論します。

14.1.1 自分の情熱を注げるものを見つける

開発者一人ひとりが、独自のスキル、興味、価値観の組み合わせを持っています。そして、この組み合わせがそれぞれのキャリアを形作ります。コードを書くことは技術者としてのキャリアの基盤となります。コーディングのどの部分にワクワクするかを発見できれば、充実感があり、長く続けられるキャリアパスを築く助けとなります（自分の興味を見極め、育むための戦略については、12章のコラム「情熱に導かれて」を参照してください）。

14.1.1.1 さまざまな分野やテクノロジーを探求する

すでに興味のあるテクノロジーや分野を見つけているかもしれませんが、もしそうでない場合でも、ソフトウェア開発の世界には数え切れないほどのビジネス分野やテクノロジーが存在します。キャリアの初期段階では、さまざまな分野の仕事を行い、自分が本当に情熱を注げるものを見つけてみてください。以下に、取り組むべきプロジェクトの例をいくつか紹介します。

- 毎日何百万人もが利用するコンシューマ向けアプリケーション
- ビジネスを支えるエンタープライズシステム
- 分析やインサイトを得るためのデータ集約型アプリケーション
- システムの稼働を維持するインフラストラクチャと DevOps
- ゲームやインタラクティブな体験
- 学習を支援する教育テクノロジー
- 患者の治療成果を向上させる医療システム
- 買い手と売り手を結びつける EC プラットフォーム
- モバイルアプリケーション開発
- 人工知能と機械学習システム

分野ごとに、異なる課題や影響、文化が存在します。例えば、顧客に直接影響を与えることにやりがいを感じるなら、コンシューマ向けアプリケーションが天職かもしれません。もし、以前からビデオゲームが大好きで、自分だけのゲームを作りたいと夢見てきたなら、ゲームやインタラクティブ体験の分野に進むべきかもしれません。

自分がどのテクノロジーや分野に興味があるのかわからない場合は、それを知るには、実際にいろいろ試してみるしか方法はありません。

14.1.1.2 サイドプロジェクトに挑戦する

キャリアの初期段階では、担当するプロジェクトの種類が限られてしまうおそれがあります。毎日同じ分野で同じテクノロジーを使って仕事をしているだけでは、新しいことに触れる機会がありません。

サイドプロジェクトは、自分が興味を持ちそうなテクノロジーや分野を探求するために、リスクの低い環境を提供してくれます。本業とは異なり、何を作るか、どのように作るかについて、完全に自由に決められます。もし、読者が主にバックエンド開発者であり、フロントエンドの仕事に興味があるなら、見栄えの良いフロントエンドを構築する必要があるプロジェクトに取り組んでみてください。

何を使って開発するかを選ぶ際は、自分が働きたいと思う 10 社の求人情報を参考にしてみてください。それらの求人情報で繰り返し登場するテクノロジーを特定しましょう。このようなテクノロジーは明らかに需要が高く、時間を割く価値があります。

ただし、現在流行しているものだけに限定しないでください。普段の業務とは**異なる**テクノロジーを、あえて選んでみるのもよいでしょう。普段とは異なるプログラミング言語や開発環境、プラットフォームに取り組むことは、自分への挑戦となり、ソフトウェアエンジニアとしての真の成長につながります。こうした取り組みは、ソフトウェアの仕組みに対する理解を深め、自分が何にワクワクするかを発見する機会をさらに広げてくれるでしょう。

個人的なサイドプロジェクトに取り組んだり、オープンソースに貢献したりすることは、履歴書で自分をアピールするための素晴らしい方法です。必須ではありませんが、社内や他社への応募の際、こうした活動は他の候補者との差別化につながります。

さまざまなことを試していく中で、どの要素が活力を与えてくれるか、逆にどの要素がエネルギーを奪うか、注意深く観察してみてください。

コーディング中に確認すべき点は以下のとおりです。

- どのような問題に取り組むとワクワクしますか？
- ユーザーインターフェースの構築と、データやアルゴリズムの扱いのどちらを好みますか？
- 複雑なシステムのデバッグは、やる気が出るものですか、それともイライラするものですか？
- デザインの創造的な側面と、バックエンドシステムの論理性と厳密性が求められる側面のどちらを楽しみますか？

さまざまなコーディング活動について「+1」（活力が湧く）または「-1」（エネルギーを消耗する）を記入し、その理由を簡単にメモしたシンプルな日記をつけてみましょう。そこに現れるパターンに注目してみてください。ピクセル単位で正確に CSS を揃える作業は自分には向いていないと気づくかもしれません。あるいは、インタラクティブな Web 体験を構築することが、まさにプログラミングの醍醐味だと実感するかもしれません。

自分の問題を解決するサイドプロジェクト

Dan です。サイドプロジェクトに取り組む際は、自分にとって役立つものや、自分の問題を解決するものを構築するとよいでしょう。私は物心ついた頃から個人サイト（http://danvega.dev/）を持っています。このサイトのおかげで、インターネット上に自分だけの小さなスペースを持ち、取り組んでいることを紹介できます。オープンソースプロジェクトに参加し、そこで学んだことをコミュニティに還元するためにブログを始めました。何より、この作業が本当に楽しいということです。そして、本業では決して使うことのなかったフレームワークやツールに触れる機会を得られたということです。

14.1.2　キャリアの選択肢を探る

キャリアの初期段階では、技術の習得に集中するあまり、木を見て森を見ずの状態になりがちです。最初の職に就くためにソフトウェア開発の基礎を学んだり、昇進を目指してスキルを磨いたりしていることでしょう。その時点では、コードを書き、自分のスキルを向上させることが、キャリアのすべてだと考えているのです。

ご存じないかもしれませんが、ソフトウェア開発には、単にコードを書くことにとどまらない、さまざまなキャリアパスが存在します。充実したキャリアを築くための鍵は、自分の興味や強みに沿った方向性を意識しつつ、新たな機会を柔軟に受け入れる姿勢でいることです。

この節では、技術専門職としての道からリーダーシップの道、さらにはその他の選択肢に至るまで、さまざまなキャリアパスを探っていきます。重要なのは、どのような選択肢が存在するのかを知り、今すぐ決断を下して、自分に合った道を進み始めることです。



キャリアに関する決断のうち、真に取り返しのつかないものは 2 つだけです。それは、重大な倫理違反と、同僚との関係を断ち切ってしまうことです。それ以外はすべて変えることができます。適切な計画と努力さえあれば、テクノロジーや会社、さらにはキャリアパスさえも、方向転換できます。

14.1.2.1　技術専門職のキャリアパス

技術専門職のキャリアパス（インディビジュアルコントリビュータ《IC、Individual Contributor》トラックとも呼ばれます）は、開発者が管理職へ転向することなく、技術の第一線で実務に携わりながらキャリアを積む道を提供します。このキャリアパスでは、深い技術的知識、システム設計スキル、そして技術的リーダーシップが評価されます。企業では通常、シニアエンジニアからスタッフ、プリンシパル、場合によってはディスティンギッシュドエンジニアやフェローエンジニアといったレベルへと昇進するキャリアパスを設けています。役職名や組織構造は企業によって異なります。

アーキテクト

ソフトウェアアーキテクトは、システムの全体構造を設計し、プロジェクトや組織全体の方向性を決定するような大局的な技術的判断を下します。ビジネス要件を技術的な解決策へと変換すると同時に、開発チームをリードするための開発基準を確立します。全体を俯瞰する視点に立つ一方で、有能なアーキテクトは、自身の設計が実用的であることを確認し、開発チームからの信頼を維持するために、自らコードを書く実務にも携わります。テクニカルアーキテクトは、以下の業務を行います。

- システムアーキテクチャを設計し、重要な技術的判断を下す
- ビジネス上のニーズを考慮しつつ、技術的なトレードオフのバランスを取る
- 技術に詳しくないステークホルダーに技術的な概念を説明するために、優れたコミュニケーション能力が求められる
- 進化しつづける技術の最新動向を把握するために、常に好奇心と学習意欲を持ちつづける
- 通常、8 年以上の開発経験を有している

スタッフエンジニア

スタッフエンジニアは、コーディングの実務と広範な影響力を兼ね備えた、シニア技術リーダーです。複雑な問題を解決すると同時に、自身の直属チームの枠を超えて技術的な方向性を示します。このキャリアパスは、コードの実務に携わりつつ、より広範な影響力を発揮したいと考えている開発者のためのものです。

- 技術的な卓越性とエンジニアリングのベストプラクティスに注力する
- 他の開発者を指導し、技術的な意思決定に影響を与える
- 複数のチームやプロジェクトにまたがって業務を行うことが多い
- 通常、特定の分野における深い技術的専門知識が必要とされる
- シニアレベルでは、管理職と同等かそれ以上の報酬が得られる場合がある
- 役職としては、シニアデベロッパー、テックリード、プリンシパルエンジニア、スタッフエンジニアなどがある

これらの技術専門職のキャリアパスは、管理職とは異なる充実した選択肢をエンジニアに提供します。難解な技術的課題の解決に情熱を持って取り組みつつ、自身の影響力と報酬を拡大していくことを可能にします。

14.1.2.2 リーダーシップのキャリアパス

リーダーシップのキャリアパスでは、開発者がチームを率い、プロダクトの方向性を決定し、組織の成功を牽引する機会が得られます。このキャリアパスでは、戦略的思考、対人スキル、そして技術的な取り組みをビジネス目標と整合させる能力が評価されます。企業では通常、これらの役割を管理職の階層構造として位置づけ、チームリーダーからディレクター、バイスプレジデント、そ

してCレベル（経営幹部）の役職へと昇進していく仕組みとなっています。ただ、その形態は組織によって異なります。

エンジニアリングマネージャー

エンジニアリングマネージャーは、人材、プロセス、成果に重点を置いて開発チームを率います。効果的なチームを構築し、障害を取り除き、技術的な取り組みをビジネス目標に整合させながら、個人の成長を支援します。マネジメントは誰にでも向いているわけではありませんが、人や組織づくりに関心のある方にとっては、やりがいのあるキャリアパスとなります。

- チームビルディング、キャリア開発、プロジェクトの遂行に重点を置く
- 優れた対人スキルと組織運営能力が求められる
- 技術的な知識は有するものの、コードを書くことはほとんど、あるいはまったく行わない
- チームの満足度とビジネス目標のバランスを取る必要がある
- 報酬には、多額のボーナスや株式報酬が含まれることが多い

テクニカルプロダクトマネージャー

テクニカルプロダクトマネージャーは、ビジネスニーズと技術的な実装の橋渡し役を務めます。プロダクトビジョンを策定し、機能の優先順位を決定し、技術チームやビジネスのステークホルダーと連携して、価値あるソリューションを提供します。ビジネスとテクノロジーの架け橋となる役割を楽しく感じる開発者向けです。

- ビジネス要件を技術的なソリューションへと変換する
- 組織全体のステークホルダーと緊密に連携する
- 技術的な理解力とビジネスセンスの両方が求められる
- コーディングよりも戦略的な思考を重視することが多い
- シニアプロダクトマネージャーや戦略担当の役職へとキャリアアップする可能性がある

このようなリーダーシップのキャリアパスを通じて、技術志向の専門家は、これまでの経験を活かしながら、人材管理、戦略的思考、およびビジネス戦略との整合などの新たなスキルを身につけられます。

14.1.2.3 その他のキャリアパス

従来の技術専門家やリーダーシップのキャリアパス以外にも、ソフトウェアエンジニアは、技術的な専門知識と他の分野を融合させた新たなキャリアパスを歩むことができます。多くの場合、こうした新たなキャリアパスでは、プログラミングの知識を活かしながら、コミュニケーション、教育、ビジネス、あるいは専門分野における補完的なスキルを身につけます。テクノロジーが進化しつづける中、こうしたハイブリッドな役割には、テクノロジーエコシステムに独自の貢献ができるような魅力的な機会が数多く提供されます。

デベロッパーアドボケイト

デベロッパーアドボケイトは、企業と技術コミュニティの架け橋としての役割を果たします。ユーザーが技術を理解し、活用できるよう支援すると同時に、コミュニティからのフィードバックをプロダクトチームに伝える役割も担っています。この比較的新しいキャリアパスは、技術的なスキルとコミュニティとの関わりを兼ね備えたものです。

- コンテンツの作成、講演、開発者コミュニティの構築
- 優れたコミュニケーション能力と指導力が求められる
- 出張や人前で話す機会が頻繁にある
- テクノロジーのトレンドを常に把握しておく必要がある
- 将来的には、デベロッパーリレーションズやテクニカルマーケティングへのキャリアアップにつながる

セールスエンジニア

セールスエンジニアは、深い技術的知識と情熱を兼ね備え、顧客が抱える課題に対して最適なソリューションを見つけるお手伝いをします。営業プロセスにおいては、プロダクトのデモや技術的な懸念への対応、見込み客との信頼関係の構築を通じて、技術の専門家としての役割を果たします。技術の解説や人との関わりを好む開発者向けです。

- 技術的な能力と顧客にとってのビジネス価値を結びつける
- 技術的な専門知識に加え、優れたプレゼンテーション能力と対人スキルが求められる
- 顧客先や展示会への出張が頻繁にある
- 複雑な技術的概念を理解し、さまざまな人々に向けてわかりやすく説明する能力が必要
- シニアセールス職、テクニカルセールスマネジメント、カスタマーサクセス職へのキャリアアップの可能性がある

技術系起業家

技術系起業家は、自身のエンジニアリングの専門知識を活かし、プロダクトやビジネスをゼロから構築します。市場の機会を見極め、ソリューションを考案し、持続可能なベンチャーを立ち上げるうえでの課題を乗り越えていきます。自身のベンチャーを立ち上げたいと考えている方向けです。

- 技術的なスキルとビジネス開発を融合させる
- リスクは高いが、大きな見返りが得られる可能性がある
- 技術とビジネスの両分野にわたる幅広い知識が必要
- 本業を続けながら、サイドプロジェクトとして始めることが多い
- 成功した企業の設立や、スタートアップへの早期参画につながる可能性がある

ソフトウェアエンジニアにとって、デベロッパーアドボケイト、セールスエンジニア、技術系起業家だけがその他の選択肢というわけではありません。SRE（Site Reliability Engineer）や、DevOps（Development and Operations）など、他にも多くのキャリアパスが存在します。この

節の冒頭でも触れたように、キャリア全体を通じて、新たな機会に対して柔軟な姿勢を保ちつつ、大まかな方向性を定めることが、成長には不可欠であることを忘れないでください。

興味のある分野で働いている人たちに話しかけてみてください。アドバイスを求めてみましょう[†1]。多くの人は、喜んで指導してくれます。上司にも自分の興味や関心について伝えてください。上司が読者の希望を知らなければ、上司もそれを実現するための手助けはできないのです。多くの企業では、公式・非公式を問わず、他社の業務を見学する機会や、一定期間別の部署で働く機会を提供しています。自分のキャリア形成は自分自身に責任があるのですから、積極的に自分の意見を主張するようにしてください！

14.1.3 目標から逆算する

ソフトウェアエンジニアとして、複雑な問題を、より小さく扱いやすい単位に分解するという重要なスキルを身につけてきました。このアプローチにより、単一の問題として捉えていては手に負えないような課題にも取り組むことができるのです。この原則は、キャリア形成においても同じように適用できます。

キャリアの目標を達成するには、長期的なビジョンを、達成可能な小さなマイルストーンに分割し、そこから逆算して計画を立てることが効果的です。3年から5年後に自分がどうなっているかを考えてみてください。複雑なシステムを設計するテクニカルアーキテクトとして働いているでしょうか？ 開発チームを統括するエンジニアリングマネージャーとして働いているでしょうか？ あるいは、クライアントと協力してプロダクトの方向性を決めているのでしょうか？ どのような道を選ぶにせよ、その過程にある中間目標を明確にすることで、計画的に目標に到達できるのです。

例えば、テクニカルアーキテクトになることを目標としている場合、以下のようなマイルストーンになるでしょう。

5年目以降

- 主要プロジェクトにおけるアーキテクチャの決定を主導する
- チーム横断的な技術標準を策定する
- 直属のチームの枠を超えた影響力を築く
- プレゼンテーションや効果的な会議の進行など、技術以外のスキルを磨く

3〜5年目

- 技術プロジェクトを主導する
- ジュニア開発者を指導する
- 分散システムに関する経験の習得
- スケーラビリティおよびパフォーマンス最適化に関する専門知識の習得
- アーキテクトと連携し、成果物の作成を支援する

†1 コーヒーやランチに誘うと、うまくいく可能性がさらに高まるでしょう。

1〜2 年目

- 現在の技術スタックを習得する
- 複雑化しつづける技術的課題に取り組む
- システム設計の原則を学び始める
- 組織内のアーキテクト 1〜2 名と定期的に 1on1 の面談を設定する



自分が目指すポジションにいる人々と交流しましょう。アーキテクトと仕事をする機会があれば、彼らがジュニアからシニア、そしてアーキテクトへと至った道のりについて尋ねてみてください。彼らの経験から学ぶことは、自身のキャリアに関する決断を下すうえで役立つでしょう。

14.1.4 計画的なスキル習得

学ぶことが好きなら、素晴らしいキャリアを選択したと言えます。ソフトウェア開発者として、生涯にわたって学びつづけることを決意したわけですから。しかし、現実には、すべてを学ぶことはできません。どんなに学びたいと思っても、単純にそれだけの時間を持つことは不可能なのです。

前節では、キャリアのロードマップを作成する方法について学びました。目標に向かって着実に進むためには、単に流行っているという理由だけで、あらゆる新しいテクノロジーを追いかけないようにしてください。実のところ、こうした流行のツールの多くは、数年後には消えていることでしょう。流行を追いかけることは、時に、「履歴書駆動開発」と呼ばれます。有意義なキャリアアップのためではなく、単に履歴書に書き加えるためだけにテクノロジーを学ぶことを指します。

12 章では、学習方法に関する有益なヒントをいくつか学びました。この重要なスキルと、計画的なスキル習得を組み合わせることで、目標をより早く達成できるようになります。行き当たりばったりの学習では、行き当たりばったりの結果しか得られません。キャリアにおいて効果的に成長するためには、新しいスキルを習得するための体系的なアプローチが必要です。

計画的なスキル習得を促進するためには、基礎をしっかりと固め、T 字型の専門性を養い、戦略的な学習に注力するようにしてください。

14.1.4.1 コアスキル

コアスキルは技術的専門性の基盤となり、技術トレンドの変化にかかわらず常に価値を持ちつづけます。これらのスキルは、T 字型スキルセットにおける「縦棒」に相当する基礎部分です。複合的なスキルを追加する前に、強固な基盤を築くことが不可欠です。まずは、ソフトウェアエンジニアリングの以下の基本を習得することに注力しましょう。

- データ構造とアルゴリズム
- デザインパターン
- テスト手法

- バージョン管理
- データベース設計

14.1.4.2　T字型の人材育成

しっかりとした基盤を築き上げたなら、専門性を広げていきましょう。T字型の人材育成では、専門分野の深い知識と幅広い知識を兼ね備えることで、絶えず進化しつづける業界において、高い専門性と柔軟性を兼ね備えたエンジニアへと成長できます。

- 縦棒は、主要なテクノロジースタックに関する深い知識を表しています。
- 横棒は、関連する技術全般にわたる幅広い知識を表しています。

例えば、バックエンドのJava開発者であれば、次のようになります。

- 深い知識：Java、Spring Framework、データストレージ（SQL／NoSQL）
- 幅広い知識：基本的なフロントエンドスキル、DevOpsの実践、システム設計

14.1.4.3　戦略的学習

限られた時間を新しいスキルの習得に費やす前に、以下の基準に基づいて、それぞれの学習機会を評価してください。

- これは読者のキャリア目標と合致していますか？
- このテクノロジーやスキルは、3〜5年後も有用であると考えられますか？
- この知識は、読者に競争上の優位性をもたらしますか？
- これにより、現在の職務やチームにおいて、読者の価値が高まりますか？
- この新しいことにワクワクしますか？

計画的なスキルの習得は、学習を「行き当たりばったりのもの」から「戦略的なもの」へと変えます。まずコアスキルに焦点を当て、T字型の専門性を養い、キャリア目標に照らして学習の機会を評価します。その結果、どこに時間を投資すべきかについて、より適切な判断を下せるようになります。この的を絞ったアプローチは、成長を加速させるだけでなく、読者自身のキャリアパスにおいて、真に重要な分野でしっかりとしたマスタリーを身につけることにつながります。

14.1.4.4　自分だけのテクノロジーレーダーを作成する

計画的なスキルの習得が重要であることを理解したところで、次は「テクノロジーレーダー」について学び、それを実践しましょう。**テクノロジーレーダー**は意思決定およびコミュニケーションツールです。組織が新たなテクノロジー、ツール、フレームワーク、プラットフォーム、手法を長期的に追跡・分類するのに役立ちます。特に、ソフトウェア開発チーム、プロダクトイノベータ、そしてテクノロジーリーダーにとって非常に有用です。もともとは、Thoughtworks

（https://oreil.ly/n3p-r）が組織向けに開発した概念ですが、個人的にも利用することが可能です。レーダーには 4 つの象限があります。

1. 言語とフレームワーク
2. ツールとインフラストラクチャ
3. プラットフォームと API
4. 技術と方法論

各象限内で、テクノロジーをリング状に分類してください。

Adopt（採用）
現在使用し、習得しているテクノロジー

Trial（試用）
現在積極的に学んだり試したりしているテクノロジー

Assess（評価）
現在調査中ですが、まだ学習を始めていないテクノロジー

Hold（非推奨）
現時点では導入を見送ることにしたテクノロジー

テクノロジーレーダーを初めて作成するときは、少し気が重く感じるかもしれません。まずは既知の領域から始めてみましょう。現在の技術的な状況を整理することから着手し、徐々に範囲を広げていくのです。作成方法は、紙に書き出すようなシンプルな方法から、Thoughtworks が提供する Build Your Own Radar（BYOR）（https://oreil.ly/MxSJv）というツールを使う方法までさまざまです。自分自身やチームにとって最適な方法を選べばよいのです。ただし、重要なのは、有意義な議論を促し、将来の意思決定の指針となるように、自社のテクノロジーエコシステムを可視化することです。

1. 現在使用しているテクノロジースタックを列挙してください。日常的に使用している言語、フレームワーク、ツールを含めてください。それぞれの習熟度を明記し、知識が不足していて補うべき点があれば特定してください。
2. 興味のある求人情報に記載されているテクノロジーを追加してください。今後 1〜2 年の間に就きたいと思うポジションを確認してください。それらの求人情報には、どのようなテクノロジーが頻繁に挙げられていますか？ また、「必須」と「あれば尚可」の条件として挙げられているテクノロジーを比較してみてください。
3. テクノロジー関連のブログやカンファレンスから、最新の技術を取り入れましょう。自分自身の専門分野で注目を集めつつあるソリューションに注目してください。あらゆるトレンドを追

いかけるのではなく、業界のリーダーたちが採用しているものの中にどのような傾向があるかを見極めましょう。

4. 定期的に見直しと更新を行ってください。四半期ごとにリマインダーを設定し、テクノロジーレーダーを見直しましょう。テクノロジーは、読者のキャリアとの関連性が変化するにつれて、別のカテゴリに移動する場合があります。



忘れないようにしてください。読者の「レーダー」は、現在主流のテクノロジーと将来のトレンドの両方を反映しているべきです。このレーダーは、短期的および中期的に何を学びたいかに意識を集中させるためのツールなのです。

具体的に示すために、**表14-1** では、ジュニア開発者のテクノロジーレーダーがどのようなものになるかの一例を挙げています。

表14-1　ジュニア開発者のテクノロジーレーダーの例

	Adopt（採用）	Trial（試用）	Assess（評価）	Hold（非推奨）
技術と方法論	システム設計パターン	マイクロサービスアーキテクチャ		
ツールとインフラストラクチャ	バージョン管理用のGit	Docker とKubernetes		
プラットフォームと API	データアクセス用のSQL		クラウドプラットフォーム／GraphQL	
言語とフレームワーク	Java、Spring Boot			衰退しつつあるテクノロジー、実験的なフレームワーク

定期的に「レーダー」を見直し、更新する習慣をつけてみてください。テクノロジーは急速に進化しており、今日重要だと思われていることが、明日には時代遅れになっているかもしれません。キャリアロードマップと「レーダー」を組み合わせて活用し、目標に向かって着実に進んでいきましょう。これらの決断はどれも絶対的なものではありません。自分のニーズや希望が変われば、それに合わせて柔軟に対応すべきです。自分自身の興味や関心も変化していくものです。何よりも大切なのは、自分の情熱です。そのテーマにワクワクできないのであれば、最も貴重な資産である「時間」と「注意力」を注ぎ込むことは決してできないでしょう。

14.1.5　人生の節目に合わせたキャリアの選択

キャリアに関する決断は、単独で下されるものではありません。それは、現在のライフステージや個人的な状況に深く影響されます。なぜソフトウェア開発の道に進んだのか、そしてどのようにして今の立場に至ったのか、その経緯は、人それぞれ異なります。卒業直後には理にかなっていたキャリアパスも、幼い子供がいる時期には状況が変わります。ましてや子供が独立して自由な時間

ができた時期になれば、さらに事情は異なるかもしれません。

キャリアの決断には、通常、5つの主な要因が影響します。

報酬（給与、福利厚生、株式報酬）

仕事に対して、十分な報酬は得られていますか？

チームと職場の文化

その組織やそこで働く人たちは気に入りましたか？

業務内容と技術的な課題

その仕事は好きですか？

成長の機会と将来の見通し

この仕事は、次の仕事につながるでしょうか？ また、機会損失についても考えてみてください。他の選択肢ではなくこの道を選んだことで、どんな経験やスキル、人脈を得る機会を逃してしまうおそれがあるでしょうか？

ワークライフバランス

この仕事では、私生活や心の健康を保つことができますか？

これらの要素をすべて考慮に入れるべきですが、人生の段階によって優先順位が変わる可能性があります。以下に例を示します。

キャリア初期

成長の機会や、やりがいのある仕事を優先し、最先端のスタートアップで経験を積むために、多少の給与の減額を受け入れることもあるでしょう。

家族を築く時期

健康保険、安定した勤務時間、そして一定の収入が重要になるため、大企業がより魅力的に映るようになるかもしれません。

中堅期

経験を積んだ後は、企業間を戦略的に転職することで、報酬をできるだけ引き上げることに注力することでしょう。

キャリア終盤

ワークライフバランスやチーム文化のほうが、キャリアアップよりも重要に感じられることが多いでしょう。また、経験豊富な開発者の多くは、長く残る功績を生み出すことにやりがいを見出します。他者を指導したり、意義あるプロジェクトに貢献したり、あるいは永続的な影響を残すものを生み出したりすることなどです。

キャリアの選択を人生の節目に合わせて考える際には、どのような企業で働きたいかという点を考慮するだけでなく、転勤や出張を受け入れるかどうかといった現実的な要素についても検討する必要があります。

14.1.5.1 企業の種類とワークライフバランス

次のキャリアの機会を探す際には、どのような企業があるのか、そしてそれらが自分の人生の節目とどのように合致するかを検討してみるとよいでしょう。

スタートアップ

斬新なプロダクトやサービスの開発に注力する、新しく設立された企業

- メリット：多岐にわたる課題に取り組み、さまざまな役割を担うことになるため、迅速かつ多様な学びの機会が得られる
- デメリット：長時間労働、失敗のリスクが高いこと、そしてリソースや福利厚生が限られていること

従来型の企業

テクノロジーを活用して本業を支えているものの、会社自体はテクノロジー企業ではない、さまざまな業界（医療、経理、小売、製造など）の企業

- メリット：安定性、規則正しい勤務時間、充実した福利厚生、良好なワークライフバランス
- デメリット：一般的に開発ペースが緩やかで、最先端の技術が採用されにくく、総報酬額が低め

ビッグテック

Alphabet（Google）、Amazon、Apple、Meta（Facebook）、Microsoft など、規模が大きく影響力のあるテクノロジー企業

- メリット：経済的な安定、充実した福利厚生、社会的信頼、最先端技術に携わる機会
- デメリット：採用プロセスが非常に厳格で、アルゴリズムの学習など、面接の準備に時間がかかる

コンサルティング会社

コンサルティング会社として、あるいは独立したフリーランスとして、他の組織に専門的な助言やサービスを提供する役割

- メリット：複数の企業や技術分野にわたる幅広い経験、さまざまな業界への関与、そして多くの場合、より高い報酬が得られる。独立コンサルタントとして、自身のプロジェクトを選定でき、より柔軟な働き方が可能
- デメリット：新しい環境や技術スタックに迅速に順応する必要があり、出張の可能性がある。また、独立系コンサルタントとして活動する場合、自らクライアントを開拓し、

営業活動や事業運営を自ら管理するという課題も生じる

14.1.5.2 現実的な面での考慮事項

就職先を検討する際、単に「どのような会社で働きたいか」という点だけが決め手になるとは限りません。他の条件がすべて同じであれば、現在のライフステージによっては、以下のような現実的な要素のほうが重要になる場合もあります。

- 健康保険は必要ですか、それともパートナーを通じて加入できますか？
- 勤務地が変わらないことは重要ですか？
- 給与ではなく株式報酬による収入という、経済的な不確実性に耐えられますか？
- 学びの多い環境に対応する時間とエネルギーはありますか？
- 出張は多めがよいですか、それとも少なめがよいですか？
- この転職は、自身のキャリア形成に役立ちますか？
- 在宅勤務とオフィス勤務、どちらを好まれますか？ 後者の場合、通勤時間は無理のない範囲ですか？
- リスクに対してどの程度許容できますか？
- この仕事では、読者が求めるワークライフバランスを実現できますか？
- そのビジネス分野やプロダクトに興味がありますか？ それを学ぶことにワクワクしますか？

「唯一の正解」というものはなく、そのときの状況に合った選択をすることが大切です。スタートアップ、ビッグテック、コンサルティング会社など、企業のタイプは人生の節目に合わせて選ぶことができます。また、健康保険の必要性、勤務地が変わらないこと、ワークライフバランスの好みといった現実的な要素も、意思決定の指針となるでしょう。

また、柔軟な計画を立てましょう。キャリアに関する決断をする際は、野心的な目標を設定しつつも、予期せぬ困難や私生活の変化に備えて、ある程度の余裕を持たせるようにしてください。キャリアパスが、計画通りに進むことはめったにありませんが、それでかまいません。むしろ、そうした状況こそが、価値のある成長や学びの機会となることが多いのです。あらかじめ決めた厳しいスケジュールに追われて燃え尽きてしまうよりも、長期的に着実な進歩を続けることに重点を置きましょう。

14.2 キャリアパスを歩む

キャリアパスの計画の立て方がわかったところで、これから歩む道のりにおいて役立つ、実践的なヒントについて見ていきましょう。キャリアを積み重ねていく中で、その道のりの一歩一歩を楽しむことを忘れないでください。私たちは往々にして、目的地にばかり目を向けてしまい、現在の成果を祝うことを忘れてしまいがちです。この節では、成果を記録する方法、インポスター症候群

への対処法、プロフェッショナルネットワーク（専門技術上の人脈）の築き方、そして面接を成功させるための戦略について学びます。

14.2.1 成果を祝い、記録に残す

ソフトウェア開発の世界は目まぐるしいスピードで動いています。ある問題を解決するために全力を注いだかと思えば、すぐに次の課題へと移っていきます。複雑なバグの修正であれ、新機能の実装であれ、あるいは同僚やコミュニティのメンバーから寄せられる肯定的なフィードバックであれ、どんなに些細なことのように思えても、すべての成果は重要です。大小を問わず、こうした成果が生まれたら、その場でぜひ記録に残すようにしましょう。

「成果記録」には、さまざまな役割があります。インポスター症候群（本章で後ほど説明します）に襲われた際に、強力な励みとなってくれます。困難な時期には自信を取り戻す助けとなります。また、業績評価や履歴書の更新、就職の面接においても貴重な資料となります。さらに、自身のキャリアの中で「成果」とみなしてきたものの中にどのような傾向があるのかを把握するのにも役立ちます。

成果を記録するために、自分の達成したことを継続的に記録できる場所を見つけてください。デバイス間で同期され、クラウドにバックアップされるデジタルツールが理想的です。もしも昔ながらの紙に書き留めるほうがお好きなら、ぜひそうしてください。専用のナレッジマネジメントアプリケーションである必要も、凝った仕組みである必要もありません。Apple のメモや Google ドキュメントなら、スマートフォンやノート PC から簡単にアクセスできます。記録に含めるべき項目をいくつか紹介します[†2]。

- 技術的な成果（実装に成功した機能、バグの修正）
- 自ら主導したプロセスの改善
- 同僚、ステークホルダー、コミュニティメンバーからの肯定的なフィードバック
- 新たに習得したスキル
- 自ら主導したプレゼンテーションや知識共有セッション

業務内容を記録する際は、網羅的に書くよりも具体的に書くことを心がけてください。そうすれば、後で詳細を思い出しやすくなります。「主力アプリケーションのパフォーマンスを向上させた」と書くのではなく、「仮想スレッドを有効化し、顧客体験を向上させることで、主力アプリケーションのパフォーマンスを 25% 向上させた」と記述しましょう。後者の表現のほうが、はるかに説得力があります。このような具体的な成果は、現在の会社内であれ他社であれ、新たな機会を求める際に、履歴書にそのまま活用できます。

†2　日付を記録しておくと、振り返りや履歴書の更新の際に非常に役立ちます。

14.2.2 インポスター症候群を克服する

「自分はここにふさわしいのだろうか？」「本当にこれができるのだろうか？」「この仕事をするだけの能力があるのだろうか？」といった疑問が、頭の中にしつこくよぎることはありませんか？これは**インポスター症候群**と呼ばれます。実のところ、こうした感情はキャリアを通じてずっとつきまとうかもしれません。しかし、良いニュースもあります。それは、こうした感情はまったく健全なものであり、むしろ成長の原動力になり得るということです。

14.2.2.1 インポスター症候群とは一体何なのか

キャリアを積んでいくうちに、いつの日か、当初感じていたインポスター症候群は、自分がその場にふさわしくないからではなく、良い仕事をしたいと真剣に考えている証だったのだと気づくことでしょう。インポスター症候群は、自分が知らないことがいかに広大であるかを理解している証拠でもあります。「何でも知っている」という人には注意してください。彼らは実際には何も知っておらず、プロジェクトにおいてしばしば危険な存在になりかねません。ソフトウェアは専門性の高い分野であり、すべてのことについて詳しく知ることは不可能です。謙虚さと自信の両方が求められます。知っていることもあれば、知らないこともある、それこそが、まったくもって自然なことです。

ソフトウェア開発者として、インポスター症候群によく見られる兆候をいくつか紹介しましょう。

- 技術的な議論で発言をためらってしまう
- 発言する前にすべてを理解しておかなければならないと感じてしまう
- 経験豊富な開発者と自分を比べてしまう
- 自分の成功をスキルではなく運のせいにしてしまう
- 自分が場違いだと他人に「見抜かれる」のではないかと心配してしまう

こうした気持ちはごく自然なものです。多くの人は、身につけなければならないと思う膨大な知識の量に、圧倒されてしまうものです。ソフトウェア開発という分野は広大であり[†3]、すべてを網羅することは不可能です。しかし、それでまったく問題ありません。

14.2.2.2 インポスター症候群を克服する方法

「自分はふさわしくない」と感じてしまうことはよくあることですが、その感情に足を引っ張られる必要はありません。むしろ、それは自分自身が挑戦し、成長を目指している証かもしれません。この節では、そうした感情を、前向きな変化をもたらす力強い原動力へと変えていく方法に焦点を当てます。

†3 すべてを学ぶことはできません。

「疑い」を原動力にする

インポスター症候群を単なる否定的な自己不信と捉えるのではなく、こうした感情を成長のための強力な原動力へと変えることができます。

失敗を悪いことだと捉えるのではなく、学ぶ機会だと考えましょう。確かに、道のりには紆余曲折があるでしょうが、それは問題ありません。失敗を恐れることをやめたとき、真の成長が始まります。ある言語やフレームワークに関して、技術的な自信のなさを感じていますか？ それを、その分野を極めるための原動力にしてください。前節で触れたような感情も、次のように乗り越えることができます。

技術的な議論で発言をためらってしまう

自分の経験に基づいて話してください。読者の個人的な経験を否定できる人は誰もいません。もし間違っていたとしても、そこから学びましょう。

貢献する前に、すべてを知っておく必要があると感じてしまう

「すべてを知ることはできない」という事実を受け入れましょう。このことを受け入れるのが早ければ早いほど、成長も早くなります。

経験豊富な開発者と自分を比べてしまう

自分より知識のある人は必ずいます。それを受け入れましょう。そのような人々と接することは、学ぶための絶好の機会なのです。

自分の成功を、実力ではなく運のせいにしてしまう

運とは、準備と機会が出会うことです。失敗から学び、絶えず向上しつづけることで、成功は偶然ではなく、実力から生まれるのです。

自分がそこにふさわしくないことを、他人に「見抜かれる」のではないかと心配してしまう

読者はここにふさわしい存在です。このことを早く受け入れれば、それだけ早くキャリアを満喫できるようになります。簡単ではないでしょうが、ひたむきな努力さえあれば、どんなことでも成し遂げられます。

行動を通じて自信を築く

同じ問題に何度も直面することで、自信がつくものです。初めて課題に直面したときは、「どうすれば解決できるのか」とパニックになり、途方に暮れてしまうことでしょう。

バグ修正や新機能の開発を終え、コードレビューに提出する際は、いただいたフィードバックを前向きに受け止めてください。レビュー担当者が必ずしも読者より優秀というわけではありません。ただ、彼らは同様の問題に頻繁に直面してきただけなのです。簡単な振り返りを行ってみましょう。何がうまくいったか、何がうまくいかなかったか、そして今後はどう対応すべきか、考えてみてください。

フィードバックを受け入れることは重要ですが、自分が強く確信している決断については、しっ

かりと主張できるようになることも必要です。これは、自分の考えに対する自信を示すためのものです。単に議論したいから議論するというものではありません。次にこのような課題に直面したとき、これまでの経験を活かし、どうすべきかを正確に判断できるようになるでしょう。やがて、自信と経験は一体のものとなります。自転車に乗るのと同じように、同じ問題に繰り返し直面することで、それが自然と身についていくのです。かつては怖かったこと（自転車に乗るときの、あの最初のふらふらとしたときのようなもの）も、やがて何の苦労もなくできるようになります。

成長の機会を見極める

課題管理ツールを閲覧する際は、あえて複雑なタスクを選んでみてください。直近の 10 件の修正と似たようなチケットを選びたくなるかもしれませんが、それでは成長にはつながりません。不慣れな分野の課題でも、積極的に取り組んでみてください。そうすることで、システム全体に対する知識が広がるだけでなく、多様な課題に取り組むことで問題解決能力も高まるでしょう。

他の開発者が敬遠しがちなアプリケーションの課題に取り組むことで、チームにとってより価値のある存在になれるでしょう。「決済システム」など自分が熟知している分野において、ドメインエキスパートとしての地位を確立できるはずです。

インポスター症候群はごく自然なことであり、その感情はキャリアを通じて続くこともあるということを覚えておいてください。それを障害と捉えるのではなく、日々の成長の原動力として活用しましょう。成長のもう一つの原動力となるのが、どのような人々と関わるかです。次の節では、プロフェッショナルなコミュニティを築く方法について学びます。

14.2.3 プロフェッショナルなコミュニティを築く

ネットワーキングは、コンピュータだけの話ではありません。キャリアアップに関して言えば、開発者は往々にして技術的なスキルだけに注目しがちです。もちろん、技術的スキルはキャリアにとって重要です。しかし、キャリアを通じて築く人的なつながりも、それと同じくらい価値のあるものです。プロフェッショナルなコミュニティを築くということは、ソーシャルネットワーク上のフォロワー数を増やすことではありません。キャリアを通じて築き上げる真摯な人間関係のことです。アドバイスや仕事の機会を求めて相談したり、プロジェクトで起きたことについてただ愚痴をこぼしたりできる相手がいることは、成功への鍵となります。

「ネットワーキング」という言葉を聞いて、混雑したカンファレンスの交流会や、気乗りしない世間話を思い浮かべるのは、決して読者だけではありません。しかし、効果的なネットワーキングとは、そのようなものではありません。ネットワーキングにおいて、社交的な人物になる必要も、あらゆる懇親会に参加する必要もありません。成功の秘訣は、自分の強みを活かし、自分らしい人との接し方でネットワーキングの方法を見つけることにあるのです。

この節では、プロフェッショナルなコミュニティを築くために重要となる点をいくつか学びます。地域やオンラインなど、コミュニティの種類について理解を深めます。これらのコミュニティへ関わり始めると、知識の共有から積極的な関与に至るまで、コミュニティに貢献するための効

果的な方法を学ぶことができます。また、誠実さ、継続性、相互価値に基づいた、永続的でプロフェッショナルな人間関係を築くための原則についても学びます。最後に、質より量を優先してしまうといった、よくある避けるべき落とし穴について検討し、しっかりとしたプロフェッショナルネットワークがキャリアを通じてもたらす長期的なメリットについても探っていきます。

私のキャリアにおけるプロフェッショナルネットワークの影響

Danです。私のキャリアにおけるあらゆるチャンスは、プロフェッショナルなコミュニティを築いてきたことによるものだと言っても、決して大げさではありません。人脈が必ずしも直接的にチャンスにつながったわけではありませんが、常に何らかの形で私のキャリアパスに影響を与えてきました。キャリアの初期には、ユーザーグループのミーティングやカンファレンスに参加し、その中で素晴らしい人々に出会い、今でも友人として交流を続けています。私は、人生における偶然の出会いを信じてはいません。新たな機会をもたらしてくれる人との出会いがあり、その機会を活かすかどうかは自分次第だと考えています。

プロフェッショナルなコミュニティは、職場の同僚だけではありません。このネットワークには、メンター、元同僚、カンファレンスの講演者、オープンソースの貢献者、採用担当者、そして仲間の開発者などが含まれます。こうしたつながりの一つひとつが、学び、協働、そしてキャリアアップの機会となります。これから地域コミュニティとオンラインコミュニティの両方について見ていきましょう。

14.2.3.1 地域のテックコミュニティ

地域のテックコミュニティとは、お住まいの地域で活動するグループのことです。対面での交流や有意義な人間関係の構築を促すものです。具体的には以下のようなものがあります。

- 特定の技術に特化したユーザーグループ
- 地域のハッカソンやコーディングイベント
- 技術交流会や懇親会
- 専門職団体の支部

地域コミュニティの最大のメリットは、定期的な対面での交流を通じて、より深いつながりを築けることです。ほとんどのユーザーグループのミーティングでは、本編のプレゼンテーションの前後にネットワーキングの機会が設けられており、そこで他のコミュニティメンバーと交流できます。また、自宅でマルチタスクをしながらバックグラウンドでオンライン会議を見る場合と比べ、プレゼンテーションの間も、より集中して参加できるでしょう。

もちろん、考慮すべきトレードオフもあります。会場の場所やスケジュールによっては、移動時間が問題になるかもしれません。車での移動が短距離であれ長距離であれ、移動時間、ネットワー

キング、そしてイベント本体の時間を合わせると、合計で 3〜4 時間ほどかかる可能性があります。

お住まいの地域や、どのようなテクノロジーに興味をお持ちかにかかわらず、さまざまなプログラミング言語やテクノロジー、関心分野を扱うミートアップやユーザーグループが数多く見つかるはずです。もし自分の興味に合うグループが見つからない場合は、自ら率先して立ち上げてみてはいかがでしょうか。

14.2.3.2 オンラインコミュニティ

オンラインコミュニティとは、デジタル空間に存在するグループのことです。地理的な場所に関係なく、テクノロジー分野の専門家同士のつながりを促進するものです。具体的には以下のようなものがあります。

- GitHub リポジトリとディスカッション
- Stack Overflow
- 技術に特化した Discord サーバー
- 専門的な Twitter ／ X コミュニティ
- LinkedIn グループ

オンラインコミュニティの最大のメリットは、さまざまな専門知識や視点を持つ専門家たちのグローバルなネットワークにアクセスできることです。ほとんどのオンラインプラットフォームでは非同期型のコミュニケーションが可能であり、都合の良い時間に参加できます。また、複数のコミュニティに同時に参加するのも容易ですので、技術や専門分野を横断して知識を広げることができます。

もちろん、考慮すべきトレードオフもあります。オンラインでの交流には、対面でのやり取りのような深みや個人的なつながりが欠けていることがよくあります。テキストベースのコミュニケーションだけでは、有意義な関係を築くのが難しい場合もあります。さらに、オンラインコミュニティは 24 時間 365 日稼働している性質上、情報過多に陥ったり、常に参加しつづけなければならないというプレッシャーを感じたりすることもあります。

どのようなテクノロジー分野に関心を持っていても、ほぼあらゆるプログラミング言語、プラットフォーム、専門分野をカバーしたオンラインコミュニティが数多く存在します。もし自分自身の興味に合ったコミュニティが見つからない場合でも、多くのプラットフォームでは、独自のスペースを簡単に作成でき、同じ志を持つ専門家を招待できます。

14.2.4 プロフェッショナルな人間関係を築く

有意義なつながりを通じてネットワークを築くことは、キャリアアップに不可欠です。他の人間関係と同様、こうしたつながりも時間をかけて育み、大切にしていかなければなりません。コミュニティを築くことは、そこから何を得られるかということではないことを忘れないでください。このような「見返りを求める」考え方は、有意義な関係にはつながりません。真摯な貢献を通じて相

互に価値を生み出すことに注力すれば、自然とプロフェッショナルとしての成長もついてくるでしょう。

14.2.4.1 プロフェッショナルな関係における基本原則

長続きするプロフェッショナルな関係は、技術的な専門知識と対人スキルの両方を反映した、以下の基本原則に基づいて築かれます。

誠実さ

どんな場面でも、ありのままの自分であり、誠実な姿勢を心がけましょう。会議やコードレビューの最中に技術的な概念が理解できない場合は、素直に認め、それを学びの機会として活かしてください。同僚たちは読者の正直さを評価し、より積極的に手助けしてくれるでしょう。

信頼性

信頼性は、あらゆる分野において重要な資質です。約束は確実に果たしましょう。木曜日までに機能のレビュー準備が整うと約束したなら、必ずそのとおりにしてください。締め切りに間に合わない場合は、早めにその旨を伝えましょう。ネットワークの方々と定期的にコーヒーやランチの約束をしている場合は、時間どおりに現れてください。直前にキャンセルするようなことは避けてください。こうした信頼性こそが、チームメンバーやプロフェッショナルなコミュニティとの信頼関係を築くのです。

相互価値

フロントエンドのスキルに長けた同僚に答えを求めるだけでなく、お互いにメリットのある関係を築きましょう。自分の知識を惜しみなく共有しつつ、他者から学ぶ姿勢も持ちましょう。

敬意

自分はすべての答えを知っているわけではない、という考えを柔軟に受け入れてください。さまざまな視点や経験を認め、尊重しましょう。技術的なアプローチについて議論する際は、たとえ自分の好む方法とは異なっていても、他者の解決策に積極的に耳を傾けてください。ソフトウェアエンジニアリングでは、多くの場合、問題を解決する有効な方法が複数あるということを忘れないでください。

コミュニケーション

コミュニケーションはあらゆる人間関係の基盤であり、それはプロフェッショナルネットワークにおいても同様です。ソフトウェア開発においては、明確で効果的なコミュニケーションが不可欠です。コードの変更点を説明する場合でも、アーキテクチャの決定について議論する場合でも、進捗状況を報告する場合でも、わかりやすさを心がけ、相手の技術的な背景を考慮するようにしてください。

14.2.4.2 ネットワークを維持し、交流を深める

人脈を築くことは重要ですが、長期的にキャリアを成功に導くためには、構築したネットワークとの有意義な関係を維持することも同じように重要です。そのためには、時間とエネルギーを注ぎつづける必要があります。

独自の視点を共有し、積極的に参加することで、コミュニティと意識的に関わっていきましょう。自分が解決した課題について記事を書いたり、GitHub や Stack Overflow での議論に参加したり、ミーティングでは建設的な質問を投げかけたりしましょう。解決策は必ず記録しておきましょう。後で同じ問題に直面したとき、その記録がきっと役に立つはずです。ただ傍観しているだけでは不十分です。できる限り手助けをし、関連するリソースを共有し、他者の仕事に対して建設的なフィードバックを提供しましょう。

定期的に連絡を取り、相手の近況を尋ねたり、現在取り組んでいることを確認したりしましょう。何か用事があるときだけ連絡すると、相手はそれに気づき、読者を避けるようになるかもしれません。

少し機械的すぎるように思えるかもしれませんが、定期的に連絡を取るよう自分にリマインダーを設定するようにしてください。定期的にコーヒーを飲んだり、ランチをしたりする予定を組みましょう。誰かが気に入りそうな記事やポッドキャストを見つけたら、ぜひ送ってみてください。ネットワークを育むことは、決して無駄にはならない投資です。

これらの基本原則を念頭に置きながら、コミュニティと積極的に関わり、長期にわたって人間関係を築いていくことで、キャリアのあらゆる段階を支えてくれる、プロフェッショナルネットワークを構築できるでしょう。

> **量より質を選ぶ**
>
> 言うは易く行うは難し、というところですが、特定のソーシャルメディアプラットフォームでのフォロワー数を気にするのは避けたほうがよいでしょう。その代わりに、既存のフォロワーと積極的に交流し、真のつながりを築くことに注力してください。
>
> LinkedIn のつながりが 5,000 人いても、その誰一人として読者に仕事を推薦してくれなければ、何の意味もありません。社会で重要視されがちな「見せかけの指標」ではなく、**有意義な**人間関係を築くことに注力しましょう。

14.2.5 次回の面接を成功させる

面接は、単に次の仕事を得るためだけのものではありません。それは、キャリアを通じて大きな成果をもたらす基本的なスキルなのです。社内の昇進を目指す場合でも、社外での新たな機会を探る場合でも、あるいは初めての就職を目指す場合でも、面接の技術を習得することは、ソフトウェアエンジニアリングにおけるキャリアアップにとって重要です。

これまで築いてきたプロフェッショナルネットワークも重要な役割を果たします。人脈を通じて、企業に関する有益な情報を得たり、紹介を受けたり、さらには推薦者になってもらったりすることもできます。

たとえ現在の職務に満足しているとしても、面接のスキルを磨いておくことで、予期せぬチャンスや社内昇進、あるいはこの業界では珍しくない「リストラ」といった事態に備えることができます。

次の節では、面接の準備方法、面接当日の対応、そしてプロフェッショナルなフォローアップの方法について学びます。これらのスキルは、ソフトウェアエンジニアリングの他のスキルと同様、練習と準備を重ねることで上達します。

14.2.5.1 面接の準備

この節では、他の候補者との差別化を図り、自信を持って面接に臨むための準備について解説します。リサーチツールやプロフェッショナルネットワークの活用から、よくある質問への対応練習、ソフトスキルの磨き方まで、技術的な専門知識とプロフェッショナルとしての価値を効果的にアピールする方法を学びます。準備とは単に技術的な知識のことだけではありません。組織の成功に大きく貢献できる、バランスの取れたプロフェッショナルとしての自分をアピールするための準備です。

戦略的調査

どのポジションに応募する場合でも、応募する前に、その企業と具体的な職務内容について徹底的に調査し、それらが自身のキャリア目標や価値観と合致しているかを確認してください。この事前の調査は、どのポジションが自分に適しているかを判断するうえで役立ちます。面接の機会を得た後は、調査をさらに深め、企業の文化、技術スタック、ビジネス上の課題、そして面接プロセスについて理解を深めてください。ほんの少しの時間をかけて、こうした調査を行うだけで、その日だけで100通目の履歴書を送っているかもしれない他の候補者たちとは一線を画すことができます。

調査に活用できるリソースは数多くあります。それらを活用して、求める結果を得ることは読者自身の役割です。まだLinkedInを利用していない場合は、ぜひ登録することをおすすめします。LinkedInは、プロフェッショナルネットワークを構築するためのツールとなるからです。また、面接官や企業の背景、職場文化に関する情報も得られます[†4]。さらに、読者のネットワークからは、面接プロセスや社内の雰囲気に関する貴重な内部情報、さらには面接官に関する有益な情報さえも得られるでしょう。

面接の準備をする際、LLM（大規模言語モデル）やリサーチアシスタントといったAIツールを利用すれば、求人情報だけでは得られない、企業に関する詳細な情報を得ることができます。これらのツールは、企業の最新ニュースを分析し、技術スタックに関する詳細情報を引き出します。また、市場での位置づけや競合環境を明らかにするだけでなく、採用パターンの傾向や企業文化を特

†4 さらに、他の候補者との差別化に役立つつながりを発見できるかもしれません。

定することさえできます。こうした詳細な調査機能を活用することで、候補となる企業をより深く理解し、面接に向けてより効果的に準備を進めることができます。以下は、お気に入りの AI ツールを使って、今後の面接に向けた調査を行う際に活用できるプロンプトの例です。

> 近日中に[会社名]の[職種]のポジションについて面接を受けることになりました。以下の点について、詳細な調査結果を提供いただき、面接の準備に協力いただけますか。
>
> 1．会社の概要：沿革、ミッション、価値観、および現在の経営陣について。
>
> 2．最近の動向：過去 6〜12 か月間の主なニュース、新製品のリリース、買収、または戦略の転換など。
>
> 3．技術情報：主に使用している技術スタック、注目すべきオープンソースへの貢献、エンジニアリングブログの注目記事、および直面している可能性のある技術的な課題。
>
> 4．市場における位置づけ：主な競合他社、市場シェア、独自の強み、および最近の業績指標。
>
> 5．企業文化と職場環境：社員の口コミ、ワークライフバランス、リモート／ハイブリッド勤務に関する方針、およびキャリアアップの機会。
>
> 6．採用面接のプロセスに関する情報：社員の体験談に基づいた、よくある面接質問、技術力評価、および重視されるスキルについて。
>
> 7．面接の際に、私の調査内容や真摯な関心を伝えるために尋ねられる可能性のある質問。
>
> 上記の情報を、確認しやすいように整理し、面接の際に特に言及すべき重要な点を強調してください。



ChatGPT、Google Gemini、Anthropic の Claude などの AI アシスタントは、複数のオンライン情報源から情報を統合することで、「ディープリサーチ（深堀り調査）」を行えます。これらのツールは、単に検索結果を返すだけではありません。ニュースサイト、テックブログ、財務報告書、従業員のレビューなど、多くの情報源から企業データを分析します。この分析に基づき、テクノロジースタック、ビジネス戦略、企業文化に関する包括的な情報が提供されます。具体的な質問を投げかけ、フォローアップの質問を用いて関連分野を掘り下げることで、面接の準備に役立ててください。

AI ツールは調査に役立ちますが、使いすぎたり、結果を確認せずに使用したりしないよう注意が必要です。人事部門や採用担当者は、AI が生成したカバーレターや履歴書を容易に見抜くことができます。スペルや文法、文章全体の改善に助けが必要な場合は、AI ツールを慎重に活用してください。ただし、カバーレター全体を無条件に受け入れることは避けてください。これは読者の第一印象となるものですから、誠実な印象を与え、読者らしさを際立たせる独自の資質をアピールすることが大切です。

もう一つの効果的な調査方法は、その企業の面接スタイルを把握することです。技術的な質問とソフトスキルの質問が混在している場合や、FAANG 企業のようなアルゴリズム課題や、あるいは

持ち帰り型のコーディング課題が出される場合もあります[†5]。どのような形式の面接になるかを理解しておくことで、より万全の準備ができ、次の面接を成功させるための自信を持てるでしょう。

最後に、実際に現地へ赴き、足を使って調査を行う必要があるかもしれません。具体的には、現在そこで働いている人や、過去にそこで働いていた人たちに話を聞くといったことが考えられます。

事前準備をしておくことで、ストレスの多い面接プロセスを乗り切るための安心感と自信を得ることができます。どのような展開になるかを把握しておけば、不意を突かれることもありません。面接で事前調査の結果を示すことは、採用担当者に、読者がその機会に対して真剣に取り組んでいる姿勢を伝えることにつながります。これは、内定の可否を分ける要因となり得るだけでなく、より良い待遇条件を引き出すことにもつながる可能性があります。

お互いの相性を評価する

事前調査は、面接の質問に答えるためだけのものではありません。面接官に尋ねる質問を決める際にも役立ちます。面接は、面接官が読者を見極めるだけでなく、読者も相手を見極めている場であることを忘れないでください。事前調査を活かし、真摯な関心を示す質問を準備すると同時に、その機会が自分のキャリア目標と合致しているかを見極める手助けにしましょう。例えば、次のようなことを尋ねてみてください。

- チームの雰囲気と文化
- 技術的な課題と意思決定プロセス
- 技術スタックと開発手法
- キャリアアップとメンターシップの機会
- 出張の必要性と出張の割合

よくある質問への対応

面接官から投げかけられるすべての質問に完璧に答えられるよう準備することはできません。しかし、多くの面接でよく聞かれる一般的な質問に対しては、ある程度準備をしておくことができます。まず準備しておくべき質問は「なぜ現在の職場を辞めるのですか？」というものです。この場合、正直であると同時に、プロフェッショナルな態度を示すことが重要です。以下のような回答が適切です。

- より良い待遇を求めている
- 新たな技術的な課題に取り組みたい
- 成長の機会を求めている

†5　FAANG とは、Facebook（現 Meta）、Amazon、Apple、Netflix、Google（現 Alphabet）を指します。

現在の勤務先について否定的な発言は避けてください。それは読者のプロ意識に疑問を投げかけることになり、面接官もその点を問題視するでしょう。プロフェッショナルネットワークは密接につながっていることを忘れないでください。たとえ詳細を匿名化したとしても、言及した人物が他の人に特定されてしまうおそれがあります。

もう一つ、準備しておくべきよくある質問は、「N 年後、自分はどのような立場にいたいと考えていますか？」というものです[†6]。この質問は、採用担当者が読者の野心や熱意、そしてキャリア目標が会社の方向性と合致しているかどうかを評価するためのものです。採用担当者は、読者が技術的なスキルとリーダーシップの両面において、自己研鑽に熱心に取り組んでいるかどうかを知りたいと考えています。

回答する際は、自身の専門能力の向上や、組織への貢献に向けた計画に焦点を当ててください。テックリーダーやアーキテクトとしての役割、あるいは他者への指導など、将来への展望について触れるのもよいでしょう。ただし、目標は現実的なものであり、一般的なキャリアパスに沿ったものであるようにしてください。これは、本章で説明したキャリアのロードマップに言及する絶好の機会となります。

ソフトウェアエンジニアが想定すべき技術的な質問

一般的な面接の質問に加え、ソフトウェアエンジニア職の面接では、採用担当者が読者の問題解決へのアプローチや経験を理解するために、技術的な質問が投げかけられます。以下に、事前に準備しておくとよい代表的な質問をいくつか紹介します。

「本番環境でのパフォーマンスの問題をデバッグするとしたら、どのように取り組むか、手順を詳しく教えてください」

この課題では、体系的な思考力とデバッグ手法に対する理解力が試されます。問題の特定から解決策の実施に至るまでの手順を一つひとつ説明できるよう練習してください。

「これまでに解決した困難な技術的問題と、その際の思考プロセスについて話してください」

最終的な答えだけでなく、その過程にも意識を向けましょう。

「新しいテクノロジーの最新情報をどのように把握し、何を学ぶかを決めていますか？」

自身のスキルアップに積極的に取り組んでいることを示してください。具体的にどのようなリソースを活用しているか、また新しいツールやフレームワークをどのように評価しているかについて触れてください。

「コードレビューやフィードバックのやり取りについては、どのように取り組んでいますか？」

チームワークのスキルとコードの品質へのこだわりを示すことができます。効果的なコードレビューにおける技術的側面と対人関係の側面の両方について議論しましょう。

†6 「N 年後の私はどのような立場になっているとお考えですか？」という質問も、面接官に投げかけるのに最適な質問です。そうすることで、面接官は読者が採用され、この組織でキャリアを積んでいる姿を想像することになるでしょう。

「URL短縮サービスやチャットアプリケーションのようなシンプルなシステムを、どのように設計するか説明してください」

この課題では、システムアーキテクチャ、データモデリング、スケーラビリティに関して体系的に検討する能力が問われます。基本的な要件から始め、設計上の判断や想定されるトレードオフについて、その過程を順を追って説明してください。



これまで積み重ねてきた成果を覚えていますか？ それらは、これから読者の強力な武器となるでしょう。解決した技術的な課題、修正したバグ、予定より早く完了させたプロジェクト、これらは、ぜひともアピールすべき成果です。ぜひ、それらの成果を振り返り、会話の中にうまく織り交ぜる方法を見つけてください。

練習は完璧を生む

次の面接を成功させるためには、事前準備が極めて重要です。その中でも、模擬面接は非常に効果的な手段と言えます。これは単に受け答えを練習するだけのものではありません。こうした練習を通じて築かれる土台こそが、自信と落ち着き、そして明確な思考を持って本番の面接に臨むための支えとなるのです。定期的に模擬面接を行うことで、本来ならストレスを感じがちな面接も慣れ親しんだものとなり、肝心な場面で、読者の真のプロフェッショナルとしての姿が存分に発揮できるようになります。

練習を最大限に活用するためには、有意義なフィードバックをくれるパートナーを慎重に選びましょう。ただ「はい」と相槌を打つ人や「素晴らしかった」と言う人ではなく、実際に面接官の立場を経験したことのある人から、具体的なフィードバックをもらうことが大切です。そのような友人や同僚が見つからない場合は、プロフェッショナルネットワークの中で、協力してくれる人を探してみてください。また、interviewing.io（http://interviewing.io/）のような技術面接に特化したオンラインプラットフォームもあります。経験豊富な専門家による体系的な練習の機会を提供してくれます。先ほど紹介した地域の技術コミュニティやオンラインの技術コミュニティも、適切な練習パートナーを見つけるための良い場となるでしょう。

AI模擬面接

次の面接準備のために適切な練習相手を見つけるのは、必ずしも簡単ではありません。そこで、AIツールが大いに役立ちます。AIチャットボットは、特定の職種に合わせたリアルな面接質問を生成したり、コーディング課題を作成したり、技術的な質問に明確に答える練習をサポートしたりできます。現在、多くのAIチャットボットには音声モードが搭載されており、答えを声に出して練習できます。チャットボットは、実際の面接と同じように、追加の質問を投げかけてくれます。こうした口頭での練習は、未知の質問に答える自信をつけ、即座に思考を整理して答えることに慣れるうえで非常に価値があります。AIが人間のフィードバックを

完全に代替できるわけではありませんが、24 時間 365 日利用可能であり、人間による模擬面接を行う前の面接準備を始めたばかりの頃には、大きな助けとなるでしょう。

練習セッションを最大限に活用するためには、実際の面接と同じくらいの真剣さを持って臨む必要があります。

- セッションを録画して自己評価を行い、第三者の視点から自分の長所と短所の両方を客観的に確認しましょう
- 言葉の内容だけでなく、非言語的なコミュニケーションのサインにも細心の注意を払いましょう
- 技術的な説明については、わかりやすさと深みに重点を置き、具体的なフィードバックを求めましょう
- 自身の経験に基づく具体的な事例を用いて、行動面に関するよくある質問への回答練習を行いましょう
- プロフェッショナルなボディランゲージと適切なアイコンタクトを保つよう、常に心がけましょう

面接対策に定期的かつ体系的な練習を取り入れることで、面接というプロセスを、ただただ恐ろしいハードルから、自身の資質や可能性を存分にアピールできる、しっかりと準備された機会へと変えることができます。さて、今後の面接に向けて準備が整ったところで、実際の面接で活用できる実用的なヒントについて説明しましょう。

14.2.5.2 面接中

この節では、面接官と直接対面する重要な場面に備えるための準備を行います。対面でもオンラインでも、技術的な会話で主導権を握り、単なるプログラミングの知識にとどまらない問題解決能力を示し、そしてプロフェッショナルな姿勢で自分をアピールするための実践的な戦略を学びます。

技術面接を成功させる

この面接に向けてしっかりと準備してきたことを忘れず、自信を持って必ず成功させようという気持ちで臨んでください。コーディング課題が出された際、すぐにキーボードを打ち始めたり、ホワイトボードに書き始めたりしたくなる衝動を抑えることが重要です。その代わりに、じっくりと考えを巡らせ、その思考プロセスを言葉で説明するようにしてください。これは、面接官が求めている重要なスキルであり、コードの書き方そのものよりも重視されることが多いのです。コードの書き方については、すでに習得しているものと想定されています。まず要件や制約条件を明確にし、解決策を実装する前に、アプローチの概要を説明してください。

初めて取り組む課題を解決する際は、明確なアプローチを通じて自身の価値を示しましょう。具体的には、課題を小さな要素に分解し、さまざまな解決策を検討したうえで、最終的に選んだアプローチを説明してください。過去のプロジェクトについて話す際は、実績として記録された技術的な成果を例に挙げ、自身の専門知識をアピールするとともに、それが今回の課題にどのように関連するかを説明しましょう。

プロフェッショナルな態度で臨む

技術的な能力に加え、プロフェッショナルな態度も面接の結果に大きな影響を与えることがあります。以下に、対面またはオンラインの面接で効果的に自分をアピールするための実践的なヒントを紹介します。

面接の準備として、以下の一般的な手順を参考にしてください。

- 面接官の名前や役職を調べておきましょう。こうすることで、細部への配慮が伝わり、相手一人ひとりに合わせた対応がしやすくなります。
- 求人票には記載されていない職務内容について、よく考え抜いた質問を用意しておきましょう。
- オンライン面接であっても、履歴書を数部持参しましょう（面接中に参照できます）。
- 身だしなみを整えましょう。これは対面でもオンラインでも同様です。
- 前日は十分な睡眠を取っておきましょう。
- 端末の通知音をオフにしておきましょう。

対面での面接の際は、予期せぬトラブルに備えて、早めに到着し、心の準備をする時間を確保するようにしましょう。これは当たり前のことのように思えるかもしれません。しかし、遅刻したり、慌てて会場に向かったりすると、精神状態やパフォーマンスに悪影響を及ぼすおそれがあります。

オンライン面接においても、一般的な面接のヒントはすべて当てはまりますが、技術的な点についても注意が必要です。以下は、環境を整えるためのヒントです。

- 機器の動作確認を行ってください。カメラ、マイク、照明がすべて正常に動作するか確認してください。また、オンライン会議アプリもテストし、面接の予定時間にログインできることを確認してください。
- アプリケーションの通知をすべてオフにしてください。
- ライブコーディングの演習中にシステムの動作を遅くするおそれのある不要なアプリケーションは閉じてください。
- カメラに写る背景が清潔で整頓されていることを確認してください。
- 家族に、面接中は邪魔されない時間が必要であることを伝えておいてください。

この体験を存分に楽しむ

面接とは、双方にとって最適な相手を見つけるためのものです。最初の面接、あるいは最初の3回の面接で採用が決まらない可能性もありますが、それは問題ありません。それぞれの面接を、経験や人脈を築く機会として捉えてください。もし良い印象を残すことができれば、その経験から将来的なチャンスが生まれるかもしれません。その機会や会社に対して、心からの熱意を示しましょう。

14.2.5.3 面接後

面接は終了し、読者はベストを尽くしました。そして、ここから待ち時間が始まります。幸いなことに、通常、結果は2つしかあり得ないということです。つまり、内定が得られるか、得られないかのどちらかです。もし内定が得られたなら、おめでとうございます！ これからは、その機会を評価し、報酬条件について交渉し、初出社の準備をする時期です。しかし、すべての面接が内定につながるわけではありません。ソフトウェアエンジニアとしての道のりにおいて、それはごく当たり前のことです。

以下の節では、採用に至らなかった場合の対処法に焦点を当てています。これは、そうしたケースがより一般的だからという理由ではなく、貴重な成長の機会にもかかわらず、見過ごされがちだからです。

この経験から学ぶ

デートと同じように、すべての面接がうまくいくわけではないという現実を受け入れ、不採用になることもあると覚悟しておく必要があります。たいていの場合、採用を見送られるのは読者の能力とはほとんど関係がありません。企業が社内の人材を採用することに決めた、優先順位が変わった、採用凍結になった、あるいは単に求めるスキルに対してより合致する候補者を見つけた、といった理由によるものです。個人的な資質に問題があると受け止めてはいけません。

フィードバックを糧に成長する

もし採用されなかった場合、落ち込んで「なぜ面接官は自分を採用しなかったのか」と一人で悩むこともできますが、これを成長の新たな機会と捉えることもできます。採用判断についてフィードバックを求める絶好の機会です。この情報は、読者のキャリア形成にとって非常に貴重なものとなるでしょう。もし改善すべき点が指摘された場合は、それを学びの指針やモチベーションとして活かし、次回の面接でより良い結果を出せるよう準備しましょう。なぜなら、必ず次があるからです！

繰り返しになりますが、何が起こるかは誰にもわかりません。もっと自分に合った、あるいはより魅力的なポジションが空くかもしれません。あるいは、別の会社で採用活動を行っている知人を紹介してくれるかもしれません。さらに、その人が新しい組織に移り、採用担当者に読者を推薦してくれるかもしれません。要するに、大切なのは「関係を断ち切らない」ということです。

14.2.6　ワークライフバランスを実現する

ソフトウェアエンジニアという仕事は、放っておけば人生のすべてを飲み込んでしまうおそれがあります。最新のテクノロジーを習得し、ますます複雑化する課題に取り組み、時には非現実的な納期に間に合わさなければならないというプレッシャーの中で、仕事と私生活の境界線は簡単に曖昧になってしまいます。

14.2.6.1　ソフトウェア開発における課題を理解する

どの職業にも固有の課題がありますが、ソフトウェア開発も例外ではありません。まずこれらの課題を特定できれば、それらに注意を払い、対処する準備を整えることができます。以下に、よくある課題とその対処方法を紹介します。

急速なテクノロジーの変化に対応する

直面する最大の課題の一つは、テクノロジーの変化の速さです。ツールやフレームワーク、プログラミング言語の最新動向についていくためには、絶えず学び、適応しつづける必要があります。

対処方法：

継続的な学習

- 12 章でこのことについて多くのことを学びましたが、何を学ぶかについて計画的に考え、書籍や講座、ワークショップ、カンファレンスなどを通じて学びつづけ、常に最新の情報を取り入れるようにしてください。

テックレーダー

- 本章で学んだように、テックレーダーは、次に何を学ぶべきかを決める際や、その選択肢を自分のキャリア目標と照らし合わせる際に役立つツールです。

ネットワーキング

- 同僚や友人と協力し合い、知識を共有し、業界のトレンドを常に把握するようにします。

頭を使うような問題は、「切り替える」のが難しい

頭を使うような難しい問題に取り組んでいると、その問題から離れられなくなり、あっという間に私生活にまで影響が出てしまうことがあります。

対処方法：

距離を置く

- キーボードから離れて、問題について無意識に任せておくことで、最高の解決策が浮かんでくることがあります。散歩やジョギングに出かけ、思考を自由に巡らせてみてください。

ラバーダッキング

- 時には、問題のすべてを誰か（あるいはゴム製のアヒルや AI チャットボットのような無生物）に話してみることで、自分の論理の穴に気づいたり、問題をより明確に理解したりできることがあります。

分散型チームとフレックスタイム制の運用

アジャイル環境において分散型チームと協働することは、チームが異なるタイムゾーンや文化、コミュニケーションスタイルで活動しているため、困難を伴うことがあります。

対処方法：

効果的なコミュニケーション

- チームが目標に向かって一致団結できるよう、明確なコミュニケーション手段と定期的なミーティングを設けてください。

標準化されたプロセス

- 一貫性を保つため、すべてのチームで統一されたツールと手法を導入してください。

心身の健康を維持する

ソフトウェア開発者にかかる負担は、適切に管理されなければ、ストレスや燃え尽き症候群につながるおそれがあります。

対処方法：

ワークライフバランス

- 燃え尽き症候群を防ぐために、仕事と私生活の境界線をはっきりさせてください。

セルフケア

- 友人や家族と過ごす時間、定期的な運動、健康的な食事、マインドフルネスの実践など、心身の健康を促進する活動を優先しましょう。自分に優しく接し、必要なときには休息を取ってください。

ソフトウェア開発者として直面する課題を認識しておくことで、それらを乗り越えるための準備がより万全になります。

14.2.6.2 リモートワーク

近年、ソフトウェア開発は大きな変革を遂げ、リモートワークが現実的かつ一般的な選択肢となっています。ソフトウェア開発者はかねてよりリモートワークが成功し得ることを実証してきましたが、現在では他の多くの職種でもこの働き方が取り入れられるようになりました。開発者にとって、リモートでのソフトウェア開発のメリットと課題を理解することは、キャリアアップのために極めて重要です。

もしソフトウェア開発が単にコードを書くことだけなら、リモートワークにデメリットなどない

でしょう。しかし、本書で紹介したように、この仕事にはコーディング以上の多くの要素が含まれています。ソフトウェア開発の中心となるのは、コラボレーション、効果的なコミュニケーション、メンタリング、そしてチームの力学であり、これらはすべてリモート環境において特有の課題をもたらします。

リモートワークのメリット

ソフトウェアエンジニアとしてリモートワークを行うことには、いくつかメリットがあり、ワークライフバランスを大幅に改善してくれます。

コストと時間の削減

ラッシュアワーの渋滞や長い通勤時間とはお別れです。毎週数時間の時間を節約でき、荒っぽいドライバーへのストレスも解消されます。

生産性の向上

リモートワークであれば、誰にも邪魔されない時間を確保でき、複雑な問題解決やコーディングに集中できます。Slackで送れば済むような質問をしようと、同僚がデスクにふらりと立ち寄って、集中している最中に作業の流れを遮られることもなくなります。

柔軟性

従来の「9時から5時まで」という勤務形態が苦手ですか？ 多くのリモートチームでは非同期型の働き方を採用しており、最も生産性の高い時間帯に仕事に取り組むことができます。気分転換をしたいですか？ お気に入りのカフェへ行って、次のタスクに取り掛かりましょう。

健康上のメリット

リモートワークでは、病原菌が蔓延しやすい混雑したオフィスを避けることができるため、体調を崩すことが少なくなるかもしれません。同僚と外食ランチをとる代わりに、自宅でより健康的な食事を用意することもできます。さらに、朝や昼休みに簡単に運動を取り入れることができるので、活力を得たり、気分転換をしたりすることもできます。

グローバルな活躍の機会

通勤の制約がなければ、住む場所や働く場所の選択肢は無限に広がります。より幅広い機会への道が開かれ、多くの場合、より充実した報酬が得られることになります。

専用開発環境

作業環境にはこだわりがありますか？ お気に入りの椅子やデスク、モニターを所有していますか？ 多くの企業では作業環境を整えるための費用を支援してくれるので、標準的なオフィスの個室とは比べ物にならない、快適で理想的なリモートワーク環境を構築できます。

リモートでのソフトウェア開発における課題

リモートワークに課題がないわけではありません。しかし、その課題を把握しておけば、それらに対処するための準備をより万全に整えられます。

孤立と孤独感

誰もが孤独な環境でうまくやっていけるわけではなく、多くの人は人との交流を求めています。オフィスでの人間関係がないと、在宅勤務は孤独に感じられることがあります。もし孤独を感じているなら、1日の始まりにジムのクラスに参加したり、週に数回カフェで仕事をしたりして、自宅という単調な環境から抜け出すことを検討してみてください。

自宅での気が散るもの

自宅のオフィス環境が何にも邪魔されないように整っている人もいれば、家族の喧騒や、居心地の良いベッドの誘惑、あるいは娯楽などの邪魔に悩まされている人もいます。テレビを見たり家事をしたりしたくなる気持ちはわかりますが、仕事に集中しつづけるには自制心が必要です。

コミュニケーション

リモートでのソフトウェア開発はデジタルコミュニケーションツールに依存していますが、それによって誤解が生じることがあります。チームメンバーが異なるタイムゾーンに分散しているため、すぐに回答を得られるとは限りません。オフィスで同僚の肩を叩いてすぐにフィードバックが得られるのとは異なり、リモートでのコミュニケーションにはより多くの忍耐が必要です。

技術的な指導

この課題は、キャリアの初期段階において特に大きな負担となります。オフィス勤務であれば、メンターに気軽に質問を投げかけることができますが、リモートワークでは、その問題についてビデオ通話を設定するべきかどうかを判断しなければなりません。

仕事と私生活の境界

リモートワークはワークライフバランスを改善する一方で、仕事と私生活という重要な境界線を曖昧にしてしまうこともあります。自宅がオフィスになると、プライベートな時間に仕事の用事をさっと片付けたくなってしまうものです。仕事と私生活の間に明確な境界線を引くことが、極めて重要になります。

テクノロジーへの依存

リモートワークには安定したインターネット環境と技術的なインフラストラクチャが不可欠であり、接続障害が発生すると生産性が低下してしまいます。技術的な問題が発生した際にも安心して作業できるよう、自宅の近くに少なくとも2か所の代替作業場所を確保しておくのが賢明です。

キャリアの展望と成長

リモートワーカーは、評価や昇進に関して見過ごされると感じたり、「目に見えなければ気にもかけられない」と思ったりすることがあります。

ソフトウェアエンジニアとしてのキャリアには、特有の課題があります。充実した仕事をするためには、明確な境界線が必要です。仕事では成果を最大化し、同時に私生活を守るための戦略に注力しましょう。

14.3　まとめ

本章で何か一つだけ覚えておくとしたら、それはソフトウェアエンジニアとしてのキャリアは「目的地」ではなく「旅」であるということです。キャリアのどの段階にいても、ソフトウェア開発者という仕事は時に気が重く感じられるかもしれません。ぜひ時間をかけて、現在の自分の立場や歩んできた道のりを振り返ってみてください。

キャリアパスを明確に定め、計画的にスキルを磨き、有意義でプロフェッショナルな人間関係を築き、健全なワークライフバランスを維持してください。そうすれば、絶えず変化しつづけるソフトウェア開発の世界をうまく切り拓いていけるでしょう。重要なのは、キャリアの目標を見失わずに、柔軟に対応しつづけることです。

ソフトウェア開発者のキャリアには、成長と成功への道が数多くあります。技術的な専門知識を深めることも、管理職への道を進むことも、あるいは起業家として独自の道を切り拓くことも可能です。今日から意識して選択を始めれば、どのキャリアパスを選んでも実現可能です。

困難や失敗を成長の機会として受け止め、インポスター症候群は、自分が学び、成長しようと努力している証であることを忘れないでください。読者がこれから歩むソフトウェアエンジニアとしての道のりは、読者だけのものです。ぜひ、その道のりを存分に楽しんでください。

14.4　実践に移す

学んだことを実践するには、知識だけでなく行動が必要です。以下の実践的なステップを参考に、これらのキャリア開発の考え方をすぐに適用し、学んだことを具体的なキャリアアップへとつなげましょう。

1. 自分だけのテクノロジーレーダーを作成し、各段階（「採用」、「試用」、「評価」、「非推奨」）に該当するテクノロジーを特定しましょう。
2. 自分のキャリアのロールモデルにしたい人物に連絡を取りましょう。コーヒーをご馳走して、その方のキャリアの歩みについて話を伺えないか尋ねてみてください。無料のコーヒーを嫌がる人はいないでしょう？
3. 1年後、3年後、5年後のキャリア目標を書き出しましょう。

4. 地元で、少なくとも 1 つのミートアップグループに参加しましょう。定期的に対面での集まりがあるグループを見つけられれば理想的でしょう。
5. 自分の成果や成長を記録するために、「成果記録」を作成しましょう。
6. たとえノート PC 上の単なるメモ書きであっても、ブログを始めましょう。「今日は、とても興味深い問題を解決しました（理解するために一度、そしてそれを明確に説明するためにもう一度、計 2 回学ぶ必要がありました）」といった内容でよいのです。
7. AI ツールを活用して模擬面接の練習を行いましょう。可能であれば音声モードを使用して、実際の面接環境を再現してください。AI に、自分が志望している特定の職種向けの面接官役を演じてもらい、技術的な質問や行動面に関する質問に対して、声に出して答える練習をしましょう。練習が終わったら、自分の回答や改善すべき点についてフィードバックを求めましょう。
8. 同僚やメンターを見つけて、1 時間の模擬面接の時間を設定しましょう。（許可を得たうえで）その様子を録画し、自分のパフォーマンスを振り返りましょう。
9. 希望する企業を選び、その面接プロセスを調べてみましょう。その企業の技術スタック、最近の動向、そして面接で尋ねたい具体的な質問を 3〜5 つまとめた準備資料を作成してください。

14.5 関連リソース

- 『Developer Career Masterplan』Heather VanCura・Bruno Souza 共著（Packt Publishing、2023 年）
- 『The Manager's Path』Camille Fournier 著（O'Reilly、2017 年）
  和書『エンジニアのためのマネジメントキャリアパス』（オライリー・ジャパン、2018 年）
- 『Developer, Advocate!』Geertjan Wielenga 著（Packt Publishing、2019 年）
- 『Help Your Boss Help You』Ken Kousen 著（Pragmatic Bookshelf、2021 年）
- 『The Passionate Programmer』Chad Fowler 著（Pragmatic Bookshelf、2009 年）
  和書『情熱プログラマー』（オーム社、2010 年）
- 『The Pragmatic Programmer』David Thomas, Andrew Hunt 共著（Addison-Wesley Professional、1999 年）
  和書『達人プログラマー（第 2 版）』（オーム社、2020 年）[†7]
- 『Never Eat Alone』Keith Ferrazzi, Tahl Raz 共著（Crown Currency、2014 年）
  和書『一生モノの人脈力』（パンローリング、2012 年）

†7 訳注：和書は、1999 年版ではなく、2019 年発行の 20 周年記念版の邦訳です。現在ではこちらのほうが手に入りやすいはずです。

15章
AIを駆使するソフトウェアエンジニア

生き残るのは、その種の中で最も強いものでも、最も賢いものでもなく、
変化に最もよく適応できるものです。
— Charles Darwin

1801 年、Joseph Marie Jacquard は、パンチカードによって制御され、複雑な模様を自動的に織り上げることができる織機を発明しました。かつては長年の修練を要した作業を、この新しい機械が再現する様子を、プロの織工たちは不安げに見守っていました。多くの人が、彼らの知る伝統的な織物技術の終焉を覚悟しました。

しかし、予想外のことが起こりました。ジャカード織機は、織工を排除するどころか、彼らを変革させたのです。熟練した職人は、織物の模様のデザイナーや機械操作員、繊維技術者へと転身しました。最も成功したのは、伝統的な技術と新しい技術の両方を理解していた織工たちでした。生産量は急増し、それまで存在しなかったまったく新しい役割が生まれました。

200 年が経った今、私たちは似たような状況に直面しています。AI がコードを書き、バグを修正し、システム全体を設計するようになっています。開発者たちは、当時の織工たちが抱いていたのと同じように、「自分はすぐに職を失うのだろうか？」という不安を抱えています。しかし、重要なのはここです。歴史が教えてくれるように、たいていの場合、そうはならないのです。このパターンは、あらゆる大きな技術的革新のたびに繰り返されてきました。そのたびに、変化に適応した人々が最終的に成功を収めてきました。

ジャカード織機は、熟練した織工たちの仕事を奪いませんでした。むしろ、彼らに「超能力」を与えたのです。それは、同じ時間内でより多くの量を生産できる能力です。かつては複雑な模様を織り上げるのに数日かかっていた織工も、今では 1 日で複数の作品を生産できるようになりました。開発者にとっての人工知能（AI）も、同じように根本的な変化を引き起こしています。今では、アプリケーションの構築、コードのデバッグ、そして問題解決を、かつてないほど迅速に行えるようになりました。

では、今日のような状況において、読者のような開発者はどのような立場にあるのでしょうか？ソフトウェアエンジニアリングの基礎を習得し、AI をペアプログラマとして活用する方法を学べ

ば、読者は時代遅れになるどころか、むしろ引く手あまたな人材となるでしょう。学習し、問題を解決し、状況に適応する能力こそが読者の最大の強みです。これらのスキルを磨きつづける限り、読者は常に必要とされる存在でありつづけます。次の節では、開発者として知っておくべき概念を紐解きながら、AIとは何かについて学んでいきます。

15.1　AIとは一体何なのか？

今日、AIは私たちの会話の隅々にまで浸透しているように見えます。しかし、AIは決して新しいものではありません。コンピュータ科学者たちは、1950年代からAIという夢を追いかけてきました。当時はコンピュータが部屋一つを占めるほど巨大なものでしたが、処理能力は現在のポケットに入るスマートフォンよりも劣っていました。AIの研究は、熱狂の波と、汎用AIへの資金が枯渇する「AIの冬」が交互に訪れる中で進歩してきました。それでも、実用的なAIが消え去ることは決してありませんでした。表舞台からは見えませんが、AIは裏方で、不正検知や画像認識、検索エンジンなどを静かに支えてきました。そして2022年11月、OpenAIがChatGPTを一般公開しました。公開から5日で100万人のユーザーを獲得し、2か月後には1億人に達しました。突如として、AIは単なる研究上の興味の対象ではなく、誰もが活用できるツールとなったのです。

「AIとは何か」という問いに答える際、AIの分野が非常に広範であるため、単純な答えは存在しません。**AI**とは、単純なルールベースのシステムから、人間の脳が情報を処理する仕組みを模倣するニューラルネットワークに至るまで、あらゆるものを包括する総称です。ソフトウェア開発者にとって関連性のある内容にするため、ここでは特定の技術群に焦点を当てていきます。ここで紹介する技術は、問題を解決するのに有効であることが実証されているものです。

この節では、AIについて、一見難しそうに思える用語を紐解き、AIをより身近なものにしていきます。AIが得意とする分野と、不得意な分野について学んでいきます。最後に、ソフトウェアエンジニアである読者にとって、AIが具体的にどのような役割を果たせるのかを探っていきます。

15.1.1　AIの用語を紐解く

新しいテクノロジーを学ぶことは、しばしば気が重く感じられるものです。特に、新しい分野で見られる専門用語をすべて習得しようとする場合はなおさらです。AIの分野では、**図15-1**に示すように、機械学習（ML）、ディープラーニング、生成AI（GenAI）、大規模言語モデル（LLM）といった用語を目にすることになります。モダンAIの本質は、コンピュータにパターンや確率を認識させ、統計的に最も可能性の高い答えを選ばせることにあります。これは、一連の手順を段階的に記述する従来のプログラミングとは異なります。

ソフトウェアエンジニアとして、AI研究者になる必要はありません。ただし、これらの基本概念を理解しておくことで、アプリケーションにAI機能を組み込む際や、チーム内のデータサイエンティストやAIエンジニアと連携する際に役立ちます。

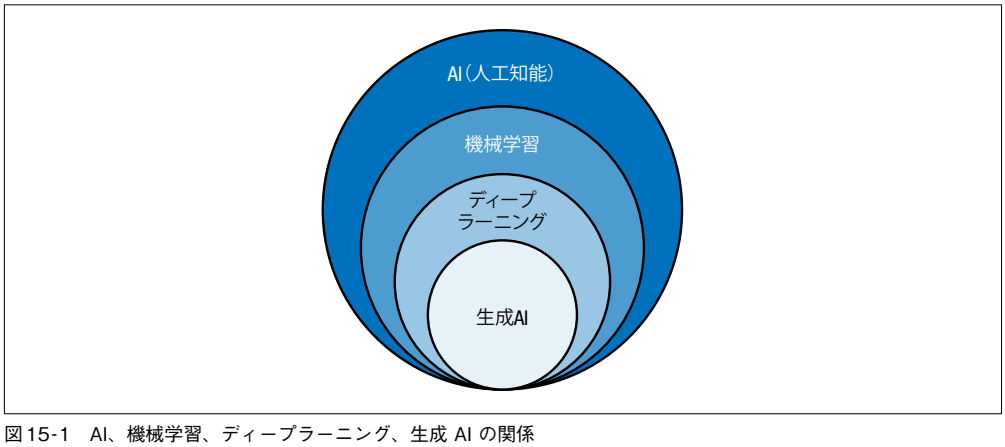


図 15-1　AI、機械学習、ディープラーニング、生成 AI の関係

15.1.1.1　機械学習

機械学習（ML、Machine learning） は、モダン AI の基盤となっています。機械学習アルゴリズムは、あらゆる判断を明示的にプログラミングするのではなく、データからパターンを学習します。その典型的な例が顔認識ソフトウェアです。すべての顔の形状を記述するコードを書く必要はありません。その代わりに、システムに何千枚もの顔画像を学習させると、システムはそれぞれの顔を特徴づけるパターンを識別できるようになります。

従来のプログラミングでは、「入力＋プログラム＝出力」というモデルに従い、明示的なルールを記述します。機械学習では、これとは逆に、「入力＋出力＝プログラム」というモデルを採用し、入力例とその正しい出力を提供します。そうすることで、アルゴリズムが自動的にルールを導き出します。

このアプローチが効果的なのは、人間にとっては容易に判断できるものの、明示的にプログラミングするのが難しい問題においてです。例えば、どのメールをスパムとして分類すべきかを検出するタスクを任されたと想像してみてください。これまでに何千通ものスパムメールを見てきた知識と経験があれば、メールを開いてさらっと眺めるだけで、どれがスパムであるかを素早く見分けることができるでしょう。

では、こうしたパターンを識別するコードをどのように書けばよいのでしょうか？ 不審なフレーズ、送信者のパターン、書式の癖、その他数千もの要因について、ルールを設定する必要があります。こうしたルールをすべて設定したとしても、スパマーは絶えず手口を進化させ、ルールをすり抜けてしまいます。機械学習（ML）は、スパムメールや正当なメールの例から学習します。新しいパターンが発見されるたびに自動的に適応するので、こうした問題を解決します。

開発者にとって、機械学習は主に以下の 3 つの課題で活用されます。

分類

あるものがどのカテゴリに属するかを判断することです。「これは何ですか？」という問い

に答えるようなものと考えてください。例としては、スパム検出（スパムか正当なメールか）、医療診断（病気か健康か）、コンテンツのフィルタリング（適切か不適切か）などが挙げられます。アルゴリズムは、カテゴリを区別するためにパターンを認識するよう学習します。

回帰分析

数値の予測を行うことです。カテゴリではなく「どのくらいですか？」や「いくつですか？」といった問いかけを行います。例としては、立地や条件に基づいた住宅価格の予測、配達時間の見積もり、売上高の予測などが挙げられます。アルゴリズムは、入力された特徴量と数値的な結果との間の関係を学習します。

クラスタリング

事前にグループを指定せずに、類似した項目をまとめて分類する手法です。これは「どのような自然なグループ分けが存在するか？」という問いに対する答えとなります。例としては、顧客セグメンテーション（さまざまな購買者像の特定）、大量のドキュメント群の整理、ユーザーの行動パターンの特定などが挙げられます。分類とは異なり、どのグループを探すべきかをアルゴリズムに指示するのではなく、アルゴリズムが自らグループを発見します。

開発者としては、アルゴリズムを一から構築するのではなく、APIや事前学習済みモデルを通じて機械学習を活用するのが一般的です。一般的なアプリケーションに機械学習を組み込む方法は、以下の通りです。

```
// APIを通じて事前学習済みの機械学習モデルを使用する
public class EmailClassifier {
    public boolean isSpam(String emailContent) {
        MLPrediction prediction = mlService.classify(
            emailContent,
            "spam-detection-model"
        );

        return prediction.getConfidence() > 0.85
            && prediction.getLabel().equals("spam");
    }
}
```

15.1.1.2　ディープラーニング

ディープラーニングは、機械学習の一分野です。多層構造を持つニューラルネットワークを使用するため、「ディープ（深層）」と呼ばれています。では、ニューラルネットワークとは何でしょうか？ これは、人間の脳が情報を処理する仕組みに着想を得て、人間の脳を簡略化したモデルだと考えてください。人間の脳には、互いに信号をやり取りする約860億個のニューロン（神経細胞）が相互につながっています。ニューラルネットワークには、この機能を模倣したノード（人工ニュー

ロン）が含まれており、それらを接続してデータを伝達します。

ニューラルネットワークにおいて、各層は処理層として機能し、それぞれに特定の役割が割り当てられています。深層ニューラルネットワークでは、生データを受け取る入力層と、そのデータを処理・変換する複数の隠れ層が存在します。各層によるデータ処理が完了すると、出力層が最終結果を出力します。これらのネットワークは複数の層で構成されており（通常3層以上あれば「深層」と呼ばれます）、これにより複雑なパターンを学習できます。

ニューラルネットワークは、データの中から複雑なパターンを見つけ出すのが非常に得意です。従来の機械学習ではコンピュータがパターンを認識できるようにしました。これに対し、ディープラーニングはニュアンスやコンテキストを捉える能力をコンピュータに与えます。顔認識の場合、最初の層では単純なエッジや線を検出します。次の層で、それらを組み合わせて基本的な形状を認識し、さらに深い層では目や顔の構造といったより複雑な特徴を識別します。そして最終層がこれらすべてを統合し、画像全体を「Dan」や「Nate」として分類します。

開発者としては、ニューラルネットワークを一から構築するよりも、事前学習済みモデルを通じてディープラーニングを活用するケースがほとんどでしょう。これらは、膨大なデータセットを用いてすでに学習済みのモデルであり、アプリケーションに組み込むことができます。以下に、Javaアプリケーションで事前学習済み画像認識モデルを、どのように活用できるか具体的な例を紹介します。

```
// 事前学習済みのディープラーニングモデルを使用する
public class ImageClassifier {
    public String identifyImage(byte[] imageData) {
        DeepLearningModel model = ModelLoader.load("face-recognition-v1");
        return model.classify(imageData);
    }
}
```

15.1.1.3 生成AI

生成AI（GenAI） は、AIの能力を根本から変革しています。従来のAIが既存データを分析・分類するのに対し、生成AIはこれまで存在しなかった新しいコンテンツを生み出します。機械学習が、コンピュータに「優れた絵画とはどのようなものか」を認識させることだとすれば、生成AIはコンピュータに「新しいオリジナルの絵画を創作させる」ことだと言えます。これが実現できるのは、AIモデルが、すべて人間によって作成された、複数のモダリティにわたる膨大な量のデータを用いて学習されているからです。モデルは、人間が創作する際のパターン、スタイル、構造を学習し、そのパターンを活用して、人間が作成したような新しいコンテンツを生成するのです。

生成AIは多種多様なコンテンツに対応しており、それぞれが開発者に新たな可能性をもたらします。

テキストとコード

AIは、説明文に基づいて記事やドキュメント、コードの関数、さらにはアプリケーション

全体を作成することができます。ユーザーにわかりやすいエラーメッセージを生成することから、現実的なテストデータを作成することまで、あらゆるものが含まれます。

画像

オリジナルのアートワークを作成したり、アプリのスタイルに合わせたアイコンを生成したり、マーケティング用のビジュアルを作成したり、コンテンツに関連するダミーの画像を作成したりできます。AIは、「モダンなフラットデザインスタイルの、親しみやすいロボットのアイコン」といった抽象的な概念を理解します。

音声

アクセシビリティ機能向けに自然な話し声を生成したり、アプリケーション用のBGMを作成したり、効果音を作成したりできます。

動画

デモ動画の制作、複雑な概念のアニメーション解説の作成、マーケティングコンテンツの生成が可能です。AIは、「このモバイルアプリの機能の使い方を紹介する30秒の動画を作成してください」といったリクエストを理解することができます。

以下は、開発ワークフローで生成AIを活用する具体例です。

```
// 生成AIを活用したテストデータの作成
public class TestDataGenerator {

   private final ChatClient chatClient;

   public TestDataGenerator(ChatClient.Builder builder) {
       this.chatClient = builder.build();
   }

   public List<User> generateTestUsers(int count) {
       var userList = chatClient.prompt()
               .user( (u) -> {
                   var prompt = """
                       Generate {n} realistic user profiles with
                       name, email, age, and interests""";
                   u.text(prompt).param("n", count);
               })
               .call()
               .entity(UserList.class);

       return userList.users();
   }
}
```



生成 AI が新しいコンテンツを生成する際は、その出力を必ず検証するようにしてください。生成されたコードはテストし、生成されたデータは確認し、ユーザー向けのコンテンツについては、正確性と妥当性をチェックする必要があります。

多くのソフトウェア開発者は、機械学習アルゴリズムの開発や、基盤となる言語モデルの構築に直接関与しているわけではありませんが、API やサービスを通じて AI 技術と日常的に接しています。これらの基礎を理解することで、より優れたアプリケーションを構築できるだけでなく、チーム内の AI 専門家とも円滑に連携できるようになります。

15.1.1.4 大規模言語モデル

大規模言語モデル（LLM）は、本章でこれまで取り上げてきた概念を統合したものです。これらは深層学習ニューラルネットワークを活用しており、生成 AI の代表的な形態です。これらのモデルは、膨大なデータからパターンを抽出し、人間のようなコンテンツを理解・生成します。ChatGPT、Claude、GitHub Copilot は、いずれも LLM によって動作しています。

LLM が特に強力である理由は、人間の言語におけるコンテキストやニュアンスを理解できる点にあります。キーワードを認識するだけだった従来の AI とは異なり、LLM は意味を理解し、会話を続け、さらには暗に示された情報さえも把握することができます。だからこそ、ChatGPT はコードのデバッグを支援し、Claude は複雑な概念を説明し、GitHub Copilot は関連性の高いコードの補完を提案できるのです。

LLM は、言語の理解や生成に関わるタスクに優れています。ここでは、LLM を活用してドキュメントを自動生成する方法を実例で紹介します。この例では、コードを入力として受け取り、開発者向けにコードの目的と使用法を説明する JavaDoc コメントを生成します。

```
// アプリケーションに LLM を組み込んでドキュメントを生成する
public class CodeDocumentationGenerator {
    private final LLMService llm;

    public String generateDocumentation(String code) {
        String prompt = String.format(
            "Generate JavaDoc comments for this method:\n%s",
            code
        );

        return llm.complete(prompt, new CompletionOptions()
            .setMaxTokens(200)
            .setTemperature(0.3)  // 数値が低いほど、より焦点の合った出力になる
        );
    }
}
```



LLMの設定における**temperature（温度）**は、ランダム性の度合いを制御します。値が低いほど（0.0〜0.5）、出力は予測したとおりのものになります。値が高いほど（0.7〜1.0）、創造性が高まります。コード生成を行う場合、temperatureを低く設定してください。temperatureを高くすると創造性は高まりますが、自信満々に聞こえるものの実際には誤った回答である、ハルシネーション（幻覚）が生じるリスクも高まります。事実に基づくタスクでは、temperatureを低く設定することで正確性を維持しやすくなります。

15.1.2　AIの能力と限界を理解する

AIは誕生以来、著しい進化を遂げてきました。現在では高い能力を持ちますが、それでも限界があります。この限界をあらかじめ認識しておくことで、日々の業務において、いつ、どのようにAIを効果的に活用すべきかをより的確に判断できるようになります。

この節では、AIの強みについて探求し、ワークフローにAIを取り入れるべきタイミングを見極めるお手伝いをします。同様に重要なこととして、LLMの限界についても検証し、より慎重に対応すべき分野について知見を提供します。

15.1.2.1　ソフトウェア開発者にとってAIが得意とする分野

AIは多くの職種において効率と生産性を向上させていますが、ソフトウェア開発者である読者にはどのような影響があるのでしょうか？ 日々の業務の足を引っ張っているルーチンワークを、AIがどのように自動化できるか、具体例を見てみましょう。こうした業務から解放されることで、より意義のある仕事に集中でき、全体的な満足感も高まることでしょう。

反復的なコーディング作業

反復的なコーディング作業こそ、AIが最大の投資対効果（ROI）をもたらす分野です。AIは知的なコーディングパートナーとして、複数のファイルにまたがる類似パターンの実装や、一貫性のあるAPIの作成、さらには機能全体の開発を支援します。これにより、時間を節約でき、定型コードの記述ではなく、アプリケーション固有の課題に集中できるようになります。

自動化スクリプトと「手間のかかる作業」

これは、AIが日々のワークフローにおいて価値を提供できる分野です。開発者は、サーバーからのファイルダウンロード、ディレクトリの整理、デプロイ手順の自動化といった、日常的な運用タスクを頻繁に処理しなければなりません。AIは、こうした「手間のかかる作業」のためのシェルスクリプトやbashファイルの作成、およびスクリプトの自動化に非常に優れています。

コードの説明

コードの解説機能は、新しい言語やフレームワークを学んでいる開発者や、新しいコードベースに参画する開発者にとって、AIが提供する非常に価値のある機能です。コードベー

スにアクセスできれば、AI に次のような質問を投げかけることができます。「このアプリケーションは、大まかにどのような動作をするのですか？」「主なエントリーポイントはどこですか？」「主要な依存関係は何ですか？」また、新しいことを学ぶ際には、そのトピックに関して包括的な学習計画を AI に作成してもらうことも可能です。

ドキュメントの作成

これもまた、AI が得意とする分野の一つです。正直なところ、開発者は義務付けられていない限り、ドキュメントの作成を行わないものです。AI は、個々のメソッドやクラスから、包括的なプロジェクトドキュメントに至るまで、あらゆるレベルのコードドキュメントを生成するのに役立ちます。また、ドキュメントをある形式（Markdown など）から別の形式（HTML など）に変換することも可能です。

リファクタリングと最適化の提案

AI にリファクタリングや最適化をお願いすると、経験豊富なエンジニアがコードをレビューしフィードバックしてくれたかのような提案が得られます。ボトルネックを特定したり、より洗練された実装方法を提案したり、コードの改善につながる最新の言語機能やライブラリを推奨したりしてくれます。

テストケースの生成

これもまた、AI が活用できる分野の一つです。これにより、より創造的で複雑な問題に集中できるようになります。ドキュメントと同様に、テストもまた、通常は後回しにされがちなものですが、AI の支援を受けることで大きなメリットが得られます。包括的なテストスイートがあったとしても、重要なエッジケースに対するテストを見落としてしまうおそれがあります。アプリケーションに関するコンテキストを AI に伝えれば、AI は既存のテストスイートのスタイルやパターンに沿ったテストを生成することができます。

コード変換

AI は、言語、フレームワーク、データ型をまたいだコード変換に役立ちます。これは Python、JavaScript、Java などの高水準言語では特に効果的です。ただし、限界もあります。低水準言語やハードウェア固有のコード、あるいは複雑な依存関係を持つシステムでは、信頼性が低下するおそれがあります。AI は些細ながらも重要な詳細を見落とす場合があるため、変換後のコードは常に徹底的にテストするように心がけてください。

ユーザーインターフェースの設計とモックアップの作成

ユーザーインターフェースの設計やモックアップの作成において、AI はユーザーの説明に基づいて、UI レイアウトやコンポーネント構造、さらにはインタラクティブなプロトタイプまでを迅速に生成することができます。AI は HTML/CSS レイアウトを作成したり、レスポンシブデザインのパターンを提案してくれます。実装に着手する前に、さまざまなインターフェース案を視覚化する際に役立ちます。

言語とフレームワークの機能の理解

適切なプロンプトを与えれば、LLM はドキュメントをくまなく調べる手間を大幅に省いてくれます。チャットボットに API の説明やライブラリの細かい点を尋ねてみてください。まるで無限の忍耐力と豊富な経験を持つパートナーがすぐそばにいるような心強さが感じられます。

15.1.2.2　AI の現在の限界

AI は多くの分野で生産性の向上に役立ちますが、完璧というわけではありません。コードの品質を損なうことなく AI を効果的に活用するためには、AI の限界を理解することが重要です。

以下に挙げる制限事項を検討する際は、モデルとプロダクトを区別してください。チャットボットを利用する際、ユーザーがやり取りしているのは、特定のモデルの上に構築され、そのモデルと通信するプロダクトです。これらの限界は主にモデルレベルに存在します。しかし、プロダクト側で機能を拡張して対処していることがよくあります。例えば、ChatGPT にお気に入りの会社の現在の株価を尋ねても答えを知りません。ChatGPT は特定の時点までのデータセットで学習されているからです。しかし、ChatGPT はツールを呼び出して、必要な情報を Web から検索できます。

リアルタイムの知識がないというのは、現在の AI システムの大きな制約です。最新のドキュメントにアクセスしたり、最新のツールやフレームワークを理解したり、リアルタイムのイベントを処理したりすることができません。こうした制約により、正確で適切な回答を提供できない場合があります。

ハルシネーション（幻覚）は、AI の非常に危険な限界です。AI システムは意図的に誤った答えを出すわけではありません。しかし、事前学習されたデータと与えられたコンテキストのみに基づいて応答を生成することがあります。その結果、一見もっともらしいのに実際には誤っている答えを、自信を持って提示してしまいます。これは、AI が事実を知っているからではなく、学習データに含まれるパターンに基づいて応答を生成するためです。

コードベースに関するコンテキストが不足していると、AI はプロジェクトのアーキテクチャ、既存の規約、あるいはビジネスルールを理解することができません。こうしたコンテキストがなければ、AI は単独でコードを生成することになり、既存のシステムとの適切な統合や、チームのコーディング標準や好みに沿ったコード生成が困難になります。

学習データに内在するバイアスは、AI が問題のあるパターンを生成したり、多様な視点を反映していない仮定を立てたりする原因となります。AI モデルは既存のコードリポジトリやドキュメントを用いて学習されます。しかし、これらには時代遅れの慣行や文化的バイアス、あるいはアクセシビリティや包摂性を考慮していない考え方などが含まれているおそれがあります。その結果、生成されたコードが意図せず特定のグループを排除してしまうことがあります。

ビジネス要件を理解できないことは、AI の意思決定能力を制限します。AI は、性能と保守性のトレードオフを比較検討したり、規制順守の要件を理解したり、特定の業界やユーザー層に関する専門知識を必要とする意思決定を行ったりできません。

分野によって性能にばらつきがあるのは、AI は一般的な言語やよくあるパターンでは優れた性能を発揮しますが、ニッチな技術や新しいフレームワーク、あるいは学習データが限られている専門的な分野では苦戦することを意味します。

プライバシーやセキュリティ上のリスクが、AI ツールとコードやデータを共有する際に生じます。多くの AI プラットフォームでは、会話履歴を保存したり、入力データを利用してモデルの精度を高めたりしており、意図せず機密情報が漏洩してしまうおそれがあります。独自のコード、API キー、データベーススキーマ、あるいは機密性の高いビジネスロジックを AI ツールと共有することは、セキュリティ上の脆弱性や知的財産権に関する懸念を引き起こすおそれがあり、慎重な検討が必要です。



AI が生成したコードに対しても、人間が書いたコードと同じく厳格なプロセスを常に適用してください。徹底的なテスト、コードレビュー、慎重な統合計画が必要な「草案」と捉えてください。コード生成で節約できた時間は、徹底的な検証に充てるべきです。

AI の強みと限界の両方を理解することで、コードの品質を維持しつつ、これらのツールを開発ワークフローに取り入れることができます。次の節では、日々の生産性を向上させる具体的なツールについて紹介します。

15.2 ペアプログラマとしての AI

知識豊富な同僚がすぐ隣に座り、質問に答えたり、解決策を提案したり、問題の解決に向けて一緒に考えたりしてくれる姿を想像してみてください。24 時間 365 日いつでも対応可能で、疲れを知らず、「当たり前の」質問をしたからといって決して読者を責めたりしません。まさにそれが、開発者である読者にとっての AI の役割、つまり究極のペアプログラミングパートナーなのです。

コーディングに関して言えば、AI はインラインでの支援から本格的なプロジェクト計画に至るまで、幅広いタスクを非常に得意としています。この節では、AI コーディングアシスタントに分類されるものについて解説します。また、本書執筆時点で利用可能な具体的なプロダクトについても解説していきます。AI の急速な進歩により個々のプロダクトは入れ替わっていくかもしれませんが、こうした基本的な分類自体は今後も変わらないでしょう。

15.2.1 スタンドアロン型チャットボットアシスタント

ChatGPT、Google Gemini、Anthropic Claude などのスタンドアロン型チャットボットアシスタントは、開発環境とは独立して、テキストや音声インターフェースを通じてやり取りを行う対話型 AI システムです。いつでも利用できるコーディングのメンターのようなものだと考えてください。

これらのツールは、概念の理解、複雑な問題のデバッグ、アーキテクチャ決定において特に役立ちます。説明付きの丁寧なコード生成、複雑な問題解決、あるいはコーディング上の課題に関する

詳細な議論が必要な際にはぜひ活用ください。

以下は、コードに関する議論を行ううえでのベストプラクティスです。

- 達成したい目標について、コンテキストを説明してください。
- 具体的な問題について質問する際は、関連するコードスニペットを共有してください。
- 質問する際は、自分の立場だけでなく、想定されるユーザー像も伝えてください。単に解決策を求めるのではなく、説明を求めてください。
- 新しいテクノロジーやフレームワークを学ぶために、AIアシスタントを活用してください。

例えば、ソートが必要なオブジェクトのリストがある場合、「これをどうソートすればいいですか？」といったプロンプトでは、意味のある回答が得られる可能性は低いでしょう。このプロンプトでは、どのようなコードが必要なのかがAIに伝わらないからです。その代わり、次のようなプロンプトの方が良いでしょう。「Customerオブジェクトのリストがあり、Java 24を使用して、登録日順（新しい順）にソートしたいのですが」

「15.3 プロンプトエンジニアリングの基礎」でも繰り返し説明するように、AIに与えるプロンプトの質は、得られる結果の質に大きく影響します。

15.2.2 インラインIDEアシスタント

インラインIDEアシスタントは、IDE内で直接動作するAI搭載ツールです。入力中にリアルタイムで提案や自動補完を行います。これらのツールは、プラグインや組み込み機能を通じて開発環境に統合され、日常的な作業を代行することで、コーディングの効率化を支援します。

人気のあるものとしては、GitHub Copilot、JetBrains AI Assistant、Amazon Q Developerなどが挙げられます。モダンなコードエディタのほとんどは、これらのAIアシスタントに対応しており、あらゆるレベルの開発者が利用できるようになっています。

次のような場合には、インラインアシスタントを使用してください。

- ゲッター／セッターやコンストラクタなど、繰り返し発生するコードの記述
- 標準的なパターン（シングルトン、ビルダーなど）の実装
- 既存のコードに基づいたユニットテストの生成
- コードのコメントやドキュメントの作成
- APIや構文に関してすぐにアドバイスが必要な場合

AIアシスタントを最大限に活用したいのであれば、以下の点に留意してください。

- 生成されたコードを受け入れる前に確認してください。
- 提案は最終的な解決策ではなく、出発点として活用してください。
- AIが読者のコードベースから学習するパターンに注意を払ってください。
- AIの提案によって、自身のアーキテクチャ上の決定が左右されないようにしましょう。

- まずは単純で反復的なタスクから始め、徐々に複雑なパターンに応用してください。

インラインアシスタントが、コードとドキュメントの両方でどのように役立つか見てみましょう。

```
public class Calculator {
    // "public int add"と入力し始めると、AI が次のように提案する
    public int addNumbers(int a, int b) {
        return a + b;
    }

    // また、"/** Calculate"と入力し始めると次のようなドキュメントが作成される
    /**
     * Calculates the sum of two integers
     * @param a the first number
     * @param b the second number
     * @return the sum of a and b
     */
}
```



AI が生成したコードについては、そのコードをそのまま受け入れて次に進むという落とし穴にはまらないようにしてください。「このコードが何を実行するのか、なぜこのように動作するのかを理解しているか？」と自問してください。もし理解できていない場合は、より明確に理解できるよう、自分で調べてみてください。コードレビューの際には、すべての行について説明できなければなりません。次のことをよく覚えておいてください。AI は生産性を最大化するためのツールであり、理解しなくて済むようにするものではありません。目標は、学習を完全に避けることではなく、より早く学ぶことです。

15.2.3 エージェント型 AI 搭載 IDE 環境

最新の AI ペアプログラミングツールとしては、Cursor、Junie、Cline といった AI 搭載の開発環境が挙げられます。これらのツールは単なる提案にとどまらず、コーディング作業に積極的に参加し、ユーザーの要件に基づいて関数やファイル全体を記述できます。

これらの環境は、コードベース全体のコンテキストを把握し、複数のファイルにわたって、より高度な変更を加えることができます。特に、リファクタリングや機能の実装、プロジェクト全体の一貫性の維持において、その威力を発揮します。

以下に、エージェント型環境を使用する例を挙げます。

- 複数ファイルにわたる大規模なリファクタリング
- 複数の箇所の変更を必要とする機能の実装
- コードのマイグレーションやモダナイゼーション作業
- AI にプロジェクト全体のコンテキストを理解させる必要がある場合

エージェント型の環境を活用する際のベストプラクティスは以下のとおりです。

- 明確かつ具体的な要件から始めましょう。
- 非機能要件（コーディングスタイル、セキュリティ基準、テスト手法など）を事前に定義しましょう。
- コミットする前に、すべての変更内容を慎重に確認しましょう。
- バージョン管理システムを使用して、AIによって生成された変更を追跡してください。
- 大きなタスクは、管理しやすい小さな単位に分割しましょう。

具体的な例を考えてみましょう。WeatherWidget、StockPriceWidget、NewsWidget、CalendarWidget、TaskListWidgetという5つのウィジェットがすでに実装されているダッシュボードアプリケーションがあるとします。そこで、システムの健全性メトリクスを表示する新しいStatusWidgetを作成するよう、エージェント型AIに依頼します。

AIエージェントがこの課題に取り組む方法は以下のとおりです。

分析フェーズ

AIはまず、既存のウィジェットを分析してパターンを把握します。

- 基底Widgetクラスの構造を確認します
- `render()`、`updateData()`、`getConfig()`などの共通メソッドを特定します。
- データサービスパターンが一貫して使用されていることを確認します
- スタイルの規約とコンポーネントの構成を確認します。

計画フェーズ

AIが計画を立てます。

- 「基底Widgetクラスを継承したStatusWidgetを作成する必要があります」
- 「システムメトリクスを取得するためにStatusServiceが必要になります」
- 「このウィジェットは、他のウィジェットと同じ更新パターンに従う必要があります」
- 「新しいウィジェットをダッシュボードレジストリに追加する必要があります」

実装

この分析を終えて初めて、AIはコードの記述を開始し、次のようなものを作成します。

- 確立されたパターンに従ったStatusWidget.java
- 既存のサービスアーキテクチャに準拠したStatusService.java
- 新しいウィジェットを含めるよう、ダッシュボードの設定を更新
- 他のウィジェットと統一感のあるスタイル

これは、エージェント型AIの威力を示しています。単にコードを生成するだけでなく、プロジェクトのコンテキストを理解し、コードベース全体の一貫性を維持してくれるのです。



忘れないでください。読者はパイロットであり、乗客ではありません。AI ツールは強力なアシスタントですが、コードを理解し、アーキテクチャ上の判断を下し、品質を確保する責任は依然として読者にあります。AI は思考の代わりとなるものではなく、読者の能力を強化するためのツールとして活用してください。結局のところ、プロジェクト内のすべてのコードに対する責任は、それが読者自身によって生成されたものであれ、AI によって生成されたものであれ、読者にあるのです。

どのような AI コーディングツールにおいても、成功の鍵はその強みと限界を理解し、状況に応じて適切なツールを選ぶことにあります。開発者として成長していくにつれ、これらの AI アシスタントも読者と共に進化し、コーディングの道のりにおいてより洗練されたパートナーとなっていくことでしょう。

15.3 プロンプトエンジニアリングの基礎

AI を駆使するソフトウェア開発者を目指すうえで、プロンプトエンジニアリングは理解しておくべき重要なスキルです。完全な初心者であっても、経験豊富なユーザーであっても、明確で効果的なプロンプトの作成方法を習得すれば、AI アシスタントやコード生成ツール、エージェント型 IDE を活用する際のパフォーマンスが飛躍的に向上します。

実のところ、多くの開発者が AI ツールの活用に苦労しているのは、技術が未熟だからではなく、自分の要望を効果的に伝える方法を学んでいないからです。このスキルは、チームメンバーに向けて明確に要件を記述したり、コードのドキュメントを作成したりするといった、実際の業務でもよく必要とされます。AI を活用する際は、同僚との会話と同じように捉え、明確かつ丁寧なコミュニケーションを心がける必要があります。

15.3.1 プロンプトエンジニアリングとは何か？

本章で学んだように、AI を扱う際には、少しハードルが高く感じられる言葉がいくつかあります。**プロンプトエンジニアリング**もその一例です。プロンプトエンジニアリングとは、AI と効果的にコミュニケーションを行う方法を学ぶことに他なりません。検索エンジンを効果的に使いこなす方法を学ぶようなものだと考えてください。これは専門家だけのものではなく、誰もが身につけるべきスキルなのです。

オフィスで、隣のブースにいるジュニア開発者に「ログインのバグを直して」と頼む場面を想像してみてください。コンテキストが伝わらないままでは、この単純な依頼が複雑な問題に発展してしまうおそれがあります。相手はチケットシステムを検索して未解決の課題を探すのに時間を浪費してしまうかもしれません。ログインフォームは複数あるため、どれを修正すればよいのかわからなくなるからです。何時間もかけてログインシステムをテストしても、問題が見つからないという事態も起こり得ます。しかし、もし「ユーザーダッシュボードのログインフォームがメールアドレスの形式を正しく検証しておらず、無効なメールアドレスが送信されてしまい、下流で問題が発生

している」と伝えたとしたらどうでしょう。この場合、はるかに良い結果が得られるはずです。

AIについても同じことが言えます。プロンプトの質は、AIの応答の質と直接関連しています。

15.3.2 プロンプトエンジニアリングの必須テクニック

AIを最大限に活用するには、単に質問をするだけでは不十分です。重要なのは、適切な方法で「正しい質問」をすることです。この節では、AIツールとのやり取りを一変させるテクニックを紹介します。何度も手直しが必要なありきたりな回答を得るのではなく、常に期待どおりの結果を生み出すプロンプトの書き方を学んでいきます。

15.3.2.1 明確なコミュニケーションが重要

AIから良い結果を引き出したいのであれば、具体的に指示する必要があります。これは、期待どおりの出力が得られなかった際に、プロンプトを繰り返し調整していくことで、徐々に身につくテクニックです。例えば、以下のプロンプトを考えてみてください。

```
ソートを行うJavaのコードを書いてください
```

このプロンプトは曖昧すぎます。何をソートするのか、どのようにソートするのか、どのデータ構造を使うのかが明示されていません。AIは、数字、文字列、オブジェクト、あるいはそれ以外の何かをソートしたいのか、推測しなければなりません。以下のように記述するのがプロンプトとしては適切です。

```
EmployeeオブジェクトのArrayListを給与の降順でソートするJavaメソッドを作成してください。null値が
入力された場合のエラー処理も実装してください。
```

このプロンプトには、AIが必要とするすべての情報が含まれています。具体的には、特定のデータ構造（`ArrayList`）、ソート対象（`Employee`オブジェクト）、ソート基準（給与）、順序（降順）、さらには考慮すべきエッジケース（null値の入力）まで含まれています。

15.3.2.2 構成こそが成功の鍵

構成の整ったプロンプトは、AIが読者の要望を理解するだけでなく、どのような形式で出力を提供してほしいのかまで理解するのに役立ちます。例えば、次のプロンプトを見てください。

```
データベースのインデックス作成について説明してください
```

このプロンプトには、内容の深さや視点、形式に関する指示は一切含まれていません。AIの回答は、一文だけのものから、大学院レベルのコンピュータサイエンスの解説に至るまで、あらゆる内容になる可能性があります。以下のように記述するのが適切です。

```
経験豊富なJava開発者として、データベースのインデックスについて以下の3つのポイントに分けて説明してください。

1．インデックスとは何か、そしてパフォーマンスにとってなぜ重要なのか
2．クラスタ化インデックスと非クラスタ化インデックスの使い分け
3．PostgreSQL でインデックスを作成する実用的な例

それぞれの項目は 2〜3 文にまとめ、コード例を 1 つ含めてください。
```

このプロンプトは、明確な構成（3 つの番号付き項目）を示し、説明対象（Java 開発者）を明示し、文章の長さの目安（各項目につき 2〜3 文）を定め、具体的な例を求める内容となっています。

15.3.2.3　命令するのではなく、教えるのだと考える

もし読者がジュニア開発者にとって良いメンターであるなら、彼らに近づいていって特定のタスクの成果を一方的に求めたりはしないでしょう。その代わりに、可能な限り多くのコンテキストを提供し、コードベースにすでに存在する例を通じて指導するはずです。結果を命令するのではなく、自分の思考プロセスを AI に示して導いてください。このアプローチは**思考連鎖（CoT、chain-of-thought）プロンプト**と呼ばれ、より正確な応答につながります。例えば、次のプロンプトを見てください。

```
このコードを最適化してください。

[ここにコードを貼り付ける]
```

このプロンプトからは、「最適化」が何を意味するのか、具体的なコンテキストが読み取れません。AI は、実行速度の向上、メモリ使用量の削減、可読性の向上、あるいはそれとはまったく異なる何かを求めているのか、判断できません。以下のように記述するのが適切です。

```
大規模なデータセットを検索する、このJavaメソッドを最適化する必要があります。まず、現在の時間計算量を特定する手助けをしてください。次に、具体的な改善策を提案してください。最後に、パフォーマンスの向上を説明するコメントを添えて、リファクタリングされたコードを示してください。

[ここにコードを貼り付ける]
```

このプロンプトでは、タスクを論理的な手順に分解し、最適化の目標（大規模データセットにおけるパフォーマンス）を明確にし、コードの変更とともにその説明を求めるようになっています。

15.3.2.4　すぐに実践できる改善のヒント

以下は、AI が生成する結果を改善するために、すぐに活用できる実用的なプロンプトエンジニアリングのテクニックです。

コンテキストを具体的に説明する

常に適切な背景情報を提供してください。新規プロジェクトでしょうか、それともモダンなベストプラクティスを活用できない既存のレガシーシステムでしょうか？ 個人的なプロジェクトでしょうか、それとも職場での業務に不可欠なアプリケーションでしょうか？

例を示す

AIに、どのような出力が適切なのかを示してください。コードを特定の形式で書きたい場合は、例を提示してください。

自身の環境について、コンテキストを説明する

お使いのJavaのバージョン、使用しているフレームワーク、または作業上の制約について記述してください。

役割を指定する

AIに、特定の役割や専門知識を持たせてください。例えば、「このコードをレビューするシニアJavaアーキテクトとして振る舞ってください」や、「パフォーマンス最適化の専門家として、このクエリを分析してください」といった具合です。これにより、適切な詳細度と視点に基づいた回答が得られます。

繰り返し、改善する

最初から完璧な結果が出ると思わないでください。AIの返答を参考にプロンプトを調整し、希望する結果に近づけていきましょう。

AIを活用してプロンプトを改善する

期待通りの結果が得られない場合は、AIに「より適切なプロンプトの作成」を手伝ってもらいましょう。例えば、「このパフォーマンスの問題のデバッグを手伝ってほしいのですが、あなたの回答は私が求めていたものとは少し違いました。より具体的なデバッグ手順を得るために、プロンプトをどのように言い換えるべきか提案していただけますか？」といった感じで試してみてください。

良いプロンプトを保存する

効果的なプロンプトを作成したら、保存しておきましょう。将来、同じようなリクエストが必要になる可能性が高いからです。このトピックに関するより詳しいヒントやコツについては、「11.3.2 個人のナレッジマネジメント」を参照してください。

15.3.3 プロンプトエンジニアリングの高度なテクニック

基本を習得したら、高度なテクニックを活用することで、より複雑な開発タスクに取り組むことができます。ここでは、高度なテクニックを主に2つのカテゴリに分けて解説します。1つ目は、リクエストの構成や枠組みを整えるのに役立つ「構造化テクニック」です。2つ目は、複雑で複数

の部分からなるプロンプトを管理するのに役立つ「情報整理テクニック」です。

15.3.3.1 構造化テクニック

ここでは、よく使われるプロンプトのテクニックと、その使い方をいくつか見ていきましょう。

ゼロショットプロンプティング（zero-shot prompting）とは、例を一切提示せずに AI にタスクを実行させることです。このアプローチに何ら問題はなく、場合によっては、まさに求めていた結果が得られることもあります。例を示します。

```
株価監視システム向けに、オブザーバーパターンを実装する Java クラスを作成してください。
```

ワンショットプロンプティング（one-shot prompting）とは、AI に守ってほしいパターンを示すために、単一の例を示す手法です。例えば、AI に以下のように指示します。

```
// 私が好むコーディングスタイルの例を以下に示します

public Optional<User> findUserByEmail(String email) {
    if (email == null || email.trim().isEmpty()) {
        return Optional.empty();
    }
    return userRepository.findByEmail(email.toLowerCase());
}

// それでは、Long型のidパラメータを受け取るfindUserByIdという同様のメソッドを作成してください。
```

この例では、特定の種類のメソッドを書く際の自分の好みのスタイルを AI に学習させました。AI は、そのスタイルに沿った同様のメソッドを生成してくれるはずです。

フューショットプロンプティング（few-shot prompting）とは、単なる指示ではなく、例を用いて AI モデルを学習させる手法です。自分が何を求めているかを説明する代わりに、望ましい入出力パターンの例を AI にいくつか提示します。例えば、AI に顧客レビューの感情を分類させたい場合を考えてみましょう。

```
以下の顧客レビューの感情を分類してください。

例 1：「このソフトウェアは直感的で、何時間も作業時間を節約できます」→肯定的
例 2：「ドキュメントが充実しており、サポートチームも素晴らしい」→肯定的
例 3：「インターフェースに不具合が多く、頻繁にクラッシュする」→否定的

それでは、次の文を分類してください：「新機能は便利ですが、UI がわかりにくい」
```

以前に紹介した AI の用語を思い出してもらえれば、これは**分類**の問題であり、モデルが今後提示される例を正しく分類できるよう、十分な数の例を与えることが重要だということがわかることでしょう。

15.3.3.2　情報整理テクニック

情報整理テクニックを活用することで、AIシステムからより適切で正確な回答を引き出すことができます。プロンプトを明確に整理することで、不確実性を減らし、AIが最も重要な点に集中できるようになります。

XMLタグ（およびMarkdownやJSONなどの構造化形式）は、異なる種類の情報を明確に区別することで、複雑なリクエストを整理するのに役立ちます。これにより、AIは各情報の目的や優先順位を把握しやすくなります。以下に例を示します。

```
<task>
ユーザー管理用のRESTful APIエンドポイントを作成する
</task>

<requirements>
- ユーザーの作成にはPOST /usersを使用する
- メールアドレスとパスワードの検証を行う
- 適切なHTTPステータスコードを返す
- Spring Bootのアノテーションを使用する
</requirements>

<constraints>
- Java 17
- Spring Boot 3.0
- Spring Starter以外の外部依存関係は無し
</constraints>
```

このXML構造により、AIは、構築すべきもの（task）、要件（requirements）、制約条件（constraints）を区別できるようになります。この構造がなければ、すべての情報が混在してしまい、AIが重要な情報を見落としたり、優先順位を誤ったりするおそれがあります。

タスクの分解とは、複雑な問題を扱いやすい単位に分割することです。これは、ソフトウェアエンジニアである私たちが日常的に行っていることです。自分自身やジュニア開発者のためにタスクを細分化するのと同様に、AIに対しても同じことを行うことで、より良い結果を得ることができます。AIモデルに対して大規模な要求を行うと、混乱し誤った回答や不完全な回答を返してしまうおそれがあります。リクエストを段階的に分割することで、各要素をより詳細に分析し、その過程で調整を加えられるようになります。例を示します。

```
Javaでファイル処理システムを構築しています。以下の手順についてご教示ください。

フェーズ1：ファイル読み込み方式の設計（ストリーム方式 vs バッチ方式）
フェーズ2：データ検証ロジックの作成
フェーズ3：エラー処理とロギングの実装
フェーズ4：ユニットテストの追加

まずはフェーズ1から始めて、1GB以上のファイルを処理する際のそれぞれの手法のメリットとデメリットを分析してください。
```

AIツールを効果的に活用するための鍵は、フレームワークやテクニックを暗記することではありません。明確なコミュニケーションの習慣を身につけ、どのようなアプローチが最も効果的か理解することにあります。まずは、具体的なプロンプトを作成して、システムに期待する出力を伝えることから始めましょう。自分自身やプロジェクトに関するコンテキストを提供し、試行錯誤を重ねながらアプローチを調整していく姿勢を持ちましょう。

AIを最大限に活用する方法がわかったところで、ソフトウェア開発の未来がどのようなものになるのか、見ていきましょう。

15.4 AIがソフトウェアエンジニアリングに与える影響

ソフトウェアエンジニアにとって、未来の仕事はどのようなものになるのでしょうか？ 将来がどうなるかを確実に予測できる人はいませんが、AIがソフトウェア開発において重要な役割を果たすことは間違いありません。AIが時とともに成長し変化しつづける中で、他のエンジニアと自分を差別化する要素について考えることが重要です。AIがコードを書く能力をますます高めていくにつれ、コラボレーション能力、共感力、メンタリング能力など、読者独自の資質やソフトスキルが、これまで以上に重要になっていくでしょう。

この節では、AIが開発者の役割をどのように変革しているかについて解説します。AIは開発者に取って代わるものではなく、生産性を高め、コードの記述やレビューの方法を一新する、強力なツールとして機能します。既存のスキルを活かしながら、AIを活用してより効果的な問題解決能力を身につける方法について学びましょう。

15.4.1 AIに仕事を奪われてしまうのか？

本章では、AIがどのようにタスクの自動化、コードの生成、テストの作成、ドキュメントの作成を行うかについて見てきました。こうした機能が向上するにつれ、あらゆるレベルのソフトウェア開発者が「AIに仕事を奪われてしまうのではないか？」と懸念を抱いています。この疑問には、過去の傾向や現在の動向から、ある程度答えを推定できます。

本書を通じて学んだように、ソフトウェアエンジニアという仕事には、単にコードを書くこと以上の多くのことが求められます。2章では、コードを書く時間よりも、コードを読む時間のほうがはるかに長くなることを学びました。この基本的なスキル、つまりコードを理解し、把握する能力は、そのコードが他の開発者によるものであれ、AIによるものであれ、依然として不可欠なものです。

コードを読むことは、コードを書くこと以外に、ソフトウェア開発者が行う多くの業務の一例に過ぎません。この機会に、開発者の役割を構成するいくつかの業務について振り返ってみましょう。コードを書く前に、不完全で曖昧な要件を、実際に動作するシステムへと形にする責任があります。9章で学んだように、パフォーマンスや保守性といった機能間のトレードオフについて、「状況による」という判断を下すことになります。また、社内の人間関係をうまくまとめ、ジュニア開発者を指導し、技術に詳しくないステークホルダーに技術的な概念を説明することも求められま

す。単に**どのように**作るかだけでなく、**なぜ**作るのか、**誰**のために作るのかを理解している必要があります。

AIは、質の良い例も、質の悪い例も含め、何百万もの事例を用いて学習されており、この事前学習データに基づいて新しいコードを生成できます。しかし、AIは自社の具体的なビジネスモデルや、ユーザー固有のニーズ、あるいはCEOの「シンプルな要望」が実際にはアーキテクチャ上の悪夢である理由を理解できません。また、会議に出席して、プロダクトマネージャーの要件が営業チームが約束した内容と一致していないことに気づけません。ソフトウェア開発におけるこうした人間的な要素は、例外的なケースではなく、まさに仕事そのものなのです。

> AIは開発者に取って代わることはありませんが、AIを活用する開発者が、AIを活用しない開発者に取って代わるでしょう。
>
> — Jeff Atwood（の発言とされる）、ソフトウェア開発者、著者、ブロガー、起業家

本章の冒頭で触れたジャカード織機を覚えていますか？ 成功を収めた織工たちは、その技術に抵抗した人たちではなく、その操作法を学び、そのための模様をデザインし、それを使って以前は不可能だったものを生み出した人たちでした。

今日、ソフトウェア開発者の間でも同じような傾向が見られます。AIは、プログラマが退屈に感じるような単調な作業を排除しつつあります。定型コードの記述から、反復的なテストの作成、ドキュメントの作成など、あらゆる作業が含まれます。これを脅威と捉えることもできますが、本来の姿、つまり「解放」として捉えることもできるでしょう。やりがいを感じない作業に費やす時間が減れば、ソフトウェア開発を刺激的なものにする、創造的かつ戦略的な業務に充てる時間をより多く確保できるようになります。

プログラミングの進化を振り返ってみると、この業界はアセンブリ言語から高水準言語へ、シンプルなテキストエディタから強力な開発ツールや洗練されたIDE、そして自動化されたCI/CDパイプラインへと移行してきました。プログラマを不要にしてしまうかもしれないような技術の進歩が相次ぐ中、開発者への需要はむしろ年々高まりつづけています。

AIの未来で成功を収める開発者とは、AIをコーディングのパートナーとして捉えられる人たちです。そのパートナーは、超人的な能力を持ち、決して疲れを知らず、インターネット上の歴史的な知識を備え、基本的なタスクを代行してくれます。このため、開発者はより大規模で複雑な問題に取り組めるのです。読者の価値を決めるのは、求められたときにforループを書ける能力や、バブルソートアルゴリズムを実装することではありません。それらを**いつ**使うべきか、**なぜ**重要なのか、そして自分が構築している大規模なアプリケーションの中で**どのように**機能するのかを理解しているかどうかです。

では、AIは一部の仕事を奪うのでしょうか？ その可能性はあります。しかし、大多数の仕事を奪うことになるのでしょうか？ そうなるのは、まだ先の話です。一部の役割は再定義されるでしょうが、それは問題ありません。もしその変化を受け入れることができれば、生産性が向上し、ほんの数年前には不可能だったようなものを生み出せるようになるでしょう。問題は、AIが読者

に取って代わるかどうかではありません。重要なのは、この強力な新しいツールを活用する開発者の一人になるのか、それとも2010年の頃のような単にコードを書くことだけにしがみつく数少ない人になるのか、ということです。

では、AIが私たちの仕事を奪うわけではないとしたら、具体的に私たちの日常業務はどのように変化するのでしょうか？ 開発者の役割がどのように進化していくのか、そしてどのような新しいスキルが読者を他者と差別化するのか、探ってみましょう。

15.4.2 バイブコーディング

2025年2月5日、AIとディープラーニング分野で影響力のあるAndrej Karpathyが、Xに投稿を行い（https://oreil.ly/0zwB9）、自身のソフトウェア開発における現在の経験を要約するとともに、**バイブコーディング（vibe coding）**という用語を提唱しました。この用語自体は、意図的に皮肉を込めて使われています。Karpathyがこの言葉を考案したのは、問題のある開発手法を指摘するためであり、それを推奨するためではありません。

15.4.2.1 バイブコーディングとは何か？

バイブコーディングとは、エージェント型IDEを用い、洗練されたプロンプトを通じてアプリケーション全体を生成する手法を指します。開発者は、コードの一行一行を手作業で入力するという単調な作業を行う代わりに、構築したいものを記述するだけで、AIツールがコードベースの大部分を自律的に生成します。

このアプローチにより、これまでとはまったく異なる層のクリエイターがソフトウェア開発に取り組めるようになりました。素晴らしいアイデアを持った人は、基本的なプロトタイプを作るためだけなら、開発チームを雇ったり、何年もかけてプログラミングを学んだりする必要はありません。AIアシスタントに自分のビジョンを伝えるだけで、目の前でそのコンセプトが形になっていくのを見ることができるのです。

その意味について、少し考えてみてください。シンプルな注文システムを作成したいレストランのオーナー、生徒向けのインタラクティブなテストを作成している先生、ポートフォリオサイトを開発しているアーティストなど、プログラミングの経験がなくても、今では自分のアイデアを形にすることができるのです。

15.4.2.2 メリットとリスク

Karpathyが最初のポストで指摘したように、バイブコーディングは「週末の遊びのプロジェクト」には適しています。ここで重要なのは「遊び」という点です。自分だけが使う個人プロジェクトを作るのと、フォーチュン500企業の基盤を支えるソフトウェアを開発するのとでは、天と地ほどの違いがあります。

次の2つのケースを考えてみてください。

シナリオ 1

毎月の支出を分類するために、個人用支出管理アプリを作りたいと考えています。週末を使って、バイブコーディングによりそのアプリを作成しました。もしバグによってコーヒー代が誤って計算されてしまったとしても、最悪の場合で収支が少し不正確になる程度です。

シナリオ 2

読者の会社では、数千人の従業員が複数の州に住んでおり、それぞれの州で異なる税制要件を管理できる給与システムが必要です。ここで不具合が生じると、従業員への給与が正しく支払われなかったり、納税義務が履行されなかったりします。その結果、会社が法的措置に直面するおそれがあります。

そのリスクの大きさは、これ以上ないほど異なります。

15.4.2.3　企業の現状認識

「大手企業のコードの 25% は、現在 AI によって記述されている」といった見出しを目にする際、それが実際にはどのような状況なのかを理解することが重要です。この AI の支援によるコードは、単純なプロンプトからアプリケーション全体が魔法のように生成される、というお手軽コーディングの結果ではありません。

現実的な例として、Microsoft のような企業が実証しているワークフローが挙げられます。開発者が問題の概要を詳細に記述した課題を作成します。AI コーディングエージェントがその課題を読み取り、実際のコード変更を生成し、その変更内容を盛り込んだプルリクエスト（PR）を作成します。人間の開発者（または開発チーム）がコードをレビューし、この変更をコードベースにマージするかどうかを決定します。AI が日常的な実装作業を担当していますが、品質保証や意思決定の主導権は依然として人間が握っています。

読者が働く企業環境には、「週末プログラマ」たちには足りないものが一つあります。それは厳格なプロセスです。企業内でも、ある程度の「バイブコーディング」が行われていないわけではありませんが、そこにはチェックとバランスが機能しています。コードレビュー、テストフレームワーク、デプロイパイプライン、品質保証の手順などが整っているのです。AI 生成コードも、人間が書いたコードと同じように、厳格な審査を経る必要があります。実際、こうしたプロセスはこれまで以上に重要になっています。

コードレビューがこれまで以上に重要である理由

AI ツールはかつてない速さでコードを生成できます。だからといって開発期間が飛躍的に短縮されるわけでも、隠れたコストが伴わないわけでもありません。生成されたコードには（人間が書いたコードと同様に）、見過ごされがちなバグやセキュリティ上の脆弱性、あるいはアプリケーションにふさわしくないアーキテクチャ上の判断が含まれているおそれがあります。Neal Ford のような専門家は、チームが適切なレビュープロセスを経ずに、AI 生成のソリューションを急いで

実装しようとすることで、将来的に「質の悪いコードの津波」が押し寄せるおそれがあると警告しています。

だからこそ、コードレビューは重要性を増しており、決して軽視できるものではありません。この機能の開発者としては、生成されたコードの一行一行が何を実行するのか、またなぜ特定の判断がなされたのかを理解する必要があります。レビュー担当者としては、誰が書いたか（人間であれ機械であれ）にかかわらず、細部に至るまで注意を払う必要があることを理解しなければなりません。週末の緊急対応中に問題を発見するよりも、時間を割いて、コードレビューの段階でこうした問題を見つけ出したいと思いませんか？

コードレビューは SDLC においてますます重要な役割を担うようになります。AI を使ってメソッドやクラス、あるいは機能全体を生成する場合でも、「自分の」コードを理解し、説明できる必要があります。コードレビューのためにプルリクエストを送信する前に、コードを徹底的に確認し、そのコードが存在する理由や、どのような動作をするのかを説明できるようにしておく必要があります。「そのコードは AI が生成したものであり、自分の責任ではない」という言い訳は通用しません。自分で書いたものであれ、AI が書いたものであれ、そのコードに対する責任は読者にあり、その結果についても説明責任を負うことになります。

開発者の新しい考え方

AI が活用されるこの時代において、開発者にとって重要なのは、「バイブコーディング」の落とし穴に陥らないことです。AI ツールは開発を加速させることができますが、Karpathy が風刺したような、漫然とプロンプトを入力して「うまくいくことを祈る」ようなアプローチは、まさに避けるべきものです。その代わりに、自分自身をコードの「創造者」であり「キュレーター」であると捉える必要があります。厳格な基準を維持しつつ、AI を賢く活用すべきです。

つまり、以下のスキルを身につける必要があります。

- より優れた AI コードを生成するための効果的なプロンプトの作成
- 生成されたコードの迅速な確認と理解
- 人間が書いたコードと AI が書いたコードの両方における潜在的な問題の特定
- 潜在的なバイアスや排他性の特定
- コードレビュー時の効果的なコミュニケーション

バイブコーディングはソフトウェア開発における刺激的な変化ですが、慎重かつ考え抜かれた開発手法の必要性をなくすものではありません。むしろ、こうしたプロセスはこれまで以上に重要になっています。キャリアを積むにつれ、効率と生産性を高めるツールを積極的に取り入れていくべきです。しかし、信頼性が高く、保守しやすいソフトウェアをリリースするという基本的な責任を見失わないようにしましょう。

15.4.3 AIを戦力増幅器として活用する：コード記述から問題解決へ

戦力増幅器（フォースマルチプライヤー、force multiplier）は、軍事分野とビジネスの両方で用いられる概念です。これは、既存のスキルを飛躍的に高めるツールや能力のことです。例えば、多くの人がスマートフォンのカメラで写真を撮っています。現代のスマートフォンのカメラは性能が優れており、高品質な写真を撮影するための機能も備わっています。しかし、その上をいく選択肢として、デジタル一眼レフカメラがあります。これらのプロ仕様のカメラは数万円から数十万円もしますが、使い方を理解していれば、息をのむような美しい写真を撮影することができます。

もし、5,000ドルのカメラを与えられた人と、20年の経験を持つプロの写真家にスマートフォンを1台だけ渡した場合、どちらがより良い写真を撮れるでしょうか？ おそらくプロの写真家が勝つでしょう。なぜなら彼らは絞り、露出、照明、構図といった概念を理解しているだけでなく、自分が望むイメージを正確に捉えるための芸術的なビジョンを持っているからです。高価なカメラは、いわば「戦力増幅器」のようなものです。それは写真家の既存のスキルを強化しますが、もともと持っていないスキルを新たに生み出すことはできません。

これは、ソフトウェア開発におけるAIの現状にも通じるものです。高価なカメラが読者を写真家に変えるわけではないのと同じように、このツールが読者をより優れた開発者に変えるわけではありません。読者を他者と差別化するものは、読者の知識、経験、そして問題解決能力なのです。AIは読者の「一眼レフカメラ」のようなものです。読者がすでに持っている知識や能力をさらに引き立てる強力なツールなのです。

結局のところ、どのような技術を使おうと、ソフトウェアを作っていることに変わりはありません。開発者が考慮すべき本質的な変化は、何を作るかではなく、それを構築するためにどう時間を使うかという点です。従来のソフトウェア開発には決まったパターンがあります。時間の70%を、定型コードの記述、構文エラーのデバッグ、そして繰り返しの多いパターンの実装に費やしていたことでしょう。その結果、創造的な問題解決や、その他の管理業務に充てられる時間は、わずか30%しか残らないことになります。

AIの登場により、この構図を逆転できるようになりました。AIが日常的なコード生成や不具合のデバッグを処理してくれるようになったのです。これにより、開発者はエネルギーの70%を要件分析、アーキテクチャの決定、そして複雑なビジネス課題の解決に注力できるようになります。理論上は、プロダクトのリリース頻度は変わらないものの、より綿密に検討された状態で本番環境に投入されるようになり、対処すべきバグやエッジケースも減少することになります。

身近な問題について見てみましょう。自分が携わっている分野向けにREST APIを構築すると仮定してください。AIが登場する前は、ドメインオブジェクトや検証ロジック、コントローラ、リポジトリなどを作成するのに何時間も費やす必要があったかもしれません。しかし、AIと適切なプロンプト作成のテクニック、そして組織内でREST APIがどのような構成になっているかを示したテンプレートがあれば、これらのコンポーネントを数分で生成することができます。

定型コードの記述にかかる時間を節約できた分、重要な戦略的な判断に集中できるようになります。単にAPIをリリースするだけでなく、APIの設計や直感的な操作性について、より多くの

時間を割いて検討できます。この API が、高スループットにどのように対応するか検討しましたか？ 使用している言語やフレームワークの並行処理機能を十分に活用していますか？ この API は、負荷がかかった際にもスケーラビリティを確保しつつ、正常にフェイルオーバーできるでしょうか？ もちろん、アプリケーションがまず正常に動作することは必須ですが、こうした複雑な課題こそ、時間を割いて取り組むべきものなのです。



Dan です。AI によって自分が書くコード行数が減ってしまうのではないかと心配されているかもしれませんが、その心配は無用です。コード行数は、実際には生産性や価値をはかるには、意味のある指標ではありません。私は長年にわたり数え切れないほどの履歴書を見てきましたが、「1 万行のコードを書いた」という実績を挙げている人は一人も見たことがありません。仮にそう書かれていたとしても、感心することはないでしょう。真の実績とは、その「影響力」に焦点を当てたものです。「アプリケーションアーキテクチャを最適化し、クラウド利用料を 20% 削減した」という実績は、コード行数などでは決して伝えられない、はるかに説得力のあるストーリーを語っています。

コードを書くことから問題解決へと役割が移行しても、読者の役割が軽くなることは決してありません。むしろ、その役割はより重要なものへと昇華されるのです。読者はより大きな課題に取り組み、担当するチームやアプリケーションに大きな影響を与える意思決定を行うことになるのです。

15.5 まとめ

AI がソフトウェア開発者の代わりになるわけではありません。しかし、AI を効果的に活用できる開発者は、そうでない開発者よりも大きな優位性を得ることになるでしょう。ジャカード織機が織工を排除するのではなく、その働き方を変革したのと同様に、AI もまた、ソフトウェア開発への取り組み方を変革しつつあります。

AI は、人間の代わりとなるのではなく、ペアプログラミングのパートナーとして最も効果を発揮します。アーキテクチャの議論にスタンドアロンのチャットボットを活用する場合でも、定型コードの作成に IDE 内のアシスタント機能を利用する場合でも、あるいは複雑なリファクタリングにエージェント型環境を利用する場合でも、その目的は同じです。それは、既存のスキルを強化し、より高度な問題解決に集中できるよう、精神的な余裕を生み出すことです。

AI の能力と限界を理解することは、AI を効果的に活用するための重要な要素です。AI は、反復的な作業、コードの説明、ドキュメントの自動生成、パターン認識において優れた能力を発揮します。しかし、最新の情報をリアルタイムで反映することは難しく、誤った情報を生成するハルシネーションを起こすこともあります。また、ビジネス上のコンテキストを理解できず、セキュリティリスクをもたらすおそれもあります。AI が生成したコードに対しても、人間が書いたコードと同様に、厳格で徹底したテストやコードレビューを行うことは、単なるベストプラクティスでは

なく不可欠なことです。

ソフトウェア開発の未来は、コードを書くことよりも問題解決に焦点を当てるものへと変化しています。時間の 70% を定型コードの実装作業に費やすのではなく、そのエネルギーをアーキテクチャの決定、要件分析、そして複雑なビジネスロジックに注げるようになります。これは開発者としての役割を軽くするものではありません。より有意義で影響力の大きい課題に取り組むことで、その役割を高めるものです。

開発ワークフローに AI を取り入れ始める際は、プロンプトエンジニアリングは習得すべきスキルです。このスキルにより、成果を劇的に向上できるということを忘れないでください。コンテキストを具体的に示し、例を挙げ、リクエストを明確に構成し、プロンプトを繰り返し改善するようにしてください。そして何よりも重要なのは、コードを生成したのが誰であれ、何であれ、そのコードに対する責任を持ちつづけることです。

AI が活用される未来において成功を収める開発者とは、コードの品質、継続的な学習、そしてソフトウェアエンジニアリングの基礎を重視しつつ、こうしたツールを積極的に取り入れる人々です。AI は読者に超人的な力を与えてくれますが、それを正しく使いこなす責任は読者自身にあります。

本書では、私たちがこれまでのキャリアを通じて苦労の末に得た教訓を、読者と共有してきました。次世代のソフトウェアエンジニアである読者が目標を達成できるよう、私たちの経験を共有できることを光栄に思います。私たちが執筆を楽しんだのと同じくらい、読者も本書を楽しんでいただけたなら幸いです。

私たちはよくミートアップやカンファレンスに参加していますので、もしお会いする機会がありましたら、ぜひ声をかけてください。本書を読んでいただき、**誠にありがとうございます**。言葉では言い表せないほど感謝しております！

15.6　実践に移す

学んだことを実践するには、知識だけでなく行動が必要です。以下の実践的な手順に従うことで、コードの品質を維持しつつ、AI ツールを開発ワークフローに組み込み、プロンプトエンジニアリングの重要なスキルを身につけることができます。

1. コードベースの既存のメソッドを選び、AI にその動作の説明を求め、改善の余地を特定してもらい、テストケースを提案してもらってください。その分析結果を、自分自身の理解している内容と比較してみてください。
2. 本章で学んだ手法を用いて、デバッグ、コードレビュー、ドキュメント作成などの一般的なコーディング作業向けに、構成がしっかりとした詳細でコンテキストに即したプロンプトを 3 つ作成し、プロンプトエンジニアリングを実践してください。
3. コードレビュー、ドキュメント作成、デバッグといった一般的な開発タスクのために、効果的なプロンプトを 5〜10 個作成し保存し、自分専用の「AI プロンプトライブラリ」を作成しま

しょう。

4. AI コーディングアシスタントを 1 つ選び、1 週間、個人的なプロジェクトで活用してみてください。そのアシスタントが得意とするタスクと、苦手とする点を記録しておきましょう。
5. AI に簡単なクラスやメソッドを生成させ、それをまるでジュニア開発者が書いたかのように徹底的なコードレビューを行う「バイブコーディングレビュー」を実践してください。見つかった問題を記録しておきましょう。
6. AI を活用して、あるプログラミング言語から別の言語（Python から Java など）へのコード変換を行い、その後、検証を行って変換が正しいこと、およびベストプラクティスに従っていることを確認してください。
7. AI アシスタントを使用する場面と使用しない場面、特に注意が必要なコードの種類、および機密情報や知的財産情報の取扱方法などを明確に定義し、AI 利用ガイドラインを策定してください。

15.7 関連リソース

- 『AI Engineering』Chip Huyen 著（O'Reilly、2024 年）
  和書『AI エンジニアリング』（オライリー・ジャパン、2025 年）
- 『Beyond Vibe Coding』Addy Osmani 著（O'Reilly、2025 年）
  和書『バイブコーディングを超えて』（オライリー・ジャパン、2025 年）
- 『Prompt Engineering for LLMs』John Berryman, Albert Ziegler 共著（O'Reilly、2024 年）
  和書『LLM のプロンプトエンジニアリング』（オライリー・ジャパン、2025 年）
- "No Silver Bullet"、Frederick P. Brooks Jr. 著（University of North Carolina at Chapel Hill）
- 『Rebooting AI: Building Artificial Intelligence We Can Trust』Gary Marcus 著（Vintage、2020 年）
- 『Co-Intelligence: Living and Working with AI』Ethan Mollick 著（Portfolio、2024 年）
  和書『これからの AI、正しい付き合い方と使い方』（KADOKAWA、2024 年）
- 『Human Compatible: Artificial Intelligence and the Problem of Control』Stuart Russell 著（Penguin Books、2020 年）
  和書『AI 新生』（みすず書房、2021 年）
- 『Artificial Intelligence: A Modern Approach』Stuart Russell, Peter Norvig 共著（Pearson、2021 年）
  和書『エージェントアプローチ人工知能（原著第 4 版）』（共立出版、2026 年）
- 『Deep Learning with Python』Francois Chollet 著（Manning、2021 年）
  和書『Python によるディープラーニング』（マイナビ出版、2022 年）
- 『Taming Silicon Valley』Gary Marcus 著（Tantor Media, Inc.、2025 年）

- Simon Willison's Weblog（https://simonwillison.net/）
- Marcus on AI（https://garymarcus.substack.com/）
- Practical AI podcast（https://oreil.ly/WZtuI）

訳者あとがき

本書は、Nathaniel Schutta 氏と Dan Vega 氏による『Fundamentals of Software Engineering』の日本語版です。タイトルのとおり、ソフトウェアエンジニアリングの「基礎」を扱った一冊です。ここでいう「基礎」とは、特定のプログラミング言語の文法や、特定のフレームワークの使い方だけを意味するものではありません。

原著者が述べているように、ソフトウェアエンジニアであるということは、単に動くコードを書けるということにとどまりません。既存のコードを読み解くこと、読みやすく保守しやすいコードを書くこと、テストすること、設計すること、本番環境で動かすこと、チームやステークホルダーと協働すること、そして学びつづけること。こうした一連の取り組みを通じて、ソフトウェアを価値あるものとして育てていくことが、ソフトウェアエンジニアリングなのです。

訳者自身も、ソフトウェアに関わる仕事を長く続ける中で、コードを書くことそのものだけでなく、チーム内でコラボレーションし、関係者と認識を合わせ、ユーザーが直感的に使えるものを作る難しさと大切さを何度も感じてきました。その意味で、本書で語られている内容は、決して「新人だけが学ぶべきこと」ではなく、ソフトウェアに関わるすべての人が、ときどき立ち止まって振り返るべき基礎なのだと思います。

本書の特徴は、ソフトウェアエンジニアリングという広い世界を、エンジニアになったばかりの人にもわかりやすく示しているところにあります。コードの読み方から、テスト、モデリング、アーキテクチャ、デプロイ、ユーザーインターフェース、データの扱い、AI、そしてキャリア形成まで、本書で扱う範囲は非常に広いものです。一つひとつのテーマを極端に深掘りするというよりも、ソフトウェアエンジニアとして歩んでいくうえで、どのような考え方や視点を持っておくべきかを示してくれる本になっています。

訳していて特に印象的だったのは、本書が技術だけでなく「人」にもかなりの注意を向けていることです。ソフトウェア開発は、コンピュータに向かって一人でコードを書く作業のように思えることもあります。しかし実際には、多くの場合、既存のコードを書いた誰かがいます。そのコードを読む誰かがいます。そのソフトウェアを使うユーザーがいます。そして、チームや組織の中でさまざまな判断が行われています。コードはコンピュータ上で実行されるものですが、同時に人間に読まれ、理解され、変更され、引き継がれていくものでもあります。本書は、こうした当たり前でありながら忘れられがちなことを繰り返し伝えています。

また、この分野において現在では避けて通ることができない AI についても、本書では触れられています。ソフトウェア開発の現場では、AI を使ってコードを書くことが急速に一般的になりつつあります。こうした状況を見て、「もう基礎は必要ない」と感じる人もいるかもしれません。しかし、本書を読むと、むしろその逆であることがわかります。AI が生成したコードをレビューするにも、AI に適切なプロンプトを与えるにも、ソフトウェアエンジニアリングの基礎が必要です。コードを書くのは AI かもしれませんが、そのコードをソフトウェアとして仕上げていくための判断は、それを使う「人」に委ねられているのです。

本書は、これからソフトウェアエンジニアとして働き始める方にとって、貴重なガイドブックになるはずです。いま自分が何を知っていて、何をまだ知らないのか。これからどの方向に学びを広げていけばよいのか。そうしたことを考えるきっかけを与えてくれます。

一方で、すでに経験豊富な方にとっても、本書には多くの発見があると思います。日々の業務の中でなんとなく身につけてきたことを言語化したり、自分の経験を整理したり、ジュニア開発者やチームメンバーに何を伝えるべきかを考えたりするうえでも役立つことでしょう。特に、コードを書く力だけでなく、読む力、設計する力、伝える力、学びつづける力をどう育むかという観点は、経験年数を問わず重要なテーマです。

翻訳にあたっては、原著の親しみやすさをできるだけ残しつつ、日本語として自然に読めることを意識しました。本書は入門的な位置づけの本ではありますが、扱っている内容は決して軽いものではありません。そのため、初めてソフトウェアエンジニアリングの世界に触れる読者にもわかりやすい表現になるよう心がけました。

この分野には、学ぶべきことが本当にたくさんあります。その広さに圧倒されることもあるでしょう。しかし、本書が伝えているように、そのすべてを最初から知っている必要はありません。大切なのは、自分がまだ知らないことに気づき、必要に応じて学びつづけ、少しずつ実践してスキルを磨いていくことです。

本書が、これからソフトウェアエンジニアとして歩み始める方にとっては心強い案内役となり、すでに現場で働いている方にとっては自分の仕事を見つめ直すきっかけとなれば、訳者としては幸いです。

最後に、この書籍の翻訳作業では、オライリー・ジャパンの高恵子氏に多大なご協力をいただきました。心より感謝申し上げます。

2026 年 9 月
村上 列

索引

数字

5 つのなぜ……9

A

ACID 特性…… 176
ADR（アーキテクチャ決定レコード）…… 216
AES（Advanced Encryption Standard）…. 239
AI……34, 283, 353
　図作成 …… 109
　テスト ……84
　ペアプログラマ …… 363
　模擬面接 …… 342
API テストツール …… 102
AP システム…… 178

C

CAP 定理 …… 177
CA システム…… 178
CCPA（カリフォルニア州消費者プライバシー法）
　…… 240
CI/CD…… 253
　CI/CD パターン …… 256
　CI/CD ワークフロー …… 254
CoT（思考連鎖）プロンプト …… 369
CP システム…… 178
CSV…… 161

D

DRY 原則 ……45

E

E2E テスト ……80, 81
ER 図（エンティティ・リレーションシップ図）
　…… 111
ETL プロセス…… 196
EU 一般データ保護規則（GDPR）…… 240

F

FOMO（取り残されることへの恐れ）…… 285

G

GDPR（EU 一般データ保護規則）…… 240
GenAI（生成 AI）…… 357

GitHub ワークフロー …… 254

H

HIPAA …… 241
HTTPS による通信の保護 …… 235

I

I18N（国際化）…… 129
IDE（統合開発環境）…… 22
　コードナビゲーションツール …… 26
IntelliJ IDEA …… 23
is-a 関係 …… 43

J

JSON …… 159

L

L10N（ローカライゼーション）…… 129
LLM（大規模言語モデル）…… 359
L モード …… 278

O

ORM（オブジェクトリレーショナルマッピング）
…… 173

P

package by feature（機能別パッケージング）
…… 96
package by layer（レイヤー別パッケージング）
…… 95
PCI DSS（Payment Card Industry Data Security Standard）…… 241
PII（個人識別情報）…… 238

R

R モード …… 278

S

SBOM（Software Bill Of Materials）…… 241
Software Bill Of Materials（SBOM）…… 241
SSL（Secure Sockets Layer）…… 236

T

TTL（有効期間）付きのキャッシュアサイド
…… 182
T 字型の人材育成 …… 324

U

UI（ユーザーインターフェース）…… 125
UML（統一モデリング言語）…… 60

X

XML …… 160

Y

YAML …… 161

あ行

アーキテクチャ …… 203
　アーキテクチャと設計 …… 208
　ステークホルダーとの合意形成 …… 211

戦術的な決定 …… 208
戦略的思考 …… 208
トレードオフ …… 205
品質特性の特定 …… 210
アーキテクチャ決定レコード（ADR）……18, 216
アーキテクチャ図 …… 216
アーキテクチャスタイル …… 212
アーキテクトの肩書き …… 214
アクセシビリティ …… 128
アサーション …… 84
アジャイルアーキテクト …… 213
暗号化 …… 239
依存関係 …… 86
依存関係の管理 …… 259
一括デプロイ …… 252
一貫性モデル …… 176
イノベーションデイ …… 289
イベントストーミング …… 68
インポスター症候群 …… 331
インライン IDE アシスタント …… 364
影響力の行使 …… 306
エージェント型 AI 搭載 IDE 環境…… 365
エッジケース …… 33
エラー処理 …… 234
エンジニアのスケジュール …… 311
エンジニアリングの原則 ……2
エンティティ・リレーションシップ図（ER 図）
…… 111
エンドツーエンド（E2E）テスト ……80, 81
エントリーポイントの特定 …… 98
黄金律 …… 10
大文字化 ……6
オブジェクトリレーショナルマッピング（ORM）
…… 173
オプティマイザ …… 189
オンボーディングドキュメント …… 95
オンラインコミュニティ …… 335
オンライン面接 …… 344

か行

会議 …… 299
解決策に飛びつかない ……7
開発環境の最適化 …… 263
開発者 ……2
外部依存関係の特定 …… 106
可読性の高いコード ……4
カナリアデプロイ …… 252
カナリアリリース …… 256
カリフォルニア州消費者プライバシー法（CCPA）
…… 240
環境変数 …… 231
監視 …… 257
キーバリューストア …… 168
キーボード …… 266
キーボードショートカット …… 268
機械学習（ML）…… 355
機能別パッケージング（package by feature）
……96
キャッシュ ……179, 183
キャッシュアサイド（遅延読み込み）…… 179
キャッシュヒット …… 179
キャッシュミス …… 179
キャリアの選択肢 …… 318
キャリアパス …… 315
技術専門職 …… 318
リーダーシップ …… 319
凝集度 ……42
許可フラグ …… 232
近接、デザイン …… 138
クエリ …… 185
クエリプランナー …… 189
クラス図 ……63
グラフデータベース …… 169

グリーンフィールド開発 …… 15
継続的インテグレーションと継続的デプロイ …… 253
結果整合性 …… 178
結合度 …… 42
険悪な雰囲気 …… 304
コアスキル …… 323
コアブランチ …… 244
構造化データ …… 156
コーダー …… 2
コードカバレッジ …… 19, 82
コードスメル …… 45
コードナビゲーションツール …… 26
コードの修正 …… 114
コードレビュー …… 49, 122
　チェックボックス式 …… 49
　ピラミッド …… 49
コードを書く …… 37
　コミュニケーション …… 303
　コメント …… 45
　車輪の再発明は避ける …… 38
　少ないほど良い …… 42
　メソッドは短く …… 44
　メトリクスを追加 …… 40
　良いコードとは …… 39
　読まれることを意識する …… 44
コードを読む …… 15
　IDE（統合開発環境） …… 22, 25
　テストを読む …… 31
国際化（I18N） …… 129
小賢しいコード …… 48
個人識別情報（PII） …… 238
コネクションプール …… 174
コマンドライン …… 266
コマンドラインインターフェース（CLI） …… 266
コミット …… 121
コミュニケーション …… 296
　コード …… 303
　プログラミングの本質 …… 52
コメント …… 45
コンテキスト図 …… 61
コントラスト、デザイン …… 136
コンプライアンス要件 …… 240
コンポーネント図 …… 62, 111

さ行

災害復旧モデル …… 69
サイドプロジェクト …… 317
シークレット …… 232
シーケンス図 …… 64, 111
シェルコマンド …… 267
思考連鎖（CoT）プロンプト …… 369
システムの更新 …… 258
システムの分解 …… 110
システムの保守 …… 258
実験フラグ …… 232
実行フロー …… 97
自動テスト …… 75
　コードカバレッジ …… 82
　種類 …… 79
　メリット …… 75
シフトレフト …… 8
車輪の再発明は避ける …… 38
守・破・離 …… 279
情熱を注げるもの …… 316
信頼の構築 …… 51
図
　AI …… 109
　クラス図 …… 63
　コンテキスト …… 70
　コンテキスト図 …… 61
　コンポーネント図 …… 62
　シーケンス図 …… 64

セキュリティ図 …… 66
ソフトウェアモデリング …… 60
データモデル …… 67
配置図 …… 65
モデル …… 58
スカウトのルール …… 117
スケーリング …… 183
スタンドアロン型チャットボットアシスタント …… 363
ステークホルダー
扱い方 …… 308
合意形成 …… 211
生産性の向上 …… 263
生成 AI（GenAI） …… 357
整列、デザイン …… 137
セキュリティ …… 235
セキュリティ図 …… 66
セッション一貫性 …… 178
接続プール …… 174
ゼロショットプロンプティング …… 371
戦力増幅器（フォースマルチプライヤー） …… 378
ソフトウェアアーキテクチャ …… 203
ソフトウェアエンジニア …… 1
ソフトウェア考古学 …… 20
ソフトウェアのコンプライアンス …… 240
ソフトウェアモデリング …… 57
図 …… 60
ソフトスキル …… 295
ソルト …… 237

た行

大規模言語モデル（LLM） …… 359
大規模なデータセット …… 164
怠惰なプログラマの精神 …… 5
対面面接 …… 344
タスクの分解 …… 372
段階的デプロイ …… 252
地域のテックコミュニティ …… 334
チェックボックス式のコードレビュー …… 49
遅延読み込み（キャッシュアサイド） …… 179
直接データベースアクセス …… 171
強一貫性 …… 178
ディープラーニング …… 356
定型コードに注意する …… 42
ディザスタリカバリモデル …… 69
データ …… 155
依存関係と変換 …… 198
大規模なデータセット …… 164
データモデル …… 67
データ量増加とパフォーマンス維持 …… 184
日付と時刻 …… 163
保存 …… 165
マイグレーション …… 194
データベース …… 165
データベースクエリプランナー …… 189
データベーススキーマの変更 …… 197
データベース接続 …… 174
データベーストランザクション …… 175
適応度関数 …… 214
適切なセッション管理 …… 238
テクノロジーレーダー …… 324
デザインの原則 …… 136
テスト …… 75
AI …… 84
テスト駆動リファクタリング …… 116
テスト対象システム（SUT） …… 85
テストのピラミッド …… 79
テストファースト手法 …… 83
テストラスト手法 …… 84
テストを読む …… 31
ドキュメントとして活用 …… 47
テストダブル …… 86
デバッグ …… 104

デプロイ
一括デプロイ …… 252
自動化 …… 248
戦略 …… 251
段階的デプロイ …… 252
デプロイ環境 …… 243
デプロイメントパイプライン …… 242
伝言ゲーム …… 303
統一モデリング言語（UML）…… 60
統合開発環境 …… → IDE
統合テスト …… 80
ドキュメントデータベース …… 167
トランザクション …… 175
取り残されることへの恐れ（FOMO）…… 285
ドレイファスモデル …… 279

な行

内製フレームワークの特定 …… 107
名前をつける …… 45
ナレッジマネジメント …… 272
二次的なユーザー、ユーザーインターフェース設計 …… 132
認証 …… 236
認知バイアス …… 17
ヌメロニム …… 126
ネットワーキング …… 333

は行

バージョン管理
戦略 …… 243
ベストプラクティス …… 120
配置図 …… 65
バイナリデータ …… 162
バイブコーディング …… 375
初めて触れるシステム …… 93
パスワード管理 …… 237
ハッカーデイ …… 289
ハッカソン …… 289
パフォーマンスの最適化 …… 228
バリューストリームマッピング …… 68
ハルシネーション（幻覚）…… 362
反復、デザイン …… 137
非機能要件 …… 209
非構造化データ …… 158
ビッグバンマイグレーション …… 195
日付と時刻のデータ …… 163
批判される …… 50
品質特性 …… 209
フィーチャートグル …… 232
フィーチャーフラグ …… 232
フェーズドマイグレーション …… 195
フォースマルチプライヤー（戦力増幅器）…… 378
複雑なコード …… 48
複数の認証段階を追加する …… 238
フューショットプロンプティング …… 371
ブラウザの開発者ツール …… 100
ブラウンフィールド開発 …… 15
ブランチ …… 122
ブランチ名 …… 246
プリペアドステートメント …… 186
ブルーグリーンデプロイ …… 256
プルリクエスト …… 122
プレゼンテーション …… 302
フロー状態 …… 264
フローチャート …… 111
プログラマ …… 2
プログラミング言語を学ぶ …… 51
プロフェッショナルな人間関係 …… 335
プロンプトエンジニアリング …… 367
ヘキサゴナルアーキテクチャ …… 96
ベクトル埋め込み …… 170
ベクトルデータベース …… 170

変更管理戦略 …… 118
ポートとアダプタ …… 96
補助ブランチ …… 245
ポモドーロ・テクニック …… 312
本番環境 …… 223
　コードの作成 …… 228
本番システム …… 257
　監視 …… 257

ま行

マイグレーション …… 194
マイクロサービスアーキテクチャ …… 97
マインドマップ …… 111
昔からそうしてきたから …… 7
メソッドは短く …… 44
メッセージング …… 298
メトリクスの活用、コードを書く …… 40
面接 …… 337
　オンライン …… 344
　技術的な質問 …… 341
　事前調査 …… 340
　対面 …… 344
メンタルモデル …… 109
　可視化 …… 110
モーニングコーヒー …… 283
モック …… 86
モデリングツール …… 71
モデル …… 57
　図 …… 58
モデルからのコード生成 …… 72
モデル作成のベストプラクティス …… 69

や行

ユーザーアカウントの保護 …… 238
ユーザーインターフェース（UI）設計 …… 125
　エラーメッセージ …… 149
　デザインの原則を適用 …… 139
　デフォルト設定の弊害 …… 133
　破壊的な操作 …… 150
　文化の影響 …… 134
　わかりやすい …… 143
ユーザーストーリーマッピング …… 69
ユーザーデータの保護 …… 238
ユーザーの理解
　ユーザーインターフェース設計 …… 130
　ユーザーは予測不能 …… 224
ユーザビリティ …… 127, 135
ユニットテスト …… 79, 80
良いコードとは …… 39

ら行

ライトスルー …… 180
ライトビハインドキャッシュ …… 180
リーダーシップ …… 319
リードレプリカ …… 183
リファクタリング …… 115
リポジトリパターン …… 172
リモートワーク …… 347
リリースフラグ …… 232
リレーショナルデータベース …… 166
類似度検索 …… 170
レイヤー別パッケージング（package by layer）
…… 95
レガシーコード …… 16
ローカライゼーション（L10N） …… 129
ローリングデプロイ …… 252
ロギング …… 104, 234

わ行

ワークライフバランス …… 346
ワンショットプロンプティング …… 371

● 著者紹介

Nathaniel T. Schutta（ナサニエル・T・シュッタ）

クラウドコンピューティング、開発者体験の向上、使いやすいアプリケーションの構築に注力するソフトウェアアーキテクトであり、Java Champion である。ポリグロットプログラミングの支持者であり、複数の書籍を執筆し、数多くの動画やポッドキャストにも出演している。また、経験豊富な講演者でもあり、世界各地のカンファレンス、ミートアップ、大学、ユーザーグループで定期的に登壇している。本業に加えて、ミネソタ大学の非常勤講師として、技術の変化を受け入れ、評価する姿勢を学生に教えている。世の中から悪いプレゼンテーションをなくすことを目指し、Neal Ford、Matthew McCullough とともに『Presentation Patterns』を共著した。また、O'Reilly Media から刊行されている『Thinking Architecturally』および『Responsible Microservices』の著者でもある。

Dan Vega（ダン・ベガ）

Broadcom の Spring デベロッパーアドボケイトであり、Java Champion である。20 年以上にわたるソフトウェア開発の経験を持つ。情熱的な問題解決者である Dan は、ブロガー、YouTuber、コース制作者、講演者として積極的に知識を共有し、継続的な学びを通じて開発者たちに刺激を与えている。

● 訳者紹介

村上 列（むらかみ れつ）

兵庫県生まれ。京都大学大学院工学研究科修了。生来の新しもの好きがこうじて、計算機やネットワークの世界に足を踏み入れ現在にいたる。計算機やネットワーク上での人と人とのコミュニケーションのあり方について強い興味を持つ。

表紙の動物

表紙に描かれている動物は、キンセンアイゴ（Siganus lineatus）である。西太平洋に生息する色鮮やかな魚で、ラグーンやサンゴ礁をすみかとしている。

キンセンアイゴは淡い青色の体を持ち、頭部から二叉に分かれた尾びれにかけて、波状のオレンジ色の線が走っている。体は左右に平たく、最大で約 43 センチメートルまで成長するが、一般的な体長は約 23 センチメートルである。背びれには鋭い棘があり、捕食者から身を守るための毒を備えている。

独特の体色によって、キンセンアイゴはサンゴ礁の環境に自然に溶け込むことができる。草食性で、主に藻類を食べる。浜辺の岩やサンゴ礁に付着した藻類を削り取るほか、海藻や海草も摂取する。

キンセンアイゴはサンゴ礁の生態系において重要な役割を担っている。藻類の過剰な繁殖を抑えるとともに、より大型のサンゴ礁魚類の餌にもなっている。

キンセンアイゴは絶滅危惧種ではなく、国際自然保護連合（IUCN）のレッドリストでは低懸念（Least Concern）に分類されている。ただし、乱獲、生息地の喪失、気候変動、観賞魚取引など、いくつかの脅威に直面している。O'Reilly の表紙に登場する動物は、いずれも世界にとって重要な存在である。

ソフトウェアエンジニアリングの基礎
コーダーからエンジニアになるための実践ガイド

2026年 9 月14日　初版第 1 刷発行
2026年10月 9 日　初版第 2 刷発行

著　　者　Nathaniel Schutta（ナサニエル・シュッタ）、Dan Vega（ダン・ベガ）
訳　　者　村上 列（むらかみ れつ）
発 行 人　ティム・オライリー
制　　作　アリエッタ株式会社
印刷・製本　三美印刷株式会社
発 行 所　株式会社オライリー・ジャパン
〒 105-0003　東京都港区西新橋一丁目 18 番 6 号
Tel （03）6257-2177
Fax （03）6257-3380
電子メール　japan@oreilly.co.jp
発 売 元　株式会社**オーム**社
〒 101-8460　東京都千代田区神田錦町 3-1
Tel （03）3233-0641（代表）
Fax（03）3233-3440

Printed in Japan（ISBN978-4-8144-0178-9）
乱本、落丁の際はお取り替えいたします。

本書は著作権上の保護を受けています。本書の一部あるいは全部について、株式会社オライリー・ジャパンの承諾を得ずに、著作権法の範囲を超えて無断で複写、複製することは禁じられています。